LINUX MEMORY MANAGER

(*LINUX KERNEL 5.10.0*)



com.lee

LBB_3001@163.com

https://github.com/iamlixiaolong/linux_mem_mgt_notes

Contents

1 slub 组件 **7**
1.1 创建 slub cache 描述符 kmem_cache . . . 7
1.1.1 kmem_cache_create_usercopy() . . . 10
1.2 ___kmem_cache_alias() . . . 13
1.3 create_cache() . . . 17
1.3.1 ___kmem_cache_create() . . . 19
1.3.2 kmem_cache 结构体部分成员 . . . 27
1.3.3 calculate_sizes() . . . 28
1.4 专用缓存的创建 . . . 40
1.5 通用缓存的创建 . . . 41

2 slub object 的分配 **47**
2.1 kmem_cache_alloc_node() . . . 59
2.1.1 快速路径 . . . 66
2.1.2 慢速路径 和 超慢路径 . . . 66
2.2 缓存分配的 KPI 入口 . . . 83
2.2.1 通用缓存的分配 . . . 84
2.2.2 kmalloc() . . . 84
2.2.3 专用缓存的分配 . . . 88

3 SLUG debug **89**
3.1 slub obj 中与 debug 相关的区域 . . . 89
3.2 slub 分配时内存的初始填充值 . . . 90
3.3 slub obj 使用过程中填充数据的变化 . . . 91
3.3.1 分配 slub obj . . . 91
3.4 SLUB debug 的检测点 . . . 91
3.4.1 内存释放时篡改检测 . . . 91
3.5 分配时的内存篡改检测 . . . 97
3.6 slabinfo 工具 . . . 98

4 页分配 **100**
4.1 水位线初始化 . . . 103
4.1.1 init_per_zone_wmark_min() . . . 103
4.1.1.1 refresh_zone_stat_thresholds() . . . 110
4.1.1.2 ___setup_per_zone_wmarks() . . . 118
4.2 ___alloc_pages_nodemask() • 页分配的快速路径和慢速路径 . . . 124
4.2.1 ___alloc_pages_slowpath() . . . 126
4.2.2 prepare_alloc_pages() . . . 138

4.2.2.1 gfp_zone() . . . 140
4.2.3 alloc_flags_nofragment() . . . 144
4.2.4 get_page_from_freelist() . . . 145
4.2.4.1 prep_new_page() . . . 150
4.2.4.2 reserve_highatomic_pageblock() . . . 152
4.2.4.3 zone_watermark_fast() . . . 153
4.2.4.4 rmqueue() . . . 155

5 内存回收 181

5.1 do_try_to_free_pages() . . . 181
5.1.1 vmpressure_prio() . . . 188
5.1.2 内存压力事件上报 . . . 189
5.1.3 shrink_zones() . . . 195
5.1.3.1 compaction_ready() . . . 198
5.1.4 shrink_node() . . . 202
5.1.5 lru_cache_add() . . . 212
5.1.6 回收损耗 . . . 213
5.1.6.1 回收损耗 • lru_note_cost_page() . . . 213
5.1.6.2 workingset_refault() . . . 214
5.1.6.3 匿名页误回收开销 • ___read_swap_cache_async() . . . 219
5.1.7 页回收时工作集信息的存储 . . . 219
5.1.7.1 文件页 workingset 信息的保存 . . . 220
5.1.7.2 匿名页 workingset 信息的保存 . . . 220
5.1.8 shrink_node()(继续) . . . 220
5.1.9 shrink_node_memcgs() . . . 224
5.1.10 shrink_lruvec() . . . 230
5.1.10.1 get_scan_count() 以及工作页集 . . . 234
5.1.11 shrink_list() . . . 246
5.1.12 shrink_inactive_list() . . . 248
5.1.13 shrink_page_list() . . . 253
5.1.13.1 add_to_swap() . . . 276
5.1.13.2 get_swap_page() . . . 278
5.1.13.3 add_to_swap_cache() . . . 282
5.1.13.4 XA_STATE_ORDER() . . . 285
5.1.13.5 page_check_dirty_writeback() . . . 286
5.1.13.6 文件页链表上的匿名页: lazyfree 页 . . . 288
5.1.13.7 page_check_references() . . . 289
5.1.13.8 pageout() . . . 292
5.1.14 shrink_active_list() . . . 295
5.1.14.1 isolate_lru_pages() . . . 299
5.2 kswapd 非直接回收 . . . 305
5.2.1 balance_pgdat . . . 311
5.2.2 snapshot_refaults() . . . 318

5.2.3 pgdat_balanced . . . 318
5.2.4 kswapd_shrink_node() . . . 318
5.3 try_to_freeze() . . . 320
5.4 备忘 . . . 323
5.4.1 结构体 scan_control 成员说明 . . . 323

6 内存整理 328
6.1 compact_zone() . . . 329
6.1.1 compaction_restarting() . . . 331
6.1.2 compaction_suitable() . . . 332
6.1.2.1 fragmentation_index() . . . 334
6.1.2.2 ___compaction_suitable() . . . 337
6.1.3 ___reset_isolation_suitable() . . . 340
6.1.3.1 ___reset_isolation_pfn() . . . 342
6.2 compact_zone()(续) . . . 344
6.2.1 isolate_migratepages() . . . 349
6.2.1.1 suitable_migration_source() . . . 352
6.2.1.2 isolate_migratepages_block() . . . 353
6.2.2 compact_finished() . . . 362
6.2.2.1 ___compact_finished() . . . 362
6.2.3 migrate_pages() . . . 369
6.2.3.1 split_huge_page_to_list() . . . 373
6.3 end_page_writeback() . . . 385

7 脏页限流 388
7.1 balance_dirty_pages_ratelimited() . . . 388
7.2 进程脏页阈值和 per-cpu 的脏页阈值 . . . 389
7.2.1 balance_dirty_pages() . . . 394
7.3 全局脏页上限阈值 . . . 407
7.4 允许不限速产生脏页的上限 . . . 408
7.5 回写带宽的更新 . . . 409
7.5.1 ___wb_update_bandwidth() . . . 409
7.6 控制脏页产生速率的调速系数 . . . 413
7.6.1 wb_position_ratio() . . . 413
7.6.1.1 pos_ratio_polynom() . . . 421
7.7 脏页阈值和后台回写阈值 . . . 424
7.8 单设备的脏页阈值 . . . 426
7.8.1 wb_dirty_limits() . . . 426
7.8.1.1 ___wb_calc_thresh() . . . 427
7.9 脏页轮询的间隔 . . . 428
7.9.1 dirty_poll_interval() . . . 428
7.10 脏页速率限制值 . . . 429
7.10.1 wb_update_dirty_ratelimit() . . . 429

7.11 写 page cache 线程的暂停间隔 . . . 434
7.11.1 wb_min_pause() . . . 435
7.11.2 wb_max_pause() . . . 437

8 swapcache 439

9 反向映射 444
9.1 匿名页和文件页反向映射的入口函数 . . . 445
9.1.1 rmap_walk_anon() . . . 446
9.1.1.1 try_to_unmap_one() . . . 451
9.2 PTE(页表项) . . . 465
9.2.1 PTE 什么时候加载进 TLB . . . 468
9.2.1.1 TLB 加载和 invalid . . . 468

10 异构内存管理 470
10.1 GPU 私有内存迁移到系统 RAM . . . 470
10.1.1 单页回迁 . . . 471
10.1.1.1 和异构内存管理无关但重要的主题 · signal 的组装和发送 . . . 473
10.1.1.2 驱动的->migrate_to_ram() 回调 . . . 483
10.1.2 批量回迁 . . . 484
10.1.2.1 migrate_vma_setup() . . . 485
10.1.2.2 migrate_vma_collect() . . . 487
10.1.2.3 migrate_vma_prepare() . . . 488
10.1.2.4 migrate_vma_pages() . . . 494
10.1.2.5 (AMD GPU) 用户空间通过 ioctl() 进行批量迁移的流程 . . . 497
10.1.2.6 (AMD GPU) 因显存压力内核空间触发批量迁移的流程 . . . 499
10.2 RAM 数据迁移到 GPU 私有内存 . . . 500
10.2.1 批量 (单页) 迁移 . . . 500
10.3 PTE 状态转换图 . . . 500
10.4 用户空间访问设备私有内存的流程 . . . 501
10.5 设备私有内存映射 . . . 503
10.5.1 devm_memremap_pages() . . . 503
10.5.2 memremap_pages() . . . 504
10.5.3 pagemap_range() . . . 506
10.5.4 memmap_init_zone_device() . . . 510

11 DMA-BUF 513
11.1 dma-buf 在用户空间的共享 . . . 513
11.2 dma-buf 的映射接口 . . . 514
11.2.1 dma_buf_ops->mmap() . . . 514
11.2.1.1 drm_gem_dmabuf_mmap() . . . 515
11.2.1.2 drm_gem_prime_mmap() . . . 515
11.2.2 dma_buf_vmap() . . . 517

12 NUMA blance **519**
12.1 实现方法 . . . 519
12.2 NUMA balance 执行的三个阶段 . . . 520
12.2.1 扫描和标记 (NUMA hinting PTE) . . . 520
12.2.2 缺页采样 . . . 527
12.2.3 决策迁移 . . . 528
12.2.3.1 迁移进程 . . . 528
12.2.3.2 迁移页 . . . 529
12.3 防止 NUMA balance 破坏全局负载均衡 . . . 529

13 与用户空间的交互 **530**
13.1 查看空闲内存 . . . 530
13.1.1 free . . . 530
13.1.2 /proc/meminfo 节点 . . . 532
13.1.2.1 查看根 LRU 向量 . . . 533
13.2 检测内存碎片 . . . 533
13.2.1 /proc/buddyinfo . . . 533
13.3 查看进程的内存占用 . . . 533
13.3.1 /proc/$pid/status . . . 533
13.4 查看内核工作集页的状态 . . . 534
13.4.1 /proc/vmstat . . . 534

14 初始化 **536**
14.1 bootmem_init() . . . 536
14.1.1 arm64_hugetlb_cma_reserve() . . . 538
14.1.2 hugetlb_cma_reserve() . . . 538
14.1.3 arm64_numa_init() . . . 540
14.1.4 numa_init() . . . 540
14.1.5 numa_register_nodes() . . . 542
14.1.6 numa_alloc_distance() . . . 543
14.2 CMA 内存 . . . 544
14.2.1 预留通用 CMA . . . 550
14.2.1.1 通过 dts 节点预留通用 CMA . . . 550
14.2.1.2 通过启动参数预留通用 CMA . . . 551
14.2.2 创建设备私有的或默认全局的 DMA 池 . . . 557
14.2.3 预留 per-node DMA 专用的 CMA . . . 558

15 其它 **560**
15.1 xarrays(可扩展数组) . . . 560
15.1.1 xas_store() . . . 566
15.1.2 xas_create() . . . 570
15.2 外设寄存器 (MMIO) 的映射 . . . 573
15.2.1 ioremap() . . . 573

A 附录 **574**

1

slub 组件

slab allocator 是一系列数据结构、算法、接口的集合体，用于小对象 (obj) 内存的管理和分配。它在中文中一般被翻译成"slab 分配器"，但是称为" 组件" 更符合实际也更容易理解。

slab allocator 管理数量可变的缓存 (caches)。这里的缓存指的是一些常用结构体内存 (buffer) 的缓存 cache。它可以用于加速常用结构体的快速分配，是软件 cache, 和硬件 cache(hardware cache) 不是一个概念。

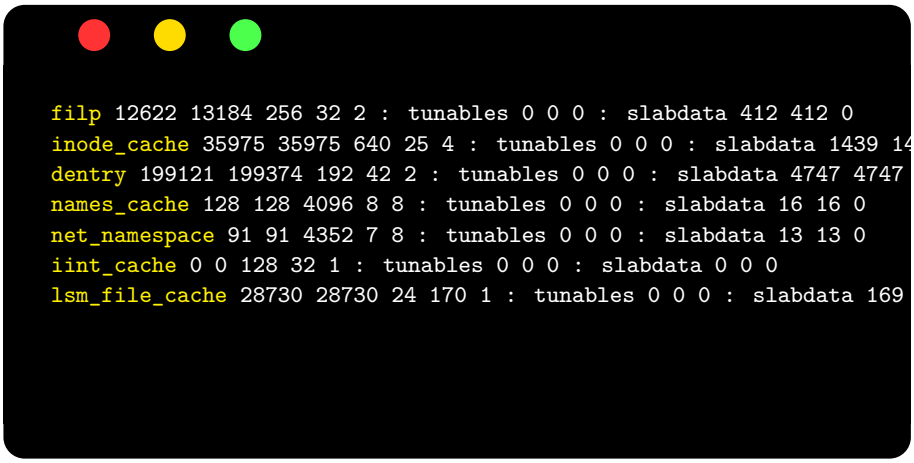
1.1 创建 slub cache 描述符 kmem_cache

kmem_cache_create() 函数用于创建 (或分配) 一个管理特定 size 大小 obj 对象的 slub cache 缓存结构体 kmem_cache, 以应对不同大小内存块的申请。原型如下:

```
struct kmem_cache *
kmem_cache_create(const char *name, unsigned int size, unsigned int align,
                                        slab_flags_t flags, void (*ctor)(void *))
```

各参数意义如下:

- ***name*** 这是字符串表示的 slub cache 名称。可以通过 *ls /sys/kernel/slab* 或 *cat /proc/slabinfo* 看到该名称。*cat /proc/slabinfo* 的输出示意如下 (截取的一部分)，其中最左一栏就是 slub cache 名称:



- ***size*** 单个 obj 的字节大小。这 size 是纯 payload(有效载荷) 的大小。
- ***align*** obj 最小对齐字节。

  $align = 0$: 使用默认对齐（按 size 自然对齐、cpu 原生对齐）。

  $align > 0$: 强制每个 obj 起始地址必须是该值整数倍。适用场景: 硬件 DMA 结构体、需要特殊对齐的数据结构。
- ***flags*** slub cache 行为控制标志，位掩码。常用宏如下:

控制宏	意义
SLAB_HWCACHE_ALIGN	*obj* 按 *cpu* 的硬件 *cache line* 对齐。
SLAB_CACHE_DMA	强制 *obj* 从 *ZONE_DMA* 内存域分配，专门用于 *DMA* 硬件访问。
SLAB_POISON	开启内存污化调试项，检测越界/野指针
SLAB_PANIC	创建失败直接 *panic*，不返回 *NULL*
SLAB_CONSISTENCY_CHECKS	分配/释放时强制做内存一致性校验。判断当前 *obj* 是否属于本 *slub*、是否落在 *slub* 页合法区间。检测双重释放。检测 *redzone* 警戒区是否被改写。校验 *freelist* 链表完整性。校验对象污化标记。，仅用于内存 *bug* 排查，生产环境严禁开启。
SLAB_RED_ZONE	在每个 *obj* 的 *payload* 前后插入警戒红区，用来检测内存越界读写。

- ***ctor*** obj 构造回调函数指针，可为 NULL。分配新 slub 首次初始化 obj 时自动调用。仅在整块 slub 新建时执行，每次 kmalloc 分配不会重复调用。传 NULL 代表无构造函数，内核不做额外初始化。

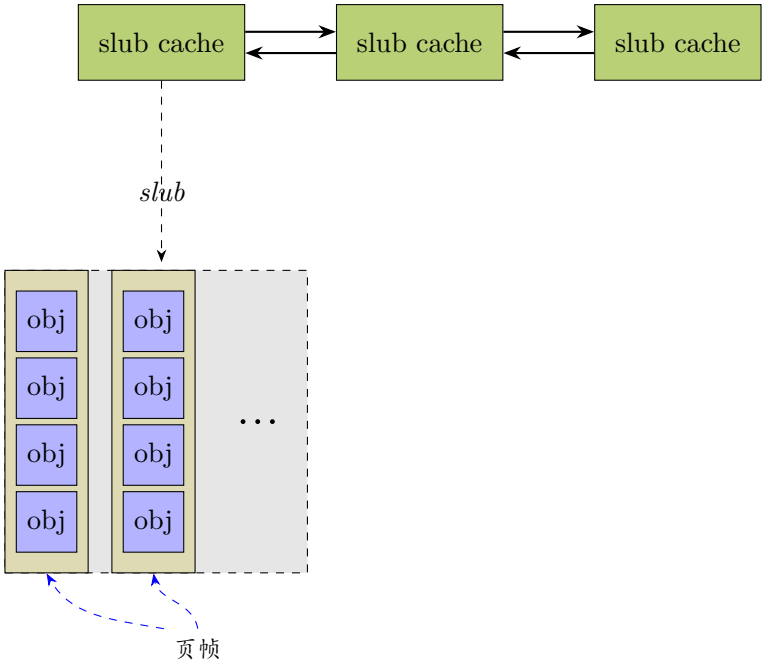
函数 kmem_cache_create() 会初始化缓存池的元数据 (譬如 obj 对象大小、对齐方式、标志等)，但是该函数不会直接分配物理内存或创建对象 (slub obj)。之后调用 kmem_cache_alloc() 时才会分配 slub 缓存块。

slub、slub obj、kmem_cache 的概念如下:

- ***slub cache*** 也称为***slub 缓存***、***slub 缓存池***。它是同尺寸、同属性的 slub 块的集合，因此被称为 (slub) 内存池。
- ***slub*** 或者称为***slub 块***、***slub 缓存块***。它是一组连续物理页组成的内存块，是承载多个 slub obj 的物理载体，隶属于某个 kmem_cache 结构体管理。
- ***slub obj*** 又称为***slub 对象***、***slub 缓存对象***。是 slub 内部切割出来的等大小内存块，是用户实际申请读写的最小内存单元。同一个 kmem_cache 管理下的所有 obj 大小完全相同。

- ***kmem_cache*** 该结构体也被称为***slub* 缓存描述符**，也是同尺寸 slub 对象的全局管理结构体。固定大小的内存对象唯一对应一个 kmem_cache 管理结构体来管理。

总而言之,slub cache 是 slub 的池子。slub 是 slub obj 的缓存池。

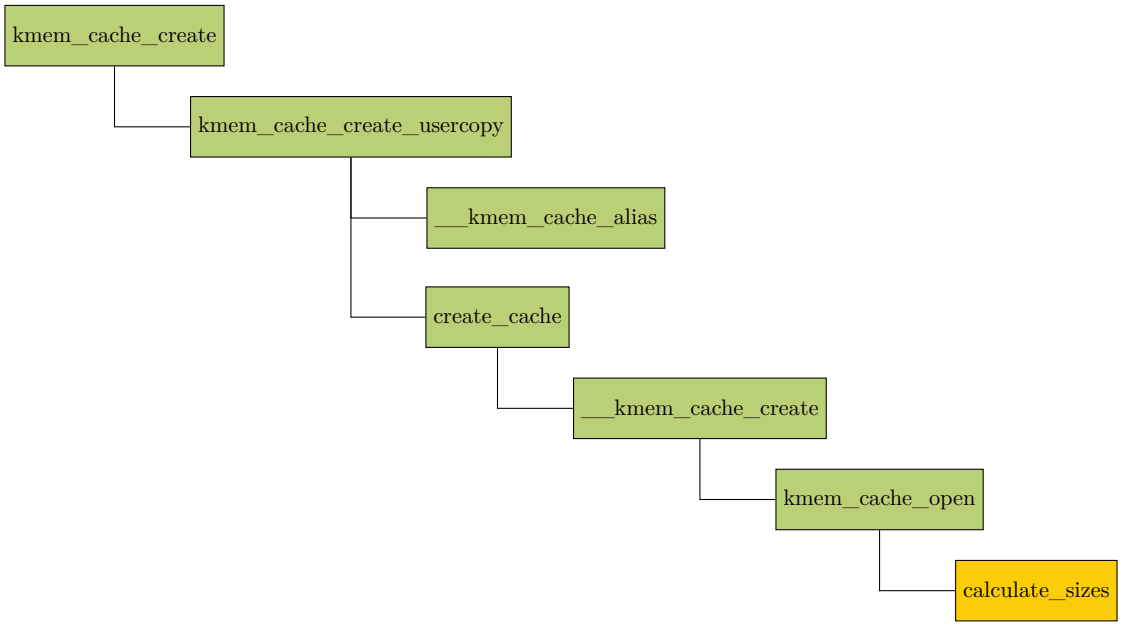


如图，每个缓存都由一个缓存描述符结构体 kmem_cache 表示。

在 slub 内存管理中经常出现的 slub 和 slub 对象分别与 c++ 语言中的“类型”和“对象”的概念有差异。譬如 c++ 中一个结构体类型 X, 实例化或分配内存后可以称之为结构体类型 X 的对象 Y。X 和 Y 的关系就像一个物件在蓝图上和实际生产出来的关系，而且蓝图上标注的 size 和实际生产出的 size 是一致的。但是 slub 和 slub 对象的关系却不是这样的，slub 就好像一个 slub 对象的池子，由多个 (极端情况下可能是一个)slub 对象组成。

为了尽量避免和 c++ 的概念混淆，本文之后碰到 slub 对象都以 slub obj 来称之，希望在中文语境下以英语单词造成的疏离感产生概念上的显著区别。

kmem_cache_create() 函数的主要子函数调用如下：



1.1.1 kmem_cache_create_usercopy()

kmem_cache_create_usercopy() 的调用图如下：

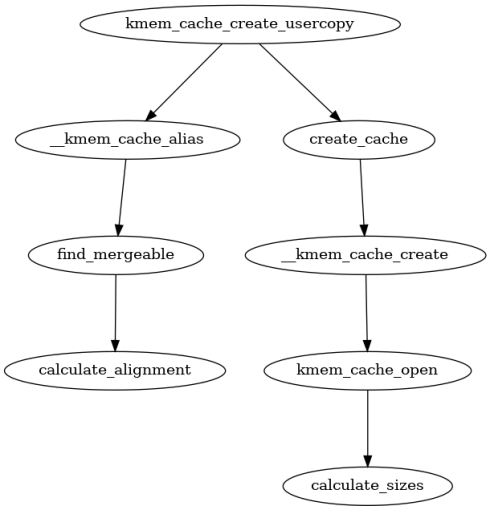


图 1–1: 调用图：kmem_cache_create_usercopy()

slub cache(slub 缓存) 的整个创建过程其实是封装在 kmem_cache_create_usercopy 函数中，kmem_cache_create 直接调用了该函数，并将参数透传过去。

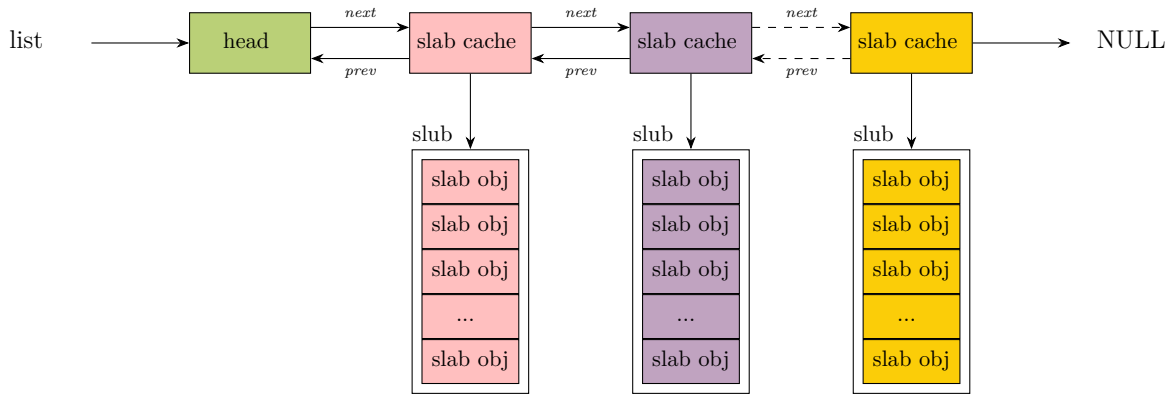
kmem_cache_create() *kmem_cache_create_usercopy()*

```
300 struct kmem_cache *
301 kmem_cache_create_usercopy(const char *name,
302           unsigned int size, unsigned int align,
303           slab_flags_t flags,
304           unsigned int useroffset, unsigned int usersize,
305           void (*ctor)(void *))
306 {
307     struct kmem_cache *s = NULL;
308     const char *cache_name;
309     int err;
```

```
310
311     get_online_cpus();
312     get_online_mems();
313
314     mutex_lock(&slab_mutex);
315
316     err = kmem_cache_sanity_check(name, size);
317     if (err) {
318         goto out_unlock;
319     }
320
321     /* Refuse requests with allocator specific flags */
322     if (flags & ~SLAB_FLAGS_PERMITTED) {
323         err = -EINVAL;
324         goto out_unlock;
325     }
326
327     /*
328      * Some allocators will constraint the set of valid flags to a subset
329      * of all flags. We expect them to define CACHE_CREATE_MASK in this
330      * case, and we'll just provide them with a sanitized version of the
331      * passed flags.
332      */
333     flags &= CACHE_CREATE_MASK;
334
335     /* Fail closed on bad usersize of useroffset values. */
336     if (WARN_ON(!usersize && useroffset) ||
337         WARN_ON(size < usersize || size - usersize < useroffset))
338         usersize = useroffset = 0;
339
340     if (!usersize)
341         s = __kmem_cache_alias(name, size, align, flags, ctor);
342     if (s)
343         goto out_unlock;
344
345     cache_name = kstrdup_const(name, GFP_KERNEL);
346     if (!cache_name) {
347         err = -ENOMEM;
348         goto out_unlock;
349     }
350
351     s = create_cache(cache_name, size,
352              calculate_alignment(flags, align, size),
353              flags, useroffset, usersize, ctor, NULL);
354     if (IS_ERR(s)) {
355         err = PTR_ERR(s);
356         kfree_const(cache_name);
357     }
358
359 out_unlock:
360     mutex_unlock(&slab_mutex);
361
362     put_online_mems();
363     put_online_cpus();
364
365     if (err) {
366         if (flags & SLAB_PANIC)
367             panic("kmem_cache_create: Failed to create slab '%s'. Error %d\n",
368                 name, err);
369         else {
370             pr_warn("kmem_cache_create(%s) failed with error %d\n",
371                 name, err);
372             dump_stack();
373         }
```

```
374             return NULL;
375         }
376         return s;
377 }
378 EXPORT_SYMBOL(kmem_cache_create_usercopy);
```

- 311-312 获取 cpu_hotplug_lock 和 mem_hotplug_lock，防止 cpu 热插拔改变在线 cpu 的 map 位图，及防止访问内存的时候进行内存热插拔。
- 314 内核中使用一个全局的双向链表来串联起系统中所有的 slub cache(slub 缓存池) (如下图)，这里获取全局链表 list 的锁，防止并发操作对 list 进行修改。



- 316-319 入参检查，校验 name 和 size 的有效性。并且防止在中断上下文中进行创建过程，因为 slub cache 的创建并不是原子的。
- 322-325 检查有效的 slab flags 标记位，如果传入的 flag 是无效的，则拒绝本次创建请求
- 333 创建 slab cache 时用得到的标志位.
- 336 检验 useroffset 和 usersize 的有效性。

  useroffset 表示允许拷贝至用户态的内存区域起始偏移量。usersize 是允许拷贝至用户态的内存区域长度。
- 341 如果是普通内核对象缓存 (usersize==0)，那么遍历全局 slab_caches 链表。查找是否存在规格完全匹配的现有缓存，可以直接复用，就不需要创建新的 slub 缓存描述符了，直接和已有的 slab cache 合并，并且在 sys 文件系统中使用指定的 name 作为已有 slab cache 的别名。

  只有不需要用户态拷贝的普通 slub 缓存，才去查找复用同规格已有缓存；带 usercopy 安全区域的缓存不参与复用，直接新建。

  从 kmalloc() 调用时传递的参数 useroffset 和 usersize 都为 0。因此不需要用户态拷贝。
- 345-349 根据传入的字符串指针 name，存 slub 分配一块新内存区域，把传入的字符串完整复制过去，返回新字符串地址存入 cache_name。cache_name 就是将要创建的 slab cache 名称，用于在 ***/proc/slabinfo*** 中显示。

  不直接用 name 是因为 name 可能是只读段常量、栈临时字符串，或者外部传入临时指针。如果直接赋值，原字符串销毁后，cache_name 会变成野指针。单独拷贝一份内存副本，变量的生命周期由自己控制。

- 351 找不到可以被复用的 slub cache，那么调用 create_cache 开始创建新的 slab cache

1.2 ___kmem_cache_alias()

___kmem_cache_alias 函数的核心是在 find_mergeable() 函数中，find_mergeable() 会遍历 slab cache 的全局链表，查找与当前创建参数贴近可以被复用的 slub cache。

kmem_cache_create() > *kmem_cache_create_usercopy()* > *___kmem_cache_alias()*

```
4406 struct kmem_cache *
4407 __kmem_cache_alias(const char *name, unsigned int size, unsigned int align,
4408            slab_flags_t flags, void (*ctor)(void *))
4409 {
4410     struct kmem_cache *s;
4411
4412     s = find_mergeable(size, align, flags, name, ctor);
4413     if (s) {
4414         s->refcount++;
4415
4416         /*
4417          * Adjust the object sizes so that we clear
4418          * the complete object on kzalloc.
4419          */
4420         s->object_size = max(s->object_size, size);
4421         s->inuse = max(s->inuse, ALIGN(size, sizeof(void *)));
4422
4423         if (sysfs_slab_alias(s, name)) {
4424             s->refcount--;
4425             s = NULL;
4426         }
4427     }
4428
4429     return s;
4430 }
```

- 4412 在全局 slab cache 链表中查找与当前创建参数相匹配的 slab cache，如果查找到一个 slab cache 的核心参数和指定的创建参数很贴近，那么就没必要再创建新的 slub cache 了，可以复用已有的 slub cache。
- 4414 如果存在可复用的 kmem_cache，则将它的引用计数 +1。
- 4420-4421 采用较大的值，更新已有的 kmem_cache 相关的元数据。object_size 为 slab 中 obj 的实际大小 (不包含填充的字节数)；->inuse 为 obj 的 object_size 按照 word 字长对齐之后的大小。
- 4423 由于会复用已有的 kmem_cache 并不会创建新的，为了看起来像是创建了一个名称为 name 的新 kmem_cache，所以要给被复用的 kmem_cache 起一个别名，这个别名就是指定的参数 name。这样一来会在 sys 文件系统中出现这样的目录 /sys/kernel/slab/name，该目录下的文件包含了对应 slab cache 运行时的详细信息。这是内核需要让用户感觉已经按照指定的创建参数创建了一个新的 slab cache，但是该目录下的文件需要软链接到原有 slab cache 在 sys 文件系统对应目录下的文件。这就相当于给原有 slab cache 起了一个别名。可以通过 /sys/kernel/slab/<cachename>/aliases 文件查看该 slab cache 的所有别名个数，也就是说有多少个 slab cache 复用了该 slab cache 。

  系统中的所有 slab cache 都会在 sys 文件系统中有一个专门的目录：/sys/kernel/slab/<cachename>，该目录下的所有文件都是 read only 的，每一个文件代表 slab cache 的一项运行时信息，如下：

align: 标识该 slab cache 中的 slab 对象的对齐 align。

alloc_fastpath: 记录该 slab cache 在快速路径下分配的对象个数。

alloc_from_partial: 记录该 slab cache 从本地 cpu 缓存 partial 链表中分配的对象次数。

alloc_slab: 记录该 slab cache 从伙伴系统中申请新 slab 的次数。

cpu_slabs: 记录该 slab cache 的本地 cpu 缓存中缓存的 slab 个数。

partial: 记录该 slab cache 在每个 NUMA 节点缓存 partial 链表中的 slab 个数。

objs_per_slab: 记录该 slab cache 中管理的 slab 可以容纳多少个对象。

遍历全局所有 kmem_cache，查找一个可以复用的现有 slub 缓存，找不到就返回 NULL。

kmem_cache_create() > *kmem_cache_create_usercopy()* > *__kmem_cache_alias()* > *find_mergeable()*

```
185 struct kmem_cache *find_mergeable(unsigned int size, unsigned int align,
186         slab_flags_t flags, const char *name, void (*ctor)(void *))
187 {
188     struct kmem_cache *s;
189 
190     if (slab_nomerge)
191         return NULL;
192 
193     if (ctor)
194         return NULL;
195 
196     size = ALIGN(size, sizeof(void *));
197     align = calculate_alignment(flags, align, size);
198     size = ALIGN(size, align);
199     flags = kmem_cache_flags(size, flags, name, NULL);
200 
201     if (flags & SLAB_NEVER_MERGE)
202         return NULL;
203 
204     list_for_each_entry_reverse(s, &slab_caches, list) {
205         if (slab_unmergeable(s))
206             continue;
207 
208         if (size > s->size)
209             continue;
210 
211         if ((flags & SLAB_MERGE_SAME) != (s->flags & SLAB_MERGE_SAME))
212             continue;
213         /*
214          * Check if alignment is compatible.
215          * Courtesy of Adrian Drzewiecki
216          */
217         if ((s->size & ~(align - 1)) != s->size)
218             continue;
219 
220         if (s->size - size >= sizeof(void *))
221             continue;
222 
223         if (IS_ENABLED(CONFIG_SLAB) && align &&
224             (align > s->align || s->align % align))
225             continue;
226 
227         return s;
228     }
229     return NULL;
230 }
```

- 190-191 内核启动参数（slab_nomerge），开启后全局禁止任何缓存复用，则直接返回 NULL，不复用任何缓存。
- 193-194 存在构造函数则禁止合并。

  不同缓存不能共用同一个初始化构造函数 ctor，复用后对象初始化逻辑会冲突；因此只要传入构造函数，直接放弃复用，必须新建缓存。
- 196 对象 size 向上对齐到指针宽度。

  内核分配要求所有 slab 对象 (obj) 保证指针对齐，方便存放链表指针、元数据
- 197 根据指定的对齐参数 align 并结合 CPU cache line 大小，计算出一个合适的对齐参数。
- 198 obj size 重新按照 align 进行对齐。确保对象整体尺寸满足对齐要求。
- 201 如果 flag 设置的是不允许复用，则停止并退出。
- 204 开始遍历内核中已有的 slab cache，寻找可以复用的 slab cache。
- 205-206 跳过不能合并的缓存。

  slab_unmergeable() 判断也有缓存是否带调试功能（KASAN、红区、POISON、追踪、RCU 特殊标记等），开启 slub 调试功能的 slub 必须完全独立、不能和其他缓存复用。
- 208 缓存单个对象大小 s->size 要能装下本次要分配的数据。如果本次需要更大 size 缓存，这个目标缓存不能用，跳过。
- 211 校验指定的 flag 是否与已有 slab cache 中的 flag 一致。
- 217-218 检验已存在的缓存对象的 size 是否满足对齐要求。

  已有目标缓存对象的 s->size 必须是本次需要 size 的 align 的整数倍。
- 220-221 目标 size 和已存在的 slub 缓存中对象大小的差值不能超过一个指针宽度。这是为了避免大量小 size 请求复用超大对象，造成严重内存浪费。
- 223-225 已有 slab cache 中 obj 的对齐 align 要大于等于指定的 align 并且可以整除 align。

  SLUB 分配器不会执行该代码段。

事实上，内核并不会完全按照我们指定的 align 进行内存对齐，而是会综合考虑 cpu 硬件 cacheline 的大小，以及处理器字长 (word size) 计算出一个合理的 align 值。内核在对 slab 对象进行内存布局的时候，会按照这个最终的 align 进行内存对齐。

通过将对象与 L1 缓存对齐，可以更好的利用硬件 cache 加速对象的存取。L1 缓存是最靠近 CPU 核心的缓存，访问速度最快。将 obj 对象按 L1 缓存行对齐能最大限度减少” 跨缓存行访问”, 以及缓存抖动，确保数据对象高效加载到 L1 缓存中，这对内核性能 (尤其是频繁访问的对象，譬如进程描述符等结构体) 至关重要。

```
137 static unsigned int calculate_alignment(slab_flags_t flags,
138         unsigned int align, unsigned int size)
139 {
140     /*
141      * If the user wants hardware cache aligned objects then follow that
142      * suggestion if the object is sufficiently large.
143      *
144      * The hardware cache alignment cannot override the specified
145      * alignment though. If that is greater then use it.
146      */
147     if (flags & SLAB_HWCACHE_ALIGN) {
148         unsigned int ralign;
149 
150         ralign = cache_line_size();
151         while (size <= ralign / 2)
152             ralign /= 2;
153         align = max(align, ralign);
154     }
155 
156     if (align < ARCH_SLAB_MINALIGN)
157         align = ARCH_SLAB_MINALIGN;
158 
159     return ALIGN(align, sizeof(void *));
160 }
```

- 147 SLAB_HWCACHE_ALIGN 表示需要按照硬件 cacheline 对齐。
- 150 获取 cache line 大小作为对齐基准, 通常为 64 字节。
- 151 根据指定对齐参数 align，object size(对象大小) 以及 cacheline(硬件缓存行) 大小综合计算出一个合适的对齐参数 ralign。
- 156 ARCH_SLAB_MINALIGN 为 slab 设置的最小对齐参数，8 字节大小，align 不能小于该值。
- 159 与 word size 进行对齐。

NOTE

为结构体添加 ___cacheline_aligned 关键字 (<linux/cache.h>)，可强制其按 L1 缓存行大小对齐。

___aligned(x): 通用对齐宏，可手动指定对齐字节数（如 ___aligned(64) 强制按 64 字节对齐），本质与 ___cacheline_aligned 一致，但需显式指定大小。

___cacheline_aligned 是内核通过宏定义的对齐属性，其定义依赖于 cpu 架构的缓存行大小（如 x86 为 64 字节），本质是利用 GCC 的 attribute((aligned(...))) 扩展实现。

___cacheline_aligned 和 kmem_cache_create() 中的 SLAB_HWCACHE_ALIGN 标志目标一致但应用场景不同，两者都用于保证内存对象按 CPU 缓存行（cache line）对齐，以优化硬件缓存利用率，但作用对象和触发时机不同。

特性	___cacheline_aligned	SLAB_HWCACHE_ALIGN
作用对象	静态定义的结构体	由 slub 分配器动态分配的对象
使用方式	作为结构体定义的属性 (编译时生效)	作为创建缓存池时的参数 (运行时生效)
对齐时机	编译时由编译器保证对齐	运行时由 slab 分配器分配 obj 内存对象时保证对齐
典型场景	全局数据结构 (如内核中的锁、计数器、SMP 共享数据)	频繁动态分配的小对象 obj(如 task_struct 等)

1.3 create_cache()

就内存视角而言，不同对象的缓存 (cache) 在最顶层通过一个“缓存链 (cache chain)” 的双向循环链表链接在一起。从 slab 分配器的视角而言，缓存链表上的“缓存 (cache)” 是特定类型对象 (如 mm_struct 结构体等) 的管理结构体，分别对应各个 kmem_cache 结构体实例。不同类型对象的缓存通过 kmem_cache 结构体中的 next 字段相互链接成链表。

每个特定对象类型的 kmem_cache 全局唯一。但它会为每个内存节点（node）分别维护一个独立的子缓存（per-node cache），以适配 NUMA 架构的本地内存访问优化。

可以把 kmem_cache 想象成一家全球性汽车制造公司 (比如特斯拉)。

- 汽车制造公司总部 (kmem_cache)
- 汽车制造公司分部 (kmem_cache_node):

  为了利用各地的生产和人力资源，以及便于销售，特斯拉在各洲建立了公司分部。struct kmem_cache 中包含一个 struct kmem_cache_node *node 数组，每个元素对应系统中的一个 NUMA 节点。
- 分布式的工厂 (slab):

  为了降低运输成本、提高效率，特斯拉在世界各地（美洲、亚洲、欧洲）建立工厂。每个洲可能会分别建立多个工厂。每个洲的工厂分别由各洲的分公司管理。即每个节点的 slab 由对应的 kmem_cache_node 管理，而非 kmem_cache 直接管理。
- 特斯拉 Mode Y(slab obj)

  不同大洲上的多个工厂生产同一车型 Mode Y。用于生产“Mode Y” 的模具和设计图纸是特斯拉全球唯一的，并且规格统一。这就像内核中分配不同线程的特定管理结构体 task_struct，这个对象的大小是一样的。
- 本地的原材料 (zone):

  每个洲上如果有多个工厂，通常这些工厂会建在这个洲的不同国家或城市。为了节省成本会从离工厂近的当地的仓库获取生成原材料。这个“当地仓库” 就是 node 节点的内存域（zone）。一个在 Node 0 的 slab，它的物理页面就是从 Node 0 的 ZONE_NORMAL 等 zone 通过伙伴系统分配来的。

结构体 struct kmem_cache 用于跟踪特定对象的 slab 块相关的状态，slab 分配器负责管理缓存块中可供分配的对象（slab obj）。

当一个 slub 未被完全分配 (仍有空闲对象) 时，会被标记为 “partial”，并通过 next 指针与其他 partial slub 串联在一起形成链表，便于 CPU 快速访问本地缓存的 slub 空闲对象。

特定某个对象的 slab 块所占用的页数是在创建缓存池时通过计算确定的固定值，同一类对象的所有 slab 块页数完全相同，不会因分配顺序或时机而变化

create_cache 函数的主要任务就是为 slub cache 创建它的内核管理数据结构 struct kmem_cache。

随后内核会为其创建 slub cache 的本地 cpu 结构 kmem_cache_cpu，每个 cpu 对应一个这样的缓存结构，用于无锁化快速分配释放对象。

```
struct kmem_cache {
    struct kmem_cache_cpu __percpu *cpu_slab;
    ... ...
}
```

最后为 slab cache 创建 NUMA 节点缓存结构 kmem_cache_node，每个 NUMA 节点对应一个。

```
struct kmem_cache {
    ... ...
    struct kmem_cache_node *node[MAX_NUMNODES];
}
```

当 slab cache 的整个骨架被创建出来之后，内核会为其在 sys 文件系统中创建 ***/sys/kernel/slab/-name*** 目录节点，用于详细记录该 slab cache 的运行状态以及行为信息。最后将新创建出来的 slab cache 添加到全局双向链表 list 的末尾。

create_cache() 源码如下:

kmem_cache_create() > *kmem_cache_create_usercopy()* > *create_cache()*

```
137 static struct kmem_cache *create_cache(const char *name,
138         unsigned int object_size, unsigned int align,
139         slab_flags_t flags, unsigned int useroffset,
140         unsigned int usersize, void (*ctor)(void *),
141         struct kmem_cache *root_cache)
142 {
143     struct kmem_cache *s;
144     int err;
145 
146     if (WARN_ON(useroffset + usersize > object_size))
147         useroffset = usersize = 0;
148 
149     err = -ENOMEM;
150     s = kmem_cache_zalloc(kmem_cache, GFP_KERNEL);
151     if (!s)
152         goto out;
153 
154     s->name = name;
155     s->size = s->object_size = object_size;
156     s->align = align;
157     s->ctor = ctor;
158     s->useroffset = useroffset;
159     s->usersize = usersize;
160 
161     err = __kmem_cache_create(s, flags);
162     if (err)
163         goto out_free_cache;
164 
165     s->refcount = 1;
166     list_add(&s->list, &slab_caches);
167 out:
168     if (err)
169         return ERR_PTR(err);
170     return s;
171 
172 out_free_cache:
173     kmem_cache_free(kmem_cache, s);
174     goto out;
175 }
```

- 146-147 用户使用区域起始偏移 + 用户区域长度超过 obj 总大小，直接清零配置，避免越界访问。
- 150 为将要创建的 slub cache 分配管理结构体 kmem_cache ，此处会从 kmem_cache 专属的 slab cache 中申请一个 kmem_cache 对象。内核在启动阶段，专门为 struct kmem_cache 创建其专属的 slab cache 缓存，保存在全局变量 ***kmem_cache*** 中。

```
        struct kmem_cache *kmem_cache;
```

- 154-159 利用指定的参数初始化 kmem_cache 结构。
- 161 这是创建 slab cache 的核心函数，会初始化 kmem_cache 结构中的其他重要属性，包括创建并初始化 kmem_cache_cpu 和 kmem_cache_node 结构。
- 165 slab cache 初始状态下，引用计数为 1。
- 166 将刚刚创建出来的 slab cache 加入到 slab cache 在内核中的全局链表管理。

1.3.1 ___kmem_cache_create()

___kmem_cache_create() 函数的主要工作就是建立 slub cache 的基本骨架，包括初始化 kmem_cache 结构中的其他重要属性，创建初始化本地 cpu 缓存结构以及 NUMA 节点缓存结构，这一部分的重要工作封装在 kmem_cache_open 函数中完成。

kmem_cache_create() → *kmem_cache_create_usercopy()* → *create_cache()* → *__kmem_cache_create()*

```
4432 int __kmem_cache_create(struct kmem_cache *s, slab_flags_t flags)
4433 {
4434     int err;
4435
4436     err = kmem_cache_open(s, flags);
4437     if (err)
4438         return err;
4439
4440     /* Mutex is not taken during early boot */
4441     if (slab_state <= UP)
4442         return 0;
4443
4444     err = sysfs_slab_add(s);
4445     if (err)
4446         __kmem_cache_release(s);
4447
4448     return err;
4449 }
```

- 4436 这一步完成缓存底层资源初始化，是整个缓存创建的核心
- 441-4442 系统处于未完全就绪的早期启动阶段，缓存创建成功，但 sysfs 子系统未准备好，直接返回 0。

  全局变量 slab_state 存储着 slab 组件当前状态。各值分别代表内核 slab 启动的多个阶段 (如下表所示):

枚举值	意义
DOWN	尚未提供任何 *slab* 内存分配功能
PARTIAL	部分就绪 *(SLUB* 专属阶段*):* 已可分配 *NUMA* 节点管理结构体 *kmem_cache_node*，但 *CPU* 本地缓存、*sysfs*、统计调试等配套功能未就绪。
PARTIAL_NODE	节点部分就绪 *(SLAB* 分配器专属*):* 已能通过 *kmalloc* 分配节点管理结构体内存
UP	分配器可用，但 *sysfs* 等附加配套功能未就绪。
FULL	全部功能就绪

- 4444 在 ***/sys/kernel/slab/*** 下为当前缓存创建专属 sysfs 属性目录，对外提供缓存运行时信息。
- 4445-4446 如果 sysfs 节点创建失败 (譬如:sysfs 挂载异常、节点重名、内存不足)，回滚 kmem_cache_open 分配的底层资源，防止资源泄漏。

kmem_cache_create() ··· *create_cache()* *__kmem_cache_create()* *kmem_cache_open()*

```
3783 static int kmem_cache_open(struct kmem_cache *s, slab_flags_t flags)
3784 {
3785     s->flags = kmem_cache_flags(s->size, flags, s->name, s->ctor);
3786 #ifdef CONFIG_SLAB_FREELIST_HARDENED
3787     s->random = get_random_long();
3788 #endif
3789 
3790     if (!calculate_sizes(s, -1))
3791         goto error;
3792     if (disable_higher_order_debug) {
3793         /*
3794          * Disable debugging flags that store metadata if the min slab
3795          * order increased.
3796          */
3797         if (get_order(s->size) > get_order(s->object_size)) {
3798             s->flags &= ~DEBUG_METADATA_FLAGS;
3799             s->offset = 0;
3800             if (!calculate_sizes(s, -1))
3801                 goto error;
3802         }
3803     }
3804 
3805 #if defined(CONFIG_HAVE_CMPXCHG_DOUBLE) && \
3806     defined(CONFIG_HAVE_ALIGNED_STRUCT_PAGE)
3807     if (system_has_cmpxchg_double() && (s->flags & SLAB_NO_CMPXCHG) == 0)
3808         /* Enable fast mode */
3809         s->flags |= __CMPXCHG_DOUBLE;
3810 #endif
3811 
3812     /*
3813      * The larger the object size is, the more pages we want on the partial
3814      * list to avoid pounding the page allocator excessively.
3815      */
3816     set_min_partial(s, ilog2(s->size) / 2);
3817 
3818     set_cpu_partial(s);
3819 
3820 #ifdef CONFIG_NUMA
3821     s->remote_node_defrag_ratio = 1000;
3822 #endif
3823 
3824     /* Initialize the pre-computed randomized freelist if slab is up */
3825     if (slab_state >= UP) {
3826         if (init_cache_random_seq(s))
3827             goto error;
3828     }
3829 
3830     if (!init_kmem_cache_nodes(s))
3831         goto error;
3832 
3833     if (alloc_kmem_cache_cpus(s))
3834         return 0;
3835 
3836     free_kmem_cache_nodes(s);
3837 error:
```

```
3838     return -EINVAL;
3839 }
```

- 3790 calculate_sizes() 根据传入的 slub obj 的 size, 和内核 config 配置项 (条件编译) 来确定最终 obj 的整体大小和警戒区、slub tracking、KASAN 等元数据区的排布。
- 3792-3803 若禁止大阶 slub 的调试功能，那么在 obj+ 元数据 (警戒区、调试跟踪区、填充区等) 之后的大小对应的阶数 >obj 的真实 payload 折算的 order 时，清除 slub 调试标记，重新计算或考虑 obj 的大小和布局。
- 3805-3810 ======CMPXCHG 待补充
- 3816 用 slub obj+ 元数据后的 size 做对数缩放计算，得到 NUMA**每个 node 下 partial 链表最低保留 slab 的数量**。

  partial 链表会保留最低 slub 数量。即使某些 slub 已经完全空 (empty, 换言之,slub 内的 obj 全部空闲，一个也没被占用) 了，也依旧留在 slub partial 链表**不回收**。只有调用 kmem_cache_shrink() 主动回收缓存时才会回收。

  因为一有空 (empty)slub 就立刻释放到伙伴系统，下次分配对象时又要重新向伙伴系统申请连续页创建 slub，这样频繁操作开销很大。所以，设置保留最小 slub 阈值，缓存少量空的 slub 做缓冲，减少频繁操作物理页。主动通过 kmem_cache_shrink() 回收时才会无视阈值。

  obj 的 size 越大，单个 slub 容纳的 obj 数量相应越少，就越容易变成 empty 的 slub。因此内核希望 obj 的 size 越大，保留的最小 slub 数量的阈值也大，也就是多保留 slub 数量。

  函数 set_min_partial() 之所以取对数是为了 size 增大时，取值平缓。另外该函数将保留 slub 数量的阈值钳位在 $[5, 10]$。
- 3818 设置**单个 CPU 本地 partial 链表最多缓存的空闲 obj 的上限**。

  每个 CPU core 都有一个私有 kmem_cache_cpu->partial 存放空闲对象的 slub。slub 被 frozen 冻结，仅当前 CPU 操作，无锁竞争。而->cpu_partial 用于设定每个 core 的 partial 缓存链表可存放空闲 obj的最大数量。这个设定值决定了两项行为:

  - 当 cpu core 的私有 partial 缓存链表的 obj 数量达到上限时，需要转移至 node 缓存链表的对象数量。
  - 当本地 cpu 缓存的空闲 obj 耗尽时，从 node 缓存链表拉取至本地 cpu 私有缓存的对象数目。内核只会拉取总量 (这里的总量就是 cpu 私有 partial 链表可存放空闲 onj 的最大数量) 的**50%**，以此保留一部分缓存容量，用于后续 obj 的释放操作。

  每个 core 的 partial 链表主要存放仅释放出一个空闲对象的 slab。这些 slub 不会进入各个 node 的 partial 缓存链表。

  后续分配内存时复用这些 slub 时，操作无需获取锁。譬如: 某个 slab 共 10 个 obj，都已经被申请占用。0 空闲。现在其中一个 obj 被释放，那目前 1 个 obj 空闲，9 个被占用。这个 slab 就会被移入本地 cpu 的 partial 链表。需要注意的是，移入仅针对反向操作 (释放 obj) 的场景，正向操作不会将 slab 移入。譬如: 某个 slab 共 10 个 obj，8 个已经被申请，2 个空闲。如果再占用 1 个，此时 1 个 obj 空闲，9 个被占用。此种操作不会导致 slab 被移入本地 cpu 的 partial 链表。

  cpu_partial 和 min_partial 的对比:

变量	归属	管控对象	含义
min_partia	*NUMA node* 共享链表	*slub* 缓存数量下限	*node* 全局 *partial* 链表至少保留多少 *slub(*空 *slub* 也不回收*)*
cpu_partial	单 *CPU* 私有缓存	空闲 *obj* 数量上限	本地 *CPU partial* 链表最多存多少空闲 *obj*, 超出则刷到 *NUMA node* 共享链表。

该函数的实现逻辑是:obj 越大,cpu 本地缓存空闲 obj 数量要越小。缓存多了会浪费内存。

- 3820-3822 在 CONFIG_NUMA 开启时存在，控制 slub 分配时是否可以跨 NUMA 节点去其他 node 借用 partial 链表上的 slab。

  这个比例值控制一个取舍: 分配时本地 node 无可用 partial slab, 是新建本地 slub 还是遍历 node 节点借用其他 node 的 partial slab 来填充。当值 = 0 时绝不跨节点; 当 = 1000(也就是 100%)，优先跨节点借用，减少内存碎片化，但引入了延时。

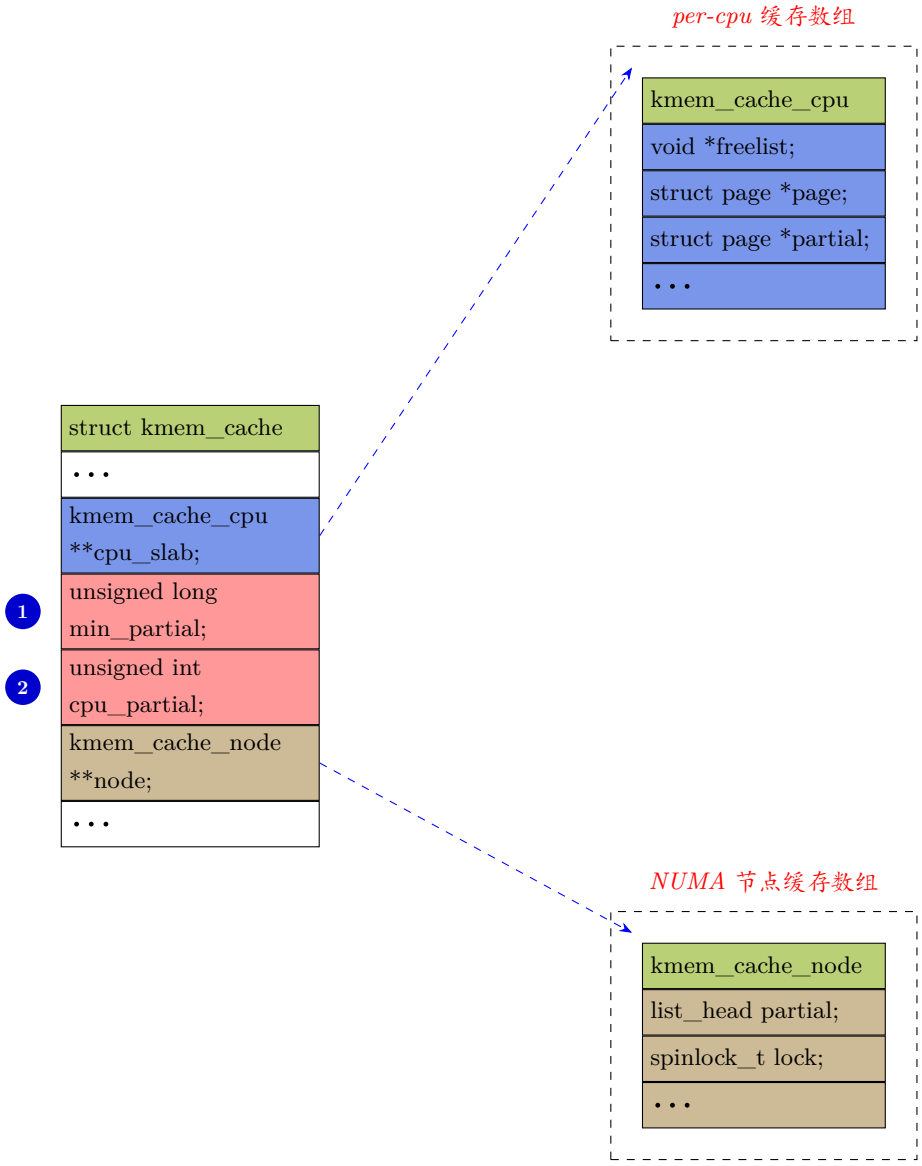
- 3825-3828 当前 slub 初始化已完成，初始化随机空闲对象链表。

  UP 表示 slub 组件的状态 (见 ___kmem_cache_create() 函数)。

- 3830 为新创建的 slub 描述符 struct kmem_cache 的每个 NUMA 节点，分配并初始化专属的节点管理结构 ***kmem_cache_node***。

- 3833 为新的 kmem_cache 结构分配并初始化 per-CPU 的本地私有缓存的管理结构体 ***kmem_cache_cpu***, 这是 per-cpu 缓存无锁操作的底层支撑。

  下图示意了 kmem_cache 的重要成员和指向的 kmem_cache_node 和 kmem_cache_cpu 结构。



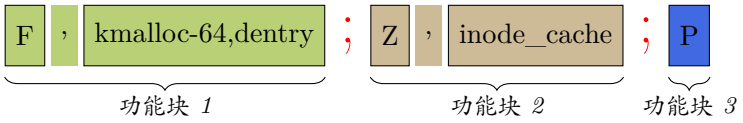
说明:

1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。

========== 待添加 3830,3833 两个函数的解析

slub_debug_flags() 函数用于从内核启动时传递的 init 选项 ***slub_debug=*** 中，解析出 slub 调试 flag 参数。

slub_debug 参数可以同时传递多个调试策略 (功能块)，在传入的字符串中这些功能块之间以";" 分隔 (如下图)。



kmem_cache_flags() 对 ***slub_debug*** 传递进来的参数进行解析。

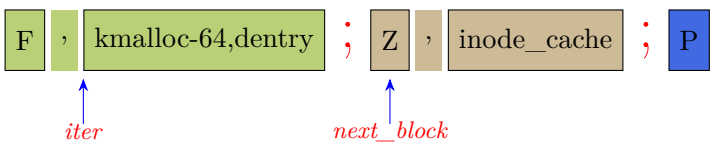
kmem_cache_create() → ··· → *__kmem_cache_create()* → *kmem_cache_open()* → *kmem_cache_flags()*

```
1407 slab_flags_t kmem_cache_flags(unsigned int object_size,
1408     slab_flags_t flags, const char *name,
1409     void (*ctor)(void *))
1410 {
1411     char *iter;
1412     size_t len;
1413     char *next_block;
1414     slab_flags_t block_flags;
1415 
1416     len = strlen(name);
1417     next_block = slub_debug_string;
1418     /* Go through all blocks of debug options, see if any matches our slab's name */
1419     while (next_block) {
1420         next_block = parse_slub_debug_flags(next_block, &block_flags, &iter, false);
1421         if (!iter)
1422             continue;
1423         /* Found a block that has a slab list, search it */
1424         while (*iter) {
1425             char *end, *glob;
1426             size_t cmplen;
1427 
1428             end = strchrnul(iter, ',');
1429             if (next_block && next_block < end)
1430                 end = next_block - 1;
1431 
1432             glob = strnchr(iter, end - iter, '*');
1433             if (glob)
1434                 cmplen = glob - iter;
1435             else
1436                 cmplen = max_t(size_t, len, (end - iter));
1437 
1438             if (!strncmp(name, iter, cmplen)) {
1439                 flags |= block_flags;
1440                 return flags;
1441             }
1442 
1443             if (!*end || *end == ';')
1444                 break;
1445             iter = end + 1;
1446         }
1447     }
1448 
1449     return flags | slub_debug;
1450 }
```

- 1417 获取启动时通过 ***slub_debug*** 传递的字符串参数的指针。
- 1420 解析目标字符串，从目标字符串的第一个功能块中获取对应的 slub 调试参数项。

  next_block 指向目标字符串的下一个功能块。iter 指向当前处理的功能块中 slub 缓存名称相关子块的首地址。

  假设传入字符串参数进行第一次进入 1420 行处理。那么 next_block 和 iter 指针指向如下图:



- 1421-1422 当前功能块的策略字符串中没有携带 slub 缓存名称，那么跳过下面的处理，直接重新迭代，处理下一个功能块。

策略参数中不带 slub 缓存名称，代表是全局策略。

- 1424 下面的代码块对每个目标功能块携带的缓存名称参数部分迭代解析。每一次迭代只对一个目标功能块中的参数部分处理。
  - 1428 找到下一个”,” 号的位置。
  - 1429-1430 如果”,” 在下一个功能块内部，那么代表当前缓存名称字串中不再有”,”, 当前处理字串是目标功能块的最后一个 slaub 缓存名称。因此，slub 缓存字串的结束位置在下一个功能块前一个字节。

    如果”,” 不在下一个功能块中，那么当前字串的首地址到”,” 之间部分就是待取出的 slub 缓存名称。
  - 1432 在前面检索出的 slub 缓存名称参数字符串中，检索出”*” 号的位置。

    * 号类似 shell 中的通配符。
  - 1433-1434 如果缓存名称字串中有”*” 号，那么，和当前要创建缓存描述符的 slub 名称匹配比较时，就只比较到通配符”*” 之前的字符，长度是 $(glob - iter)$。
  - 1435-1436 如果缓存名称字符串中没有”*” 号，就判断需要创建缓存描述符 kem_cache 的缓存名称，和目标参数字串哪个 len 更长. 按最长 len 进行字符串比较。
  - 1438-1441 如果 slub_debug 传递的调试参数中配置了当前缓存的调试策略—也就是 slub 名称字符串和传入 name 匹配—就返回当前 slub 缓存的调试策略 flag。
  - 1443-1444 如果当前功能块的缓存名称块部分结束了，就跳出解析，进入下一个功能块的处理。
  - 1445 否则，$end + 1$ 指向下一个名称，再继续检索共用同一个 slub 调试策略的不同名称缓存。

parse_slub_debug_flags() 负责解析出传递过来字符串中的首个功能块。对整个 slub_debug 参数中的多个功能块的迭代解析，则是由上一层函数循环迭代中调用 parse_slub_debug_flags() 实现的。

kmem_cache_create() ⋯ *kmem_cache_open()* *kmem_cache_flags()* *parse_slub_debug_flags()*

```
1267 static char *
1268 parse_slub_debug_flags(char *str, slab_flags_t *flags, char **slabs, bool init)
1269 {
1270     bool higher_order_disable = false;
1271 
1272     /* Skip any completely empty blocks */
1273     while (*str && *str == ';')
1274         str++;
1275 
1276     if (*str == ',') {
1277         /*
1278          * No options but restriction on slabs. This means full
1279          * debugging for slabs matching a pattern.
1280          */
1281         *flags = DEBUG_DEFAULT_FLAGS;
1282         goto check_slabs;
1283     }
1284     *flags = 0;
1285 
1286     /* Determine which debug features should be switched on */
1287     for (; *str && *str != ',' && *str != ';'; str++) {
1288         switch (tolower(*str)) {
1289         case '-':
1290             *flags = 0;
```

```
1291                break;
1292            case 'f':
1293                *flags |= SLAB_CONSISTENCY_CHECKS;
1294                break;
1295            case 'z':
1296                *flags |= SLAB_RED_ZONE;
1297                break;
1298            case 'p':
1299                *flags |= SLAB_POISON;
1300                break;
1301            case 'u':
1302                *flags |= SLAB_STORE_USER;
1303                break;
1304            case 't':
1305                *flags |= SLAB_TRACE;
1306                break;
1307            case 'a':
1308                *flags |= SLAB_FAILSLAB;
1309                break;
1310            case 'o':
1311                /*
1312                 * Avoid enabling debugging on caches if its minimum
1313                 * order would increase as a result.
1314                 */
1315                higher_order_disable = true;
1316                break;
1317            default:
1318                if (init)
1319                    pr_err("slub_debug option '%c' unknown. skipped\n", *str);
1320            }
1321        }
1322    check_slabs:
1323        if (*str == ',')
1324            *slabs = ++str;
1325        else
1326            *slabs = NULL;
1327
1328        /* Skip over the slab list */
1329        while (*str && *str != ';')
1330            str++;
1331
1332        /* Skip any completely empty blocks */
1333        while (*str && *str == ';')
1334            str++;
1335
1336        if (init && higher_order_disable)
1337            disable_higher_order_debug = 1;
1338
1339        if (*str)
1340            return str;
1341        else
1342            return NULL;
1343    }
```

- 1273-1274 如果传递的参数字符串首个字符是";", 则字符指针 +1。这是为了把冗余的前置";" 跳过，使指针指向真正的有效字符。
- 1276-1283 第一个字符如果是"," 则使用默认 debug 选项。然后跳转至 check_slabs 处执行。

  在一个功能块中，首个"," 号前是调试功能选项，之后是 slub cache 名称。如果首个"," 之前是空的，也就是无自定义 flag。那么默认开启全套 slub debug 选项标志。

  check_slabs 标签处，首先判断是否存在slab 缓存名的参数块 ("," 号之后的部分)，没有则 $*slabs =$

NULL。

有则将”,” 字符之后的部分赋值给指针 slabs(传出参数), 上层函数会从 slabs 指针解析出缓存名称。

如果存在缓存名称的参数块，那么继续迭代字符指针指向在此之后的第一个”;” 之后的字符。换言之，也就是指向下一个功能块的第一个有效字符。然后把该指针返回给上一层的调用函数。

- 1287-1321 如果第一个有效字符不是”,”, 那么这个代码块解析当前功能块中第一个”,” 之前的各个参数 (每个字符表示不同的调试项)，并设置对应的 flag 位。迭代遍历当前代码块的功能项字符直到碰到字符”,” 表示功能项部分结束，在”,” 之后是 slab 缓存的名称。如果功能项是针对全局的，没有”,” 字符，那么碰到”;” 字符就跳出循环，表示下面是另一个功能块。

  SLUB Debug 调试选项功能说明如下:

参数	功能说明
f	一致性校验：分配/释放 *obj* 时，完整校验 *slab obj* 元数据完整性，捕获内存越界、结构体损坏。
z	警戒区防护：在每个 *obj* 首尾插入警戒区，检测缓冲区写越界，越界会触发 *Oops* 告警
p	内存污化：释放对象时填充毒魔数值，分配前校验污化值，捕获释放后的写 *(UAF)*、重复释放
u	保存调用栈：记录 *obj* 分配、释放的调用函数栈，报错时打印完整用户调用栈信息
t	完整追踪：开启 *trace*，跟踪记录分配/释放 *slab* 页面的线程 *ID*、本地 *cpu*、以及分配/释放时刻。
a	故障 *slab* 标记：检测到 *obj* 被破坏时直接标记整个 *slab*，快速隔离被破坏的缓存
o	大阶页禁止调试：若开启调试会导致 *slub* 最小分配占用更多连续物理页，则禁止对该缓存启用调试，避免内存碎片化加剧

  若开启调试选项导致目标 slub 的最小分配阶数变大，则禁止为该 slub 启用调试功能。

- 1322 下面就是 check_slubs 标签处的处理 (见前面)。
  - 1336-1337 需要注意的是, 如果是系统启动时调用该函数 ($init == true$), 并且由于进展因为调试参数的打开提高了对应 slub 缓存块页的阶数，那么需要置 $disable_higher_order_debug = 1$。如果是在正常运行过程中 ($init = false$) 调用该函数，那么不考虑这种情况。因为启动时如果需要为目标 slub 配置该项，那么目前已经配置了。如果后续通过 sysfs 配置，也已经太晚了，已经可能 slub 缓存已经分配了。

1.3.2 kmem_cache 结构体部分成员

- ***size*** 包括了元数据和和为了对齐而填充的字节之后 obj 的大小。
- ***object_size*** 不包括元数据和填充字节的 obj 的实际大小，该成员在调用 calculate_sizes() 之前已经初始化。
- ***offset*** 内核会在 slab 管理的 obj 内存中，存储下一个空闲 obj 的指针。这个存放位置距离 obj 首地址的偏移就是 offset.

include/linux/slub_def.h :: struct kmem_cache 部分成员

```
struct kmem_cache {
                ... ...
        unsigned int size;      /* The size of an object including metadata */
        unsigned int object_size;/* The size of an object without metadata */
        ... ...
        unsigned int offset;    /* Free pointer offset */
                ... ...
        struct kmem_cache_order_objects oo;

        /* Allocation and freeing of slabs */
        struct kmem_cache_order_objects max;
        struct kmem_cache_order_objects min;
                ... ...
        unsigned int inuse;             /* Offset to metadata */
```

- ***oo*** 该结构体实际是一个 unsigned int 型变量，包括 slab cache 所申请的页面数量对应的 order 值，以及所包含 obj 数量。其中低 16 位表示一个 slab cache 中所包含的 obj 总数，高 16 位表示 slab cache 所占有的内存页数对应的 order。
- ***max*** slab cahe 中所能包含 obj 以及内存页数的最大值对应的 order
- ***min*** 当按照 oo 的尺寸为 slab 申请内存时，如果内存紧张，会采用 min 的尺寸为 slab 申请内存，可以容纳一个 obj 即可
- ***inuse*** obj 的 object_size 按 word 对齐后的大小，也是 meta 数据区距离 obj 首地址的偏移。

1.3.3 calculate_sizes()

calculate_sizes() 用来确定 slab obj 的 order, 以及 obj 的内存分布情况. 还会初始化 kmem_cache 结构体的一些核心参数。calculate_sizes() 调用图如下：

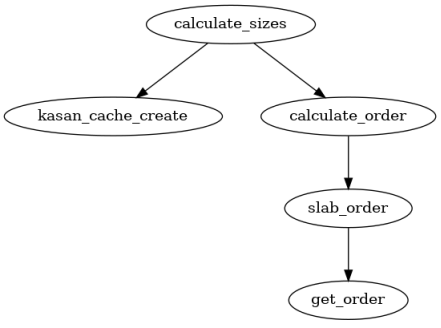


图 1–2: 调用图：calculate_sizes()

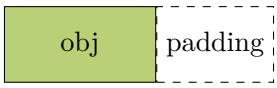
kmem_cache_create() ··· *__kmem_cache_create()* *kmem_cache_open()* *calculate_sizes()*

```
3637 /*
3638  * calculate_sizes() determines the order and the distribution of data within
3639  * a slab object.
3640  */
3641 static int calculate_sizes(struct kmem_cache *s, int forced_order)
3642 {
3643         slab_flags_t flags = s->flags;
3644         unsigned int size = s->object_size;
3645         unsigned int freepointer_area;
3646         unsigned int order;
```

```
3647
3648            /*
3649             * Round up object size to the next word boundary. We can only
3650             * place the free pointer at word boundaries and this determines
3651             * the possible location of the free pointer.
3652             */
3653            size = ALIGN(size, sizeof(void *));
3654            /*
3655             * This is the area of the object where a freepointer can be
3656             * safely written. If redzoning adds more to the inuse size, we
3657             * can't use that portion for writing the freepointer, so
3658             * s->offset must be limited within this for the general case.
3659             */
3660            freepointer_area = size;
```

- 3653 将 obj 按字 (word) 的大小对齐, 这里四舍五入向上对齐到下一个 word 的边界. 如果 obj size 本来就是与 word 对齐的，此处就不需要填充 padding.
- 3660 下一个空闲 obj 的指针 (FP) 可能放在 word 对齐的边界处. 也就是此时的偏移 size 处。

此时,slub obj 的 layout 如下所示:



接下来, 如果配置了 SLUB_DEBUG 就会进入下面代码:

calculate_sizes()

```
3662
3663    #ifdef CONFIG_SLUB_DEBUG
3664            /*
3665             * Determine if we can poison the object itself. If the user of
3666             * the slab may touch the object after free or before allocation
3667             * then we should never poison the object itself.
3668             */
3669            if ((flags & SLAB_POISON) && !(flags & SLAB_TYPESAFE_BY_RCU) &&
3670                            !s->ctor)
3671                    s->flags |= __OBJECT_POISON;
3672            else
3673                    s->flags &= ~__OBJECT_POISON;
3674
3675
3676            /*
3677             * If we are Redzoning then check if there is some space between the
3678             * end of the object and the free pointer. If not then add an
3679             * additional word to have some bytes to store Redzone information.
3680             */
3681            if ((flags & SLAB_RED_ZONE) && size == s->object_size)
3682                    size += sizeof(void *);
3683    #endif
```

- 3669–3674 这段代码用于决定是否” 污化”obj. 如果使用者在 obj 被 free 后或 alloc 之前会接触 obj, 那么就不能对 obj 进行” 污化”.SLAB_TYPESAFE_BY_RCU 类型意味着 obj free 之后仍可以 touch. 有构建函数 (s->ctor) 意味着 obj 的内容会保留到下一次 alloc.

- 3681-3682 如果配置了警戒区 (redzone), 那么先检查在 obj 和空闲指针之间是否有空闲区域. 如果没有则需要给 size 增加一个 word 来保存警戒区信息. 假设 obj 和空闲指针放置的位置 (对齐的边界) 之间没有空闲区域, 那么 layout 将如下:



如果 object_size 和对齐的边界之间有 padding 区，那么这段 padding 区可以作为右侧的警戒区，而不需要额外为右侧警戒区分配内存。示意图如下：

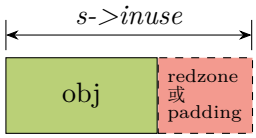


继续往下:

calculate_sizes()

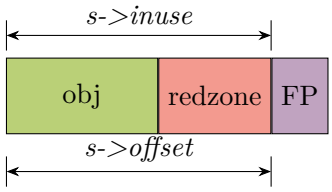
```
3686
3687        /*
3688         * With that we have determined the number of bytes in actual use
3689         * by the object. This is the potential offset to the free pointer.
3690         */
3691        s->inuse = size;
3692
3693        if (((flags & (SLAB_TYPESAFE_BY_RCU | SLAB_POISON)) ||
3694                s->ctor)) {
3695                /*
3696                 * Relocate free pointer after the object if it is not
3697                 * permitted to overwrite the first word of the object on
3698                 * kmem_cache_free.
3699                 *
3700                 * This is the case if we do RCU, have a constructor or
3701                 * destructor or are poisoning the objects.
3702                 *
3703                 * The assumption that s->offset >= s->inuse means free
3704                 * pointer is outside of the object is used in the
3705                 * freeptr_outside_object() function. If that is no
3706                 * longer true, the function needs to be modified.
3707                 */
3708                s->offset = size;
3709                size += sizeof(void *);
3710        } else if (freepointer_area > sizeof(void *)) {
3711                /*
3712                 * Store freelist pointer near middle of object to keep
3713                 * it away from the edges of the object to avoid small
3714                 * sized over/underflows from neighboring allocations.
3715                 */
3716                s->offset = ALIGN(freepointer_area / 2, sizeof(void *));
3717        }
```

- 3691 这个 offset 是空闲指针潜在的放置位置。

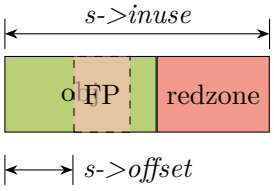


为方便起见，之后示意图中右侧 padding 仍仅用 redzone 表示。

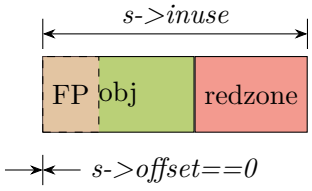
- 3693-3709 当使用 RCU, 拥有构建函数, 析构函数或需要污化 obj 时, 才进入该分支. 即, 如果在 kmem_cache_free() 中不允许覆盖 obj 的第一个 word, 那么需要把空闲指针放置到 obj 之后. 此时将 size 再增加一个 word. 此时 layout 如下:



- 3710-3716 如果 obj 的 size 大于一个 word, 那么可以把空闲指针放在 obj 大小的 1/2 左右处. 这个位置也需要按照 word 对齐. 之所以这么做是为了远离 obj 的边界, 防止邻近 obj 的 alloc 造成小的 overflow 或 underflow. 此时 layout 如下:



如果 obj 的 size 不大于一个 word, 并且也没有使用 RCU, 也不存在构建函数, 也没有打开” 污化” 配置. 那么默认 FP 在 offset 为 0 处, 如下所示:

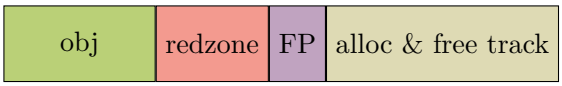


如果配置了 SLUB_DEBUG 则继续往下:

calculate_sizes()

```
3719
3720 #ifdef CONFIG_SLUB_DEBUG
3721         if (flags & SLAB_STORE_USER)
3722                 /*
3723                  * Need to store information about allocs and frees after
3724                  * the object.
3725                  */
3726                 size += 2 * sizeof(struct track);
3727 #endif
```

- 3721-3726 如果需要跟踪 slub obj 使用者的信息 (包括 alloc 和 free), 则 size 增加两个 struct track 的大小. 此时 layout 如下 (假设 FP 此时在 obj 外):



接下来是 3731 行:

calculate_sizes()

```
3730
3731         kasan_cache_create(s, &size, &s->flags);
```

该函数如下

calculate_sizes() > *kasan_cache_create()*

```
224 void kasan_cache_create(struct kmem_cache *cache, unsigned int *size,
225                          slab_flags_t *flags)
226 {
227         unsigned int orig_size = *size;
228         unsigned int redzone_size;
229         int redzone_adjust;
230
231         /* Add alloc meta. */
232         cache->kasan_info.alloc_meta_offset = *size;
233         *size += sizeof(struct kasan_alloc_meta);
234
235         /* Add free meta. */
236         if (IS_ENABLED(CONFIG_KASAN_GENERIC) &&
237             (cache->flags & SLAB_TYPESAFE_BY_RCU || cache->ctor ||
238              cache->object_size < sizeof(struct kasan_free_meta))) {
239                 cache->kasan_info.free_meta_offset = *size;
240                 *size += sizeof(struct kasan_free_meta);
241         }
242
243         redzone_size = optimal_redzone(cache->object_size);
244         redzone_adjust = redzone_size - (*size - cache->object_size);
245         if (redzone_adjust > 0)
246                 *size += redzone_adjust;
```

```
247
248         *size = min_t(unsigned int, KMALLOC_MAX_SIZE,
249                       max(*size, cache->object_size + redzone_size));
250
251         /*
252          * If the metadata doesn't fit, don't enable KASAN at all.
253          */
254         if (*size <= cache->kasan_info.alloc_meta_offset ||
255                       *size <= cache->kasan_info.free_meta_offset) {
256                 cache->kasan_info.alloc_meta_offset = 0;
257                 cache->kasan_info.free_meta_offset = 0;
258                 *size = orig_size;
259                 return;
260         }
261
262         *flags |= SLAB_KASAN;
263 }
```

- 232-233 实际 size 增加一个 struct kasan_alloc_meta 结构体的大小, 以便储存 alloc 的 meta 信息.
- 236-241 判断 slub obj size 是否小于 kasan_free_meta 结构体 size 大小. 如果是, 则 240 行将 size 再增加一个 kasan_free_meta 结构体 size 大小. 此时 layout 如下:



如果 slub obj size 大于 kasan_free_meta 结构体 size 大小, 则 free 时会将 kasan_free_meta 结构体信息从 obj 起始 0 处开始存储.layout 如下:



- 243 计算最佳警戒区长度. 从注释看,optimal_redzone() 函数中这段代码来自用户空间 AddressSanitizer 工具运行时的警戒区适配策略.
- 244-246 将最佳警戒区长度减去 (目前实际 size - slub obj 的 size), 求得差值. 如果差大于 0, 说明各种 meta 数据占用的空间长度仍为达到最优警戒区长度. 这时继续增大 size, 以达到最优警戒区长度.
- 248-249

  首先, 这个 max 会判断当前 size 和 (slub obj + 最优警戒区 size) 长度的大小, 并取其中最大值. 这里和 244-246 行合起来看就是, 如果 slub obj 和各种 meta 数据合起来的长度小于最优警戒区长度, 则 size 扩大; 反之, 按实际加起来的长度取 size.

  然后, 这个 size 长度又受 kmalloc 最大分配长度的限制.KMALLOC_MAX_SIZE 各平台和架构或有不同. 这里取二者中较小的.

- 254-260 当 metadata 数据不合适, 就恢复 cache 结构体到传递进来时的处时值.(comlee: 此处没有完全看懂在什么情况下进入该条件, 也许是 KMALLOC_MAX_SIZE 在某些情况下定义得太小? 亦或是 size 太大溢出回绕了?) 并且不使能 KASAN 功能.

回到 mm/slub.c :: calculate_sizes() 函数, 如果配置了 SLUB_DEBUG 功能, 则进入下面的代码:

calculate_sizes()

```
3732 #ifdef CONFIG_SLUB_DEBUG
3733         if (flags & SLAB_RED_ZONE) {
3734                 /*
3735                  * Add some empty padding so that we can catch
3736                  * overwrites from earlier objects rather than let
3737                  * tracking information or the free pointer be
3738                  * corrupted if a user writes before the start
3739                  * of the object.
3740                  */
3741                 size += sizeof(void *);
3742
3743                 s->red_left_pad = sizeof(void *);
3744                 s->red_left_pad = ALIGN(s->red_left_pad, s->align);
3745                 size += s->red_left_pad;
3746         }
3747 #endif
```

- 3733-3745 如果配置了 SLAB_RED_ZONE 警戒区, 需要继续扩展 size 大小, 以增加左侧警戒区.

  此处先增加了 1 个 word 的 padding, 这里 padding 的作用是: 如果用户在 obj 的实际内存开始之前写入内容, 那么我们就能捕获这种更早分配的 obj 踩到内存的情况. 而不会让当前 obj 的 tracking 信息或空闲指针被破坏.

  然后又给 red_left_pad 增加了一个 word. 需要注意的是此时 red_left_pad 的大小是和 HW cacheline(s->align) 对齐后的大小.size 也相应增加.

- 至此 layout 如下 (假设 free meta 信息在后面):



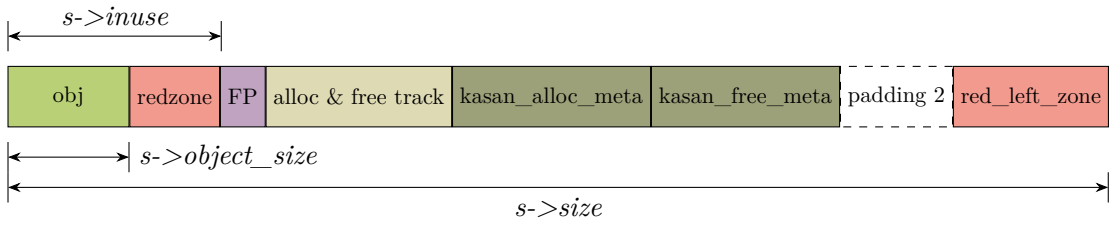
下面是 calculate_sizes() 剩余代码:

calculate_sizes()

```
3748
3749         /*
3750          * SLUB stores one object immediately after another beginning from
3751          * offset 0. In order to align the objects we have to simply size
3752          * each object to conform to the alignment.
3753          */
3754         size = ALIGN(size, s->align);
3755         s->size = size;
```

```
3756         s->reciprocal_size = reciprocal_value(size);
3757         if (forced_order >= 0)
3758                 order = forced_order;
3759         else
3760                 order = calculate_order(size);
3761
3762         if ((int)order < 0)
3763                 return 0;
3764
3765         s->allocflags = 0;
3766         if (order)
3767                 s->allocflags |= __GFP_COMP;
3768
3769         if (s->flags & SLAB_CACHE_DMA)
3770                 s->allocflags |= GFP_DMA;
3771
3772         if (s->flags & SLAB_CACHE_DMA32)
3773                 s->allocflags |= GFP_DMA32;
3774
3775         if (s->flags & SLAB_RECLAIM_ACCOUNT)
3776                 s->allocflags |= __GFP_RECLAIMABLE;
3777
3778         /*
3779          * Determine the number of objects per slab
3780          */
3781         s->oo = oo_make(order, size);
3782         s->min = oo_make(get_order(size), size);
3783         if (oo_objects(s->oo) > oo_objects(s->max))
3784                 s->max = s->oo;
3785
3786         return !!oo_objects(s->oo);
3787 }
```

- 3754-3755 将获取的实际 size 按硬件 cacheline 对齐后的值填充 kmem_cache 的 size 成员. 长度示意图如下:



  至此，获得了一个 slab obj 真实占用内存的大小 size，内核会根据这个 size，在物理内存页中划分出一个一个的 obj 出来。

- 3756-3763 从 kmem_cache_open() 调用 calculate_size() 的话, 传递的参数 forced_order 是-1. 因此会走 else 分支，通过 size 计算 order(阶). 也就是计算一个 slab 到底需要多少个物理内存页，并将结果转换成 order 值。calculate_order() 函数根据一定的算法计算出一个合理的 order 值, 函数原型见后面代码.

- 3781-3784 slub 所需的物理内存页个数计算出来之后，内核会根据 slub obj 占用内存的大小 size，计算出一个 slub 可以容纳的 obj 个数。并将这个结果保存在 kmem_cache 结构中的 oo 属性中。属性类型 struct kmem_cache_order_objects 这样一个结构保存 slub 所需要的物理内存页个数对应的

order 以及 slub 所能容纳的 obj 个数。它是一个 unsigned int 型值. 低 16 位存储 obj 总数，高 16 位存储 order 值.

3782 get_order() 代码分析在后面。

一个 slub 需要多少个物理内存页是在 calculate_sizes() 中计算出来的，内核会根据一定的算法，尽量保证 slub 中的内存碎片最小化，综合计算出一个合理 order 值的连续物理页。

calculate_sizes() > *calculate_order()*

```
3420 static inline int calculate_order(unsigned int size)
3421 {
3422         unsigned int order;
3423         unsigned int min_objects;
3424         unsigned int max_objects;
3425 
3426         /*
3427          * Attempt to find best configuration for a slab. This
3428          * works by first attempting to generate a layout with
3429          * the best configuration and backing off gradually.
3430          *
3431          * First we increase the acceptable waste in a slab. Then
3432          * we reduce the minimum objects required in a slab.
3433          */
3434         min_objects = slub_min_objects;
3435         if (!min_objects)
3436                 min_objects = 4 * (fls(nr_cpu_ids) + 1);
3437         max_objects = order_objects(slub_max_order, size);
3438         min_objects = min(min_objects, max_objects);
3439 
3440         while (min_objects > 1) {
3441                 unsigned int fraction;
3442 
3443                 fraction = 16;
3444                 while (fraction >= 4) {
3445                         order = slab_order(size, min_objects,
3446                                         slub_max_order, fraction);
3447                         if (order <= slub_max_order)
3448                                 return order;
3449                         fraction /= 2;
3450                 }
3451                 min_objects--;
3452         }
3453 
3454         /*
3455          * We were unable to place multiple objects in a slab. Now
3456          * lets see if we can place a single object there.
3457          */
3458         order = slab_order(size, 1, slub_max_order, 1);
3459         if (order <= slub_max_order)
3460                 return order;
3461 
3462         /*
3463          * Doh this slab cannot be placed using slub_max_order.
3464          */
3465         order = slab_order(size, 1, MAX_ORDER, 1);
3466         if (order < MAX_ORDER)
3467                 return order;
3468         return -ENOSYS;
3469 }
```

- 3434-3436 slab 能容纳的最少 obj 数目是 slub_min_objects。也就是理论上一个 slub 最优容纳的目标 obj 数。

  slub_min_objects 的数值可以在系统启动时通过命令行参数"***slub_min_objects=***" 来传入，默认为 0. 如果还是默认值 0, 此处会给 min_objects 另外赋值。

  nr_cpu_ids 表示当前系统中的 cpu core 的数目。fls() 是位操作函数, 返回 32bit 无符号整数中最左边置 1 的 bit 位的索引 (函数 ffs() 则是查找最右边置 1 的 bit 位的索引)。返回的数置是从 1 开始计数, 也就是 bit[0] 被置位则返回 1。返回 0 有特殊意义, 代表传入参数值全 0。

  此处通过计算 $4 \times (fls(nr_cpu_ids) + 1)$ 来计算 min_objects 是经验算法。它是一种对数级缩放的启发式算法。通过 fls(nr_cpu_ids) 计算最小 obj，这样最小 obj 数量会按 cpu 数目对数缓慢增长，而不会随 cpu core 数量线性暴涨。

- 3437-3438 计算按默认最大页面分配 order 可以分配指定 size 下的最多 slab obj 的数目。该数值和前面得到的 min_objects 值比较后取最小值, 来对 min_objects 更新或修正.

  slub_max_order 是 slub 组件设定的页阶数上限：slub 组件计算出的所需连续页阶数一旦超过 slub_max_order，就会放弃申请大阶连续物理内存，降低单个 slub 容纳的对象数量，改用更小 order($\leq slub_max_order$) 的连续页。不过，会产生更多数量的小 slub。也就是说把原本一块大 slub 承载的对象，分摊到多个更小的独立 slub 上。

  slub 最大允许占用的最大连续物理页阶 slub_max_order 的值，也可以通过启动参数"***slub_max_order***=" 传递。内核默认把 slub_max_order 设为 PAGE_ALLOC_COSTLY_ORDER(3)。PAGE_ALLOC_COSTLY_ORDER(3) 表示分配 $order \geq 3$(≥ 8 页) 的连续物理页属于高代价分配。

  order_objects() 利用 slub 最大允许占用的连续物理页阶，和申请的 obj 的 size, 来计算得到当前申请的目标 obj 在一个 slub 里允许的最多数目。公式如下:

$$\begin{aligned} max_objects &= \frac{PAGE_SIZE \times 2^{order}}{size} \\ &= \frac{\text{连续物理页的总字节数}}{\text{目标 } obj \text{ 的字节数}} \\ &= \text{连续物理页可容纳的 } obj \text{ 数量} \end{aligned}$$

  3438 行将系统允许的目标 slub 的最大和最小 obj 数目比较，取两者中最小的作为 obj 的最低数量。

- 3440-3452 利用给定 obj 的 size，算出合适的 slab order。

  系统希望一块 slub 装更多 obj。实在算不出合适 order，就逐步降低单 slub 能容纳的最少 obj 数。

  根据当前 fraction 计算 order，查找出能够使 slab 产生最小化碎片的 order 值. 但同时, 要满足 order 小于 slub_max_order 的条件, 否则就放宽对碎片大小的限制, 再次循环以找到合适的 order. 如果在内循环未找到, 则将 slab 可以容纳的最少 obj 数减少, 再次进入内循环计算, 直到获取合适的 order, 或者退出外循环.

  slab_order() 函数实现见后面.

- 3441-3443 fraction 是碎片比例控制参数，代表允许 slub 尾部空闲内存占总 slub 大小的比例为 $\frac{1}{fraction}$。初始赋值为 16，表示允许最多 slub 1/16 的空间作为内部碎片浪费。
- 3444 最大容忍 1/4 slub 空间的浪费，不再放宽碎片限制。
- 3445-3446 按给定的目标 slub obj 的 size 计算页分配 order。
- 3447-3448 如果本轮算出的 order 在限制以内，说明找到了合适 order，直接返回。
- 3449 本轮碎片限制行不通，把 $fraction/2$，放大允许的碎片比例，再次迭代。

  从代码看，允许的碎片比例会依次放宽:1/16 →1/8 →1/4。
- 3451 内层循环走完，在放宽碎片限制的情况下，内层循环每轮计算出的所需连续页的 order (页阶) 都超过 slub_max_order 时，减少每个 slub 存放的 obj 的数量，再次重试。这也是外循环的处理，目标 slub 每轮循环减少 1 个 obj。

- 3458-3467 进入到这个代码块意味着前面代码找寻在 slab 中放置 > 一个 obj 的努力失败了. 继续尝试是否能只放置一个 obj。

  如果失败则会放宽 order 限制到 ***MAX_ORDER***。也就是允许分配大阶连续页不受 ***slub_max_order*** 限制。

  MAX_ORDER 值一般是 11，$order < MAX_ORDER$, 所以对应的页数：$2^{(11-1)} = 2^{10} = 1024$ 页。一页通常默认 4KB，那么总大小是 $1024 \times 4\text{KB} = 4\text{MB}$。

> ***NOTE***
>
> *MAX_ORDER 是内核页分配器伙伴系统空闲链表数组 free_area[MAX_ORDER] 的长度。*
> *所有大阶连续物理页分配的硬上限对应就是:$MAX_ORDER - 1$。*

给定单个 slub obj 的大小，需要计算 slub 分配页阶 (order)。

页面分配阶 (order) 对系统性能、内存碎片、伙伴系统行为影响极大。常规场景下应当优先选用 order-0 ($2^0 = 1$) 页分配：order-0 仅申请单页，不会占用大阶连续物理页，避免造成伙伴系统外部碎片。

若单个 slub 中空闲浪费的空间占该 slub 总容量的比例达到 1/16 及以上，说明剩余空白内存占比很高，内存利用率差。则增大本次 slub 分配的连续页阶，也就是一次性分配更多连续页面，放大整块 slub 的总内存容量，这样同一块连续内存会容纳更多同规格对象。

若配置了该 slub 的最小分配阶 min_order，且该 min_order 大于适配当前 obj 尺寸的天然最小 order，则 slab 页分配将强制从 min_order 起步，而非选用刚好能放下 obj 的最低阶的连续页。

计算给定 obj 的 slub 分配页阶的函数如下:

calculate_sizes() > *calculate_order()* > *slab_order()*

```
3395 static inline unsigned int slab_order(unsigned int size,
3396                 unsigned int min_objects, unsigned int max_order,
3397                 unsigned int fract_leftover)
3398 {
3399         unsigned int min_order = slub_min_order;
3400         unsigned int order;
3401
3402         if (order_objects(min_order, size) > MAX_OBJS_PER_PAGE)
3403                 return get_order(size * MAX_OBJS_PER_PAGE) - 1;
3404
3405         for (order = max(min_order, (unsigned int)get_order(min_objects * size));
3406                         order <= max_order; order++) {
```

```
3407
3408                unsigned int slab_size = (unsigned int)PAGE_SIZE << order;
3409                unsigned int rem;
3410
3411                rem = slab_size % size;
3412
3413                if (rem <= slab_size / fract_leftover)
3414                        break;
3415        }
3416
3417        return order;
3418 }
```

- 3402-3403 按最小 order 值和 size 计算出可以容纳的 obj 的数量. 如果该值大于 ***MAX_OBJS_PER_PAGE***, 那么用 ***MAX_OBJS_PER_PAGE*** 钳位。将该值做为目标 slub 能容纳的 obj 数目。

  用obj 的数目×obj 的 size算出的总大小, 并折算成页阶值返回 (见后面 get_order() 函数的解析)。

  MAX_OBJS_PER_PAGE 宏值是 32767, 字面意思是每个 page 能容纳的最多 obj 数目。32767 = 2^{15}, 对于 4k bytes 的 page 来讲, 平均最少每个 obj 是 1/8byte. 这应该是不可能的, 每个 obj 加上 meta 数据 >1 byte。从该文件定义的该宏使用的地方看, 正确的名称恐怕应该是*MAX_OBJS_PER_ORDER*.

  3403 行为何减 1? 见 get_order() 函数的说明.

- 3405-3415 从 slab 所需要的最小 order 到最大 order 之间开始遍历, 查找能够使 slab 碎片最小的 order 值

  3411 按当前页分配 order 分配完 slab obj 之后剩余的内存, 即所产生的碎片大小. 满足条件则 break 返回. 此处 fract_leftover 越大碎片就限制得越小, 反之则放宽了碎片限制.

get_order() 根据指定 size 确定要分配多少内存页, 并换算成 2 的 order(阶):

calculate_sizes() > *calculate_order()* > *slab_order()* > *get_order()*

```
10 /**
11  * get_order - Determine the allocation order of a memory size
12  * @size: The size for which to get the order
13  *
14  * Determine the allocation order of a particular sized block of memory.  This
15  * is on a logarithmic scale, where:
16  *
17  *      0 -> 2^0 * PAGE_SIZE and below
18  *      1 -> 2^1 * PAGE_SIZE to 2^0 * PAGE_SIZE + 1
19  *      2 -> 2^2 * PAGE_SIZE to 2^1 * PAGE_SIZE + 1
20  *      3 -> 2^3 * PAGE_SIZE to 2^2 * PAGE_SIZE + 1
21  *      4 -> 2^4 * PAGE_SIZE to 2^3 * PAGE_SIZE + 1
22  *      ...
23  *
24  * The order returned is used to find the smallest allocation granule required
25  * to hold an object of the specified size.
26  *
27  * The result is undefined if the size is 0.
28  */
```

```
29 static inline __attribute_const__ int get_order(unsigned long size)
30 {
31         if (__builtin_constant_p(size)) {
32                 if (!size)
33                         return BITS_PER_LONG - PAGE_SHIFT;
34
35                 if (size < (1UL << PAGE_SHIFT))
36                         return 0;
37
38                 return ilog2((size) - 1) - PAGE_SHIFT + 1;
39         }
40
41         size--;
42         size >>= PAGE_SHIFT;
43 #if BITS_PER_LONG == 32
44         return fls(size);
45 #else
46         return fls64(size);
47 #endif
48 }
```

- 31-39 如果 size 是常量才会进入这个代码块。***__builtin_constant_p*** 是编译器内置函数，用于判断一个值在编译时是否是常量. 如果是，函数返回 1 ，否则返回 0。仅编译阶段生效，运行时无任何逻辑，不会生成实际代码。

  在前面的调用 $slab_order() -> get_order$ 看, 我们的代码流不进入这个代码块.

- 41-42 size 右移 PAGE_SHIFT(即,size 除以 2^{PAGE_SHIFT}), 这里 2^{PAGE_SHIFT} 是一页可以容纳的字节数。因此 $\frac{size}{2^{PAGE_SHIFT}}$ = 页数目。

  需要注意的是,size 在右移之前首先自减 1 得到从 0 开始计数的最大字节数. 譬如 size 为 4 byte, 假设一个页可以容纳 4 byte. 如果 size 不自减 1, $4/4 = 1$ 阶, 对应 $2^1 = 2$(页)。就会多一页, 而实际应该是 $2^0 = 1$ 页. 所以 size 必须自减 1。也可以理解为 size 按 PAGE_SIZE 向下对齐。

  如果 size 已经是 PAGE_SIZE 的整数倍，减 1 会让它变成非整数倍，从而在后续计算中向下对齐 -1，而不是向上对齐 $+1$。如果 size 不是 PAGE_SIZE 的整数倍，减 1 不影响计算结果。

- 43-47 将上面的计算结果 (页数目) 换算成阶，并返回给上一层调用。

  这里区分了 32bit 和 64bit 的架构下因为 long 型的 size 不同而导致 fls() 的不同实现。如前面所提到的,fls() 函数返回 0 有特殊意义。因此正常返回值从 1 开始索引。即返回 1 时, 表示 order 0。

  这也说明了前面 slab_order() 的 3403 行为什么要减 1。因为 -1 后得到的才是真实 order。

  fls(n) 用来快速获取数值中最高不为 0 的位的索引 (1 为初始索引)，等价于求 n 的以 2 为底的对数 (也就是页阶), 如下:

  $$fls(n) = \log_2(n) + 1$$

  $n > 0$ 时,fls(n) 和内核的 $ilog2(n) + 1$ 函数等价。

1.4 专用缓存的创建

从前面的讨论可以知道: 内核业务代码是通过 kmem_cache_create 动态创建专用缓存。譬如用于 inode、vm_area_struct、task_struct 等这些内核常用结构体。换言之，专用缓存拥有自定义字符串名称

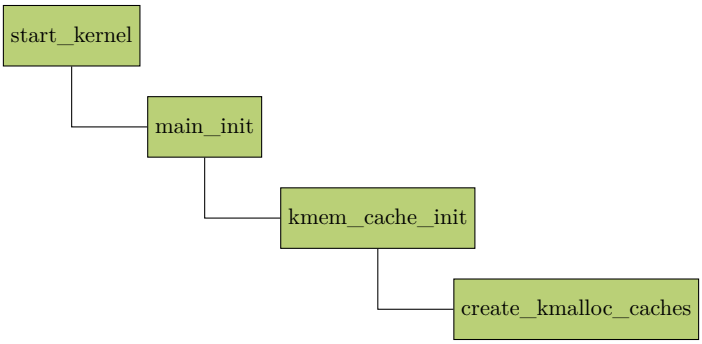
(inode_cache、vm_area_struct)。可手动调用 kmem_cache_destroy() 销毁整套缓存 (slub cache)。缓存描述符 (kmem_cache) 自身从系统最底层缓存 cache_cache 数组 (存放所有 kmem_cache 实例的全局专用 slub) 分配。

专用缓存保留有 ctor 构造 (constructor) 函数指针，新建 obj 时自动执行结构体初始化，内核很多描述符（mm_struct、file）依靠回调完成字段清零、链表初始化。

1.5 通用缓存的创建

kmalloc 通用缓存的 ctor 被强制置空。kmalloc 仅分配裸内存，不会做任何 obj 初始化。如需清零只能手动 memset。

kmalloc 通用缓存在内核启动阶段由 create_kmalloc_caches() 函数静态批量生成 kmalloc-32/64/128 …。命名格式统一为:*kmalloc-xxx*、*dma-kmalloc-xxx*，属于内核常驻基础缓存，系统运行全程不会被销毁，不存在对外释放接口。



create_kmalloc_caches()

```
767 void __init create_kmalloc_caches(slab_flags_t flags)
768 {
769     int i;
770     enum kmalloc_cache_type type;
771
772     for (type = KMALLOC_NORMAL; type <= KMALLOC_RECLAIM; type++) {
773         for (i = KMALLOC_SHIFT_LOW; i <= KMALLOC_SHIFT_HIGH; i++) {
774             if (!kmalloc_caches[type][i])
775                 new_kmalloc_cache(i, type, flags);
776
777             /*
778              * Caches that are not of the two-to-the-power-of size.
779              * These have to be created immediately after the
780              * earlier power of two caches
781              */
782             if (KMALLOC_MIN_SIZE <= 32 && i == 6 &&
783                     !kmalloc_caches[type][1])
784                 new_kmalloc_cache(1, type, flags);
785             if (KMALLOC_MIN_SIZE <= 64 && i == 7 &&
786                     !kmalloc_caches[type][2])
787                 new_kmalloc_cache(2, type, flags);
788         }
789     }
790
791     /* Kmalloc array is now usable */
792     slab_state = UP;
793
794 #ifdef CONFIG_ZONE_DMA
795     for (i = 0; i <= KMALLOC_SHIFT_HIGH; i++) {
```

```
796         struct kmem_cache *s = kmalloc_caches[KMALLOC_NORMAL][i];
797
798         if (s) {
799             kmalloc_caches[KMALLOC_DMA][i] = create_kmalloc_cache(
800                 kmalloc_info[i].name[KMALLOC_DMA],
801                 kmalloc_info[i].size,
802                 SLAB_CACHE_DMA | flags, 0,
803                 kmalloc_info[i].size);
804         }
805     }
806 #endif
807 }
```

- 767 ___init 标记该函数仅内核初始化阶段执行，启动完成后这段内存会被回收，节省运行内存。
- 772 循环遍历普通、可回收两大类通用缓存，DMA 缓存不在这里创建，走 794-806 分支单独创建。

  kmalloc 缓存类型分三类:

  - ***KMALLOC_NORMAL*** 常规 kmalloc 缓存（普通内存），内存回收线程（kswapd/slab shrinker）不会主动回收该缓存里空闲 slub。
  - ***KMALLOC_RECLAIM*** 可回收 kmalloc 缓存（内存不足时 slub 可回收归还伙伴系统）。
  - ***KMALLOC_DMA*** DMA 专用缓存 (打开配置项 CONFIG_ZONE_DMA 后在下方分支单独创建)
- 773 内层循环, 从低到高遍历所有 2 的幂次档位。
  - ***KMALLOC_SHIFT_LOW*** 最小分配阶。

    配置项 *CONFIG_SLUB* 打开时,*KMALLOC_SHIFT_LOW* 默认值为***3***。
  - ***KMALLOC_SHIFT_HIGH*** 最大分配阶。

    配置项 *CONFIG_SLUB* 打开时, 在 arm 平台上 *KMALLOC_SHIFT_HIGH* 默认值为***13***。

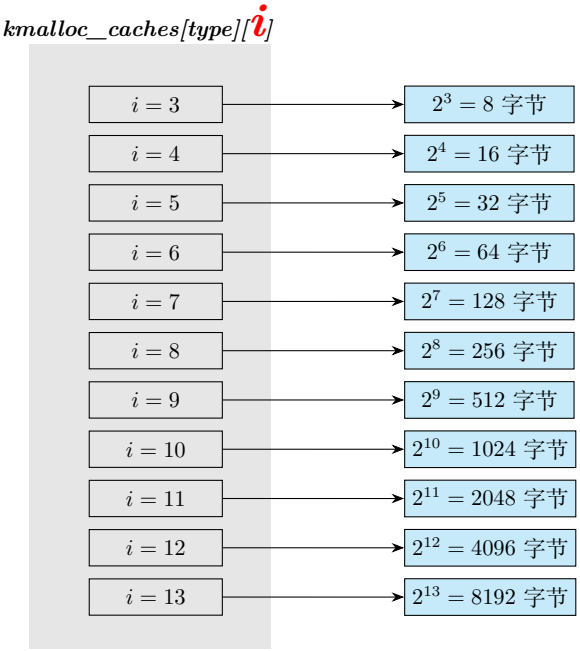
  每轮对应创建的大小是 $1 \ll i, i \in [3,13]$(即 2^i 字节,$i \in [3,13]$)。所以 kmalloc 支持字节的范围 $\in [2^3, 2^{13}]$。
- 774-775 kmalloc_caches[type][i] 是二维全局数组，保存每种类型、每个尺寸对应的 struct kmem_cache * slub 缓存描述符指针。后续 kmalloc 分配时直接读取该数组对应位置指针找到 slub 缓存描述符指针 struct kmem_cache *。如果当前类型 + 大小的缓存未创建 (指针为空), 新建对应 slab 缓存，存入全局数组。(部分标准 kmalloc 缓存可能已提前创建完毕，因为创建 slab 缓存的内存分配流程本身就依赖这些缓存。)

  从前面可知，***KMALLOC_SHIFT_LOW***默认是 3。所以显而易见，$i = 0\ 1\ 2$ 时不会进入这两行执行。也意味着 kmalloc_caches [type][0]—kmalloc_caches [type][2] 即使没有创建也不会在此处创建。
- 782-784, 785-787 ***KMALLOC_MIN_SIZE*** 是 kmalloc 分配接口支持的最小内存尺寸。默认值是 $1 \ll KMALLOC_SHIFT_LOW = 1 \ll 3 = 2^3 = 8$(字节)。

  若申请的 $size < KMALLOC_MIN_SIZE$(即 3 字节)，内核不会创建更小的 obj，统一从最小规格缓存分配。譬如，如果最小默认 8 字节，那么 kmalloc(4, GFP_KERNEL)(GFP 的意思是:get free page) 申请 4 字节时返回 kmalloc-8 的 obj，实际拿到 8 字节。

  kmalloc_caches 索引有两套规则:

- 首先，$KMALLOC_SHIFT_LOW \leq i \leq KMALLOC_SHIFT_HIGH$ 时，也就是 $3 \leq i \leq 13$，对应的 obj 是标准 2 次幂 (2^i 字节) 大小的缓存。那么据此建立的 kmalloc 基础内存池数组索引和缓存大小对应关系如下图:



- 下标索引 $i = 1, 2$ 时，代表特殊的非标准缓存。换言之，下标索引 $= 1, 2$ 时是独立规则，obj 大小和下标索引移位的计算无关。

  内核硬编码规定: $i = 1$ 时，***kmalloc_caches[type][1]*** 对应 96 字节的缓存池 (kmalloc-96)。承接申请 65 ~ 96 字节时的分配，避免造成申请的字节 $\in [65, 96]$ 时 ($[65, 96] \in (64, 128]$，即 $(64, 96] \in (2^6, 2^7]$) 直接返回 $2^7 = 128$ 字节给申请者，从而造成浪费和内存碎片。

  同理，$i = 2$ 时，***kmalloc_caches[type][2]*** 对应 192 字节的缓存池 (kmalloc-192)。承接 129 ~ 192 字节 ($\in (128, 256]$, 即 $\in (2^7, 2^8]$) 的分配。避免申请字节 $\in (129, 192]$ 时返回 $256(2^8)$ 字节给申请者。

  如图黄色高亮部分表示 $i = 1, 2$ 时，对应的 obj 的大小。



综上，*kmalloc_caches[type][i]* 的索引从 *KMALLOC_SHIFT_LOW*(通常是 3) 开始。索引 *1* 和 *2* 的成员描述符指针指向特殊大小的缓存 *96* 字节和 *192* 字节。而索引 *0* 不会用到。

之所以在 $i = 6$ 时，创建 kmalloc—96;$i = 7$ 时，创建 kmalloc—192。并没有特别的深意，仅在于 $2^6, 2^7$ 分别是 96 和 192 的前置 size。

条件 $KMALLOC_MIN_SIZE \leq 32$。如果把 32 字节作为 base 的分配单元，那么 kmalloc—96 肯定是它的整数倍。如果 $32 < KMALLOC_MIN_SIZE \leq 64$，那么把 64 字节作为最小 base 的分配单元，kmalloc—96 就不是最小 base 的整数倍，因此就跳过创建 kmalloc—96，但是此时 kmalloc—128 是最小分配单元的整数倍，因此创建 kmalloc—128。这样既减少了内存的浪费，又保证了是最小分配 base 的整数倍。这个思想类似于用页 (伙伴系统)和slub机制分别减少外部碎片和内部碎片。这里的 base 分配单元可以看成是"页"，而 KMALLOC_MIN_SIZE 则可以看做slub。

- 789 外层循环结束：NORMAL、RECLAIM 两套普通内存池全部初始化完毕。

  此时可以从/proc/slabinfo 节点看到如下信息:

```
kmalloc-rcl-8k         0      0   8192    4    8 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-4k         0      0   4096    8    8 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-2k         0      0   2048   16    8 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-1k         0      0   1024   32    8 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-512        0    160    512   32    4 : tunables    0    0    0 : slabdata      5      5      0
kmalloc-rcl-256       13     64    256   32    2 : tunables    0    0    0 : slabdata      2      2      0
kmalloc-rcl-192       71    168    192   42    2 : tunables    0    0    0 : slabdata      4      4      0
kmalloc-rcl-128     1337   1376    128   32    1 : tunables    0    0    0 : slabdata     43     43      0
kmalloc-rcl-96      2394   2394     96   42    1 : tunables    0    0    0 : slabdata     57     57      0
kmalloc-rcl-64      6016   6016     64   64    1 : tunables    0    0    0 : slabdata     94     94      0
kmalloc-rcl-32         0      0     32  128    1 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-16         0      0     16  256    1 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-rcl-8          0      0      8  512    1 : tunables    0    0    0 : slabdata      0      0      0
kmalloc-8k           350    356   8192    4    8 : tunables    0    0    0 : slabdata     89     89      0
kmalloc-4k          3139   3152   4096    8    8 : tunables    0    0    0 : slabdata    394    394      0
kmalloc-2k          2112   2112   2048   16    8 : tunables    0    0    0 : slabdata    132    132      0
kmalloc-1k          2980   3008   1024   32    8 : tunables    0    0    0 : slabdata     94     94      0
kmalloc-512        16970  17376    512   32    4 : tunables    0    0    0 : slabdata    543    543      0
kmalloc-256        10039  11136    256   32    2 : tunables    0    0    0 : slabdata    348    348      0
kmalloc-192         6216   6216    192   42    2 : tunables    0    0    0 : slabdata    148    148      0
kmalloc-128         3230   3552    128   32    1 : tunables    0    0    0 : slabdata    111    111      0
kmalloc-96          4956   4956     96   42    1 : tunables    0    0    0 : slabdata    118    118      0
kmalloc-64         21763  23680     64   64    1 : tunables    0    0    0 : slabdata    370    370      0
kmalloc-32         40843  41600     32  128    1 : tunables    0    0    0 : slabdata    325    325      0
kmalloc-16         47104  47104     16  256    1 : tunables    0    0    0 : slabdata    184    184      0
kmalloc-8          14009  14848      8  512    1 : tunables    0    0    0 : slabdata     29     29      0
```

图 1–3: *cat /proc/slabinfo* 显示的信息

- 792 普通 kmalloc 缓存数组就绪，内存分配功能现已正常可用。
- 794-806 在内核配置项 ***CONFIG_ZONE_DMA*** 开启后生效。建立 DMA 类型的 kmalloc 全局数组。
  - 795-796 遍历所有普通缓存下标，如果该大小缓存存在，才创建 DMA 副本。。
  - 799-803 创建 DMA 专用缓存存入 DMA 缓存数组。复用普通缓存的尺寸、对齐、标志，仅内存域改为 DMA。

create_kmalloc_caches() > *new_kmalloc_cache()*

```
750 static void __init
751 new_kmalloc_cache(int idx, enum kmalloc_cache_type type, slab_flags_t flags)
752 {
753     if (type == KMALLOC_RECLAIM)
754         flags |= SLAB_RECLAIM_ACCOUNT;
755 
756     kmalloc_caches[type][idx] = create_kmalloc_cache(
757                     kmalloc_info[idx].name[type],
758                     kmalloc_info[idx].size, flags, 0,
759                     kmalloc_info[idx].size);
760 }
```

- 753-754 判断当前要创建的是可回收类型缓存 (***KMALLOC_RECLAIM***)。若是，则将该 slub 内存纳入可回收内存的统计。内存紧张时 kswapd 可以回收该缓存内空闲 obj。如果是普通缓存 *KMALLOC_NORMAL*，不追加 *SLAB_RECLAIM_ACCOUNT* 标记，内存不可主动回收。
- 756-759 创建 kmalloc 的 slub 缓存描述符。create_kmalloc_cache 底层调用 kmem_cache_create 生成 slub 缓存描述符，返回缓存描述符指针存入 kmalloc_caches 数组。

  从入参 ctor 构造函数指针为 0 可以看出 kmalloc 通用缓存无自定义构造器，填 NULL。

create_kmalloc_caches() > *new_kmalloc_cache()* > *create_kmalloc_cache()*

```
571 struct kmem_cache *__init create_kmalloc_cache(const char *name,
572         unsigned int size, slab_flags_t flags,
573         unsigned int useroffset, unsigned int usersize)
574 {
575     struct kmem_cache *s = kmem_cache_zalloc(kmem_cache, GFP_NOWAIT);
576 
577     if (!s)
578         panic("Out of memory when creating slab %s\n", name);
579 
580     create_boot_cache(s, name, size, flags, useroffset, usersize);
581     list_add(&s->list, &slab_caches);
582     s->refcount = 1;
583     return s;
584 }
```

- 575-578 分配一块内存存放 slub 缓存管理结构体 struct kmem_cache，并且整块内存清零。

  入参 GFP_NOWAIT 标注分配管理结构体内存时不阻塞休眠。此时是内核极早期初始化流程，调度器/内存回收还没就绪，不能等待回收内存，如果失败直接 panic。
- 580 初始化 slub 描述符 struct kmem_cache 内部所有字段。这是启动阶段专用初始化逻辑，区别于运行时动态创建缓存，所有 kmalloc 内置缓存都走这套启动初始化流程。

- 581 把新建的缓存描述符插入全局 slub cache 链表 ***slab_caches***。

  slab_caches 是一个双向链表头，不管是 kmalloc 通用缓存还是 kmem_cache_create() 创建的专用缓存，都会通过自己的 slab 缓存描述符 (kmem_cache->list) 把自身挂入这个链表。如果要遍历系统中的 slab 缓存，可以以 *slab_caches* 为起点。

- 582 设置缓存引用计数为 1。kmalloc 内置缓存永久有效，初始化时置 1，全程不会释放。而动态创建的私有缓存引用计数归 0 时会销毁 slub，kmalloc 系统缓存生命周期随内核全程存在。

2

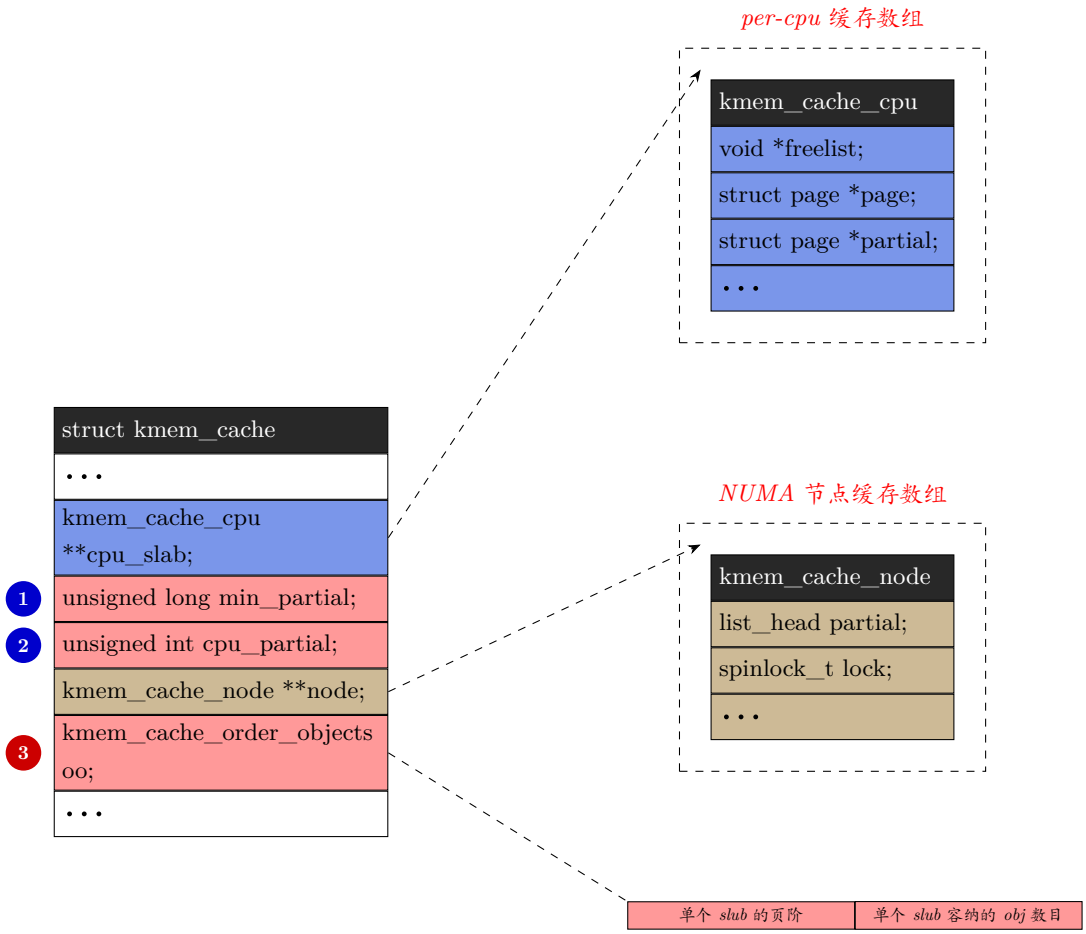
slub object 的分配

obj 的分配总是从 active slab(kmem_cache.cpu_slabs.page) 中进行的。

slub 或 slub 创建时会包括固定数量的 slub obj，如有需要，后续可以扩展。更确切的说就是，单个 slub 创建后内部包含固定数量不变的 obj。但是，slab 本身的数量并没有限定，运行时可以不断新增 slub 实现动态扩容。

一个 slab 可以是满的、部分满的或空的。满 slab 的所有 obj 都被分配，而空 slab 的所有 obj 都是空闲的。如有需要，可以销毁/回收空 slab，并将其页返回给伙伴系统。slab 可以作为 per-cpu 的活跃 slub，也可以位于 per-cpu 的 partial slub 列表中，或者位于 per-node 的 partial slub 列表中。在任何 partial 列表中的 slab 要么是部分空，要么完全空。满 slab 不会存在于上述提到的任何列表中。除了 slub 调试外，无需维护满 slub 列表，因为当从满 slab 释放一个 obj 时，我们可以通过 obj 地址获取 slub 地址，并将这个现在成为 partial slub 的 slub 放入适当的 slub 列表中。

一个 slub 可以由一个或多个页组成，这并不依赖于 obj 的大小，即使其 obj 小于一页，slub 仍可能由多个页组成。组成 slub 的连续物理页的页阶和单个 slub 能容纳的目标 obj 的总数 (不同的 obj 对应的总数不一定相同)，都在 ___kmem_cache_create() 阶段由 ___kmem_cache_create()->kmem_cache_open()->calculate_sizes() 依次调用，在 calculate_sizes() 中一次性计算得到。同一个 kmem_cache 下的所有 slub, 都具有相同的 order、相同的 slub obj 数。在 slub 描述符 kmem_cache 初始化阶段一次性计算完成并存入 kmem_cache.oo 后。后续分配、释放 slub 全过程都不会改变。如下图说明 3:



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目，和组成单个 slub 的连续物理页的页阶，拼接成一个 unsigned int 大小的值。

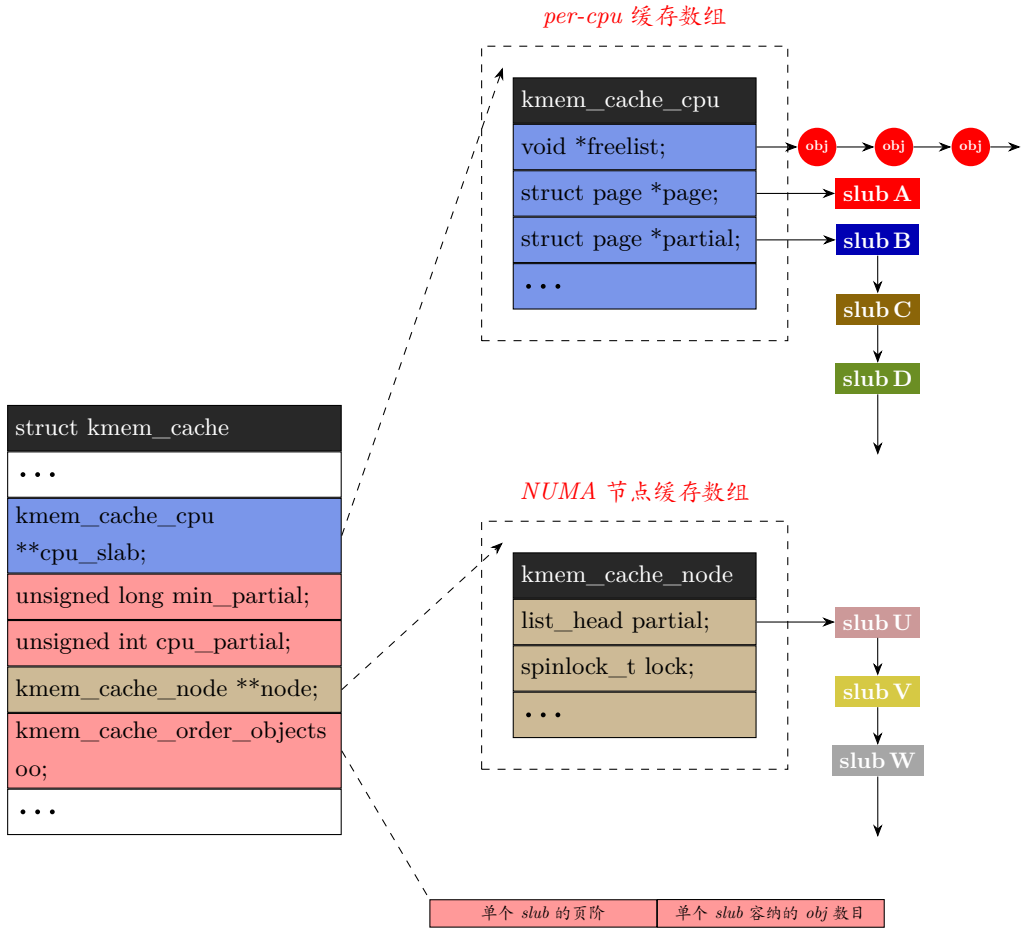
由多个连续物理页组成的 slub($order \geq 1$) 只有复合页首页的 slab_cache、freelist 成员具备有效数据。

- ***page->slab_cache*** 指向该 slab 所属的 kmem_cache。
- ***page->freelist*** 存储本 slub 内部空闲 obj 链表头。

复合页中所有**尾部**子页帧的 page 结构体的 slab_cache、freelist 字段不会被 slub 组件维护。不能依靠这两个字段反向查找 slub 缓存，也无法籍此获取 slub 空闲 obj 链表。需要注意的是，这里的**尾部**是指复合页中除了头部首页外的其余页，而**不是末页**。

- ***kmem_cache_cpu->freelist*** 是本地 CPU slab 缓存的空闲 obj 的链表。
- ***kmem_cache_cpu->partial*** 是单链表，链表上每个节点都是 struct page*，每个节点的指针指向各个 partial 的复合页或单页 slub 的首页。
- ***kmem_cache_cpu->page*** 指向当前使用的复合页或单页 slub 的首页。之后为了简便，在不需要特别处理时，对复合页和单页的 slub 都无差别称之为 slub。

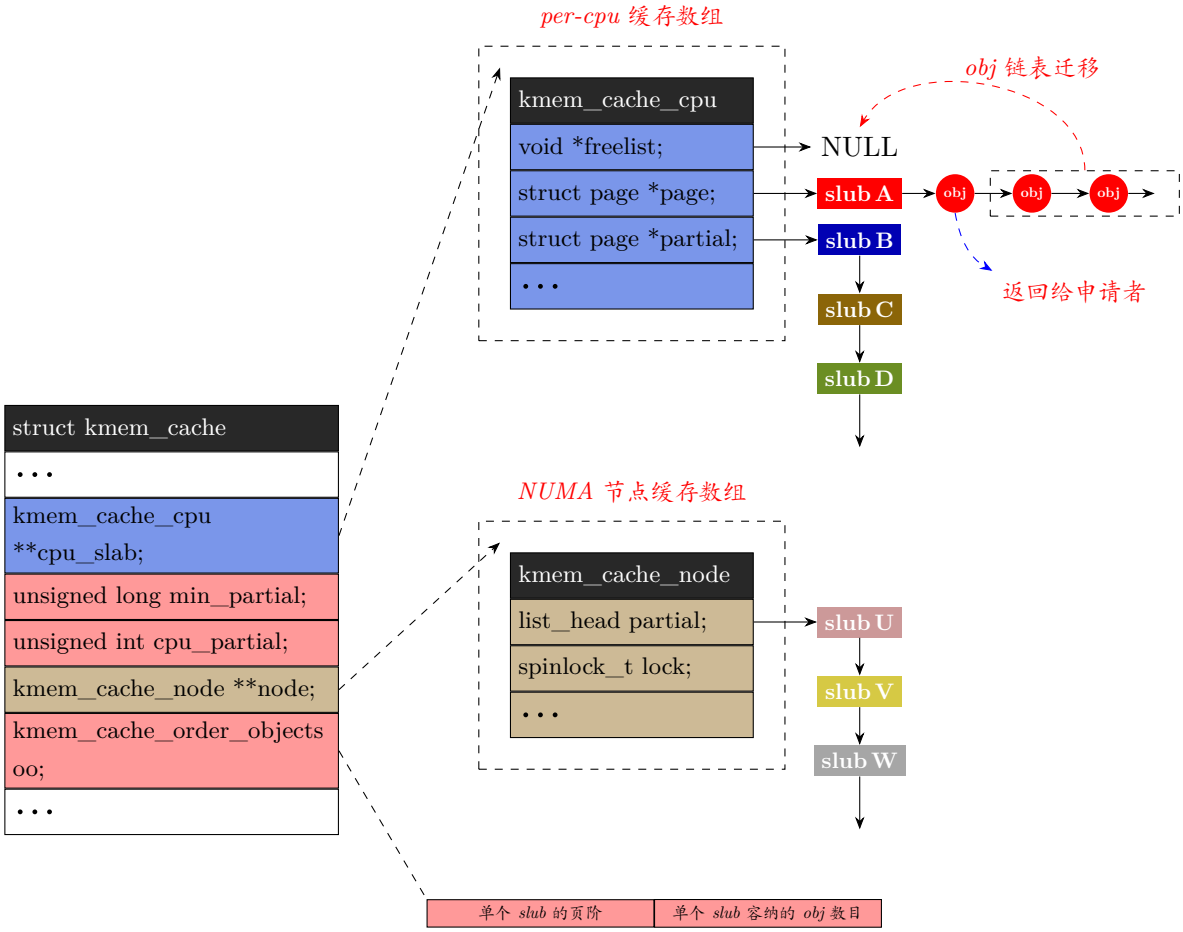
最初 kmem_cache_cpu->freelist 会被更新成 kmem_cache_cpu->page->freelist。也就是 kmem_cache_cpu->page->freelist 和 kmem_cache_cpu->freelist 在刚切换新 slub 的瞬间二者指向同一个链表头部。但是紧接着 kmem_cache_cpu->page->freelist 会置 NULL。之后只要该 slub 上的 obj 有跨 cpu core 的 free，那么 free 的 obj 会被挂到 kmem_cache_cpu->page->freelist 链表。并且只有 kmem_cache_cpu->freelist 都分配出去或耗尽时，才会从 slub 的 kmem_cache_cpu->page->freelist 查找空闲 obj 分配。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目，和组成单个 slub 的连续物理页的页阶，拼接成一个 unsigned int 大小的值。

图 2–1: cpu 本地缓存链表 freelist 非空时的状态

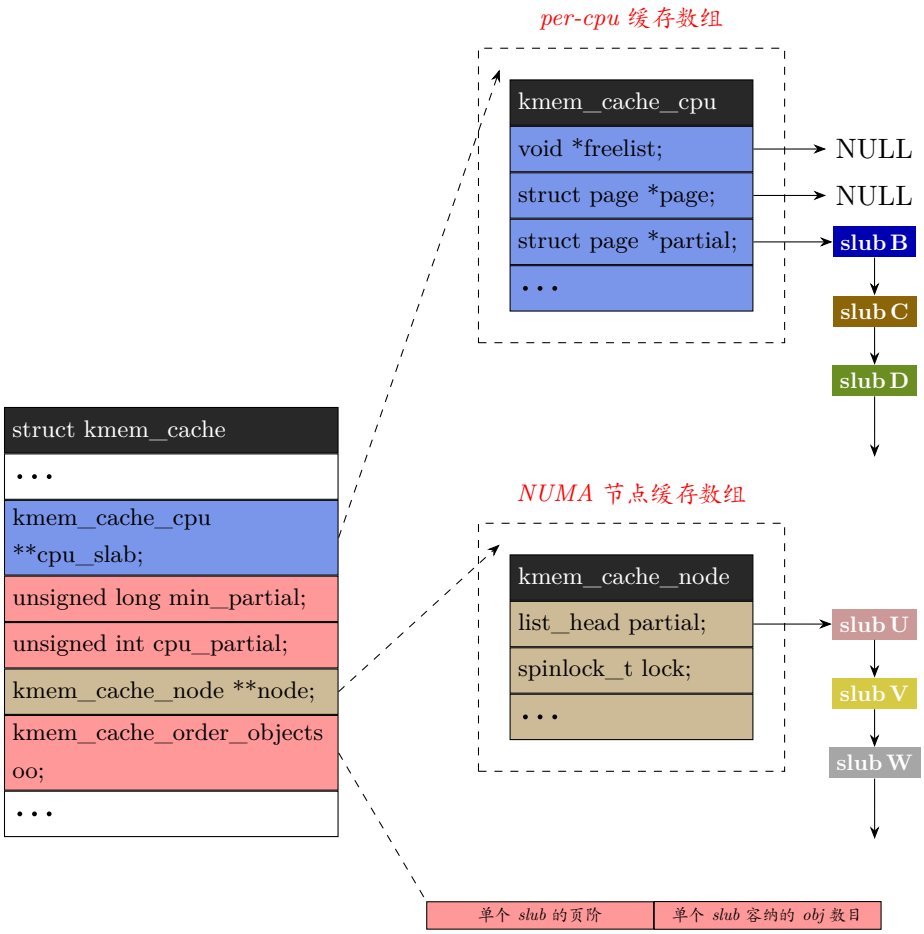
上图中，如果 kmem_cache_cpu->freelist 上的 obj 耗尽，那么系统会查找本地 cpu 优先用于分配的 slub 自身的 freelist(kmem_cache_cpu->page->freelist) 上是否有空闲 obj，如有则取出首个 obj 返回，并将剩余 freelist 更新到 kmem_cache_cpu->freelist，同时 kmem_cache_cpu->page->freelist 置 NULL。如下图。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–2: cpu 本地缓存的 freelist 耗尽，缓存的 slub 自身的 freelist 链表非空时的状态

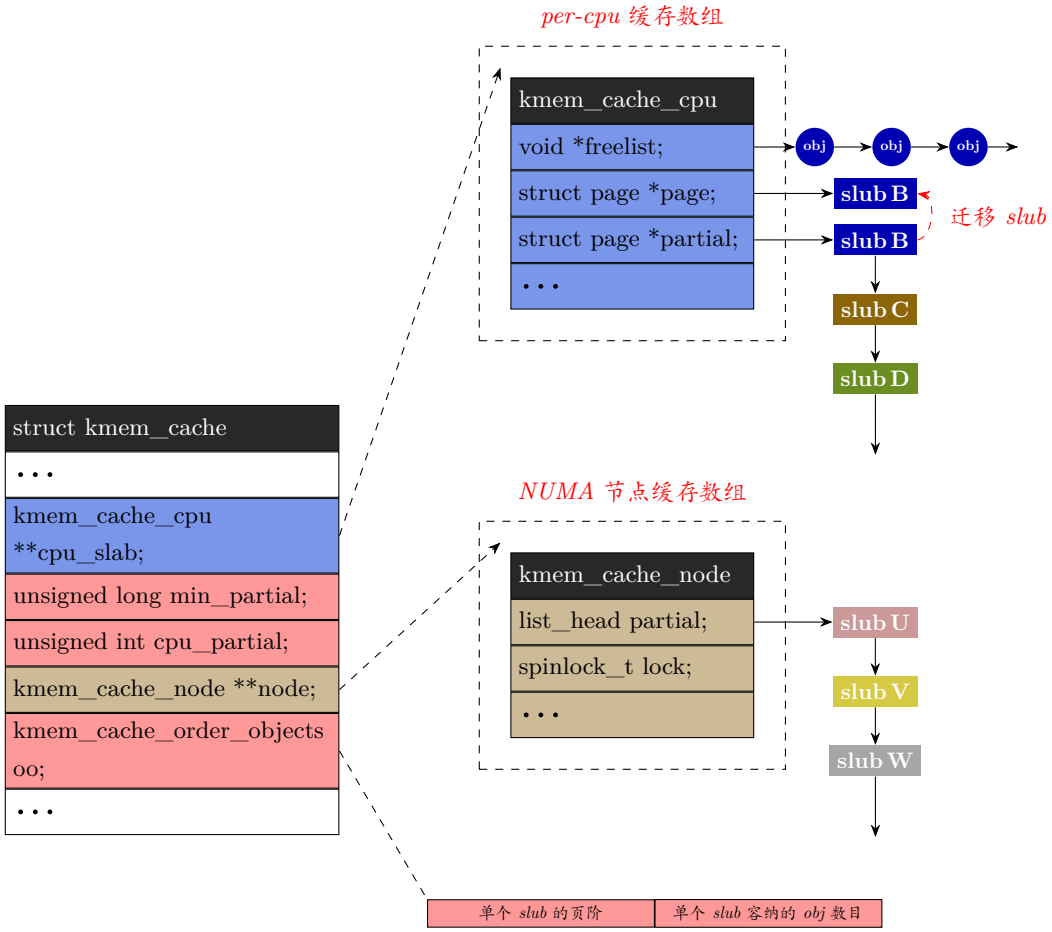
如果本地缓存的 slub 自身的 freelist 为 NULL，那么意味着本地缓存的 slub 耗尽。因此会将 kmem_cache_cpu->page 置为 NULL—不再管理该->page 指向的 slub—不再持有指向当前 slub 首页的指针 (如下图)。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–3: cpu 本地缓存链表 freelist 耗尽时的状态

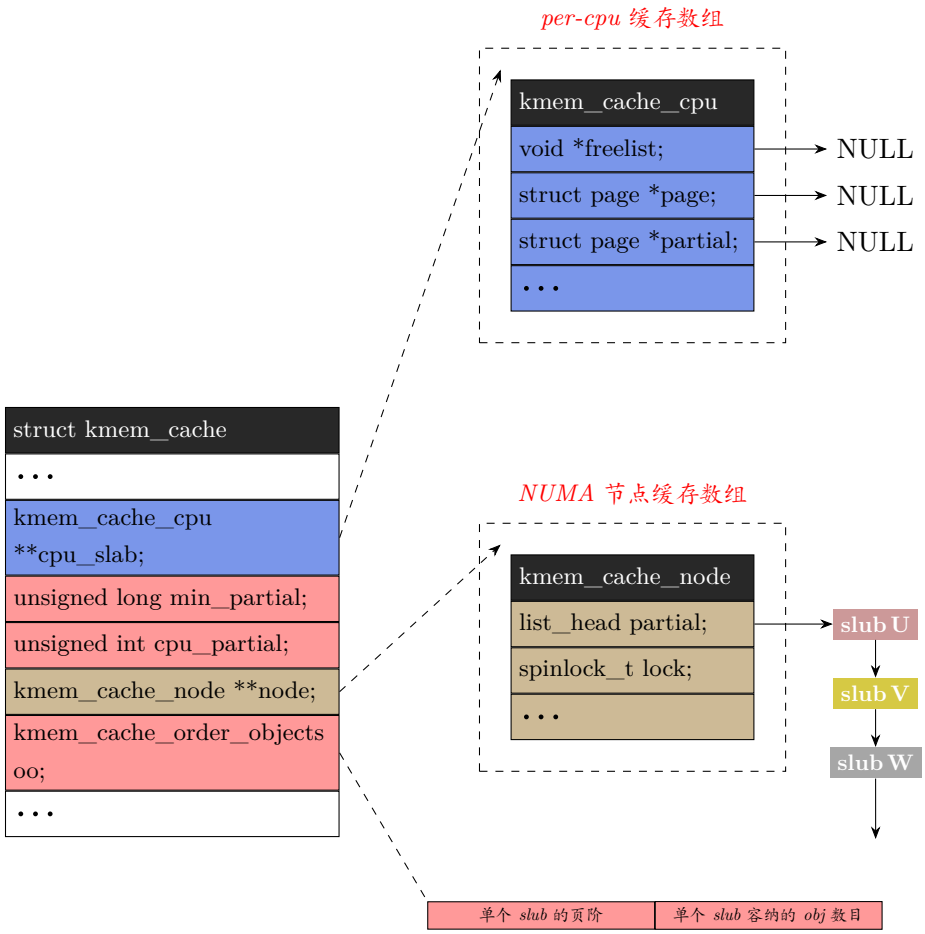
那下一步，系统会去 kmem_cache_cpu->partial 链表头部去取 partial 空闲的 slub。将该 slub 自身 (首页) 的 page->freelist 整条迁移给 kmem_cache_cpu->freelist, 置空自身的 page->freelist, 。同时把这个 slub 的首页赋值给 kmem_cache_cpu->page，使之成为本地 cpu 上新激活的 slub。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–4: cpu 本地缓存链表 freelist 耗尽，从 partial 链表迁移 partial slub 到本地 cpu

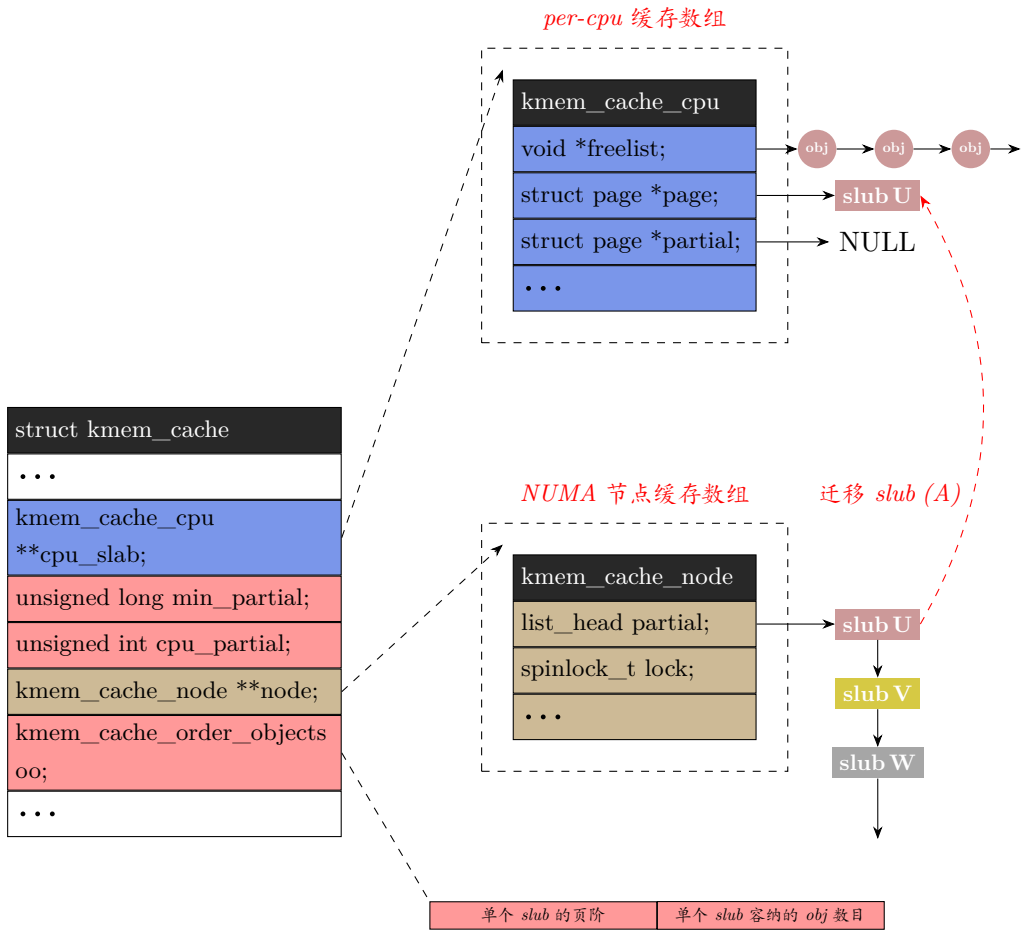
如果本地 freelist 耗尽,kmem_cache_cpu->partial 链表也为 NULL。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–5: cpu 本地缓存链表 freelist 耗尽，本地 partial 链表也无 partial slub

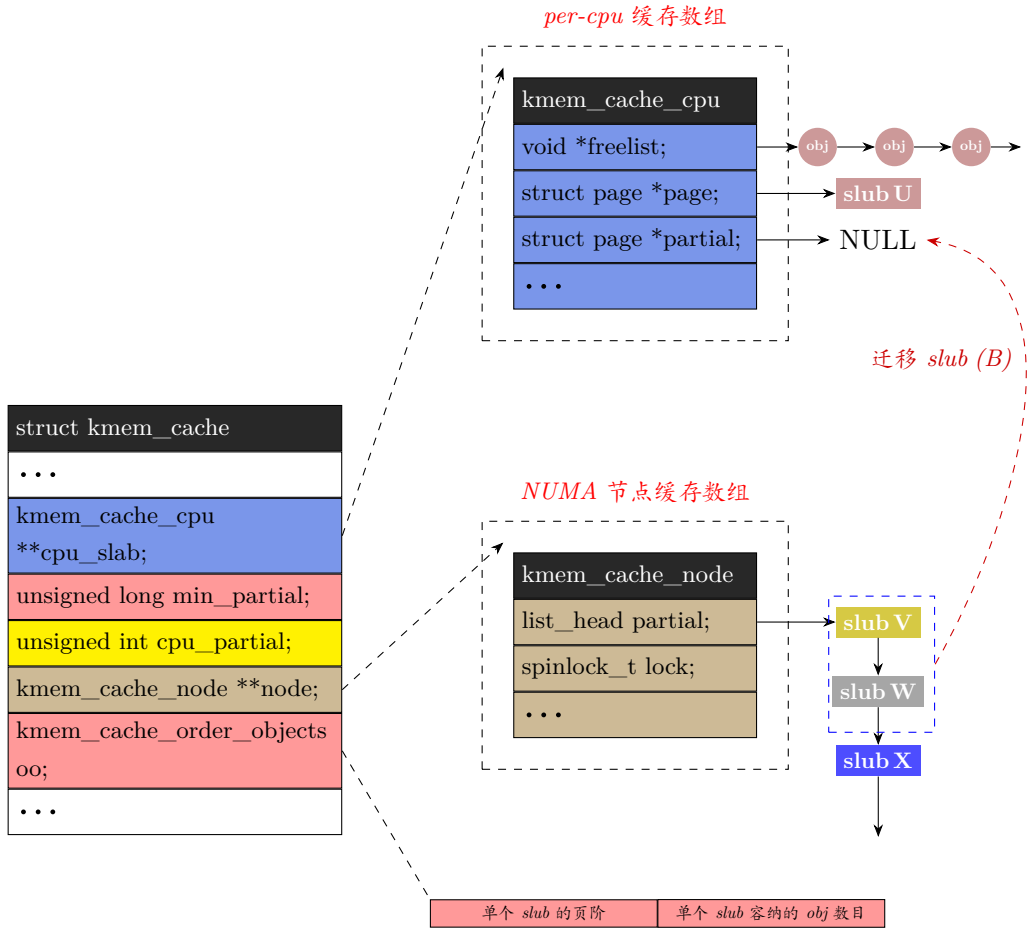
那么从本 NUMA 节点 kmem_cache_node->partial 链表头部摘下第一个有空闲 obj 的 slub，直接给 kmem_cache_cpu->page(指向该 slub 首页)。同时该 slub 的 freelist 直接迁移给 kmem_cache_cpu->freelist。如果本地 cpu 对应的 node 节点上也无空闲 slub, 那么就跨节点分配，从 kmem_cache_node 中其他 NUMA node 节点去查找和分配内存。(下图只是示意，没有区分本地 node 和其他 node)



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–6: cpu 本地缓存链表 freelist 耗尽，从全局 partial 链表迁移 partial slub 到本地 cpu (***A***)

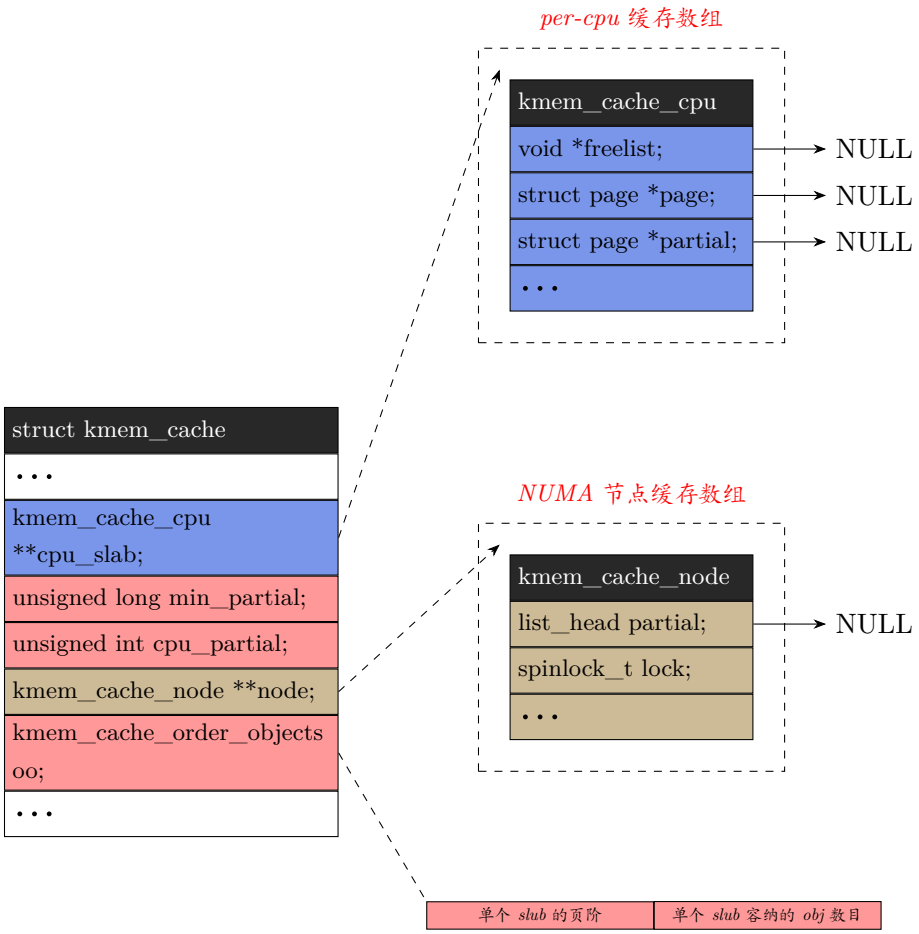
在上面图中步骤执行完，也就是将 NUMA 节点的 partial 链表的第一个 slub 摘取激活成本地 cpu 优先分配的 slub 后，内核会继续轮询该 NUMA 节点的 partial 链表。将 partial 链表上剩余的 slub 继续摘取出来，放到本地 cpu 的 partial 链表 (kmem_cache_cpu->partial)。继续摘取的 slub 的空闲 obj 总数目若超过 $\frac{kmem_cache->cpu_partial}{2}$，则停止从 kmem_cache_node 的 partial 链表继续迁移 slub 至本地 cpu 的 partial 链表的动作。*kmem_cache->cpu_partial* 在下图黄色高亮处。



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目，和组成单个 slub 的连续物理页的页阶，拼接成一个 unsigned int 大小的值。

图 2–7: cpu 本地缓存链表 freelist 耗尽，从全局 partial 链表迁移 partial slub 到本地 cpu (***B***)

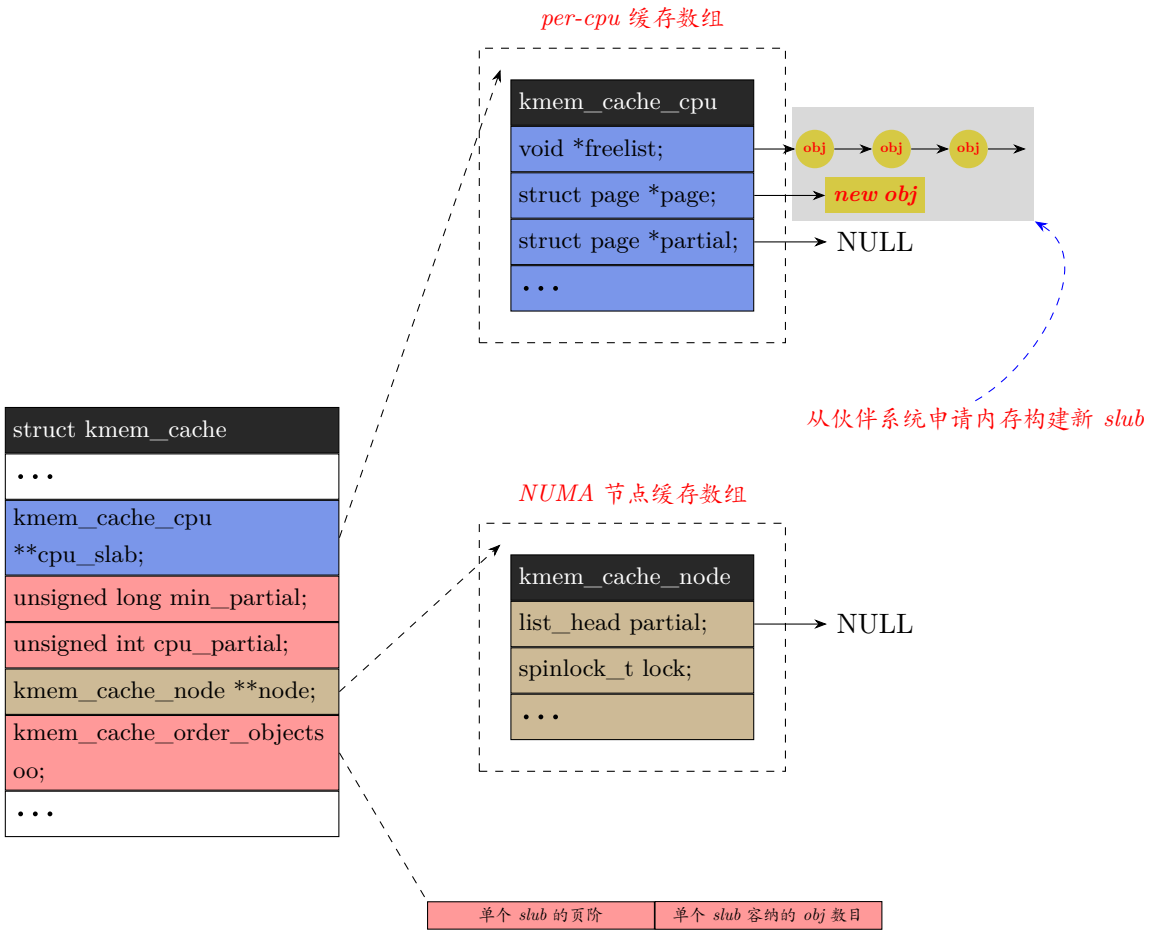
如果本地 slub 缓存池和全局 slub 缓存池中的 obj 都已耗尽，如下:



说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–8: cpu 本地缓存 slub 池和全局 slub 池的 obj 都耗尽

那么向伙伴系统申请连续物理页来构建新的 slub, 新建的 slub 被激活成本地 cpu 优先分配的 slub:

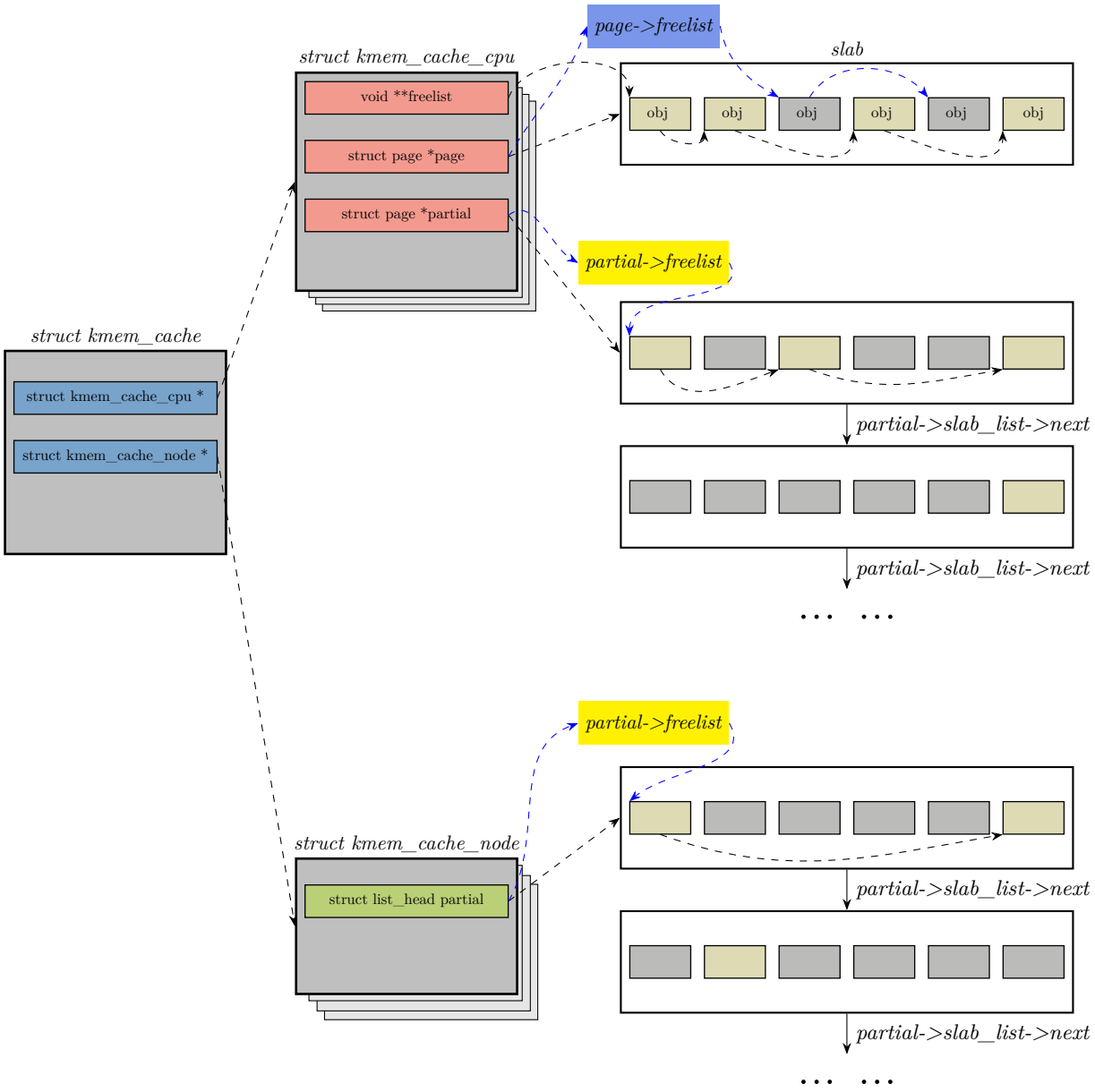


说明:
1. NUMA 每个 node 下 partial 链表最低要保留的 slab 数量。
2. 单个 CPU 本地 partial 链表最多可缓存的空闲 obj 数量的上限。
3. 单个 slub 容纳的 obj 数目, 和组成单个 slub 的连续物理页的页阶, 拼接成一个 unsigned int 大小的值。

图 2–9: cpu 本地缓存 slub 池和全局 slub 池的 obj 都耗尽，向伙伴系统申请内存

对于由复合页组成的 slub，头部页的 freelist 指向 slub 上的第一个空闲 obj，且这个空闲 obj 可以在 slub 的任何位置，即它可能在头部页上，也可能在某个尾部页上。

各结构体及成员之间的关系如下 (注意: 其中蓝色高亮的 page->freelist 串联的 obj 是跨 cpu 释放回 slub 的 obj。):



struct kmem_cache_cpu: 是对本地 slub 缓存池的描述，每一个 cpu 对应一个 kmem_cache_cpu 结构体。可以使用无锁访问，提高缓存对象分配速度。

struct kmem_cache_node: 是对共享 slub 缓存池的描述，用于管理 NUMA 每个 node 的 slab 页面。

每一个 node 对应一个 kmem_cache_node 结构体。从 node 的 partial 链表中获取新 slub 需要获取锁。若失败，系统会转而从伙伴系统获取新 slub。但是从 node 链表或伙伴系统获取新 slub 是超慢的执行路径，因为这两个操作的时间是不确定的。

slub obj 分配有三条路径：快速路径、慢速路径，以及超慢路径 (如下图)。

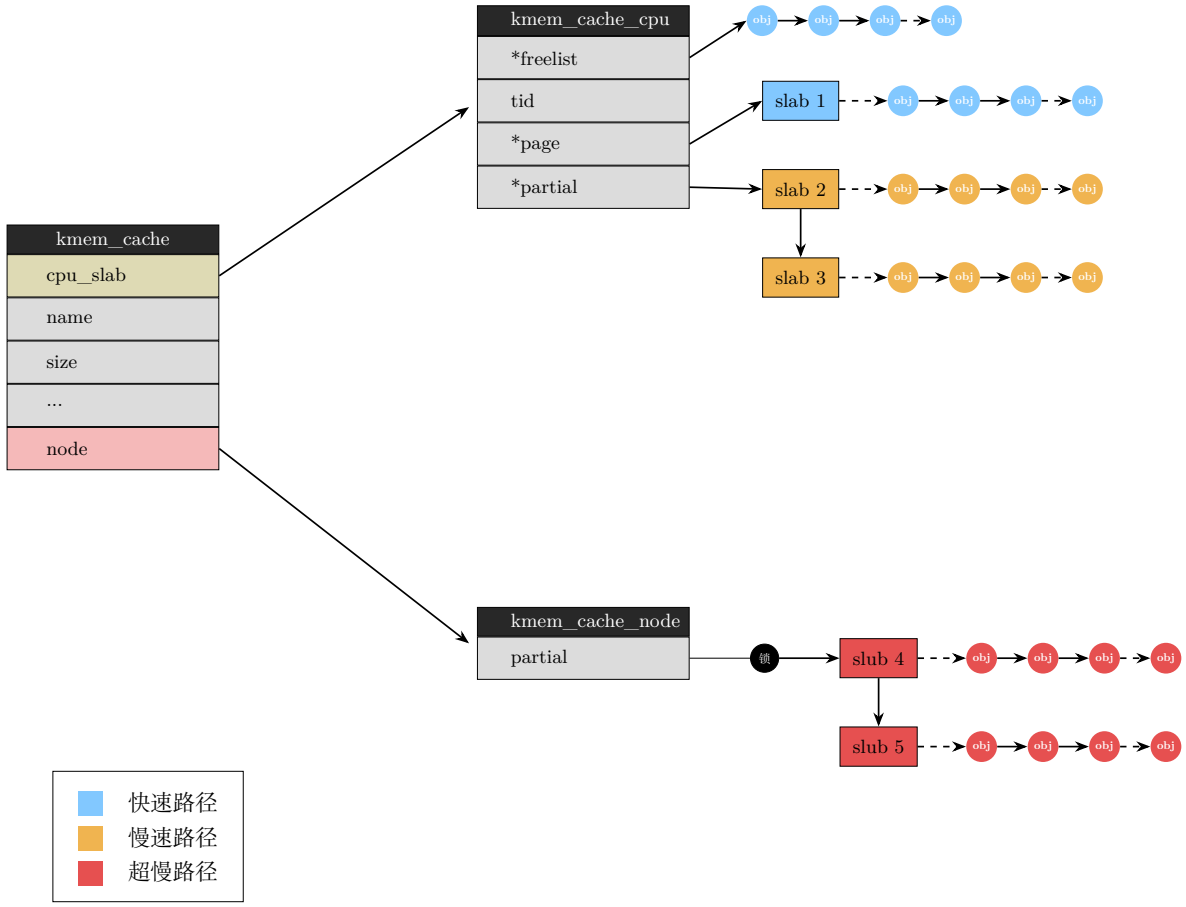


图 2–10: slub kmem_cache 缓存分层结构

当 per-cpu 的无锁 freelist（kmem_cache.cpu_slab->freelist）包含空闲对象，就会走**快速路径**分配。这是最简单而且开销最小的分配路径，不涉及任何锁定或 irq/preemption 禁用。该空闲 obj（即由 kmem_cache.cpu_slab->freelist 指向的）链表第一个 obj 会作为分配的 obj 返回，链表中下一个可用 obj 成为链表新表头被 kmem_cache.cpu_slab->freelist 指向。如果分配耗尽了无锁 freelist 中的所有 obj，那么该列表将变为 NULL，并在下次尝试分配时获得新 obj。如前所述，*如果 CPU 不支持两个 word 的 cmpxchg 操作，或者启用了 slub 调试（slub_debug），则不会使用快速分配路径。*

慢速分配和超慢分配路径的分配开销是不一样的。下面按照开销增加的顺序解析这些路径的代码。

2.1 kmem_cache_alloc_node()

分配 slub obj 的入口函数是 kmem_cache_alloc_node()，如下：

mm/slub.c :: kmem_cache_alloc_node()

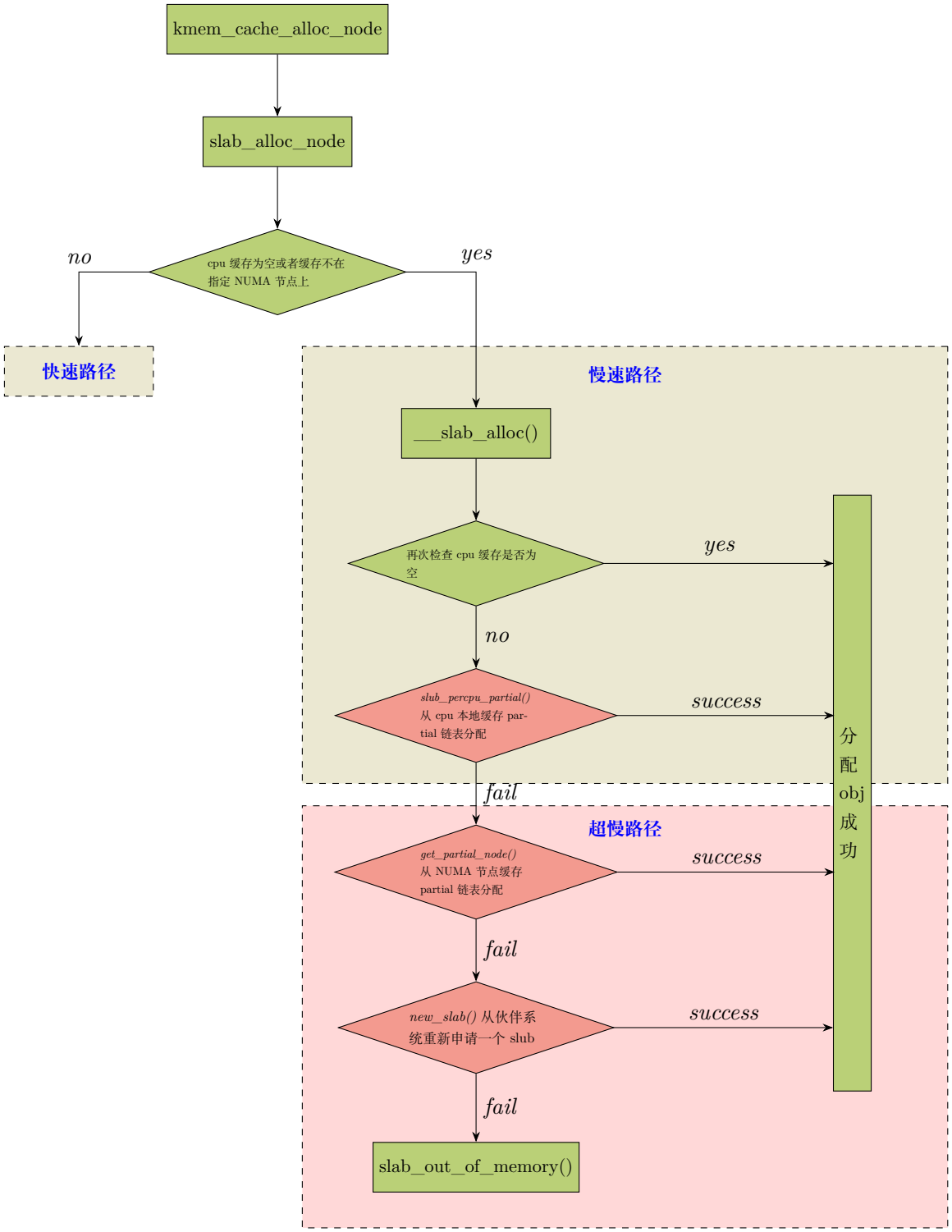
```
2925 void *kmem_cache_alloc_node(struct kmem_cache *s, gfp_t gfpflags, int node)
2926 {
2927     void *ret = slab_alloc_node(s, gfpflags, node, _RET_IP_);
2928
2929     trace_kmem_cache_alloc_node(_RET_IP_, ret,
2930                     s->object_size, s->size, gfpflags, node);
2931
2932     return ret;
2933 }
2934 EXPORT_SYMBOL(kmem_cache_alloc_node);
```

该函数会调用 slab_alloc_node() 并在其中判断：cpu 的缓存是否为空，或者缓存是否在指定 NUMA 节点上。从而走慢速或超慢速执行路径。

slowpath(包括慢速路径和超慢路径。为了简便起见，除非特别说明，下文的" 慢速路径" 一词包含这两种含义) 分为几种情况 (基本场景在前文已经讨论过，此处不吝冗词再次列出)：

- node unmatch，也就是在 NUMA 架构中使用了不匹配节点的内存，需要尝试释放 slub, 并从本节点的内存中申请新的 slub。
- kmem_cache 的本地 cpu 缓存管理结构体指针是 struct kmem_cache_cpu *cpu_slab。如果 cpu_slab->page 为空，也就意味着本地 cpu 缓存上已经没有可以优先分配的 slub 了，这种情况多发生在当前 slub 上的 obj 已经被分配完。此时 cpu_slab->freelist 也是 NULL, 实际上也意味着优先分配的 slab 已分配完，。一个需要注意的点是，当 slab_cpu->page 对应的优先分配 slab 被分配完之后，该页面就被 slab 组件直接抛弃了，也就是 slub 组件并不维护管理该 slub，实际上也不需要维护，因为在 object 的释放时，完全可以通过 obj 的虚拟地址来确定所属的 page。而在 debgug 模式下，被分配完的 slab 将会链接到 per-node 的 full 链表中用作调试。
- 如果 cpu 本地缓存的 freelist 上的 obj 已经耗尽，那么查看 cpu_slab->partial 链表上是否存在 partial slab，该 partial 同样是 percpu 的，相对于 cpu_slab->page 来说是备用分配区，该链表上的 slab 通常是 partial 状态，如果 cpu_slab->partial 上存在可分配的 slub，就将其 slab 对应的首页 page 赋值给 slab_cpu->page 来作为本地 cpu 新的可供优先分配的 slub，然后用新集合 slub 上空闲的 obj 更新 cpu_slab->freelist 列表。
- 如果 per-cpu 类型的 cpu_slub->partial 链表上也不存在 slub 节点，那就只能从 NUMA 节点的 partial 链表中找可使用的 slub，如果找到了 partial slub，就将其交由 per-cpu 的 struct kmem_cache_cpu cpu 类型的结构体指针 _slab 管理，然后从其中分配空闲的 obj。
- 最后一种情况是：供本地 cpu 优先分配的 slub 上没有空闲 object，两个备用的缓存 partial 链表 (本地 cpu 的 partial 链表 +NUMA 的 partial 链表) 上也都没有 partial slub，于是只能重新构建一个新的 slub。于是从 buddy 子系统中申请页面，申请页面的数量取决于 kmem_cache->oo 成员中的 order 值 (如前所述，这个 order 是构成单个目标 slub 的所需页阶)。然后将新申请的页面初始化为一个新的 slub，其中包括：page->objects、page->slab_cache、page->freelist、page->inuse、page->frozen 等 struct page 相关成员的初始化。之后将新的 slub 交由 cpu_slab 管理。创建了新的 slub 之后，取出 freelist 指向的第一个 obj 返回给申请者。需要注意的是，尽管这种情况下申请的页面会交由 per-cpu 的 cpu_slab 管理，但是并不是从 per-cpu 的内存区进行申请，而是从普通内存区申请。

下面是一个流程示意图:



slab_alloc_node() 函数从指定的 NUMA 节点分配单个 obj，如下：

kmem_cache_alloc_node() *slab_alloc_node()*

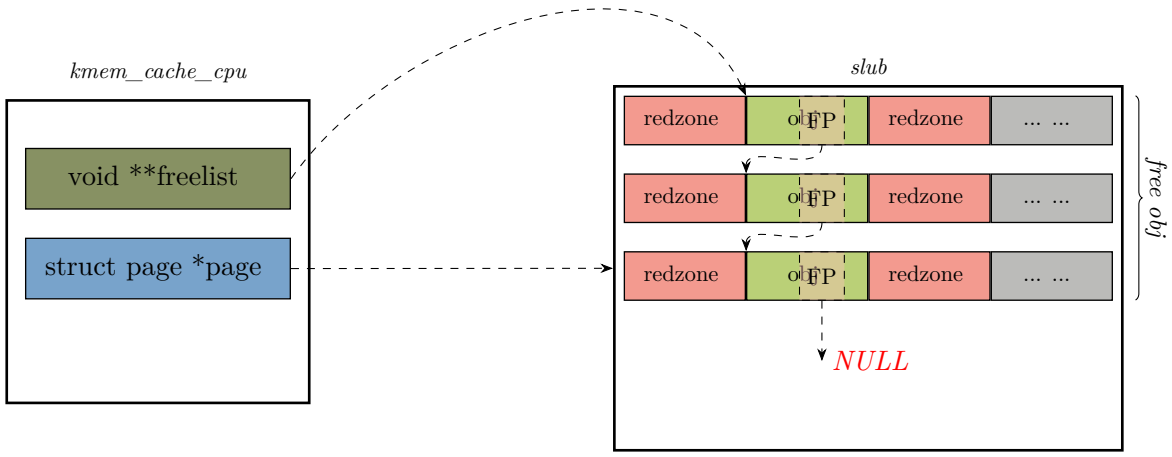
```
2807 static __always_inline void *slab_alloc_node(struct kmem_cache *s,
2808                 gfp_t gfpflags, int node, unsigned long addr)
2809 {
2810         void *object;
2811         struct kmem_cache_cpu *c;
2812         struct page *page;
2813         unsigned long tid;
```

```
2814         struct obj_cgroup *objcg = NULL;
2815
2816         s = slab_pre_alloc_hook(s, &objcg, 1, gfpflags);
2817         if (!s)
2818                 return NULL;
2819 redo:
2820         /*
2821          * Must read kmem_cache cpu data via this cpu ptr. Preemption is
2822          * enabled. We may switch back and forth between cpus while
2823          * reading from one cpu area. That does not matter as long
2824          * as we end up on the original cpu again when doing the cmpxchg.
2825          *
2826          * We should guarantee that tid and kmem_cache are retrieved on
2827          * the same cpu. It could be different if CONFIG_PREEMPTION so we need
2828          * to check if it is matched or not.
2829          */
2830         do {
2831                 tid = this_cpu_read(s->cpu_slab->tid);
2832                 c = raw_cpu_ptr(s->cpu_slab);
2833         } while (IS_ENABLED(CONFIG_PREEMPTION) &&
2834                  unlikely(tid != READ_ONCE(c->tid)));
2835
2836         /*
2837          * Irqless object alloc/free algorithm used here depends on sequence
2838          * of fetching cpu_slab's data. tid should be fetched before anything
2839          * on c to guarantee that object and page associated with previous tid
2840          * won't be used with current tid. If we fetch tid first, object and
2841          * page could be one associated with next tid and our alloc/free
2842          * request will be failed. In this case, we will retry. So, no problem.
2843          */
2844         barrier();
2845
2846         /*
2847          * The transaction ids are globally unique per cpu and per operation on
2848          * a per cpu queue. Thus they can be guarantee that the cmpxchg_double
2849          * occurs on the right processor and that there was no operation on the
2850          * linked list in between.
2851          */
2852
2853         object = c->freelist;
2854         page = c->page;
2855         if (unlikely(!object || !page || !node_match(page, node))) {
2856                 object = __slab_alloc(s, gfpflags, node, addr, c);
2857         } else {
2858                 void *next_object = get_freepointer_safe(s, object);
2859
2860                 /*
2861                  * The cmpxchg will only match if there was no additional
2862                  * operation and if we are on the right processor.
2863                  *
2864                  * The cmpxchg does the following atomically (without lock
2865                  * semantics!)
2866                  * 1. Relocate first pointer to the current per cpu area.
2867                  * 2. Verify that tid and freelist have not been changed
2868                  * 3. If they were not changed replace tid and freelist
2869                  *
2870                  * Since this is without lock semantics the protection is only
2871                  * against code executing on this cpu *not* from access by
2872                  * other cpus.
2873                  */
2874                 if (unlikely(!this_cpu_cmpxchg_double(
2875                                 s->cpu_slab->freelist, s->cpu_slab->tid,
2876                                 object, tid,
2877                                 next_object, next_tid(tid)))) {
```

```
2878
2879                        note_cmpxchg_failure("slab_alloc", s, tid);
2880                        goto redo;
2881                }
2882                prefetch_freepointer(s, next_object);
2883                stat(s, ALLOC_FASTPATH);
2884        }
2885
2886        maybe_wipe_obj_freeptr(s, object);
2887
2888        if (unlikely(slab_want_init_on_alloc(gfpflags, s)) && object)
2889                memset(object, 0, s->object_size);
2890
2891        slab_post_alloc_hook(s, objcg, gfpflags, 1, &object);
2892
2893        return object;
2894 }
```

每个 kmem_cache 对象表示一个 slab cache. 每个 kmem_cache 对象有一个 per-cpu 的指针 cpu_slab 指向一个 kmem_cache_cpu 对象。换言之,kmem_cache_cpu 对象保存着 slab cache 的 per-cpu 的 slub 信息.kmem_cache_node 表示这个 slab 组件可以使用的内存节点.

本地 cpu(per-cpu) 缓存的空闲 slub obj 通过链表维护。kmem_cache->cpu_slab 的 freelist 成员指向这个链表的第一个空闲对象 (free obj), 如下图:



每个 slub cache(由 kmem_cache 对象表示) 都有一个 per-cpu 的指针 kmem_cache.cpu_slabs.page、一个 per-cpu 的 partial list(kmem_cache.cpu_slabs.partial, 依赖于 config 的配置)、一个 per-node 的 partial list(kmem_cache.nodes[node_no].partial).

kmem_cache_cpu->partial 链表是本地 cpu 私有的 partial, 仅属于单个 cpu 的 per-cpu 变量, 可以无锁访问。而 kmem_cache_node->partial 是全局的, 整个 NUMA 节点共享, 所有 cpu 都能访问, 访问时需要锁保护。当 cpu 本地的 partial 上 slub 过多时, 会迁移部分 slub 到该 cpu 对应 node 的 (node->)partial 链表。如果 cpu 本地的 partial 耗尽了, 那么会优先从 cpu 对应 node 的 partial 链表取空闲 slub。如果对应 node 的 partial 上也无空闲 slub, 那么就会从其他 cpu 对应 node 的 partial 上查找 kxslub。

slub obj 总是优先从 per-cpu 的 slub 上分配。如果 slub 上所有的 obj 都已经分配出去, 那么其他某个 partial slub 上返回的第一个 slub 的第一个空闲 obj 会做为被分配的 obj。每个 active 的 slub 包含

一个 per-cpu 的空闲链表 (freelist), 这个 freelist 由该 active slub 上的空闲 obj 组成. 所以 active 的 slub 上的 obj 在任何时候都会在两个链表的其中之一 (如前所述，这两个链表不一定同步). 一个是无锁链表 (kmem_cache_cpu.cpu_slabs.freelist), 另一个是常规链表 (page.freelist). 在支持用 cmpxchg 指令原子交换 2 个 word 值的架构 (如 x86_64, aarch64 等) 上, 从无锁的空闲链表进行 alloc 和 free 无需获取任何锁, 以及关闭中断和抢占.

- 2830-2834 this_cpu_read() 是一个宏, 隐含了抢占和中断保护. 因此使用它时用户无需担心抢占和中断.
- 2853-2854 获取本地 cpu 的 kmem_cache->cpu_slab->freelist 指向的空闲 obj(对象) 链表, 以及对应的 (复合页或单页的) 首页指针。

  这个首页指针 kmem_cache->cpu_slab->page 指向的是，本地 cpu 会优先分配的 freelist 上的 obj 所在 slub 的首页。
- 2855-2884 如果本地 cpu 上的空闲 obj 链表里已经没有空闲 obj, 或本地 cpu 不存在激活的 slub。换言之, 也就是 slub cache 在本地 cpu 上不存在空闲对象了, 那么走慢车道 (慢速分配路径). 另外, 如果本地 cpu 缓存的可以优先分配 (或称之为激活) 的 slub 不属于该次分配指定的 NUMA 节点, 此时也走慢车道。

  – 2830-2834 在开启了 ***CONFIG_PREEMPT*** 的情况下, 内核中优先级更高的线程运行就会抢占当前 cpu。发生抢占后, 被强占线程被重新调度运行时, 可能调度到其他 core 上运行。这样一来，在被强占前线程运行的 cpu core 和抢占后重新调度运行时的 cpu core 不是同一个。这样导致了->kmem_cache_cpu 不同, 相应的 kmem_cache_cpu->freelist 和 kmem_cache_cpu->tid 也就和被抢占前不一致。

  这里的循环检测是要求: 读取 kmem_cache_cpu->freelist 和读取 kmem_cache_cpu->tid 时运行的 cpu 一致，也就是**读取瞬间**的一致性，避免拿到错误的快照。

  NOTE

  per-cpu 变量的值不会随着线程调度迁移到不同 *cpu core* 上运行而自动更新。
  必须通过 *this_cpu_read()* 或类似函数读出其当前值。不通过这些函数读出，直接把它当做全局变量使用会造成编译时错误。
  当然一旦通过 *this_cpu_read()* 等类似函数读出到普通变量，就脱离了 *per-cpu* 机制，无论线程调度到哪个 *core* 运行，该普通变量值都会不变。

  后续 2874 行 cmxchg_double() 进行原子交换时会保证**修改瞬间**的一致性。在原子操作中，该函数会比较彼时 cpu core 上缓存的 tid 和 freelist 和此处保存的快照中的这两个值是相同的，以此保证是同一个 cpu core, 而且 freelist 未被并发操作。

  NOTE

  per-cpu 读取函数 *this_cpu_read()* 和 *raw_cpu_ptr()* 的区别

	this_cpu_read()	***raw_cpu_ptr()***
返回值	变量 *(value)*	指针 *(pointer)*
抢占安全	安全	不安全 *(需手动禁用抢占)*
用途	快速获取单个值	高效访问结构体

  – 2856 这里是慢车道的流程, 最终返回一个空闲 obj。___slab_alloc() 函数的实现见后面.

- 2858 进入这个代码块意味着本地缓存中有空闲 slub obj 可以直接分配, 也就是快车道. 首先获取下一个空闲对象 obj.
- 2861-2881 以原子的方式 (不使用锁!) 更新 kmem_cache_cpu->freelist 链表, 使 freelist 指向下一个空闲的 obj(next_object)。

  这里的 tid 不是 task id 或 thread id, 而是当前 cpu 本地 slub 缓存全局唯一的事务编号。用来标记本地 cpu slub 缓存是否被外部修改过。它以 cpu core 在系统中的编号作为初始值。在配置项 ***CONFIG_PREEMPT=y*** 时, 自增的步长 = 系统最大 cpu core 数量向上取整到的最接近的 2 的幂次。***CONFIG_PREEMPT=n*** 则自增的步长为 1。

  譬如, 假设 CONFIG_PREEMPT=y, 处理器的 core 为 n, 向上对齐到 2 的幂 N。那么各个 cpu core 的事务编号如下:

  - core 0 的事务编号:0、N、$2N$、$3N$.....
  - core 1 的事务编号:1、$1+N$、$1+2N$、$1+3N$.....
  - core n 的事务编号:n、$n+N$、$n+2N$、$n+3N$.....

  由上述事务编号可以看出任意两个 cpu core 的 tid 永远不会重叠或碰撞, 这样不用全局锁也能保证 tid 的全局唯一性。并且这里 tid 可以通过位运算快速得到 cpu 编号。N 是 2 的幂, 那么 $tid\&(N-1)$ = cpu core 的编号, 无需除法计算。

  这是无锁操作, 保护措施仅针对在这个 cpu core 上执行的代码, 而不防止其他 cpu 访问。

  this_cpu_cmpxchg_double() 用于在本地 cpu 上进行双字段比较和原子的将双字段交换。普通的 cmpxchg 只能原子的更新 1 个变量。而 cmpchg_double 能同时更新两个值。是硬件级原子操作, 无锁。

  2874-2877 代码中,*obj*、*tid*预期更新成*next_object*、*next_tid(tid)*。其中, 前两个入参:*->cpu_slab->freelist*、*->cpu_slab->tid*分别是当前 cpu 私有缓存的 freelist 和 tid(全局事务号)。整个操作的流程如下:

  - 获取新的 obj 指针, 为更新 freelist 做准备。
  - 计算新的全局事务号 next_tid(tid)。如前所述, 新的 tid = 旧的 tid + 步长。
  - 原子操作: 确认 当前 cpu 缓存的 freelist 指针 == 旧的 obj 的地址&&当前 cpu 缓存的 tid == 旧的 tid, 如果是, 更新 freelist 指针和 tid 为新值。

    2879-2880 如果因为并发操作的原因导致原子操作失败 (譬如原子操作中发现当前 cpu 私有缓存的 tid 不等于旧的 tid), 那么跳至 redo 标签处, 重试。
- 2882 该函数由架构相关的汇编指令实现. 目的是预取 next_object 的 FP 到 cpu 高速缓存, 加快下一次分配 obj 的速度.
- 2883 将本次快速执行路径下 obj 成功分配的事件进行记录。主要用于内核统计和调优。
- 2886 如果开启了 free 时自动擦除的特性, 那么把 obj 内部区域存放的 FP 指针 (指向下一个空闲 obj) 清 0。避免 free 后残留指针泄漏信息。
- 2888-2889 如果分配出的 obj 有效 (不为 NULL), 并且需要分配时自动清 0, 那么执行清 0 操作。slab_want_init_on_alloc() 的实现中会判断是否自定义了构造函数 ctor, 若是, 则表明用户自行处理。内核不再执行 memset 自动清 0, 直接优先从 slab_want_init_on_alloc() 返回 false。
- 2893 返回分配到的 obj。

2.1.1 快速路径

上面代码的 2858-2884 行也就是所谓的快速路径。内核进入快速路径分配 obj, 尝试从 cpu 本地缓存的 kmem_cache_cpu->freelist 中快速获取 slub obj。

随后内核会通过 kmem_cache_cpu->freelist 来获取该 cpu 缓存 slub 中的第一个空闲的 obj。

如果当前 cpu 缓存的 slub 是空的（没有空闲 obj 可供分配）或者该 slab 所在的 NUMA 节点并不是我们指定的。那么就会通过 ___slab_alloc 进入到慢速分配路径中。

如果当前 cpu 缓存 slab 有空闲的 obj 并且 slab 所在的 NUMA 节点正是我们指定的，那么将当前 kmem_cache_cpu->freelist 指向的第一个空闲 obj 分配出去。随后通过 get_freepointer_safe() 获取当前被分配出去 obj 的 FP(free pointer) 指针（其指向其下一个空闲 obj），然后将 kmem_cache_cpu->freelist 更新为该 free pointer 指向的空闲 obj。

kmem_cache_alloc_node() > *slab_alloc_node()* > *get_freepointer_safe()*

```
286 static inline void *get_freepointer_safe(struct kmem_cache *s, void *object)
287 {
288     unsigned long freepointer_addr;
289     void *p;
290 
291     if (!debug_pagealloc_enabled_static())
292         return get_freepointer(s, object);
293 
294     freepointer_addr = (unsigned long)object + s->offset;
295     copy_from_kernel_nofault(&p, (void **)freepointer_addr, sizeof(p));
296     return freelist_ptr(s, p, freepointer_addr);
297 }
```

- 294 获取 free_pointer 的指针。free_pointer 位于 object 起始地址的 offset 偏移处。
- 295 对 free_pointer 指针解引用。也就是获取 FP 指针存储的下一个空闲对象的地址。

  普通的解引用操作 (*free_pointer) 在指针是非法地址时可能会产生 Oops 等异常，而 copy_from_kernel_nofault() 会避免读取非法地址，安全容错的读取指针内容。
- 296 返回下一个空闲对象地址。

2.1.2 慢速路径 和 超慢路径

分配一个 slab obj(对象)。内核首先会走**快速路径**在对应的 cpu 的本地缓存中查找空闲的 freelist，如有则返回其指针。如果当前 freelist 已经耗尽，则走**慢速路径**从本地的 partial slub 链表查找 partial slub。如没有，则**超慢路径**从 NUMA 节点中查找 partial slub。还没有，则可能会触发从伙伴系统分配新的页面来创建新的 slub。

慢速路径的入口也就是在下面 ___slab_alloc() 函数的 2781 行:

kmem_cache_alloc_node() > *slab_alloc_node()* > *__slab_alloc()*

```
2765 static void *__slab_alloc(struct kmem_cache *s, gfp_t gfpflags, int node,
2766                           unsigned long addr, struct kmem_cache_cpu *c)
2767 {
2768         void *p;
2769         unsigned long flags;
2770 
2771         local_irq_save(flags);
2772 #ifdef CONFIG_PREEMPTION
2773         /*
2774          * We may have been preempted and rescheduled on a different
2775          * cpu before disabling interrupts. Need to reload cpu area
```

```
2776          * pointer.
2777          */
2778         c = this_cpu_ptr(s->cpu_slab);
2779 #endif
2780
2781         p = ___slab_alloc(s, gfpflags, node, addr, c);
2782         local_irq_restore(flags);
2783         return p;
2784 }
```

- 2771 屏蔽本地所有可屏蔽中断。

  屏蔽本地中断是为了防止中断打断临界区，杜绝在中断上下文中分配或释放 slub、修改本地 freelis 等并行操作的风险。

  NOTE

  本地时钟中断 *tick* 属于可屏蔽中断.

- 2772-2779 如果内核打开了 ***CONFIG_PREEMPT*** 支持内核抢占。那么重新获取 per-cpu 的值 cpu_slab(kmem_cache->kmem_cache_cpu)。

  因为在该函数入口到关中断之前的这段时间里，函数的执行随时可能被中断打断，从而在重新调度时调度到不同的 cpu core 运行。所以需要重新获取当前运行 core 的 per-cpu 变量 kmem_cache_cpu。

  另外，内核的调度点是可屏蔽中断处理返回的执行路径上。那么在 2771 行屏蔽中断后，到 2782 行开中断之间显而易见不会再产生调度，没有切换 core 的风险。因此，关中断后获取的 per-cpu 的缓存指针全程稳定不变。

  NOTE

  为什么此处 *(2771)* 行不使用 *preempt_disable()*?

  preempt_disable() 只能禁止进程抢占或进程调度，不能禁止中断。所以硬件中断仍然可以打断执行，从而在中断上下文中操作修改 *cpu* 的本地缓存。
  如果禁止本地中断，那么不会进中断处理流程，也就不会执行中断返回时进行的调度检查的代码，于是就不会有线程调度切换的机会。

- 2781 慢速路径更底层的入口是 ____slab_alloc()(见后面).

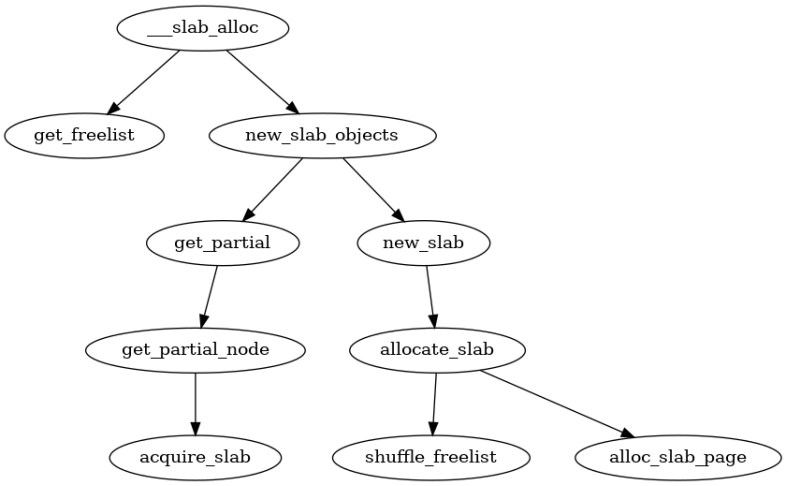


图 2–11: 调用图: ____slab_alloc()

如前所述,____slab_alloc() 是在本地中断已关闭的情况下慢速路径的执行。但如果在执行中发现有新 obj 被释放到了空闲链表，那么执行速度依然很快.

kmem_cache_alloc_node() > *slab_alloc_node()* > *__slab_alloc()* > *____slab_alloc()*

```
2641 /*
2642  * Slow path. The lockless freelist is empty or we need to perform
2643  * debugging duties.
2644  *
2645  * Processing is still very fast if new objects have been freed to the
2646  * regular freelist. In that case we simply take over the regular freelist
2647  * as the lockless freelist and zap the regular freelist.
2648  *
2649  * If that is not working then we fall back to the partial lists. We take the
2650  * first element of the freelist as the object to allocate now and move the
2651  * rest of the freelist to the lockless freelist.
2652  *
2653  * And if we were unable to get a new slab from the partial slab lists then
2654  * we need to allocate a new slab. This is the slowest path since it involves
2655  * a call to the page allocator and the setup of a new slab.
2656  *
2657  * Version of __slab_alloc to use when we know that interrupts are
2658  * already disabled (which is the case for bulk allocation).
2659  */
2660 static void *___slab_alloc(struct kmem_cache *s, gfp_t gfpflags, int node,
2661                            unsigned long addr, struct kmem_cache_cpu *c)
2662 {
2663         void *freelist;
2664         struct page *page;
2665
2666         stat(s, ALLOC_SLOWPATH);
2667
2668         page = c->page;
2669         if (!page) {
2670                 /*
2671                  * if the node is not online or has no normal memory, just
2672                  * ignore the node constraint
2673                  */
2674                 if (unlikely(node != NUMA_NO_NODE &&
2675                              !node_state(node, N_NORMAL_MEMORY)))
2676                         node = NUMA_NO_NODE;
2677                 goto new_slab;
2678         }
```

- 2666 如果开启了 slub 组件的事件统计功能，那么对本次进入慢车道分配 (次数) 进行统计数 +1。
- 2668 获取本地缓存的优先分配 slub 对应的首页 page 指针.
- 2674-2676 如果指定的 NUMA node 在线但没有普通内存，那么忽略 node 节点的限定约束。
- 2677 如果 page 指针为 NULL, 那么缓存的 slub 没有空闲 obj，意味着要获取新的 slub. 那么跳转到 new_slab 分支执行。

kmem_cache_alloc_node() > *slab_alloc_node()* > *__slab_alloc()* > *____slab_alloc()*

```
2679 redo:
2680
2681         if (unlikely(!node_match(page, node))) {
2682                 /*
2683                  * same as above but node_match() being false already
2684                  * implies node != NUMA_NO_NODE
2685                  */
2686                 if (!node_state(node, N_NORMAL_MEMORY)) {
2687                         node = NUMA_NO_NODE;
2688                         goto redo;
```

```
2689                } else {
2690                        stat(s, ALLOC_NODE_MISMATCH);
2691                        deactivate_slab(s, page, c->freelist, c);
2692                        goto new_slab;
2693                }
2694        }
2695
2696        /*
2697         * By rights, we should be searching for a slab page that was
2698         * PFMEMALLOC but right now, we are losing the pfmemalloc
2699         * information when the page leaves the per-cpu allocator
2700         */
2701        if (unlikely(!pfmemalloc_match(page, gfpflags))) {
2702                deactivate_slab(s, page, c->freelist, c);
2703                goto new_slab;
2704        }
2705
2706        /* must check again c->freelist in case of cpu migration or IRQ */
2707        freelist = c->freelist;
2708        if (freelist)
2709                goto load_freelist;
2710
2711        freelist = get_freelist(s, page);
2712
2713        if (!freelist) {
2714                c->page = NULL;
2715                stat(s, DEACTIVATE_BYPASS);
2716                goto new_slab;
2717        }
2718
2719        stat(s, ALLOC_REFILL);
2720
2721load_freelist:
2722        /*
2723         * freelist is pointing to the list of objects to be used.
2724         * page is pointing to the page from which the objects are obtained.
2725         * That page must be frozen for per cpu allocations to work.
2726         */
2727        VM_BUG_ON(!c->page->frozen);
2728        c->freelist = get_freepointer(s, freelist);
2729        c->tid = next_tid(c->tid);
2730        return freelist;
```

在 slab cache 的子系统中存在两个 freelist：

一个是 page->freelist，也就是 kmem_cache_cpu->page->freelist。系统中同种对象的 slub 是通过链表连在一起的，而这个链表的每个节点都是通过 slub 的首页的 page 串联。因此可以看出 kmem_cache_cpu->page->freelist 是单纯的站在一个特定的 slub 的视角来表示这个 slub 中的空闲 obj 列表。

另一个是 kmem_cache_cpu->freelist，特指 slab 被本地 cpu 缓存的空闲 obj 链表。当某个 slab 被本地 cpu 缓存之后，这个 slub 的首页 page->freeelist 直接赋值给 kmem_cache_cpu->freelist，然后 page->freeelist 置 NULL。同时 page->frozen 设置为 1，表示 slab 被冻结在当前 cpu 的本地缓存中。

而 slab 一旦被当前 cpu 缓存，它的状态就变为了冻结状态（slab->frozen = 1），当前 cpu 可以从处于冻结状态下的 slub 中分配或者释放 obj，但是其他 cpu 只能释放 obj 到该 slub 中，不能从该 slub 中分配 obj。

如果一个 slab 被一个 cpu 缓存之后，那么这个 cpu 在该 slub 看来就是本地 cpu，当本地 cpu 释放 obj 回这个 slab 的时候会释放回 kmem_cache_cpu->freelist 链表中；如果其他 cpu 想要释放 obj 回该

slab 时，其他 cpu 只能将 obj 释放回该 slub 的 kmem_cache_cpu->page->freelist 中。

譬如：cpu1 在本地缓存了 slub A，cpu B 在本地缓存了 slab B(它们都同属于一个 kmem_cache 管理)，进程先从 slab A 中获取了一个 obj，正常情况下如果进程一直在 cpu B 上运行的话，当进程释放该 obj 回 slab A 中时，会直接释放回 (kmem_cache_cpu A)->freelist 链表中。但如果进程在 slab A 中获取完 obj 之后，被调度到了 cpu B 上运行，这时进程想要释放对象回 slab A 中时，就不能走快速路径了，因为 cpu B 本地缓存的是 slab B，所以 cpu B 只能将该 obj 释放至 slab A 自己的 freelist 中。这种情况下，在 slab A 的内部视角里，就有了两个 freelist 链表，它们的共同之处都是用于组织 slab A 中的空闲 obj，但是 kmem_cache_cpu A->freelist 链表中组织的是缓存在 cpu A 本地的空闲 obj, (kmem_cache_cpu A->)page->freelist 链表组织的是由其他 cpu 释放的 slab A 的空闲 obj。

- 2707-2709 这里需要再次检查 slab cache 本地 cpu 缓存中的 freelist 是否有空闲对象. 因为当前进程可能被中断，当重新调度之后，其他进程可能已经释放了一些 obj 到 cpu 本地缓存中，kmem_cache_cpu->freelist 可能此时就不为空了。如果此时不为空了，则从 cpu 本地缓存的 freelist 中直接分配 obj, 也就是下面的 2721-2730.

  2727 被本地 cpu 缓存的 slub 所属的 page 必须是 frozen 冻结状态，只允许本地 cpu 从中分配 obj。

  2728 kmem_cache_cpu 中的 freelist 指向被 cpu 缓存 slub 中第一个空闲 obj。由于第一个空闲 obj 马上要被分配出去，所以这里需要获取下一个空闲 obj 来更新 freelist.

  2729 更新本地 cpu slab 缓存分配 obj 时的全局事务号 (transaction id)。每分配完一次 obj，kmem_cache_cpu 中的全局事务号 (tid) 都需要改变。

- 2711-2717 现在检查 kmem_cache_cpu->page->freelist ，它表示由其他 cpu 所释放的本地 cpu 上缓存的 slub 的 obj 的列表。因为此时可能其他 cpu 释放了一些本地缓存 slub 的 obj，这时 slub 对应的 kmem_cache_cpu->page->freelist 就不为空了，就可以直接分配. 如果仍然没有空闲 obj, 则走慢速路径分配新的 slub。get_freelist() 函数的实现见后。

kmem_cache_alloc_node() > *slab_alloc_node()* > *__slab_alloc()* > *____slab_alloc()* > *get_freelist()*

```
2617 static inline void *get_freelist(struct kmem_cache *s, struct page *page)
2618 {
2619         struct page new;
2620         unsigned long counters;
2621         void *freelist;
2622
2623         do {
2624                 freelist = page->freelist;
2625                 counters = page->counters;
2626
2627                 new.counters = counters;
2628                 VM_BUG_ON(!new.frozen);
2629
2630                 new.inuse = page->objects;
2631                 new.frozen = freelist != NULL;
2632
2633         } while (!__cmpxchg_double_slab(s, page,
2634                 freelist, counters,
2635                 NULL, new.counters,
2636                 "get_freelist"));
2637
```

```
2638            return freelist;
2639    }
```

- 2624 获取 (kmem_cache_cpu->)page 结构的 freelist，freelist 指向当其他 cpu 释放的空闲 obj。

  当一个 slub 被本地 cpu 缓存时，slub 的 freelist 被更新到 kmem_cache_cpu->freelist, 然后 slub 的 freelist(kmem_cache_cpu->freelist) 置为 null。但是这个 slub 上被分配的 obj 被不同的 cpu 跨 cpu 释放后，会依次又挂到 slub 自身的 freelist(kmem_cache_cpu->page->freelist)。需要注意的是，目标 slub 上的 obj 被缓存它的 cpu 释放后，这个 obj 会挂到本地缓存的 freelist(kmem_cache_cpu->freelist)。这两种场景下，释放回的 obj 挂载的 freelist 是不同的。

- 2630 更新 inuse 字段，置为目标 slub 的总的 obj 的数目，表示 page 中的 obj 全部被分配出去了

- 2631 如果 freelist != null，表示其他 cpu 又释放了一些 obj 到 page 中（slub）。则 page->frozen = 1 , slub 依然冻结在 cpu 本地缓存中; 如果 freelist == null, 则 page->frozen = 0，slub 从 cpu 本地缓存中解冻脱离。

- 2633-2636 原子的同时更新 page->freelist 和 page->counters。当 page 被本地 cpu 缓存时，(slub 自身的)page->freelist 需要置空。因为在本地 cpu 缓存场景下 page->freelist 指向其他 cpu 释放的空闲 obj 列表;kmem_cache_cpu->freelist 指向的是被本地 cpu 缓存的空闲 obj 列表。这两个列表中的空闲 obj 共同组成了 slub 中的空闲 obj。

  代码中 page->counters 成员是用联合体 (union) 封装的，如下:

```
union {
    ... ...
    unsigned long counters;     /* SLUB */
    struct {            /* SLUB */
        unsigned inuse:16;
        unsigned objects:15;
        unsigned frozen:1;
    };
};
```

  各字段如下:

  - ***counters*** 位域结构体中各位域合并后的值，用于双字比较交换的原子操作。
  - ***inuse*** 目标 slub 已经分配出去的 obj 的数目。
  - ***objects*** 目标 slub 中包含的 obj 的总数目。
  - ***frozen*** 冻结标志。表示目标 slub 被 cpu 独占。

  所以，这里原子更新 counters 其实是同时更新了 new.inuse 和 new.frozen。

- 2638 如果返回的 freelist 不为 NULL, 那么从 2631 行可以看出，slub 会保持冻结状态 (frozen 标记); 如果为 NULL, 意味着目标 slub 已经耗尽，并且已经解冻。

接下来回到 ____slab_alloc()。

```
2731
2732 new_slab:
2733
2734         if (slub_percpu_partial(c)) {
2735                 page = c->page = slub_percpu_partial(c);
2736                 slub_set_percpu_partial(c, page);
2737                 stat(s, CPU_PARTIAL_ALLOC);
2738                 goto redo;
2739         }
2740
2741         freelist = new_slab_objects(s, gfpflags, node, &c);
2742
2743         if (unlikely(!freelist)) {
2744                 slab_out_of_memory(s, gfpflags, node);
2745                 return NULL;
2746         }
2747
2748         page = c->page;
2749         if (likely(!kmem_cache_debug(s) && pfmemalloc_match(page, gfpflags)))
2750                 goto load_freelist;
2751
2752         /* Only entered in the debug case */
2753         if (kmem_cache_debug(s) &&
2754                         !alloc_debug_processing(s, page, freelist, addr))
2755                 goto new_slab;  /* Slab failed checks. Next slab needed */
2756
2757         deactivate_slab(s, page, get_freepointer(s, freelist), c);
2758         return freelist;
2759 }
```

- 2734-2739 查看 kmem_cache_cpu->partial 链表中是否有可供分配 slub obj。slub_percpu_partial(c) 实际是 (c)->partial 的封装宏。

  2735 获取 cpu 本地缓存 kmem_cache_cpu partial 列表中的第一个 slub(用该 slub 的首页 page 指示), 并将这个 slub 提升为 cpu 本地缓存中激活的 slub，该 slub 首页指针赋值给 c->page。

  2736 将 partial 列表中第一个 slub (kmem_cache_cpu->page) 从 partial 列表中摘下, 并将列表中的下一个 slub 更新为 partial 列表的首结点。

  2737 更新状态统计信息，记录本次分配是从 kmem_cache_cpu 的 partial 列表中分配。

  2738 重新回到 tag: redo，这下就可以从 page->freelist 中获取 obj 了。并且在 tag: load_freelist 中将 page->freelist 更新到 c->freelist 中，page->freelist 设置为 null。此时 slab cache 中的 cpu 本地缓存 kmem_cache_cpu 的 freelist 以及 page 就更新为了 partial 列表中的 slub。

- 2741 流程走到这里表示 cpu 本地缓存 partial 列表中也没有 slub 了, 需要进一步到 numa node cache —- kmem_cache_node 中的 partial 列表去查找, 如果还是没有, 就只能去伙伴系统中申请新的 slub, 然后分配 obj。该函数为 slab cache 在**超慢路径**下分配 obj 的核心逻辑，见后面代码。

- 2743-2746 如果伙伴系统中无法分配 slub 所需的 page，那么就提示内存不足，分配失败。

- 2750 此时从 kmem_cache_node->partial 列表中获取的 slub 或者从伙伴系统中重新申请的 slub 已经被激活为本地 cpu 缓存的 kmem_cache_cpu->page。这里需要跳转到 tag: load_freelist，从本地 cpu 缓存 slub 中获取第一个 obj 返回。

new_slab_objects() 函数如下。该函数会向伙伴系统申请一组物理连续的页，然后把这些页初始化成一个全新的 slub，并按需求切割成 obj 链表。最后返回这个 slub 首页对应 page 的指针。

kmem_cache_alloc_node() > *slab_alloc_node()* > *__slab_alloc()* > *____slab_alloc()* > *new_slab_objects()*

```
2564 static inline void *new_slab_objects(struct kmem_cache *s, gfp_t flags,
2565                         int node, struct kmem_cache_cpu **pc)
2566 {
2567         void *freelist;
2568         struct kmem_cache_cpu *c = *pc;
2569         struct page *page;
2570 
2571         WARN_ON_ONCE(s->ctor && (flags & __GFP_ZERO));
2572 
2573         freelist = get_partial(s, flags, node, c);
2574 
2575         if (freelist)
2576                 return freelist;
2577 
2578         page = new_slab(s, flags, node);
2579         if (page) {
2580                 c = raw_cpu_ptr(s->cpu_slab);
2581                 if (c->page)
2582                         flush_slab(s, c);
2583 
2584                 /*
2585                  * No other reference to the page yet so we can
2586                  * muck around with it freely without cmpxchg
2587                  */
2588                 freelist = page->freelist;
2589                 page->freelist = NULL;
2590 
2591                 stat(s, ALLOC_SLAB);
2592                 c->page = page;
2593                 *pc = c;
2594         }
2595 
2596         return freelist;
2597 }
```

- 2573-2576 尝试从指定 node 节点的 kmem_cache_node 的 partial 列表，获取可以分配空闲 obj 的 slub，如果指定 numa 节点的内存不足，则会根据 cpu 访问距离的远近，进行跨 numa 节点分配。
- 2578 流程走到这里说明 numa cache 缓存的 slub 也用尽了，无法找到可以分配 obj 的 slub。只能向底层伙伴系统重新申请内存页创建 slub，然后从新的 slub 中分配 obj。
- 2578-2594 将新申请的 slub 缓存到本地 cpu 的 slub 缓存中。

  2580 获取 cpu 本地缓存的管理结构体的指针:per-cpu 变量 kmem_cache_cpu。

  2582 刷新本地 cpu 缓存，将旧的 slub 缓存与 cpu 本地缓存解绑. 该函数细节随后再详看

  2588-2589 将新申请的 slub 与 cpu 本地缓存绑定，page->freelist 赋值给 kmem_cache_cpu->freelist. 并将 page->freelist 置 NULL.

  2592 将新申请的 slub 对应的首页 page 指针赋值给 kmem_cache_cpu->page。

内核首先会在 get_partial 函数中找到我们指定的 NUMA 节点缓存结构 kmem_cache_node ，然后开始遍历 kmem_cache_node->partial 链表直到找到一个可供分配 obj 的 slub。然后将这个 slub 激活为本地 cpu 缓存的优先分配的 slub，并从 kmem_cache_node->partial 链表中依次填充 slub 到 kmem_cache_cpu->partial。

如果我们指定的 NUMA 节点 kmem_cache_node->partial 链表也是空的，随后内核就会跨 NUMA 节点进行查找，按照访问距离由近到远，开始查找其他 NUMA 节点 kmem_cache_node->partial 链表。

如果还是不行，最后就只能通过 new_slab 函数到伙伴系统中重新申请一个 slub，并将这个 slub 激活为本地 cpu 缓存。

kmem_cache_alloc_node() ··· ··· *___slab_alloc()* *new_slab_objects()* *get_partial()*

```
2061 static void *get_partial(struct kmem_cache *s, gfp_t flags, int node,
2062                 struct kmem_cache_cpu *c)
2063 {
2064         void *object;
2065         int searchnode = node;
2066 
2067         if (node == NUMA_NO_NODE)
2068                 searchnode = numa_mem_id();
2069 
2070         object = get_partial_node(s, get_node(s, searchnode), c, flags);
2071         if (object || node != NUMA_NO_NODE)
2072                 return object;
2073 
2074         return get_any_partial(s, flags, c);
2075 }
```

get_partial 函数的主要任务是选取合适的 NUMA 缓存节点来获取空闲 obj 返回。优先使用指定的 NUMA 节点。如果指定的 NUMA 节点中没有足够的内存，内核就会跨 NUMA 节点按照访问距离的远近，选取一个合适的 NUMA 节点。

- 2067-2068 如果没有限定 numa node 节点，则首先选取本地 cpu 对应的 numa node 进行内存分配。
- 2070-2072 从选定 node 的 kmem_cache_node 缓存的 partial 列表中获取对象。
  - 2071-2072 如果顺利获取到对象，则返回。若没有从指定的 numa node 节点的 partial 链表找到空闲 obj, 并且又不允许从其他 NUMA node 节点获取内存，那么返回 NULL(失败)。
- 2074 因为选定 node 的 kmem_cache_node 缓存的 partial 列表无空闲 obj，没有任何可供分配的 slub, 那么按照访问距离远近，继续遍历 searchnode 之后的 numa node，进行跨节点内存分配

kmem_cache_alloc_node() ··· ··· *___slab_alloc()* *new_slab_objects()* *get_partial_node()*

```
1948 static void *get_partial_node(struct kmem_cache *s, struct kmem_cache_node *n,
1949                                 struct kmem_cache_cpu *c, gfp_t flags)
1950 {
1951         struct page *page, *page2;
1952         void *object = NULL;
1953         unsigned int available = 0;
1954         int objects;
1955 
1956         /*
1957          * Racy check. If we mistakenly see no partial slabs then we
1958          * just allocate an empty slab. If we mistakenly try to get a
1959          * partial slab and there is none available then get_partial()
1960          * will return NULL.
1961          */
1962         if (!n || !n->nr_partial)
1963                 return NULL;
1964 
1965         spin_lock(&n->list_lock);
1966         list_for_each_entry_safe(page, page2, &n->partial, slab_list) {
1967                 void *t;
1968 
1969                 if (!pfmemalloc_match(page, flags))
1970                         continue;
```

```
1971
1972                t = acquire_slab(s, n, page, object == NULL, &objects);
1973                if (!t)
1974                        break;
1975
1976                available += objects;
1977                if (!object) {
1978                        c->page = page;
1979                        stat(s, ALLOC_FROM_PARTIAL);
1980                        object = t;
1981                } else {
1982                        put_cpu_partial(s, page, 0);
1983                        stat(s, CPU_PARTIAL_NODE);
1984                }
1985                if (!kmem_cache_has_cpu_partial(s)
1986                        || available > slub_cpu_partial(s) / 2)
1987                        break;
1988
1989        }
1990        spin_unlock(&n->list_lock);
1991        return object;
1992 }
```

get_partial_node() 在限定的 NUMA 节点的 kmem_cache_node->partial 链表中查找 slub。

- 1966 依次遍历 kmem_cache_node 的 partial 列表，检索链表每个节点 (->partial->slab_list) 对应的 slub ，看是否有空闲 obj 分配以及填充 kmem_cache_cpu。
- 1972 page 表示当前遍历到的 slub，这里会从该 slub 中获取空闲对象赋值给 t，并将 slub 从 kmem_cache_node 的 partial 列表上摘下。acquire_slab() 函数见后。
- 1973-1974 如果 partial 列表上已经没有可供分配 obj 的 slub 了 (t 为 NULL)。意味着 slub 都耗尽了，那么退出循环，进入伙伴系统重新申请 slub。
- 1976 objects 表示当前 slub 中包含的剩余空闲 obj 个数。available 用于统计目前已经遍历的 slub 中所有空闲 obj 的总和, 后面会根据 available 的值来判断是否继续填充 kmem_cache_cpu 的 partial 链表。
- 1978-1980 由于 *object==NULL*, 因此第一次循环会走到这里，第一次循环主要是满足当前 obj 分配的需求, 将 partila 列表中第一个 slub 缓存进 kmem_cache_cpu 中作为本地 cpu 优先分配的 slub。
- 1982-1983 第二次以及后面的循环就会走到这里，目的是从 kmem_cache_node 的 partial 列表中摘下 slub，然后填充进 kmem_cache_cpu 的 partial 列表里。
- 1985-1987 判断是否继续填充 kmem_cache_cpu 中的 partial 列表。

  kmem_cache_has_cpu_partial 用于判断 slab cache 是否配置了 CONFIG_SLUB_CPU_PARTIAL。配置了 CONFIG_SLUB_CPU_PARTIAL 选项意味着开启了 kmem_cache_cpu 中的 partial 列表，没有配置的话，本地 cpu 缓存中就不会有 partial 列表。kmem_cache_cpu 中缓存被填充之后的空闲 obj 数目（包括 partial 列表）不能超过 kmem_cache 结构中 cpu_partial 指定的 个数/2。

get_partial_node 函数通过遍历 NUMA 节点缓存结构的 kmem_cache_node->partial 链表主要做两件事情:

- 1. 将第一个遍历到的 slub 从 node 的 partial 链表中摘下，激活为本地 cpu 缓存的 slub (kmem_cache_cpu->page)。

- 2. 继续遍历 partial 链表，后面遍历到的 slub 会串联挂到本地 cpu 缓存 kmem_cache_cpu->partial 链表中，直到当前 cpu 缓存的所有空闲 obj 数目 available 超过了 $kmem_cache- > cpu_partial / 2$ 的限制。

现在本地 cpu 缓存的 slub 已经被激活，随后内核会从 kmem_cache_cpu->freelist 中分配一个空闲 obj 出来给进程使用。

kmem_cache_alloc_node() ··· ··· *get_partial()* *get_partial_node()* *acquire_slab()*

```
1902 static inline void *acquire_slab(struct kmem_cache *s,
1903                 struct kmem_cache_node *n, struct page *page,
1904                 int mode, int *objects)
1905 {
1906         void *freelist;
1907         unsigned long counters;
1908         struct page new;
1909 
1910         lockdep_assert_held(&n->list_lock);
1911 
1912         /*
1913          * Zap the freelist and set the frozen bit.
1914          * The old freelist is the list of objects for the
1915          * per cpu allocation list.
1916          */
1917         freelist = page->freelist;
1918         counters = page->counters;
1919         new.counters = counters;
1920         *objects = new.objects - new.inuse;
1921         if (mode) {
1922                 new.inuse = page->objects;
1923                 new.freelist = NULL;
1924         } else {
1925                 new.freelist = freelist;
1926         }
1927 
1928         VM_BUG_ON(new.frozen);
1929         new.frozen = 1;
1930 
1931         if (!__cmpxchg_double_slab(s, page,
1932                         freelist, counters,
1933                         new.freelist, new.counters,
1934                         "acquire_slab"))
1935                 return NULL;
1936 
1937         remove_partial(n, page);
1938         WARN_ON(!freelist);
1939         return freelist;
1940 }
```

从 kmem_cache_node 的 partial 列表中摘下一个 slub 分配 obj。随后将摘下的 slub 放入 cpu 本地缓存 kmem_cache_cpu 中缓存，后续分配 obj 直接就会优先从 cpu 缓存的 slub 中分配。

- 1917-1918 page 是即将从 kmem_cache_node 的 partial 列表摘下的 slub 的首页指针。获取 slub 自身的空闲 obj 列表 freelist (page->freelist), 以及一个表面上是 *unsigned long* 类型的变量 counters。*page->counters* 用来存储 slub 的状态数据。它其实是几个位域的拼接后组成的，是一个 union 联合体中的成员:

```
            union {
                ... ...
                unsigned long counters;     /* SLUB */
                struct {            /* SLUB */
                    unsigned inuse:16;
```

```
                    unsigned objects:15;
                    unsigned frozen:1;
                };
            };
```

counters 对应的位域成员的含义在前面部分已经讨论过。

- 1920 objects 获取目标 slub 当前还剩下的空闲 obj 数量。
- 1921-1926
  - *mode == true* 表示将 slub 从 kmem_cache_node 的 partial 链表摘下之后，激活到 cpu 本地缓存中 (kmem_cache_cpu->page)。

    acquire_slab() 函数被调用的地方在整个内核代码中只有一处，也就是 get_partial_node()->acquire_slab()。我们可以看到在 get_partial_node() 函数循环调用 acquire_slab() 时，传递的入参 *mode* 是一个 bool 表达式:***object == NULL***。

    由于 get_partial_node() 中 object 初始化就是 NULL，所以第一次进循环中调用 acquire_slab() 时，*mode=true*。因此，我们可以知道，如果 kmem_cache_node 的 partial partial 链表如果有空闲 slub，那么首个 slub 会被摘取挂到 cpu 本地缓存 (kmem_cache_cpu->page),slub 的 freelist 会挂到 kmem_cache_cpu->freelist，而 kmem_cache_cpu->page->freelist 会置 NULL。
  - *mode == false* 表示将 slub 摘取之后更新到 kmem_cache_cpu 缓存的 partial 链表中。

  这里的 ***new*** 是一个临时变量，主要来存储 union 成员 counters 中各位域的值。主要目的在于为后续 1931 行 ___cmpxchg_double_slab() 进行双字原子替换操作预先准备 counters 的新值。
- 1929 slub 激活成 cpu 本地缓存之后需要置冻结标记。冻结标记意味着其他 cpu 不能从这个 slub 分配 obj，而只能跨 cpu core 释放 obj 到这个 slub。
- 1931-1935 更新 slub (page 表示) 中的 freelist 和 counters。
- 1937 将目标 slub 从 kmem_cache_node 的 partial 列表上摘除，完成 slub 从 kmem_cache_node->partial 到 kmem_cache_cpu->page 迁移的最后一步。

再回到 new_slab_objects()，当缓存中无空闲 obj 时。最后调用 new_slab() 向伙伴系统申请连续物理页新建 slub。

kmem_cache_alloc_node() ··· ··· *___slab_alloc()* *new_slab_objects()* *new_slab()*

```
1814 static struct page *new_slab(struct kmem_cache *s, gfp_t flags, int node)
1815 {
1816         if (unlikely(flags & GFP_SLAB_BUG_MASK))
1817                 flags = kmalloc_fix_flags(flags);
1818
1819         return allocate_slab(s,
1820                 flags & (GFP_RECLAIM_MASK | GFP_CONSTRAINT_MASK), node);
1821 }
```

- 1816-1817 进行新建 slub 时的非法 flag 拦截，如果碰到非法 flag 位就对非法 bit 位清 0 修正。

  GFP_SLAB_BUG_MASK 是非法 flag 掩码，如下:

```
#define GFP_SLAB_BUG_MASK (__GFP_DMA32|__GFP_HIGHMEM|~__GFP_BITS_MASK)
```

- **禁止 *___GFP_HIGHMEM*** slub 组件返回的是内核线性地址，需要直接指针访问或解引用。但是 ZONE_HIGHMEM 区域的页面只能用 kmap() 进行临时映射，它是用内核预留的一段虚拟地址窗口动态或临时建立页表映射。没有下面这种内核线性地址的运算关系:

$$VA(\text{虚拟地址}) = PAGE_OFFSET + PA(\text{物理地址})$$

  这种永久固定的、一对一的、无需动态页表的线性映射。
- **禁止 *___GFP_DMA32*** 内核没有配套的全局 DMA32 kmalloc 缓存，如果传入可能引起 bug。
- **禁止 *(___GFP_BITS_MASK)*** 这个取反运算得到的是 mask 范围外的数值，也就是非法值。

其中调用 allocate_slab() 用于新构建一个 slub。源码如下:

kmem_cache_alloc_node() ··· ··· *new_slab_objects()* *new_slab()* *allocate_slab()*

```
1734 static struct page *allocate_slab(struct kmem_cache *s, gfp_t flags, int node)
1735 {
1736         struct page *page;
1737         struct kmem_cache_order_objects oo = s->oo;
1738         gfp_t alloc_gfp;
1739         void *start, *p, *next;
1740         int idx;
1741         bool shuffle;
1742 
1743         flags &= gfp_allowed_mask;
1744 
1745         if (gfpflags_allow_blocking(flags))
1746                 local_irq_enable();
1747 
1748         flags |= s->allocflags;
1749 
1750         /*
1751          * Let the initial higher-order allocation fail under memory pressure
1752          * so we fall-back to the minimum order allocation.
1753          */
1754         alloc_gfp = (flags | __GFP_NOWARN | __GFP_NORETRY) & ~__GFP_NOFAIL;
1755         if ((alloc_gfp & __GFP_DIRECT_RECLAIM) && oo_order(oo) > oo_order(s->min))
1756                 alloc_gfp = (alloc_gfp | __GFP_NOMEMALLOC) & ~(__GFP_RECLAIM|__GFP_NOFAIL
                            );
1757 
1758         page = alloc_slab_page(s, alloc_gfp, node, oo);
1759         if (unlikely(!page)) {
1760                 oo = s->min;
1761                 alloc_gfp = flags;
1762                 /*
1763                  * Allocation may have failed due to fragmentation.
1764                  * Try a lower order alloc if possible
1765                  */
1766                 page = alloc_slab_page(s, alloc_gfp, node, oo);
1767                 if (unlikely(!page))
1768                         goto out;
1769                 stat(s, ORDER_FALLBACK);
1770         }
1771 
1772         page->objects = oo_objects(oo);
1773 
1774         page->slab_cache = s;
1775         __SetPageSlab(page);
1776         if (page_is_pfmemalloc(page))
1777                 SetPageSlabPfmemalloc(page);
1778 
1779         kasan_poison_slab(page);
```

```
1780
1781            start = page_address(page);
1782
1783            setup_page_debug(s, page, start);
1784
1785            shuffle = shuffle_freelist(s, page);
1786
1787            if (!shuffle) {
1788                    start = fixup_red_left(s, start);
1789                    start = setup_object(s, page, start);
1790                    page->freelist = start;
1791                    for (idx = 0, p = start; idx < page->objects - 1; idx++) {
1792                            next = p + s->size;
1793                            next = setup_object(s, page, next);
1794                            set_freepointer(s, p, next);
1795                            p = next;
1796                    }
1797                    set_freepointer(s, p, NULL);
1798            }
1799
1800            page->inuse = page->objects;
1801            page->frozen = 1;
1802
1803    out:
1804            if (gfpflags_allow_blocking(flags))
1805                    local_irq_disable();
1806            if (!page)
1807                    return NULL;
1808
1809            inc_slabs_node(s, page_to_nid(page), page->objects);
1810
1811            return page;
1812    }
```

- 1737 kmem_cache_order_objects 的高 16 位表示构建一个目标 slub 需要的物理连续的内存页数目，低 16 位表示目标 slub 能够容纳的 obj 数目。
- 1745-1746 当分配标志允许阻塞休眠时，打开本地中断。

  这是为了让 tick 中断等正常运行，避免关中断后休眠导致死锁。因为我们在前面已经讨论过，内核中调度除了发生在线程主动让出 cpu，还发生在中断处理返回的路径上。因为原子分配不能休眠，所以允许保持中断关闭。

  另外，我们在函数 ___slab_alloc() 中关闭本地中断是因为要操作 per-cpu 的结构体变量 (kmem_cache->)***kmem_cache_cpu***。但是代码运行至此，说明本地 cpu 或其他全局 node 的缓存 slub 已经耗尽，需要重新向伙伴系统申请物理内存，暂时无需操作本地 per-cpu 的 kmem_cache_cpu。等申请到物理内存需要重新操作本地 freelist 链表时，再重新关本地中断。

- 1754 内存紧张时，首次分配大阶内存允许失败，降阶重试用最小页阶完成分配。此处置允许这些动作的相关标记。
- 1755-1756 本次分配允许直接回收内存，并且组成目标 slub 的理想最优页阶 (oo_order())> 能容纳至少一个目标 slub obj 的最小连续页阶数，那么禁止访问紧急预留内存、禁止回收、允许分配失败。
- 1758 向伙伴系统申请 oo 属性限定的内存页。(该函数见后面)

- 1760 如果伙伴系统无法满足 oo 指定的内存页数目, 那么这里再次尝试用 (kmem_cache->)min 中指定的最小能容纳一个 obj 的连续页的数目向伙伴系统申请内存页。***kmem_cache->min*** 表示当内存不足或者内存碎片的原因无法满足内存分配时，至少要保证容纳一个 obj 时内存页数目。
- 1761 此处 flags 重置。不再禁止: 访问紧急预留内存、回收。不再允许分配失败。
- 1766 (降阶后) 再次向伙伴系统申请容纳所需要的内存页。
- 1772 初始化 slub 对应的 struct page 结构中的属性。获取 slub 可以容纳的对象个数
- 1774 将 slub cache 与 page 结构关联。通过 slub 的首页 page 指针指向 kmem_cache，也就是将 slub 和 slub cache 描述符 kmem_cache 关联。
- 1775 将 PG_slab 标识设置到 struct page 的 flag 属性中，表示该内存页 page 被 slub 组件管理。如果是复合页，那么也通过首页的 flag 属性表示整个这个复合页被 slub 组件管理。
- 1776-1777 ========== 待分析
- 1779 用 0xFC 填充 slub 中的内存，用于 KASAN 机制对内存访问越界检查。
- 1781 获取内存页对应的虚拟内存地址。
- 1783 开启内核内存页的调试能力。如果开启了内核配置项 CONFIG_SLUB_DEBUG，那么对获取的内存页 (可能是复合页) 填充 ***POISON_INUSE(0x5a)*** 进行污化 (具体**污化**的功能见 **SLUB debug** 章)。顾名思义，将页填充这个字节可以在之后用来判断页是否未经初始化就使用。

- 1785 在配置了 CONFIG_SLAB_FREELIST_RANDOM 选项的情况下,shuffle_freelist() 才会生效。作用是在 slub 中以随机顺序排列 freelist 列表。这是为了安全考虑，防止攻击者预测下一个分配 obj 的地址，如果新 slub 内部的 obj 在 freelist 中不按物理地址的顺序串联，那么就大幅提高了安全门槛。

  shuffle_freelist() 返回值 *shuffle == true* 表示随机初始化 freelist。*shuffle == false* 表示按照正常的顺序初始化 freelist。

  配置了 CONFIG_SLAB_FREELIST_RANDOM 选项时的实现如后面所示。否则直接返回 false.

- 1788-1797 将按正常地址顺序从小到大排列空闲 obj 的 freelist 进行初始化。

  1788 获取 slub 第一个空闲 obj 的真正起始地址。

  slub 可能配置了 SLAB_RED_ZONE，这样会在 slub obj 内存空间两侧填充 red zone 监测内存访问越界。这里需要跳过 red zone 获取真正存放 obj 的内存地址 (或者说是 payload 的部分)。fixup_red_left() 函数见后面。

  1789 填充 obj 的警戒区以及初始化空闲 obj、初始化 slub debug 的跟踪 (track) 区域 (要开对应配置项才会有)。

  1791 从 slub 中的第一个空闲 obj 开始，按照地址从小到大顺序，将 obj 通过 FP(freepoint 指针) 串联起来形成一个 freelist。

  1791-1795 获取下一个 obj 的内存地址, 填充下一个 obj 的内存 debug 区域以及初始化。设置 freepointer 指针指向下一个空闲 obj. 循环遍历把 slub 中的空闲 obj 串联在 freelist 中.

从 1792 行可以看出在新建 slub 中，空闲 obj n 紧挨着空闲 obj (n-1)，而空闲 obj (n-1) 又紧挨着 obj (n-2)..... 直到 obj 1。如果按 freelist 上空闲 obj 按顺序串联，那么就如下图:

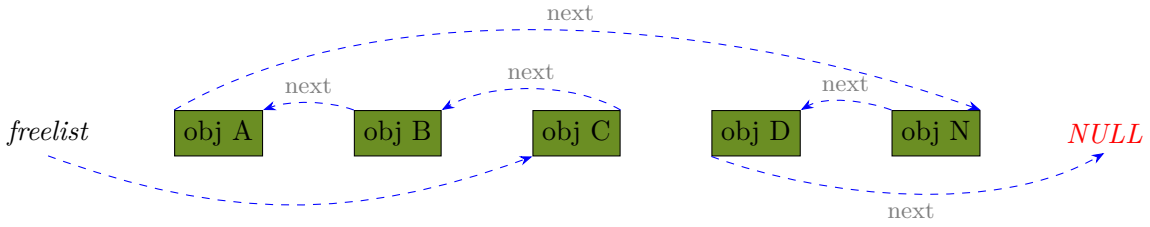


1797 freelist 中的最后一个空闲 obj 的 freepoint(FP 指针) 指向 null。

- 1800-1801 slub 的初始状态 inuse 的值为所有空闲 obj 数目。slub 被创建出来之后需要放入 cpu 本地缓存 kmem_cache_cpu 中，不能给其他 cpu 扫描到使用，所以对外标记为已全部使用。并且 $->frozen = 1$ 来**冻结**。
- 1804-1805 重新打开本地中断，因为返回之后要在无锁场景下操作 per-cpu 变量 kmem_cache_cpu。
- 1809 更新 page 所在 numa 节点缓存 kmem_cache_node 结构中的相关统计计数. kmem_cache_node 中包含的 slub 个数加 1，包含的总 obj 个数加 page->objects。

需要注意的是，新构建的 slub 自身的 freelist 会赋值给 kmem_cache_cpu->freelist，slub 的首页给 kmem_cache_cpu->page，而 kmem_cache_cpu->page->freelist=NULL。

shuffle_freelist() 会将 freelist 中的节点随机排列 (譬如下图示意)。shuffle 的本意就是**洗牌**的意思。



kmem_cache_alloc_node() ··· ··· *new_slab()* *allocate_slab()* *shuffle_freelist()*

```
1689 static bool shuffle_freelist(struct kmem_cache *s, struct page *page)
1690 {
1691         void *start;
1692         void *cur;
1693         void *next;
1694         unsigned long idx, pos, page_limit, freelist_count;
1695
1696         if (page->objects < 2 || !s->random_seq)
1697                 return false;
1698
1699         freelist_count = oo_objects(s->oo);
1700         pos = get_random_int() % freelist_count;
1701
1702         page_limit = page->objects * s->size;
1703         start = fixup_red_left(s, page_address(page));
1704
1705         /* First entry is used as the base of the freelist */
1706         cur = next_freelist_entry(s, page, &pos, start, page_limit,
1707                                   freelist_count);
1708         cur = setup_object(s, page, cur);
1709         page->freelist = cur;
1710
1711         for (idx = 1; idx < page->objects; idx++) {
```

```
1712                    next = next_freelist_entry(s, page, &pos, start, page_limit,
1713                            freelist_count);
1714                    next = setup_object(s, page, next);
1715                    set_freepointer(s, cur, next);
1716                    cur = next;
1717            }
1718            set_freepointer(s, cur, NULL);
1719
1720            return true;
1721    }
```

随机 dreelist 初始化和顺序 freelist 初始化唯一不同的点在于，获取空闲 obj 起始地址的方式不同：顺序 freelist 初始化的方式是直接获取 slub 中第一个空闲 obj 的地址，然后通过 $start + kmem_cache- > size$ 方式顺序一个一个地获取后面 obj 地址。随机初始化则是通过随机的方式获取 slab 中空闲 obj，也就是说 freelist 中的头结点可能是 slub 中的第一个对象，也可能是第三个对象。后续也是通过这种随机的方式来获取每个随机的空闲 obj。

- 1696 slub 实际容纳的 obj 数目 < 2, 或者与计算的全局偏移数组为 NULL, 则不对 freelist 进行随机初始化。
- 1699 获取 slub 理想最优可以容纳的 obj 数目。
- 1700 获取用于随机初始化 freelist 的随机位置。
- 1702 计算得到 slub 中 obj 的总大小 (obj 数目 $\times$ obj 的大小 (包括各填充区))。
- 1703 获取 slub 第一个空闲 obj 的真正起始地址 (payload 的地址) 最为 base 地址，这里的**第一个**指的是按地址从小到顺序有序找到的第一个 obj。slub 可能配置了 SLAB_RED_ZONE, 这样会在 slub 中对象内存空间两侧填充 red zone 来监测内存访问越界。但是 FP 指向的是 obj 的 payload 地址，因此需要跳过 red zone 获取真正存放 obj payload 的内存地址。
- 1706 以 1703 行得到的 base 地址作为基址 start，获取 *s->random_seq[pos]* 里读出的随机偏移值，用 *start* + 随机偏移值 算出第一个随机的 obj 地址并返回。
- 1708 填充 obj 的内存区域以及初始化空闲 obj, 以及可能的 debug 区域。
- 1709 将随机选出的第一个 obj 赋值给 freelist。
- 1711-1717 以 cur 为头结点初始化 freelist（每一个空闲对象都是随机计算选取的），通过循环遍历，把 slub 中的空闲对象随机的串联在 freelist 中.

  1712 next_freelist_entry() 函数中用到的 kmem_cache->random_seq[] 数组是在 kmem_cache_init() 中初始化的。([comlee]: 待补充)
- 1718 slub 中最后一个 obj 的 FP 指向 NULL。

回到 allocate_slab() 中的 alloc_slab_page()，该函数用来分配物理页。

kmem_cache_alloc_node() ··· ··· *new_slab()* *allocate_slab()* *alloc_slab_page()*

```
1608    static inline struct page *alloc_slab_page(struct kmem_cache *s,
1609                    gfp_t flags, int node, struct kmem_cache_order_objects oo)
1610    {
1611            struct page *page;
1612            unsigned int order = oo_order(oo);
1613
```

```
1614         if (node == NUMA_NO_NODE)
1615                 page = alloc_pages(flags, order);
1616         else
1617                 page = __alloc_pages_node(node, flags, order);
1618
1619         if (page)
1620                 account_slab_page(page, order, s);
1621
1622         return page;
1623 }
```

- 1615 alloc_pages 函数用于分配 2^{order} 个连续物理页，order 是向底层伙伴系统申请连续物理页的页阶。该函数返回值是一个 struct page 类型的指针，用于指向申请的内存块中第一个物理内存页的页帧。而 CPU 可以直接访问的却是虚拟内存，所以内核又提供了一个函数 ___get_free_pages ，该函数直接返回物理内存页对应的虚拟地址。用户 (内核空间) 可以直接使用。___get_free_pages 函数在使用方式上和 alloc_pages 是一样的，函数参数的含义也是一样，只不过虚拟地址。事实上 ___get_free_pages 函数的底层也是基于 alloc_pages 实现的，只不过多了一层虚拟地址转换的工作。

  当 $node == NUMA_NO_NODE$, 也就是不限定分配的 NUMA node 时运行 alloc_pages()。

- 1617 当指定了分配的 NUMA node 节点时，运行 ___alloc_pages_node()。
- 1619-1620 给 page 打上 slub 标识、绑定所属 kmem_cache。

allocate_slab() 中通过 kasan_poison_slab() 函数将 slab 中的内存用 0xFC 填充，用于 kasan 对于内存越界相关的检查。

kmem_cache_alloc_node() ··· ··· *new_slab()* *allocate_slab()* *kasan_poison_slab()*

```
1608 void kasan_poison_slab(struct page *page)
1609 {
1610         unsigned long i;
1611
1612         for (i = 0; i < compound_nr(page); i++)
1613                 page_kasan_tag_reset(page + i);
1614         kasan_poison_shadow(page_address(page), page_size(page),
1615                         KASAN_KMALLOC_REDZONE);
1616 }
```

最后会初始化 slab 中的 freelist 链表，将内存页中的空闲内存块通过 page->freelist 链表组织起来。

如果内核开启了 CONFIG_SLAB_FREELIST_RANDOM 选项，那么就会通过 shuffle_freelist 函数将内存页中空闲的内存块按照随机的顺序串联在 page->freelist 中。

如果没有开启，则会在 if (!shuffle) 分支中，按照正常的顺序初始化 page->freelist。

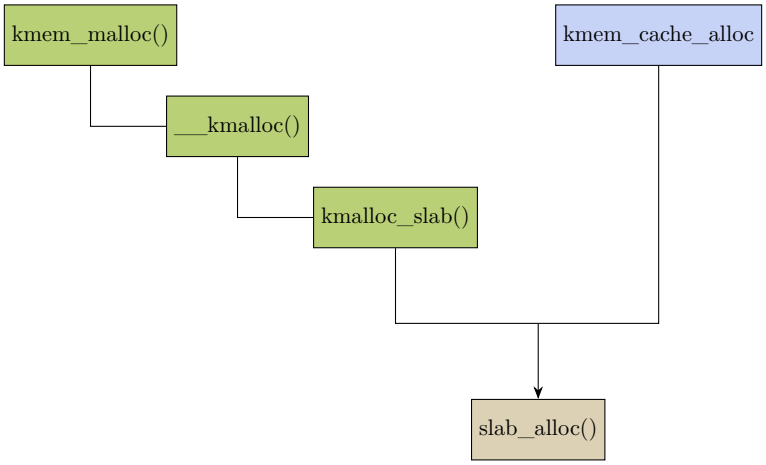
最后通过 inc_slabs_node 更新 NUMA 节点缓存 kmem_cache_node 结构中的相关计数。

2.2 缓存分配的 KPI 入口

通用缓存和专业缓存提供的对外 KPI 的接口不同。通用 kmalloc 缓存不同 slub obj 的描述符统一存放在全局二维数组 ***kmalloc_caches*** 中，分配入口函数是 kmalloc()。该函数的实现中会先通过 kmem_slab() 在全局数组 kmalloc_caches 中查表找到对应 obj 大小的 slub 描述符指针 kmem_cache，再进入底层分配。

专业缓存对外提供的入口函数是 kmem_cache_alloc(), 它会利用 kmem_cache_create() 提前创建好的 slub 描述符指针作为入参，无需查表。

但是无论是通用缓存还是专用缓存，他们的底层分配最终会汇入 slab_alloc() 函数。slub 组件底层不区分是通用缓存还是专用缓存，只操作 slub 描述符指针 struct kmem_cache *, 只是获取这个缓存指针的方式不同。如下图:



2.2.1 通用缓存的分配

当需要分配小块、高频小对象内存时，会使用kmalloc()函数首先从 **slab** 中分配。需要分配整页/多页、物理连续、大块内存时一般使用alloc_pages()函数直接从 ***Buddy***(伙伴系统) 分配。

值得注意的是 kmalloc() 从 slub 分配 obj 存在大小限制，通常是 2 页。在不同 cpu 架构下页大小 (宏 ***PAGE_SHIFT***) 会有所差异，因此这个限制的字节大小值也相应会有差异。在 arm 下，PAGE_SHIFT 是 12(页大小:$2^{12} = 4k$), 因此 slub 可分配的单个 slub obj 最大不超过 8k Bytes。arm64 下 PAGE_SHIFT 则可由内核配置项 *CONFIG_ARM64_PAGE_SHIFT* 来配置。x86、risc–v 架构下页大小则和 arm 相同，最大限制为 8k 字节。

2.2.2 kmalloc()

kmalloc() 的底层实现是 ___kmalloc(), 函数如下:

kmalloc()() ___kmalloc()

```
3953 void *__kmalloc(size_t size, gfp_t flags)
3954 {
3955     struct kmem_cache *s;
3956     void *ret;
3957
3958     if (unlikely(size > KMALLOC_MAX_CACHE_SIZE))
3959         return kmalloc_large(size, flags);
3960
3961     s = kmalloc_slab(size, flags);
3962
3963     if (unlikely(ZERO_OR_NULL_PTR(s)))
3964         return s;
3965
3966     ret = slab_alloc(s, flags, _RET_IP_);
3967
3968     trace_kmalloc(_RET_IP_, ret, size, s->size, flags);
3969
3970     ret = kasan_kmalloc(s, ret, size, flags);
3971
3972     return ret;
```

```
3973 }
3974 EXPORT_SYMBOL(__kmalloc);
```

- 3955 本地临时 slab 缓存描述符结构体指针，指向内核为不同固定大小内存块预先创建的内存池 (slab 缓存)。
- 3958-3959 如果申请的内存大小超出上限，就走大内存路径，从伙伴系统分配。

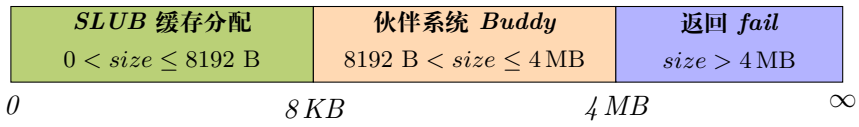
  KMALLOC_MAX_CACHE_SIZE 是 kmalloc 能从 slub 缓存中分配的最大 size 内存的阈值。若申请大小超出该值 (arm 平台下该值大小是 8k 字节)，那么就转而调用子函数 kmalloc_large() 从伙伴系统 (buddy) 直接分配连续物理页。

  进一步的，从 kmalloc_large() 分配连续大内存时，会有最大限制 (参考 ___alloc_pages_nodemask() 函数的源码解析)。当申请分配内存的字节大小换算成页，并且页数量换算成 2 的指数 (order)，这个 order≥MAX_ORDER(2^{11}) 个页时会返回 fail，换言之，order≤(MAX_ORDER-1)(即 $2^{11-1} = 2^{10} = 1k$) 个页时才可以从伙伴系统分配。也就是最大 4M 字节 (1k 个页)。

  总而言之,kmalloc() 单次分配会根据需要分配的内存大小走不同 (以 arm 为例) 的分配路径:

  - $size \leq 8K\ Bytes$ 直接在 slub 缓存中分配内存。即，$(0, 8K]$(单位:Byte)
  - $8K\ Bytes < size \leq 4M\ Bytes$ 从伙伴系统分配。即，$(8K, 4M]$(单位:Byte)
  - $4M < size$, 返回 fail。

  kmalloc() 分配内存的大小和来源示意如下图:



- 3961-3964 查找匹配的 slab 缓存，返回对应内存池的 kmem_cache 指针，后续从该池取内存 (slub obj)。

  kmalloc 在内核中不是任意大小分配，而是预定义一组离散值 (2^n bytes)。因此，函数会根据传入的 size，向上取整到 slab 缓存中预定义的 obj 大小。同时该函数会根据 flags(如 DMA、原子分配) 选择对应类型的 kmem_cache。

  - 3963-3964 如果 kmalloc_slab() 找不到可用 slab 缓存 (系统内存极度紧张、缓存创建失败等) 或申请的内存长度非法 (为 0)，则直接返回。

    这里会判断返回的是否 NULL 指针和 ZERO 指针 (详见后面 kmalloc_slab() 函数的讨论)。
- 3966 从目标 kmem_cache 内存池中取出一个空闲 slub 对象 (obj)。该函数专用缓存也会调用，是专用缓存和通用缓存的汇聚点，后面有详细解析。
- 3968 内核 tracepoint 跟踪点函数. 记录本次 kmalloc 行为：调用地址、分配地址、请求大小、slab 实际对象大小、GFP 标志。

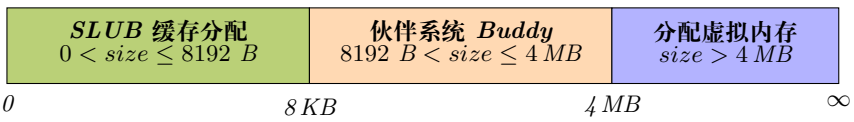
```
mount -t debugfs debugfs /sys/kernel/debug
# 启用 kmalloc tracepoint 追踪
echo 1 > /sys/kernel/debug/tracing/events/kmem/kmalloc/enable
```

默认未开启 tracepoint 时，trace_kmalloc 会被编译成 nop 空指令，无运行时开销；需要手动开启编译配置 + 运行时开关才会真正执行逻辑。它的原型是由 TRACE_EVENT 宏展开生成。内核未开启 tracepoint 功能时，整个 tracepoint 基础设施被裁减掉。但是即使开启了 tracepoint 内核配置项，在运行时该功能仍然默认是关闭的。通过 /sys/kernel/debug/tracing/ 接口启用 kmalloc 跟踪 (如上图所示)。

NOTE

kvmalloc 融合了 *kmalloc* 和 *vmalloc* 的能力，优先物理连续分配，失败则改用虚拟连续、物理不连续内存进行分配。

要分配的内存 $\leq 8K$ *bytes* 时，*kvmalloc()* 会从 *slub* 中分配。
要分配的内存 $\leq 4M$ *bytes* 并且 $> 8K$ *bytes* 时，*kvmalloc()* 会从伙伴系统分配。
要分配的内存 $> 4M$ *bytes* 时，*kvmalloc()* 会调用 *vmalloc* 函数族函数分配物理不连续的内存。



---

kmalloc()() → ___kmalloc() → kmalloc_slab()

```
633 struct kmem_cache *kmalloc_slab(size_t size, gfp_t flags)
634 {
635     unsigned int index;
636 
637     if (size <= 192) {
638         if (!size)
639             return ZERO_SIZE_PTR;
640 
641         index = size_index[size_index_elem(size)];
642     } else {
643         if (WARN_ON_ONCE(size > KMALLOC_MAX_CACHE_SIZE))
644             return NULL;
645         index = fls(size - 1);
646     }
647 
648     return kmalloc_caches[kmalloc_type(flags)][index];
649 }
```

- 637 从前面讨论已经得知:< 192字节 的细粒度的 kmalloc 缓存档位不全是 2 的 order 次幂 (2^{order}) 字节大小。因此无法用位运算快速计算在 kmalloc_caches[type][i] 二维数组中的索引 i。于是系统提前建好静态索引，通过查表直接找到缓存索引。

- 638-639 如果申请的缓存大小是 0，返回指向地址为 ((void *)16) 的指针 (*ZERO_SIZE_PTR*)。之所以不直接返回 NULL 指针是为了区分两种不同的错误场景。返回 指针 = *NULL* 表示内存不足，分配失败。返回 指针 = 16 表示系统申请 0 字节内存。

  无论是 0 还是 16，对系统来说都是非法地址，解引用时会导致出错。将这两种错误区分开，可以在分析时通过指针值快速看出是代码逻辑错误 (申请了 0 字节内存去读写)，还是分配失败未做空指针检测。

- 641 查表, 返回申请的目标大小缓存对应的 slub 描述符指针在全局数组 kmalloc_caches[type][i] 中的索引 i。

  之前已经讨论过，默认 kmalloc 分配的最小的原子单位是 8 字节 (若申请的内存 <8 字节，则返回 8 字节给申请者)。所以此处索引表将 [8, 192] 范围的字节，每隔 8 字节建一个索引。以 kmalloc 分配的最小原子单位 8 字节对应的索引作为表格第一个成员。如下，右侧注释中的数字为要申请的内存大小, 左侧则是它对应的 slub 描述符在全局数组 kmalloc_caches[type][i] 中的索引 i。

```
static u8 size_index[24] __ro_after_init = {
    3,  /* 8 */
    4,  /* 16 */
    5,  /* 24 */
    5,  /* 32 */
    6,  /* 40 */
    6,  /* 48 */
    6,  /* 56 */
    6,  /* 64 */
    1,  /* 72 */
    1,  /* 80 */
    1,  /* 88 */
    1,  /* 96 */
    7,  /* 104 */
    7,  /* 112 */
    7,  /* 120 */
    7,  /* 128 */
    2,  /* 136 */
    2,  /* 144 */
    2,  /* 152 */
    2,  /* 160 */
    2,  /* 168 */
    2,  /* 176 */
    2,  /* 184 */
    2   /* 192 */
};
```

- 642 如果申请的 kmalloc 通用内存 > 192字节, 并且 > $KMALLOC_MAX_CACHE_SIZE$。那么超出通用内存的分配上限，返回 NULL。

- 643-644 *KMALLOC_MAX_CACHE_SIZE* 在 arm 平台 (*CONFIG_SLUB* 开启情况下) 一般是 13，也就是说 kmalloc 的分配上限是 *8k* 字节。

- 645 返回 kmalloc 大内存分配对应的描述符索引。将 *(size - 1)* 看成 x, 那么此处等价于 $index = i\log_2(x) + 1$。此处 +1 是为了向后对齐到最近的 2 的次幂。

譬如，kmalloc 申请 250 字节 ($2^7 < 250 < 2^8$) 的内存，那么会返回给申请者 256 字节, 对应:

$$\begin{aligned} index &= i\log_2(250) + 1 \\ &= 7 + 1 \\ &= 8 \end{aligned}$$

- 648 返回对应类型和大小的 slub 描述符指针。

2.2.3 专用缓存的分配

专用缓存的分配入口是 kmem_cache_alloc()。该函数直接就是 slab_alloc() 函数的包裹函数。slab_alloc 详见后面解析。

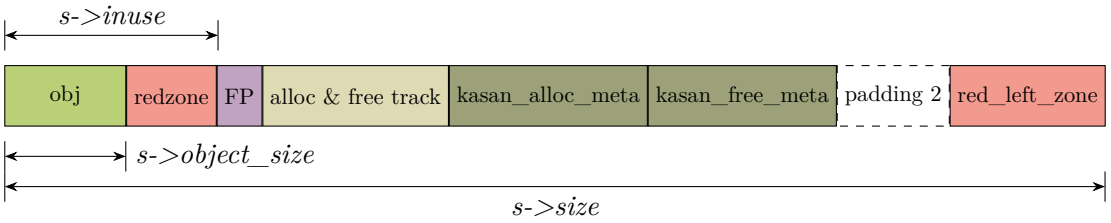
3

SLUG debug

SLUB debug 就是利用特殊区域填充特殊的 magic num(魔术数字、魔法数字)，在每一次 alloc/free 的时候检查 magic num 是否被意外修改。之所以这些填充的数值被称为**魔术数字**、**魔法数字**是因为: 在填充这些数值后，就像使用了**魔法**一样可以快速的发现被不当操作的内存区域。

3.1 slub obj 中与 debug 相关的区域

假设内核中 slub debug 等选项都打开, 此时 slub obj 如前所示:



- **red_left_pad**

  red_left_pad 和一般右侧警戒区的作用一致。都是为了检测 oob。区别就是警戒区检测右侧 oob，而 red_left_pad 是检测左侧 oob。看上面图片的 obj layout。可能会疑问，如果发生 left oob，那么应该是前一个 obj 的 red_left_pad 区域被踩，而不是当前 obj 的 red_left_pad。为了避免这种情况的发生，SLUB allocator 在初始化 slab 缓存池的时候会做一个转换。此时 layout 如下:



在 struct page 结构中有一个 freelist 指针，freelist 会指向第一个空闲的 obj。在构建 object 之间的单链表的时候，obj 首地址加上一个 red_left_pad 的偏移才是实际 obj 的内容的地址，这样实际的 layout 就如同图片中转换之后的 layout。之所以如此是因为在有 SLUB DEBUG 功能的时候，并没有检测 left oob 功能。这种转换是后续一个补丁的修改。补丁就是为了增加 left oob 检测功能。做了转换之后的 red_left_pad 就可以检测 left oob。所以 page 结构体的 freelist 包括每个 slub obj 中的 FP 都指向 slab obj 偏移 s->red_left_pad 处.(此处假设没有置 SLAB_POISON 等标志, 并且 object_size 也不大于一个 word, 否则 FP 应该对应在 obj payload 的外部或在 obj 起始地址)

page 指向构成 slab 的第一个物理页.

- **redzone**

  在 object 后面紧接着就是警戒区 (redzone), "slub_debug=Z" 使能时该区域会存在. 在 kmem_cache 结构体里没有成员显示的指示该区域有多大. 如果 kmem_cache->object_size 正好对齐到 sizeof(void *), 那么该区域大小是 sizeof(void *). 否则, 这个区域按实际情况有可能借用 padding 部分 (区别起见, 后面的 padding 区标准成 padding2).

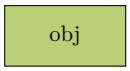
  警戒区的作用是检测右边界越界访问（OOB）。原理是: 在警戒区填充 magic num，检查警戒区数据是否被修改即可知道是否发生右侧 OOB。但是如果越过警戒区，直接改写了 FP，岂不是检测不到 oob 了? 其实在 check_object() 函数中会调用 check_valid_pointer() 来检查 FP 是否 valid，如果 invalid，同样会报错。

- **padding2 区**

  padding 是 sizeof(void *) 的填充区域，在分配 slab cache 时，会将所有的内存填充 0x5a。同样在 free/alloc object 的时候作为检测的一种途径。如果 padding 区域的数据不是 0x5a，就代表发生了 "Object padding overwritten" 问题。越界跨度很大时这也是有可能的。

- **obj payload**

  该区域用于 obj 保存实际的 payload 数据, 而且相对于其他区域而言始终存在. 如果没有打开任何诸如 SLUB_DEBUG,SLAB_POISON,KASAN 之类的选项, 那么 slub object 就会保持最纯粹状态只由该区域组成. 它的大小是 kmem_cache->object_size.

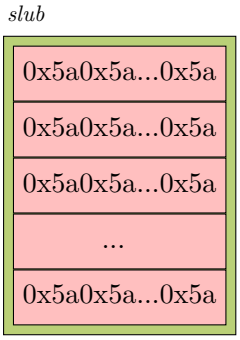


- **meta 数据区**

  这个区域的内容取决于各种 debug 选项是否打开. 它可能包括以下一个或多个信息区域:FP, track 信息, Kasan 分配信息.

3.2 slub 分配时内存的初始填充值

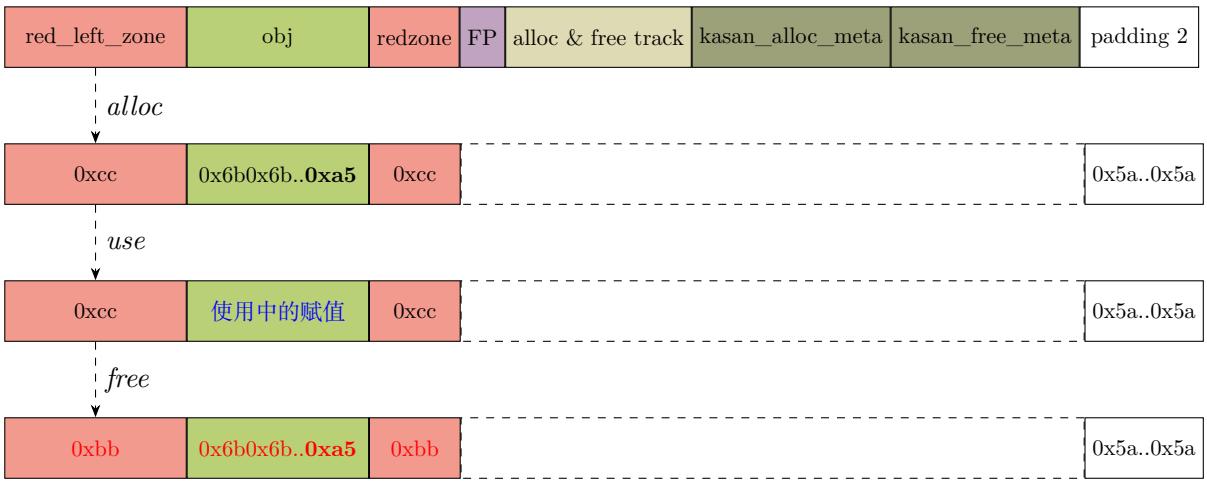
开启 SLUB debug 机制后，在申请一块内存作为 slub 缓存池时，会给整块内存池填充值 POISON_INUSE(0x5a) 以用于 debug。如图 (图中的每一行表示一个 obj)：



3.3 slub obj 使用过程中填充数据的变化

3.3.1 分配 slub obj

在打开 SLUB debug 机制后，obj 中各区域内存填充值在不同的使用阶段会有不同值，示意如下：



从 slub 中申请 (alloc) 到一个 slub obj 时会自动初始化物理内存中的该 obj，内核会给 obj 区域填充 0x6b, 并且用 0x5b 填充 slub obj 的 object size 大小内存区域的最后一个字节，表示填充结束。

- 通过判断图中左 (red_left_zone) 右 (redzone)redzone 可以确认是否存在 obj 左右越界的情况。

  左侧越界时，red_left_zone 中填充值会被破坏，不再是填充值 0xcc; 右侧越界相似。

- 通过”obj + redzone” 可以判断如下情况：

  未初始化：此时 redzone 中填充值为 0xcc, 而 obj 中值为 0x6b0x6b...0x5a

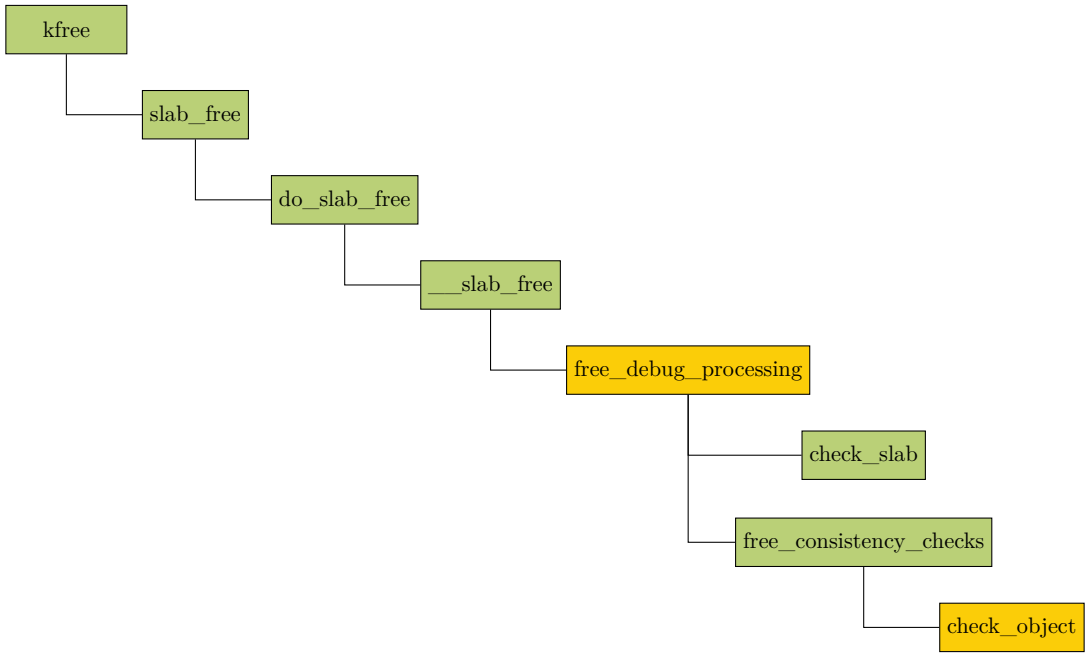
  free 后使用 (UAF): 此时 redzone 中填充值为 0xbb, 而 obj 中值为 0x6b0x6b...0x5a

3.4 SLUB debug 的检测点

在填充字节后，内核会在内存分配/释放的时候进行内存完备性检测。

3.4.1 内存释放时篡改检测

kfree() 在 free_consistency_checks() 函数中调用 check_object() 进行检测。流程图如下:



========= 释放过程中检测后一些魔术数字会重新置值，以便下次 alloc 时检测是否有 UAF 的错误场景。这段代码待分析

====new_slab->alloc_slab->...setup_object->init_object 会在第一次分配 slub 分隔 obj 时填充魔术数字，这样不影响 alloc 时的检测，待添加这段分析

其中 free_debug_processing() 函数在 slub obj 被释放回 slab cache 时，会执行一系列一致性检查，确保释放过程的正确性。代码如下：

mm/slub.c :: free_debug_processing()

```
1205 static noinline int free_debug_processing(
1206     struct kmem_cache *s, struct page *page,
1207     void *head, void *tail, int bulk_cnt,
1208     unsigned long addr)
1209 {
1210     struct kmem_cache_node *n = get_node(s, page_to_nid(page));
1211     void *object = head;
1212     int cnt = 0;
1213     unsigned long flags;
1214     int ret = 0;
1215
1216     spin_lock_irqsave(&n->list_lock, flags);
1217     slab_lock(page);
1218
1219     if (s->flags & SLAB_CONSISTENCY_CHECKS) {
1220         if (!check_slab(s, page))
1221             goto out;
1222     }
1223
1224 next_object:
1225     cnt++;
1226
1227     if (s->flags & SLAB_CONSISTENCY_CHECKS) {
1228         if (!free_consistency_checks(s, page, object, addr))
1229             goto out;
1230     }
1231
1232     if (s->flags & SLAB_STORE_USER)
1233         set_track(s, object, TRACK_FREE, addr);
1234     trace(s, page, object, 0);
```

```
1235     /* Freepointer not overwritten by init_object(), SLAB_POISON moved it */
1236     init_object(s, object, SLUB_RED_INACTIVE);
1237
1238     /* Reached end of constructed freelist yet? */
1239     if (object != tail) {
1240         object = get_freepointer(s, object);
1241         goto next_object;
1242     }
1243     ret = 1;
1244
1245 out:
1246     if (cnt != bulk_cnt)
1247         slab_err(s, page, "Bulk freelist count(%d) invalid(%d)\n",
1248              bulk_cnt, cnt);
1249
1250     slab_unlock(page);
1251     spin_unlock_irqrestore(&n->list_lock, flags);
1252     if (!ret)
1253         slab_fix(s, "Object at 0x%p not freed", object);
1254     return ret;
1255 }
```

- 1219-1222 如果启用了一致性检查 (SLAB_CONSISTENCY_CHECKS 标志), 则调用 check_slab() 检查整个 slab 的状态.check_slab() 见后面.
- 1228 接下来会 free_consistency_checks() 继续 slub obj 也就是 obj 级别的检查。检查 obj 是否被破坏 (溢出等)。
- 1232-1233 如果启用了对内存使用者的跟踪（SLAB_STORE_USER 标志), 则记录 obj 释放操作的调用者的地址。set_track() 见后面。
- 1236 根据 obj 的状态 SLUB_RED_INACTIVE，重置释放的 obj 的属性和内存填充区。其中会将警戒区设置成 SLUB_RED_INACTIVE(0xbb)。init_object() 见后。

  SLUB debug 机制会在 free 时对 obj 的警戒区 (red zone) 检测，判断该区域值是否都为 SLUB_RED_ACTIVE(0xcc)。若否，则会打印报错日志。检测完成后重置警戒区的填充值为 SLUB_RED_INACTIVE(0xbb)。而 0xbb 在 alloc 检测时会进行检测，也就是 alloc 时会判断目标 slab obj 在此之前是否处于不活跃状态，且内存没有被破坏（UAF 错误等）。
- 1239-1242 遍历整个 obj 链表, 重复 1236 行开始的处理，直到遍历完。
- 1246-1248 检查实际处理的 obj 数是否与预期（bulk_cnt）一致, 不一致返回报错。

1220 行调用的 check_slab() 如下：

free_debug_processing() ▸ *check_slab()*

```
947 static int check_slab(struct kmem_cache *s, struct page *page)
948 {
949     int maxobj;
950
951     VM_BUG_ON(!irqs_disabled());
952
953     if (!PageSlab(page)) {
954         slab_err(s, page, "Not a valid slab page");
955         return 0;
956     }
```

```
957
958     maxobj = order_objects(compound_order(page), s->size);
959     if (page->objects > maxobj) {
960         slab_err(s, page, "objects %u > max %u",
961             page->objects, maxobj);
962         return 0;
963     }
964     if (page->inuse > page->objects) {
965         slab_err(s, page, "inuse %u > max %u",
966             page->inuse, page->objects);
967         return 0;
968     }
969     /* Slab_pad_check fixes things up after itself */
970     slab_pad_check(s, page);
971     return 1;
972 }
```

- 953-956 检查页面是否确实是 slab 页面, 如果否，则无需处理。
- 958 获取页面的阶数 (可能是复合页), 根据页面阶数和 obj 大小，计算一页理论上最多能容纳 obj 的数目。
- 959-963 判断实际记录的 obj 数量是否超过理论最大值。若超过，说明元数据被破坏，报错退出。
- 964-968 已分配（正在使用）的 obj 数是否超过总 obj 数。若超过，说明内存管理状态不一致，返回。
- 970 检查 slab 页面的填充字节。

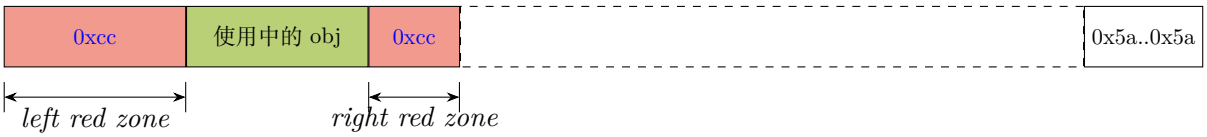
实际执行 slub obj 检测的函数 check_objects() 如下：

free_debug_processing() > *check_slab()* > *check_object()*

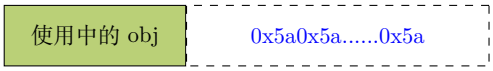
```
891 static int check_object(struct kmem_cache *s, struct page *page,
892                     void *object, u8 val)
893 {
894     u8 *p = object;
895     u8 *endobject = object + s->object_size;
896
897     if (s->flags & SLAB_RED_ZONE) {
898         if (!check_bytes_and_report(s, page, object, "Redzone",
899             object - s->red_left_pad, val, s->red_left_pad))
900             return 0;
901
902         if (!check_bytes_and_report(s, page, object, "Redzone",
903             endobject, val, s->inuse - s->object_size))
904             return 0;
905     } else {
906         if ((s->flags & SLAB_POISON) && s->object_size < s->inuse) {
907             check_bytes_and_report(s, page, p, "Alignment padding",
908                 endobject, POISON_INUSE,
909                 s->inuse - s->object_size);
910         }
911     }
912
913     if (s->flags & SLAB_POISON) {
914         if (val != SLUB_RED_ACTIVE && (s->flags & __OBJECT_POISON) &&
915             (!check_bytes_and_report(s, page, p, "Poison", p,
916                     POISON_FREE, s->object_size - 1) ||
917              !check_bytes_and_report(s, page, p, "Poison",
918                 p + s->object_size - 1, POISON_END, 1)))
919             return 0;
```

```
920         /*
921          * check_pad_bytes cleans up on its own.
922          */
923         check_pad_bytes(s, page, p);
924     }
925 
926     if (!freeptr_outside_object(s) && val == SLUB_RED_ACTIVE)
927         /*
928          * Object and freepointer overlap. Cannot check
929          * freepointer while object is allocated.
930          */
931         return 1;
932 
933     /* Check free pointer validity */
934     if (!check_valid_pointer(s, page, get_freepointer(s, p))) {
935         object_err(s, page, p, "Freepointer corrupt");
936         /*
937          * No choice but to zap it and thus lose the remainder
938          * of the free objects in this slab. May cause
939          * another error because the object count is now wrong.
940          */
941         set_freepointer(s, p, NULL);
942         return 0;
943     }
944     return 1;
945 }
```

- 897 判断 obj 缓存是否启用了警戒区。SLAB_RED_ZONE 标志只在内核配置了 CONFIG_DEBUG_SLAB 选项后才有效。
- 898-904 检测 slab obj 的左警戒区 ($object - red_left_pad$) 和右警戒区的填充值是否为 SLUB_RED_ACTIVE(0xcc)，也就是验证下图蓝色部分字节是否被篡改。

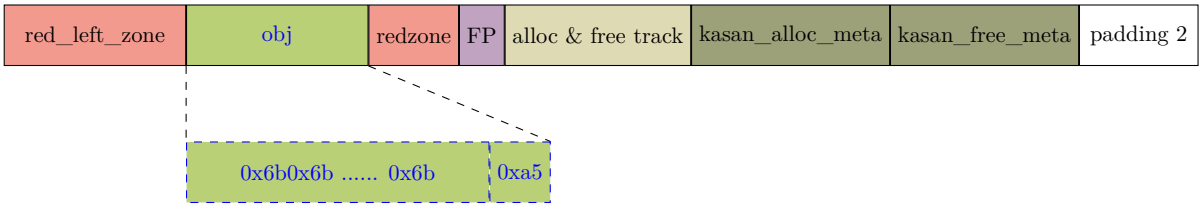


- 906-910 如果没有启用警戒区，但是启用了 slab 污化 (SLAB_POISON), 那么在 obj 的结束地址开始检测填充的值是否 POISON_INUSE(0x5a)，下图蓝色部分。



- 913-919 在启用了 slab 污化 (SLAB_POISON) 和 slab obj 污化 (___OBJECT_POISON) 功能的前提下，检测 obj 对象内存是否被非法覆写。

  检查从开始到 $object_size - 1$ 的字节是否为 POISON_FREE(0x6b)。检查最后一个字节是否为 POISON_END()0xa5), 下图蓝色字体区域。



需要注意的是，入参 val 为 SLUB_RED_ACTIVE 时 (即 obj 处于已分配状态)。**不进入** 913-919 行检测 obj 污化的代码块。因为此时待检测的 slab obj 未完成释放，obj 里有使用的脏数据，此时无法检测填充数据。

在 obj 处于分配状态调用 check_object() 函数的地方，在内核里有两处：

- free 内存时，此时会传递 SLUB_RED_ACTIVE 给 check_object()。
- 应用层调用 *slabinfo -v* 主动触发 slab 检测，在检测 active freelist 时会传递 SLUB_RED_ACTIVE，此时 active freelist 链表上的 obj 处于已分配状态。

- 923 检测填充区，见后面 check_pad_bytes() 函数。
- 926-931 若空闲指针 FP 嵌在 obj 内部（非独立存储），且 slab obj 处于已分配状态，跳过空闲指针检测 (避免误判)。
- 934-943 获取 obj 内嵌的 下一个空闲 obj 地址 (指针)，并对该指针 FP 进行有效性检测。如果 FP 指针非法，报错并清空指针 ，防止后续操作链式崩溃。

923 行调用的 check_pad_bytes() 函数:

free_debug_processing() > *check_slab()* > *check_pad_bytes()*

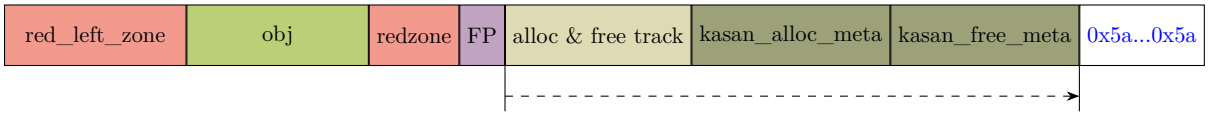
```
837 static int check_pad_bytes(struct kmem_cache *s, struct page *page, u8 *p)
838 {
839     unsigned long off = get_info_end(s);    /* The end of info */
840
841     if (s->flags & SLAB_STORE_USER)
842         /* We also have user information there */
843         off += 2 * sizeof(struct track);
844
845     off += kasan_metadata_size(s);
846
847     if (size_from_object(s) == off)
848         return 1;
849
850     return check_bytes_and_report(s, page, p, "Object padding",
851             p + off, POISON_INUSE, size_from_object(s) - off);
852 }
```

- 839 获取 obj **元数据**距离起始的偏移量。如果 FP 指针在 obj 外部存储会比嵌在 obj 内部多一个字节的 size 大小。
- 841-843 如果启用了 SLAB_STORE_USER, 要加上 $2 * sizeof(structtrack)$。因为启用该 flag 后会为 alloc 和 free slab 内存的跟踪信息 预留空间。这些信息包括调用地址、调用栈、cpu core、进程 ID、操作时间等。alloc 和 free 各自占用一个 struct track 大小。结构体如下:

```
struct track {
    unsigned long addr; /* Called from address */
#ifdef CONFIG_STACKTRACE
    unsigned long addrs[TRACK_ADDRS_COUNT]; /* Called from address */
#endif
    int cpu;        /* Was running on cpu */
    int pid;        /* Pid context */
    unsigned long when; /* When did the operation occur */
};
```

- 845 增加 KASAN 用于跟踪 alloc 和 free 信息的元数据长度

- 847-848 实际分配的总大小 (含元数据) 与 off 相等，说明 无填充区 ，直接返回。
- 如果有填充区，则当前 off 偏移的位置就是填充区的起始位置。检测填充区数据是否被破坏。下图蓝色字体是要检测的区域和期望的值，箭头显示了计算偏移量时的移动计算过程。



回到 check_object()。该函数 898,907,915 行调用的 check_bytes_and_report() 如下:

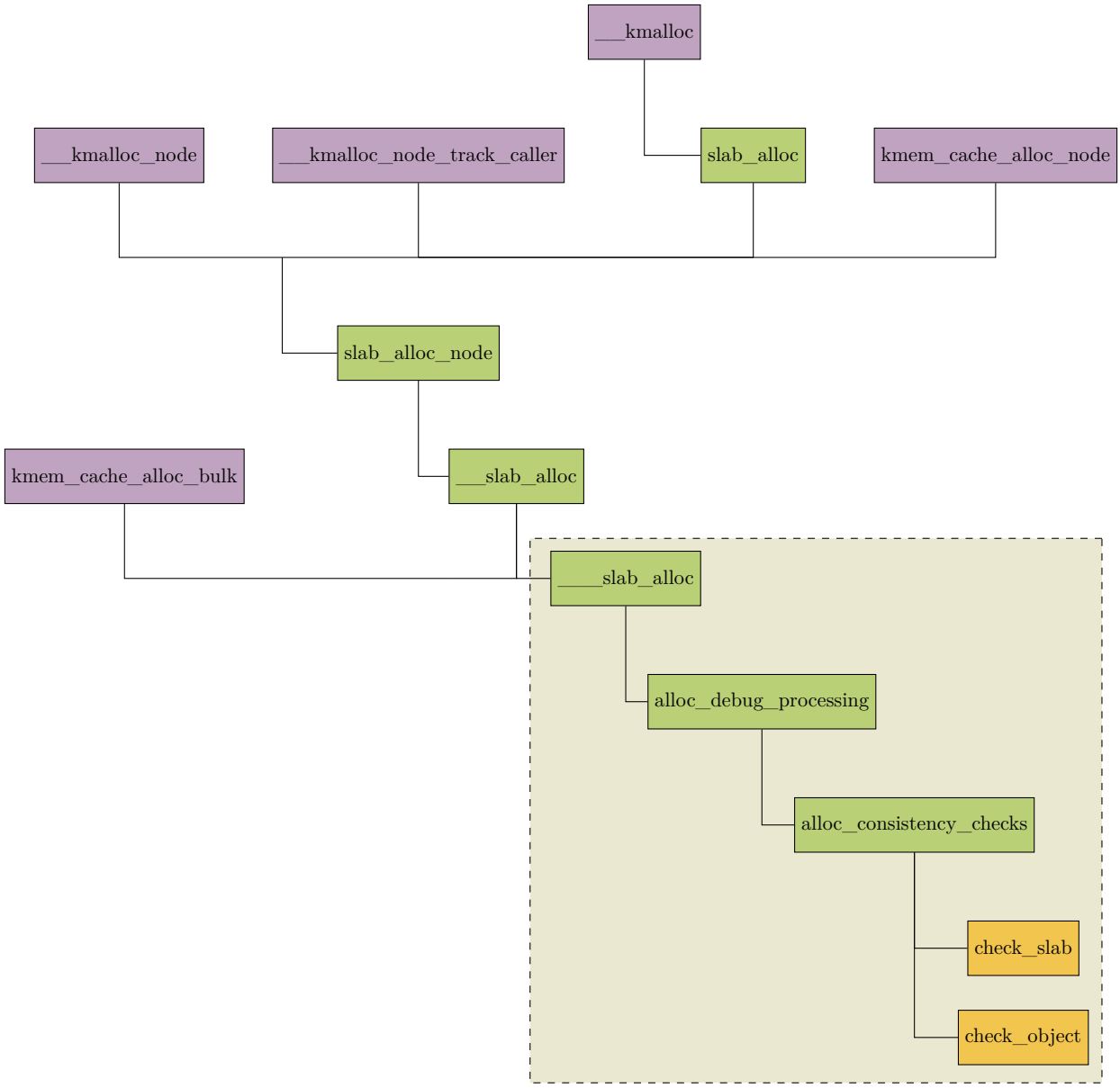
free_debug_processing() > *check_slab()* > *check_bytes_and_report()*

```
771 static int check_bytes_and_report(struct kmem_cache *s, struct page *page,
772             u8 *object, char *what,
773             u8 *start, unsigned int value, unsigned int bytes)
774 {
775     u8 *fault;
776     u8 *end;
777     u8 *addr = page_address(page);
778 
779     metadata_access_enable();
780     fault = memchr_inv(start, value, bytes);
781     metadata_access_disable();
782     if (!fault)
783         return 1;
784 
785     end = start + bytes;
786     while (end > fault && end[-1] == value)
787         end--;
788 
789     slab_bug(s, "%s overwritten", what);
790     pr_err("INFO: 0x%p-0x%p @offset=%tu. First byte 0x%x instead of 0x%x\n",
791                     fault, end - 1, fault - addr,
792                     fault[0], value);
793     print_trailer(s, page, object);
794 
795     restore_bytes(s, what, value, fault, end);
796     return 0;
797 }
```

- 780 查找一段内存区域中首个和目标字节**不匹配**的字节，并返回不匹配字节的地址。memchr_inv() 是 memchr() 函数的反向操作，后者是查找匹配字节，所以此处的”_inv” 代表反向操作，而不是从内存尾地址向前**逆向**查找。
- 782-783 如果 780 行检测结果显示并无区域被篡改，则退出。
- 785 计算待检测目标内存区域的结束地址。
- 786-787 从目标内存区域的末尾向前扫描，遇到第一个值不等于 value 的字节或到达 fault 位置时终止。循环结束后，[fault, end] 即是被破坏或篡改区域的起止区间。
- 795 将被篡改区域恢复成正确的检测用的填充字符。

3.5 分配时的内存篡改检测

内存分配的各函数调用 ____slab_slab() 时 (见下图)，会调用到 alloc_consistency_checks()。



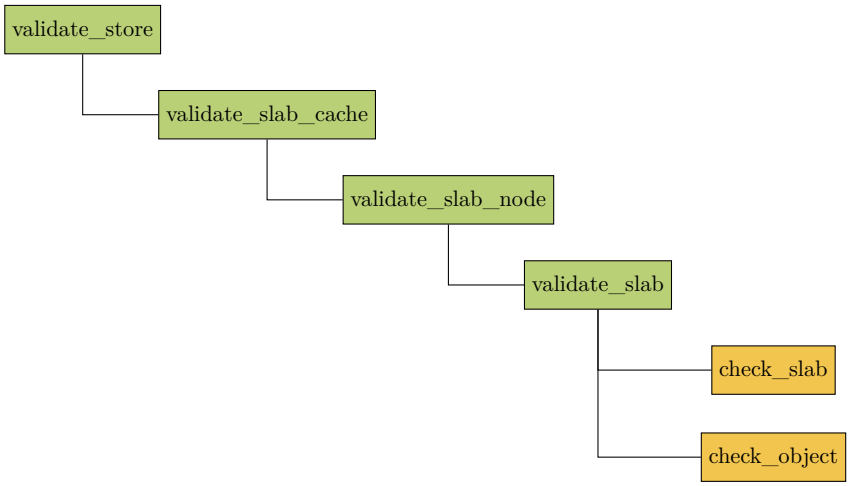
函数 alloc_consistency_checks() 的执行流程和 free 时调用的 free_consistency_checks() 函数类似，最终会调用 alloc_consistency_checks->check_slab->slab_pad_check() 对 slub 的填充区进行字段检测，另外也会通过 check_object 对 slub obj 及警戒区的污化字段检测。不同的是，此时对警戒区检测的字段是 SLUB_RED_INACTIVE(0xbb)。

3.6 slabinfo 工具

由上可知 SLUB debug 发生在 alloc 和 free slub obj 时。但是如果越界踩内存，或 UAF(free 内存后又使用这块内存) 这些不当操作已经发生，那么在未调用到 alloc/free(这里指内核分配和释放 slub 的函数，不是字面上的函数名称) 时，不会对目标内存触发检测。换言之，这些检测是被动进行的。

幸运的是我们可以在应用空间通过调用 *slabinfo -v* 工具主动触发 SLUB debug 检测。等效操作是向*/sys/kernel/slab/<**slab name**>/validate* 节点写"1" 触发 slab 检测功能。

该节点对应的内核写函数是 validate_store()。对节点置 1 后最终会调用对填充区的检测函数 check_slab(), 以及对 obj, 和 obj 警戒区的检测函数 check_object()。示意流程如下：


validate_store
validate_slab_cache
validate_slab_node
validate_slab
check_slab
check_object

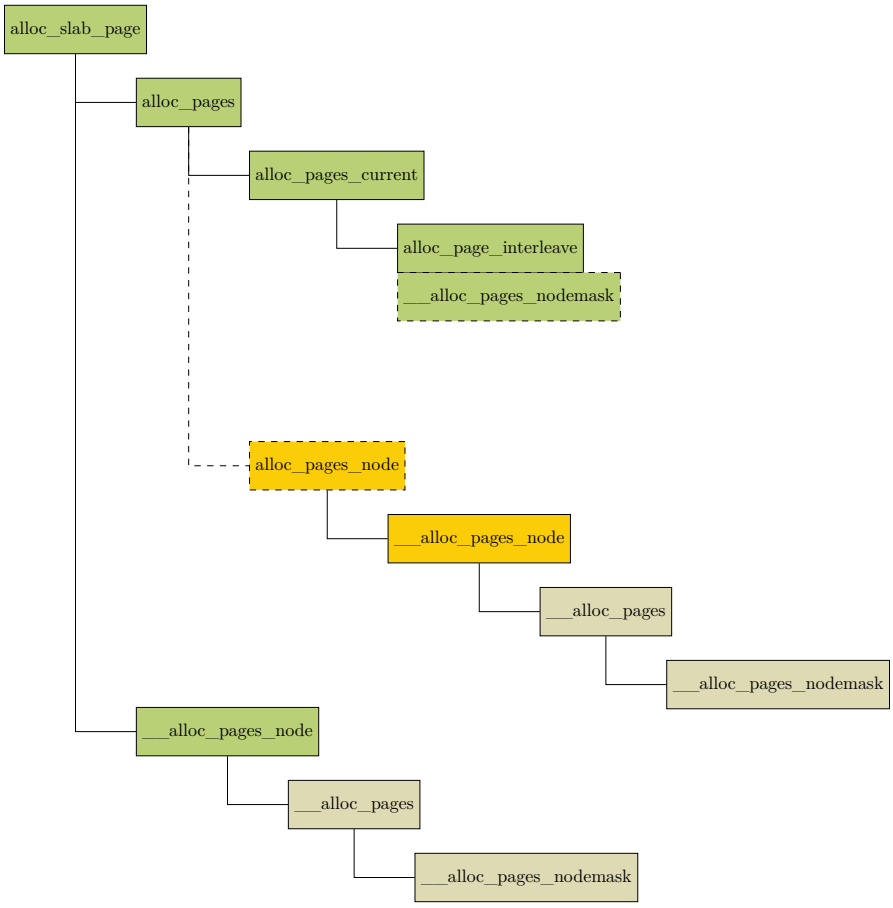
4

页分配

前面 2.1.2章节提到，如果本地和远端 NUMA 节点中的空闲内存都已经不足，或者由于内存碎片的原因导致伙伴系统无法满足 slab 所需要的内存页个数，导致 alloc_slab_page() 分配失败。那么内核会降级采用 kmem_cache->min 指定的尺寸，向伙伴系统申请只容纳一个对象所需要的最小内存页个数。

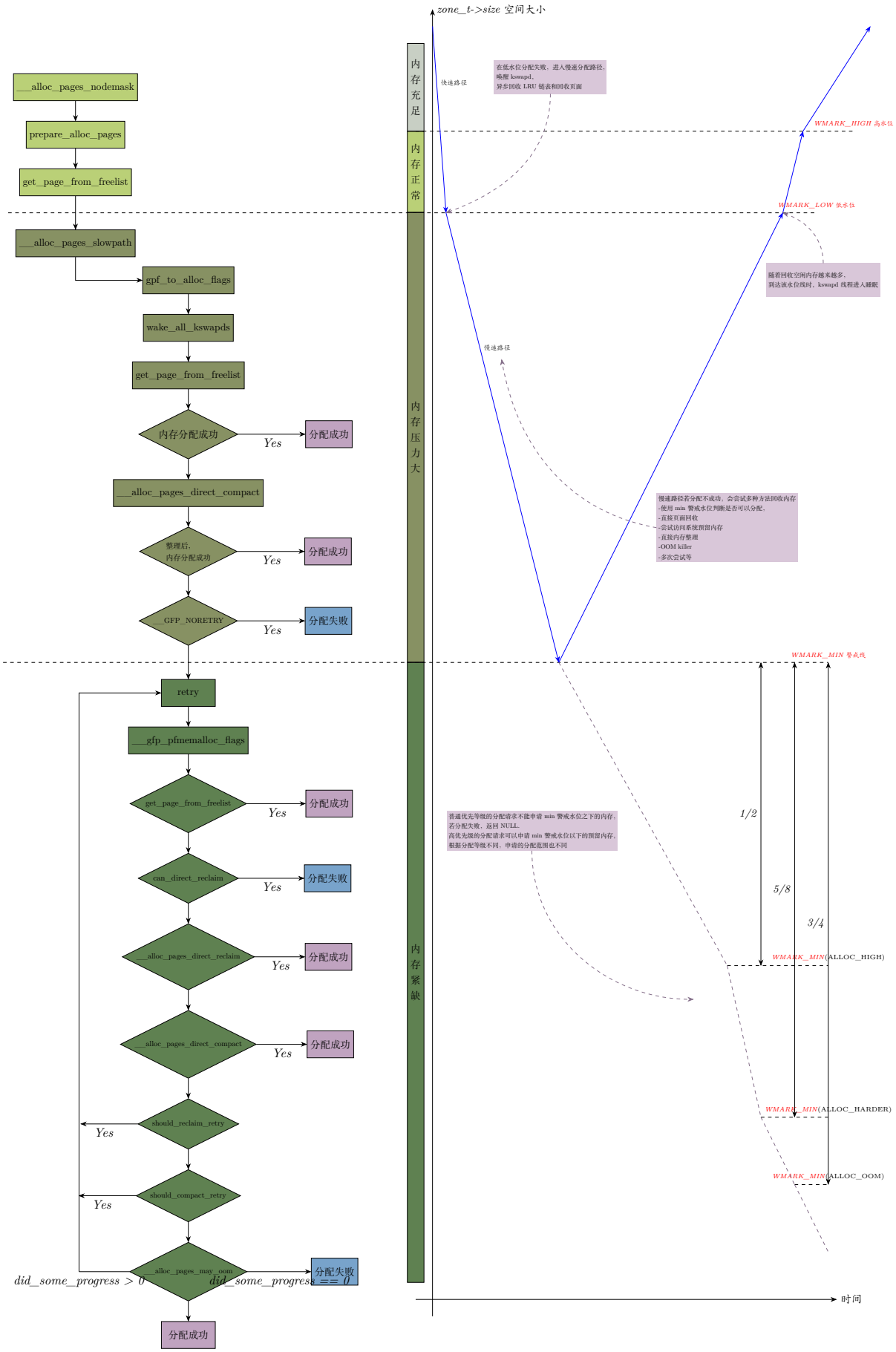
如果成功地向伙伴系统申请到了 slab 所需要的内存页 page。紧接着就会初始化 page 结构中与 slab 相关的属性。

基本所有的内核内存分配 KPI 最终都会调用 2.1.2的 alloc_slab_page() 来完成内存的分配。该函数会根据入参 *node* 的不同来确定走两条不同的分配路径。如果要由内核任意选择可用 NUMA 节点分配则执行 alloc_pages()。如果要限定从某个 NUMA 节点上分配内存，则走 ___alloc_pages_node() 的执行路径。但是实际上 alloc_pages() 在底层会最终调用 ___alloc_pages_node()。如下图:



图中虚线代表条件编译的部分函数。

因为最底层的调用都是 ___alloc_pages_node()，所以我们就从它的代码开始进行解析。此函数的调用流程以及水位线示意图如下：


__alloc_pages_nodemask
prepare_alloc_pages
get_page_from_freelist
__alloc_pages_slowpath
gpf_to_alloc_flags
wake_all_kswapds
get_page_from_freelist
内存分配成功
Yes
分配成功
__alloc_pages_direct_compact
整理后，
内存分配成功
Yes
分配成功
__GFP_NORETRY
Yes
分配失败
retry
__gfp_pfmemalloc_flags
get_page_from_freelist
Yes
分配成功
can_direct_reclaim
Yes
分配失败
__alloc_pages_direct_reclaim
Yes
分配成功
__alloc_pages_direct_compact
Yes
分配成功
should_reclaim_retry
Yes
should_compact_retry
Yes
__alloc_pages_may_oom
did_some_progress > 0
did_some_progress == 0
分配失败
分配成功
zone_t->size 空间大小
内存充足
内存正常
内存压力大
内存紧缺
快速路径
在低水位分配失败，进入慢速分配路径，
唤醒 kswapd，
异步回收 LRU 链表和回收页面
WMARK_HIGH 高水位
WMARK_LOW 低水位
随着回收空闲内存越来越多，
到达该水位线时，kswapd 线程进入睡眠
慢速路径
慢速路径若分配不成功，会尝试多种方法回收内存
-使用 min 警戒水位判断是否可以分配，
-直接页面回收
-尝试访问系统预留内存
-直接内存整理
-OOM killer
-多次尝试等
WMARK_MIN 警戒线
普通优先等级的分配请求不能申请 min 警戒水位之下的内存，
若分配失败，返回 NULL.
高优先级的分配请求可以申请 min 警戒水位以下的预留内存，
根据分配等级不同，申请的分配范围也不同
1/2
5/8
3/4
WMARK_MIN(ALLOC_HIGH)
WMARK_MIN(ALLOC_HARDER)
WMARK_MIN(ALLOC_OOM)
时间

内核会给 NUMA 的每个内存 node 节点的每个内存 zone 区域定义一个对应的 _watermark[NRWMARK] 数组，其中存放着该 zone 的 min、low 和 high 三个内存水位线。它们是衡量当前 zone 剩余内存的一个标尺。当 zone 中的剩余内存高于 high 时说明剩余内存充足，低于 low 但高于 min 时说明内存短缺但是仍可分配内存，若低于 min 则说明剩余内存极度短缺将停止分配并将进行直接内存回收。每个 zone 区域可以通过 struct zone->_watermark 来访问各自的水位线。

上面左图展示了内存水位线和内存回收之间的联系：

- WMARK_HIGN(高水位线): 高于 WMARK_HIGH 时，kswapd 将会休眠。
- WMARK_LOW(低水位线): 低于 WMARK_LOW 时，kswapd 会被唤醒。
- WMARK_MIN(警戒线): 低于 WMARK_MIN 时，说明在低水位线和警戒线之间 kswapd 的回收速度低于分配的速度，因此阻塞分配并将进行直接内存 (direct reclaim) 回收。
- 内核中的一些紧急分配或是高优先级的分配请求，会设置不同的分配标识 (例如 PE_MEMALLOC)，来放宽对 WMARK_MIN 水位线的要求，将 WMARK_MIN 水位线适度降低，具体降低范围和过程参看后面 ___zone_watermark_ok() 函数。

NOTE

紧急预留内存通俗的讲也就是:*WMARK_MIN* 水位线以下的内存。*(*这里的 *WMARK_MIN* 水位线指放宽之前的 *min* 水位。*)*

4.1 水位线初始化

4.1.1 init_per_zone_wmark_min()

系统中对应每个 zone 的 min、low、high 水位线的默认值是在系统启动时计算并写入 _watermark[NRWMARK] 数组的。这个工作在 init_per_zone_wmark_min() 函数中完成。该函数除了完成水位线初始化，还设置了内存区域 (zone) 页面差值进行同步的阈值，各个 zone 预留内存 lowmem_reserve，各节点不进行映射页的最少数目，各节点 slab 可回收内存最多可以达到多少页等。

mm/page_alloc.c :: init_per_zone_wmark_min()

```
7929 int __meminit init_per_zone_wmark_min(void)
7930 {
7931     unsigned long lowmem_kbytes;
7932     int new_min_free_kbytes;
7933
7934     lowmem_kbytes = nr_free_buffer_pages() * (PAGE_SIZE >> 10);
7935     new_min_free_kbytes = int_sqrt(lowmem_kbytes * 16);
7936
7937     if (new_min_free_kbytes > user_min_free_kbytes) {
7938         min_free_kbytes = new_min_free_kbytes;
7939         if (min_free_kbytes < 128)
7940             min_free_kbytes = 128;
7941         if (min_free_kbytes > 262144)
7942             min_free_kbytes = 262144;
```

```
7943        } else {
7944            pr_warn("min_free_kbytes is not updated to %d because user defined value %d is
                preferred\n",
7945                    new_min_free_kbytes, user_min_free_kbytes);
7946        }
7947        setup_per_zone_wmarks();
7948        refresh_zone_stat_thresholds();
7949        setup_per_zone_lowmem_reserve();
7950
7951    #ifdef CONFIG_NUMA
7952        setup_min_unmapped_ratio();
7953        setup_min_slab_ratio();
7954    #endif
7955
7956        khugepaged_min_free_kbytes_update();
7957
7958        return 0;
7959    }
7960    postcore_initcall(init_per_zone_wmark_min)
```

- 7934 获取 ZONE_DMA、ZONE_DMA32 和 ZONE_NORMAL 区比各自 high 水位线高出的页的数量总和。换言之，是这三个 zone 区域富余页的总和。并换算成以 ***kbyte(千字节)*** 为单位的值。

  这里的 nr_free_buffer_pages() 的执行过程中会调用 gfp_zone(GFP_USER) 获取首选分配 zone 是 ZONE_NORMAL。所以该行的计算结果是基于 *首选 zone 和它所有的备选 zone* 而得到的。

- 7935 根据上一行获得的值算出新的 min。

  首先，上一行得到:

  $$
  \begin{aligned}
  &lowmem_kbytes \\
  &= 4 \times (\text{ZONE_DMA+ZONE_DMA32+ZONE_NORMAL 区中高于 high 水位的页数总和}) \\
  &(\text{单位: 千字节 (kbytes)})
  \end{aligned}
  $$

  此处进一步基于 lowmen_kbytes 进行调整，得到新的 min 值 (目标 zone min 水位线要保证的千字节值):

  $$
  \begin{aligned}
  &new_min_free_kbytes \\
  &= \sqrt{\text{lowmem_kbytes} \times 16} \\
  &= 4 \times \sqrt{\text{lowmem_kbytes}} \\
  &= 4 \times \sqrt{\text{ZONE_NORMAL 及备选 zone 区高于 high 水位的千字节值总和}} \\
  &(\text{单位: 千字节 (kbytes)})
  \end{aligned}
  $$

  因为 ZONE_DMA,ZONE_DMA32,ZONE_NORMAL 这三个区是常规内存分配区域，所以这个值也称为**全局最小预留内存**。

> ***NOTE***
>
> 在计算全局内存最小预留值时，我们只统计了 *ZONE_DMA,ZONE_DMA32,ZONE_NORMAL* 三个内存区，特别把 *ZONE_HIGHMEM* 排除在外。这是因为:*ZONE_HIGHMEM* 不是线性映射的，需要通过 *kmap()* 映射，会阻塞。而预留内存是为了紧急使用，可能在原子上下文、回收路径中申请，不能休眠。

- 7947 进一步调整得到最终的**全局最小预留内存值**。

  这里的***user_min_free_kbytes***是**用户**手动通过*/proc/sys/vm/min_free_kbytes* 设定的**期望的**全局最小预留空闲内存。系统会将通过该节点传递的值赋给全局变量 *user_min_free_kbytes*。

  这里***min_free_kbytes***则是内核通过计算和钳位得到的真正的全局最小预留内存值。

  所以，如果前面计数结果 > 用户期望的值，则更新 min_free_kbytes。并且全局最小预留值 $\in [128, 262144]$(单位: 千字节)。

  $min_free_kbytes = \sqrt{lowmem_kbytes \times 16}$ 这是一个经验公式。之所以没有使用线性公式是为了使嵌入式小内存平台按比例预留，防止分配失败。对于大内存平台则为了避免预留太多，造成浪费。所以随着富余内存逐渐变大，全局最小预留内存的变化越来越缓。内核注释中列出了一个利用常规内存区富余字节数和公式算得的全局最小预留内存字节数的对照表的例子，如下:

```
* 16MB:     512k
* 32MB:     724k
* 64MB:     1024k
* 128MB:    1448k
* 256MB:    2048k
* 512MB:    2896k
* 1024MB:   4096k
* 2048MB:   5792k
* 4096MB:   8192k
* 8192MB:   11584k
* 16384MB:  16384k
```

  左侧是 zone 的富余内存总和，右侧是根据公式计算到的全局最小预留内存。据此画出的曲线如下图，可以看到 zone 的富余内存总和越来越大时，全局最小预留内存的增长则越来越缓。

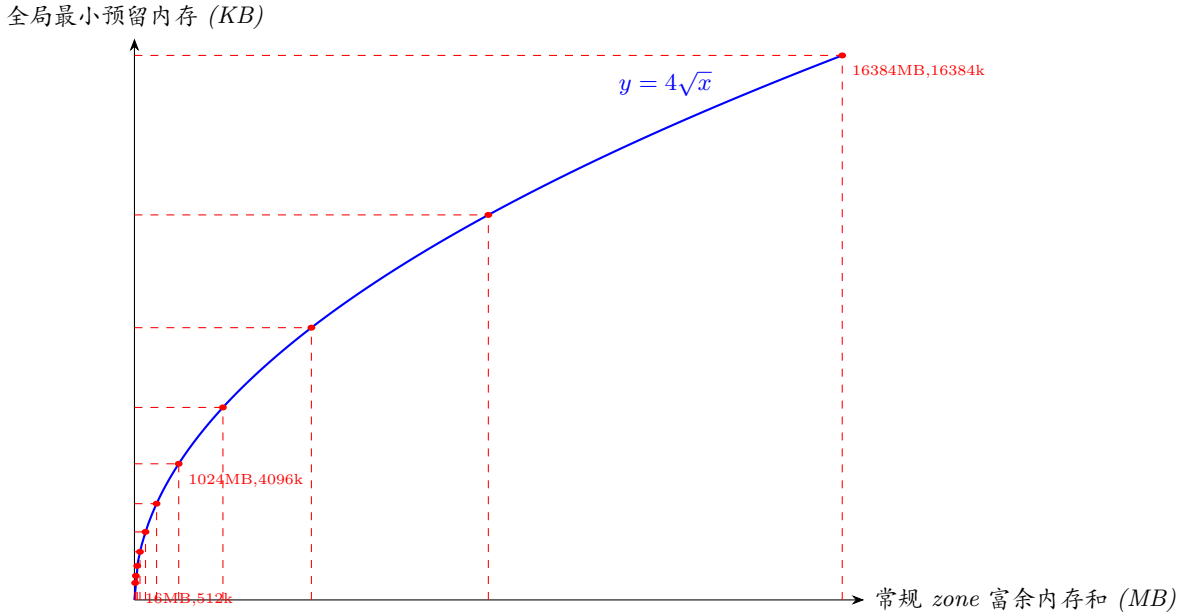


图 4–1: 常规 zone 富余内存总和与全局最小预留内存的关系曲线

- 7947 根据总的 min 值和各个 zone 在总内存中的占比，计算出每个 zone 各自的 min 值，进而计算出各个 zone 的水位大小。
- 7948 更新各个 zone 的本地 (或 per-cpu) 空闲页面统计差值 (Δ) 同步到全局页面统计值的阈值门槛

(该函数讨论见后面)。

- 7949 设置每个 zone 的 lowmem_reserve 大小。

  lowmem_reserve（低端内存预留）——这一机制通过分级保护，避免本来对“资源更充足 (或资源更廉价) 内存域”的请求无法被该域满足，分配器/组件尝试从其下一级内存域（资源较少或资源更珍贵的内存域）中分配内存 (fallback 到下一级内存域) 时，过度占用“资源较少内存域”的有限内存。换言之，其核心作用是：限制高级内存域对低级廉价内存的“侵占”，确保低级内存有足够空间供内核核心功能和 DMA 设备使用。因此当从非指定请求的首要内存域分配页时，该域应保留一定的页，防止耗尽该 zone 的内存。该值在 setup_per_zone_lowmem_reserve() 函数中设置，并由 vm.lowmem_reserve_ratio 可调参数控制。其值可以通过/proc/sys/vm/lowmem_reserve_ratio 来修改。

  页分配时，如果首选的 node 和 zone 不能满足请求，内核就会从备选 (fallback) 的内存 zone 区借用物理内存。fallback 的核心是“备用选择”，而非“等级降低”，在内存管理中仅表示分配顺序上的“次选”，这里的“备选”不意味着“级别更低”，而是“优先级靠后”。备选 zone 分两种情况：

  – 1) 一个内存 node 的某个 zone 向另一个 node 相同类型的 zone 借用物理内存。
  – 2) 高区域内存向低区域内存借用物理页。

  以 UMA 系统为例，内核在分配内存时，会按照 HIGHMEM->NORMAL->DMA 的方向进行遍历，如果当前 zone 分配失败，就会尝试低区域 (或者从 index 来说，是更高 index) 的 zone。譬如，应用进程通过内存映射申请 HIGHMEM 的内存，如果此时 HIGHMEM zone 无法满足分配，则会尝试从 NORMAL zone 进行分配。这就有一个问题，来自 HIGHMEM zone 的请求可能会耗尽 NORMAL zone 的内存，最终的结果就是 NORMAL zone 无内存满足内核对 NORMAL zone 内存的正常分配请求。因此针对这个场景，内核通过保留 lowmem_reserve[NORMAL] 大小的内存来满足对 NORMAL zone 本身的内存请求，限制因高区域 zone 分配失败降级到低区域 NORMAL zone 请求的内存的大小。

  同样当从 NORMAL zone 失败后，会尝试从 zonelist 中的 DMA zone 申请，通过 lowmem_reserve[DMA]，限制因 HIGHMEM 或 NORMAL zone 分配失败，从而请求的低区域 zone 内存的大小。

  可调参数 lowmem_reserve_ratio 代表内核对保留这些 lower zone 页的积极程度。如果系统使用 highmem 或 ISA DMA，而且应用程序使用 mlock()，或者没有运行时 swap 机制，那么可能应该修改 lowmem_reserve_ratio 的设置值。

  lowmem_reserve_ratio 是一个数组，可以通过以下方式读到：

```
% cat /proc/sys/vm/lowmem_reserve_ratio
256   256   32   0   0
```

但这些值不会被直接使用。内核会从中计算出每个 zone 的保留页数量。这些数据在 /proc/zoneinfo 中打印显示为”protection(保护)”(之所以称为”protection”，是因为历史原因，以下统一称为” 保留”)，如下所示。(这是一个 x86-64 系统的例子)。每个 zone 都有一个保留页数组，如下所示：

```
Node 0, zone DMA32
pages free     358269
       min      1445
       low      1806
       high     2167
    spanned   1044480
   present    376093
    managed    359671
        cma     0
     protection: (0, 0, 14257, 14257, 14257)
```

这些页面数目会被用于计算统计该 zone 应该保留多少页面，以判断该 zone 是否应该用于页面分配或被回收。

现在，假设/proc/sys/vm/lowmem_reserve_ratio 显示如下：

```
% cat /proc/lowmem_reserve_ratio
256 256 32
```

假设/proc/zoneinfo 内容如下:

```
% cat /proc/zoneinfo
Node 0, zone DMA
pages free      1355
      min       3
      low       3
      high      4
      ... ...
  numa_other    0
      protection : (0, 2004 , 2004 , 2004)
  pagesets
     cpu: 0 pcp: 0
```

如果 normal 页（index=2）向 DMA zone 请求，且 watermark[WMARK_HIGH] 用于水位线，内核会判断该 zone 不应被使用，因为 pages_free 为 (1355) 小于 watermark + protection[2] (4 + 2004 = 2008)。如果保留值为 0，则该区域将允许来自于 normal 区域的请求。如果是 DMA zone (index=0) 请求页，则使用 protection[0] (=0)。

因为 zone[j] (j 区) 页不足而向 zone[i] (i 区) 请求页时，zone[i] 对本区域的页是否要有保留、保留多少？可以通过以下表达式计算：

```
(i < j):
        zone[i]->protection[j]
                = (total sums of managed_pages from zone[i+1] to zone[j] on the node
                   ) / lowmem_reserve_ratio[i];
(i = j):
        (should not be protected. = 0;)
(i > j):
        (not necessary, but looks 0)
```

上面截图中，”i<j” 时 lowmem_reserve_ratio[i] 的值即”cat /proc/sys/vm/lowmem_reserve_ratio” 所打印的值。256 表示” 保留页面” 的数量约占节点上所有更高区域 (即更小 index) 的zone 管理的页面总数的 1/256 = 0.39%。所以，如果想保留更多页面，小数值会更有效。最小值为 1（1/1 = 100%）。小于 1 的值将完全禁用保留页面。

执行”cat /proc/zoneinfo” 的打印输出还有三个重要的量:

```
Node 0, zone DMA32
pages free     ...
      min      ...
      low      ...
      high     ...
   spanned   1044480
   present    376093
   managed    359671
      cma      ...
    protection: (......)
```

spanned: 表示当前 zone 包含的所有 pages(包含空洞) 的数量，它的值计算公式如下：

$$spanned_pages = zone_end_pfn - zone_start_pfn$$

present: 表示当前 zone 确实存在的所有物理页，计算如下:

$$present_pages = spanned_pages - absent_pages(pagesinholes)$$

managed: 表示由该 zone 的伙伴系统管理的页，也就是由确实存在的物理页减去保留的页. 式子如下：

$$managed_pages = present_pages - reserved_pages$$

- 7952-7953 计算各 NUMA 节点要保留不映射的页的最少数量, 和各 NUMA 节点可回收 slab 页最多数量。
- 7956 透明大页相关 (===== 待分析)

init_per_zone_wmark_min() > *nr_free_buffer_pages()*

```
5272 unsigned long nr_free_buffer_pages(void)
5273 {
5274     return nr_free_zone_pages(gfp_zone(GFP_USER));
5275 }
```

- 5274 gfp_zone() 会根据入参的 GFP 掩码，返回本次允许分配的最高 zone 的索引编号。用于限制页分配器只能遍历 ≤ 该编号 (或者该编号以下) 的内存 zone 区域。详见 4.2.2.1章节。

  这里传入的 gfp 参数是 *GFP_USER*, 查表可知，首选 zone 是 ZONE_NORMAL。

init_per_zone_wmark_min() > *nr_free_buffer_pages()* > *nr_free_zone_pages()*

```
5243 static unsigned long nr_free_zone_pages(int offset)
5244 {
5245     struct zoneref *z;
5246     struct zone *zone;
5247 
5248     /* Just pick one node, since fallback list is circular */
5249     unsigned long sum = 0;
5250 
5251     struct zonelist *zonelist = node_zonelist(numa_node_id(), GFP_KERNEL);
5252 
5253     for_each_zone_zonelist(zone, z, zonelist, offset) {
5254         unsigned long size = zone_managed_pages(zone);
5255         unsigned long high = high_wmark_pages(zone);
5256         if (size > high)
5257             sum += size - high;
5258     }
5259 
5260     return sum;
5261 }
```

- 5251 根据当前 cpu 所在的 NUMA 节点和分配掩码，找出本地 NUMA 节点对应的 zonelist。

  首先，zonelist 虽然名称中带”list”, 但其实是**数组**，它按索引顺序定义了 NUMA 节点分配内存时 zone 的首选顺序, 索引越小越优先选择 (注意，这是 zonelist 的数组成员的索引，和数组成员里存放的 zone 的编号索引不是同一个概念)。每个 NUMA 节点都会维护两套 zonelist，分别是 fallback 和 nofallback 的 zonelist。

fallback 意味着可以跨 node 选择备选 zone, 而 nofallback 意味着不能跨 node 选择 zone, 只能在本地 node 选择备选 zone。

NOTE

numa_node_id() 可用于找出当前正在执行的 cpu 对应的 NUMA node 的节点号。也就是本地 NUMA 节点号。

而分配掩码 flag 中的标记 ***___GFP_THISNODE*** 是否存在，决定了返回不跨 *node* 备选的 zonelist，还是返回跨 *node* 备选的 zonelist。

整个查找过程其实是: 根据 node 号先从存放 node 描述符的全局指针数组 struct pglist_data *node_data[] 中，找到节点的描述符指针，然后以宏 ZONELIST_FALLBACK 或 ZONELIST_NOFALLBACK 为索引从 pglist->zonelist[] 中返回本地节点的 zonelist。

- 5253 遍历目标 zonelist 数组中给定索引的 zone 及该 zone 之后的 zone。

  ===== 此处可以插入图

- 5154 返回目标 zone 中可正常动态分配的总页数。通常这个值 = *present_pages* − *reserved_pages*。即: 可正常分配的总页数 = 当前 zone 的实际物理页数 − 保留的页数。

- 5255-5257 high 水位线对应的页数目。

  如果可正常分配的总页数 >high 水位线对应的页数，那么将这个差值 Δ 累加到 *sum*。

- 5260 综上可知, 该函数会统计出给定 zone—包括它能备选的 zone 的—高出 high 水位线的 (可动态分配) 页的总和。

4.1.1.1 refresh_zone_stat_thresholds()

7948 行调用的函数:

init_per_zone_wmark_min() → *refresh_zone_stat_thresholds()*

```
246 void refresh_zone_stat_thresholds(void)
247 {
248     struct pglist_data *pgdat;
249     struct zone *zone;
250     int cpu;
251     int threshold;
252
253     /* Zero current pgdat thresholds */
254     for_each_online_pgdat(pgdat) {
255         for_each_online_cpu(cpu) {
256             per_cpu_ptr(pgdat->per_cpu_nodestats, cpu)->stat_threshold = 0;
257         }
258     }
259
260     for_each_populated_zone(zone) {
261         struct pglist_data *pgdat = zone->zone_pgdat;
262         unsigned long max_drift, tolerate_drift;
263
264         threshold = calculate_normal_threshold(zone);
265
266         for_each_online_cpu(cpu) {
267             int pgdat_threshold;
268
269             per_cpu_ptr(zone->pageset, cpu)->stat_threshold
270                             = threshold;
```

```
271
272             /* Base nodestat threshold on the largest populated zone. */
273             pgdat_threshold = per_cpu_ptr(pgdat->per_cpu_nodestats, cpu)->stat_threshold;
274             per_cpu_ptr(pgdat->per_cpu_nodestats, cpu)->stat_threshold
275                 = max(threshold, pgdat_threshold);
276         }
277
278         /*
279          * Only set percpu_drift_mark if there is a danger that
280          * NR_FREE_PAGES reports the low watermark is ok when in fact
281          * the min watermark could be breached by an allocation
282          */
283         tolerate_drift = low_wmark_pages(zone) - min_wmark_pages(zone);
284         max_drift = num_online_cpus() * threshold;
285         if (max_drift > tolerate_drift)
286             zone->percpu_drift_mark = high_wmark_pages(zone) +
287                     max_drift;
288     }
289 }
```

- 248 pglist_data 是 NUMA 架构下 node 节点物理内存管理的结构体。物理内存被划分成了多个 NUMA 节点。内核使用了一个大小为 MAX_NUMNODES，类型为 pglist_data 结构体的全局数组 node_data[ ] 来管理所有的 NUMA 节点。pglist_data 结构体中包含了节点中各个 zone 的管理结构。

  在每个 NUMA 节点内部又将其所管理的物理内存按照功能不同划分成不同的内存区域 zone ，每个内存区域 zone 管理用于特定功能的物理内存页 page，内核为每个内存区域 zone 分配一个 struct free_area 类型的数组 free_area[MAX_ORDER], 数组的每一个成员用于管理不同 size 物理内存块的分配和释放。数组的索引对应不同的 order, 不同的 order(分配阶) 对应不同 size 的内存块。每个 order 对应的内存块由 2^{order} 个连续物理页组成。相同 size 的内存块用会用双向链表组织起来。

  各 zone 中相同 size 的内存块在 struct free_area 结构中通过 struct list_head 组织成了 per zone 意义上的一个双向链表。内核根据物理内存页的迁移类型将相同 size 的内存块通过不同的双向链表重新组织起来，这些双向链表组成一个数组 free_list[MIGRATE_TYPES]，每个数组成员代表不同迁移类型的链表。

  迁移类型是内核为物理页帧分组划分的标签, 用于页回收、内存碎片整理、页迁移时的分类批量管理。把物理页帧按用途分为不同的迁移组，碎片整理时只在同组内移动页面，也就是页存储的数据只能迁移到同类型的物理页帧上，避免不同 (迁移) 类型的页面互相穿插形成无法合并的碎片。简言之，同类页面集中存放。

  页迁移或页整理需要严格同类型物理页帧。但是页分配时内存不足的话，可以从备选迁移类型的空闲链表借用页。
  物理内存页面的迁移类型有如下几种:

```
enum migratetype {
    MIGRATE_UNMOVABLE,
    MIGRATE_MOVABLE,
    MIGRATE_RECLAIMABLE,
    MIGRATE_PCPTYPES,   /* the number of types on the pcp lists */
    MIGRATE_HIGHATOMIC = MIGRATE_PCPTYPES,
#ifdef CONFIG_CMA
    /*
     * MIGRATE_CMA migration type is designed to mimic the way
```

```
     * ZONE_MOVABLE works.  Only movable pages can be allocated
     * from MIGRATE_CMA pageblocks and page allocator never
     * implicitly change migration type of MIGRATE_CMA pageblock.
     *
     * The way to use it is to change migratetype of a range of
     * pageblocks to MIGRATE_CMA which can be done by
     * __free_pageblock_cma() function.  What is important though
     * is that a range of pageblocks must be aligned to
     * MAX_ORDER_NR_PAGES should biggest page be bigger then
     * a single pageblock.
     */
    MIGRATE_CMA,
#endif
#ifdef CONFIG_MEMORY_ISOLATION
    MIGRATE_ISOLATE,    /* can't allocate from here */
#endif
    MIGRATE_TYPES
};
```

每个 zone 内部都有几条不同迁移类型的空闲页链表:free_area[order][迁移类型]。当要分配某一迁移类型的内存，但是该类型的空闲页不足。那么会根据预先定义的规则顺序，在同一 zone 内借用其他迁移类型的空闲页。
内核根据不同迁移类型定义了不同的备选/候补 (fallback) 规则:

```
static int fallbacks[MIGRATE_TYPES][3] = {
    [MIGRATE_UNMOVABLE]   = { MIGRATE_RECLAIMABLE, MIGRATE_MOVABLE,   MIGRATE_TYPES
        },
    [MIGRATE_MOVABLE]     = { MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE, MIGRATE_TYPES
        },
    [MIGRATE_RECLAIMABLE] = { MIGRATE_UNMOVABLE,   MIGRATE_MOVABLE,   MIGRATE_TYPES
        },
#ifdef CONFIG_CMA
    [MIGRATE_CMA]         = { MIGRATE_TYPES }, /* Never used */
#endif
#ifdef CONFIG_MEMORY_ISOLATION
    [MIGRATE_ISOLATE]     = { MIGRATE_TYPES }, /* Never used */
#endif
};
```

譬如: MIGRATE_UNMOVABLE 类型的 free_list 内存不足时，内核会备选到 MIGRATE_RECLAIMABLE 的空闲链表中去获取。如果还是不足，则再次候补 (fallback) 到 MIGRATE_MOVABLE 的空闲页链表中获取。同时会从候补迁移类型的最高阶空闲链表开始查找。

另外，前面提到每个 NUMA 节点的 struct pglist_data 结构中都会包含一个 node_zonelists，其中包含了当前 NUMA 节点以及备选 NUMA 节点的所有 zone 区域。当前前 NUMA 节点内存不足时，内核会从 node_zonelists 中的备选 NUMA 节点分配内存。如下:

```
typedef struct pglist_data {
    /*
     * node_zones contains just the zones for THIS node. Not all of the
     * zones may be populated, but it is the full list. It is referenced by
     * this node's node_zonelists as well as other node's node_zonelists.
     */
    struct zone node_zones[MAX_NR_ZONES];

    /*
     * node_zonelists contains references to all zones in all nodes.
     * Generally the first zones will be references to this node's
     * node_zones.
     */
```

```
    struct zonelist node_zonelists[MAX_ZONELISTS];
        .... ....
```

如前所述，node_zones 数组包含了目标 NUMA 节点中各个 zone 区域的管理结构。至于 node_zonelists[MAX_ZONELISTS] 数组，在 NUMA 系统里一共有两个元素，node_zonelists[ZONELIST_NOFALLBACK] 也就是索引成员 [1] 是只包含只把当前 node 的各个 zone 区作为备选的 zone 数组，该数组第一个成员当然是目标 zone。node_zonelists[ZONELIST_FALLBACK] 也就是索引成员 [0]，是包含把 NUMA 所有 node 节点的 zone 作为备选的 zone 数组，当然包括本地节点的 zone 在内。”距离”越近的邻居 node 的 zone 在数组中的位置越靠前 (后面会有详细说明)。如果分配内存时有___GFP_THISNODE标志则不会从其它 node 的 zone 备选。

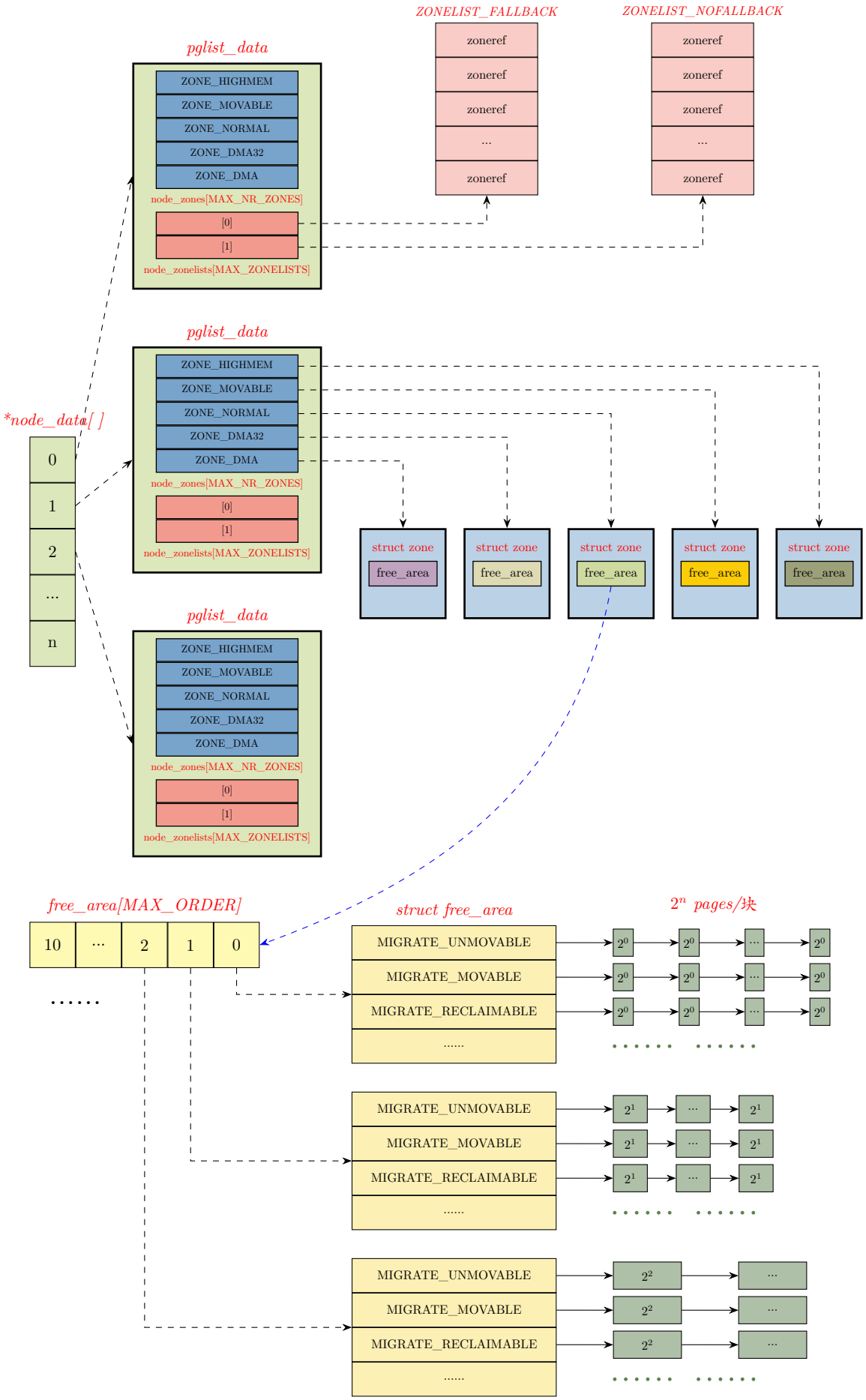
如前面章节所述，备选的顺序有两种：按照node 优先的顺序或按照zone 类型优先的顺序。可以使用 numa_zonelist_order 指定优先顺序：’d’ 表示默认顺序、’n’ 表示 node 优先、’z’ 表示 zone 优先。也可以使用节点/proc/sys/vm/numa_zonelist_order 进行运行中修改。需要注意的是，这两种备选顺序都只针对 ZONELIST_FALLBACK 而言。ZONELIST_NOFALLBACK 只从本地 zone 备选。

从上述讨论可以看出，存在两套备选 (fallback) 机制:

- ***zonelist* 备选**机制。
- ***migratetype*(迁移类型) 备选**机制。

这两套备选机制嵌套串行执行。核心逻辑是: 先选择一个 zone 分配，先在目标 zone 尝试目标迁移类型页的空闲链表。如果该链表上没有空闲页，就备选 *(zonelist fallback)*到本 zone 的其他迁移类型的空闲链表上借页。如果仍然没有空闲页，则按规则备选 *(migratetype fallback)*到本地 node 或跨 node 的其他 zone, 再次重复尝试寻找空闲页。

上述机制各管理结构之间关系的示意图如下 (图中内存块的双向链表没有画出来):


ZONELIST_FALLBACK
ZONELIST_NOFALLBACK
zoneref
zoneref
zoneref
...
zoneref
pglist_data
ZONE_HIGHMEM
ZONE_MOVABLE
ZONE_NORMAL
ZONE_DMA32
ZONE_DMA
node_zones[MAX_NR_ZONES]
[0]
[1]
node_zonelists[MAX_ZONELISTS]
*node_data[ ]
0
1
2
...
n
struct zone
free_area
free_area[MAX_ORDER]
10
...
2
1
0
struct free_area
MIGRATE_UNMOVABLE
MIGRATE_MOVABLE
MIGRATE_RECLAIMABLE
.....
2^n pages/块
2^0
2^1
2^2

- 254-258 将 per-cpu 变量 stat_threshold 初始化为 0，per-cpu 结构体变量 per_cpu_nodestats 是保存在各 cpu 里的 node 信息.
  per_cpu_nodestats 结构体如下：

```
struct per_cpu_nodestat {
    s8 stat_threshold;
    s8 vm_node_stat_diff[NR_VM_NODE_STAT_ITEMS];
};
```

- 260-276

  264 算出目标 zone 空闲页统计差值 (Δ) 同步的阈值 threshold，并赋值给 zone->pageset->stat_threshold.

  这里的 zone->pageset->stat_threshold 是一个 per-cpu 变量，和下面的 pgdat->per_cpu_nodestats->stat_threshold 都称为空闲页统计差值 (Δ) 同步的阈值。只是它们针对不同域的统计对象。

  274-275 比较 threshold 和存储在 per-cpu 里的 node 信息 stat_threshold，取最大值来更新 node 信息里的 stat_threshold。换言之,node 对应的 stat_threshold 在循环结束后为该 node 下所有 zone 的最大 threshold，即取各 zone->pageset->stat_threshold 的最大者。

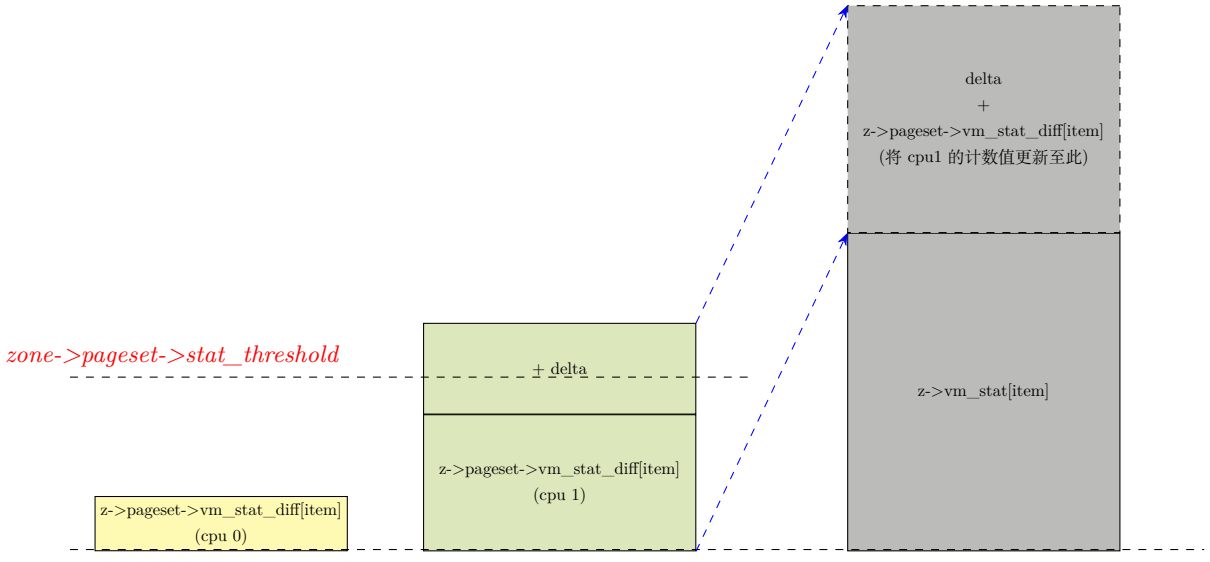
  那***stat_threshold***有什么作用?

  内核在进行物理内存分配时，需要读取空闲页面的数量值与 watermark 水位线值进行比较。而 zone->vm_stat 是 zone 分区的页面统计，zone->vm_stat_diff 是本地 cpu(或 per-cpu) 缓存由于 malloc/free 造成的增减计数。为了减少锁竞争和开销每次的增减不能也不会立刻刷到 zone 的全局统计值 zone->vm_stat 中。这时就由 per-cpu 的值 zone->pageset->stat_threshold 来控制何时将增减计数刷新到 zone->vm_stat。当差值 (增减计数) 的绝对值，也就是 $|\Delta| \geq stat_threshrold$ 这个门槛时才会更新。

  从计算也可以看出，zone 的内存越大，算得的结果值就越大，门槛也就越高，因此同步频次就越低。

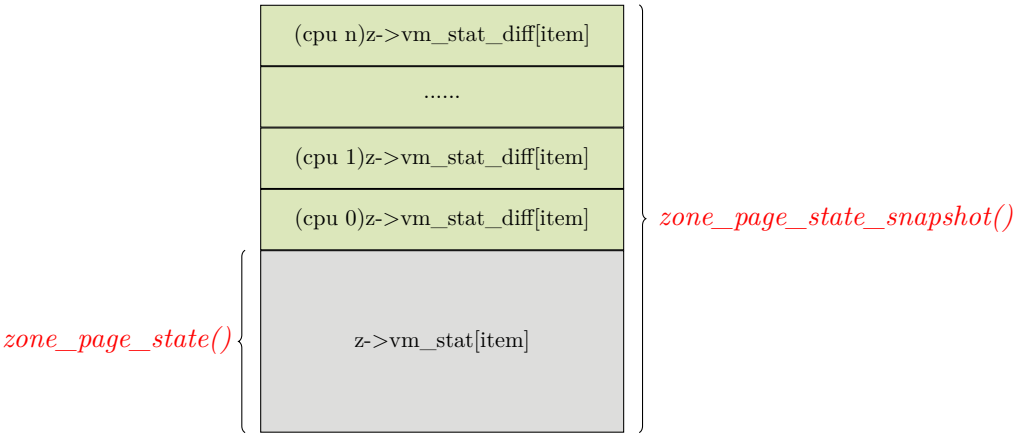
精确的空闲页面值 = z->vm_stat[NR_FREE_PAGES] + z->pageset->vm_stat_diff[NR_FREE_PAGES]

  当 per-cpu 的 zone->pageset->vm_stat_diff[ ] 的值 ≥zone->pageset->stat_threshold 时，会把值累加到 zone->vm_stat[ ] 中，同时对应 zone->pageset->vm_stat_diff[ ] 清 0。示意图如下：



因为 zone 的 pageset->vm_stat_diff[ ] 是 per-cpu 的，所以每次都要针对各 cpu 进行读取的话会影响系统效率。因此 zone 结构体中引入了 percpu_drift_mark 成员，只有在低于这个值的时候，才触发精确的页面读取。否则只从 zone->vm_stat[item] 读取页面计数。

读取精确页面值的函数是 zone_page_state_snapshot()(此处的精确是相对的)，读取 zone->vm_stat[item] 值的函数是 zone_page_state()。示意如图：



上面提到的几个 zone 中成员如下：

```
struct zone {
        ...
        struct per_cpu_pageset __percpu *pageset;
        ...
        unsigned long percpu_drift_mark;
        ...
        /* Zone statistics */
        atomic_long_t           vm_stat[NR_VM_ZONE_STAT_ITEMS];
}
```

pgdat->per_cpu_nodestats->stat_threshold 是 node 级别的 per-cpu 统计差值同步的门槛值。

- 283-287 这里计算前面提到的->percpu_drift_mark 值，这是一个启发式的经验值。

init_per_zone_wmark_min() → refresh_zone_stat_threshold() → calculate_normal_threshold()

```
196 int calculate_normal_threshold(struct zone *zone)
197 {
198     int threshold;
199     int mem;    /* memory in 128 MB units */
200
201     /*
202      * The threshold scales with the number of processors and the amount
203      * of memory per zone. More memory means that we can defer updates for
204      * longer, more processors could lead to more contention.
205      * fls() is used to have a cheap way of logarithmic scaling.
206      *
207      * Some sample thresholds:
208      *
209      * Threshold    Processors  (fls)   Zonesize    fls(mem+1)
210      * ------------------------------------------------------------------
211      * 8            1           1    0.9-1 GB    4
212      * 16           2           2    0.9-1 GB    4
213      * 20           2           2    1-2 GB      5
214      * 24           2           2    2-4 GB      6
```

```
215      * 28        2        2    4-8 GB      7
216      * 32        2        2    8-16 GB     8
217      * 4         2        2    <128M       1
218      * 30        4        3    2-4 GB      5
219      * 48        4        3    8-16 GB     8
220      * 32        8        4    1-2 GB      4
221      * 32        8        4    0.9-1GB     4
222      * 10        16       5    <128M       1
223      * 40        16       5    900M        4
224      * 70        64       7    2-4 GB      5
225      * 84        64       7    4-8 GB      6
226      * 108       512      9    4-8 GB      6
227      * 125       1024         10  8-16 GB     8
228      * 125       1024         10  16-32 GB    9
229      */
230
231     mem = zone_managed_pages(zone) >> (27 - PAGE_SHIFT);
232
233     threshold = 2 * fls(num_online_cpus()) * (1 + fls(mem));
234
235     /*
236      * Maximum threshold is 125
237      */
238     threshold = min(125, threshold);
239
240     return threshold;
241 }
```

根据目标 zone 内存大小，在线 cpu 的数量，计算 zone 的 vmstat 刷新的门槛。本地 cpu 的页面计数差值超过门槛时，才将 per-cpu 的缓存统计合并到全局 zone 统计值里，降低频繁同步带来的开销。

- 231 将目标 zone 管理的页总数折算成以 128M;bytes 为 base 单元的倍。换言之，目标页总数可以换算成多少个 128M 大小的块。这里 zone_managed_pages() 用于获取当前目标 zone 管理的页总数。PAGE_SHIFT 是 1 页换算成字节时的偏移量 ($1 \gg PAGE_SHIFT$)，通常是 12。于是 $27 - PAGE_SHIFT$ 等价于 $27 - 12 = 15$。$27 - PAGE_SHIFT$ 其实等价于这个式子:

$$\begin{aligned}
\text{128M 字节是多少页} &= \frac{128M}{\text{一页的字节数}} \\
&= \frac{2^{27}}{1 << PAGE_SHIFT} \\
&= \frac{2^{27}}{1 << 12} \\
&= \frac{2^{27}}{2^{12}} \\
&= \log_2(2^{27}) - \log_2(2^{12}) \\
&= 27 - 12 \\
&= 5
\end{aligned}$$

  所以，该行的计算等价于:$\frac{\text{zone 管理的页数目}}{128M}$

- 233 fls(x) 等价于:

$$\text{fls}(x) = \begin{cases} \log_2(x) + 1, & x > 0 \\ 1, & x = 0 \end{cases}$$

这里的 fls(mem)+1 是为了避免 $mem < 128M$ 导致整个 $fls(mem) = 0$, 从而 threashold 为 0。所以这个相当于 $threshold = 2 \times \log_2$(在线 cpu 数量) $\times \log_2$(当前 zone 有多少 128M 的块), 利用了低成本的对数缩放计算。

- 238 threshold 最大值钳位在 125。

4.1.1.2 ___setup_per_zone_wmarks()

init_per_zone_wmark_min()7035 行调用的函数 ___setup_per_zone_wmarks() 如下:

init_per_zone_wmark_min() > *___setup_per_zone_wmarks()*

```
7827 static void __setup_per_zone_wmarks(void)
7828 {
7829     unsigned long pages_min = min_free_kbytes >> (PAGE_SHIFT - 10);
7830     unsigned long lowmem_pages = 0;
7831     struct zone *zone;
7832     unsigned long flags;
7833 
7834     /* Calculate total number of !ZONE_HIGHMEM pages */
7835     for_each_zone(zone) {
7836         if (!is_highmem(zone))
7837             lowmem_pages += zone_managed_pages(zone);
7838     }
7839 
7840     for_each_zone(zone) {
7841         u64 tmp;
7842 
7843         spin_lock_irqsave(&zone->lock, flags);
7844         tmp = (u64)pages_min * zone_managed_pages(zone);
7845         do_div(tmp, lowmem_pages);
7846         if (is_highmem(zone)) {
7847             /*
7848              * __GFP_HIGH and PF_MEMALLOC allocations usually don't
7849              * need highmem pages, so cap pages_min to a small
7850              * value here.
7851              *
7852              * The WMARK_HIGH-WMARK_LOW and (WMARK_LOW-WMARK_MIN)
7853              * deltas control async page reclaim, and so should
7854              * not be capped for highmem.
7855              */
7856             unsigned long min_pages;
7857 
7858             min_pages = zone_managed_pages(zone) / 1024;
7859             min_pages = clamp(min_pages, SWAP_CLUSTER_MAX, 128UL);
7860             zone->_watermark[WMARK_MIN] = min_pages;
7861         } else {
7862            /*
7863             * If it's a lowmem zone, reserve a number of pages
7864             * proportionate to the zone's size.
7865             */
7866             zone->_watermark[WMARK_MIN] = tmp;
7867         }
7868 
7869         /*
7870          * Set the kswapd watermarks distance according to the
7871          * scale factor in proportion to available memory, but
7872          * ensure a minimum size on small systems.
7873          */
7874         tmp = max_t(u64, tmp >> 2,
7875                 mult_frac(zone_managed_pages(zone),
```

```
7876                        watermark_scale_factor, 10000));
7877
7878        zone->watermark_boost = 0;
7879        zone->_watermark[WMARK_LOW]  = min_wmark_pages(zone) + tmp;
7880        zone->_watermark[WMARK_HIGH] = min_wmark_pages(zone) + tmp * 2;
7881
7882        spin_unlock_irqrestore(&zone->lock, flags);
7883    }
7884
7885    /* update totalreserve_pages */
7886    calculate_totalreserve_pages();
7887 }
```

- 7829 将全局预留最小内存 (千字节) 总量值 min_free_kbytes 换算成页数目。
- 7835-7838 计算所有处于非 ZONE_HIGHMEM 区域的 zone 管理下的页数总和。换言之, 获取常规内存区域 ZONE_DMA,ZONE_DMA32,ZONE_NORMAL 区域的 zone 管理的内存页的总和。
- 7844-7845 do_div() 宏执行除法操作，商保存在第一个参数 (tmp) 里，余数作为返回值。

  结合 7835-7838 行可以看出，在针对各个 zone 的每次迭代循环中，根据该 zone 管理的页在所有 zone(除 ZONE_HIGHMEM 外的常规 zone) 管理页的总和中占的比例为系数，同比例获取目标 zone 对应的 page_min(也就是目标 zone 在全局预留最小内存中，要贡献多少预留内存)，公式如下:

$$\text{目标 zone 需要预留的最小内存值} = \text{全局预留最小内存值} \times \frac{\text{目标 zone 管理的页数目}}{\text{常规 zone 管理的页数总和}}$$

- 7846-7860 ___GFP_HIGH 和 PF_MEMALLOC 等紧急分配不依赖和使用高端内存页 (ZONE_HIGHMEM)，所以对于 ZONE_HIGHMEM 区域 watermark[WMARK_MIN] 限制在小值 32K-128K 个页即可。不使高端内存 zone 的 WMARK_MIN 值为 0 是为了避免一使用高端内存就触发直接回收，或 kswapd 回收，给 ZONE_HIGHMEM 留一点缓冲。
- 7866 对于常规 (非 ZONE_HIGHMEM)zone 区，上述按比例得到的值就是目标 zone 需要预留的最小内存值 WMARK_MIN。
- 7874-7880

  7874-7876 multi_frac() 函数的功能是将整数 × 分数相乘。同时尽量防止溢出和精度丢失。

  譬如,multi_frac(x, a, b) 等价于式子:

$$x \times \frac{a}{b} \tag{1}$$

  编程中传统的做法是有三种:(1) 是 $x \times a$ 后再除以 b。但这样造成的可能是 $x \times a$ 的结果可能很大，有可能造成溢出。(2) 是先计算 $\frac{a}{b}$, 然后得到的结果 $\times x$。但这样有一个问题是这里是整型相除，如果 $a < b$, 那么 $\frac{a}{b}$ 得到的结果就是 0。(3) 是 x 先除以 b, 得到的结果再 $\times a$。这样的话由于是整数相除，$\frac{x}{b}$ 会丢失余数。

  multi_frac() 的做法是: 先把 x 以 b 作为 base 拆成整数和余数两部分。即:

$$x = \text{整数部分} \cdot b^1 + \text{余数部分} \cdot b^0 \tag{2}$$

  然后 (2) 式代入 (1)，

$$(\text{整数部分} \cdot b^1 + \text{余数部分} \cdot b^0) \times \frac{a}{b}$$

得,

$$(\text{整数部分} \cdot b^1 \times \frac{a}{b}) + (\text{余数部分} \cdot b^0 \times \frac{a}{b})$$

得,

$$(\text{整数部分} \times a) + \frac{\text{余数部分} \times a}{b}$$

这个式子也就是 multi_frac() 代码中实现的算式。

(1) 对于 7874 行，如果不考虑 wtermark_scale_factor 这个参数值的影响，那么 7874 行等价于 $tmp \cdot \frac{1}{4}$. 此时对应 zone 的 WMARK_LOW 的页数 = tmp + (1/4) * tmp = (5/4) * tmp; 对应 WMARK_HIGH 的页数 = tmp + 2 * (1/4) * tmp = (1 + 2/4) * tmp = (3/2) * tmp = (6/4) * tmp。

那么 min、low、high 水位线之间的关系是:

$$min : low : high = tmp : (\frac{5}{4} * tmp) : (\frac{6}{4} * tmp) = 4 : 5 : 6$$

所以此时, 非高端内存 zone 区域的 min、low、high 水位线之间页数目比例: min:low:high = 4:5:6

(2) **wtermark_scale_factor** 这个参数是内存水位线的缩放因子，用于在不影响 min 水位的情况下，控制 low、high 两个水位之间，以及这两个水位距离 min 水位线的距离。参数默认值为 10，对应内存占比 10/10000=0.1%。

```
int watermark_scale_factor = 10;
```

考虑该参数影响，7874 行会在 $(\frac{1}{4} \cdot$ 目标 zone 预留内存) 和 (目标 zone 管理的页总数 $\cdot (\frac{\text{watermark_scale_factor}}{10000})$) 中取较大者赋值给 tmp, 也就是 WMARK_MIN 和 WMARK_LOW 之间的差值 $|\Delta|$。因为是取两者之间较大者，所以这个 Δ 至少是 $\frac{1}{4} \cdot$ 目标 zone 的 min 水位线.

这么做的目的在于: 根据可用内存和比例参数设置 kswapd 运行的水位线之间的距离，但同时确保在小内存的系统上有合适的最小水位线间隔，防止频繁唤醒 kswapd, 甚至频繁阻塞去直接回收 (direct reclaim) 内存。

wtermark_scale_factor 这个参数有对应的*/proc/sys/vm/watermark_scale_factor* 节点的实现 (kernel/sysctl.c 中), 用于暴露给用户空间来设置该参数。每次修改该值时都会通过回调函数来重新设置水位，如下:

```
int watermark_scale_factor_sysctl_handler(struct ctl_table *table, int write,
        void *buffer, size_t *length, loff_t *ppos)
{
    int rc;

    rc = proc_dointvec_minmax(table, write, buffer, length, ppos);
    if (rc)
        return rc;

    if (write)
        setup_per_zone_wmarks();

    return 0;
}
```

7879-7880 计算获取 WMARK_LOW 和 WMARK_HIGH 水位对应的内存 (单位: 千字节)。可以看到这几个水位线之间的关系:

- $\Delta_{low-min} = \frac{1}{2} \cdot \Delta_{high-low}$。
- $\Delta_low - min \geq \frac{1}{4} \cdot$ min 水位线的内存。

- 7886 更新每个节点的 totalreserve_pages 参数值.

init_per_zone_wmark_min() ··· *__setup_per_zone_wmarks()* *calculate_totalreserve_pages()*

```
7750 static void calculate_totalreserve_pages(void)
7751 {
7752     struct pglist_data *pgdat;
7753     unsigned long reserve_pages = 0;
7754     enum zone_type i, j;
7755
7756     for_each_online_pgdat(pgdat) {
7757
7758         pgdat->totalreserve_pages = 0;
7759
7760         for (i = 0; i < MAX_NR_ZONES; i++) {
7761             struct zone *zone = pgdat->node_zones + i;
7762             long max = 0;
7763             unsigned long managed_pages = zone_managed_pages(zone);
7764
7765             /* Find valid and maximum lowmem_reserve in the zone */
7766             for (j = i; j < MAX_NR_ZONES; j++) {
7767                 if (zone->lowmem_reserve[j] > max)
7768                     max = zone->lowmem_reserve[j];
7769             }
7770
7771             /* we treat the high watermark as reserved pages. */
7772             max += high_wmark_pages(zone);
7773
7774             if (max > managed_pages)
7775                 max = managed_pages;
7776
7777             pgdat->totalreserve_pages += max;
7778
7779             reserve_pages += max;
7780         }
7781     }
7782     totalreserve_pages = reserve_pages;
7783 }
```

当 sysctl_lowmem_reserve_ratio 或 min_free_kbytes 发生修改时，会调用该函数。

- 7756 遍历 NUMA 节点依次取出对应节点的描述符指针 (struct pglist_data *)pgdat。
- 7760 这是内循环。在取出 node 描述符后，顺序遍历目标 node 的各个 zone 区域。
- 7763 获取目标 zone 管理的总页数
- 7766-7769 轮询迭代所有能备选到目标 zone 的高端 zone(包括目标 zone 本身) 索引，比较这些 zone 备选到目标 zone 时要求目标 zone 预留的值，最大者赋给变量 max。这些值存储在目标 zone 的 lowmem_reserve[] 数组中。
- 7772 我们将高水位线 (包括可能的临时抬高值 boost) 也视为**预留内存**。如果不把高水位线作为保护的预留内存，那么哪怕高端 zone 有大量可回收的内存，也仍会吃光低端 zone 的内存。

这里的**预留内存**和 min 水位线相关的**紧急预留内存**是两个概念。

- 7774 如果算出的 max 大于目标 zone 管理的页数量，则修正 max= 目标 zone 管理的页总量.
- 7777 将目标 zone 的高水位线的页数量 + 最大预留值 lowmem_reserve，累计到目标 node 节点的总的保留内存变量中。
- 7779-7782 在内外循环结束后，将各个 node 节点的总预留内存计算总和赋给全局变量 totalreserve_page。

init_per_zone_wmark_min() › *setup_per_zone_lowmem_reserve()*

```
7791 static void setup_per_zone_lowmem_reserve(void)
7792 {
7793     struct pglist_data *pgdat;
7794     enum zone_type j, idx;
7795 
7796     for_each_online_pgdat(pgdat) {
7797         for (j = 0; j < MAX_NR_ZONES; j++) {
7798             struct zone *zone = pgdat->node_zones + j;
7799             unsigned long managed_pages = zone_managed_pages(zone);
7800 
7801             zone->lowmem_reserve[j] = 0;
7802 
7803             idx = j;
7804             while (idx) {
7805                 struct zone *lower_zone;
7806 
7807                 idx--;
7808                 lower_zone = pgdat->node_zones + idx;
7809 
7810                 if (!sysctl_lowmem_reserve_ratio[idx] ||
7811                     !zone_managed_pages(lower_zone)) {
7812                     lower_zone->lowmem_reserve[j] = 0;
7813                     continue;
7814                 } else {
7815                     lower_zone->lowmem_reserve[j] =
7816                         managed_pages / sysctl_lowmem_reserve_ratio[idx];
7817                 }
7818                 managed_pages += zone_managed_pages(lower_zone);
7819             }
7820         }
7821     }
7822 
7823     /* update totalreserve_pages */
7824     calculate_totalreserve_pages();
7825 }
```

该函数用于确保每个 zone 区域维护正确的预留页面数量，保证在分配页面后仍有充足的预留页面。此处是在 init 时，从 init_per_zone_wmark_min() 函数调用。但是在 sysctl_lowmem_reserve_ratio 值 (也就是用户手动写 */proc/sys/vm/lowmem_reserve_ratio*) 发生变动时也会自动调用。

- 7796 外层大循环遍历 NUMA 节点，获取 node 节点描述符指针。
- 7797 内循环遍历目标 node 下的各个 zone, 编号由小到大一次迭代。
- 7799 获取目标 zone 管理的页总数目。
- 7804 这里 idx 采用由大到小倒序遍历是为了累加高序列的 zone 的内存。因为编号更高序列的 zone 都具备借用低序列号 zone 内存的能力。它们总内存的规模决定了低序列 zone 被耗尽的风险大小。

低序列 zone 需要借助高序列所有 zone 的总内存计算预留的内存，$idx--$ 倒序遍历正好可以累加高序列 zone 的内存。

假设进入循环前 $j = 3$, 并且中途不通过关键字 continue 跳出。那么会循环 3 次，每次得到的 lowmem_reserve 如下:

- idx = 3, 得到 zone2->lowmem_reserve[3]
- idx = 2, 得到 zone1->lowmem_reserve[3]
- idx = 1, 得到 zone0->lowmem_reserve[3]

以此类推，j=2、1 时分别获得不同 zone 的 lowmem_reserve[2]、[1]。

init_per_zone_wmark_min() › *setup_min_unmapped_ratio()*

```
7999 static void setup_min_unmapped_ratio(void)
8000 {
8001     pg_data_t *pgdat;
8002     struct zone *zone;
8003 
8004     for_each_online_pgdat(pgdat)
8005         pgdat->min_unmapped_pages = 0;
8006 
8007     for_each_zone(zone)
8008         zone->zone_pgdat->min_unmapped_pages += (zone_managed_pages(zone) *
8009                                 sysctl_min_unmapped_ratio) / 100;
8010 }
```

- 8004-8005 遍历所有在线的 NUMA 节点，并对每个节点描述符的 min_unmapped_pages 清 0。
- 8007-8009 根据*/proc/sys/vm/min_unmapped_ratio* 计算每个 zone 的 *min_unmapped_pages*(每个 zone 要求保留的未映射页至少要有多少页)。并累加到 zone 所属的 NUMA 节点的 min_unmapped_pages。

  遍历系统中的每个 zone 内存区，并利用式子:

$$\text{目标 zone 管理的页数量} \times \frac{sysctl_min_unmapped_ratio}{100}$$

  即,

$$\text{目标 zone 管理的页数量} \times \frac{\text{预留比例}}{100}$$

  8008 根据上述算出的各个 zone 预留不映射的页，累加到各个 zone 所属的 NUMA 节点的->min_unmapped_pages。

  从上式也可以看出写入*/proc/sys/vm/min_unmapped_ratio* 节点的值的单位是:%。

这里各个 zone 指向的 zone_pgdat 是各个 zone 自己所属的 NUMA 节点，所以 zone->zone_pgdat->min_unmapped_pages 这个值是目标 node 下内存区域 (zone) 共享的。

init_per_zone_wmark_min() › *setup_min_slab_ratio()*

```
8027 static void setup_min_slab_ratio(void)
8028 {
8029     pg_data_t *pgdat;
8030     struct zone *zone;
8031 
8032     for_each_online_pgdat(pgdat)
8033         pgdat->min_slab_pages = 0;
8034 
8035     for_each_zone(zone)
8036         zone->zone_pgdat->min_slab_pages += (zone_managed_pages(zone) *
8037                              sysctl_min_slab_ratio) / 100;
8038 }
```

根据*/proc/sys/vm/min_unmapped_ratio* 计算每个 zone 允许的可回收 slab 内存最多达到多少页，并累加到所属节点的 *min_slab_pages*。

该函数的实现和前面的 setup_min_unmapped_ratio() 类似，此处不赘述。

4.2 ___alloc_pages_nodemask() • 页分配的快速路径和慢速路径

再回到页分配函数 ___alloc_pages_nodemask(), 子函数调用层次如下:

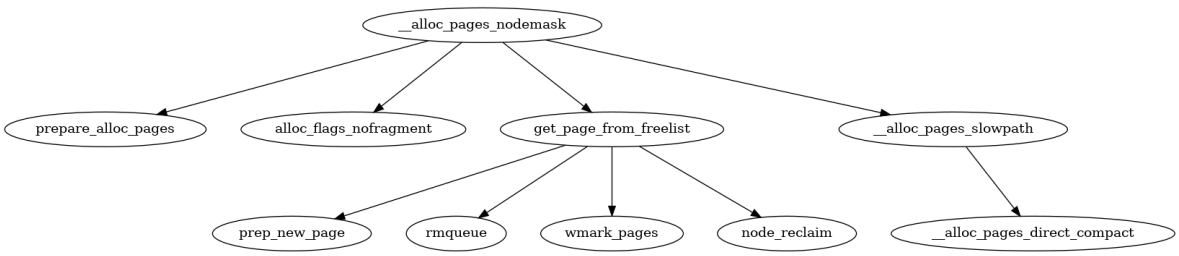


图 4–2: 调用图: ___alloc_pages_nodemask()

mm/page_alloc.c :: ___alloc_pages_nodemask()

```
4916 struct page *
4917 __alloc_pages_nodemask(gfp_t gfp_mask, unsigned int order, int preferred_nid,
4918                        nodemask_t *nodemask)
4919 {
4920     struct page *page;
4921     unsigned int alloc_flags = ALLOC_WMARK_LOW;
4922     gfp_t alloc_mask; /* The gfp_t that was actually used for allocation */
4923     struct alloc_context ac = { };
4924 
4925     /*
4926      * There are several places where we assume that the order value is sane
4927      * so bail out early if the request is out of bound.
4928      */
4929     if (unlikely(order >= MAX_ORDER)) {
4930         WARN_ON_ONCE(!(gfp_mask & __GFP_NOWARN));
4931         return NULL;
4932     }
4933 
4934     gfp_mask &= gfp_allowed_mask;
4935     alloc_mask = gfp_mask;
```

```
4936     if (!prepare_alloc_pages(gfp_mask, order, preferred_nid, nodemask, &ac, &alloc_mask,
             &alloc_flags))
4937         return NULL;
4938 
4939     /*
4940      * Forbid the first pass from falling back to types that fragment
4941      * memory until all local zones are considered.
4942      */
4943     alloc_flags |= alloc_flags_nofragment(ac.preferred_zoneref->zone, gfp_mask);
4944 
4945     /* First allocation attempt */
4946     page = get_page_from_freelist(alloc_mask, order, alloc_flags, &ac);
4947     if (likely(page))
4948         goto out;
4949 
4950     /*
4951      * Apply scoped allocation constraints. This is mainly about GFP_NOFS
4952      * resp. GFP_NOIO which has to be inherited for all allocation requests
4953      * from a particular context which has been marked by
4954      * memalloc_no{fs,io}_{save,restore}.
4955      */
4956     alloc_mask = current_gfp_context(gfp_mask);
4957     ac.spread_dirty_pages = false;
4958 
4959     /*
4960      * Restore the original nodemask if it was potentially replaced with
4961      * &cpuset_current_mems_allowed to optimize the fast-path attempt.
4962      */
4963     ac.nodemask = nodemask;
4964 
4965     page = __alloc_pages_slowpath(alloc_mask, order, &ac);
4966 
4967 out:
4968     if (memcg_kmem_enabled() && (gfp_mask & __GFP_ACCOUNT) && page &&
4969         unlikely(__memcg_kmem_charge_page(page, gfp_mask, order) != 0)) {
4970         __free_pages(page, order);
4971         page = NULL;
4972     }
4973 
4974     trace_mm_page_alloc(page, order, alloc_mask, ac.migratetype);
4975 
4976     return page;
4977 }
4978 EXPORT_SYMBOL(__alloc_pages_nodemask);
```

- 4921 设置分配时水位线限制的标识。其实这个宏值 =WMARK_LOW, 常常被用于作为 $zone->watermark[]$ 的索引

  当一个内存区域 zone 的空闲内存水位线处于 low 或 low 之上时，分配内存时会尝试走”快速路径 (fast path)” 进行分配。或者，换言之，” 快速路径” 的执行必须满足空闲页水位线 $\geq WMARK_LOW$。此时会给函数 get_page_from_freelist() 传递参数 ALLOC_WMARK_LOW(见 4946 行) 告知这样的限制。

  ALLOC_WMARK_LOW 被用来限定在 WMARK_LOW 及之上分配页。如果是 ALLOC_WMARK_MIN 传递给 get_page_from_freelist()，则是限制在水位线 WMAEK_MIN 及之上分配页。

- 4923 该结构体变量 ac 用于在不同的内存分配辅助函数间传递参数。和后面” 内存回收” 章节的 struct scan_control(实例化后的名称通常简写为 sc) 用处相似, sc 被用于在回收辅助函数间传递上下文参数。

- 4929 检查向伙伴系统申请内存的容量分配阶 order 是否在合法范围。内核定义的伙伴系统最大的页分配阶为 $(MAX_ORDER-1)$, MAX_ORDER 值默认定义为 11。所以 $2^{MAX_ORDER-1}=2^{10}$, 也就意味着, 一次最大可以申请 1k 数目的内存页, 如果按一页是 4k bytes 计算, 那么对应 $1k\times 4k=4M$ 字节的连续物理内存。

  如果配置了 CONFIG_FORCE_MAX_ZONEORDER, 那么 MAX_ORDER 会按该配置项配置的 order 的值设置。

- 4934 用 mask 清除不允许的 gfp 标志。这是一个全局的 gfp 标志掩码，用于标记在当前上下文可以使用哪些分配特性。

- 4936 初始化要传递的参数结构 alloc_context，并为接下来的快速内存分配设置相关 gfp。

- 4943 设置" 避免内存碎片化" 的相关分配标识。在第一遍尝试考察完本地所有 zone 之前，禁止备选 (fallback) 到可能造成内存碎片的迁移类型。换言之，也就是**首轮分配**禁止备选到容易造成碎片化的迁移类型 (函数见 4.2.3章节)。

- 4946 这是内存分配快速路径:第一次尝试从伙伴系统分配内存。此时是在阀值 WMARK_LOW (4921 行默认设置) 之上分配内存。

- 4947-4952 如果从快速路径分配成功则跳到 out 退出。

- 4956 将从 current->flags 传递的 gfp flags 和 gfp_mask 合并，生成最终生效的分配掩码。

- 4957 进入慢速路径时，允许单节点突破脏页限额，先保证分配能成功。

  譬如: node 1,node 2 是业务节点。node 0 内存极小，几乎没有配额。一旦 node ,node 2 脏页满了，那么按照均衡策略。只允许在未超限的 node 0 上分配，那这时就几乎没有任何节点满足要求了。

- 4963 恢复成最初的 nodemask, 因为它可能在第一次内存分配的过程中被改变。

- 4965 第一次快速内存分配失败说明内存已经不足了，内核需要做更多的工作. 比如通过 kswapd 后台异步回收内存，或者直接同步内存回收等方式获取更多的空闲内存以满足内存分配的需求。该过程称之为慢速分配路径。

- 4968-4971 伙伴系统还有空闲页，但是 cgropu 限额满了。因此将申请到的页释放回去。

4.2.1 ___alloc__pages__slowpath()

___alloc__pages__nodemask() → *___alloc__pages__slowpath()*

```
4607 static inline struct page *
4608 __alloc_pages_slowpath(gfp_t gfp_mask, unsigned int order,
4609                        struct alloc_context *ac)
4610 {
4611     bool can_direct_reclaim = gfp_mask & __GFP_DIRECT_RECLAIM;
4612     const bool costly_order = order > PAGE_ALLOC_COSTLY_ORDER;
4613     struct page *page = NULL;
4614     unsigned int alloc_flags;
4615     unsigned long did_some_progress;
```

```
4616     enum compact_priority compact_priority;
4617     enum compact_result compact_result;
4618     int compaction_retries;
4619     int no_progress_loops;
4620     unsigned int cpuset_mems_cookie;
4621     int reserve_flags;
4622 
4623     /*
4624      * We also sanity check to catch abuse of atomic reserves being used by
4625      * callers that are not in atomic context.
4626      */
4627     if (WARN_ON_ONCE((gfp_mask & (__GFP_ATOMIC|__GFP_DIRECT_RECLAIM)) ==
4628                 (__GFP_ATOMIC|__GFP_DIRECT_RECLAIM)))
4629         gfp_mask &= ~__GFP_ATOMIC;
4630 
4631 retry_cpuset:
4632     compaction_retries = 0;
4633     no_progress_loops = 0;
4634     compact_priority = DEF_COMPACT_PRIORITY;
4635     cpuset_mems_cookie = read_mems_allowed_begin();
4636 
4637     /*
4638      * The fast path uses conservative alloc_flags to succeed only until
4639      * kswapd needs to be woken up, and to avoid the cost of setting up
4640      * alloc_flags precisely. So we do that now.
4641      */
4642     alloc_flags = gfp_to_alloc_flags(gfp_mask);
4643 
4644     /*
4645      * We need to recalculate the starting point for the zonelist iterator
4646      * because we might have used different nodemask in the fast path, or
4647      * there was a cpuset modification and we are retrying - otherwise we
4648      * could end up iterating over non-eligible zones endlessly.
4649      */
4650     ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
4651                     ac->highest_zoneidx, ac->nodemask);
4652     if (!ac->preferred_zoneref->zone)
4653         goto nopage;
4654 
4655     if (alloc_flags & ALLOC_KSWAPD)
4656         wake_all_kswapds(order, gfp_mask, ac);
4657 
4658     /*
4659      * The adjusted alloc_flags might result in immediate success, so try
4660      * that first
4661      */
4662     page = get_page_from_freelist(gfp_mask, order, alloc_flags, ac);
4663     if (page)
4664         goto got_pg;
4665 
4666     /*
4667      * For costly allocations, try direct compaction first, as it's likely
4668      * that we have enough base pages and don't need to reclaim. For non-
4669      * movable high-order allocations, do that as well, as compaction will
4670      * try prevent permanent fragmentation by migrating from blocks of the
4671      * same migratetype.
4672      * Don't try this for allocations that are allowed to ignore
4673      * watermarks, as the ALLOC_NO_WATERMARKS attempt didn't yet happen.
4674      */
4675     if (can_direct_reclaim &&
4676             (costly_order ||
4677                (order > 0 && ac->migratetype != MIGRATE_MOVABLE))
4678             && !gfp_pfmemalloc_allowed(gfp_mask)) {
4679         page = __alloc_pages_direct_compact(gfp_mask, order,
```

```
4680                        alloc_flags, ac,
4681                        INIT_COMPACT_PRIORITY,
4682                        &compact_result);
4683        if (page)
4684            goto got_pg;
4685
4686        /*
4687         * Checks for costly allocations with __GFP_NORETRY, which
4688         * includes some THP page fault allocations
4689         */
4690        if (costly_order && (gfp_mask & __GFP_NORETRY)) {
4691            /*
4692             * If allocating entire pageblock(s) and compaction
4693             * failed because all zones are below low watermarks
4694             * or is prohibited because it recently failed at this
4695             * order, fail immediately unless the allocator has
4696             * requested compaction and reclaim retry.
4697             *
4698             * Reclaim is
4699             *  - potentially very expensive because zones are far
4700             *    below their low watermarks or this is part of very
4701             *    bursty high order allocations,
4702             *  - not guaranteed to help because isolate_freepages()
4703             *    may not iterate over freed pages as part of its
4704             *    linear scan, and
4705             *  - unlikely to make entire pageblocks free on its
4706             *    own.
4707             */
4708            if (compact_result == COMPACT_SKIPPED ||
4709                compact_result == COMPACT_DEFERRED)
4710                goto nopage;
4711
4712            /*
4713             * Looks like reclaim/compaction is worth trying, but
4714             * sync compaction could be very expensive, so keep
4715             * using async compaction.
4716             */
4717            compact_priority = INIT_COMPACT_PRIORITY;
4718        }
4719    }
4720
4721retry:
4722    /* Ensure kswapd doesn't accidentally go to sleep as long as we loop */
4723    if (alloc_flags & ALLOC_KSWAPD)
4724        wake_all_kswapds(order, gfp_mask, ac);
4725
4726    reserve_flags = __gfp_pfmemalloc_flags(gfp_mask);
4727    if (reserve_flags)
4728        alloc_flags = current_alloc_flags(gfp_mask, reserve_flags);
4729
4730    /*
4731     * Reset the nodemask and zonelist iterators if memory policies can be
4732     * ignored. These allocations are high priority and system rather than
4733     * user oriented.
4734     */
4735    if (!(alloc_flags & ALLOC_CPUSET) || reserve_flags) {
4736        ac->nodemask = NULL;
4737        ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
4738                    ac->highest_zoneidx, ac->nodemask);
4739    }
4740
4741    /* Attempt with potentially adjusted zonelist and alloc_flags */
4742    page = get_page_from_freelist(gfp_mask, order, alloc_flags, ac);
4743    if (page)
```

```
4744        goto got_pg;
4745
4746    /* Caller is not willing to reclaim, we can't balance anything */
4747    if (!can_direct_reclaim)
4748        goto nopage;
4749
4750    /* Avoid recursion of direct reclaim */
4751    if (current->flags & PF_MEMALLOC)
4752        goto nopage;
4753
4754    /* Try direct reclaim and then allocating */
4755    page = __alloc_pages_direct_reclaim(gfp_mask, order, alloc_flags, ac,
4756                        &did_some_progress);
4757    if (page)
4758        goto got_pg;
4759
4760    /* Try direct compaction and then allocating */
4761    page = __alloc_pages_direct_compact(gfp_mask, order, alloc_flags, ac,
4762                    compact_priority, &compact_result);
4763    if (page)
4764        goto got_pg;
4765
4766    /* Do not loop if specifically requested */
4767    if (gfp_mask & __GFP_NORETRY)
4768        goto nopage;
4769
4770    /*
4771     * Do not retry costly high order allocations unless they are
4772     * __GFP_RETRY_MAYFAIL
4773     */
4774    if (costly_order && !(gfp_mask & __GFP_RETRY_MAYFAIL))
4775        goto nopage;
4776
4777    if (should_reclaim_retry(gfp_mask, order, ac, alloc_flags,
4778                 did_some_progress > 0, &no_progress_loops))
4779        goto retry;
4780
4781    /*
4782     * It doesn't make any sense to retry for the compaction if the order-0
4783     * reclaim is not able to make any progress because the current
4784     * implementation of the compaction depends on the sufficient amount
4785     * of free memory (see __compaction_suitable)
4786     */
4787    if (did_some_progress > 0 &&
4788            should_compact_retry(ac, order, alloc_flags,
4789                compact_result, &compact_priority,
4790                &compaction_retries))
4791        goto retry;
4792
4793
4794    /* Deal with possible cpuset update races before we start OOM killing */
4795    if (check_retry_cpuset(cpuset_mems_cookie, ac))
4796        goto retry_cpuset;
4797
4798    /* Reclaim has failed us, start killing things */
4799    page = __alloc_pages_may_oom(gfp_mask, order, ac, &did_some_progress);
4800    if (page)
4801        goto got_pg;
4802
4803    /* Avoid allocations with no watermarks from looping endlessly */
4804    if (tsk_is_oom_victim(current) &&
4805        (alloc_flags & ALLOC_OOM ||
4806         (gfp_mask & __GFP_NOMEMALLOC)))
4807        goto nopage;
```

```
4808
4809        /* Retry as long as the OOM killer is making progress */
4810        if (did_some_progress) {
4811            no_progress_loops = 0;
4812            goto retry;
4813        }
4814
4815    nopage:
4816        /* Deal with possible cpuset update races before we fail */
4817        if (check_retry_cpuset(cpuset_mems_cookie, ac))
4818            goto retry_cpuset;
4819
4820        /*
4821         * Make sure that __GFP_NOFAIL request doesn't leak out and make sure
4822         * we always retry
4823         */
4824        if (gfp_mask & __GFP_NOFAIL) {
4825            /*
4826             * All existing users of the __GFP_NOFAIL are blockable, so warn
4827             * of any new users that actually require GFP_NOWAIT
4828             */
4829            if (WARN_ON_ONCE(!can_direct_reclaim))
4830                goto fail;
4831
4832            /*
4833             * PF_MEMALLOC request from this context is rather bizarre
4834             * because we cannot reclaim anything and only can loop waiting
4835             * for somebody to do a work for us
4836             */
4837            WARN_ON_ONCE(current->flags & PF_MEMALLOC);
4838
4839            /*
4840             * non failing costly orders are a hard requirement which we
4841             * are not prepared for much so let's warn about these users
4842             * so that we can identify them and convert them to something
4843             * else.
4844             */
4845            WARN_ON_ONCE(order > PAGE_ALLOC_COSTLY_ORDER);
4846
4847            /*
4848             * Help non-failing allocations by giving them access to memory
4849             * reserves but do not use ALLOC_NO_WATERMARKS because this
4850             * could deplete whole memory reserves which would just make
4851             * the situation worse
4852             */
4853            page = __alloc_pages_cpuset_fallback(gfp_mask, order, ALLOC_HARDER, ac);
4854            if (page)
4855                goto got_pg;
4856
4857            cond_resched();
4858            goto retry;
4859        }
4860    fail:
4861        warn_alloc(gfp_mask, ac->nodemask,
4862                "page allocation failure: order:%u", order);
4863    got_pg:
4864        return page;
4865    }
```

- 4611 对内存分配标识进行合法性检查。

  ___GFP_DIRECT_RECLAIM 表示可以进行会导致休眠的直接回收的操作。而 ___GFP_ATOMIC 表示原子上下的分配，不能执行会导致阻塞休眠的任何动作，包括直接回收操作。通常用在中断上

下文。所以这两个标识相互矛盾不能同时使用。慢速路径碰到这两个标识一起使用时，会把其中的 ___GFP_ATOMIC 标识位清 0.

___GFP_ATOMIC 通常和 ___GFP_HIGH 一起联用。

- 4612 当需分配的页阶数大于 ***PAGE_ALLOC_COSTLY_ORDER*** 时 (PAGE_ALLOC_COSTLY_ORDER 被硬编码成 3)，表示这是一次” 代价昂贵” 的分配，这个” 代价昂贵” 意味着在正常压力水平下，这个页块的回收不能和伙伴页进行自然合并。
- 4642 把上层业务传递的分配标识转换成底层伙伴系统使用的分配水位控制标识。

  快速路径使用保守的分配标志，仅需确保在不需要唤醒 kswapd 的情况下尽快分配成功，避免精确计算和设置标志带来的开销。而当前慢速路径下，需要在唤醒 kswapd 之前精确设置这些标志。
- 4650-4653 确定在哪个首选 zone 上进行内存申请。如果找不到合适的 zone 则跳转到 nopage 处的处理逻辑。
- 4655-4656 如果设置了 ALLOC_KSWAPD 标志，则在所有允许分配的 node 上唤醒 kswapd，对所有 zone 进行间接内存回收。

  NOTE

  在 NUMA 架构下，每个 node 节点对应一个 kswapd 线程，负责节点自身的内存回收。
- 4662-4664 在 4642 行调整了分配标识为 ALL_WMARK_MIN 后，意味着在此处调用 get_page_from_freelist() 时允许最低在WMARK_MIN水位线上尝试获取得内存。如果还没无法申请到内存，表示此时内存压力较高，那么接下来会进行直接内存回收。
- 4675-4682 首先尝试直接内存整理，换言之，也就是阻塞式的内存整理。

  进行直接内存整理需要同时满足以下三个条件条件:

  – 允许直接回收。

  – 目前申请的内存页阶是” 代价较高”($order > 3$) 的分配，或者当前申请的是大阶页并且申请的内存类型是不可移动的 (MIGRATE_UNMOVABLE)。

  代价较高的连续页指:$order > 3$ 的连续页。大阶页指:$order > 0$ 的连续页。

  – 当前内存分配请求不具备使用 PF_MEMALLOC 标记的紧急物理页的申请权限。换言之，当前分配必须严格遵守水位线限制。

  PF_MEMALLOC 标记的页是系统内存耗尽时的兜底资源，只能给内核紧急流程使用，普通流程禁止使用。如果普通分配占用了这些紧急内存，内核的自救关键操作线程会因拿不到内存而卡死，最终导致系统内存死锁、彻底崩溃。内核通过 ALLOC_NO_WATERMARKS 标志（无视内存水位线）紧急分配的物理页，会被 set_page_pfmemalloc(page) 打上该标记，

  对于高代价分配，应优先尝试直接内存整理，因为此时系统很可能具备足够的基页，无需执行内存回收。对于不可移动的大阶页分配，优先执行内存整理；内存整理通过对相同迁移类型的内存块进行迁移，可以有效避免产生永久性碎片。
- 4723-4724 如果 ALLOC_KSWAPD 标志存在，我们再次为该 node 上的所有 zone 唤醒 kswapd。

  这是为了保证仍然在进行分配尝试时，进行非直接回收的后台 kswapd 线程不会休眠。或者说，防止 kswapd 意外进入休眠。

- 4726 通过 ___gfp_pfmemalloc_flags() 判断本次分配是否可以使用紧急预留内存。
- 4727-4728 如果本次分配为紧急分配，那么更新分配标识。函数根据两个输入标志的组合，设置 alloc_flags 中的关键位，最终生成可直接指导页分配器执行物理页分配的标志。解决上层标志通用、下层执行需要精细化规则的问题。
  - 第一个参数 gfp_mask：上层传递的通用内存分配标志（指定分配场景、是否允许阻塞、是否允许回收等通用规则）。
  - 第二个参数 reserve_flags：本次获取的 PF_MEMALLOC 紧急标志；
- 4735 如果满足如下两个条件之一则可以突破 cpuset 限制:
  - 当 alloc_flags 中置位 ALLOC_CPUSET, 表示分配必须只能在 cpuset 限定的内存节点中分配。当该位未置位（即!(alloc_flags & ALLOC_CPUSET) 为真），所以本次可在系统所有内存节点中分配。
  - 或者可以使用预留的紧急内存 (reserve_flags 不为 0)。
- 4736-4737 通过以上判定确认本次是紧急分配，并且不限制当前进程只允许从所属的 cpuset 内存节点中分配页面。那么可通过设置 nodemask 为 NULL 来突破 cpuset 的限制。同时通过 first_zones_zonelist() 重新确定首选 zone 来分配内存。
- 4742-4744 重新尝试获取页，看分配标志改变或者非直接回收是否已经起效，从而促使这次分配成功。如果没有成功则进入下一步。否则得到页进行跳转。

  这里有可能是在忽略 min 水位的条件 (4726-4728 行) 下执行该函数。
- 4747-4748 到了这里，我们将尝试直接回收。但是如果 can_direct_reclaim 没有设置，那么意味着不能进行回收。最后努力失败。
- 4751-4752 如果设置了 PF_MEMALLOC, 则意味着不直接回收从紧急预留内存申请内存。仍然跳转到 no page 分支处理。” 跳转” 这一步很关键，因为 4755 行直接回收逻辑中会自己设置这个标志，如果不调整的话会导致反复递归的进行回收操作。
- 4755 执行直接回收和分配的操作。
- 4761 如果直接回收未能分配到页，那么执行直接内存整理和分配的操作。

  内存整理是内核迁移可移动页，将伙伴系统中的页块合并成更大阶数连续页。通过该机制，可避免大阶页因内存碎片导致无法分配内存的问题
- 4767-4768 判断是否要重新尝试。如果设置了 ___GFP_NORETRY 标志，则不重试。
- 4774 如果是代价较高的页分配 (>8 页 (2^3)), 且不允许再进行可能失败的尝试，那么跳转无页处理流程。

  高代价分配 (costly allocations) 本身就意味着：如果直接回收没有立刻释放出空闲页，那么分配几乎不可能成功。因此，除非用户通过 ___GFP_RETRY_MAYFAIL 标志显式要求重试，否则在这种情况下不会再次尝试分配.
- 4777 其它场景下是否再次尝试由辅助函数 should_reclaim_retry() 来判定。

  需要注意的是，did_some_progress 会作为参数传入，该变量由 ___alloc_pages_direct_reclaim() 设置。它用于表示直接回收过程是否取得了进展，如果有进展，那么重试回收操作就是有意义的。因此，它是 should_reclaim_retry() 的判断依据之一。

- 4787-4791 如果函数 should_reclaim_retry() 不允许重试，但直接回收已经取得了进展，则通过 should_compact_retry() 对内存整理是否重试应用相同的判断逻辑。
- 4799-4801 判断该函数是否能立即获得可用页来满足分配需求。
- 4804-4807 如果 ___alloc_pages_may_oom() 不能分配到页，则会检查当前运行的线程是否是被选中的牺牲进程。并且出现以下任一情形：

  进程正在使用专门给 OOM 路径使用的预留内存。正确的说应该是 ALLOC_OOM 会降低 min 水位线去分配。

  这是为了防止已经被 OOM 杀死的牺牲者进程，还去抢内存。。

  或由于设置了 ___GFP_NOMEMALLOC(禁止使用内核的紧急内存预留)。

  这是防止牺牲者线程在退出过程中抢紧急内存，导致完全没有可用的紧急预留内存从而系统卡死。这种情况下，我们直接进入 nopage 处理流程。
- 4810-4813 否则，如果 OOM killer 执行取得了进展 (由 did_some_progress 参数指示)，就重置 no_progress_loops 计数器并重新尝试分配。直接宣告分配失败，向内核循环缓冲区写入一条警告信息，前提是未设置 ___GFP_NOWARN 标志。
- 4815 如果未取得进展，或者逻辑中的其他分支触发了无可用页面处理流程，最终会进入分配失败处理环节。
- 4824-4859 当设置了 ___GFP_NOFAIL 标志的情况下，要求内核必须满足分配请求，不允许失败，因此可能会无限重试。

  使用者应当仅在没有合理的失败处理策略时才使用 ___GFP_NOFAIL 标识。但相比于在分配器周围的代码中自行编写无限循环，使用该标志显然是更可取的方案。但不建议将该标志用于高代价分配。

  4829-4830 检查一种边缘场景：设置了 ___GFP_NOFAIL，但未设置 ___GFP_DIRECT_RECLAIM。此时内核会认为 ___GFP_NOFAIL 是被错误设置的，因此中止分配。

  4837 PF_MEMALLOC 标记是给线程开“绿色通道”，允许它使用预留内存，不受任何内存水位限制，保证它一定能分到内存。

  NOTE

  PF_MEMALLOC 是给内存回收线程或 OOM killer 线程专用的特权!
  不会触发新的内存回收，因为它自己就是为了回收内存而存在的。因为内存回收线程也需要临时内存，如果回收内存的时候也分配不到内存，系统就死锁了。

  在 nopage 和 ___GFP_NOFAI 的场景下，如果自己就是回收线程，回收线程拥有 PF_MEMALLOC 权限都没法获得内存，陷入 nopage。只能干等别人救你。因此发出告警。

  4845 在 nopage 和 ___GFP_NOFAI 的场景下，高代价分配不允许失败是难以满足的。因此发出警告，以便将其改造为其他方案。
- 4853 调用 ___alloc_pages_cpuset_fallback() 执行最后的分配尝试，该函数会设置 ALLOC_HARDER 标志，以访问更多的紧急内存预留资源。如果失败，内核将无条件地重试分配。这里不使用 ALLOC_NO_WATERMARKS，因为这可能耗尽全部预留储备内存，反而会让内存短缺的情况进一步恶化。

NOTE

内存分配标识与水位线和积极度的关系

标识	水位线算式	水位线值	积极度
ALLOC_HARDER	$min*(1-\frac{1}{4})$	$\frac{6}{8}min$	*NA*
ALLOC_HIGH	$min*(1-\frac{1}{2})$	$\frac{4}{8}min$	积极度更高
ALLOC_OOM	$min*(1-\frac{1}{2})$	$\frac{4}{8}min$	积极度 *ALLOC_OOM == ALLOC_HIGH.(CONFIG_MMU=y* 的情况下)
ALLOC_HIGH | ALLOC_HARDER	$min*(1-\frac{1}{2})*(1-\frac{1}{4})$	$\frac{3}{8}min$	积极度更高
ALLOC_HIGH | ALLOC_OOM	$min*(1-\frac{1}{2})*(1-\frac{1}{2})$	$\frac{2}{8}min$	积极度更高
ALLOC_NO_WATERMARKS	*NA*	*NA*	完全忽略 *MIN* 水位线，积极度大于任何其他标识

表格内水位线的计算见函数 ___zone__watermark__ok() 函数. 该函数在 get__page__from__freelist() 分配页时会间接调用来判断水位线。

- 4857-4858 让出调度，之后再重试。防止该函数长时间占用 cpu。

NOTE

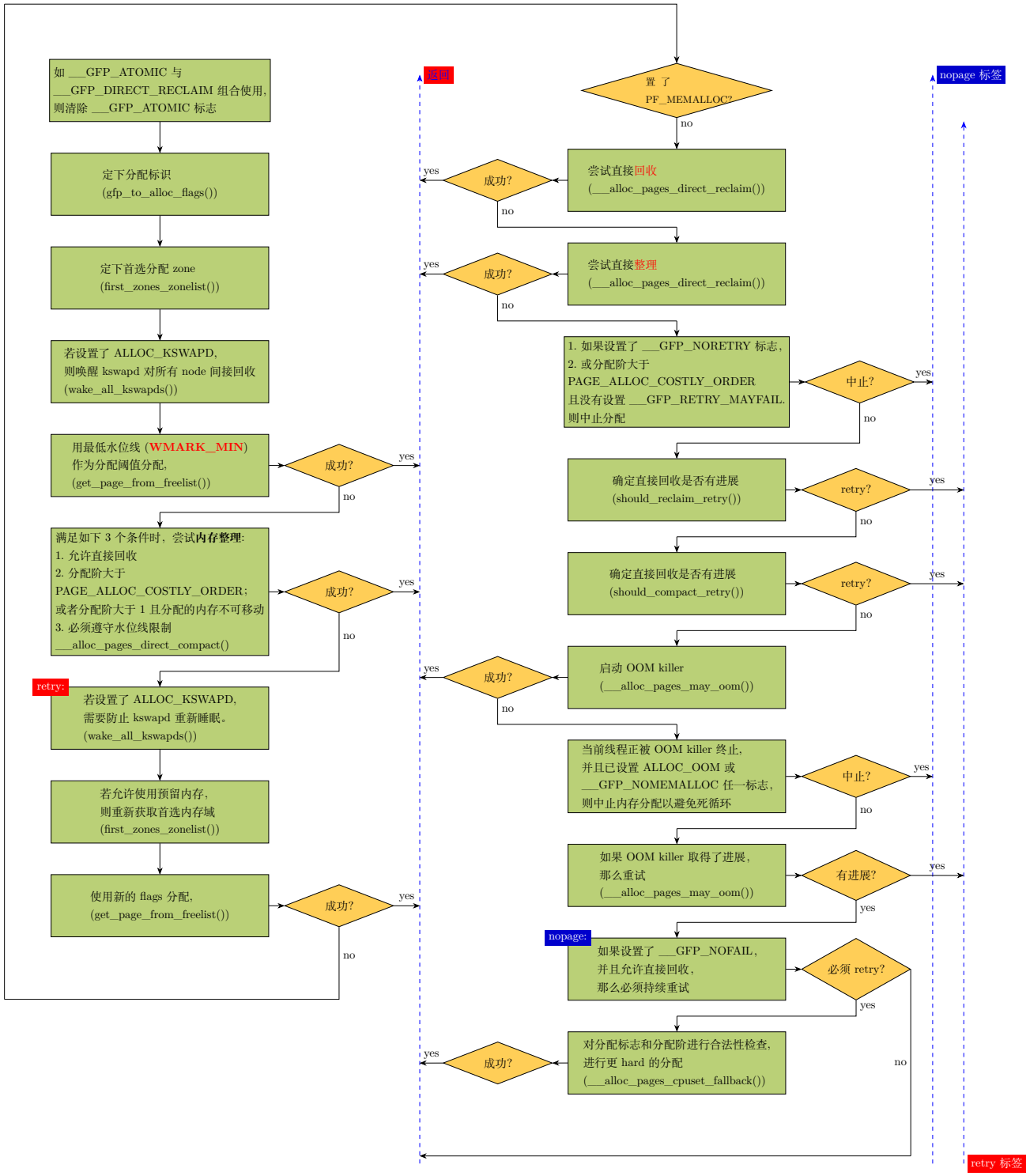
schedule() 和 *cond_reshed()* 的区别

schedule() 会无条件执行调度。而 *cond_reshed()* 会通过 *need_reshed()* 判断是否为 *true*, 如果是，才调用 *schedule()* 执行调度。*need_reshed()* 是通过 *thread_info->flags* 中 *TIF_NEED_RESHED* 位是否置位来进行判断是否要调度。

通常不推荐直接使用 *schedule()*，这样会造成频繁无意义的任务切换，开销较大。

- 4860-4861 见 4810-4813 行的说明。

___alloc__pages__slowpath() 函数执行流程图如下.



在 4726 行调用的 ___gfp_pfmemalloc_flags() 来确定是否可以使用预留内存。源码如下:

___alloc_pages_nodemask() > *___alloc_pages_slowpath()* > *___gfp_pfmemalloc_flags()*

```
4450 static inline int __gfp_pfmemalloc_flags(gfp_t gfp_mask)
4451 {
4452     if (unlikely(gfp_mask & __GFP_NOMEMALLOC))
4453         return 0;
4454     if (gfp_mask & __GFP_MEMALLOC)
```

```
4455         return ALLOC_NO_WATERMARKS;
4456     if (in_serving_softirq() && (current->flags & PF_MEMALLOC))
4457         return ALLOC_NO_WATERMARKS;
4458     if (!in_interrupt()) {
4459         if (current->flags & PF_MEMALLOC)
4460             return ALLOC_NO_WATERMARKS;
4461         else if (oom_reserves_allowed(current))
4462             return ALLOC_OOM;
4463     }
4464
4465     return 0;
4466 }
```

该函数决定了紧急预留内存能使用的场景。

- 4452-4453 传递 ___GFP_NOMEMALLOC 表示不允许使用任何紧急预留内存。

  如果传递 ___GFP_MEMALLOC 则意味着无视水位限制，可以使用全部紧急预留内存。

- 4454-4455 ___GFP_MEMALLOC 标志意味着忽略所有水位线，zone 里的所有内存都可以被申请。此时返回 ALLOC_NO_WATERMARKS。

- 4456-4457 通过在 task_struct->flags 中设置 PF_MEMALLOC 标志，可以让在当前进程执行时触发的软中断，具备 ___GFP_NOMEMALLOC 标志所具备的使用紧急内存预留区的能力。

  Linux 内核中，硬中断（hardirq）有独立的中断栈，而软中断（softirq）没有独立的栈，会复用当前进程或硬中断的栈。

  - 硬中断处理完成后，内核会检查是否有待处理的软中断，此时直接在硬中断栈上执行软中断处理函数。执行时 in_serving_softirq() 为 true，但指向当前任务的 task_struct 指针 current 为 NULL。
  - 内核空闲时、进程调度切换时触发的软中断，会复用当前 CPU 上运行进程 (current) 的内核栈。此时 in_serving_softirq() 为 true，current 指向被复用栈的进程。比如 CPU 运行 kswapd（置了 PF_MEMALLOC 标志的进程）时触发软中断，软中断就复用 kswapd 的内核栈执行。

  软中断上下文不能阻塞、不能睡眠（原子执行）。软中断是系统级高优先级任务，其内存分配失败会直接影响系统可用性。如果当前 current 进程本身就是高优先级紧急进程 (current->flags 置了 PF_MEMALLOC 标志)，此时软中断执行中的内存分配可以继承该进程的紧急内存申请权限。

- 4458-4460 如果是纯粹的进程上下文，则同样的，返回忽略水位线使用预留紧急内存的标志。

  在 4456-4457 的分支是：在紧急进程中触发的软中断获得紧急预留内存的使用权。4458-4460 的分支则是：紧急进程获得紧急预留内存的使用权。仅排除硬中断上下文是因为硬中断有自己的内存分配规则，无需这种兜底分配。

  in_serving_softirq() 为 true 时，in_interrupt() 一定为 true。因为软中断属于中断上下文的子集。

- 4461-4462 在纯粹的进程上下文中，如果当前进程是 oom killer 挑选出的牺牲者，那么该进程允许使用紧急内存。因为该进程使用的内存很快将在进程被 kill 后得到释放。这种情况返回 ALLOC_OOM.

gfp_to_alloc_flags() 主要用来将上层传入的 GFP(get free pages) 分配标志精准转换成底层伙伴系统分配时使用的水位控制标识(alloc_flags)。在快速路径没有调用它是为了在内存充足时，通过牺牲分配策略的精准度来换取低延迟。

```
4389 static inline unsigned int
4390 gfp_to_alloc_flags(gfp_t gfp_mask)
4391 {
4392     unsigned int alloc_flags = ALLOC_WMARK_MIN | ALLOC_CPUSET;
4393
4394     /*
4395      * __GFP_HIGH is assumed to be the same as ALLOC_HIGH
4396      * and __GFP_KSWAPD_RECLAIM is assumed to be the same as ALLOC_KSWAPD
4397      * to save two branches.
4398      */
4399     BUILD_BUG_ON(__GFP_HIGH != (__force gfp_t) ALLOC_HIGH);
4400     BUILD_BUG_ON(__GFP_KSWAPD_RECLAIM != (__force gfp_t) ALLOC_KSWAPD);
4401
4402     /*
4403      * The caller may dip into page reserves a bit more if the caller
4404      * cannot run direct reclaim, or if the caller has realtime scheduling
4405      * policy or is asking for __GFP_HIGH memory.  GFP_ATOMIC requests will
4406      * set both ALLOC_HARDER (__GFP_ATOMIC) and ALLOC_HIGH (__GFP_HIGH).
4407      */
4408     alloc_flags |= (__force int)
4409         (gfp_mask & (__GFP_HIGH | __GFP_KSWAPD_RECLAIM));
4410
4411     if (gfp_mask & __GFP_ATOMIC) {
4412         /*
4413          * Not worth trying to allocate harder for __GFP_NOMEMALLOC even
4414          * if it can't schedule.
4415          */
4416         if (!(gfp_mask & __GFP_NOMEMALLOC))
4417             alloc_flags |= ALLOC_HARDER;
4418         /*
4419          * Ignore cpuset mems for GFP_ATOMIC rather than fail, see the
4420          * comment for __cpuset_node_allowed().
4421          */
4422         alloc_flags &= ~ALLOC_CPUSET;
4423     } else if (unlikely(rt_task(current)) && !in_interrupt())
4424         alloc_flags |= ALLOC_HARDER;
4425
4426     alloc_flags = current_alloc_flags(gfp_mask, alloc_flags);
4427
4428     return alloc_flags;
4429 }
```

- 4392 设置页分配时的限定标识。这里是设置 default 值，下面可能会重置成其他值。

  ALLOC_WMARK_MIN 表示在该 zone 上的页分配必须在 WMARK_MIN 水位线之上。这也是慢速路径的水位分配区间。

- 4399-4400 内核约定 ___GFP_HIGH、___GFP_KSWAPD_RECLAIM 这两个上层标识的标识位分别和底层伙伴系统对应的 ALLOC_HIGH、ALLOC_KSWAPD 标识的标识位一致。这样就省去了两个 if 代码分支: 通过判断上层标识 ___GFP_XXX，再换成底层对应的标识赋给 alloc_flags。

  但是这种方式有一个限定条件，就是替换的标识和原始标识虽然宏名称不同，但是他们对应位在源和目的标识变量里都是在同一个 bit 位置。

  代码这里加了一个宏用于编译时报警，当源和目的标识的 bit 位置不是在同一位置就编译报错。目的也是为了避免源和目的标识的宏值在代码演进过程中发生变化。

- 4408-4409 由前面两行代码的讨论，我们可以知道现在直接用上层的 GFP 标识给底层水位控制标识变量赋值是可行的。因为这两个标识位对应在底层标识的位置是相同的，无需转化。

- 4411-4417 调用者常常无法执行直接内存回收，或是由于采用实时调度策略，或是申请 ___GFP_HIGH 类型的内存，那么页分配器会允许调用者更多的使用页面紧急预留内存。

  GFP_ATOMIC 请求会同时置位 ALLOC_HIGH 和 ALLOC_HARDER。ALLOC_HARDER 会将水位阈值在 ALLOC_HIGH 基础上进一步降低。

  但是，如果置了 ___GFP_NOMEMALLOC 标识，即使本次上下文处于原子的，不可休眠的场景，也不开启 ALLOC_HARDER 进行激进分配。

- 4422 对于原子紧急分配忽略 cpuset 内存 node 节点的限制。因此清除 ALLOC_CPUSET 标识。

- 4423-4424 如果上层没有传递原子分配 (___GFP_ATOMIC) 请求，但是当前线程是实时线程，并且当前处在 task 上下文中，那么允许置 ALLOC_HARDER 来从预留的紧急内存区获得空闲内存。因为实时任务对延迟要求极高不能阻塞。

- 4426 在开启 CONFIG_CMA 配置的情况下，确定是否需要加上 ALLOC_CMA 标识。(函数见后面)

==========current_alloc_flags()

当前进程没有禁止从 CMA 区域分配内存，并且本次请求的是页的迁移类型是 MIGRATE_MOVABLE。那么运行从 CMA 区域分配内存。

4.2.2 prepare_alloc_pages()

___alloc_pages_nodemaskn() > *prepare_alloc_pages()*

```
4867 static inline bool prepare_alloc_pages(gfp_t gfp_mask, unsigned int order,
4868         int preferred_nid, nodemask_t *nodemask,
4869         struct alloc_context *ac, gfp_t *alloc_mask,
4870         unsigned int *alloc_flags)
4871 {
4872     ac->highest_zoneidx = gfp_zone(gfp_mask);
4873     ac->zonelist = node_zonelist(preferred_nid, gfp_mask);
4874     ac->nodemask = nodemask;
4875     ac->migratetype = gfp_migratetype(gfp_mask);
4876 
4877     if (cpusets_enabled()) {
4878         *alloc_mask |= __GFP_HARDWALL;
4879         /*
4880          * When we are in the interrupt context, it is irrelevant
4881          * to the current task context. It means that any node ok.
4882          */
4883         if (!in_interrupt() && !ac->nodemask)
4884             ac->nodemask = &cpuset_current_mems_allowed;
4885         else
4886             *alloc_flags |= ALLOC_CPUSET;
4887     }
4888 
4889     fs_reclaim_acquire(gfp_mask);
4890     fs_reclaim_release(gfp_mask);
4891 
4892     might_sleep_if(gfp_mask & __GFP_DIRECT_RECLAIM);
4893 
4894     if (should_fail_alloc_page(gfp_mask, order))
4895         return false;
4896 
4897     *alloc_flags = current_alloc_flags(gfp_mask, *alloc_flags);
```

```
4898
4899        /* Dirty zone balancing only done in the fast path */
4900        ac->spread_dirty_pages = (gfp_mask & __GFP_WRITE);
4901
4902        /*
4903         * The preferred zone is used for statistics but crucially it is
4904         * also used as the starting point for the zonelist iterator. It
4905         * may get reset for allocations that ignore memory policies.
4906         */
4907        ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
4908                        ac->highest_zoneidx, ac->nodemask);
4909
4910        return true;
4911    }
```

- 4872 从 gfp_mask 掩码中得到内存区域修饰符，然后通过内存区域修饰符获取到此次内存分配的首选 zone 的索引编号 (该函数的解析在后面)。
- 4873 通过 gfp_mask 获取当前节点内存域的 zonelist 备用链表 (其实是数组)。zone 链表最多包含 $MAXNODES \times MAX_NR_ZONES$ 个 zone。每个 NUMA node 有两个 zonelist，一个链表包含了内存中的所有 zone，另一个只包含了当前 zonelist 所属 node 的所有 zone。除非 NUMA node 的 gfp_mask 中置了 ___GFP_THISNODE 标志，否则获取的 zonelist 数组是索引值为 ZONELIST_FALLBACK(即 0) 的 zonelist。
- 4874 NUMA 节点的限制掩码。掩码中每一位代表一个 NUMA 节点，如果对应 bit 是 1，表示该节点允许参与本次内存分配。后续页分配遍历 NUMA 节点时，只在掩码标记的节点上分配内存
- 4875 从 gfp_mask 掩码中获取本次申请内存的迁移属性，迁移属性分为：不可迁移 (unmovable)，可回收 (reclaimable)，可迁移 (movable)。
- 4877-4887 如果使用 cgroup 将进程绑定限制在了某些 CPU 上，那么内存分配只能在这些被绑定 CPU 相关联的 NUMA 节点中进行。
- 4892 如果设置了允许直接内存回收，那么当前内存分配进程则可能会休眠导致被重新调度。判断本次分配是否允许休眠，如果允许，就校验当前场景是否允许休眠；若在自旋锁/中断等禁止休眠场景下执行，打印栈告警，提前捕获内存分配错误。
- 4897 只有允许 CMA、且申请分配可迁移页面时，才打开 ALLOC_CMA 标记，让分配器可以从 CMA 内存区分配页面。

  该行只在开启了 CONFIG_CMA 配置项时才有效。

  CMA 区域的设计初衷：仅允许存放可移动页，需要连续大块内存时，可以把这些页迁移走，腾出连续物理内存给设备驱动。不可迁移页不能放 CMA zone，否则无法回收整块 CMA。
- 4900 判断本次分配是否属于写 *IO* 的执行路径。开启脏页均衡后，分配时尽量把新脏页分配到不同 zone，避免单 zone 脏页过载。

  脏页均衡只在快速路径 (get_page_from_freelist 快速空闲页查找路径) 执行。
- 4907 获取最高优先级的内存区域 zone 后续内存分配则首先会在该内存区域中进行分配。

  首选 zone 会作为 zonelist 迭代器的遍历起点；对于忽略内存策略的内存分配，该起点会被重新覆盖重置。

gfp_zone() 会根据入参 GFP, 来返回这个分配掩码下允许分配的最高 zone 区域的索引编号。

4.2.2.1 gfp_zone()

__alloc_pages_nodemaskn() → *prepare_alloc_pages()* → *gfp_zone()*

```
450 static inline enum zone_type gfp_zone(gfp_t flags)
451 {
452     enum zone_type z;
453     int bit = (__force int) (flags & GFP_ZONEMASK);
454
455     z = (GFP_ZONE_TABLE >> (bit * GFP_ZONES_SHIFT)) &
456                      ((1 << GFP_ZONES_SHIFT) - 1);
457     VM_BUG_ON((GFP_ZONE_BAD >> bit) & 1);
458     return z;
459 }
```

- 453 gfp 中低 4 位的标志组合用来表示从哪些 zone 区申请内存页, 其掩码为GFP_ZONEMASK, 目的是筛选出 gfp 标志里和内存域限定相关的 bit 位。宏定义如下:

  ```
  #define GFP_ZONEMASK    (__GFP_DMA|__GFP_HIGHMEM|__GFP_DMA32|__GFP_MOVABLE)
  ```

  代表各 zone 的内存区域修饰符为:

  ```
  #define __GFP_DMA    ((__force gfp_t)___GFP_DMA)
  #define __GFP_HIGHMEM    ((__force gfp_t)___GFP_HIGHMEM)
  #define __GFP_DMA32 ((__force gfp_t)___GFP_DMA32)
  #define __GFP_MOVABLE    ((__force gfp_t)___GFP_MOVABLE)  /* ZONE_MOVABLE allowed */
  ```

  可以看到上面没有定义***___GFP_NORMAL***, 如果不指定物理内存的分配区域，那么内核会默认从 ZONE_NORMAL 区域中分配内存。如果 ZONE_NORMAL 区域中的空闲内存不够，内核则会备选到 ZONE_DMA 区域中分配。

  *___GFP_MOVABLE*是页面存放分类标记，不限制可用 zone 范围。只代表分配出来的页面归入 *MT_MOVABLE* 迁移类型链表, 可移动。仅当同时开 ___GFP_HIGHMEM 时，分配器额外允许申请 ZONE_MOVABLE。

  而 ___GFP_DMA、___GFP_DMA32、___GFP_HIGHMEM 用于限定分配可访问的物理内存域，分别表示:

  - *___GFP_DMA*：仅允许申请 ZONE_DMA
  - *___GFP_DMA32*：允许申请 ZONE_DMA32 + ZONE_DMA
  - *___GFP_HIGHMEM*：允许申请 ZONE_NORMAL + ZONE_HIGHMEM

  四个标志理论上有 16 种组合 ($C_4^0 + C_4^1 + C_4^2 + C_4^3 + C_4^4 = 16$), 不过 ____GFP_DMA,____GFP_HIGHMEM 和 ____GFP_DMA32 这三个标志中的任何两个不能同时存在于 gfp 标志中。因为这样会造成申请的内存区矛盾。

  所有标志组合如下 (其中err部分标示的是非法 gfp 组合):

值	____GFP_MOVABLE	____GFP_DMA32	____GFP_HIGHMEM	____GFP_DMA	首选分配区域
0x0	0	0	0	0	ZONE_NORMAL
0x1	0	0	0	1	ZONE_NORMAL 或 ZONE_DMA
0x2	0	0	1	0	ZONE_NORMAL 或 ZONE_HIGHMEM
0x3	0	0	1	1	error
0x4	0	1	0	0	ZONE_NORMAL 或 ZONE_DMA32
0x5	0	1	0	1	error
0x6	0	1	1	0	error
0x7	0	1	1	1	error
0x8	1	0	0	0	ZONE_NORMAL 或 ZONE_MOVABLE
0x9	1	0	0	1	ZONE_NORMAL 或 (ZONE_DMA+ZONE_MOVABLE)
0xA	1	0	1	0	ZONE_MOVABLE
0xB	1	0	1	1	error
0xC	1	1	0	0	ZONE_DMA
0xD	1	1	0	1	error
0xE	1	1	1	0	error
0xF	1	1	1	1	error

将表中 error 即不允许出现的标志组合去掉，剩下的对应各种内存分配 zone 区组合的 $(16-8=8)$ 种标志的组合可以组成 GFP_ZONE_TABLE 表. 其中在不同条件编译下，下面的宏会有不同定义：

OPT_ZONE_DMA 代表 ____GFP_NORMAL 或者 ____GFP_DMA

OPT_ZONE_HGIHMEM 代表 ____GFP_HIGHMEM 或者 ____GFP_NORMAL

OPT_ZONE_DMA32 代表 ____GFP_NORMAL 或者 ____GFP_DMA32

代码如下：

```
#ifdef CONFIG_HIGHMEM
#define OPT_ZONE_HIGHMEM ZONE_HIGHMEM
#else
#define OPT_ZONE_HIGHMEM ZONE_NORMAL
#endif

#ifdef CONFIG_ZONE_DMA
#define OPT_ZONE_DMA ZONE_DMA
#else
#define OPT_ZONE_DMA ZONE_NORMAL
#endif

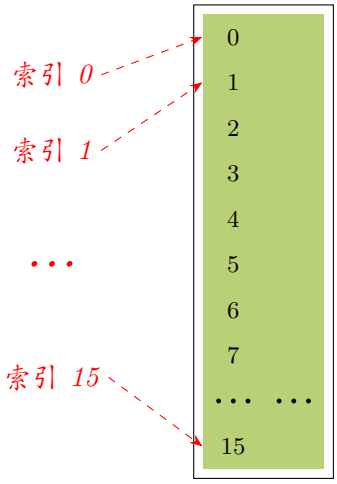
#ifdef CONFIG_ZONE_DMA32
#define OPT_ZONE_DMA32 ZONE_DMA32
#else
#define OPT_ZONE_DMA32 ZONE_NORMAL
#endif
```

内核基本分为 ZONE_DMA,ZONE_DMA32,ZONE_NORMAL 和 ZONE_HIGHMEM 4 个分配区，对应宏值分别是 $0,1,2,3$。所以最少需要 2 bit 位宽的内存才能区别或存储对应 ($2^2=4$ 个, 最大值不 >3) 分配区的宏值。

而系统是通过传递不同的 gfp 掩码来选择不同的首选 zone。系统为了快速通过掩码找到对应的分配 zone, 在代码里创建了静态的表格，这样在运行时不需要通过计算，而只要查表就能得到首选 zone。

换言之，也就是不同的 gfp 掩码组合最终得到了不同的数值，可以以这些数值为索引查表得到首选 zone 的索引编号。又因为，(前面已经提过)zone 的索引编号至少需要 2 bit。所以，该表格的每项数

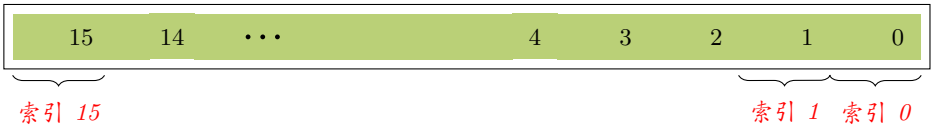
据都是 2 bit。至此，我们成功设计出了表格 ***GFP_ZONE_TABLE***。如图:



说明:
1. 从讨论可知一共有 16 种可能的组合，所以数组最多储存16个数据项。
2. 每个数据项需要储存
ZONE_DMA,ZONE_DMA32,ZONE_NORMAL 和 ZONE_HIGHMEM
四个宏值之一，数值分别是 0、1、2、3。所以每项需要是2 bit。
3. 图中每格表示 1 bit; 每行表示 1 项 (两格)。

但是，更进一步，内核按这些 2 bit 项在表格中索引顺序，将这些 2 bit 数据项拼接在一起，形成一个整型常量。这时，我们要查找表格中的 2 bit 项 (首选 zone 的索引号) 时，可以将原来表格的 索引 $\times 2$ 作为偏移来查找表项。

譬如，gfp 掩码组合成的值是 $\boldsymbol{n}$, 在原先的表格中只要找到索引是 $\boldsymbol{i}$ 的成员即可。在现在的表格中，只要偏移 $\ll (2 \cdot n)$ 即可找到对应的 2 bit 数据项。



说明:
1. 从讨论可知一共有 16 种可能的组合，所以数组最多储存16个数据项。
2. 每个数据项需要储存 ZONE_DMA,ZONE_DMA32,ZONE_NORMAL 和 ZONE_HIGHMEM
四个宏值之一，数值分别是 0、1、2、3。所以每项需要是2 bit。
3. 图中每格表示 1 bit。
4. 图中的数值代表数据项的索引而不是 bit 位。每个数据项占 2 bit

对应在代码里即是将 (标志组合的值 $\times$ *ZONES_SHIFT*) 做为位偏移，*ZONE_SHIFT* $= 2$。将首选分配 zone 区对应的宏值填入对应位偏移处位宽为 2bit 的选项里，从而创建 GFP_ZONE_TABLE 表格。即使组合标志位每个 bit 都是 1，也只是 0xf。$0xf \times 2 = 15 \times 2 = 30$, 所以 *GFP_ZONE_TABLE* 占用的内存不会超过 32 bit 长度。

GFP_ZONE_TABLE 常量 (或称表格) 的定义如下:

```
#define GFP_ZONE_TABLE ( \
    (ZONE_NORMAL << 0 * GFP_ZONES_SHIFT)                        \
    | (OPT_ZONE_DMA << ___GFP_DMA * GFP_ZONES_SHIFT)                \
```

```
    | (OPT_ZONE_HIGHMEM << ___GFP_HIGHMEM * GFP_ZONES_SHIFT)           \
    | (OPT_ZONE_DMA32 << ___GFP_DMA32 * GFP_ZONES_SHIFT)              \
    | (ZONE_NORMAL << ___GFP_MOVABLE * GFP_ZONES_SHIFT)            \
    | (OPT_ZONE_DMA << (___GFP_MOVABLE | ___GFP_DMA) * GFP_ZONES_SHIFT)    \
    | (ZONE_MOVABLE << (___GFP_MOVABLE | ___GFP_HIGHMEM) * GFP_ZONES_SHIFT)\
    | (OPT_ZONE_DMA32 << (___GFP_MOVABLE | ___GFP_DMA32) * GFP_ZONES_SHIFT)\
)
```

从上面代码页可以看出：

(1) 只要掩码 flags 中设置了 ___GFP_DMA，则不管 ___GFP_HIGHMEM 有没有设置，内存分配都只会在 ZONE_DMA 区域中分配。

(2) 如果掩码只设置了 ZONE_HIGHMEM，则优先在 ZONE_HIGHMEM 区域中进行分配，如果内存不够则备选到 ZONE_NORMAL 中请求内存，如果还是不够则进一步备选至 ZONE_DMA 中分配。

(3) 如果掩码既没有设置 ZONE_HIGHMEM 也没有设置 ___GFP_DMA，则默认优先从 ZONE_NORMAL 区域中进行内存分配，如果内存不够则备选至 ZONE_DMA 区域中分配。

(4) 单独设置 ___GFP_MOVABLE 其实并不会影响内核的分配策略，如果想要让内核在 ZONE_MOVABLE 区域中分配内存，需要同时指定 ___GFP_MOVABLE 和 ___GFP_HIGHMEM 。

上面不同的 gfp_t 掩码设置和其对应的内存区域备选策略汇总列表如下：

gfp_t 掩码	内存区域备选策略
未设置	*ZONE_NORMAL -> ZONE_DMA*
___GFP_DMA	*ZONE_DMA*
___GFP_DMA & ___GFP_HIGHMEM	*ZONE_DMA*
___GFP_HIGHMEM	*ZONE_HIGHMEM -> ZONE_NORMAL -> ZONE_DMA*

组成的 GFP_ZONE_TABLE 表格中成员映射如下 (绿色是有效值存储空间，红色是空洞)：



- 455 根据内存分配组合标志得到对应内存分配的首选 zone 的信息，并返回。

ZONE_MOVABLE 是内核定义的一个虚拟内存区域，目的是避免内存碎片和支持内存热插拔。上述的 ZONE_HIGHMEM，ZONE_NORMAL，ZONE_DMA 才是真正的物理内存区域，ZONE_MOVABLE 虚拟内存区域中的物理内存来自于上述三个物理内存区域。在 32 位系统中 ZONE_MOVABLE 虚拟内存区域中的物理内存页来自于 ZONE_HIGHMEM。在 64 位系统中 ZONE_MOVABLE 虚拟内存区域中的物理内存页来自于 ZONE_NORMAL 或者 ZONE_DMA 区域。

mm/page_alloc.c :: find_usable_zone_for_movable()

```
6452 static void __init find_usable_zone_for_movable(void)
6453 {
6454     int zone_index;
6455     for (zone_index = MAX_NR_ZONES - 1; zone_index >= 0; zone_index--) {
```

```
6456            if (zone_index == ZONE_MOVABLE)
6457                continue;
6458
6459            if (arch_zone_highest_possible_pfn[zone_index] >
6460                    arch_zone_lowest_possible_pfn[zone_index])
6461                break;
6462        }
6463
6464        VM_BUG_ON(zone_index == -1);
6465        movable_zone = zone_index;
6466    }
```

下面这段代码用于确定 ZONE_MOVABLE 从哪个内存区域借用物理页作为 ZONE_MOVABLE 区。由代码实现可以看出最高内存区域的 zone 区被划分出内存用作虚拟的内存区域 ZONE_MOVABLE. 全局变量 movable_zone 保存借用区域的索引。

至于用多大的内存作为 ZONE_MOVABLE 是在初始化时使用内核引导参数 “movablecore=nn[KMG]” 指定 ZONE_MOVABLE zone 的大小。参数 “kernelcore=” 与”movablecore=” 互补，指定保留给内核（不可移动页）的内存比例，剩余部分自动分配给 ZONE_MOVABLE。默认情况下，如果未通过命令行参数（如 movablecore 或 kernelcore）显式配置，不会启用 ZONE_MOVABLE 内存域。

默认认为巨型页是不可移动的，不会从可移动区域分配巨型页，可以通过文件 “/proc/sys/vm/**hugepages_treat_as_movable**” 配置允许从可移动区域分配巨型页。

4.2.3 alloc_flags_nofragment()

再回到下面代码。这个函数用于设置避免内存碎片化的相关分配标识。

__alloc_pages_nodemaskn() > *alloc_flags_nofragment()*

```
3737 static inline unsigned int
3738 alloc_flags_nofragment(struct zone *zone, gfp_t gfp_mask)
3739 {
3740         unsigned int alloc_flags;
3741
3742         /*
3743          * __GFP_KSWAPD_RECLAIM is assumed to be the same as ALLOC_KSWAPD
3744          * to save a branch.
3745          */
3746         alloc_flags = (__force int) (gfp_mask & __GFP_KSWAPD_RECLAIM);
3747
3748 #ifdef CONFIG_ZONE_DMA32
3749         if (!zone)
3750                 return alloc_flags;
3751
3752         if (zone_idx(zone) != ZONE_NORMAL)
3753                 return alloc_flags;
3754
3755         /*
3756          * If ZONE_DMA32 exists, assume it is the one after ZONE_NORMAL and
3757          * the pointer is within zone->zone_pgdat->node_zones[]. Also assume
3758          * on UMA that if Normal is populated then so is DMA32.
3759          */
3760         BUILD_BUG_ON(ZONE_NORMAL - ZONE_DMA32 != 1);
3761         if (nr_online_nodes > 1 && !populated_zone(--zone))
3762                 return alloc_flags;
3763
3764         alloc_flags |= ALLOC_NOFRAGMENT;
3765 #endif /* CONFIG_ZONE_DMA32 */
3766         return alloc_flags;
3767 }
```

该函数根据上层(非用户空间) 传递的 GFP 掩码标识设置将在底层分配时页分配器用到的 flag 标识。

- 3746 ___GFP_KSWAPD_RECLAIM 标志位和 ALLOC_KSWAPD 标志位的值相等，含义也相同。差异在于 ___GFP_XXX 被申请页的用户 KPI 使用，而 ALLOC_XXX 是底层页分配器内部使用的标志位宏。

  这里将用户 (不是指用户空间) 传入的上层分配掩码里，只保留 ___GFP_KSWAPD_RECLAIM 位，其余位清 0。因为这个 bit 标志位恰好和底层的页分配器使用的 alloc_flag 标志里含义相同的 ALLOC_KSWAPD 位是同一位置，所以无需通过判断、变换位、再置 alloc_flags。直接一步到位赋值 alloc_flags 的该 bit 位。

  该标志位表示：当空闲页数量降到 ≤ 低水位 WMARK_LOW 时，唤醒页回收线程 kswapd。

- 3749-3750 如果传入的首选 zone 为 NULL, 那么无对应的内存区域 (zone) 需要做碎片保护，直接返回。

- 3752-3753 如果传入的首选 zone 不是 ZONE_NORMAL, 也直接退出。因为碎片保护主要针对该区。

  不可移动页 (unmovable) 是 ZONE_NORMAL 区碎片产生的根源。内核.txt,.data 等全部放在 normal 区，它是客观的固定存在。没法移走，我们只能接受 normal 区天然的这些固定分隔点。而可移动页是加剧碎片，堵死内存整理的催化剂。因为可移动页会频繁分配释放，使空间支离破碎，内存整理失效。

  因此，ZONE_NORMAL 区碎片保护的核心是: 减少可移动页混杂到 normal 区，避免不可移动页区域之间分配可移动页，从而形成永久碎片。仅在 ZONE_MOVABLE 区无内存空闲时，才从 ZONE_NORMAL 区分配可移动页。简言之, 就是限制可移动页进入 ZONE_NORMAL 区。

- 3760 编译阶段锁死 ZONE_DMA32 的编号顺序是 $ZONE_NORMAL - 1$。

  只有 ZONE_NORMAL 和 ZONE_DMA32 之间分摊分配，才能缓解碎片压力。ZONE_DMA 容量极小不能分摊内存压力。ZONE_HIGHMEM 不能提前把压力下移到低端内存，会造成 lowmem 内存紧缺。

  这里分配碎片压力和 zone 内存不足备选 (fallback) 到下一个 zone 不是一个概念。

- 3761-3762 因为很多服务器的 NUMA 节点内存的 zone 区数量不对称，也就是说可能不一定有 ZONE_DMA32 等。因此对于这种情况就提前退出，节省开销。

- 3764 程序跑到这里，意味着首选 zone 是 normal 内存区。分配 ZONE_NORMAL 内存时添加 ALLOC_NOFRAGMENT 标志位，代表着有" 避免碎片化" 的要求。允许分摊到 ZONE_DMA32, 减轻 normal 碎片压力。

4.2.4 get_page_from_freelist()

get_page_from_freelist() 在 zonelist 中从首选 zone 开始遍历匹配 nodemask 的所有 zone，获取指定数目的连续的页帧：

- 在遍历 zone 时，如果 zone 当前空闲内存 − 申请的内存 $< WMARK_LOW$, 则该 zone 会触发内存回收。

- 首先只会尝试从首选 zone 区中获取页帧。

- 如果首选 zone 无法满足获取，则会遍历备选 zonelist 中的 zone。

__alloc_pages_nodemaskn() > *get_page_from_freelist()*

```
3787 static struct page *
3788 get_page_from_freelist(gfp_t gfp_mask, unsigned int order, int alloc_flags,
3789                                                 const struct alloc_context *ac)
3790 {
3791         struct zoneref *z;
3792         struct zone *zone;
3793         struct pglist_data *last_pgdat_dirty_limit = NULL;
3794         bool no_fallback;
3795
3796 retry:
3797         /*
3798          * Scan zonelist, looking for a zone with enough free.
3799          * See also __cpuset_node_allowed() comment in kernel/cpuset.c.
3800          */
3801         no_fallback = alloc_flags & ALLOC_NOFRAGMENT;
3802         z = ac->preferred_zoneref;
3803         for_next_zone_zonelist_nodemask(zone, z, ac->highest_zoneidx,
3804                                         ac->nodemask) {
3805                 struct page *page;
3806                 unsigned long mark;
3807
3808                 if (cpusets_enabled() &&
3809                         (alloc_flags & ALLOC_CPUSET) &&
3810                         !__cpuset_zone_allowed(zone, gfp_mask))
3811                                 continue;
3812                 /*
3813                  * When allocating a page cache page for writing, we
3814                  * want to get it from a node that is within its dirty
3815                  * limit, such that no single node holds more than its
3816                  * proportional share of globally allowed dirty pages.
3817                  * The dirty limits take into account the node's
3818                  * lowmem reserves and high watermark so that kswapd
3819                  * should be able to balance it without having to
3820                  * write pages from its LRU list.
3821                  *
3822                  * XXX: For now, allow allocations to potentially
3823                  * exceed the per-node dirty limit in the slowpath
3824                  * (spread_dirty_pages unset) before going into reclaim,
3825                  * which is important when on a NUMA setup the allowed
3826                  * nodes are together not big enough to reach the
3827                  * global limit.  The proper fix for these situations
3828                  * will require awareness of nodes in the
3829                  * dirty-throttling and the flusher threads.
3830                  */
3831                 if (ac->spread_dirty_pages) {
3832                         if (last_pgdat_dirty_limit == zone->zone_pgdat)
3833                                 continue;
3834
3835                         if (!node_dirty_ok(zone->zone_pgdat)) {
3836                                 last_pgdat_dirty_limit = zone->zone_pgdat;
3837                                 continue;
3838                         }
3839                 }
3840
3841                 if (no_fallback && nr_online_nodes > 1 &&
3842                     zone != ac->preferred_zoneref->zone) {
3843                         int local_nid;
3844
3845                         /*
3846                          * If moving to a remote node, retry but allow
3847                          * fragmenting fallbacks. Locality is more important
3848                          * than fragmentation avoidance.
```

```
3849                            */
3850                           local_nid = zone_to_nid(ac->preferred_zoneref->zone);
3851                           if (zone_to_nid(zone) != local_nid) {
3852                                   alloc_flags &= ~ALLOC_NOFRAGMENT;
3853                                   goto retry;
3854                           }
3855                   }
3856
3857                   mark = wmark_pages(zone, alloc_flags & ALLOC_WMARK_MASK);
3858                   if (!zone_watermark_fast(zone, order, mark,
3859                                            ac->highest_zoneidx, alloc_flags,
3860                                            gfp_mask)) {
3861                           int ret;
3862
3863 #ifdef CONFIG_DEFERRED_STRUCT_PAGE_INIT
3864                           /*
3865                            * Watermark failed for this zone, but see if we can
3866                            * grow this zone if it contains deferred pages.
3867                            */
3868                           if (static_branch_unlikely(&deferred_pages)) {
3869                                   if (_deferred_grow_zone(zone, order))
3870                                           goto try_this_zone;
3871                           }
3872 #endif
3873                           /* Checked here to keep the fast path fast */
3874                           BUILD_BUG_ON(ALLOC_NO_WATERMARKS < NR_WMARK);
3875                           if (alloc_flags & ALLOC_NO_WATERMARKS)
3876                                   goto try_this_zone;
3877
3878                           if (node_reclaim_mode == 0 ||
3879                               !zone_allows_reclaim(ac->preferred_zoneref->zone, zone))
3880                                   continue;
3881
3882                           ret = node_reclaim(zone->zone_pgdat, gfp_mask, order);
3883                           switch (ret) {
3884                           case NODE_RECLAIM_NOSCAN:
3885                                   /* did not scan */
3886                                   continue;
3887                           case NODE_RECLAIM_FULL:
3888                                   /* scanned but unreclaimable */
3889                                   continue;
3890                           default:
3891                                   /* did we reclaim enough */
3892                                   if (zone_watermark_ok(zone, order, mark,
3893                                           ac->highest_zoneidx, alloc_flags))
3894                                           goto try_this_zone;
3895
3896                                   continue;
3897                           }
3898                   }
3899
3900 try_this_zone:
3901                   page = rmqueue(ac->preferred_zoneref->zone, zone, order,
3902                                  gfp_mask, alloc_flags, ac->migratetype);
3903                   if (page) {
3904                           prep_new_page(page, order, gfp_mask, alloc_flags);
3905
3906                           /*
3907                            * If this is a high-order atomic allocation then check
3908                            * if the pageblock should be reserved for the future
3909                            */
3910                           if (unlikely(order && (alloc_flags & ALLOC_HARDER)))
3911                                   reserve_highatomic_pageblock(page, zone, order);
3912
```

```
3913                        return page;
3914                } else {
3915 #ifdef CONFIG_DEFERRED_STRUCT_PAGE_INIT
3916                        /* Try again if zone has deferred pages */
3917                        if (static_branch_unlikely(&deferred_pages)) {
3918                                if (_deferred_grow_zone(zone, order))
3919                                        goto try_this_zone;
3920                        }
3921 #endif
3922                }
3923        }
3924
3925        /*
3926         * It's possible on a UMA machine to get through all zones that are
3927         * fragmented. If avoiding fragmentation, reset and try again.
3928         */
3929        if (no_fallback) {
3930                alloc_flags &= ~ALLOC_NOFRAGMENT;
3931                goto retry;
3932        }
3933
3934        return NULL;
3935 }
```

get_page_from_freelist () 通过 for_next_zone_zonelist_nodemask 来遍历 NUMA 当前 node 以及备用 node的内存 zone 区域 (zonelist)，扫描顺序是：首先是 prefered_zone。也就是首选 zone。然后按照 zone_type 从高到低，逐个通过 zone_watermark_fast 检查这些内存区域 zone 中的剩余空闲内存容量是否在指定的水位线 mark 之上。如果满足水位线的要求则直接调用 rmqueue 进入伙伴系统分配内存。

如果当前正在遍历的 zone 中剩余空闲内存容量在指定的水位线 mark 之下，就需要通过 node_reclaim 触发内存回收。随后通过 zone_watermark_ok 检查，在经历了内存回收后，内核是否回收到了能满足本次分配需求的足够内存。如果已经回收了足够内存，则 $zone_watermark_ok = true$ 随后跳转到 try_this_zone 分支，在本内存区域 zone 中分配内存。否则继续遍历下一个 zone。

get_page_from_freelist 函数的内存分配逻辑会考虑到内存水位线，满足内存分配要求的物理内存 zone 中的剩余空闲内存容量必须在指定内存水位线之上。否则内核认为内存不足，不能进行内存分配。

- 3803 从首选 zone 开始，扫描备用 zone 列表中满足条件的 zone：“zone 类型小于或等于首选 zone，并且 zone 所在 node 可用 (在内存节点掩码中已经置位)”。
- 3808-3811 如果内核使能了 cpuset 功能，并且置了 ALLOC_CPUSET 标志位。那么，在 cpuset 不允许当前进程从目标 node 分配时，放弃从该 zone 分配，继续下次迭代。
- 3831-3839 spread_dirty_pages 标志表示是否允许” 匀摊” 脏页。如果允许脏页 “匀摊”，则会把脏页的数目尽量平摊到各个 node，避免某个 node 上脏页过多。在允许 “匀摊” 的情况下，分配用于写的页时，要检查当前node内存的脏页数量是否超过限制。如果超过限制，则不从当前zone分配页，重新迭代。
- 3841-3855

  内存分配时，系统会根据迁移类型（如 MIGRATE_UNMOVABLE、MIGRATE_MOVABLE 等）管理页面。若当前指定的首选迁移类型内存不足，默认会尝试从其他迁移类型内存块中备选 (fallback)。但是如果设置了***ALLOC_NOFRAGMENT***标志，则会阻止这种备选行为，强制页分配器仅在请求的原始迁移类型中分配。通过限制备选迁移类型，确保内存区域的一致性，降低碎片风险，尤其适用于对连续内存敏感的场景 (如 DMA 或大页分配)。

但是，内存访问的局部性比碎片化更重要。因为访问本地内存的速度要比访问远端，也就是其它 node 的内存要快得多。若因为无法分配到内存页，而导致需要备选到其它 node 节点的 zone，则会得不偿失。

在上面的流程中，若因设置了***ALLOC_NOFRAGMENT***标志导致需要备选到其它 NUMA 节点的 zone。那么，现在考虑去掉该标志，放宽对**碎片化**的限制，重新尝试从本地 node 的 zone 中申请满足要求的 page。因此，当枚举的目标 zone 和首选 zone 在不同 node, 则通过 3852 行对分配标志位***ALLOC_NOFRAGMENT***清 0(从远端节点申请内存的开销比在本地申请导致碎片化代价更大)，然后跳转到 retry 处去重新尝试。

ALLOC_NOFRAGMENT有两个作用:

- 同 NUMA 节点的 ZONE_NORMAL 和 ZONE_DMA32 内存区分流。

  开启该标志后，ZONE_NORMAL 会主动分摊迁移类型为 movable/reclaimable 的页的申请到 ZONE_DMA32 上分配。降低 normal 区域产生的碎片密度。

- 小块分配禁止跨不同迁移类型页块借内存。严格隔离不同迁移类型的空闲链表。

  分配阶数 < pageblock_order 大小的页块分配不允许借用不同迁移类型的页。防止小块的申请将一个完整页块 (pageblock)拆碎，影响之后的大页分配。

- 3858-3898 检查 zone 的水位线是否大于指定的水位线，满足该条件才从当前 zone 分配。如果是 ALLOC_NO_WATERMARKS 标识，则不管水位线直接分配。

  如果（zone 的空闲页数 - 申请的页数）< 水位线，则如下处理:

  3875-3876 如果函数的调用者要求不检查水位线，那么可以从当前 zone 分配页。

  3878-3880 如果没有开启 node 回收，或者当前 node 和首选 node 之间的距离大于回收距离，那么不从该 zone 分配页。

  3883-3897 从 node 回收没有映射到进程虚拟空间的文件页和块分配器申请的页，然后重新检查水位线。如果还是小于水位线，则不从该 zone 分配页。

- 3901 从当前 zone 分配页。requeue() 函数见后面。

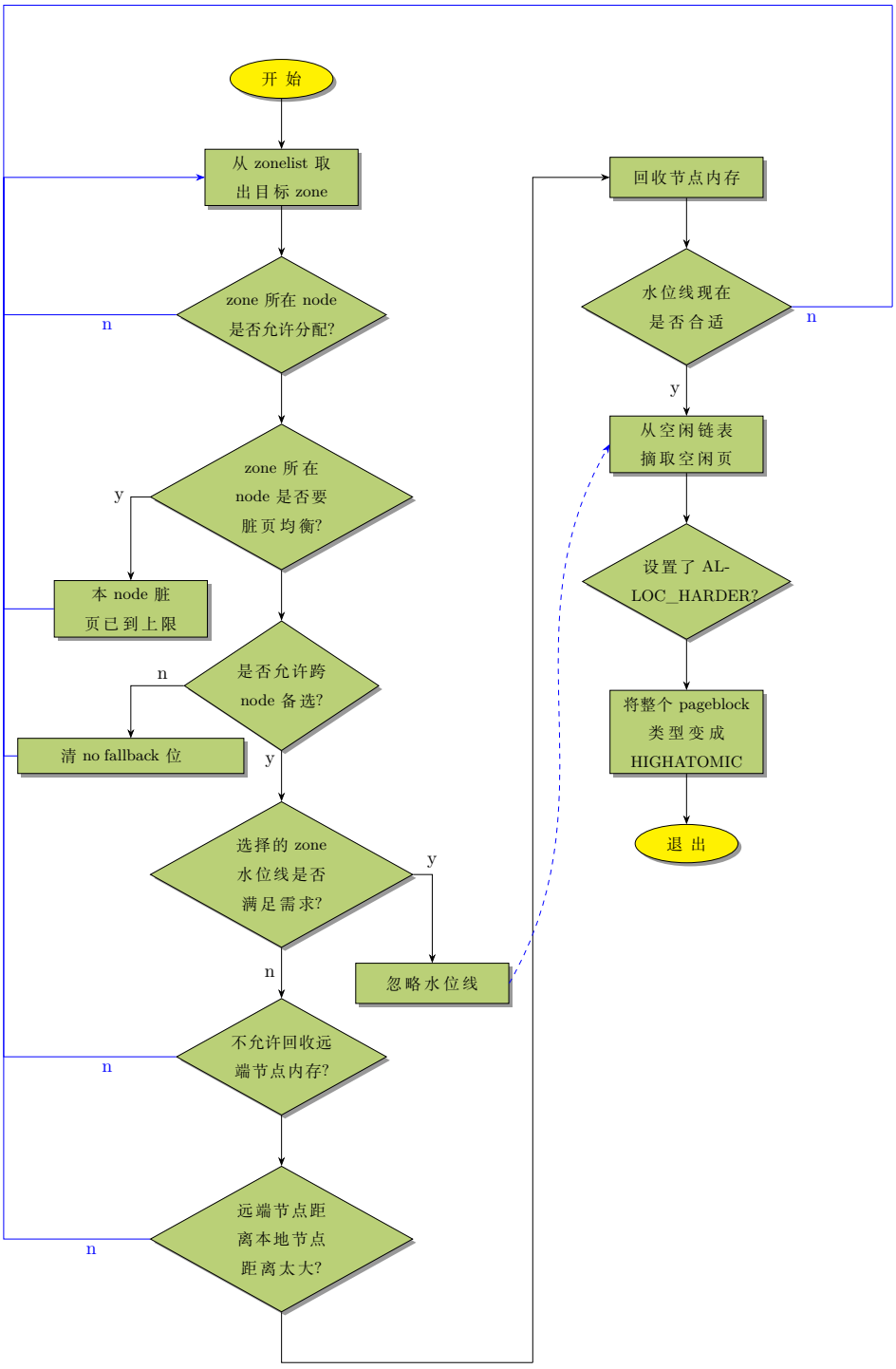
- 3904 如果分配成功，调用 prep_new_page() 来初始化页。

- 3910-3911 进入这两行首先要满足 $order > 0$(即" 大阶分配") — 单页分配（order=0）无需预留，只有多页连续分配才需要兜底。其次，该次分配需要是" 用尽所有手段 (ALLOC_HARDER)" 完成的分配。

  换言之，在高优先级场景下，当内核找到一个可用的大阶页块后，会将该页块标记为 MIGRATE_HIGHATOMIC，专门留给未来的大阶原子分配使用——既满足当前分配需求（该页块已找到，可分配），又为后续的中断上下文、自旋锁临界区等不可睡眠场景 "提前占座"，避免未来无可用页块导致分配失败。因此分配成功后这里 reserve_highatomic_pageblock() 会为了将来的使用而提前预留分配到的页块。

- 3929-3932 对于 UMA 系统而言，在枚举的那些 zone 中分配，可能都会因产生碎片化的潜在风险而失败。因此设置允许碎片化的标志，重新尝试。

get_page_from_freelist 流程图如下:



4.2.4.1 prep_new_page()

3904 行调用 prep_new_page() 对伙伴系统刚分配出来的全新物理页框 (page) 进行初始化，让物理页从”原始空闲态”变为可被内核/进程安全使用的状态。也就是说分配物理页后再调用此函数做收尾准备，函数如下:

__alloc_pages_nodemaskn() *get_page_from_freelist()* *prep_new_page()*

```
2280 static void prep_new_page(struct page *page, unsigned int order, gfp_t gfp_flags,
2281                            unsigned int alloc_flags)
2282 {
2283     post_alloc_hook(page, order, gfp_flags);
2284
2285     if (!free_pages_prezeroed() && want_init_on_alloc(gfp_flags))
2286         kernel_init_free_pages(page, 1 << order);
2287
2288     if (order && (gfp_flags & __GFP_COMP))
2289         prep_compound_page(page, order);
2290
2291     /*
2292      * page is set pfmemalloc when ALLOC_NO_WATERMARKS was necessary to
2293      * allocate the page. The expectation is that the caller is taking
2294      * steps that will free more memory. The caller should avoid the page
2295      * being used for !PFMEMALLOC purposes.
2296      */
2297     if (alloc_flags & ALLOC_NO_WATERMARKS)
2298         set_page_pfmemalloc(page);
2299     else
2300         clear_page_pfmemalloc(page);
2301 }
```

- 2283 物理页分配完成后的通用钩子逻辑。里面封装了内存分配后的公共处理逻辑 (比如内存调试统计、内核安全检查等)。所有从伙伴系统分配的物理页，都会走到这个钩子无差异化执行通用后置逻辑。
- 2286 从 page 指向的首地址开始，对 2^{order} 个连续物理页执行逐字节的内存清零操作。

post_alloc_hook() 函数如下：

__alloc_pages_nodemaskn() > *get_page_from_freelist()* > *prep_new_page()* > *post_alloc_hook()*

```
2266 inline void post_alloc_hook(struct page *page, unsigned int order,
2267                 gfp_t gfp_flags)
2268 {
2269     set_page_private(page, 0);
2270     set_page_refcounted(page);
2271
2272     arch_alloc_page(page, order);
2273     if (debug_pagealloc_enabled_static())
2274         kernel_map_pages(page, 1 << order, 1);
2275     kasan_alloc_pages(page, order);
2276     kernel_poison_pages(page, 1 << order, 1);
2277     set_page_owner(page, order, gfp_flags);
2278 }
```

- 2269 将页面的私有数据域清零。private 字段不同场景有不同用途（如 slab 分配器存储缓存信息、页面缓存存储文件 inode 指针），新分配的页面必须清空该字段，避免残留旧数据导致逻辑错误。
- 2270 将页面的引用计数（page->_refcount 字段）初始化为 1. 计数 > 0 表示页面被使用，计数 = 0 表示可被回收/释放。新分配的页面初始计数设为 1，标识“已分配且被内核使用”，防止被错误释放（如双重释放、释放未分配页面）。
- 2272 架构相关的页面分配后置处理函数.ARM 架构通常为空。
- 2276 kernel_poison_pages() 内存“污化”处理，它依赖 CONFIG_PAGE_POISONING 是否配置，如果未配置改选项则此函数实现为空函数。它根据传入参数会进行使用前的”污化”检查或进行”污化”操作。enable 参数的语义是”页面是否被内核分配并准备使用”，而非“是否执行污化”．此处 enable 为 1，会进行是否污化页被非法使用的检测。

向空闲或已释放的页面写入固定的“非法特征值”（如 0xaa/0x55）进行”污染化”，换言之，意味着标记了该页面为被污染了“不可用”；若程序非法访问“已污化的空闲页面”（如使用已释放内存 UAF、读取未初始化内存），会读取到特征值，内核可通过该特征值快速定位内存错误。

4.2.4.2 reserve_highatomic_pageblock()

__alloc_pages_nodemaskn() > *get_page_from_freelist()* > *reserve_highatomic_pageblock()*

```
2634 static void reserve_highatomic_pageblock(struct page *page, struct zone *zone,
2635                 unsigned int alloc_order)
2636 {
2637     int mt;
2638     unsigned long max_managed, flags;
2639 
2640     /*
2641      * Limit the number reserved to 1 pageblock or roughly 1% of a zone.
2642      * Check is race-prone but harmless.
2643      */
2644     max_managed = (zone_managed_pages(zone) / 100) + pageblock_nr_pages;
2645     if (zone->nr_reserved_highatomic >= max_managed)
2646         return;
2647 
2648     spin_lock_irqsave(&zone->lock, flags);
2649 
2650     /* Recheck the nr_reserved_highatomic limit under the lock */
2651     if (zone->nr_reserved_highatomic >= max_managed)
2652         goto out_unlock;
2653 
2654     /* Yoink! */
2655     mt = get_pageblock_migratetype(page);
2656     if (!is_migrate_highatomic(mt) && !is_migrate_isolate(mt)
2657         && !is_migrate_cma(mt)) {
2658         zone->nr_reserved_highatomic += pageblock_nr_pages;
2659         set_pageblock_migratetype(page, MIGRATE_HIGHATOMIC);
2660         move_freepages_block(zone, page, MIGRATE_HIGHATOMIC, NULL);
2661     }
2662 
2663 out_unlock:
2664     spin_unlock_irqrestore(&zone->lock, flags);
2665 }
```

- 2644 计算允许预留的大阶原子页块最大总页数。预留给大阶原子分配页的数量不能超过 1 个标准页面块和内存 zone 可管理页数的 1% 的总和（避免过度预留浪费内存）。针对该行的英文注释描述和代码实现不太一样，看上去原始注释有错误。
- 2645 检查 zone 中预留的大阶原子页块总页数是否超过上限；若超过，直接返回，无需执行后续操作；这样设计目的是为了“快速失败”——避免无意义的加锁 / 解锁操作，提升性能
- 2648-2652 加锁后重新检查预留上限。无锁检查是“性能优化”，加锁重新检查是“正确性保障”。因为多数场景下在无锁检查时就直接返回了，而不需要在占住自旋锁的情况下检测带来系统负担。
- 2655-2657 获取 page 所在页块的迁移类型。仅当页块的迁移类型不是以下三类时，才允许预留：
  - is_migrate_highatomic(mt)：已是大阶原子分配专用 (无需重复预留)；
  - is_migrate_isolate(mt)：隔离页块 (用于内存碎片整理，禁止修改)；

– is_migrate_cma(mt)：CMA(连续内存分配) 页块（供设备驱动专用，禁止挪用）；

这三种是**独占的** pageblock, 不能挪用。其他如 MIGRATE_(MOVABLE,UNMOVABLE,RECLAIMABLE) 的迁移类型，没有强独占约束。在内存急需原子连续内存时，可以整个 pageblock 抢占升级为 MIGRATE_HIGHATOMIC 作为 GFP_ATOMIC 的大阶原子分配的应急缓存区。

原来的 MOVABLE 块变成 HIGHATOMIC 块，内存整理不再轻易迁移这个块，会加剧内存碎片。所以只在需要满足给 HIGHATOMIC 预留内存时才会抢占这块内存，而不会无条件无限制抢占。同理，UNMOVABLE 的 pageblock 的页面虽然无法移动，但是还剩余大量可用页。原子大阶分配失败会触发 Oom, 这个风险 > 碎片风险。对于 RECLAIM 页也是，碎片风险属于性能问题，但是原子大阶分配失败属于稳定性问题。

- 2658 更新 zone 的已预留给大阶原子分配的页总数
- 2659 将该页块的迁移类型设置为 MIGRATE_HIGHATOMIC(大阶原子页分配专用)。这是“预留”的核心标记 — 后续分配时，普通分配请求会跳过 MIGRATE_HIGHATOMIC 类型的页块，仅大阶原子的分配才可使用。函数参考 4.2.4.4章节。
- 2670 将该页块内所有空闲页：从原迁移类型的空闲链表中移除，加入 MIGRATE_HIGHATOMIC 类型的空闲链表

4.2.4.3 zone_watermark_fast()

回到 get_page_from_freelist()，3858 行调用 zone_watermark_fast() 如下：

__alloc_pages_nodemaskn() > *get_page_from_freelist()* > *zone_watermark_fast()*

```
3664 static inline bool zone_watermark_fast(struct zone *z, unsigned int order,
3665                 unsigned long mark, int highest_zoneidx,
3666                 unsigned int alloc_flags, gfp_t gfp_mask)
3667 {
3668     long free_pages;
3669
3670     free_pages = zone_page_state(z, NR_FREE_PAGES);
3671
3672     /*
3673      * Fast check for order-0 only. If this fails then the reserves
3674      * need to be calculated.
3675      */
3676     if (!order) {
3677         long fast_free;
3678
3679         fast_free = free_pages;
3680         fast_free -= __zone_watermark_unusable_free(z, 0, alloc_flags);
3681         if (fast_free > mark + z->lowmem_reserve[highest_zoneidx])
3682             return true;
3683     }
3684
3685     if (__zone_watermark_ok(z, order, mark, highest_zoneidx, alloc_flags,
3686                     free_pages))
3687         return true;
3688     /*
3689      * Ignore watermark boosting for GFP_ATOMIC order-0 allocations
3690      * when checking the min watermark. The min watermark is the
3691      * point where boosting is ignored so that kswapd is woken up
3692      * when below the low watermark.
```

```
3693         */
3694        if (unlikely(!order && (gfp_mask & __GFP_ATOMIC) && z->watermark_boost
3695            && ((alloc_flags & ALLOC_WMARK_MASK) == WMARK_MIN))) {
3696            mark = z->_watermark[WMARK_MIN];
3697            return __zone_watermark_ok(z, order, mark, highest_zoneidx,
3698                        alloc_flags, free_pages);
3699        }
3700
3701        return false;
3702    }
```

- 3694-3699 对以 GFP_ATOMIC 方式 (即原子分配，指在内存分配过程中，不允许进行内存回收。例如，通过触发页面回收或交换操作) 申请的 order-0(单页) 大小内存检测 min 水位线时，忽略” 水位线提升 (watermark boost)”，使用原始水位线 (z->_watermark[WMARK_MIN]) 作为 min 水位线参数传递给 ___zone_watermark_ok()。在 ___zone_watermark_ok() 函数中会进一步应用已经忽略了” 水位线提升” 的 min 参数值，来计算 ALLOC_HIGH,ALLOC_OOM 等条件下的 min 值，以提高内存申请的成功率。

  当页块 (pageblock) 中混合了不同迁移类型的内存（易导致碎片）时，内核会通过提升水位线提前触发回收，以减少碎片积累。用于提前触发内存回收。

  启用” 水位线提升 (watermark boost)” 功能后，会观察到对 order-0 阶页 (单页) 的原子分配的失败率很高。这是由于对系统空闲页进行的水位检查是基于 ->watermark_boost 偏置的。换言之，是基于抬高后的水位进行的判断，从而无形中提高了内存申请的难度。这对于可休眠的分配是不成问题的，但对于原子分配则不然。3694-3699 行就是针对这个 bug 的 patch。

  因为在内存大小为 4GB 的 Snapdragon 硬件上运行 Android 系统时经常出现该问题。于是内核社区引入了这个 patch。譬如：当->watermark_boost 为零，如果此时系统中的水位线配置为：

  ```
  _watermark = (
         [WMARK_MIN] = 1272, --> ~5MB
         [WMARK_LOW] = 9067, --> ~36MB
         [WMARK_HIGH] = 9385), --> ~38MB
  watermark_boost = 0
  ```

  那么，在安卓系统启动了占用内存较多的应用程序后往往会导致系统中的 extfrag 事件 (内存碎片)，以至于 ->watermark_boost 被设置为最大值，即默认提升->watermark_boost 值到达 WMARK_HIGH 水位线数值的 150%。

  ```
  _watermark = (
         [WMARK_MIN] = 1272, --> ~5MB
         [WMARK_LOW] = 9067, --> ~36MB
         [WMARK_HIGH] = 9385), --> ~38MB
  watermark_boost = 14077, -->~57MB
  ```

  内核中，内存水位线（watermark）的单位是页的数目。在默认的系统配置下，要使 order-0(单页) 的原子分配成功，只要有约 2MB (min = 3/4(min/2), 同时 $min \approx 5M$, $min = 3/4(5/2) \approx 2MB$) 的空闲内存就足够了。但水位提升后 min_wmark 为 61MB($(1271 + 14077) * 4KB/page = (1271 + 14077) * 4096Bytes = 62865408/(1024 * 1024)MB \approx 60MB$)，因此，要使 order-0(单页) 的原子分配成功，系统至少要有 23MB 的空闲内存（根据 zone_watermark_ok() 的计算结果:min = 3/4(min/2), 即：$min = 3/4(61/2) \approx 23MB$)。但尽管系统有 20MB 的空闲内存，还是出现了分配失败。在测试中，这会在系统启动后的 300 秒就可复现。在更低内存配置（<2GB）时，最早会在启动后的 150 前秒就能观察到 fail。对于 oreder-0(单页) 原子分配的这种失败可以通过在计算水位

线时摒弃->watermark_boost 的偏置而避免。

4.2.4.4 rmqueue()

再回到 get_page_from_freelist() 函数 3901 行调用的函数 rmqueue()：

__alloc_pages_nodemaskn() > *get_page_from_freelist()* > *rmqueue()*

```
3420 static inline
3421 struct page *rmqueue(struct zone *preferred_zone,
3422              struct zone *zone, unsigned int order,
3423              gfp_t gfp_flags, unsigned int alloc_flags,
3424              int migratetype)
3425 {
3426     unsigned long flags;
3427     struct page *page;
3428 
3429     if (likely(order == 0)) {
3430         /*
3431          * MIGRATE_MOVABLE pcplist could have the pages on CMA area and
3432          * we need to skip it when CMA area isn't allowed.
3433          */
3434         if (!IS_ENABLED(CONFIG_CMA) || alloc_flags & ALLOC_CMA ||
3435                 migratetype != MIGRATE_MOVABLE) {
3436             page = rmqueue_pcplist(preferred_zone, zone, gfp_flags,
3437                     migratetype, alloc_flags);
3438             goto out;
3439         }
3440     }
3441 
3442     /*
3443      * We most definitely don't want callers attempting to
3444      * allocate greater than order-1 page units with __GFP_NOFAIL.
3445      */
3446     WARN_ON_ONCE((gfp_flags & __GFP_NOFAIL) && (order > 1));
3447     spin_lock_irqsave(&zone->lock, flags);
3448 
3449     do {
3450         page = NULL;
3451         /*
3452          * order-0 request can reach here when the pcplist is skipped
3453          * due to non-CMA allocation context. HIGHATOMIC area is
3454          * reserved for high-order atomic allocation, so order-0
3455          * request should skip it.
3456          */
3457         if (order > 0 && alloc_flags & ALLOC_HARDER) {
3458             page = __rmqueue_smallest(zone, order, MIGRATE_HIGHATOMIC);
3459             if (page)
3460                 trace_mm_page_alloc_zone_locked(page, order, migratetype);
3461         }
3462         if (!page)
3463             page = __rmqueue(zone, order, migratetype, alloc_flags);
3464     } while (page && check_new_pages(page, order));
3465     spin_unlock(&zone->lock);
3466     if (!page)
3467         goto failed;
3468     __mod_zone_freepage_state(zone, -(1 << order),
3469                   get_pcppage_migratetype(page));
3470 
3471     __count_zid_vm_events(PGALLOC, page_zonenum(page), 1 << order);
3472     zone_statistics(preferred_zone, zone);
```

```
3473     local_irq_restore(flags);
3474
3475 out:
3476     /* Separate test+clear to avoid unnecessary atomics */
3477     if (test_bit(ZONE_BOOSTED_WATERMARK, &zone->flags)) {
3478         clear_bit(ZONE_BOOSTED_WATERMARK, &zone->flags);
3479         wakeup_kswapd(zone, 0, 0, zone_idx(zone));
3480     }
3481
3482     VM_BUG_ON_PAGE(page && bad_range(zone, page), page);
3483     return page;
3484
3485 failed:
3486     local_irq_restore(flags);
3487     return NULL;
3488 }
```

从给定的 zone 区的 freelist 分配页。为了减少锁竞争和提供分配效率，会优先从 pcplist 中分配 0 阶单页，也就是在 per-cpu 的 pageset 缓存 (zone->pageset) 中分配，而不是从伙伴系统遍历。

- 3429-3439 分配单页时才进入这个分支使用 per-cpu 的 pageset 缓存，大阶页 ($order > 0$) 的分配直接走伙伴系统。

  - *!IS_ENABLED(CONFIG_CMA)* 检查内核是否启用了 CMA 配置选项。如果没有启用 CONFIG_CMA, 说明系统中没有 ZONE_DMA 区域，可以直接分配无需检查。
  - *alloc_flags & ALLOC_CMA* 因为是"||" 语句，所以是在 CONFIG_CMA 配置了的情况下 (换言之，条件 1 不成立)，再判定该条件的。所以，此处表示：配置了内核选项 *CONFIG_CMA=y*，并且明确指定了要从 CMA 区域分配内存。这种场景通过 rmqueue_pcplist() 分配也没有问题。
  - *migratetype != MIGRATE_MOVABLE* 同理，在前面"||" 的条件都不成立时，才执行第三个判断。也就是说: 在配置了 CONFIG_CMA, 且不允许从 CMA 区分配内存 (即没有设置 ALLOC_CMA 标志)。此时申请分配可移动页面，不能使用 rmqueue_pcplist 函数进行页面分配。可移动页面的 pcplist 可能包含 CMA 区域的页面，因此当 ALLOC_CMA 未指定时，需要跳过 CMA 区域，以避免长期占用 CMA 页面。此时 if 语句中的条件就是为了修复这一潜在问题而加的 patch, 以避免不期望的 CMA 页面的长期占用。

  申请分配可移动页时，会优先从 MIGRATE_MOVABLE 的 pcp 列表分配。但是该链表有可能混入物理地址来自 ZONE_CMA 区域的页。因为 ZONE_CMA 区域的内存可以被 MOGRATE_MOVABLE 类型的页申请借用，释放时会进入 MIGRATE_MOVABLE 的 pcplist 缓存。因此，当场景或标识不允许从 ZONE_CMA 区域分配时，必须跳过这类页面。

- 3446 如果申请 $order > 1$ 阶的页，那么不允许使用 ___GFP_NOFAIL。___GFP_NOFAIL 是高风险标识，如果用于申请大阶页，在内存碎片化的情况下，可能造成系统永久阻塞而 crash。

- 3457 在不允许分配 ZONE_CMA 区域内存的场景下，前面跳过了 pcplist 缓存链表的 order 0 阶的分配会到达该行的逻辑。但是 MIGRATE_HIGHATOMIC 是留给大阶页原子分配的。也就是说 $order > 0$, 所以 0 阶页的申请要跳过该区域。

  如果需要分配 > 0 阶的页，并且当设置了 ALLOC_HARDER 标志时，内核会在内存分配过程中采用更激进的策略去尝试获取内存。

- 3458 优先从 MIGRATE_HIGHATOMIC 迁移链表去分配。从目标迁移链表中取出最小的可用的连续页。

- 3463 如果分配不到，则从常规迁移链表进行分配尝试。
- 3464 检查分配到的页是否符合要求。否，则丢弃重新分配。
- 3466-3467 如果链表始终无空闲页，跳转错误处理的流程。
- 3468-3469 修改目标 zone 空闲页统计计数。

  ====== 待插入源码解析
- 3471 对内存 zone 区域的内存事件累加计数。
- 3472 更新 NUMA 命中/未命中统计信息。
- 3477-3479 清除目标 zone 的水位临时抬高的标识，并唤醒 kswapd 进行内存回收。

  水位线临时抬高机制是为了抬高水位，让 kswapd 尽快跑起来，持续后台释放内存。

内核管理迁移类型的基本粒度——迁移类型（如可移动、不可移动）是针对整个页块（pageblock）标记的，而不是单个页。但在未能找到满足迁移类型（migrate type）并且阶数（order）大于等于需求的页时，会从其他迁移类型的页块中迁移页到目标需求类型的页块（pageblock）中，而这就是函数 ___rmqueue_fallback() 将做的事。

调用图:

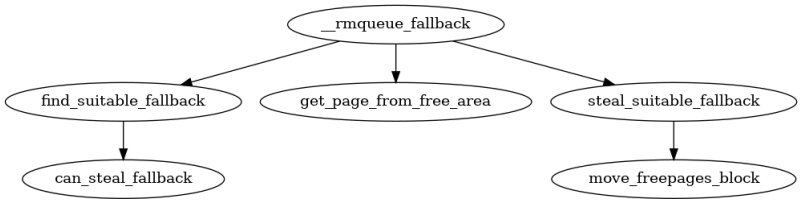


图 4–3: 调用图：___rmqueue_fallback()

___alloc_pages_nodemaskn() > *get_page_from_freelist()* > *rmqueue()* > *___rmqueue_fallback()*

```
2758 static __always_inline bool
2759 __rmqueue_fallback(struct zone *zone, int order, int start_migratetype,
2760                    unsigned int alloc_flags)
2761 {
2762     struct free_area *area;
2763     int current_order;
2764     int min_order = order;
2765     struct page *page;
2766     int fallback_mt;
2767     bool can_steal;
2768
2769     /*
2770      * Do not steal pages from freelists belonging to other pageblocks
2771      * i.e. orders < pageblock_order. If there are no local zones free,
2772      * the zonelists will be reiterated without ALLOC_NOFRAGMENT.
2773      */
2774     if (alloc_flags & ALLOC_NOFRAGMENT)
```

```
2775            min_order = pageblock_order;
2776
2777        /*
2778         * Find the largest available free page in the other list. This roughly
2779         * approximates finding the pageblock with the most free pages, which
2780         * would be too costly to do exactly.
2781         */
2782        for (current_order = MAX_ORDER - 1; current_order >= min_order;
2783                    --current_order) {
2784            area = &(zone->free_area[current_order]);
2785            fallback_mt = find_suitable_fallback(area, current_order,
2786                    start_migratetype, false, &can_steal);
2787            if (fallback_mt == -1)
2788                continue;
2789
2790            /*
2791             * We cannot steal all free pages from the pageblock and the
2792             * requested migratetype is movable. In that case it's better to
2793             * steal and split the smallest available page instead of the
2794             * largest available page, because even if the next movable
2795             * allocation falls back into a different pageblock than this
2796             * one, it won't cause permanent fragmentation.
2797             */
2798            if (!can_steal && start_migratetype == MIGRATE_MOVABLE
2799                        && current_order > order)
2800                goto find_smallest;
2801
2802            goto do_steal;
2803        }
2804
2805        return false;
2806
2807    find_smallest:
2808        for (current_order = order; current_order < MAX_ORDER;
2809                                current_order++) {
2810            area = &(zone->free_area[current_order]);
2811            fallback_mt = find_suitable_fallback(area, current_order,
2812                    start_migratetype, false, &can_steal);
2813            if (fallback_mt != -1)
2814                break;
2815        }
2816
2817        /*
2818         * This should not happen - we already found a suitable fallback
2819         * when looking for the largest page.
2820         */
2821        VM_BUG_ON(current_order == MAX_ORDER);
2822
2823    do_steal:
2824        page = get_page_from_free_area(area, fallback_mt);
2825
2826        steal_suitable_fallback(zone, page, alloc_flags, start_migratetype,
2827                                    can_steal);
2828
2829        trace_mm_page_alloc_extfrag(page, order, current_order,
2830            start_migratetype, fallback_mt);
2831
2832        return true;
2833
2834    }
```

- 2774-2775 ALLOC_NOFRAGMENT 字面意思是避免分配造成碎片化，实际上是禁止一个页块内部有不同迁移类型。因为混合类型会导致页块难以被整体回收或迁移。因此改标志的作用是限制最

小迁移粒度为页块阶数，以此减少碎片。所以，如果有避免碎片化的要求则置 min_order 为 pageblock_order。当接下来的流程中，从最高阶 order 向下搜索备选迁移类型页时的，最下限的 order 就是 min_order。当 min_order 为 pageblock_order 有助于减少碎片。

mem_section 或 zone 管理一段连续页框 (物理内存)，系统又会把这些连续物理内存划分成多个 pageblock。一个 pageblock 是 buddy 伙伴系统管理的最大连续物理页 (pageblock_nr_pages 个页框，对应 $2^{pageblock_order}$ 个页)，每个 pageblock 具有一个迁移类型或迁移属性。

- 2782-2803 从最高阶 order 向低阶 order 搜索, 这样可以尽量的将迁移列表中的大块分割, 避免形成过多的碎片。

  MAX_ORDER 定义了内核支持的最大连续物理页阶数。MAX_ORDER 限制了最大连续页分配的粒度，而 pageblock_order 定义了迁移类型管理的粒度（页块）。页块是内核管理迁移类型（migrate type）的最小单位，其阶数必须在 MAX_ORDER 支持的范围内。因此，pageblock_order 小于等于 MAX_ORDER - 1。

  - 2784 获取当前 zone 中目标 order 对应的 area。
  - 2785-2788 在当前 area 内存区域中，遍历 start_migratetype 对应的备用迁移类型数组，看是否能在备选迁移类型的列表中找到一块满足要求的内存块（order 大于等于 min_order, 小于等于 Max_ORDER-1）

- 2798-2800 为了高效找到可分配的内存，内核会优先查找”order 尽可能最大的” 可用空闲页 (即大阶连续页)，而非精确查找” 拥有最多空闲页的” 页块。

  “精确统计每个页块的空闲页数量” 需要遍历所有页块并计算，会消耗大量 CPU 资源；而 “查找 order 尽可能最大的连续可用空闲页” 是一种更高效的近似方法——较大的空闲页通常来自空闲页数量较多的页块，因此这种方式能在效率与准确性之间取得平衡。

  - 2798 当程序跑到该行，意味着已经通过遍历找到了备选迁移类型。

  can_steal 标志表示为了避免碎片化是否需要获取比所需页更多的空闲空间，从而达到一个完整的 pageblock 块。换言之，就是将目标页块 (pageblock) 中所有当前空闲的页（无论阶数大小）全部转移到当前需要的迁移类型中，供后续分配使用。已分配的页仍留在原页块中，继续被原使用者占用。(注意: 之后提到” 盗取” 或” 窃取” 整个页块, 都指的是窃取该页块内的所有空闲页，而不包括页块中已分配的页)

  如果 find_suitable_fallback() 的传出参数 can_steal 的值表明无法盗取完整的 pageblock 页块。那么在请求页的类型是 MIGRATE_MOVABLE 时，我们倾向于借用 order 尽可能最小的页，而非相反。MIGRATE_MOVABLE 类型的页可以被内核重新移动（通过页迁移机制），即使本次分配拆分了小页，后续若需要更大的连续页，内核可通过迁移这些可移动页来整理出连续的内存，避免碎片永久化。所以，即使以后再申请 MIGRATE_MOVABLE 类型内存，因内存不足—上次没有盗取到完整的页块作为 MIGRATE_MOVABLE 类型内存—备选到与当前这次不同的 pageblock 页块，也不会因剩下的 MIGRATE_MOVABLE 页不被使用，而造成当前这个 pageblock 碎片固化。因为 MIGRATE_MOVABLE 类型能迁移合并。因此跳转到 find_smallest 查找最小可用 oreder 页。

  而拆分大阶页的代价是长期难以修复的, 不拆分大阶页 (order 尽可能最大) 是优先保护稀缺的大阶连续内存。因为大阶页若拆分成小阶页后被分散使用，几乎不可能再合并回大阶页，导致以后大阶页的分配永远失败。从而由于大页的不可再生导致了永久碎片。

NOTE

在一个页块内允许同时存在不同阶数的已分配页。
页块是内核管理迁移类型（*migrate type*）的最小单位，也就是管理粒度。但它本身并不限制内部页的分配阶数。页块的核心作用是标记“整体迁移类型”，而非强制内部页必须是同一阶数 *(order)*。

- 2802 如果能盗取一整个页面块 (注意此处的含义是前面反复强调的：窃取一个页块中剩余的所有空闲页), 则跳转到 do_steal。
- 2808-2815 按阶数 (order) 从小到大重复迭代，搜索满足需求的最小 order(阶) 的可用页面。这种方式适用于希望产生更少碎片而不是更希望拥有几乎最多空闲迁移类型页的场景。一旦完成此过程，将执行 do_steal 执行窃取页的逻辑。
- 2824 从目标 order 的 area 获取 fallback_mt 迁移类型的空闲页的首页结构体。
- 2826 ” 借用”(find_smallest) 或” 盗取”(do_steal) 页的实际操作在该行真正进行。

___rmqueue_fallback() 第 2785 和 2811 行调用的，用于找寻合适备选迁移类型的函数 find_suitable_fallback() 源码如下，该函数用于检查是否有 order 大小的合适空闲备选 (fallback) 页可用：

__alloc_pages_nodemaskn() ⋯ *rmqueue()* *___rmqueue_fallback()* *find_suitable_fallback()*

```
2599 int find_suitable_fallback(struct free_area *area, unsigned int order,
2600             int migratetype, bool only_stealable, bool *can_steal)
2601 {
2602     int i;
2603     int fallback_mt;
2604
2605     if (area->nr_free == 0)
2606         return -1;
2607
2608     *can_steal = false;
2609     for (i = 0;; i++) {
2610         fallback_mt = fallbacks[migratetype][i];
2611         if (fallback_mt == MIGRATE_TYPES)
2612             break;
2613
2614         if (free_area_empty(area, fallback_mt))
2615             continue;
2616
2617         if (can_steal_fallback(order, migratetype))
2618             *can_steal = true;
2619
2620         if (!only_stealable)
2621             return fallback_mt;
2622
2623         if (*can_steal)
2624             return fallback_mt;
2625     }
2626
2627     return -1;
2628 }
```

- 2605-2606 目标阶当前剩余空闲页数量是 0，则出错返回。
- 2609-2625 在二维数组 fallbacks 里，以目标迁移类型的宏值为行号索引找到目标迁移类型数组。对目标迁移类型数组中的备选迁移类型元素进行枚举，直到找到合适的备选迁移类型。

- 2611-2612 枚举完成，找不到可用的备选迁移类型，失败返回。
- 2614-1615 每次迭代中依次判断目标 area 上当前迁移类型的备选—” 首选” 和” 次选”—类型的空闲页是否能满足需求。换言之，目标 area 中没有当前备选迁移类型空闲页则跳出本次迭代，继续枚举。
- 2617-2618 判断是否可以” 盗取” 备选页 (函数见后)。
- 当从备选迁移类型分配页时，为满足之后可能的再次分配，会尝试获取比当前所需页更大的额外空闲内存，也就是一整块 (pageblock) 内存。这样可以在下次申请同样迁移类型页时从盗取的内存中分配。从而避免再次申请同类型页时备选到不同迁移类型的页面块，造成页块内存在多种迁移类型页，对多个不同迁移类型的 pageblock 造成污染。这种获取额外空闲页达到一整个页块的方式也就是所谓的”盗取(**steal**)”。

  NOTE

  这里需要注意 ” 备选 *(fallback)* ” 和 ” 盗取 *(steal)* ” 概念的不同

  备选指当前分配类型的内存不足，从而在其他分配迁移类型的页块上获取对应阶 *(order)* 大小的内存。而盗取指在前者的基础上再获取额外的该页块 *(pageblock)* 上的剩余空闲内存，从而达到一个完整的 *pageblock* 块。

- 2620-2621 如果不需要盗取，直接返回找到的迁移类型。
- 2623-2624 如果需要盗取，那么在判断了目前迁移类型能够盗取 (一整块页面块 pageblock_order 阶大小内存) 时才返回该迁移类型，这有助于减少内存碎片，否则继续进行迭代。

can_steal_fallback() 函数用于判断是否可以从备选迁移类型中” 盗取” 一整块 (pageblock) 内存。函数如下:

__alloc_pages_nodemask() ··· ··· *find_suitable_fallback()* *can_steal_fallback()*

```
2452 static bool can_steal_fallback(unsigned int order, int start_mt)
2453 {
2454     /*
2455      * Leaving this order check is intended, although there is
2456      * relaxed order check in next check. The reason is that
2457      * we can actually steal whole pageblock if this condition met,
2458      * but, below check doesn't guarantee it and that is just heuristic
2459      * so could be changed anytime.
2460      */
2461     if (order >= pageblock_order)
2462         return true;
2463
2464     if (order >= pageblock_order / 2 ||
2465         start_mt == MIGRATE_RECLAIMABLE ||
2466         start_mt == MIGRATE_UNMOVABLE ||
2467         page_group_by_mobility_disabled)
2468         return true;
2469
2470     return false;
2471 }
```

- 2461-2462 判断目标 order 是否大于等于 pageblock_order。当前空闲块本身阶数 >= 页面块的阶数时，意味着 2^{order} 是 $2^{pagelock_order}$ 的整数倍，也就是一个当前目标 order 的内存是一个页面块的整数倍。此种情况下，始终能拿到 pageblock 页块级别的连续内存，不会因导致页面类型混合而产生碎片，所以盗取是安全的，

- 2464-2468
  - 2464 该行的”>= pageblock_order / 2” 只是一个启发式的经验规则 (可以在需要修正的时候改变)，满足该条件意味着页面块中目标阶所占空闲空间足够大，以至于超过页面块的一半。这种情况下如果盗取剩下的空闲内存，此时盗取的内存在页面块中占比较大，即使目标迁移类型和源迁移类型不同，造成的影响占比相对来说也较小。

    这个判断看上去和 2461 有冗余。但是，如果 2461 行条件满足，那么” 保证” 可以实际盗取整个 pageblock。但是 2464 行并不提供这个保证。另一个非正式的说法是：其实编译器已经提供了优化，在编译时将两个判断其中之一优化掉了。
  - 2465-2466 对于指定的迁移类型为 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 的分配请求，会不顾页大小直接返回 true。这是因为这两个迁移类型申请的备选迁移页最终备选会落到迁移类型 MIGRATE_MOVABLE 上，而这对 MIGRATE_MOVABLE 类型的页块造成的碎片化影响，会远大于迁移类型 MIGRATE_MOVABLE 从备选迁移类型 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 盗取一整个页块造成的碎片化。因为 MIGRATE_MOVABLE 类型的页面可以通过内存整理合并避免来避免造成 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 页面块内的内存碎片。

    而反之，在迁移类型 MIGRATE_MOVABLE 中盗取 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 达到一整个页面块，并且下次在此页块再次分配同样类型页时，会将剩余空间拆分成小阶页，加上迁移类型 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 不能移动合并，所以有可能造成永久碎片。

    综上，对于迁移类型 MIGRATE_UNMOVABLE 或 MIGRATE_RECLAIMABLE 允许无条件盗取。
  - 2467 无论前面的场景是否满足，只要内核主动禁用了 “按迁移性分组”，就直接允许跨类型分配—相当于 “强制关闭分组限制”。当内核全局变量 page_group_by_mobility_disabled 为 1 时，表示 “禁用页面按迁移类型分组”。

    内核的“页面按迁移类型分组”(比如 MIGRATE_MOVABLE/UNMOVABLE/RECLAIMABLE 分开管理）是为了优化内存碎片：可移动页面可以被迁移，避免碎片积累；不可移动页面固定在物理地址，不能迁移。但这种分组也可能导致 “某类页面紧缺，其他类型页面闲置” 的问题。而 page_group_by_mobility_disabled 就是为了在需要时 “打破分组限制” —当这个开关关闭时，内核严格按类型分组分配，优先从同类型页块拿页面；当开关打开时，分组逻辑失效，内核可以自由跨类型盗取页面，优先保证 “分配成功”，牺牲部分碎片优化。
  - 在系统初始化期间，所有页都被标记为 MIGRATE_MOVABLE 可移动的页面类型，初始化时内核尚未明确各页面的具体用途（如用于内核分配、CMA 预留等），统一标记为可移动类型是初始的通用配置，为后续动态调整打下基础。

__alloc_pages_nodemaskn() › … › *rmqueue()* › *__rmqueue_fallback()* › *steal_suitable_fallback()*

```
2599
2600 static void steal_suitable_fallback(struct zone *zone, struct page *page,
2601         unsigned int alloc_flags, int start_type, bool whole_block)
2602 {
2603     unsigned int current_order = buddy_order(page);
2604     int free_pages, movable_pages, alike_pages;
```

```
2605     int old_block_type;
2606
2607     old_block_type = get_pageblock_migratetype(page);
2608
2609     /*
2610      * This can happen due to races and we want to prevent broken
2611      * highatomic accounting.
2612      */
2613     if (is_migrate_highatomic(old_block_type))
2614         goto single_page;
2615
2616     /* Take ownership for orders >= pageblock_order */
2617     if (current_order >= pageblock_order) {
2618         change_pageblock_range(page, current_order, start_type);
2619         goto single_page;
2620     }
2621
2622     /*
2623      * Boost watermarks to increase reclaim pressure to reduce the
2624      * likelihood of future fallbacks. Wake kswapd now as the node
2625      * may be balanced overall and kswapd will not wake naturally.
2626      */
2627     boost_watermark(zone);
2628     if (alloc_flags & ALLOC_KSWAPD)
2629         set_bit(ZONE_BOOSTED_WATERMARK, &zone->flags);
2630
2631     /* We are not allowed to try stealing from the whole block */
2632     if (!whole_block)
2633         goto single_page;
2634
2635     free_pages = move_freepages_block(zone, page, start_type,
2636                         &movable_pages);
2637     /*
2638      * Determine how many pages are compatible with our allocation.
2639      * For movable allocation, it's the number of movable pages which
2640      * we just obtained. For other types it's a bit more tricky.
2641      */
2642     if (start_type == MIGRATE_MOVABLE) {
2643         alike_pages = movable_pages;
2644     } else {
2645         /*
2646          * If we are falling back a RECLAIMABLE or UNMOVABLE allocation
2647          * to MOVABLE pageblock, consider all non-movable pages as
2648          * compatible. If it's UNMOVABLE falling back to RECLAIMABLE or
2649          * vice versa, be conservative since we can't distinguish the
2650          * exact migratetype of non-movable pages.
2651          */
2652         if (old_block_type == MIGRATE_MOVABLE)
2653             alike_pages = pageblock_nr_pages
2654                         - (free_pages + movable_pages);
2655         else
2656             alike_pages = 0;
2657     }
2658
2659     /* moving whole block can fail due to zone boundary conditions */
2660     if (!free_pages)
2661         goto single_page;
2662
2663     /*
2664      * If a sufficient number of pages in the block are either free or of
2665      * comparable migratability as our allocation, claim the whole block.
2666      */
2667     if (free_pages + alike_pages >= (1 << (pageblock_order-1)) ||
2668             page_group_by_mobility_disabled)
```

```
2669            set_pageblock_migratetype(page, start_type);
2670
2671        return;
2672
2673    single_page:
2674        move_to_free_list(page, zone, current_order, start_type);
2675    }
```

- 2523 获取指定页的迁移类型（migrate type）。该函数会先定位目标页所属的页块 (pageblock)— 页块是迁移类型管理的最小单位 — 再返回这个页块的迁移类型。
- 2529-2530 当页面块的迁移类型因竞争 (race conditions) 被设置为 MIGRATE_HIGHATOMIC 时，内核则不尝试窃取整个页块 (的空闲页)，转而跳到”single_page” 执行分支。

  这里的*竞争关系 (race conditions)*不是指并发的竞争，而是指当系统中所有空闲页块很难提供 “符合所需阶数的连续空闲页”（比如碎片化严重）。此时，内核为了保障 “大阶 (order>=1) 页原子分配” 的成功率（这类分配失败可能导致系统崩溃），会挑选页块标记为 MIGRATE_HIGHATOMIC 并预留，强制保留其内部的连续空闲页，专门应对后续的大阶原子分配请求，避免因内存碎片导致关键分配失败。譬如 get_page_from_freelist() 函数中调用 reserve_highatomic_pageblock() 来进行 MIGRATE_HIGHATOMIC 标记。
- 2533-2535 to be contined
- 2543 ” 盗取” 最终可能会导致页面块里不同迁移类型的页混合 。可以通过尽可能” 窃取” 最大的页面 (整个页块) 来避免这种情况，理想情况下，这样就可以直接改变整个页块的迁移类型而不会产生碎片。然而，很多情况下由于因为内存不足而被迫在页块中混合了不同迁移类型页从而导致了内存碎片, 这时系统会回收 保留更多内存以防止进一步碎片化 。此时内核会通过 zone 水位临时提升（zone boosting） 以达目的: 提高水位线，从而 更早触发内存回收 。

  boost_watermark() 仅在 steal_suitable_fallback() 中调用。换言之, 当出现盗取其他迁移类型的页以达到整页块 (pageblock) 时, 内核会临时提高水位线, 以便尽早触发回收缓解内存压力。

  kswapd 在回收完成后会将将水位线恢复至正常值 (zone->watermark_boost 的恢复)，降低 kswapd 被频繁唤醒的概率，从而减少系统开销。这部分工作在 kswapd()-> balance_pgdat() 中达成。
- 2544-2545 调用 boost_watermark() 函数临时增加水位提升量后；若检测到设置了 ALLOC_KSWAPD 标志，则设置 ZONE_BOOSTED_WATERMARK 位，以确保 kswapd（执行间接回收的内核线程）被提前唤醒。

  在分配内存过程中调用 rmqueue() 会检测 ZONE_BOOSTED_WATERMARK 标志，如果置位了则 (提前/立即) 唤醒 kswapd 进行异步的非直接 (indirect) 内存回收，降低后续分配仍要备选到其他类型页的可能性。因为该节点整体可能处于内存平衡状态，kswapd 不会被自然唤醒。
- 2548-2549 不允许盗取一整个页块，则跳转到 single_page。

  这里的”single” 不是指” 单页”, 而是指” 一个完整的 order 阶页”。
- 2551-2552 当执行到当前行代码时，意味着可以窃取一整个页块，但该页块可能是原迁移类型的已分配页和空闲页的混合。页块内的目标空闲页将会通过 move_freepages_block () 函数进行迁移，迁移过程不能跨越内存 zone 的边界。已分配但可迁移的页数量会赋值给 movable_pages 变量，这些页是原类型已经分配出去的，但属于 “可迁移” 类型；统计这个值是为了后续决策：如果当前窃取的空

闲页还不够，内核可以后续迁移这些“已分配但可迁移的页”，释放更多空闲页。另外，可被迁移的空闲页数量会被返回给 free_pages 变量。

这里的“迁移”就是盗取其他类型页块的剩余空闲页，转移到当前需求类型的空闲链表供分配。链表拓扑结果的示意性代码 (待替换成示意图):zone->free_area[order]->free_list[migratetype]

- 2558-2573 该代码块计算 alike_pages 变量，该变量用于统计页块内与需求或期望的迁移类型兼容的已分配页数量。
  - 2558-2559 若目标迁移类型为可移动类型（MIGRATE_MOVABLE），则（兼容的）页数直接等于该页块内已分配但可迁移的页数量，即 movable_pages 变量的值。
  - 2568-2570 若源页块为可移动类型（MIGRATE_MOVABLE），则该数量等于该页块内所有不可移动的页的数量。可回收类型或不可迁移类型的分配请求备份到可移动类型（MOVABLE）页块，则将所有非可迁移页视为兼容。
  - 2571-2572 若源迁移类型和目标迁移类型均非可迁移类型—也就是源和目标迁移类型都是 MIGRATE_RECLAIMABLE 和 MIGRATE_UNMOVABLE 其中之一—则认为无兼容页。这样做是为保持保守策略，因为无法轻易区分不同的非可迁移迁移类型。换言之，MIGRATE_RECLAIMABLE 和 MIGRATE_UNMOVABLE 我们无法区分，因为 MIGRATE_RECLAIMABLE 类型页也是不可移动的。
- 2576-2577 如果可被迁移的空闲页数量为 0，跳转到 single_page 执行。
- 2583-2585 通过累加 free_pages（空闲页数量）和 alike_pages（兼容页数量），判断其和是否大于等于页面块 (pageblock) 大小的一半（即判断兼容页与空闲页是否在该页块中占主导），以此决定是否将页块的迁移类型改为目标迁移类型。若满足该条件或禁止分组，则通过 set_pageblock_migratetype() 函数设置页块的迁移类型。
- 2589-2590 执行到 single_branch 分支，表示不能盗取一整个页块，而只能借用一个完整的 order 阶整页。所以这里的 tag 的 single 不表示”单页”。2590 行调用函数把借用的 order 阶页迁移到对应迁移类型空闲页的 freelist 空闲链表。

NOTE

当页面被其他迁移类型借用时，这些被借用的页面的迁移类型会改变，但它们所在的整个页面块的迁移类型不一定改变。

move_freepages_block() 待补充。

move_freepages_block 的作用是迁移整个页块内的空闲页到指定迁移类型的空闲链表。该函数的核心目标是完整归属当前 zone 的页块 (pageblock), 换言之，不能跨 zone 处理页块。

每个 zone 有严格的页帧号（PFN, Page Frame Number）范围，由 zone->zone_start_pfn（起始 PFN）和 zone->zone_end_pfn（结束 PFN）界定，zone 仅管理自身范围内的 PFN。zone_spans_pfn() 作用是检查传入的 PFN 是否落在当前 zone 的合法范围内（zone->zone_start_pfn pfn < zone->zone_end_pfn）。

start_pfn 是 pageblock 对齐后的 PFN，可能跨 zone。传入的 page 是 zone 内的“锚点页” — 调整 start_page = page，是将起点从“pageblock 对齐的非法跨 zone 页”切换为“zone 内合法页”。

end_pfn 则是按页块对齐后的 end 的 PFN, 同样可能跨 zone. 这里不会调整 end_page 至 zone 区域的 end 边界, 而会立即返回 0。

页块是一个物理连续的内存单元，长度为 [start_pfn,start_pfn + pageblock_nr_pages) — 在内存域 zone 边界截断会破坏页块的物理完整性。pageblock_flags 按 start_pfn 索引，而非按 zone 索引 — 为截断的页块修改标记，会污染整个物理页块的元数据；

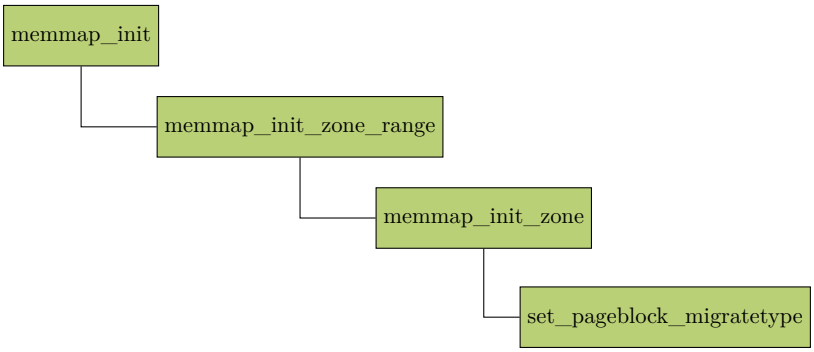
调整 start_page（针对跨 zone 的 start_pfn）是安全的：我们仅使用该 zone 内的有效页，不修改任何元数据—页块的全局标记 (zone->pageblock_flags) 仍保持不变（归属于页块真正的 start_pfn 所在的内存 zone）；

与调整 start_pfn（我们可在不触碰元数据的情况下，将 start_page 调整为内存域内的有效页）不同，调整 end_pfn 会强制为不完整的页块修改元数据,pageblock_flags 是绑定到 start_pfn 的“全局属性”，调整 end_pfn 会导致“整个 pageblock 元数据 (zone->pageblock_flags) 的修改”但“仅处理 zone 内部分页”，引发跨 zone 迁移类型误判。

steal_suitable_fallback() 首先调用 move_freepages_block() 迁移页，后续如果满足一定条件则调用 set_pageblock_migratetype() 改变整个 pageblock 页面块的迁移类型。也就是修改 zone->pageblock_flags 位图元数据中该 pageblock 对应的迁移类型位。

==========end

初始化时通过 set_pageblock_migratetype() 设置每个 pageblock 的迁移类型。到 set_pageblock_migratetype() 的整个调用流程如下所示:



memmap_init() → *memmap_init_zone_range()* → *memmap_init_zone()* → *set_pageblock_migratetype()*

```
572 void set_pageblock_migratetype(struct page *page, int migratetype)
573 {
574     if (unlikely(page_group_by_mobility_disabled &&
575                  migratetype < MIGRATE_PCPTYPES))
576         migratetype = MIGRATE_UNMOVABLE;
577 
578     set_pfnblock_flags_mask(page, (unsigned long)migratetype,
579                             page_to_pfn(page), MIGRATETYPE_MASK);
580 }
```

实际调用的函数如下:

nemmap_init() ⋯ memmap_init_zone() set_pageblock_migratetype() set_pfnblock_flags_mask()

```
542 void set_pfnblock_flags_mask(struct page *page, unsigned long flags,
543                     unsigned long pfn,
544                     unsigned long mask)
545 {
546     unsigned long *bitmap;
547     unsigned long bitidx, word_bitidx;
548     unsigned long old_word, word;
549
550     BUILD_BUG_ON(NR_PAGEBLOCK_BITS != 4);
551     BUILD_BUG_ON(MIGRATE_TYPES > (1 << PB_migratetype_bits));
552
553     bitmap = get_pageblock_bitmap(page, pfn);
554     bitidx = pfn_to_bitidx(page, pfn);
555     word_bitidx = bitidx / BITS_PER_LONG;
556     bitidx &= (BITS_PER_LONG-1);
557
558     VM_BUG_ON_PAGE(!zone_spans_pfn(page_zone(page), pfn), page);
559
560     mask <<= bitidx;
561     flags <<= bitidx;
562
563     word = READ_ONCE(bitmap[word_bitidx]);
564     for (;;) {
565         old_word = cmpxchg(&bitmap[word_bitidx], word, (word & ~mask) | flags);
566         if (word == old_word)
567             break;
568         word = old_word;
569     }
570 }
```

- 550 每个 pageblock 在 pageblock_flags 位图里占用的总比特位数内核固定定义为 4。修改该数值会直接编译失败，防止后续位运算逻辑出错。
- 553 根据传入的 struct page，找到该页所属 pageblock 的 pageblock_flags 位图。

  zone->pageblock_flags 是一个 unsigned long 全局位图指针。每一个 pageblock 占用 $NR_PAGEBLOCK_BITS = 4$ 个 bit; 多个 pageblock 的 bit 标记拼接存在一个 unsigned long 里，函数 get_page_bitmap() 的任务就是定位到目标 pageblock 的位图。
- 554 算出指定的物理页帧所属 pageblock 在 pageblock_flags 位图里的起始 bit 。也就是这个 pageblock 的 4 个标记位，在位图的第几个 bit 开始存放。(函数见后面)
- 555 算出目标 pageblack 的标记所在的 bit 在位图的第几个字 (word) ,CONFIG_64BIT=y 时，一个字 64 bit。否则通常是 32 bit。
- 560 mask 是操作掩码，只操作目标 pageblock 的 4 bit。
- 561 这是待写入的 4 bit, 也移位和 mask 对齐。
- 563 取出承载当前 pageblock 标记 4 bit 的那一个 64 bit(或 32 bit, 随平台不同会有变化) 位图单元。
- 564-569 把目标 pageblock 新的 flags 更新到位图单元。

  cmpxchg 硬件原子自旋保证并发安全，不需要上锁。

nemmap_init() ⋯ set_pageblock_migratetype() set_pfnblock_flags_mask() pfn_to_bitidx()

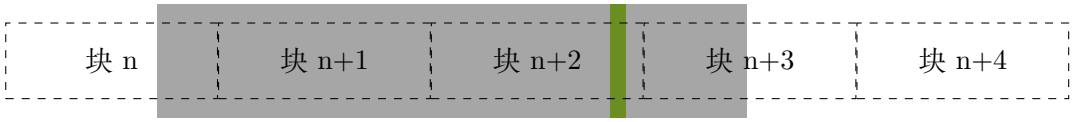
```
488 static inline int pfn_to_bitidx(struct page *page, unsigned long pfn)
489 {
490 #ifdef CONFIG_SPARSEMEM
491     pfn &= (PAGES_PER_SECTION-1);
492 #else
493     pfn = pfn - round_down(page_zone(page)->zone_start_pfn, pageblock_nr_pages);
494 #endif /* CONFIG_SPARSEMEM */
495     return (pfn >> pageblock_order) * NR_PAGEBLOCK_BITS;
496 }
```

这里先不关注 CONFIG_SPARSEMEM=y 的场景。

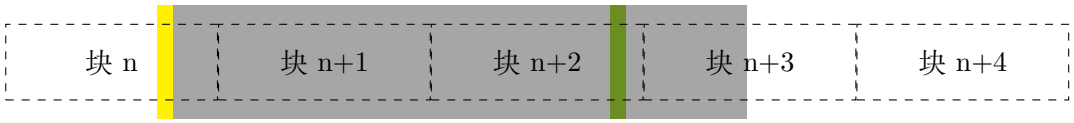
- 493 获取 目标页的物理页帧 – 目标页所在 zone 的起始页的 pageblock 的起始物理页帧 的差。

  page_zone(page)->zone_start_pfn 是目标 page 所在 zone 的起始物理页帧号。按 pageblock 的页数目取模后得到值再前向对齐，得到 zone 的起始页所在 pageblock 的起始物理页帧号。

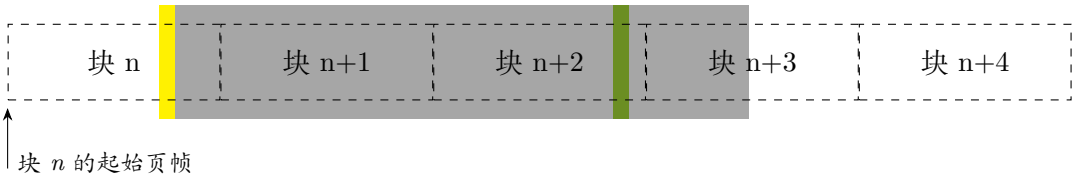
  如下图，虚线框表示 pageblock, 灰色部分表示一个 zone(内存区域), 绿色部分表示目标页帧。



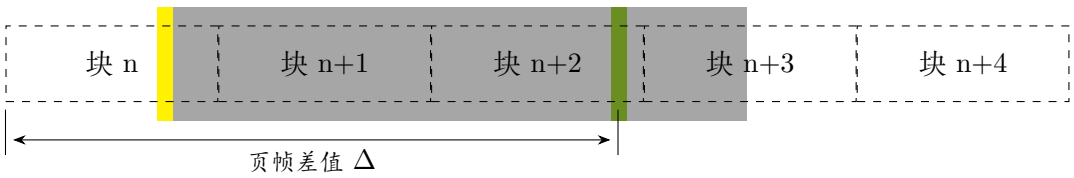
目标 page 所在 zone 的起始物理页帧是下图黄色部分。



将这个页帧 (黄色部分) 按 pageblock 的页数目取模后得到值再前向对齐，得到 zone 的起始页所在 pageblock 的起始物理页帧号。如图中箭头所示位置。



所以 493 行最终的计算结果如下图的差值:



- 495 计算得到目标 page 所在 pageblock 的标记在目标 zone 位图中的偏移。

  也就是:

$$\frac{\text{页帧差值}\Delta}{1\text{ 个页块 (pageblock) 包含的页}} \times 4 = \text{标记在位图中的偏移}$$

  仍以上图为例，假设是在 32 bit 平台。该例 pageblock 的 4 bit 标记在位图中的偏移如下图，为: $1 << (2 \times 4)$, 即 $1 << 8$。



当然这里的示意图做了简化，在真实场景下 pageblock 的标记偏移可能远大于 32 bit 或 64 bit。这时候就要先找到存储 pageblock 标记的整数的位置，然后在这个整数里再算得偏移。这也是前面 *set_pfnblock_flags_mask()* 函数 555、556 行做的事。

rmqueue()3463 行调用的 ___rmqueue() 函数如下，___rmqueue() 执行从伙伴分配器中移除一个元素的核心工作。由于它操作的是 zone 级别的全局空闲页链表，因此调用此函数前，必须先持有对应内存域（zone）的全局自旋锁 zone->lock 来保证多 CPU 并发访问时空闲链表的数据一致性。

___alloc_pages_nodemaskn() > *get_page_from_freelist()* > *rmqueue()* > *___rmqueue()*

```
2840 static __always_inline struct page *
2841 __rmqueue(struct zone *zone, unsigned int order, int migratetype,
2842                         unsigned int alloc_flags)
2843 {
2844     struct page *page;
2845 
2846 #ifdef CONFIG_CMA
2847     /*
2848      * Balance movable allocations between regular and CMA areas by
2849      * allocating from CMA when over half of the zone's free memory
2850      * is in the CMA area.
2851      */
2852     if (alloc_flags & ALLOC_CMA &&
2853         zone_page_state(zone, NR_FREE_CMA_PAGES) >
2854         zone_page_state(zone, NR_FREE_PAGES) / 2) {
2855         page = __rmqueue_cma_fallback(zone, order);
2856         if (page)
2857             return page;
2858     }
2859 #endif
2860 retry:
2861     page = __rmqueue_smallest(zone, order, migratetype);
2862     if (unlikely(!page)) {
2863         if (alloc_flags & ALLOC_CMA)
2864             page = __rmqueue_cma_fallback(zone, order);
2865 
2866         if (!page && __rmqueue_fallback(zone, order, migratetype,
2867                                 alloc_flags))
2868             goto retry;
2869     }
2870 
2871     trace_mm_page_alloc_zone_locked(page, order, migratetype);
2872     return page;
2873 }
```

- 2852-2858 为了避免 CMA 区域闲置浪费，平衡常规区域和 CMA 区域的内存使用，在 CMA 空闲页大于整个 zone 空闲页的 50% 时，直接优先从 CMA 分配。
- 2861 调用 ___rmqueue_smallest 执行最优路径分配，这是是 buddy 分配器的核心分配逻辑。该函数会先查找与 order 完全匹配的空闲块，只有找不到时才拆分更大阶的块（如拆 order=3 的 8 页内存块为两个 order=2 的 4 页内存块），目的是尽量减少大页块拆分避免内存碎片。

- 2862 进入该分支说明最优的路径执行分配失败，启动后续的” 备选 (fallback)” 策略。
- 2863-2864 常规路径走不通时，考虑第一级备选策略 — 动用 CMA 区域。如果允许，则尝试从当前 zone 的 CMA 区域分配 order 阶数的连续页块。

  如果不打开 CONFIG_CMA 的话，___rmqueue_cma_fallback() 实际上是个空函数，直接返回 NULL.
- 2866-2867 如果从 CMA 区域进行备选内存的分配也失败，那么执行第二级备选策略 — 跨迁移类型从其他迁移类型的空闲链表中“借用” 页块，甚至拆分更大阶的块，同时调整迁移类型的元数据。这就突破了 ___rmqueue_smallest() 函数” 仅在指定 zone（内存域）、指定 migratetype（迁移类型）的空闲链表中查找” 的限制。

  ” 借用” 的页块会调整迁移类型数据，迁移到指定迁移类型的 freelist 链表。因此跳回 ___rmqueue_smallest() 重新执行分配 (因为此时指定迁移类型的 freelist 上新增了” 借用” 的空闲块，此时 ___rmqueue_smallest() 会成功拿到页块)。

  ___rmqueue_fallback 仅” 准备空闲块”，不直接分配，通过跳转复用 ___rmqueue_smallest 的分配逻辑，保证了所有分配路径的统一性避免代码重复。

___rmqueue_smallest() 从指定迁移类型的空闲链表中，按 从小到大 的顺序查找满足分配请求的最小 order(阶) 的内存块。仅在指定 zone（内存域）、指定 migratetype（迁移类型）的空闲链表中查找，保证“同类型页块集中分配” 的碎片控制策略。

源码如下:

__alloc_pages_nodemaskn() ··· rmqueue() ___rmqueue() ___rmqueue_smallest()

```
2307 static __always_inline
2308 struct page *__rmqueue_smallest(struct zone *zone, unsigned int order,
2309                         int migratetype)
2310 {
2311     unsigned int current_order;
2312     struct free_area *area;
2313     struct page *page;
2314
2315     /* Find a page of the appropriate size in the preferred list */
2316     for (current_order = order; current_order < MAX_ORDER; ++current_order) {
2317         area = &(zone->free_area[current_order]);
2318         page = get_page_from_free_area(area, migratetype);
2319         if (!page)
2320             continue;
2321         del_page_from_free_list(page, zone, current_order);
2322         expand(zone, page, order, current_order, migratetype);
2323         set_pcppage_migratetype(page, migratetype);
2324         return page;
2325     }
2326
2327     return NULL;
2328 }
```

- 2316 从申请的 order 到最大 order(即 MAX_ORDER-1) 逐个迭代尝试，从特定迁移类型的空闲链表中查找合适大小的页块。
- 2317-2320 在当前 order 下特定迁移类型的空闲链表中查找空闲页块，如果不存在满足需求的空闲页块，则继续迭代。

- 2321 如果当前 order 存在空闲的合适页块，则从空闲链表摘出该页块。
- 2322 如果从大阶 (>order) 分配到了满足大于需求 order 的页块，那么该页块需按伙伴系统的行为拆分成小的块，而这就是 expand() 函数做的事。
- 2323 设置单个页面的 迁移类型缓存值 （存储在 page->index 中），用于优化页面释放到 per–CPU 页列表（pcplist） 时的效率。将从 free_area 取出空闲页的迁移类型写入 page->index，也就是为该页的 per–cpu 缓存做标记。PCP 管理该页时，直接读取 page->index 就能知道迁移类型，无需通过加锁再去查找全局数据 zone->pageblock_flags。

expand() 会将一个大阶的物理页块拆分为小阶的页块，并将拆分后的空闲页块加入对应阶的空闲链表。函数源码如下:

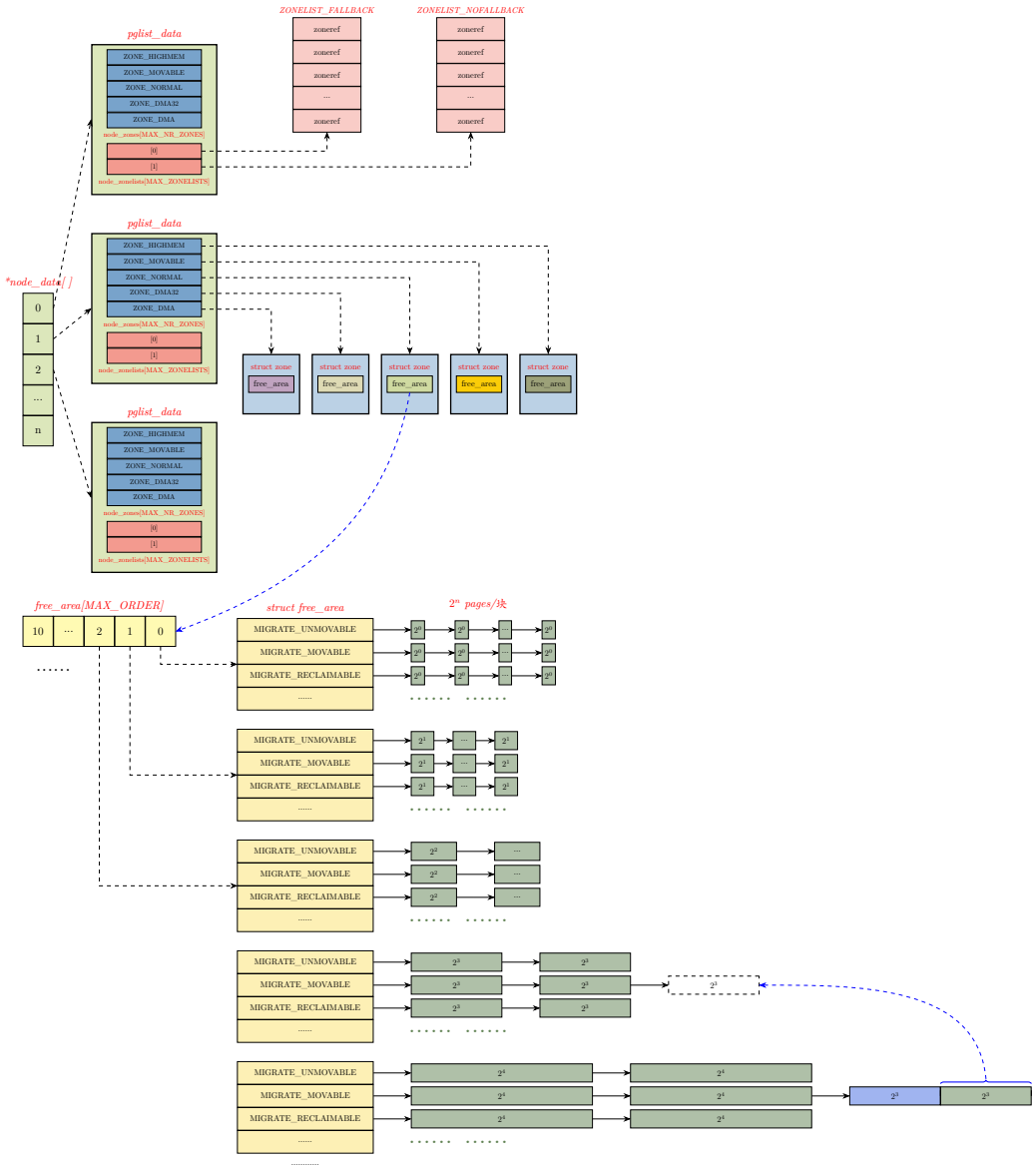
__alloc_pages_nodemaskn() › … › *__rmqueue()* › *__rmqueue_smallest()* › *expand()*

```
2161 static inline void expand(struct zone *zone, struct page *page,
2162     int low, int high, int migratetype)
2163 {
2164     unsigned long size = 1 << high;
2165
2166     while (high > low) {
2167         high--;
2168         size >>= 1;
2169         VM_BUG_ON_PAGE(bad_range(zone, &page[size]), &page[size]);
2170
2171         /*
2172          * Mark as guard pages (or page), that will allow to
2173          * merge back to allocator when buddy will be freed.
2174          * Corresponding page table entries will not be touched,
2175          * pages will stay not present in virtual address space
2176          */
2177         if (set_page_guard(zone, &page[size], high, migratetype))
2178             continue;
2179
2180         add_to_free_list(&page[size], zone, high, migratetype);
2181         set_buddy_order(&page[size], high);
2182     }
2183 }
```

- 2164 获取当前将拆分的大阶页块包含的总页数.
- 2166 只要当前拆分的大阶页块的阶数 high 高于目标阶数 low，就继续拆分。
- 2167-2168 拆分大阶页面本质上就是将一个页面拆分为两个小一阶的页面，把其中一个放到对应的空闲链表，然后对另一个页块重复此过程，直到最终得到想要的 order 大小的页面及其伙伴页 — 伙伴页是最后被放到空闲链表中的那个。

  high 被动态设置为当次迭代将拆分的大阶页块的阶数，low 为所需的阶数。循环 high - low 次，将 high - low 个页面放入空闲链表。每次循环开始时递减 high，这意味着大阶页块被平均拆成两个小一阶的页块，直到满足需求。
- 2177 set_page_guard() 的调用仅在 CONFIG_DEBUG_PAGEALLOC 被设置时才有用。
- 2180 在迭代过程中，被拆分页块的后半部分通过 add_to_free_list() 函数释放回空闲链表。

- 2181 使用 set_buddy_order() 函数正确设置其伙伴阶数。



譬如—见上图—如果从 order=4 的 freelist 中获取了一个空闲 movable 的页块，但是实际需要的是 order=2 的目标页块。那么 expand() 会把改 order=4 的页块拆成 2 个 order=3 的页块，把后半个 order=3 的页块放到 freelist oreder=3 的空闲链表，剩下的 order=3 的页块则继续按此步骤迭代拆分，直到获得目标阶 order=2 大小的页块。

__alloc_pages_nodemaskn() › ... › *__rmqueue_smallest()* › *expand()* › *set_page_guard()*

```
758 static inline bool set_page_guard(struct zone *zone, struct page *page,
759                 unsigned int order, int migratetype)
760 {
761     if (!debug_guardpage_enabled())
762         return false;
763
```

```
764     if (order >= debug_guardpage_minorder())
765         return false;
766 
767     __SetPageGuard(page);
768     INIT_LIST_HEAD(&page->lru);
769     set_page_private(page, order);
770     /* Guard pages are not available for any usage */
771     __mod_zone_freepage_state(zone, -(1 << order), migratetype);
772 
773     return true;
774 }
```

___build__all__zonelists() 用来创建各个 node 的备用 zonelist。该 zonelist 决定了在本地内存不足时，内核按何种顺序尝试其他内存域 (zone) 或节点 (node)。

该函数主要在以下两种场景下被调用:

- 系统启动阶段，在 start__kernel() 初始化流程中，build__all__zonelists() 被显示调用 (其中会调用到 ___build__all__zonelists()), 用于构建所有 NUMA 节点或 UMA 节点的内存域备用列表 zonelist.
- 当软件流程动态增删内存时，build__all__zonelists() 会被重新调用，以更新全局内存拓扑。确保内存拓扑变化被实时更新到分配策略。

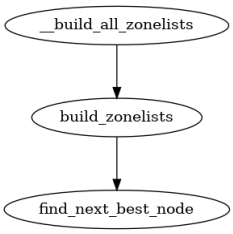


图 4–4: 调用图: ___build__all__zonelists()

mm/page__alloc.c :: ___build__all__zonelists()

```
5911 static void __build_all_zonelists(void *data)
5912 {
5913     int nid;
5914     int __maybe_unused cpu;
5915     pg_data_t *self = data;
5916     static DEFINE_SPINLOCK(lock);
5917 
5918     spin_lock(&lock);
5919 
5920 #ifdef CONFIG_NUMA
5921     memset(node_load, 0, sizeof(node_load));
5922 #endif
5923 
5924     /*
5925      * This node is hotadded and no memory is yet present.   So just
5926      * building zonelists is fine - no need to touch other nodes.
5927      */
5928     if (self && !node_online(self->node_id)) {
5929         build_zonelists(self);
```

```
5930     } else {
5931         for_each_online_node(nid) {
5932             pg_data_t *pgdat = NODE_DATA(nid);
5933 
5934             build_zonelists(pgdat);
5935         }
5936 
5937 #ifdef CONFIG_HAVE_MEMORYLESS_NODES
5938         /*
5939          * We now know the "local memory node" for each node--
5940          * i.e., the node of the first zone in the generic zonelist.
5941          * Set up numa_mem percpu variable for on-line cpus.  During
5942          * boot, only the boot cpu should be on-line;  we'll init the
5943          * secondary cpus' numa_mem as they come on-line.  During
5944          * node/memory hotplug, we'll fixup all on-line cpus.
5945          */
5946         for_each_online_cpu(cpu)
5947             set_cpu_numa_mem(cpu, local_memory_node(cpu_to_node(cpu)));
5948 #endif
5949     }
5950 
5951     spin_unlock(&lock);
5952 }
```

- 5914 ***___maybe_unused*** 是编译器标记, 该宏告诉编译器变量 cpu 可能不用，消除 ***unused*** 警告。
- 5915 获取本次操作的目标节点的描述符指针。
- 5921 构建 (重建) zonelist 意味着内存节点拓扑发生变化，旧的节点负载值失效，清 0 所有节点负载值准备重新置负载数据。

  系统开机内存管理初始化会第一次构建全系统的 zonelist；内存热插拔新增或下线 NUMA 节点，也会触发 zonelist 刷新。
- 5928-5929 当前是热插拔新增空节点 (还不在线)，只需要单独构建该节点自身的 zonelist，不需要遍历刷新全系统所有节点。
- 5931-5935 刷新所有在线节点的 zonelist。对所有 N_ONLINE 状态的内存 node 轮询迭代，在每次迭代过程中获取对应 node 的 pg_data_t 管理结构体的地址，然后调用 build_zonelists() 来构建该 node 的备选 zonelist。
- 5937-5848 部分架构存在仅含 cpu、不带本地内存的节点，需要单独修正 cpu 归属内存节点。

build_zonelists() 为单个 NUMA 节点生成一套跨节点顺序备选的数组 node_order，再基于这个节点优先级链表构建该节点专属的 zonelist(内存分配备选链表)。

核心逻辑：按 NUMA 物理距离远近 + 均衡负载，排序所有可用内存节点。本地节点放最前面，远端节点按距离递增排列，同距离节点做分流避免单节点压力过大。

___build_all_zonelists() > *build_zonelists()*

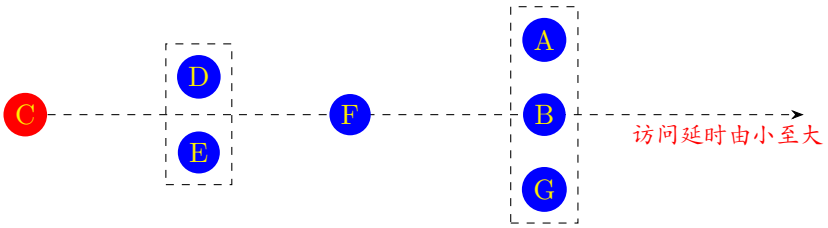
```
5799 static void build_zonelists(pg_data_t *pgdat)
5800 {
5801     static int node_order[MAX_NUMNODES];
5802     int node, load, nr_nodes = 0;
5803     nodemask_t used_mask = NODE_MASK_NONE;
5804     int local_node, prev_node;
5805 
```

```
5806     /* NUMA-aware ordering of nodes */
5807     local_node = pgdat->node_id;
5808     load = nr_online_nodes;
5809     prev_node = local_node;
5810
5811     memset(node_order, 0, sizeof(node_order));
5812     while ((node = find_next_best_node(local_node, &used_mask)) >= 0) {
5813         /*
5814          * We don't want to pressure a particular node.
5815          * So adding penalty to the first node in same
5816          * distance group to make it round-robin.
5817          */
5818         if (node_distance(local_node, node) !=
5819             node_distance(local_node, prev_node))
5820             node_load[node] = load;
5821
5822         node_order[nr_nodes++] = node;
5823         prev_node = node;
5824         load--;
5825     }
5826
5827     build_zonelists_in_node_order(pgdat, node_order, nr_nodes);
5828     build_thisnode_zonelists(pgdat);
5829 }
```

- 5801 node_order[] 是静态保存输出结果的数组，存放排序好的节点 ID 序列。数组中节点存放的顺序 = 内存分配的备选优先级，数组中靠前的节点优先分配内存，本地节点永远排第一个。
- 5807 参数 pgdat 关联的内存作为本地 node。
- 5808 在线内存 node 的数量。
- 5812 调用 find_next_best_node() 查找合适的内存 node, 并根据距离远近、访问的频繁程度 (目标 node 上负载的高低)，选择访问代价最低的 node 做为合适 node。并把选中的 node id 置位到已选中 node 的位图 used_mask 中。在后续迭代中不会再次返回已经选中过的 node id。

  find_next_best_node() 返回值 ≥ 0 表示找到了 node, 返回值 -1(NUMA_NO_NODE) 表示未找到。
- 5818-5820 一旦发现当前 node 和前一个 prev_node 不属于同一个 NUMA 距离 (相同 distance) 分组，就给当前 node 加一些惩罚值 (1) 作为负载。

  NUMA 按 distance 将 node 分组, 距离相同的 node 归为一组，优先选距离近的组里的 node；同一 distance 分组内有多个 node 时，优先选负载更低的 node。



说明：
1. 图中每个圆点表示一个 NUMA 内存节点。
2. 虚线框表示框内的内存节点访问延时相同, 是同一个” 距离组”
3. 圆点 C 代表本地节点。

如上图，本地节点 C 距离 (访问延时)NUMA 里几个 node 节点 A/B/G 的距离相同。那么 find_next_best_node() 查找时按距离 distance 访问的延迟差异是无法区分 A/B/G 优先级的。因为本地内存节点 C 对应的 cpu core 访问候选池中 A/B/G 的距离代价相同，防止分配逻辑每次都优先挑 A(同距离节点内部是按 node 编号顺序固定存放的，代码遍历候选 node 时是按编号顺序遍历，所以会始终选同一个 node)，如果每次都优先选同一个，会造成该节点内存分配压力过大。通过给同距离组第一个节点写入惩罚值 $node_load[node] = load$，下次 find_next_best_node() 计算负载权重时会避开它，实现同距离节点轮询分配。

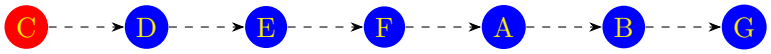
并且下次查询到 A/B/G 这个距离分组时，A 因为 load 值高会排在本距离组的其它 node 之后。这是 B 会称为新的优先节点，那么本次会给 B 写惩罚值，从而下次不会重复选择节点 B。

当前节点与本地节点的距离，不同于上一个节点与本地节点的距离时，说明进入了一个新的“距离组”。此时，为该节点设置当前的 load 值作为其“优先级权重”。换言之，当目标 node 和前一个 node 分别与本地 node 的 distance 不同时，意味着从一个 NUMA 的 distance 分组进入了另一个不同的 NUMA 的 distance 分组。如果该 distance 分组里多个 node, 那么每次都会优先选同一个。因此默认给该 distance 分组的第一个 node 的负载值 node_load[node] 临时赋予一个惩罚值。这样当**外层**下一次大循环把紧接着又选同一个 node 做为备选 node 时，会因为额外的负载值认为第一个 node 的负载值过重 (或权重相对高)，从而会放到同 distance 这组其它 node 之后来考虑。

需要注意的是，5812 行这里虽然通过 used_mask 来过滤已经选中的 node, 但是在外层的大循环为其它 node 构建 zonelist 时，used_mask 会重新初始化为 0，因此还是可能会在相同 NUMA 距离的分组优先选到同一个 node。所以同 distance 分组的惩罚值逻辑和 mask 过滤是两套独立逻辑，并不矛盾。

- 5822 在 5812 的迭代中，将每次返回的最优备选内存 node，依次顺序放入用于结果输出的 node_order 数组。

  前图中对本地节点 C，最后输出到节点访问顺序如下 (下图只是示意，真实应该是数组)。因为 C 是本地节点，所以最优选择 C 节点。

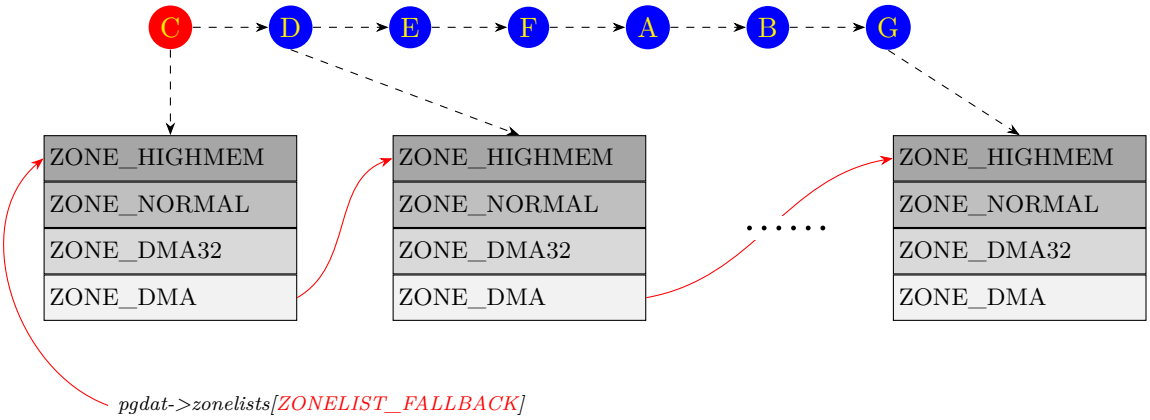


说明：
1. 图中每个圆点表示一个 NUMA 内存节点。
2. 圆点 C 代表本地节点。

- 5824 load 递减，确保下一个新距离组的第一个节点获得更低的优先级。也就是为每个不同距离组第一个节点生成递减的权重。

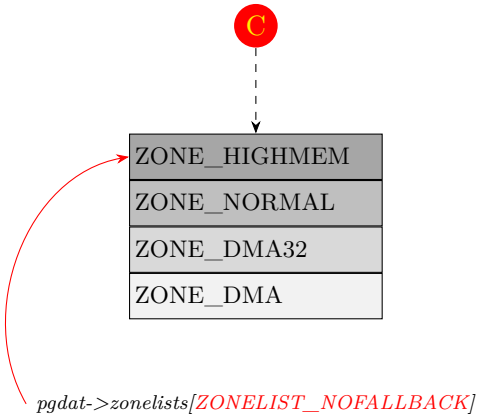
- 5827 按前面输出到节点顺序 (node order) 为本地节点构建由不同 node 上的 zone 组成的备选链表 *pgdat->zonelists[ ]*。

  譬如，为前面的 node C 节点构成的备选内存 zone 的链表如下。需要注意的是: 下面只是示意图，不同 NUMA 节点上的 zone 内存域不是对称的，也就是说每个 node 的 zone 数量不一定相同。



说明：
1. 图中每个圆点表示一个 NUMA 内存节点。
2. 圆点 C 代表本地节点。
3. 红色实线表示内存区域备选链表。

- 5828 为本地节点创建*ZONELIST_NOFALLBACK*的备选链表。只在本地 node 的内存域上备选。



说明：
1. 圆点 C 代表本地节点。
2. 红色实线表示内存区域备选链表。

按指定节点顺序填充节点的备选 (fallback) 内存域链表 (zoneref 数组)。

__build_all_zonelists() > *build_zonelists()* > *build_zonelists_in_node_order()*

```
5757 static void build_zonelists_in_node_order(pg_data_t *pgdat, int *node_order,
5758         unsigned nr_nodes)
5759 {
5760     struct zoneref *zonerefs;
5761     int i;
5762 
5763     zonerefs = pgdat->node_zonelists[ZONELIST_FALLBACK]._zonerefs;
5764 
5765     for (i = 0; i < nr_nodes; i++) {
5766         int nr_zones;
5767 
5768         pg_data_t *node = NODE_DATA(node_order[i]);
```

```
5769
5770            nr_zones = build_zonerefs_node(node, zonerefs);
5771            zonerefs += nr_zones;
5772        }
5773        zonerefs->zone = NULL;
5774        zonerefs->zone_idx = 0;
5775    }
```

- 5763 获取当前节点内存域备选 (fallback) 链表的地址。这是为了定位到 fallback 链表数组开头，下面代码准备依次填充每个备选 zone。
- 5768 获取当前循环要处理的备选节点的节点描述符指针。
- 5770 遍历目标备选节点所有的 zone，把目标 node 所有可用 zone 依次写入当前 node 的备选内存域 zone 数组。返回写入的 zone 数目。
- 5771 当前 node 的备选内存域 zone 数组向后移动，为下一次写入做准备。
- 5773-5774 置 NULL，作为终止标志。相应的 zone_idx 填 0 仅做占位，无实际业务含义。

逆序遍历该节点所有 zone，过滤掉无可用内存的空 zone，把有效 zone 写入 zoneref 数组，最后返回当前节点有效 zone 的总个数。

__build_all_zonelists() > *build_zonelists()* > *build_zonelists_in_node_order()* > *build_zonerefs_node()*

```
5640    static int build_zonerefs_node(pg_data_t *pgdat, struct zoneref *zonerefs)
5641    {
5642        struct zone *zone;
5643        enum zone_type zone_type = MAX_NR_ZONES;
5644        int nr_zones = 0;
5645
5646        do {
5647            zone_type--;
5648            zone = pgdat->node_zones + zone_type;
5649            if (managed_zone(zone)) {
5650                zoneref_set_zone(zone, &zonerefs[nr_zones++]);
5651                check_highest_zone(zone_type);
5652            }
5653        } while (zone_type);
5654
5655        return nr_zones;
5656    }
```

- 5643 MAX__NR__ZONES 是枚举的 zone 的上限。这里将用于枚举的变量 zone_type 初始赋值为最大值，用于逆序遍历 (从高编号 zone 往低编号 zone 走)。
- 5646-5653 这个循环中依次遍历目标 node 下的所有内存区域 zone, 并且目标 zone 有可管理的内存，则将该 zone 放入目标 node 的备选内存域数组。空 zone（无物理内存）直接跳过，不加入 fallback 链表。
- 5655 当前 NUMA 节点有效可用 zone 的总数量。

根据输入基准节点 node + 已选用节点掩码 used__node__mask，从未被选中的带内存的 NUMA 节点里，选出从基准节点访问该节点代价最小的做为最优节点，以之作为 zonelist 下一个备选节点。

__build_all_zonelists() > *build_zonelists()* > *find_next_best_node()*

```
5707 static int find_next_best_node(int node, nodemask_t *used_node_mask)
5708 {
5709     int n, val;
5710     int min_val = INT_MAX;
5711     int best_node = NUMA_NO_NODE;
5712 
5713     /* Use the local node if we haven't already */
5714     if (!node_isset(node, *used_node_mask)) {
5715         node_set(node, *used_node_mask);
5716         return node;
5717     }
5718 
5719     for_each_node_state(n, N_MEMORY) {
5720 
5721         /* Don't want a node to appear more than once */
5722         if (node_isset(n, *used_node_mask))
5723             continue;
5724 
5725         /* Use the distance array to find the distance */
5726         val = node_distance(node, n);
5727 
5728         /* Penalize nodes under us ("prefer the next node") */
5729         val += (n < node);
5730 
5731         /* Give preference to headless and unused nodes */
5732         if (!cpumask_empty(cpumask_of_node(n)))
5733             val += PENALTY_FOR_NODE_WITH_CPUS;
5734 
5735         /* Slight preference for less loaded node */
5736         val *= (MAX_NODE_LOAD*MAX_NUMNODES);
5737         val += node_load[n];
5738 
5739         if (val < min_val) {
5740             min_val = val;
5741             best_node = n;
5742         }
5743     }
5744 
5745     if (best_node >= 0)
5746         node_set(best_node, *used_node_mask);
5747 
5748     return best_node;
5749 }
```

- 5714-5716 本地节点永远第一优先级，如果本地节点还没被选，直接返回本地节点 id，不走下面循环进行权重打分。

  5715 在表示已使用 node 的位图中将本地节点对应 bit 置 1，标记该节点已占用，后续循环不会再选。

- 5719 对有关联内存的 NUMA 节点循环迭代。

- 5722 如果节点已经在迭代中被选中过了，则跳过该节点重新迭代。

- 5726 计算传入的基准节点和迭代的目标节点之间” 距离”(distance)。

  node_distance(a,b) 返回的是两节点间内存访问的延迟权重。值越小延迟越小。

- 5729 迭代中的目标 node 号如果小于本地 node 号，那么会对此惩罚，加一个惩罚值 (+1) 到权重分里。因为更倾向于下一个 node。同等距离时，优先选节点 ID 更大的远端节点，进行轻微均衡。

- 5732-5733 优先考虑无主 (没有 cpu 关联的内存 node) 和未使用的节点, 如果迭代的内存 node 已经关联到某 cpu，那么给权重分里再加上一个惩罚值 PENALTY_FOR_NODE_WITH_CPUS。
- 5736 略微偏好负载较轻的节点
- 5736-5737 保证距离是第一权重、节点的负载是微调权重。这里把前面距离 + CPU 惩罚的总分放大一个超大倍数, 再叠加节点负载值作为惩罚值得到总和。因此：

  不同 NUMA 距离的节点，基础分差距巨大，负载无法反转距离优先级。

  但是同一距离分组内，节点负载 (node_load[n]) 更低的节点代价 (总分) 更小，优先被选中。

- 5739-5742 对比当前节点代价与全局最小代价：如果当前候选节点打分更低，更新最小代价，并记录当前节点为临时最优节点。
- 5745-5746 如果找到有效最优节点，在表示已使用 node 的位图中标记该节点已占用，下一轮循环不会重复选取。

5

内存回收

5.1 do_try_to_free_pages()

do_try_to_free_pages() 是直接页面回收的主要入口点。

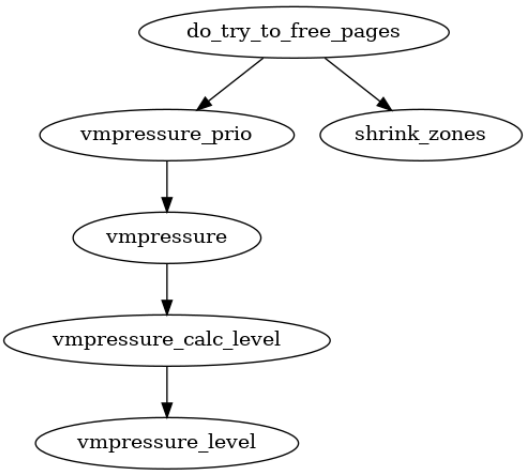


图 5–1: 调用图：do_try_to_free_pages()

系统内存不足时，do_try_to_free_pages() 会负责在 直接内存回收路径中同步释放内存 。通过回写脏页、释放缓存和降级活跃页，快速缓解内存压力。vmpressure_prio() 是内存压力统计的封装接口，接收回收优先级参数并调用 vmpressure()。vmpressure() 负责统计本轮回收的扫描页数与回收页数，监测回收效率。它调用 vmpressure_calc_level()，依据回收页数/扫描页数的效率比值计算内存压力等级。vmpressure_calc_level() 内部依靠 vmpressure_level() 将效率比值映射为压力等级。当压力等级发生变化，vmpressure 机制向用户空间发送内存压力事件通知。

mm/vmscan.c :: do_try_to_free_pages()

```
3012 static unsigned long do_try_to_free_pages(struct zonelist *zonelist,
3013                       struct scan_control *sc)
```

```
3014 {
3015     int initial_priority = sc->priority;
3016     pg_data_t *last_pgdat;
3017     struct zoneref *z;
3018     struct zone *zone;
3019 retry:
3020     delayacct_freepages_start();
3021 
3022     if (!cgroup_reclaim(sc))
3023         __count_zid_vm_events(ALLOCSTALL, sc->reclaim_idx, 1);
3024 
3025     do {
3026         vmpressure_prio(sc->gfp_mask, sc->target_mem_cgroup,
3027                 sc->priority);
3028         sc->nr_scanned = 0;
3029         shrink_zones(zonelist, sc);
3030 
3031         if (sc->nr_reclaimed >= sc->nr_to_reclaim)
3032             break;
3033 
3034         if (sc->compaction_ready)
3035             break;
3036 
3037         /*
3038          * If we're getting trouble reclaiming, start doing
3039          * writepage even in laptop mode.
3040          */
3041         if (sc->priority < DEF_PRIORITY - 2)
3042             sc->may_writepage = 1;
3043     } while (--sc->priority >= 0);
3044 
3045     last_pgdat = NULL;
3046     for_each_zone_zonelist_nodemask(zone, z, zonelist, sc->reclaim_idx,
3047                     sc->nodemask) {
3048         if (zone->zone_pgdat == last_pgdat)
3049             continue;
3050         last_pgdat = zone->zone_pgdat;
3051 
3052         snapshot_refaults(sc->target_mem_cgroup, zone->zone_pgdat);
3053 
3054         if (cgroup_reclaim(sc)) {
3055             struct lruvec *lruvec;
3056 
3057             lruvec = mem_cgroup_lruvec(sc->target_mem_cgroup,
3058                                zone->zone_pgdat);
3059             clear_bit(LRUVEC_CONGESTED, &lruvec->flags);
3060         }
3061     }
3062 
3063     delayacct_freepages_end();
3064 
3065     if (sc->nr_reclaimed)
3066         return sc->nr_reclaimed;
3067 
3068     /* Aborted reclaim to try compaction? don't OOM, then */
3069     if (sc->compaction_ready)
3070         return 1;
3071 
3072     /*
3073      * We make inactive:active ratio decisions based on the node's
3074      * composition of memory, but a restrictive reclaim_idx or a
3075      * memory.low cgroup setting can exempt large amounts of
3076      * memory from reclaim. Neither of which are very common, so
3077      * instead of doing costly eligibility calculations of the
```

```
3078         * entire cgroup subtree up front, we assume the estimates are
3079         * good, and retry with forcible deactivation if that fails.
3080         */
3081        if (sc->skipped_deactivate) {
3082            sc->priority = initial_priority;
3083            sc->force_deactivate = 1;
3084            sc->skipped_deactivate = 0;
3085            goto retry;
3086        }
3087
3088        /* Untapped cgroup reserves?  Don't OOM, retry. */
3089        if (sc->memcg_low_skipped) {
3090            sc->priority = initial_priority;
3091            sc->force_deactivate = 0;
3092            sc->memcg_low_reclaim = 1;
3093            sc->memcg_low_skipped = 0;
3094            goto retry;
3095        }
3096
3097        return 0;
3098    }
```

- 3015 sc->priority 用于控制回收扫描的激进程度 (值范围 0-12)，初始值 12 表示温和回收，之后逐轮降低以提高回收激进程度。

  有资料称之为回收优先级，也可理解为扫描力度，代表回收时是否愿意扫描更多的页，愿意多大代价回收内存。分为两大维度:

  - 控制一次扫描欠活跃链表上多少页。
  - 控制 shrink_slab() 回收容忍度，也就是控制它可以回收的对象的范围，换言之，也就是控制哪些对象可以被回收。

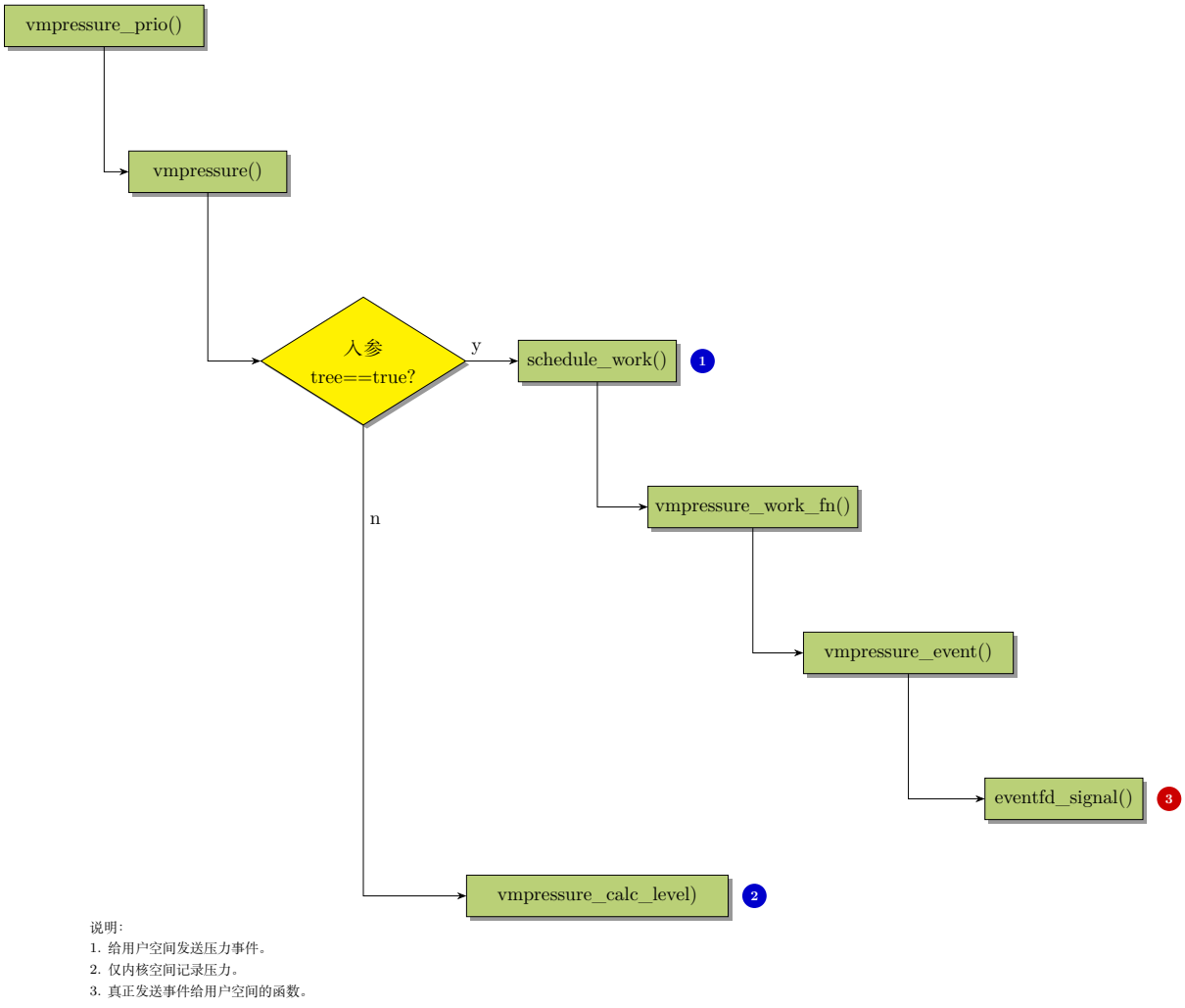
    ===== 待确认

- 3020 用于回收处理持续时间的统计。此处会调用 ktime_get_ns() 记录起始时间，3063 行会记录回收结束时间，它们的差值即是持续时间。

  ktime_get_ns() 用于获取内核 CLOCK_MONOTONIC 时钟的当前值，该时钟不受墙上时钟调整影响，不会倒流。但是会在休眠期间停止计数，醒来后继续。通常在内核被用于计算代码执行耗时，是内核高精度时间间隔测量的接口。

- 3022-3023 如果不是局部的内存控制组的回收，则统计全局内存压力事件。

- 3026-3027 根据扫描优先级通知内存压力事件至用户空间。



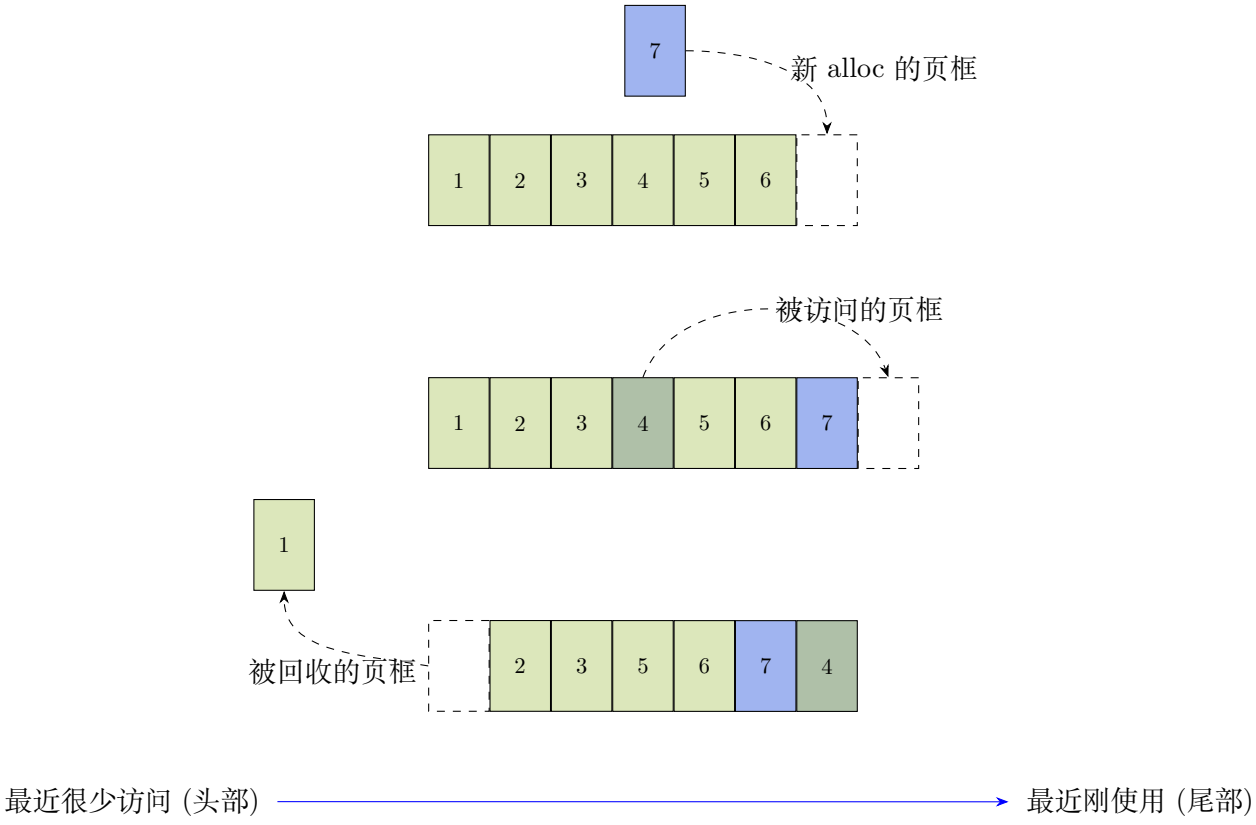
- 3028 重置本轮扫描的页面计数。
- 3029 扫描目标 zonelist 的所有内存 zone 区域回收页面。通过 LRU 算法选择页面回收，优先回收欠活跃文件页和匿名页。

NOTE

匿名页、文件页：归属用户进程使用，但由内核管理；它们是用户态页面，由伙伴系统统一通过 *LRU* 管理.

slab：归属内核空间使用，也由内核管理。不挂在任何 *LRU* 链表上，它有自己独立的回收机制。可以理解为暂时脱离了伙伴系统的管理。

为了最高效地利用系统中的内存，内核会尽可能多地使用内存来缓存磁盘上的数据。虽然内核将文件缓存在内存中这方面非常积极，但当内存压力出现时 (即内存区域达到低水位线或最小水位线)，也必须同样积极地将这些缓存释放出来 (即内存回收)。为此，需要对物理页进行重要性排序，确保保留更有用的页帧，淘汰较少使用的页帧。LRU(最近最少使用) 缓存替换策略——顾名思义：总是淘汰最近最少使用的页帧。它的理想化形式如下 (该图进行了简化，实际上应该是双向链表)：



在理想化模型中 (上图)，刚被访问的页框被加到 LRU 链表头部，而最近没有被访问的页框被从链表尾部回收。但是这种方案在工程上并不现实— 如果要在每一次页框的访问时来回收，频繁在 LRU 链表中调整位置，会引入巨大的延迟开销。因此，内核采用” 采样” 的方式—内核不再每访问一次页就操作链表，而是回收时才批量检查页的访问标志，检查最近页帧是否被访问过。

每当某个基页 (base page) 被访问时，硬件会自动置位 PTE 页表中的 _PAGE_ACCESSED 标志位。该标志位也被称作” 年轻 (young) 标志”，内核用 pte_young() 函数检测该标志位。该标志位具有” 粘性 (sticky)”——即它由硬件置位，且会保持置位状态，直到内核主动将其清除。因此，在执行内存回收时，我们可通过检查该标志位，判断自上次检查以来该页帧是否被访问过 (即，是否处于 active 状态)，并在检查后将其清零，为下一次检测做准备。这个检测过程中会从目标页帧出发，反向映射 (rmap) 遍历找到所有映射该页的页表项 (PTE) 进行检测。

若仅维护单个 LRU 链表，会面临一个严重问题：由于不会在页帧每次被访问时调整其在 LRU 链表中的位置，那么链表尾部的页帧是频繁访问的 active 页，还是极少访问的 inactive 页，必须逐一检查才能区分。这会导致大量时间被耗费在检查无法回收的页上显著拖慢系统运行速度。解决这一问题的方法就是维护彼此独立的两个 LRU 链表: 活跃 (active) 和欠活跃 (inactive)LRU 链表。同时仅尝试从 inactive 链表回收内存页帧，并将从中检测到的” 近期被访问过” 的页帧提升(promote) 到 active 活跃链表。这一设计能大幅提高效率。

此外，我们可自主选择扫描 active 链表的时机，以决定哪些页帧应降级(deactive) 至 inactive 链表：

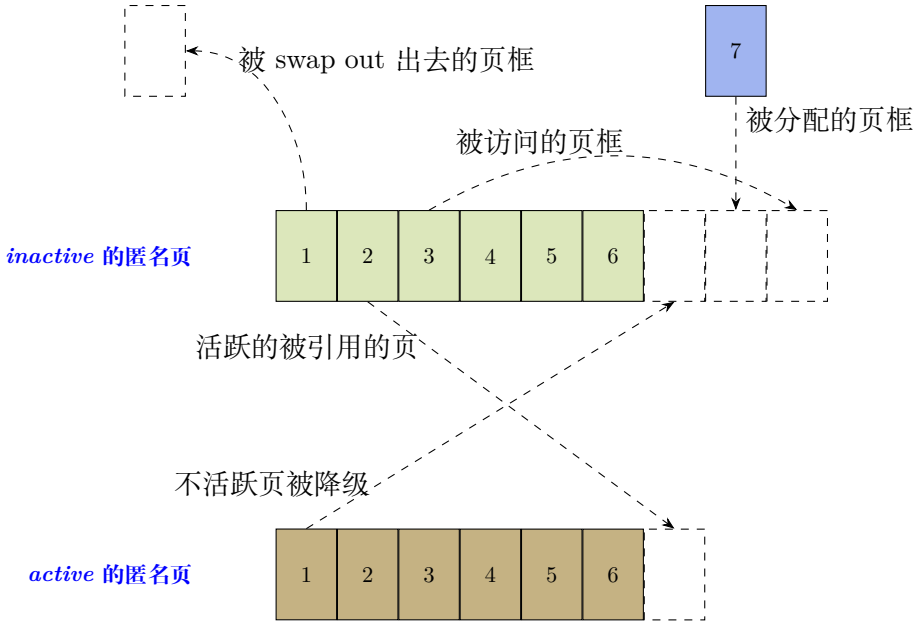
- 内存压力极大时，只聚焦于 inactive 链表，几乎能确保释放出内存；
- 其他情况下，也可检查 active 链表以期降级页帧到 inactive 链表，确保两个链表的页数量保持

合理平衡。

回收对于文件页和匿名页有着完全不同的含义：

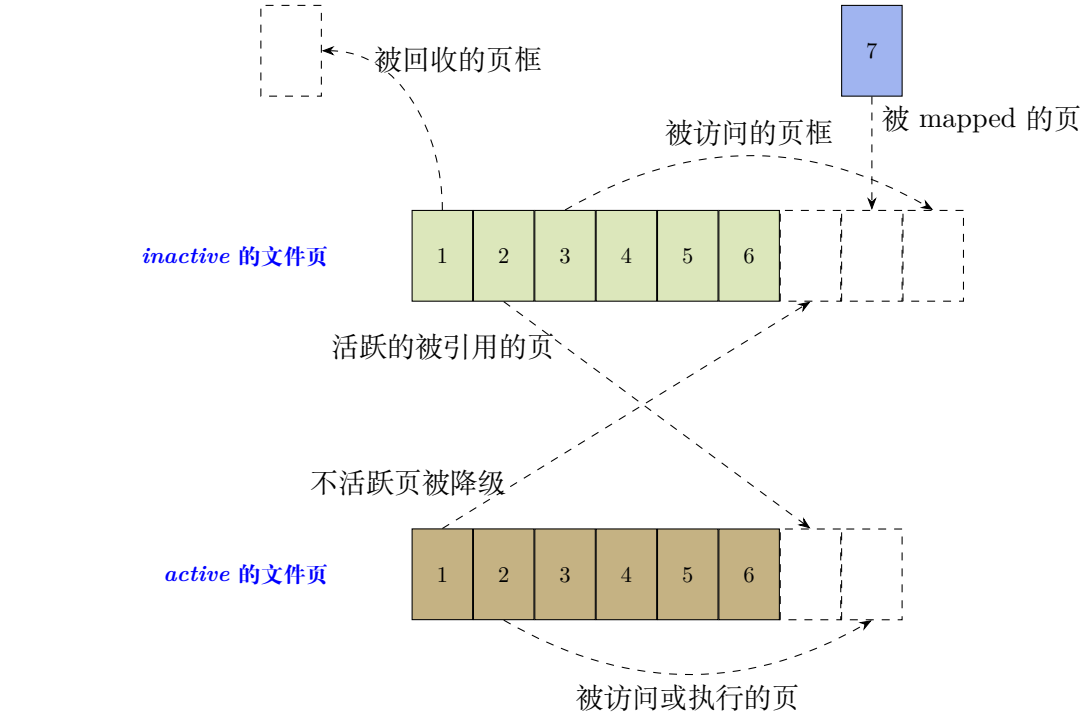
- 文件页若处于干净状态 (即 cache 中的文件页无修改不需要回写至磁盘)，可直接从 cache 中丢弃 (若后续需要，重新读取即可)；
- 匿名页则需交换至交换分区 (swap out)— 这是开销很大的操作。

基于以上原因，因此内核为文件映射页和匿名页分别维护了独立的 LRU 链表：这样就能精准控制两类页的回收策略，在两种场景下可批量执行对应的操作 (文件页直接丢弃、匿名页交换)。完全不能被回收的页面 (如通过 mlock() 锁定的内存) 不会被维护在上述任何链表中。





文件页的 LRU 链表的操作与匿名页相比有一些差异，如下：



- 3031-3032 若已回收足够页面, 则终止循环。
- 3034-3035 若回收后内存已经足够进行内存整理，则退出循环。
- 3041-3042 若优先级低于 10，表示回收压力大。将 $->may_writepage$ 置 1，忽略系统配置强制回写脏页。
- 3043 如果没有在回收中得到足够页面，则扫描优先级 (值) 递减。即，增加扫描回收力度，继续扫描。
- 3046-3047 遍历 zonelist 上的有效 zone。这些 zone 要在有效 NUMA 节点上，并且索引 ZONE_XXX 要小于给定的索引值。
- 3048-3049 通过比较最近两次处理的 node 是否同一节点来避免重复处理同一 NUMA 节点。

  这个循环代码块里的操作粒度都是针对 NUMA 节点的。所以这里避免对同一 node 重复冗余操作。
- 3052 分别获取到当前时刻止目标 lruvec 上 (匿名页和文件页) 页面重载 (refault) 次数。也就是，发生二次缺页并重新激活到活跃链表的页的数目。
- 3054-3059 若是局部内存控制组触发的回收，则清除目标 cgroup 目标控制组 lru 向量组的 lru 拥塞标志。允许快速回收该 cgroup 的内存。

  cgroup 触发的回收可以无视拥塞，允许以正常或激进方式扫描本组 lruvec。因为 cgroup 到达限额时，没有退路，必须尽力回收，否则可能触发 Oom。
- 3063 结束延迟统计 (见 3020 行的解释)。
- 3065-3066 若成功回收页面, 返回实际回收数量。

- 3069-3070 若没有成功回收，但是可以进行内存整理操作，则通知调用者尝试内存整理。
- 3081-3086 若此前没有 deactive 活跃页面，此处重置优先级，强制降级 (去激活,deactive) 重试回收。

  系统依据内存节点的内存构成情况来确定活跃页和欠活跃页的比例。但是受限于传入的最高 zone index 和 cgroup 的 memory.low 的设置，可能使大量内存被跳过回收。一开始系统会默认估算的比例值有效，不会对 cgroup 内存和子树进行代价昂贵的计算统计，只在回收效果不佳时才进行强制去激活 (deactive) 方式重试回收。也就是从活跃链表降级页到欠活跃链表。
- 3089-3095 若之前因 cgroup 内存不足跳过回收, 此处进行标记，突破 cgroup 软限制进行重试。

  这里的软限制是 cgroup 的 memory.low 来确定的，它是 cgroup v2 的内存软性保护的阈值，第一轮尽量放过，第二轮才可以回收。
- 3097 彻底失败。

5.1.1 vmpressure_prio()

vmpressure_prio() 通过回收扫描深度来计算**内存压力等级**。是回收扫描深度 (回收优先级) 驱动压力等级通知的入口，即触发压力通知的决策点。vmscan() 在回收扫描深度发生变化后，都应该在回收执行的路径调用该函数。

do_try_to_free_pages() > *vmpressure_prio()*

```
323 void vmpressure_prio(gfp_t gfp, struct mem_cgroup *memcg, int prio)
324 {
325     /*
326      * We only use prio for accounting critical level. For more info
327      * see comment for vmpressure_level_critical_prio variable above.
328      */
329     if (prio > vmpressure_level_critical_prio)
330         return;
331 
332     /*
333      * OK, the prio is below the threshold, updating vmpressure
334      * information before shrinker dives into long shrinking of long
335      * range vmscan. Passing scanned = vmpressure_win, reclaimed = 0
336      * to the vmpressure() basically means that we signal 'critical'
337      * level.
338      */
339     vmpressure(gfp, memcg, true, vmpressure_win, 0);
340 }
```

- 329-330 未达到高压力 (值越大压力越小)，直接返回。

  vmscan 的逻辑定义了以下几个内存压力紧急程度：

  - *prio == DEF_PRIORITY*: 这是初始扫描优先级，默认值是 12，此时压力处于低水平。
  - *prio <= DEF_PRIORITY - 2*: 这是内存压力的中间状态, 介于” 正常” 和” 紧急” 之间。kswapd 开始进入压力过载状态。
  - *prio == vmpressure_level_critical_prio*: 高压力状态，此时扫描深度达 lru 大小的约 10% ~ 12.5%(vmpressure_level_critical_prio 实际为 $\log_2 10$), 系统会触发紧急通知。紧急压力状态是指系统内存处于极度紧张的状态，如不及时回收内存可能导致系统无法正常分配内存。
  - *prio == 0*: 内存近乎耗尽 (OOM), 此时系统将会扫描 lru 中的每一页。

prio 值的范围在 [0,12] 之间，当前 vmpressure_level_critical_prio(即前述的高压力等级) 的值是一个经验值。

- 339 在正常情况下 vmpressure() 会积累扫描和回收数据，直到达到 vmpressure_win 阈值才计算压力。但在内存极度紧张时，这种延迟可能导致系统无法及时响应。通过主动显示传递 vmpressure_win 和 0，来立即告知系统” 内存接近无法回收”，需要触发高压力处理流程。

5.1.2 内存压力事件上报

do_try_to_free_pages() > *vmpressure_prio()* > *vmpressure()*

```
240 void vmpressure(gfp_t gfp, struct mem_cgroup *memcg, bool tree,
241         unsigned long scanned, unsigned long reclaimed)
242 {
243     struct vmpressure *vmpr = memcg_to_vmpressure(memcg);
244 
245     /*
246      * Here we only want to account pressure that userland is able to
247      * help us with. For example, suppose that DMA zone is under
248      * pressure; if we notify userland about that kind of pressure,
249      * then it will be mostly a waste as it will trigger unnecessary
250      * freeing of memory by userland (since userland is more likely to
251      * have HIGHMEM/MOVABLE pages instead of the DMA fallback). That
252      * is why we include only movable, highmem and FS/IO pages.
253      * Indirect reclaim (kswapd) sets sc->gfp_mask to GFP_KERNEL, so
254      * we account it too.
255      */
256     if (!(gfp & (__GFP_HIGHMEM | __GFP_MOVABLE | __GFP_IO | __GFP_FS)))
257         return;
258 
259     /*
260      * If we got here with no pages scanned, then that is an indicator
261      * that reclaimer was unable to find any shrinkable LRUs at the
262      * current scanning depth. But it does not mean that we should
263      * report the critical pressure, yet. If the scanning priority
264      * (scanning depth) goes too high (deep), we will be notified
265      * through vmpressure_prio(). But so far, keep calm.
266      */
267     if (!scanned)
268         return;
269 
270     if (tree) {
271         spin_lock(&vmpr->sr_lock);
272         scanned = vmpr->tree_scanned += scanned;
273         vmpr->tree_reclaimed += reclaimed;
274         spin_unlock(&vmpr->sr_lock);
275 
276         if (scanned < vmpressure_win)
277             return;
278         schedule_work(&vmpr->work);
279     } else {
280         enum vmpressure_levels level;
281 
282         /* For now, no users for root-level efficiency */
283         if (!memcg || mem_cgroup_is_root(memcg))
284             return;
285 
286         spin_lock(&vmpr->sr_lock);
```

```
287         scanned = vmpr->scanned += scanned;
288         reclaimed = vmpr->reclaimed += reclaimed;
289         if (scanned < vmpressure_win) {
290             spin_unlock(&vmpr->sr_lock);
291             return;
292         }
293         vmpr->scanned = vmpr->reclaimed = 0;
294         spin_unlock(&vmpr->sr_lock);
295 
296         level = vmpressure_calc_level(scanned, reclaimed);
297 
298         if (level > VMPRESSURE_LOW) {
299             /*
300              * Let the socket buffer allocator know that
301              * we are having trouble reclaiming LRU pages.
302              *
303              * For hysteresis keep the pressure state
304              * asserted for a second in which subsequent
305              * pressure events can occur.
306              */
307             memcg->socket_pressure = jiffies + HZ;
308         }
309     }
310 }
```

该函数通过已扫描页数量/目标回收页面数量的比例来统计并更新压力状态。

- 256-257 若 gfp 标志不包含可移动、高端内存、IO 或文件系统标志，则跳过。

  这里只关注用户空间能响应和释放的内存类型 (如可移动内存、高端内存), 忽略 DMA zone 区域的压力，因为用户空间无法有效释放 DMA 内存。向用户空间上报该压力只会造成无谓的开销。

- 267-268 如果 scanned=0, 说明回收器在当前扫描深度未找到可回收页面。但此时并不意味着要立即报告紧急压力状态，在扫描深度足够深时，将通过 vmpressure_prio() 进行压力通知。

  在正常情况下，vmpressure() 会累积扫描和回收数据，直到达到通知阈值。

- 270 针对整个 cgroup 树进行内存压力统计。

- 272 累加扫描页数到全局统计量。

- 273 累加回收页数到全局统计量。

- 276-277 检查扫描页数是否低于阈值。当累计扫描量达到 vmpressure_win(压力检测窗口) 时，才触发后续处理，减少性能开销。

- 278 利用工作队列在后台计算 cgroup 树的压力等级，并通过 event 事件通知用户空间，以便用户空间根据系统整体的不同压力等级动态采取相应决策。该工作队列处理内存压力事件的实际处理函数是 vmpressure_work_fn()。

  NOTE

  主线 *linux* 没有守护进程接收内存压力 *(vmpressure)* 事件。
  android 有专门的守护进程 *lmkd(low memory killer daemon)*。*lmkd* 监听内存压力事件，然后按照优先级 *kill* 掉 *app* 进程。同时通知 *framwork* 侧进行处理，让应用释放缓存。
  android 10+ 默认优先使用 *PSI* 代替 *vmpressure*。

- 283 以下代码段处理非 tree 模式，也就是仅对单个并且非 root 的 memcg 进行处理。根控制组包含所有进程，其压力事件无隔离意义，跳过。

- 287-288 累加扫描页数，累加回收页数。
- 289-292 检查是否到达阈值，不足则返回。
- 293 重置统计量。仅在扫描达标时继续处理，并且重置统计量。
- 298-308 仅在中高压力等级时进行如下处理.
  - 307 当压力等级超过 VMPRESSURE_LOW，也就是处于中高压力等级时，设置 memcg->socket_pressure 标志位，来通知网络协议栈 buffer 分配器维持当前压力状态，在 1 秒内不采取相应决策。采用滞后对策 (处理) 是为了避免内存压力状态频繁变化。

    此行记录截止时间戳, 意味着自此处开始计时的 1s 之后的 jiffies 计数。在该时间戳到达前网络子系统不进行相应的内存决策处理。

do_try_to_free_pages() > *vmpressure_prio()* > *vmpressure()* > *vmpressure_calc_level()*

```
121 static enum vmpressure_levels vmpressure_calc_level(unsigned long scanned,
122                                unsigned long reclaimed)
123 {
124     unsigned long scale = scanned + reclaimed;
125     unsigned long pressure = 0;
126 
127     /*
128      * reclaimed can be greater than scanned for things such as reclaimed
129      * slab pages. shrink_node() just adds reclaimed pages without a
130      * related increment to scanned pages.
131      */
132     if (reclaimed >= scanned)
133         goto out;
134     /*
135      * We calculate the ratio (in percents) of how many pages were
136      * scanned vs. reclaimed in a given time frame (window). Note that
137      * time is in VM reclaimer's "ticks", i.e. number of pages
138      * scanned. This makes it possible to set desired reaction time
139      * and serves as a ratelimit.
140      */
141     pressure = scale - (reclaimed * scale / scanned);
142     pressure = pressure * 100 / scale;
143 
144 out:
145     pr_debug("%s: %3lu  (s: %lu  r: %lu)\n", __func__, pressure,
146          scanned, reclaimed);
147 
148     return vmpressure_level(pressure);
149 }
```

该函数会计算虚拟内存压力等级，评估内存回收的压力程度。

- 124-125 作为计算基准的 *scale* 是扫描的页面数 (也就是尝试回收的页面数) 与成功回收的页面数量之和。
- 132-133 成功回收的页面数可能超过扫描数，这里处理这种特殊情况。

  这个场景通常出现在调用了 ***shrink_slab()*** 回收 slab 的场景，在这种场景下 shrink_node() 会累加已回收页的计数，但不会增加扫描页面的计数。它和常规 LRU 链表页回收有区别。

- 141-142 计算未成功回收的页面。计算公式如下:

$$
\begin{aligned}
pressure &= \left(scale - \frac{reclaimed \times scale}{scanned}\right) \times \frac{100}{scale} \\
&= \left(\frac{scale \times scanned - reclaimed \times scale}{scanned}\right) \times \frac{1}{scale} \times 100 \\
&= \frac{(scanned - reclaimed) \times scale}{scanned} \times \frac{1}{scale} \times 100 \\
&= \frac{(scanned - reclaimed)}{scanned} \times 100 \\
&= \left(1 - \frac{reclaim}{scanned}\right) \times 100
\end{aligned}
$$

  也就是:

$$
\text{压力值} = \left(1 - \frac{\text{已回收页数}}{\text{已扫描页数}}\right) \cdot 100
$$

  这里拆成两行写是因为代码要规避负数、整数相除的精度等问题。

  该值反映未成功回收内存的比例。

  把未成功回收的页转换成相对计算基准的页面百分占比。内存压力 = 扫描中未能回收的页面比例。比例越高, 回收越困难, 系统内存压力越大。

- 148 系统通过前面得到的 $\frac{\text{已回收页面数}}{\text{已扫描页面数}}$ 比值来评估内存压力。vmpressure_level() 见下面。

  vmpressure_level() 函数用来返回压力等级。

do_try_to_free_pages() ··· vmpressure() vmpressure_calc_level() vmpressure_level()

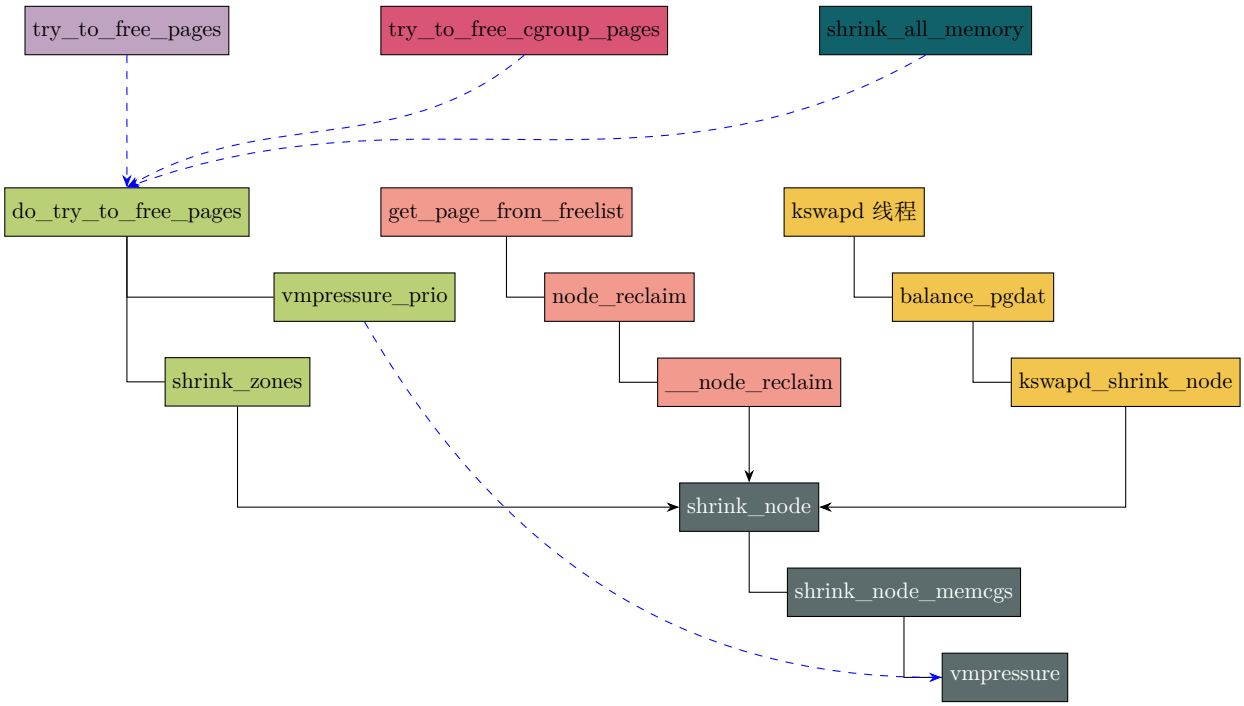
```
112 static enum vmpressure_levels vmpressure_level(unsigned long pressure)
113 {
114     if (pressure >= vmpressure_level_critical)
115         return VMPRESSURE_CRITICAL;
116     else if (pressure >= vmpressure_level_med)
117         return VMPRESSURE_MEDIUM;
118     return VMPRESSURE_LOW;
119 }
```

  该函数将压力百分比 (传入参数)映射到三级压力等级中。

- vmpressure_level_critical,vmpressure_level_med 这两个宏值分别为 95,60。
- 114-115 压力 >= 95% 时，表示紧急状态，接近 OOM，需立即采取措施.
- 116-117 60% <= 压力 < 95% 时，回收效率下降，需主动释放缓存来调整。
- 118 压力 < 60% 时，压力正常，表示回收成功率高，无需干预。

此处 vmpressure() 的压力等级，反应了系统或 cgroup 的内存全局状态；而 vmpressure_prio() 里的 prio 优先级反应的是压力回收紧急程度，针对的是单次回收操作。

vmpressure() 监测系统内存压力并调用工作队列向用户空间发送事件通知，用于内存资源管理的优化。其基于内存控制组监控内存分配成本，通过 event 事件触发用户空间对不同级别的压力响应。通过以下几条路径触发事件的上报:



如上图所示，这些函数会在执行中调用 vmpressure() 函数进行内存压力事件上报。

- ***try_to_free_pages( )*** 会在直接内存回收的过程中上报内存压力事件。
- ***try_to_free_cgroup_pages( )*** 在回收 cgroup 内存的过程中上报。
- ***get_page_from_freelist( )*** 分配物理页时，水位不满足要求，触发内存回收时上报.
- ***kswapd 线程*** kswapd 在回收内存的过程中上报.
- ***shrink_all_memory( )*** 会在系统休眠将内存转储到后备存储时进行事件上报。在启用休眠功能 CONFIG_HIBERNATION 后，该函数会负责内存回收。按以下顺序回收页面:

$$inactive > active > active\ referenced > active\ mapped$$

vmpressure_work_fn() 函数是 vmpressure() 的工作队列回调函数。。vmpressure() 函数里调用 schedule_work() 把该工作项丢进工作队列，延后到进程上下文做事件上报。

do_try_to_free_pages() > *vmpressure_prio()* > *vmpressure()* > *vmpressure_work_fn()*

```
181 static void vmpressure_work_fn(struct work_struct *work)
182 {
183     struct vmpressure *vmpr = work_to_vmpressure(work);
184     unsigned long scanned;
185     unsigned long reclaimed;
186     enum vmpressure_levels level;
187     bool ancestor = false;
188     bool signalled = false;
189
190     spin_lock(&vmpr->sr_lock);
```

```
191     /*
192      * Several contexts might be calling vmpressure(), so it is
193      * possible that the work was rescheduled again before the old
194      * work context cleared the counters. In that case we will run
195      * just after the old work returns, but then scanned might be zero
196      * here. No need for any locks here since we don't care if
197      * vmpr->reclaimed is in sync.
198      */
199     scanned = vmpr->tree_scanned;
200     if (!scanned) {
201         spin_unlock(&vmpr->sr_lock);
202         return;
203     }
204 
205     reclaimed = vmpr->tree_reclaimed;
206     vmpr->tree_scanned = 0;
207     vmpr->tree_reclaimed = 0;
208     spin_unlock(&vmpr->sr_lock);
209 
210     level = vmpressure_calc_level(scanned, reclaimed);
211 
212     do {
213         if (vmpressure_event(vmpr, level, ancestor, signalled))
214             signalled = true;
215         ancestor = true;
216     } while ((vmpr = vmpressure_parent(vmpr)));
217 }
```

- 183 从通用 work 指针找回 vmpressure 对象。
- 199-203 将 cgroup 树维度的扫描页数读到局部变量 scanned。如果扫描页数为 0 就直接 return。
- 205-207 把树维度回收页数读到局部变量 reclaimed。将计数器清零，本轮统计完成，下一次 vmpressure() 重新开始累加。
- 210 根据扫描页数、回收页数算出内存压力等级。
- 212-216 从发生内存回收的子 cgroup 开始，逐层向上，每一级父 cgroup 都收到相同的压力等级事件。子 cgroup 内存压力会通知到所有上层 cgroup。

  vmpressure_event() 负责检查该 cgroup 是否注册监听对应 level 事件，如果满足阈值，向用户态发送事件。返回 true 代表本次确实发出了信号。signalled 用来告诉上层祖先: 下层已经发过信号，可以做相应的过滤逻辑。

5.1.3 shrink_zones()

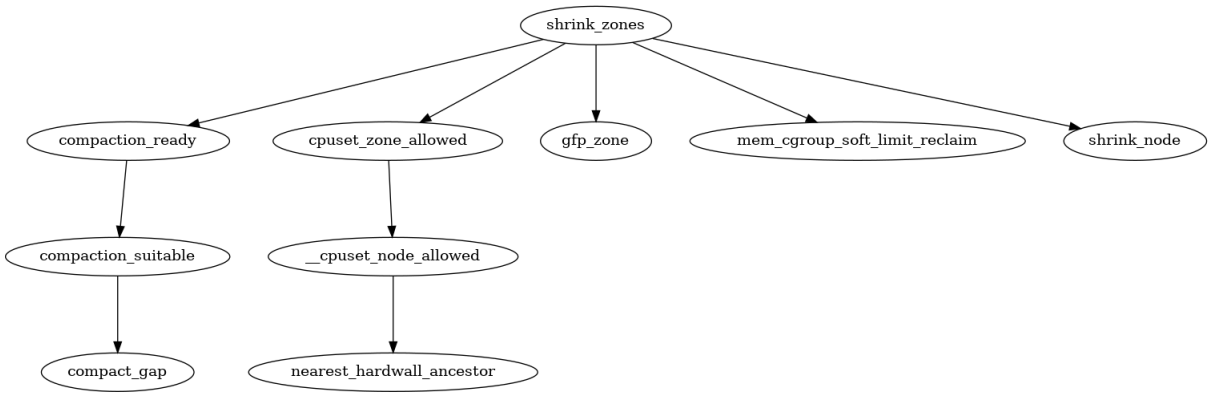


图 5–2: 调用图：shrink_zones()

shrink_zones() 是在直接内存回收路径上被 do_try_to_free_pages() 函数调用的。内核仅尝试从能够满足调用者分配请求的内存区域回收页面。如果某个内存区域充满了被锁定（pinned）的页面，则仅对其进行轻量级扫描，随后放弃对该区域的回收尝试。

do_try_to_free_pages() *shrink_zones()*

```
2899 static void shrink_zones(struct zonelist *zonelist, struct scan_control *sc)
2900 {
2901     struct zoneref *z;
2902     struct zone *zone;
2903     unsigned long nr_soft_reclaimed;
2904     unsigned long nr_soft_scanned;
2905     gfp_t orig_mask;
2906     pg_data_t *last_pgdat = NULL;
2907
2908     /*
2909      * If the number of buffer_heads in the machine exceeds the maximum
2910      * allowed level, force direct reclaim to scan the highmem zone as
2911      * highmem pages could be pinning lowmem pages storing buffer_heads
2912      */
2913     orig_mask = sc->gfp_mask;
2914     if (buffer_heads_over_limit) {
2915         sc->gfp_mask |= __GFP_HIGHMEM;
2916         sc->reclaim_idx = gfp_zone(sc->gfp_mask);
2917     }
2918
2919     for_each_zone_zonelist_nodemask(zone, z, zonelist,
2920                     sc->reclaim_idx, sc->nodemask) {
2921         /*
2922          * Take care memory controller reclaiming has small influence
2923          * to global LRU.
2924          */
2925         if (!cgroup_reclaim(sc)) {
2926             if (!cpuset_zone_allowed(zone,
2927                         GFP_KERNEL | __GFP_HARDWALL))
2928                 continue;
2929
2930             /*
2931              * If we already have plenty of memory free for
2932              * compaction in this zone, don't free any more.
2933              * Even though compaction is invoked for any
2934              * non-zero order, only frequent costly order
2935              * reclamation is disruptive enough to become a
2936              * noticeable problem, like transparent huge
```

```
2937                 * page allocations.
2938                 */
2939                if (IS_ENABLED(CONFIG_COMPACTION) &&
2940                    sc->order > PAGE_ALLOC_COSTLY_ORDER &&
2941                    compaction_ready(zone, sc)) {
2942                    sc->compaction_ready = true;
2943                    continue;
2944                }
2945
2946 ...... < to be continued > ......
```

- 2914 判断 buffer_head 的数量是否超过结点允许的最大值 buffer_heads_over_limit。

  buffer_head 结构体和文件页有关。当磁盘上一个数据块被调用到内存时，首先要储存在一个内存缓冲区，每个缓冲区与一个磁盘块对应，所以每个缓冲区独有一个对应的描述符，该描述符就是 buffer_head 结构, 在 buffer_head 中保存了对应的磁盘号。

  页缓存 (pagecache) 屏蔽了对底层磁盘或外存储的文件操作的细节。但是，最终读写文件数据还是要落到实际存储介质上，因此必须执行 I/O 操作才能从磁盘读数据或写入数据到磁盘。这些 I/O 操作的客体就是**块设备**，块设备就是以**块**为粒度进行工作的存储设备。

  块的最小尺寸是 512 bytes，最大尺寸是基页 ($order = 0$) 大小。可以把块设备看成一个数组，索引就是块编号。将页缓存 (pagecache) 中的页映射到块设备的块号的核心结构体是 struct ***buffer_head***。或者也可以说是将磁盘块 (512 bytes, 小于基页大小) 映射到页缓存内部对应的偏移位置 (offset)。

  buffer_head 结构体对象由 alloc_buffer_head() 函数从 slub 缓存分配。并且属于同一个页的 buffer_head 总是挂在 page->private 成员上。

  由上面的描述易知，当 buffer_head 的数量超过允许的阈值时，则强制回收逻辑扫描 ZONE_HIGHMEM 高端内存区。因为高端内存区的页缓存 (pagecache) 可能会固定住存放 buffer_head 的低端内存页。因为高端内存区的页缓存不释放，那么它对应的存放在低端内存的 slub obj 对象 (也就是映射结构体 buffer_head) 就不会释放，就好像把低端内存固定住不释放。所以尝试扫描回收高端内存，目的却是为了扫描回收低端内存。

  NOTE

  64 位系统总线宽度很大，可访问地址空间达到几百 T，所以没有高端内存 (ZONE_HIGHMEM) 的概念。于是这段逻辑也不起效。

- 2915-2916 在 buffer_head 数量超出限制的情况下，会从高端内存区域 ZONE_HIGHMEM 扫描回收。并根据分配标识获取对应回收 zone 的最高索引编号。

  从前面讨论可知，当一个磁盘块的数据被读入内存时，内核会在低端内存中分配一个 buffer_head 结构体，用于描述这个磁盘块的缓存信息（如物理块号、设备号、脏位等）。buffer_head 结构体是内核数据结构，需要被内核直接访问，所以存储在低端内存中。但用于存储对应该磁盘块的实际数据内容的” 块”，可能存储在从高端内存分配的一个页面上。

  如果数据页面位于高端内存且被标记为 “已使用” (例如，被用户进程映射或被 I/O 操作锁定)，则该页面不能被回收。同时，由于 buffer_head 仍在引用高端内存中的该数据页面，因此内核也不能回收存储 buffer_head 的低端内存页面。结果会间接阻止低端内存页面的回收。当高端内存中存在大量可回收但未被扫描的页面（如文件系统缓存），而低端内存中大量 buffer_head 又因映射高端内

存页面而无法释放, 最终会导致系统整体内存利用率下降。换言之，此时高端内存页” 锁定 (pin)” 住了存储 buffer_head 的低端内存页。

通过设置 ___GFP_HIGHMEM 标志，会强制内核在内存回收时主动扫描高端内存区域，释放不再使用的页面，从而间接解除对 buffer_head 的引用，最终释放低端内存。

- 2919 对指定 node 上的，索引值 (ZONE_XXX)≤ 最高可回收 zone 编号的,zonelist 链表上可用的 zone 进行迭代处理。
- 2925 如果 target_mem_cgroup 为 NULL，则是全局内存回收而不是针对某个 cgroup 进行此项操作。2925-2968 这个代码块对全局内存回收进行处理。
- 2926-2927 判断是否可以从目标 zone 分配内存。如果不能则略过，继续下一次迭代。
- 2939-2944 确认是否要进行内存整理操作还是直接往下执行回收操作。

  需要同时满足这三个条件才会置可以内存整理 ready 的标识:

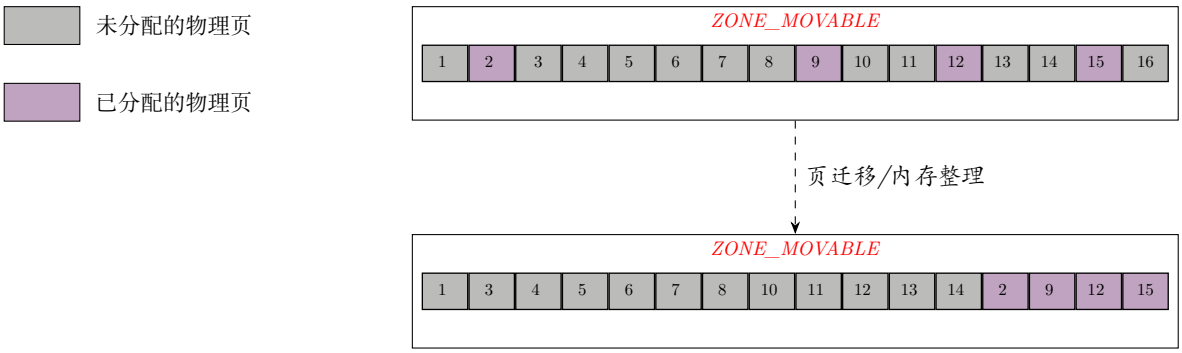
  - 内核使能了内存整理的功能。
  - 分配阶分配成本较高 ($order > 3$)。
  - 当前目标 zone 可以进行内存整理。

  PAGE_ALLOC_COSTLY_ORDER 是指内存分配成本较高的分配阶。简单来说，就是分配的内存 order 大于 PAGE_ALLOC_COSTLY_ORDER 时，会占用较多的 cpu 资源和时间，成本比较高。

  IS_ENABLED(CONFIG_COMPACTION) 判断是否使能了内存整理的功能。

随着系统的长时间运行通常会伴随着不同大小的物理内存页的分配和释放，这种不规则的分配释放，随着系统的长时间运行就会导致内存碎片，内存碎片会使得系统在明明有足够内存的情况下，依然无法为进程分配合适的内存。

内存整理时会扫描内存区域 zone 里的页面，把已分配的页记录下来，然后把所有已分配的页移动到 zone 的一端，这样就会把一个已经充满碎片的 zone 整理成一段完全未分配的区间和一段已经分配的区间，从而腾出大块连续的物理页面供内核分配。如图:



compaction_ready() 根据当前系统内存状况判断目标 zone 是否需要内存整理。

如果当前 zone 已经有足够的空闲内存用于内存整理，就不要再释放 (回收) 内存了。因为频繁的代价高昂阶的回收才会造成足够大的消耗并成为一个大问题。

- 2961-2966 前面已经提到 mem_cgroup_soft_limit_reclaim()（软限制回收）的目标是用于处理整个目标 node 节点的内存压力。同时依据内存控制组（memcg）的软限制，定向回收特定 memcg 中超出软限制的内存。软限制回收的逻辑需要当前处于全局回收 (在某个 node 节点上而言) 场景，从全局 LRU 列表中回收页面。仅在全局回收（如直接回收、kswapd 回收）时触发。

  内核维护了一个全局的 softlimit 树，树的顶端是内存 softlimit 控制层级结构的根节点 soft_limit_tree(类型是 struct mem_cgroup_tree)。根节点的成员数组->rb_tree_per_node 中的各元素又分别指向各内存 node 节点的 softlimit tree(具体为一棵红黑树)。树上的每个节点都包含某个 Memcg 在这个 node 上所用内存的 LRU 链表。只有在目标 node 上内存使用超出 softlimit 限制的 memcg，它在该 node 上的 LRU 链表才会存在于这棵红黑树上。且这棵红黑树的排序依据就是该 LRU 链表所属的 Memcg 超出 softlimit 限制的大小的差值。对一个 node 进行 softlimit 回收，即从 softlimit tree 找到一个或多个 Memcg 在目标 node 上的 LRU 链表，对这些 LRU 链表进行回收。

当需要先尝试内存回收再决定是否执行内存整理时，返回 false。

5.1.3.1 compaction_ready()

do_try_to_free_pages() > *shrink_zones()* > *compaction_ready()*

```
2864 static inline bool compaction_ready(struct zone *zone, struct scan_control *sc)
2865 {
2866     unsigned long watermark;
2867     enum compact_result suitable;
2868
2869     suitable = compaction_suitable(zone, sc->order, 0, sc->reclaim_idx);
2870     if (suitable == COMPACT_SUCCESS)
2871         /* Allocation should succeed already. Don't reclaim. */
2872         return true;
2873     if (suitable == COMPACT_SKIPPED)
2874         /* Compaction cannot yet proceed. Do reclaim. */
2875         return false;
2876
2877     /*
2878      * Compaction is already possible, but it takes time to run and there
2879      * are potentially other callers using the pages just freed. So proceed
2880      * with reclaim to make a buffer of free pages available to give
2881      * compaction a reasonable chance of completing and allocating the page.
2882      * Note that we won't actually reclaim the whole buffer in one attempt
2883      * as the target watermark in should_continue_reclaim() is lower. But if
2884      * we are already above the high+gap watermark, don't reclaim at all.
2885      */
2886     watermark = high_wmark_pages(zone) + compact_gap(sc->order);
2887
2888     return zone_watermark_ok_safe(zone, 0, watermark, sc->reclaim_idx);
2889 }
```

- 2869 对目标 zone 区域是否适合执行碎片整理的操作进行判断。(该函数详解见后面)。
- 2870-2871 compaction_suitable() 返回 COMPACT_SUCCESS，表示已经满足分配需求，无需回收。
- 2873-2875 结果为 COMPACT_SKIPPED，表示需要先执行内存回收，才能再尝试内存整理。

  若为 COMPACT_CONTINUE, 则表示需要立即进行内存整理。

- 2886 获取当前 zone 的高水位线页数目。然后根据请求的页阶数动态计算额外 gap(间隙) 页数量。

  在给定水位线基础之上，需要额外保留一定数量的空闲 $order = 0$(单页、基页) 页面。这些保留的 gap 页面在内存整理操作的过程中，用来隔离作为迁移的目标内存。以确保内存整理操作不会耗尽空闲页面。目前保留作为 gap 的页面数值，取的是请求的页阶数目的两倍 (即,$2 << order = 2 \times (1 << order) = 2 \times 2^{order} = 2^{order+1}$)。

  尽管所有用于迁移的隔离页面都是临时性的，但" 空闲页面扫描器 (free scanner)" 可能在空闲页面列表中累积最多 $1 << order$ 个页面，然后尝试拆分一个 $(order - 1)$ 阶的空闲页面。在这种情况下，仅预留 $1 << order$ 个页面的间隙可能不够，因此要求两倍数量的空闲空间会更安全。

  在这里使执行和计算复杂化并不值得，内核选择了一个简单保守的值 $(2 << order)$ 来避免" 过度设计"，简单的为大阶页请求两倍的内存间隙。这么做能提高内存碎片整理成功的概率。

  内存的水位线通常是留给内存正常分配或保底的，不应该被内存整理消耗掉。虽然内存整理本身会消耗空闲页，但是内存整理是辅助操作，重要性低于内存分配。所以，在高水位线之上的 **富余**页才可以拿来给内存整理" 折腾"，而且要不影响正常业务的分配。

- 2888 检查当前 zone 的空闲内存是否达到 2886 行计算获得的水位线。

再回到 shrink_zones()

mm/vmscan.c :: shrink_zones()

```
2946            /*
2947             * Shrink each node in the zonelist once. If the
2948             * zonelist is ordered by zone (not the default) then a
2949             * node may be shrunk multiple times but in that case
2950             * the user prefers lower zones being preserved.
2951             */
2952            if (zone->zone_pgdat == last_pgdat)
2953                continue;
2954
2955            /*
2956             * This steals pages from memory cgroups over softlimit
2957             * and returns the number of reclaimed pages and
2958             * scanned pages. This works for global memory pressure
2959             * and balancing, not for a memcg's limit.
2960             */
2961            nr_soft_scanned = 0;
2962            nr_soft_reclaimed = mem_cgroup_soft_limit_reclaim(zone->zone_pgdat,
2963                        sc->order, sc->gfp_mask,
2964                        &nr_soft_scanned);
2965            sc->nr_reclaimed += nr_soft_reclaimed;
2966            sc->nr_scanned += nr_soft_scanned;
2967            /* need some check for avoid more shrink_zone() */
2968        }
2969
2970        /* See comment about same check for global reclaim above */
2971        if (zone->zone_pgdat == last_pgdat)
2972            continue;
2973        last_pgdat = zone->zone_pgdat;
2974        shrink_node(zone->zone_pgdat, sc);
2975    }
2976
2977    /*
2978     * Restore to original mask to avoid the impact on the caller if we
2979     * promoted it to __GFP_HIGHMEM.
2980     */
```

```
2981     sc->gfp_mask = orig_mask;
2982 }
```

- 2952-2953 检查当前内存 zone 区域所属的内存节点 (node，即 zone_pgdat) 是否是上一次中处理的 node。如果是，则跳过本次处理，确保每个 node 只被处理一次。

  在回收过程中，对内存 zone 列表 (即，入参 zonelist) 中满足条件的 zone 节点所在的内存 node，只能执行一次收缩 (回收) 操作。换言之，在迭代 zone 时，对同一个内存 node 节点，不能连续执行收缩 (回收) 操作。譬如，第一次处理时，收缩 node A，那么第二次处理时收缩 node A 是不允许的。last_pgdat 用于记录上一次处理的 node，初始值为 NULL。

- 2962-2964 从当前目标节点中回收内存。函数 mem_cgroup_soft_limit_reclaim() 专门处理超过软限制的内存控制组，从超过软限制 (softlimit) 的内存控制组中” 窃取” 页面，优先回收这些组中的缓存页面，并返回成功回收的页面数。这适用于全局内存压力和平衡，而非针对特定内存控制组的限制。

  ========== 该函数待列出)

  软限制（softlimit）是一种资源管理机制，旨在平衡资源利用率与系统稳定性。用于对进程或资源组（如内存控制组 cgroup）的资源使用进行约束，但与硬限制 (hardlimit) 不同的是，软限制允许资源使用量暂时超过设定的阈值，系统不会立即终止进程或阻止分配，而是通过回收（reclaim）或节流（throttling）等策略逐步调整资源使用，使其回到限制以内。它作为资源使用的 “预警线”，触发系统自动回收资源。

  在内存管理中，当某个 cgroup 的内存使用超过软限制时，系统会优先回收该组中 inactive 的页面，而非直接拒绝内存分配请求。

- 2965-2966 更新全局计数: 累计回收和扫描的页面总数。

- 2974 对指定 numa 的内存节点执行内存回收。该函数对本节点下的所有 zone 的 LRU 链表进行扫描，回收欠回收页面。

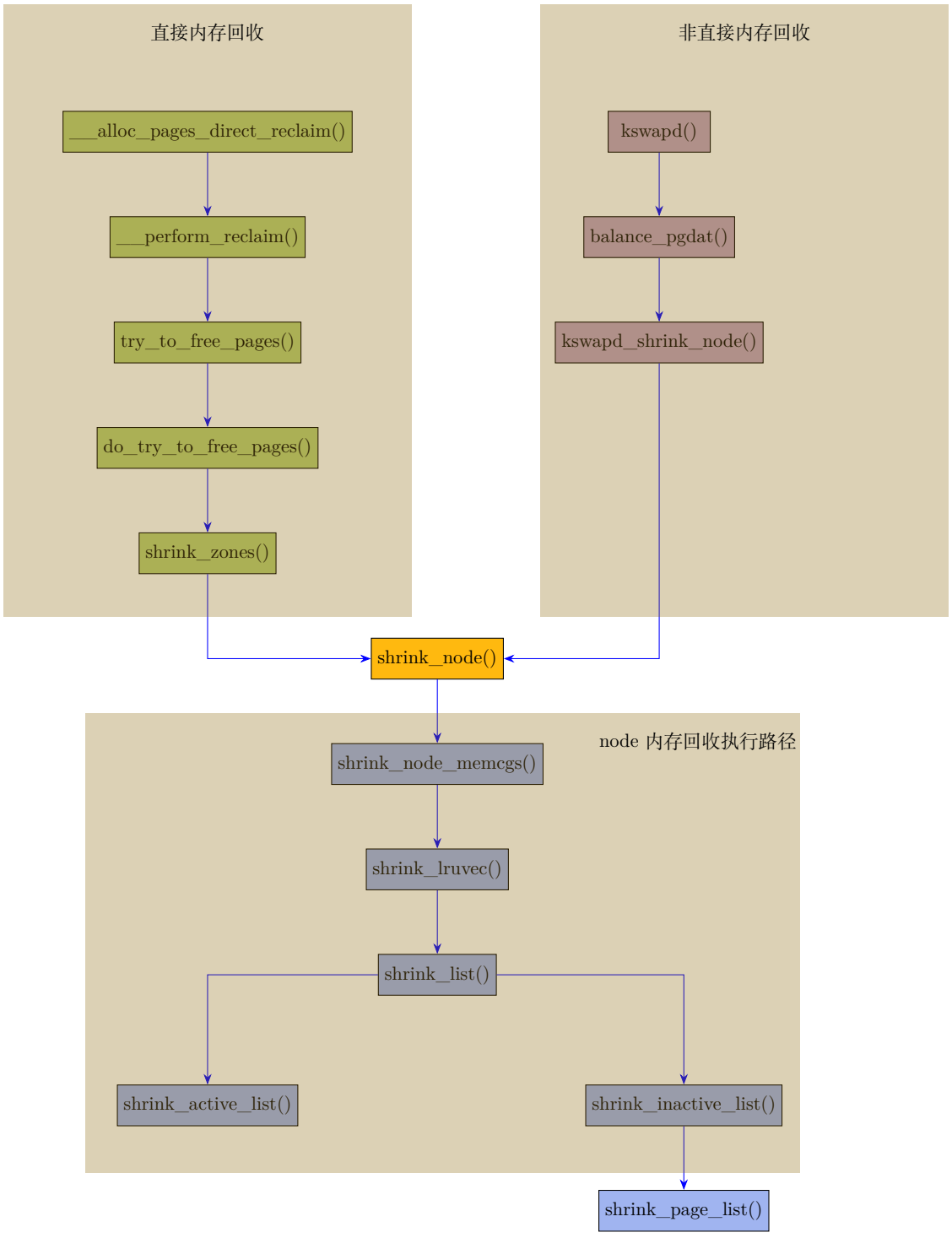
- 2981 将扫描控制结构体中的 gfp 掩码恢复。防止因为中途提升成 ___GFP_HIGHMEM 对调用者造成影响。

内存回收有两种方式:直接内存回收和非直接内存回收。

当内存分配可能涉及的所有内存区域 (zone) 都已低于最低水位线（min_watermark）时，就会触发直接内存回收（direct reclaim）。此时，发起分配的进程将阻塞，直到内存回收释放出足够的物理页，使得我们要分配的目标内存区域不再低于最低水位线为止。

非直接内存回收是作为后台线程执行的内存回收，它通过每个内存节点（node）对应的内核线程实现。当要分配内存时发现某个内存区域（zone）的水位跌破低水位线(low watermark) 时，该线程会被唤醒执行回收。执行非直接回收的内核线程名称是 kswapd，并根据它们所负责回收的内存节点编号进行命名 (例如，负责节点 node 0 的 kswapd 线程会显示为 [kswapd0])

所有内存回收调用共同的回收逻辑，都从 shrink_node() 函数开始执行。


直接内存回收
__alloc_pages_direct_reclaim()
__perform_reclaim()
try_to_free_pages()
do_try_to_free_pages()
shrink_zones()
非直接内存回收
kswapd()
balance_pgdat()
kswapd_shrink_node()
shrink_node()
node 内存回收执行路径
shrink_node_memcgs()
shrink_lruvec()
shrink_list()
shrink_active_list()
shrink_inactive_list()
shrink_page_list()

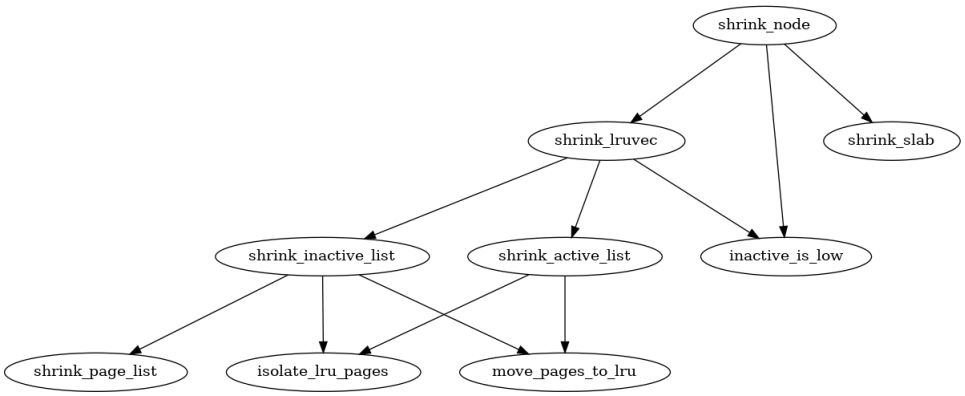


图 5–3: 调用图：shrink_node()

5.1.4 shrink_node()

下面是 shrink_node() 函数的实现。

do_try_to_free_pages() *shrink_zones()* *shrink_node()*

```
2667 static void shrink_node(pg_data_t *pgdat, struct scan_control *sc)
2668 {
2669     struct reclaim_state *reclaim_state = current->reclaim_state;
2670     unsigned long nr_reclaimed, nr_scanned;
2671     struct lruvec *target_lruvec;
2672     bool reclaimable = false;
2673     unsigned long file;
2674
2675     target_lruvec = mem_cgroup_lruvec(sc->target_mem_cgroup, pgdat);
2676
2677 again:
2678     memset(&sc->nr, 0, sizeof(sc->nr));
2679
2680     nr_reclaimed = sc->nr_reclaimed;
2681     nr_scanned = sc->nr_scanned;
2682
2683     /*
2684      * Determine the scan balance between anon and file LRUs.
2685      */
2686     spin_lock_irq(&pgdat->lru_lock);
2687     sc->anon_cost = target_lruvec->anon_cost;
2688     sc->file_cost = target_lruvec->file_cost;
2689     spin_unlock_irq(&pgdat->lru_lock);
2690
2691     /*
2692      * Target desirable inactive:active list ratios for the anon
2693      * and file LRU lists.
2694      */
2695     if (!sc->force_deactivate) {
2696         unsigned long refaults;
2697
2698         refaults = lruvec_page_state(target_lruvec,
2699                 WORKINGSET_ACTIVATE_ANON);
2700         if (refaults != target_lruvec->refaults[0] ||
2701             inactive_is_low(target_lruvec, LRU_INACTIVE_ANON))
2702             sc->may_deactivate |= DEACTIVATE_ANON;
2703         else
2704             sc->may_deactivate &= ~DEACTIVATE_ANON;
2705
2706         /*
2707          * When refaults are being observed, it means a new
2708          * workingset is being established. Deactivate to get
```

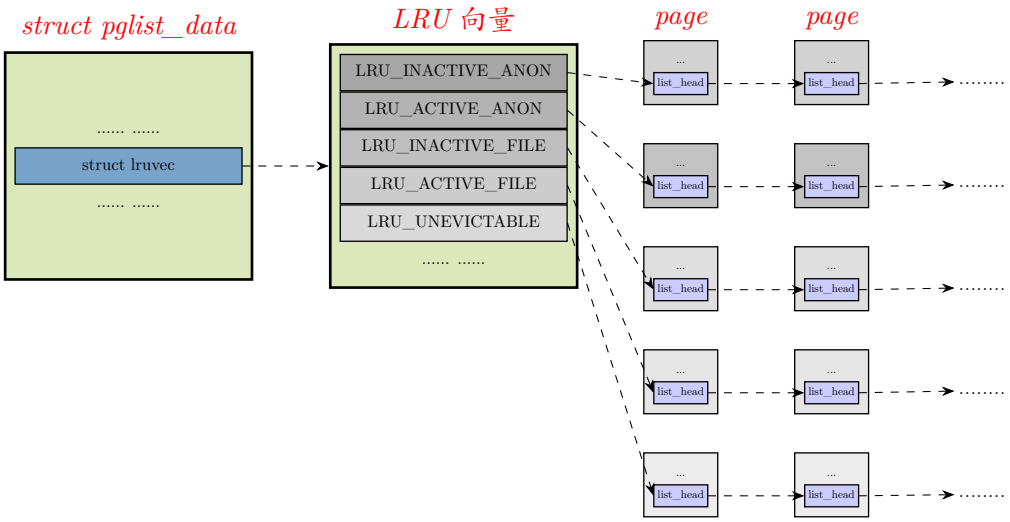
```
2709          * rid of any stale active pages quickly.
2710          */
2711         refaults = lruvec_page_state(target_lruvec,
2712                 WORKINGSET_ACTIVATE_FILE);
2713         if (refaults != target_lruvec->refaults[1] ||
2714             inactive_is_low(target_lruvec, LRU_INACTIVE_FILE))
2715             sc->may_deactivate |= DEACTIVATE_FILE;
2716         else
2717             sc->may_deactivate &= ~DEACTIVATE_FILE;
2718     } else
2719         sc->may_deactivate = DEACTIVATE_ANON | DEACTIVATE_FILE;
2720
2721     /*
2722      * If we have plenty of inactive file pages that aren't
2723      * thrashing, try to reclaim those first before touching
2724      * anonymous pages.
2725      */
2726     file = lruvec_page_state(target_lruvec, NR_INACTIVE_FILE);
2727     if (file >> sc->priority && !(sc->may_deactivate & DEACTIVATE_FILE))
2728         sc->cache_trim_mode = 1;
2729     else
2730         sc->cache_trim_mode = 0;
2731
2732         ...<to be continued>...
```

- 2675 获取目标 lruvec，内核在 LRU 向量表中选择最近最少使用的物理页回收。

  NUMA 每个 node 的 pglist_data 结构体包含一个成员 lruvec, 也称为 LRU 向量，它包含 5 个 lru 链表，分别是活跃/欠活跃匿名页，活跃/欠活跃文件页，以及不可回收链表。

  - 欠活跃匿名页链表, 即最近访问频率低的匿名页。
  - 活跃匿名页链表, 即最近访问频率高的匿名页。
  - 欠活跃文件页链表, 即最近访问频率低的文件页。
  - 活跃文件页链表, 即最近访问频率高的文件页。
  - 不可回收链表，用来链接用 mlock 锁定的、不允许回收的物理页。

  结构体之间的联系示意如下 (图中 LRU 链表只是示意，真实情况应该是双向链表)：



  除了每个 node 的 LRU 向量 (root memcg)，cgroup 也有自己管理的 LRU 向量 (memcg)。

  LRU 链表物理页及页描述符特征：

- 给页描述符结构体 struct page 的成员 flags 置 PG_lru 标志位, 表示物理页在 LRU 链表中。
- 物理页通过页描述符的成员 lru 加入 LRU 链表。
- PG_swapbacked 标志位表示是支持 swap 的物理页。
- PG_active 表示活跃的物理页。
- PG_unevictable 表示不可回收的物理页。
- PG_referenced 表示此页最近是否被访问，每次页面访问都会被置位。
- PG_mlocked 表示此页被 mlock() 锁在内存中，禁止换出和释放
- 物理页从 LRU 链表头部加入，回收算法从欠活跃 LRU 链表尾部回收。另外会从活跃 (active) 链表的尾部取物理页移动到欠活跃的 LRU 链表中。

当页面被访问时，检查该页的 PG_referenced 位，若未被置位，则置位；若发现该页的 PG_referenced 已经被置位了，则意味着该页经常被访问，这时，若该页在 inactive 链表上，则置位其 PG_active 位并移到 active 链表上去，同时清除其 PG_referenced 位的设置; 如果页面的 PG_referenced 位被置位了一段时间后，该页面没有被再次访问，那么 Linux 操作系统会清除该页面的 PG_referenced 位，因为这意味着这个页面最近这段时间都没有被访问。

PG_referenced 位同样也可以用于页面从 active 链表移动到 inactive 链表。对于某个在 active 链表上的页面来说，其 PG_active 位被置位，如果 PG_referenced 位未被置位，在给定一段时间之后，该页面如果仍没有被访问，那么该页面会被清除其 PG_active 位，挪到 inactive 链表上去。

页面根据其活跃程度会在 active 链表和 inactive 链表之间来回移动，如果要将某个页面插入到这两个链表中去，必须要通过自旋锁以保证对链表的并发访问操作不会出错。为了降低锁的竞争，Linux 提供了一种特殊的缓存：LRU 缓存，用以批量地向 LRU 链表中快速地添加页面。有了 LRU 缓存之后，新页不会被马上添加到相应的链表上去，而是先被放到一个缓冲区中去，当该缓冲区缓存了足够多的页面之后，缓冲区中的页面才会被一次性地全部添加到相应的 LRU 链表中去。Linux 采用这种方法降低了锁的竞争，极大地提升了系统的性能。

LRU 缓存用到的 pagevec 结构，如下所示:

```
/* 15 pointers + header align the pagevec structure to a power of two */
#define PAGEVEC_SIZE    15
......

struct pagevec {
    unsigned char nr;
    bool percpu_pvec_drained;
    struct page *pages[PAGEVEC_SIZE];
};
```

pagevec 这个结构就是用来管理 LRU 缓存中的这些页面的。该结构定义了一个数组，这个数组中的项是指向 page 结构的指针。一个 pagevec 结构最多可以存在 15 个这样的项 (见上面代码)。当一个 pagevec 的结构满了，那么该 pagevec 中的所有页面会一次性地被移动到相应的 LRU 链表上去。

每个 cpu 维护五个 pagevec。这五个 pagevec 由 struct lru_pvecs 管理。该结构体定义和声明如下:

```
struct lru_pvecs {
    local_lock_t lock;
    struct pagevec lru_add;
```

```
    struct pagevec lru_deactivate_file;
    struct pagevec lru_deactivate;
    struct pagevec lru_lazyfree;
#ifdef CONFIG_SMP
    struct pagevec activate_page;
#endif
};
static DEFINE_PER_CPU(struct lru_pvecs, lru_pvecs) = {
    .lock = INIT_LOCAL_LOCK(lock),
};
```

这几个 LRU 缓存管理结构对应的操作如下：

- lru_add: 缓存新加入原本不属于 LRU 链表的页。
- lru_deactivate_file: activae LRU 链表中的文件页，清除 PG_activate,PG_referenced 标志后，将降级到 inactivate LRU 链表。用于缓存这些页。
- lru_deactivate: 已经在 activae LRU 链表中的匿名页，清除 PG_activate,PG_referenced 标志后，将降级到 inactivate LRU 链表。用于缓存这些页。
- lru_lazyfree: 缓存释放的页，inactive 链表中延迟释放的页加入到这个链表。

  这是针对 ***MADV_FREE*** 页面的专有 LRU 缓存。MADV_FREE 标志可以由用户空间进程通过系统调用 madvise() 来设置。它向内核表明进程在未来一段时间不会再访问指定的内存区域。这是一种建议，告知内核可以在合适的时候回收该内存区域的物理页面，以释放内存资源。换言之，系统不会立即回收该区域内存，它会记录该区域内存已被标记为” 可回收”。在系统内存紧张时，会优先从这些被标记的区域回收物理内存。这种延迟释放机制可以减少内存回收对进程性能的影响。因为在系统内存充足时，进程仍可使用这些内存，而在需要时可以迅速回收。

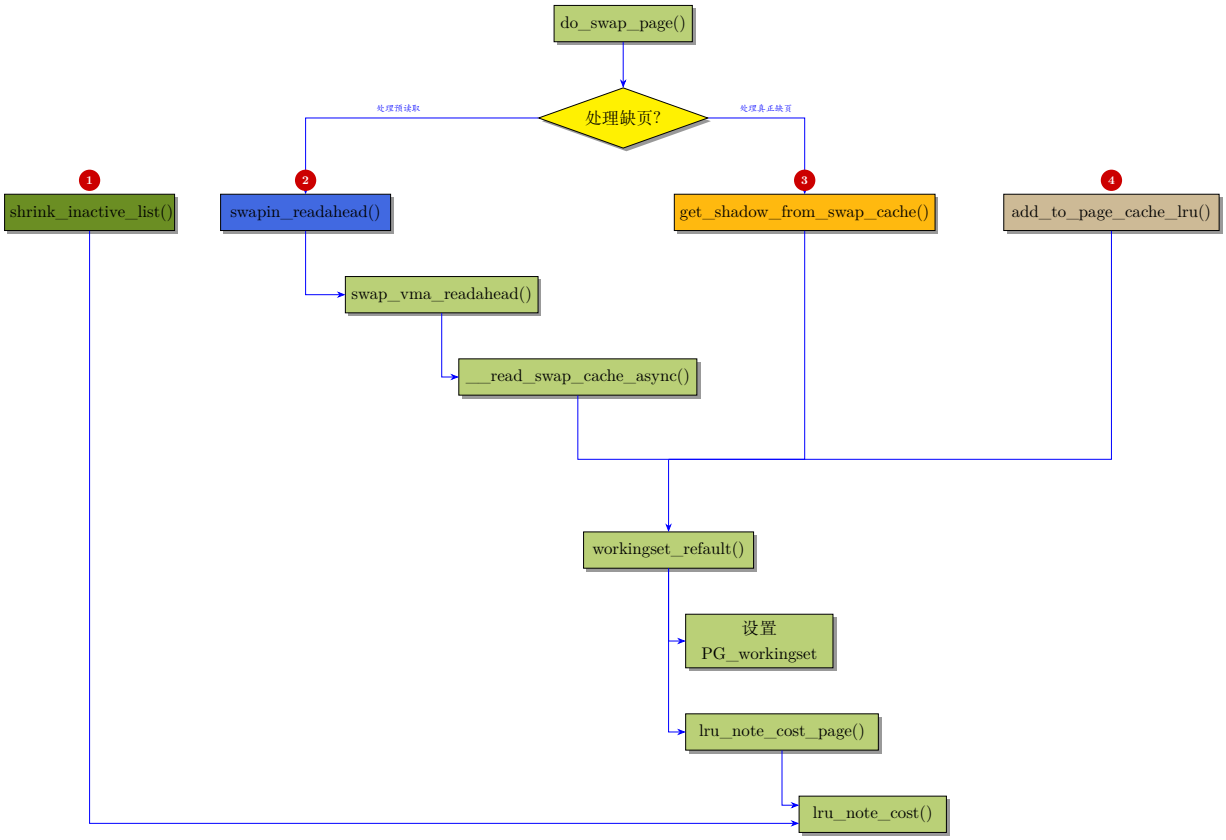
  这种机制的主要目的在于不阻塞用户进程，把 free 释放的操作丢给后台或异步操作。
- activate_page: 用于批量将页面从欠活跃 LRU 列表移动到活跃 LRU 列表，防止页面被过早回收。这是内存访问局部性优化的一部分。通过批量操作减少对 LRU 锁的竞争，提升并发性能。

- 2678 sc->nr 用于记录扫描过程中各类页框的数量
- 2680-2681 获取已经扫描的可回收和可扫描页数。
- 2687-2688 此处->anon_cost 和 ->file_cost 分别表示对匿名页和文件页回收操作的代价度量。用于匿名页和文件页 lru 链表平衡的调整。

  这两个值的计数是一直累加的。在两处场景下系统会累加**回收损耗**(cost) 计数:

  - 对欠活跃链表上的页回收时，如果要执行 I/O 操作写磁盘，说明这是代价昂贵的回收。因此进行损耗计数，计数值是要进行 IO 操作并回收的页的数目。
  - 工作集页发生 refault 时，也进行损耗累加。当某个页发生 refault 时，会去读 shadow 记录的回收时刻的 inactive 链表的 age 计数 (***E***)，并用当前计数 (***R*** 去减。如果 $R - E \leq len(activate)$，说明曾经处于 active 链表是工作集页。如果曾经是 workingset 页，那么证明之前对它进行回收是错误的。所以要记录回收损耗。

  系统中会最终调用 *lru_note_cost()* 来进行回收损耗的累加。各个调用的入口或场景如下:



说明

- 1. shrink_inactive_list() 用于回收欠活跃链表中的页，如果回收页是脏页要产生 I/O 操作，那么代表这是昂贵的回收，损耗较大，因此会记录损耗 (cost)。
- 2,3. 这两种都会在 swap in 时判断是否是工作集匿名页的 refault, 如果是，则证明之前的回收时错误的，因此产生了回收损耗，于是会累加到损耗里。
- 4. 这个是针对文件页的 page in, 原因同上。

- 2695-2719 这段及以下代码的目标是根据 active(活跃) 的匿名页和文件页的比例变化设置扫描控制参数来调整匿名页和文件页的扫描比率，以期达到匿名和文件 LRU 列表的欠活跃与活跃列表的目标理想比例。

  匿名页 (如进程堆、栈等) 无持久后备存储，回收时需写入交换分区 (swap); 文件页有文件作为后端 (如文件缓存), 回收时可直接丢弃或写回磁盘。

  - 2695-2718 的代码段是**不**强制将活跃链表里的页降级到欠活跃链表的场景下的处理。

    在内存回收效果不明显的情况下，会强制将活跃链表的页降级到欠活跃链表，以期加速回收。而整个工程里只有在 ***do_try_to_free_page()*** 的处理快结束时，发现回收效果不明显，才会置 $sc->force_deactivate = 1$ 来强制降级标志，并 retry 回收。
  - 2698 这里"refaults" 变量存储的是目标 lruvec 中因发生二次缺页 (refault)，被 workingset 提升到活跃链表的匿名页的数量。通过比较这个值的变化，以及检查" 欠活跃匿名页" 的数量，内核会动态的决定是否允许对匿名页的活跃链表进行 deactive 操作，从而优化内存管理和回收策略。

” 工作集” 顾名思义就是”工作页的集合”。

工作集（workingset）概念其核心思想是：程序在内存访问行为上具备时间局部性。在时刻 t，一个进程的工作集定义为该进程在时间段 $(t-\tau, t)$ 内访问过的页面集合，记作 $W(t,\tau)$。其中 τ 被称为工作集参数，是可自定义的值。在 τ 时间窗口内未被访问的内存页面，可以通过采样机制回收。

在 Linux 内核中，工作集对应欠活跃 LRU 链表 (inactive LRU) 与活跃 LRU 链表 (active LRU)。文件页/匿名页首次载入内存时会加入欠活跃 LRU 链表；若页面被反复读取，则提升至活跃 LRU 链表。

当内存压力触发页面回收时，内核从欠活跃链表尾部释放页面。一旦欠活跃链表长度远小于活跃链表，内核会将活跃链表中的页面迁移至欠活跃链表，平衡两条链表的容量。

欠活跃链表中的页面只有两种归宿：提升为活跃页，或是被回收；链表内所有页面持续向尾部推进 (老化)。页面需要在欠活跃链表中走完整条链表长度，抵达尾部才会被回收。

一部分文件页具备周期性访问特征：页面初次加载，放入欠活跃链表头部；随着不断载入新页面、尾部旧页面被回收，该页面逐步被推向链表尾部，最终被回收。但回收一段时间后，系统需要再次访问该页面，于是重复加载 $\rightarrow$ 后移 $\rightarrow$ 回收全过程，这类页面称为周期性访问页面。

活跃链表 + 欠活跃链表共同构成工作集；目标是尽可能保留进程工作集内正在访问的页面，避免被错误回收。

周期性访问页面的生命周期：从欠活跃链表头部逐步走到尾部，面临回收，这段路径长度等于欠活跃链表总长度。从页面被回收，到下一次再次访问它的这段时间里，欠活跃链表仍持续发生其它新的页面激活、页面回收。假设这段区间总共发生了 X 次页面激活 + 回收事件。

理想场景：页面被放置在距离欠活跃链表尾部 $X + len(inactive)$ 的位置，刚好在该页抵达欠活跃链表尾部时被访问，避免回收、消除反复震荡。但是现实约束：欠活跃链表最大长度只有 $len(inactive)$。我们需要创造出一段长度恰好是 $X + len(inactive)$ 的缓冲空间。

由于欠活跃链表上游是活跃链表，解决方案是：把页面插入活跃链表倒数第 X 个的位置。但是在链表中间插入元素的代码实现复杂而耗时。因此简化方案：直接放到活跃链表头部，以此实现保护效果。

页面放到活跃链表头部后，完整路径为：活跃链表 + 欠活跃链表，即:

$$\text{最大缓冲距离} = len(active) + len(inactive)$$

想要避免回收，需要的缓冲距离是: $X + len(inactive)$

只要满足:

$$len(active) + len(inactive) > X + len(inactive)$$

等价于:

$$len(active) > X$$

满足条件时，将页面放入活跃链表头部，就能保证页面在下一次访问到来前不会被回收; 不满足则放入欠活跃链表。

这里关键问题在于如何计算 X。X 的定义是: 周期性被访问的页面从被回收那一刻起，直到再次被访问为止，欠活跃链表发生的页面激活、页面回收总次数。

因此而达成的实现方案是: 设置全局计数器。每当欠活跃链表页面发生激活或者回收，计数器自增。周期性访问页面被回收时，把当前计数器值存入该页面对应的表项；页面再次被读取、重新载入内存时，取出旧计数值，

$$X = \text{计数器当前计数} - \text{计数器计数旧值}$$

判断

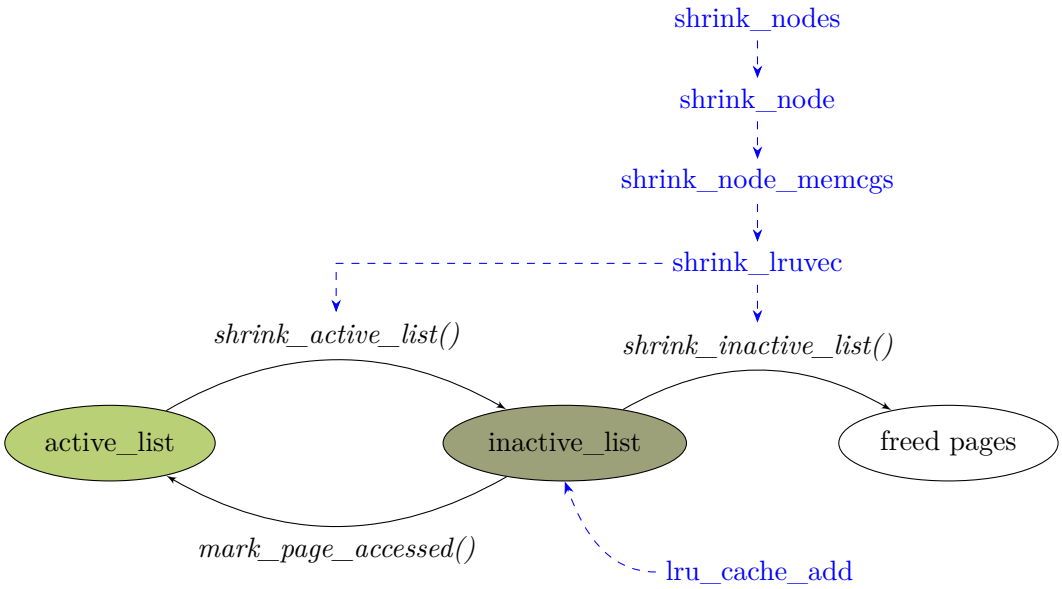
$$len(active) > X$$

是，则页面放入活跃链表头部，启用工作集保护。

否, 则将页面放入欠活跃链表。

内存页面分为活跃 (频繁访问) 和欠活跃 (较少访问) 两类，内核会优先回收欠活跃页面，避免回收频繁访问的页面导致性能损失。页面在活跃和欠活跃链表之间动态迁移，基于访问频率调整其优先级，最终将长期未访问的页面逐步淘汰回收。

工作页在不同链表之间的状态转换图及对应的接口函数如下:



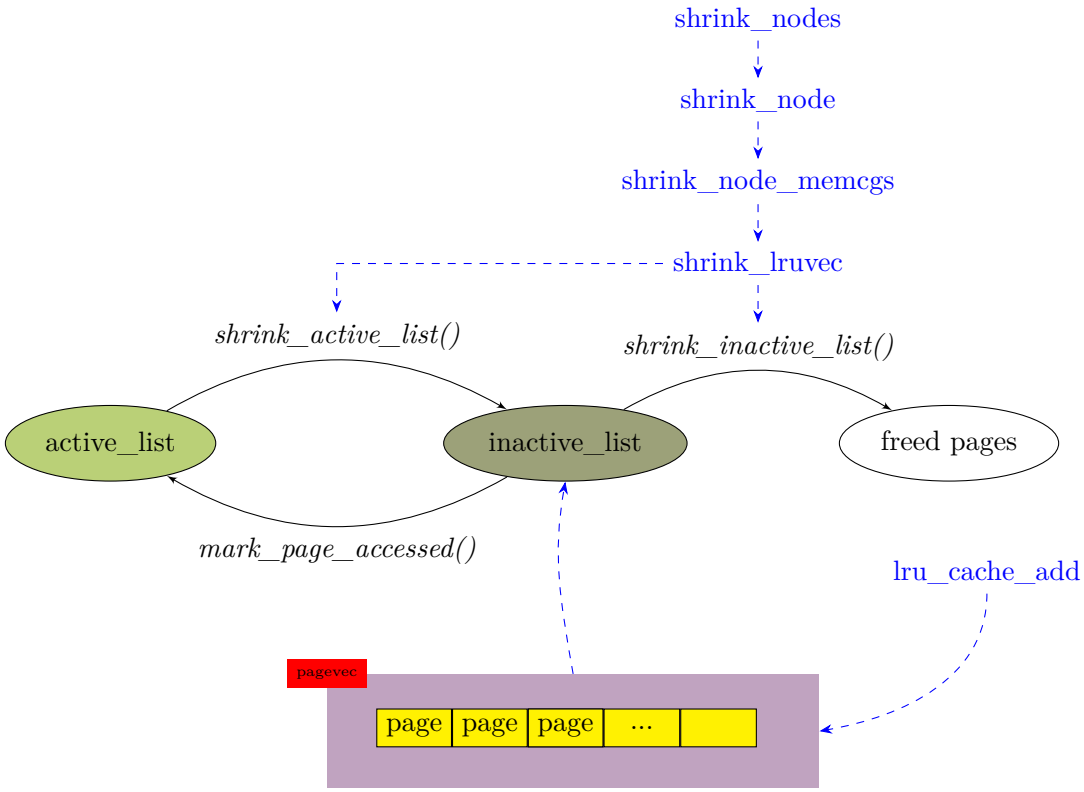
因缺页 (page fault) 而新分配的页面 (无论是匿名页还是文件页) 在首次加入 LRU 链表时, 会被放入欠活跃链表 (如 LRU_INACTIVE_ANON 或 LRU_INACTIVE_FILE), 当该缺页页面被再次访问 (refault) 时, 如果 $refault_distance(R-E) \le len(active_lru)$, 那么会被判定为工作集页直接放到到活跃链表。

需要注意的是，上图其实是简略的示意图。描述不够精确，省掉了关键的 pagevec 批处理环节。

页首次加入 LRU 链表，被放入欠活跃链表。或者是页在欠活跃链表再次被访问时提升到活跃链表。一般情况下不直接加入全局 (或 memcg) LRU 链表，而是先加入当前 CPU(per-CPU) 的页向量 (pagevec) 缓存。

通常只有当页向量的数组满了—默认是 PAGEVEC_SIZE(14) 页) — 或主动排空时，才批量刷入全局 (或 memcg) LRU。所以图中最终目标是 LRU 链表没错，但要经过中间路径

pagevec 再到最终的 LRU 链表，如下图。



内核在下面这些操作之前必须主动排空 pagevec 向量，保证页面进入 LRU 链表:

- 进程切换前
- 内存回收（kswapd）开始前
- 扫描 LRU 链表前
- 关闭 CPU、热插拔时
- sync 刷缓存时

目的在于不让页面永远留在 pagevec 里，必须进入 LRU 才能被回收、统计、扫描。

下表是 pagevec 和 LRU 相关的操作函数:

函数	功能
lru_cache_add()	将页面加入 *percpu* 的页向量缓存。当缓存满、加入第一页或页面为复合页时，批量将页面刷入 *inactive LRU* 链表。
lru_add_drain()	排空当前 *CPU* 的 *pagevec* 页向量缓存
lru_add_drain_all()	排空所有 *CPU* 的 *pagevec* 页向量缓存
lru_add_drain_cpu()	排空指定 *CPU* 的 *pagevec* 页向量缓存

- 2700-2702 如果 active(活跃) 的匿名页相比上一次循环已经有了变化，或者当前”inactive(欠活跃匿名页)” 链表上页框数量偏少，则设置” 需要扫描匿名页”。满足任意一个条件，就打开 ***DEACTIVATE_ANON*** 开关

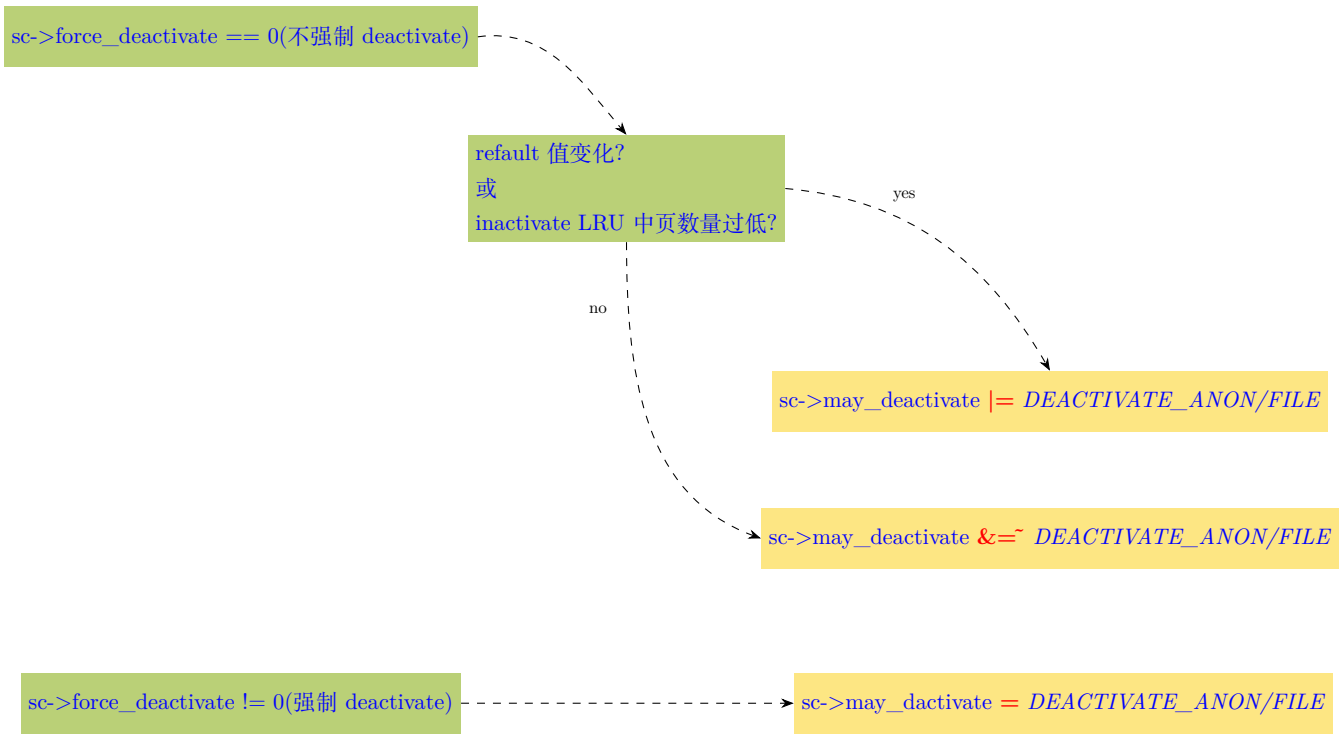
$->refaults[0]$ 里存放的是上一轮扫描时读到的 WORKINGSET_ACTIVATE_ANON 统计的快照值。当前计数器 $\neq$ 上一轮快照说明本轮周期内，发生了新的匿名页 workingset refault 激活事件。refault 本身说明有工作集页被错误回收，产生颠簸风险。但同时侧面反映 active 匿名链表在增长，链表比例失衡。所以打开 DEACTIVATE_ANON 许可，允许扫描活跃匿名链表，把其中真正的冷旧页面降级，快速清理过时的旧工作集。这里仅仅是许可，允许 shrink_active_list 去扫描匿名活跃链表，真正降级多少页面，在 shrink_active_list() 内部，会参考 refault 增量做限流。

如果 inactive 链表太少，就算 refault 没有新增，也必须允许去激活 (deactivate)，把 active 页降级，补足 inactive 链表，否则没有页面可以回收。

- 2703-2704 上面条件都不满足，则不允许降级活跃匿名页链表。
- 2711-2717 同理，按条件设置是否允许降级活跃文件页。
- 2719 如果设置了强制扫描，则匿名页和文件页都允许对活跃 LRU 链表上的页进行降级。

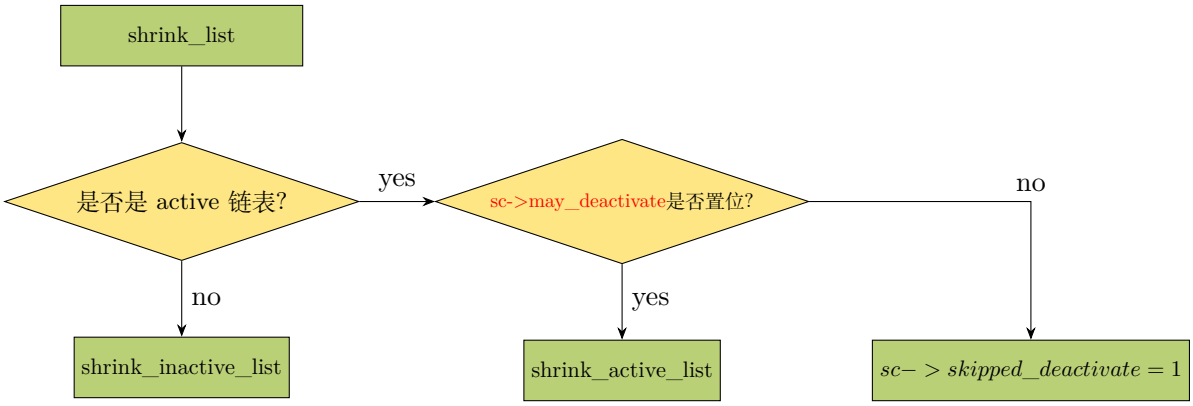
总而言之，上面 2698-2717 行根据 refaults(重缺页) 值的变化，以及匿名页和文件页 inactive 链表中的页数量来考虑设置 deactive 标志。

换言之，即使没有强制设置扫描控制参数 deactive(sc->force_deactive==0)，在 refault 值发生变化或 inactive LRU 链表中页数量过低时，也仍然可能置位 sc->may_deactive。如图：



将来在 shrink_list() 执行时可以依据 sc->may_deactivate 的值考虑是否将 active(活跃) 链表中的页降级到 inactive(欠活跃) 链表。

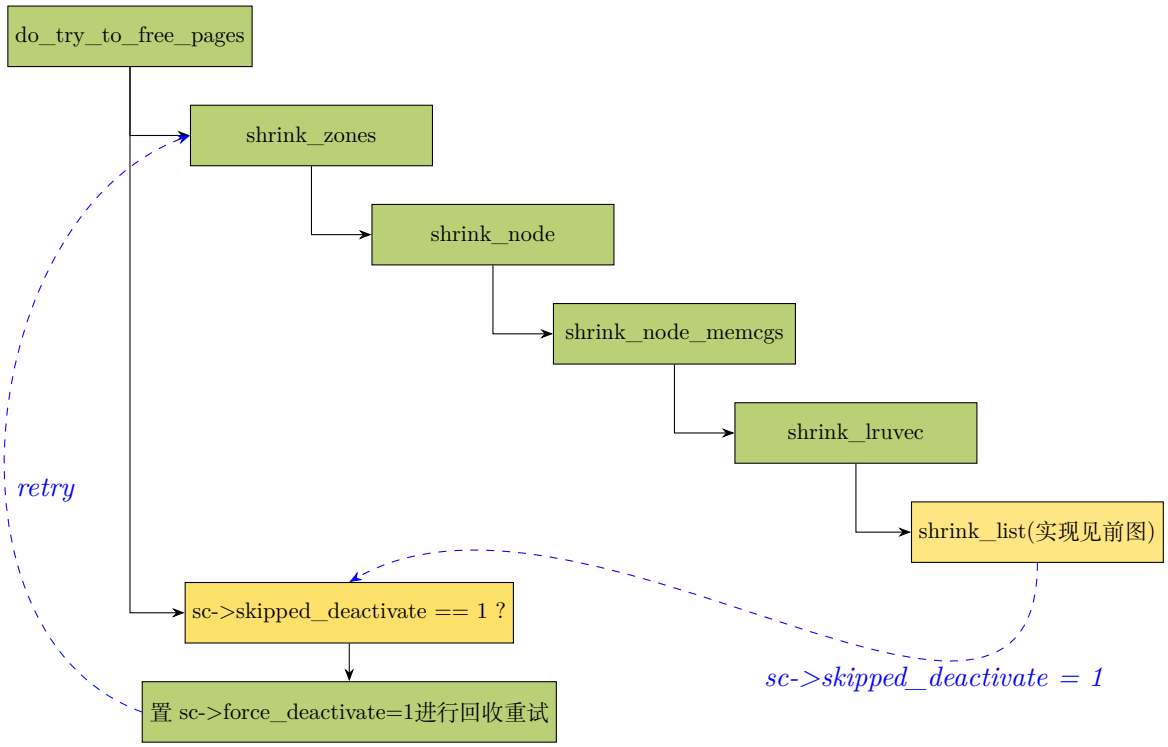
shrink_list() 函数比较简短，执行流程如图：



从上图可看出，在 sc->may_deactivate 为 0, 即 activate 链表不允许页降级到 inactive 链表时，shrink_list() 会给扫描控制参数 sc->skipped_deactivate 置位来对 shrink 时不允许页降级的行为记录。

内核基于节点 node 的内存构成来估算欠活跃 (inactive)/活跃 (active) 内存的比例，但 reclaim_idx 或 memory.low cgroup 的限制性设置可能会使大量内存豁免于回收，不过这两种设置并不常用。因此内核没有考虑这两种情况而对整个 cgroup 层级结构进行代价高昂的精确计算，而是假定普遍场景下估算准确。若回收操作失败则通过允许强制 deactive 来重试。

譬如，在回收失败时，如果检测到 sc->skipped_deactivate 已经置位，那么会将该变量清 0，并且置位”强制降级”标志 sc->force_deactivate 的值来重试回收。对扫描控制参数 sc->skipped_deactivate 的使用代码，详见直接回收最终调用的函数 do_try_to_free_pages()。



- 2726-2730 如果目前存在不会导致内存颠簸的大量欠活跃文件页，那么在回收匿名页之前，应尝试先回收或扫描这些文件页，以利扫描平衡。

  这就是所谓的 cache_trim_mode(page cache 修剪模式)。把 cache_trim_mode 值置 1，会优先进行文件页扫描。

  这里会去读取 lruvec->stat[] 数组，这是 per-lruvec 的变量，它是内核维护的原子变量。每次加页到链表，就会 *inc_lruvec_stat()*. 移除页就会调用 *dec_lruvec_stat()*。读计数器快照避免了链表遍历的巨大开销。

  2727 判断 $file >> sc->priority$ 是否为 true，等价于判断下面的表达式，即 file 页数量要 $> 2^{sc->priority}$:

$$file > (1 << sc->priority)$$

  即,

$$file > 2^{sc->priority}$$

  或者也可理解为，当扫描力度是 $sc->priority$ 时，$\frac{file}{2^{sc->priority}}$ 得到应该扫描的页数量。

  也就是比较欠活跃 (inactive) 页数量, 和当前回收优先级对应的页动态阈值。如果前者足够多，并且同时不允许通过降级文件页，将页面从活跃状态转移到欠活跃状态来回收文件页时，将启用缓存修剪 (trim) 模式 (优先回收文件页)。” 降级 (deactivate)” 操作是内存回收的前置步骤。回收时，页面先从 active 链表移到 inactive 链表 (deactivate), 然后从 inactive 链表中才能被回收 (释放或交换到后备存储)。

  禁用了 deactivate 的情况下，系统只能回收已在欠活跃链表中的页面，而无法将新的活跃页转为可回收状态。

  该行之所以检查”是否允许对文件页执行降级操作”，是因为 active 文件页当前可能被频繁访问，一旦降级到欠活跃链表，在修剪模式下可能会迅速回收，从而导致内存颠簸。

前面说明 2698 行时提到的 lru_cache_add() 源码如下:

5.1.5 lru_cache_add()

lru_cache_add()

```
462 void lru_cache_add(struct page *page)
463 {
464     struct pagevec *pvec;
465
466     VM_BUG_ON_PAGE(PageActive(page) && PageUnevictable(page), page);
467     VM_BUG_ON_PAGE(PageLRU(page), page);
468
469     get_page(page);
470     local_lock(&lru_pvecs.lock);
471     pvec = this_cpu_ptr(&lru_pvecs.lru_add);
472     if (!pagevec_add(pvec, page) || PageCompound(page))
473         __pagevec_lru_add(pvec);
474     local_unlock(&lru_pvecs.lock);
475 }
476 EXPORT_SYMBOL(lru_cache_add);
```

- 466 如果一个页同时是活跃页又是不可回收页，那么很显然是错误的，直接 crash。

  内核规定用户页只能是下面三种状态之一：

  - inactive: 非活跃、可回收
  - active: 活跃、可回收
  - unevictable: 不可回收

  一个页要么在可回收体系 (active/inactive)，要么在不可回收体系 (unevictable)，绝对不能同时在两个体系里。如果一个用户页被 mlock 锁住，那么该页不会进入 active/inactive 链表，不会被 kswapd 扫描，不会被回收.

- 467 如果页面已经在 LRU 链表上，那么直接 crash。

  lru_cache_add() 只能用于新页。而且一个页永远只能被加入 LRU 一次。这里为了防止重复加入 LRU 导致链表损坏。

- 469 页面引用计数 +1.

  保证页面在加入 pagevec 并最终进入 LRU 期间不会被释放。

- 470 获取 CPU 本地锁。保护 percpu pagevec 并发访问。

- 471 获取当前 CPU 的 pagevec 向量。所有新页都先暂存在向量数组中.

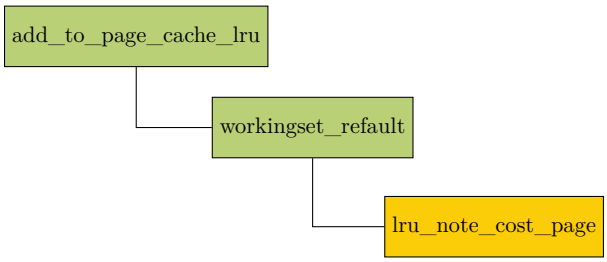
- 472-473 如果目标页加入 pagevec 向量后向量数组满了 (最大为 ***PAGEVEC_SIZE***)，或者目标页是复合大页。那么立刻把 pagevec 中所有页面排空批量加入 LRU 链表。

- 474 释放锁。

5.1.6 回收损耗

5.1.6.1 回收损耗 • lru_note_cost_page()

前面 2687-2688 提到的 lru_note_cost_page() 会在 add_to_page_cache_lru() 中调用。调用链如下:



add_to_page_cache_lru () 有两处地方调用: 首次读文件时会调用，在文件页 (page cache) 被回收后再次访问 (refault) 时，也会调用。

add_to_page_cache_lru()

```
2732 int add_to_page_cache_lru(struct page *page, struct address_space *mapping,
2733                 pgoff_t offset, gfp_t gfp_mask)
2734 {
2735     void *shadow = NULL;
2736     int ret;
```

```
2737
2738        __SetPageLocked(page);
2739        ret = __add_to_page_cache_locked(page, mapping, offset,
2740                                         gfp_mask, &shadow);
2741        if (unlikely(ret))
2742            __ClearPageLocked(page);
2743        else {
2744            /*
2745             * The page might have been evicted from cache only
2746             * recently, in which case it should be activated like
2747             * any other repeatedly accessed page.
2748             * The exception is pages getting rewritten; evicting other
2749             * data from the working set, only to cache data that will
2750             * get overwritten with something else, is a waste of memory.
2751             */
2752            WARN_ON_ONCE(PageActive(page));
2753            if (!(gfp_mask & __GFP_WRITE) && shadow)
2754                workingset_refault(page, shadow);
2755            lru_cache_add(page);
2756        }
2757        return ret;
2758    }
2759    EXPORT_SYMBOL_GPL(add_to_page_cache_lru);
```

- 2739-2742 把页面从后备存储或磁盘加入到内存 RAM/DDR 等。加入失败则直接解锁页面，返回错误。在此过程会以原子方式读取影子项数据并清 0。
- 2752 判断页是否是活跃页。如果这个新加到内存的文件页，居然是 active 状态，那就意味着内核错误，打印警告信息。
- 2753-2754 判断目标文件页是否属于重载页。若是，workingset_refault() 函数中会进一步判断目标页是否属于工作集页 (见后)，如果是，会将该页直接加入 active LRU 链表，避免短期内又被回收。

  2573 行的 if 会判断如下两项:

  首先判断这次载入页到内存的目的是否为了写入数据。写操作会重写或覆盖页面，内核社区认为改变了页的内容，不应该认为是”重载” 页。因此写操作的页不应该认为是工作集。

  其次，判断该页是否有影子项，若有影子项，则目标页才有可能是” 重载页”。
- 2755 将页加入内存管理的 LRU 链表。

5.1.6.2 workingset_refault()

上面 2754 行的 workingset_refault() 在发生 reafult 时调用。判断刚复活 (重载) 的这个页面，是不是内存不足时被误杀的工作集页。如果是，立刻激活到 active 链表保护。

add_to_page_cache_lru() > *workingset_refault()*

```
282 void workingset_refault(struct page *page, void *shadow)
283 {
284     bool file = page_is_file_lru(page);
285     struct mem_cgroup *eviction_memcg;
286     struct lruvec *eviction_lruvec;
287     unsigned long refault_distance;
288     unsigned long workingset_size;
289     struct pglist_data *pgdat;
290     struct mem_cgroup *memcg;
291     unsigned long eviction;
292     struct lruvec *lruvec;
293     unsigned long refault;
```

```
294     bool workingset;
295     int memcgid;
296 
297     unpack_shadow(shadow, &memcgid, &pgdat, &eviction, &workingset);
298 
299     rcu_read_lock();
300     /*
301      * Look up the memcg associated with the stored ID. It might
302      * have been deleted since the page's eviction.
303      *
304      * Note that in rare events the ID could have been recycled
305      * for a new cgroup that refaults a shared page. This is
306      * impossible to tell from the available data. However, this
307      * should be a rare and limited disturbance, and activations
308      * are always speculative anyway. Ultimately, it's the aging
309      * algorithm's job to shake out the minimum access frequency
310      * for the active cache.
311      *
312      * XXX: On !CONFIG_MEMCG, this will always return NULL; it
313      * would be better if the root_mem_cgroup existed in all
314      * configurations instead.
315      */
316     eviction_memcg = mem_cgroup_from_id(memcgid);
317     if (!mem_cgroup_disabled() && !eviction_memcg)
318         goto out;
319     eviction_lruvec = mem_cgroup_lruvec(eviction_memcg, pgdat);
320     refault = atomic_long_read(&eviction_lruvec->nonresident_age);
321 
322     /*
323      * Calculate the refault distance
324      *
325      * The unsigned subtraction here gives an accurate distance
326      * across nonresident_age overflows in most cases. There is a
327      * special case: usually, shadow entries have a short lifetime
328      * and are either refaulted or reclaimed along with the inode
329      * before they get too old.  But it is not impossible for the
330      * nonresident_age to lap a shadow entry in the field, which
331      * can then result in a false small refault distance, leading
332      * to a false activation should this old entry actually
333      * refault again.  However, earlier kernels used to deactivate
334      * unconditionally with *every* reclaim invocation for the
335      * longest time, so the occasional inappropriate activation
336      * leading to pressure on the active list is not a problem.
337      */
338     refault_distance = (refault - eviction) & EVICTION_MASK;
339 
340     /*
341      * The activation decision for this page is made at the level
342      * where the eviction occurred, as that is where the LRU order
343      * during page reclaim is being determined.
344      *
345      * However, the cgroup that will own the page is the one that
346      * is actually experiencing the refault event.
347      */
348     memcg = page_memcg(page);
349     lruvec = mem_cgroup_lruvec(memcg, pgdat);
350 
351     inc_lruvec_state(lruvec, WORKINGSET_REFAULT_BASE + file);
352 
353     /*
354      * Compare the distance to the existing workingset size. We
355      * don't activate pages that couldn't stay resident even if
356      * all the memory was available to the workingset. Whether
357      * workingset competition needs to consider anon or not depends
```

```
358      * on having swap.
359      */
360     workingset_size = lruvec_page_state(eviction_lruvec, NR_ACTIVE_FILE);
361     if (!file) {
362         workingset_size += lruvec_page_state(eviction_lruvec,
363                             NR_INACTIVE_FILE);
364     }
365     if (mem_cgroup_get_nr_swap_pages(memcg) > 0) {
366         workingset_size += lruvec_page_state(eviction_lruvec,
367                             NR_ACTIVE_ANON);
368         if (file) {
369             workingset_size += lruvec_page_state(eviction_lruvec,
370                             NR_INACTIVE_ANON);
371         }
372     }
373     if (refault_distance > workingset_size)
374         goto out;
375
376     SetPageActive(page);
377     workingset_age_nonresident(lruvec, thp_nr_pages(page));
378     inc_lruvec_state(lruvec, WORKINGSET_ACTIVATE_BASE + file);
379
380     /* Page was active prior to eviction */
381     if (workingset) {
382         SetPageWorkingset(page);
383         /* XXX: Move to lru_cache_add() when it supports new vs putback */
384         spin_lock_irq(&page_pgdat(page)->lru_lock);
385         lru_note_cost_page(page);
386         spin_unlock_irq(&page_pgdat(page)->lru_lock);
387         inc_lruvec_state(lruvec, WORKINGSET_RESTORE_BASE + file);
388     }
389 out:
390     rcu_read_unlock();
391 }
```

- 284 根据传入的目标页的类型获取页的类型索引。page_is_file_lru() 返回值是 0 或 1。

  内核中处理文件页或匿名页时，很多处公用相同的代码，仅通过页类型的相对索引 read/write 数据到数组的不同位置。譬如，0 代表匿名页相对索引，1 代表文件页相对索引。那么file为 0 时，data[XX_BASE+file] 就是匿名页数据的相对位置。同理，file为 1 时，data[XX_BASE+file] 就是文件页数据的相对存取位置。特别的，这里的名称”file” 是 bool 型临时变量名，代表是否文件页，更准确的命名其实应该为”is_file”, 而不应该将其理解为”file” 这个单词本身的含义。

- 297 将影子项 (shadow) 里压缩存储的信息解包。主要是如下 4 项信息:

  - ***memcgid*** 页面被回收时所属的内存 cgroup 组的 ID
  - ***pgdat*** 标识属于哪个内存 node 节点
  - ***eviction*** 页面被回收时的” 年龄” 或” 页距离”。

    就像树的横界面上年轮的圈数可以用来计量树的年龄[①]，年轮每增加一圈，树就多了一岁。这里以年轮的一圈就是作为不可分割的量子单位去” 刻度” 年。

    相似的，为了标记每次页操作时系统的时间” 刻度”，我们以一页作为量子单位去刻度两个时刻的差异。这里没有以 jiffies 或墙上时钟的客观流逝来统计。是因为就像树一样: 树因为一年四季温度、水分不一样，树长得时快时慢造成了疏密形成了一圈年轮。如果把春夏秋冬的时间拉长成 2 倍，那么即使 2 年时间，树也仍只形成一圈年轮，换言之，也就是一次从萌芽到落叶的周期。

[①]这里并非严谨的科学性描述。因为对于热带而言没有明显的四季，树一年四季长得几乎一样快，没有“快一慢”交替，因此就没有明显的年轮圈。

同样的，考虑到系统长时间休眠 (几小时、几天、几周...), 我们关注页操作的时刻也只需要关注譬如从页载入到回收等的页操作的使用周期，并不用关注客观流逝的时间。因此，系统以每个页操作时刻系统已回收的页数目做为刻度，方便计算页 Δ(增量差或减量差) 来度量页生命周期。为了方便我的理解，以后该刻度数据就称为” 页年轮”.

- ***workingset*** bool 量，只有 0 和 1 值。标识目标页曾经是否是工作集页 (即，回收前已经被激活为 active 页)。

- 299 读取 memcg 这种可能被动态删除的结构，必须加 RCU 锁防止读取过程中 cgroup 被释放。
- 316-319 利用影子项存储的 memcgid 和 pgdat，获取把这页踢出去的内存控制组的 LRU 链表。
- 320 当前时刻目标 inactive 链表上移除页事件的计数快照。

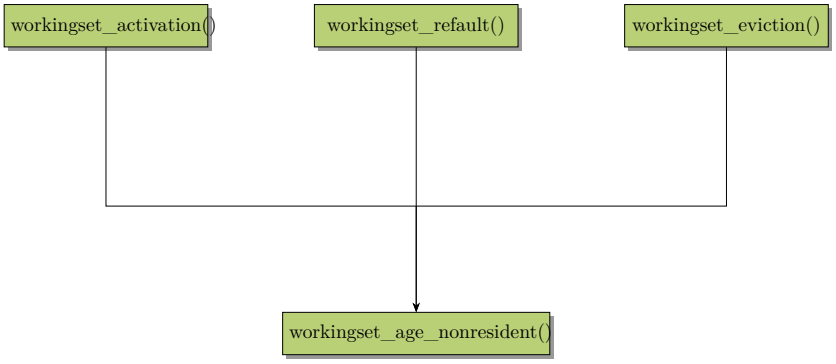
  ->nonresident_age 是全局递增计数。当一个页被换出时，内核会将当前时刻的 *->nonresident_age* 值写入该页的 eviction 字段 (见 2748 行的说明)。对于透明大页，会按折算成基页的数量进行计数。

  系统会在三种场景下调用 workinget_age_nonresident() 来增加->***nonresident*** 的计:

  - ***workingset_activation()*** 将页从 inactive 链表激活到 active 链表时。
  - ***workingset_refault()*** 页 refault 时会通过 $R - E \leq workingset_size$ 判断是否工作集页的 refault, 如果是, 直接加入 active 链表并同时增加 nonresident 计数。

    另外，在工作集页的基础上如果进一步判断，是否回收前就曾是工作集页。如果是，那么增加 cost(回收损耗) 计数。如果目标页被回收前不是工作集页，那么不增加回收损耗的计数。
  - ***workingset_eviction()*** 页从 inactive 链表中被回收时。

  总的来说，这三种情况 (如下图) 都可以认为是从 inactive 链表中产生了移除事件时，对移除 页的数量进行的计数。其他两种很容易理解，对于 workingset_refault(), 可以看成先加入 inactive 链表，又立即从 inactive 升级到 active 链表。这样也可以理解成从 inactive 链表的” 移除”。



- 338 通过两个时刻的页年轮来计算页重载距离 (refault distance)，也就是自目标页被回收的时刻到目前时刻为止，这个时间段系统 inactive 链表上 $\Delta_{\text{页移除事件}}$ 的值。
- 351 对目标页之前所属的 lruvec 记录一次页重载 (refault) 事件 (+1)。
- 360-372 计算当前 LRU 链表中最多能装下的 “活跃页总数”，也就是工作集大小。

  在这个代码段之后会进行页重载距离和工作集页的大小判断，来判定 refault 的页是否工作集页。

这里会判断系统中其他类型的 LRU 链表上的页可以回收 (踢出) 供给目标工作集使用的情况下，如果目标 refault 页仍然无法保证常驻内存，那么就不将其激活成 active 页。下面就是基于该思想计算获得的 *workingset_size*, 另外还考虑了是否支持 swap out。

- 360 获取活跃文件 LRU 链表上文件页的数目。
- 361-364 如果这次 refault 的页是匿名页 ($file == 0$), 则将欠活跃文件页也计入 workingset 的长度。

  也就是说如果 refault 页是匿名页，那么在不支持 swap 的情况下，$workingset_size$ = 活跃文件页 + 欠活跃文件页
- 365-366 在支持 swap 的情况下,workingset_size 再 +活跃匿名页。
- 368-371 如果 refault 是文件页，那么 workingset_size 再 +欠活跃匿名页。

- 373-374 判断页重载距离 (refault distance) 和工作集的大小。如果页重载距离大于工作集，在此情况下系统已经无能为力，不将该页加入工作集，退出。反之，说明该页被误回收，内存足够留住该页，继续往下执行。
- 376 通过前面的执行判定目标页是工作集页，因此激活为活跃页。
- 377 更新工作集的页年轮。
- 378 增加工作集活跃页的计数。
- 381-388 判断该页回收之前是否是工作集页，若是，则此次 rafault 再次标记为工作集页。同时记录回收开销、增加重载页计数。

  385 当工作集页发生 refault(重载)，证明之前的回收时误操作。因此这里会统计这种” 代价 (开销、损耗)”，为之后的回收或平衡提供考虑的依据。

前面 385 行 lru_note_cost_page() 最终调用 lru_note_cost() 记录回收代价。这里记录的代价指：文件页/匿名页发生了工作集页被误回收，相对而言这是回收错误，这种错误是” 昂贵的” 额外开销或成本。lru_note_cost() 会累计” 代价”:

add_to_page_cache_lru() > *workingset_refault()* > *lru_note_cost_page()* > *lru_note_cost()*

```
281 void lru_note_cost(struct lruvec *lruvec, bool file, unsigned int nr_pages)
282 {
283     do {
284         unsigned long lrusize;
285 
286         /* Record cost event */
287         if (file)
288             lruvec->file_cost += nr_pages;
289         else
290             lruvec->anon_cost += nr_pages;
291 
292         /*
293          * Decay previous events
294          *
295          * Because workloads change over time (and to avoid
296          * overflow) we keep these statistics as a floating
297          * average, which ends up weighing recent refaults
298          * more than old ones.
299          */
```

```
300         lrusize = lruvec_page_state(lruvec, NR_INACTIVE_ANON) +
301               lruvec_page_state(lruvec, NR_ACTIVE_ANON) +
302               lruvec_page_state(lruvec, NR_INACTIVE_FILE) +
303               lruvec_page_state(lruvec, NR_ACTIVE_FILE);
304
305         if (lruvec->file_cost + lruvec->anon_cost > lrusize / 4) {
306             lruvec->file_cost /= 2;
307             lruvec->anon_cost /= 2;
308         }
309     } while ((lruvec = parent_lruvec(lruvec)));
310 }
```

- 287-290 记录页发生误回收的页数目作为”代价”加入到总代价中。如果是透明巨页，lru_note_cost_page() 会换算成基页数目再传递给当前函数。

  每有一页被误回收，代价就 +1. 越频繁误回收，代价开销就越高。内核会参考这个动态值来平衡页回收。

- 300-303 计算当前 lruvec 管理的所有内存页总数:

  活跃匿名页 + 不活跃匿名页 + 活跃文件页 + 不活跃文件页

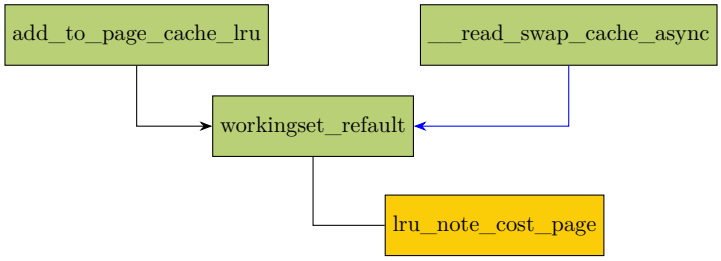
- 305-308 当总代价 > 总 lruvec 内存的 1/4 时，把 file_cost、anon_cost 全部除以 2（衰减一半）。

  这样做的目的在于: 防止数字无限变大溢出; 旧的代价不能实时代表当前状况，因此重要性越来越低，而新代价权重更高，尽量只关注最近的错误 (工作集页误回收)。当然，这里的除以 2 得到的是一个启发性值也就是经验值。

- 309 找到父 cgroup 的 lruvec，从当前 lruvec 一直向上递归到根 cgroup，全部记录一遍。

5.1.6.3 匿名页误回收开销 • ___read_swap_cache_async()

匿名页对于误回收开销的计算也是调用 workingset_refault(). 同样的，时机也是在匿名页从 swap 分区加入到内存时。调用链如下:



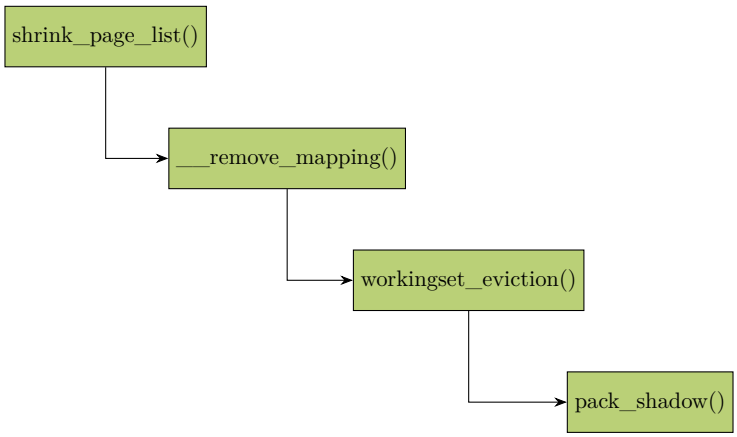
匿名页调用 workingset_refault() 的最底层函数如上图所示，为 ___read_swap_cache_async()。

5.1.7 页回收时工作集信息的存储

前面提到页在加载到内存时，会进行判断，判断该页在之前回收时是否曾存在 active LRU 中。换言之，也就是所谓的”工作集”页。那么，对应的在页回收时，我们需要把 workingset 的元信息保存，这样才能在加载 (page fault,refault) 时进行检测。

5.1.7.1 文件页 workingset 信息的保存

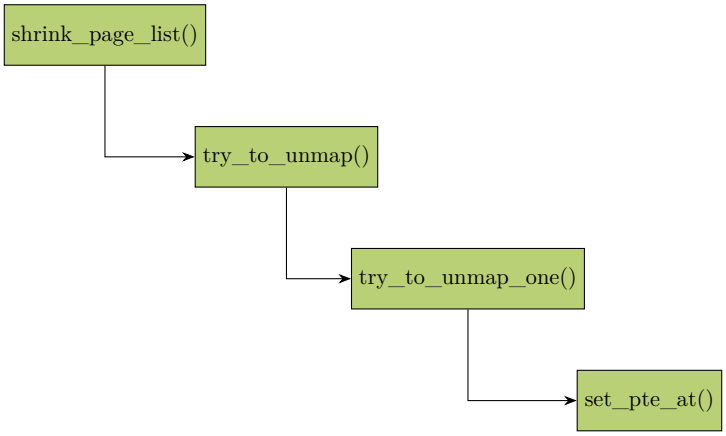
如下所示，是文件页在回收时保存 workingset 信息的执行流程:



文件页回收时，通过影子项 (shadow entry) 机制，将工作集信息编码并替换原来的 struct page * 指针存储在 *address_space->i_pages* 中，从而在物理页释放后保留页的历史信息。

5.1.7.2 匿名页 workingset 信息的保存

如下所示，是匿名页在回收时保存 workingset 信息的执行流程:



其中会通过 try_to_unmap() 遍历所有映射 PTE, 对每个读取当前 PTE、准备 workingset 信息、分配 swap entry、编码 (PTE = workingset 信息 + $swapentry$。最后调用 set_pte_at() 保存信息。

5.1.8 shrink_node()(继续)

继续回到 shrink_node() 的代码:

do_try_to_free_pages() > *shrink_zones()* > *shrink_node()*

```
2732    /*
2733     * Prevent the reclaimer from falling into the cache trap: as
2734     * cache pages start out inactive, every cache fault will tip
2735     * the scan balance towards the file LRU.  And as the file LRU
2736     * shrinks, so does the window for rotation from references.
2737     * This means we have a runaway feedback loop where a tiny
2738     * thrashing file LRU becomes infinitely more attractive than
2739     * anon pages.  Try to detect this based on file LRU size.
2740     */
2741    if (!cgroup_reclaim(sc)) {
2742        unsigned long total_high_wmark = 0;
2743        unsigned long free, anon;
2744        int z;
2745
2746        free = sum_zone_node_page_state(pgdat->node_id, NR_FREE_PAGES);
2747        file = node_page_state(pgdat, NR_ACTIVE_FILE) +
2748               node_page_state(pgdat, NR_INACTIVE_FILE);
2749
2750        for (z = 0; z < MAX_NR_ZONES; z++) {
2751            struct zone *zone = &pgdat->node_zones[z];
2752            if (!managed_zone(zone))
2753                continue;
2754
2755            total_high_wmark += high_wmark_pages(zone);
2756        }
2757
2758        /*
2759         * Consider anon: if that's low too, this isn't a
2760         * runaway file reclaim problem, but rather just
2761         * extreme pressure. Reclaim as per usual then.
2762         */
2763        anon = node_page_state(pgdat, NR_INACTIVE_ANON);
2764
2765        sc->file_is_tiny =
2766            file + free <= total_high_wmark &&
2767            !(sc->may_deactivate & DEACTIVATE_ANON) &&
2768            anon >> sc->priority;
2769    }
2770
2771    shrink_node_memcgs(pgdat, sc);
2772
2773        ... <to be continued> ...
```

- 2741-2769 如果是全局回收则走这个代码块。
- 2746 计算目标 node 中所有 zone 的空闲页面数。
- 2747-2748 计算目标 node 中所有文件页内存页数。
- 2755 统计 node 中所有 zone 的高水位值之和。
- 2763 目标 node 的匿名页数。
- 2765-2768 如果该 node 的文件页数量很少，说明文件页的回收已经失控，需要做扫描平衡选择回收匿名页。

  这里要同时满足三个条件:

  - 2766 文件页 + 空闲页小于高水位线，说明文件页已经很少，即使将文件页全回收也不满足水位线要求。
  - 2767 扫描回收的禁止对活跃匿名页降级。

- 2768 系统中实际存在大量欠活跃匿名页。

这时候就会主动降低文件页的回收权重 (置 $file_is_tiny = true$)，不再扫描已经枯竭的文件页，而是尝试推动匿名页参与回收。

- 2771 进行页面回收。

do_try_to_free_pages() > *shrink_zones()* > *shrink_node()*

```
2773    if (reclaim_state) {
2774        sc->nr_reclaimed += reclaim_state->reclaimed_slab;
2775        reclaim_state->reclaimed_slab = 0;
2776    }
2777
2778    /* Record the subtree's reclaim efficiency */
2779    vmpressure(sc->gfp_mask, sc->target_mem_cgroup, true,
2780           sc->nr_scanned - nr_scanned,
2781           sc->nr_reclaimed - nr_reclaimed);
2782
2783    if (sc->nr_reclaimed - nr_reclaimed)
2784        reclaimable = true;
2785
2786    if (current_is_kswapd()) {
2787        /*
2788         * If reclaim is isolating dirty pages under writeback,
2789         * it implies that the long-lived page allocation rate
2790         * is exceeding the page laundering rate. Either the
2791         * global limits are not being effective at throttling
2792         * processes due to the page distribution throughout
2793         * zones or there is heavy usage of a slow backing
2794         * device. The only option is to throttle from reclaim
2795         * context which is not ideal as there is no guarantee
2796         * the dirtying process is throttled in the same way
2797         * balance_dirty_pages() manages.
2798         *
2799         * Once a node is flagged PGDAT_WRITEBACK, kswapd will
2800         * count the number of pages under pages flagged for
2801         * immediate reclaim and stall if any are encountered
2802         * in the nr_immediate check below.
2803         */
2804        if (sc->nr.writeback && sc->nr.writeback == sc->nr.taken)
2805            set_bit(PGDAT_WRITEBACK, &pgdat->flags);
2806
2807        /* Allow kswapd to start writing pages during reclaim.*/
2808        if (sc->nr.unqueued_dirty == sc->nr.file_taken)
2809            set_bit(PGDAT_DIRTY, &pgdat->flags);
2810
2811        /*
2812         * If kswapd scans pages marked for immediate
2813         * reclaim and under writeback (nr_immediate), it
2814         * implies that pages are cycling through the LRU
2815         * faster than they are written so also forcibly stall.
2816         */
2817        if (sc->nr.immediate)
2818            congestion_wait(BLK_RW_ASYNC, HZ/10);
2819    }
2820
2821        ... <to be continued> ...
```

- 2773 本次直接回收过程中，已经回收的 slab 页数累加进来。

- 2779-2781 计算内存压力并向用户空间发送压力通知。

  这里通过 $\frac{\text{已回收页面数}}{\text{已扫描页面数}}$ 来表示内存压力情况。

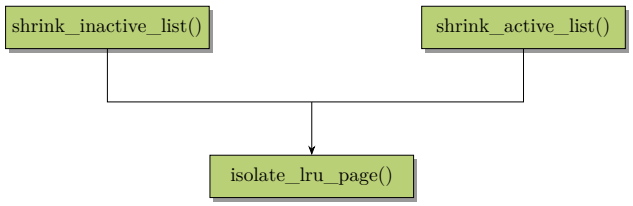
- 2786-2819 当前是在 kswapd 的上下文。

  - 2783-2784 如果本轮回收的页 >0, 那么置可回收标志。继续扫描该 zone。
  - 2804-2805 如果本次从 LRU(活跃页 + 欠活跃页) 隔离出来的所有页都在回写中，拿不到干净的文件页和可以丢弃的匿名页，全是正在 IO 的脏页，说明系统遇到回写瓶颈，则置本节点***PGDAT_WRITEBACK***标志。

    这个标志的作用是告知 kswapd，本节点回写压力大，要阻塞等待回写完成(stall)。如果不做 stall, kswapd 可能会不停扫描空转，但是回收确毫无进展。

    页面隔离发生在页面扫描的初始阶段，将符号条件的页面从全局 LRU 链表摘除，放入私有链表，确保原子性和并发安全。*sc->nr.taken* 统计的就是这一阶段成功隔离的页面总数。包括从 inactive 链表取出的要回收的页，和 active 链表要降级的页。核心操作由函数 *isolate_lru_page()* 执行。

    调用路径如下:



  - 2808-2809 本次隔离拿到的文件页都是尚未提交回写进入 IO 队列的脏页，没有干净页，则设置***PGDAT_DIRTY***标志。

    这个标志将允许 kswapd 在回收过程中主动发起脏页回写。

    PGDAT_WRITEBACK 和 PGDAT_DIRTY 对比的表格如下:

标识	意义
PGDAT_DIRTY	脏页，还没提交给回写；需要写盘。。
PGDAT_WRITEBACK	脏页已经提交给回写，正在磁盘 *IO* 回写排队中。

  - 2817-2818 如果 kswap 发现很多隔离链表上需要回收的页，都卡在回写操作中，那么说明磁盘写入太慢，IO 拥塞了。因此此处会等待异步磁盘 IO 操作，相当于回收限流。

    sc->nr.immediate 的值会在 kswapd 探测到目标隔离链表上的页面都要写回时进行递增 (见 shrink_page_list() 函数中递增的逻辑实现)。此处是对该值的判断使用。

do_try_to_free_pages() > *shrink_zones()* > *shrink_node()*

```
2821    /*
2822     * Tag a node/memcg as congested if all the dirty pages
2823     * scanned were backed by a congested BDI and
2824     * wait_iff_congested will stall.
```

```
2825         *
2826         * Legacy memcg will stall in page writeback so avoid forcibly
2827         * stalling in wait_iff_congested().
2828         */
2829        if ((current_is_kswapd() ||
2830             (cgroup_reclaim(sc) && writeback_throttling_sane(sc))) &&
2831            sc->nr.dirty && sc->nr.dirty == sc->nr.congested)
2832            set_bit(LRUVEC_CONGESTED, &target_lruvec->flags);
2833
2834        /*
2835         * Stall direct reclaim for IO completions if underlying BDIs
2836         * and node is congested. Allow kswapd to continue until it
2837         * starts encountering unqueued dirty pages or cycling through
2838         * the LRU too quickly.
2839         */
2840        if (!current_is_kswapd() && current_may_throttle() &&
2841            !sc->hibernation_mode &&
2842            test_bit(LRUVEC_CONGESTED, &target_lruvec->flags))
2843            wait_iff_congested(BLK_RW_ASYNC, HZ/10);
2844
2845        if (should_continue_reclaim(pgdat, sc->nr_reclaimed - nr_reclaimed,
2846                        sc))
2847            goto again;
2848
2849        /*
2850         * Kswapd gives up on balancing particular nodes after too
2851         * many failures to reclaim anything from them and goes to
2852         * sleep. On reclaim progress, reset the failure counter. A
2853         * successful direct reclaim run will revive a dormant kswapd.
2854         */
2855        if (reclaimable)
2856            pgdat->kswapd_failures = 0;
2857    }
```

- 2829-2832 满足下面的全部条件才置上 LRUVEC_CONGESTED 标记.
  - kswapd 线程 (间接回收)或者本次回收是 mem cgroup 维度的回收，不是全局 node 维度。
  - 扫描到了脏页，并且扫描出的脏页后端 BDI 全部处于拥塞状态, 也就是 IO 是瓶颈。
- 2840-2843 如果是直接回收，并且当前线程允许限流阻塞，扫描控制没有设成休眠等待，不过当前 lruvec 已经标记为拥塞。那么主动等待写回设备异步拥塞解除，最多等待 $\frac{HZ}{10}$。

  这是为了避免直接回收线程反复扫描 LRU，给后台 BDI 设备一点回写时间把脏页刷出去。
- 2845-2847 判断是否需继续回收。
- 2855-2856 如果 kswapd 在某些 numa 节点反复扫描始终回收不到内存，kswapd 会记录失败次数，次数超限的话，这个节点的 kswap 就会休眠。防止反复循环造成 cpu 的开销。只要该节点上某次直接回收成功回收一次，则重置 kswapd 失败计数器。来推迟 kswapd 进入休眠，或者是重新唤醒 kswapd 正常运行。

5.1.9 shrink_node_memcgs()

函数 shrink_node_memcgs() 在内存回收过程中，遍历指定 node 节点上的所有内存控制组 (memory cgroup)，并对每个控制组执行内存回收操作。

do_try_to_free_pages() > *shrink_zones()* > *shrink_node()* > *shrink_node_memcgs()*

```
2610 static void shrink_node_memcgs(pg_data_t *pgdat, struct scan_control *sc)
2611 {
2612     struct mem_cgroup *target_memcg = sc->target_mem_cgroup;
2613     struct mem_cgroup *memcg;
2614
2615     memcg = mem_cgroup_iter(target_memcg, NULL, NULL);
2616     do {
2617         struct lruvec *lruvec = mem_cgroup_lruvec(memcg, pgdat);
2618         unsigned long reclaimed;
2619         unsigned long scanned;
2620
2621         /*
2622          * This loop can become CPU-bound when target memcgs
2623          * aren't eligible for reclaim - either because they
2624          * don't have any reclaimable pages, or because their
2625          * memory is explicitly protected. Avoid soft lockups.
2626          */
2627         cond_resched();
2628
2629         mem_cgroup_calculate_protection(target_memcg, memcg);
2630
2631         if (mem_cgroup_below_min(memcg)) {
2632             /*
2633              * Hard protection.
2634              * If there is no reclaimable memory, OOM.
2635              */
2636             continue;
2637         } else if (mem_cgroup_below_low(memcg)) {
2638             /*
2639              * Soft protection.
2640              * Respect the protection only as long as
2641              * there is an unprotected supply
2642              * of reclaimable memory from other cgroups.
2643              */
2644             if (!sc->memcg_low_reclaim) {
2645                 sc->memcg_low_skipped = 1;
2646                 continue;
2647             }
2648             memcg_memory_event(memcg, MEMCG_LOW);
2649         }
2650
2651         reclaimed = sc->nr_reclaimed;
2652         scanned = sc->nr_scanned;
2653
2654         shrink_lruvec(lruvec, sc);
2655
2656         shrink_slab(sc->gfp_mask, pgdat->node_id, memcg,
2657                 sc->priority);
2658
2659         /* Record the group's reclaim efficiency */
2660         vmpressure(sc->gfp_mask, memcg, false,
2661                sc->nr_scanned - scanned,
2662                sc->nr_reclaimed - reclaimed);
2663
2664     } while ((memcg = mem_cgroup_iter(target_memcg, memcg, NULL)));
2665 }
```

- 2612 要回收的根目标 memcg。
- 2615 从根 memcg 开始，迭代遍历控制组树。
- 2617 获取当前控制组在指定内存节点上的 lru 向量，用于后续页面回收。
- 2627 条件性地让出 CPU，防止在回收过程中长时间占用 CPU 导致软锁。

- 2629 调用 mem_cgroup_calculate_protection() 实时计算当前控制组的有效保护值 (emin 和 elow)。动态调整保护值，确保层级资源管理的一致性。
- 2631-2637 检查当前控制组的内存使用量是否低于其 emin。若低于 emin (硬保护阈值) 意味着没有可回收内存，可能会触发 OOM 。
- 2637-2649 检查当前控制组的内存使用量是否低于 elow (软保护阈值)。
  - 2644-2646 判断是否允许回收受软保护的内存。若不允许，则标记本轮回收略过了受软保护的 memcg 内存，继续处理下一个。
  - 2648 目标 memcg 的内存使用量 $\leq elow$, 向用户空间上报该事件。

  memcg->memory.elow 和 memcg->memory.emin 的区别如下:

名称	来源	保护等级	*vmscan* 行为
memory.elow	用户配置的 *memory.low* 按层级折算得到	软保护	尽量少回收，内存极端不足时可以回收。
memory.emin	用户配置的 *memory.min* 按层级折算得到	硬保护	*usage <= emin* 时完全避免回收

- 2651-2652 记录 (回收前的) 已回收和已扫描页面的计数，用于后续计算。因为本次回收后这两个值会变化，所以此处临时保存变化前的值。
- 2654 回收当前控制组的 LRU 页面。
- 2656 回收控制组的 slab 缓存。
- 2660-2662 vmpressure() 记录当前控制组的回收压力和效率。

  这里第 3 个参数传递的是 false, 所以目标控制组内存压力状况不上报到用户空间，仅内核空间记录。
- 2664 循环继续遍历下一个控制组，直到所有控制组处理完毕。

do_try_to_free_pages() › … › … › *shrink_node_memcgs()* › *mem_cgroup_calculate_protection()*

```
6705 void mem_cgroup_calculate_protection(struct mem_cgroup *root,
6706                       struct mem_cgroup *memcg)
6707 {
6708     unsigned long usage, parent_usage;
6709     struct mem_cgroup *parent;
6710
6711     if (mem_cgroup_disabled())
6712         return;
6713
6714     if (!root)
6715         root = root_mem_cgroup;
6716
6717     /*
6718      * Effective values of the reclaim targets are ignored so they
6719      * can be stale. Have a look at mem_cgroup_protection for more
6720      * details.
6721      * TODO: calculation should be more robust so that we do not need
6722      * that special casing.
```

```
6723          */
6724         if (memcg == root)
6725             return;
6726 
6727         usage = page_counter_read(&memcg->memory);
6728         if (!usage)
6729             return;
6730 
6731         parent = parent_mem_cgroup(memcg);
6732         /* No parent means a non-hierarchical mode on v1 memcg */
6733         if (!parent)
6734             return;
6735 
6736         if (parent == root) {
6737             memcg->memory.emin = READ_ONCE(memcg->memory.min);
6738             memcg->memory.elow = READ_ONCE(memcg->memory.low);
6739             return;
6740         }
6741 
6742         parent_usage = page_counter_read(&parent->memory);
6743 
6744         WRITE_ONCE(memcg->memory.emin, effective_protection(usage, parent_usage,
6745                 READ_ONCE(memcg->memory.min),
6746                 READ_ONCE(parent->memory.emin),
6747                 atomic_long_read(&parent->memory.children_min_usage)));
6748 
6749         WRITE_ONCE(memcg->memory.elow, effective_protection(usage, parent_usage,
6750                 READ_ONCE(memcg->memory.low),
6751                 READ_ONCE(parent->memory.elow),
6752                 atomic_long_read(&parent->memory.children_low_usage)));
6753 }
```

该函数检测内存消耗是否在正常范围。其正确执行依赖于树结构的遍历上下文 (如父节点状态、遍历顺序)。使用时必须遵循自上而下的迭代流程，不能单独对某个节点进行孤立查询，否则会因状态缺失导致结果错误。

- 6711-6712 检查是否禁用了内存控制组功能。
- 6714-6715 若未指定 root 组，使用系统默认的内存控制 root 组 (root_mem_cgroup)。
- 6724-6725 当前组为 root 组时无需计算（root 组无父级保护)。
- 6727-6729 读取当前目标内存控制组内存的各种使用量。
- 6731-6734 获取父级内存控制组 memcg。如果无父级 memcg，表示非层级模式，无法继承保护量，直接返回。
- 6736-6740 如果父级组即是 root 组，直接将当前控制组的 min 和 low 值赋给 emin 和 elow。根 mem cgroup 节点的保护量为全局配置，其子级节点无需调整。
- 6742 读取父级组的各内存使用量。
- 6744-6752 调用 effective_protection() 函数计算当前控制组的 emin 和 elow。见下面代码。

effective_protection() 函数计算单个控制组的有效保护量，该值源自其自身的 memory.min/low 设置、父控制组和兄弟控制组的配置，以及整个控制组树中的实际内存使用情况。

do_try_to_free_pages() ··· ··· *mem_cgroup_calculate_protection()* *effective_protection()*

```
6624 static unsigned long effective_protection(unsigned long usage,
6625                       unsigned long parent_usage,
6626                       unsigned long setting,
6627                       unsigned long parent_effective,
6628                       unsigned long siblings_protected)
6629 {
6630     unsigned long protected;
6631     unsigned long ep;
6632 
6633     protected = min(usage, setting);
6634     /*
6635      * If all cgroups at this level combined claim and use more
6636      * protection then what the parent affords them, distribute
6637      * shares in proportion to utilization.
6638      *
6639      * We are using actual utilization rather than the statically
6640      * claimed protection in order to be work-conserving: claimed
6641      * but unused protection is available to siblings that would
6642      * otherwise get a smaller chunk than what they claimed.
6643      */
6644     if (siblings_protected > parent_effective)
6645         return protected * parent_effective / siblings_protected;
6646 
6647     /*
6648      * Ok, utilized protection of all children is within what the
6649      * parent affords them, so we know whatever this child claims
6650      * and utilizes is effectively protected.
6651      *
6652      * If there is unprotected usage beyond this value, reclaim
6653      * will apply pressure in proportion to that amount.
6654      *
6655      * If there is unutilized protection, the cgroup will be fully
6656      * shielded from reclaim, but we do return a smaller value for
6657      * protection than what the group could enjoy in theory. This
6658      * is okay. With the overcommit distribution above, effective
6659      * protection is always dependent on how memory is actually
6660      * consumed among the siblings anyway.
6661      */
6662     ep = protected;
6663 
6664     /*
6665      * If the children aren't claiming (all of) the protection
6666      * afforded to them by the parent, distribute the remainder in
6667      * proportion to the (unprotected) memory of each cgroup. That
6668      * way, cgroups that aren't explicitly prioritized wrt each
6669      * other compete freely over the allowance, but they are
6670      * collectively protected from neighboring trees.
6671      *
6672      * We're using unprotected memory for the weight so that if
6673      * some cgroups DO claim explicit protection, we don't protect
6674      * the same bytes twice.
6675      *
6676      * Check both usage and parent_usage against the respective
6677      * protected values. One should imply the other, but they
6678      * aren't read atomically - make sure the division is sane.
6679      */
6680     if (!(cgrp_dfl_root.flags & CGRP_ROOT_MEMORY_RECURSIVE_PROT))
6681         return ep;
6682     if (parent_effective > siblings_protected &&
6683         parent_usage > siblings_protected &&
6684         usage > protected) {
6685         unsigned long unclaimed;
6686 
6687         unclaimed = parent_effective - siblings_protected;
```

```
6688            unclaimed *= usage - protected;
6689            unclaimed /= parent_usage - siblings_protected;
6690
6691            ep += unclaimed;
6692        }
6693
6694        return ep;
6695    }
```

- 6633 计算当前 cgroup 的实际保护需求，控制组的初始保护量 (protected) 取实际使用量和配置保护阈值中的较小值。这避免了” 保护未使用的内存”, 确保资源利用率。又确保保护不超出用户预期。后续代码会结合父控制组的实际保护量 (parent_effective) 和子级控制组中内存的实际保护量 (siblings_protected), 进一步调整得到最终的实际保护量 (ep)。
- 6644-6645 当子级控制组实际被保护的页总量 siblings_protected, 超过父级控制组实际保护量 parent_effective 时触发。也就是说，父级实际保护量不足以满足子级控制组实际被保护总量时，依照各子级 cgroup 占子级控制组实际保护总量的比例，把父级实际保护量按比例分配成各子级 cgroup 的新的实际保护量。

  譬如，父级 parent_effective=100, 子级兄弟组 siblings_protected=120, 当前子级 mem cgroup 实际保护量 protected=30. 最终当前子级 cgroup 实际保护量为:$100 * 30/120 = 25$。
- 6662 如果子级控制组 cgroup 实际保护量需求不超过父级实际保护量，那么意味着父级资源充足，当前子级 cgroup 保证可以获得需求的实际保护量。
- 6680 检查是否允许把父级额外的保护量按比例分配给子级。若允许，则后续进入” 浮动保护量” 分配。
- 6682-6684 若满足以下条件则进行” 浮动保护量” 分配:
  - 父级实际保护量比子级控制组实际使用的保护量大 ($parent_effective > siblings_protected$)
  - 父级实际使用量大于子级控制组总保护量 ($parent_usage > siblings_protected$)
  - 当前子 cgroup 实际使用量超过其初始保护量 ($usage > protected$).
- 6687-6689
  - $parent_effective - siblings_protected$ 父级闲置保护量
  - $usage - protected$ 当前子 group 组需求的超额保护量
  - $parent_usage - siblings_protected$ 父级实际使用的超出子 group 组需求的超额量

  浮动保护值的数学表达式如下:

  $$floating\ proctection = (parent_effective - siblings_proctected) * \frac{(usage - protected)}{(parent_usage - siblings_protected)}$$
- 6691 把浮动保护量追加到实际保护量。

5.1.10 shrink_lruvec()

再回到 shrink_node_memcgs() 函数 2654 行的调用:

do_try_to_free_pages() > *shrink_zones()* > *shrink_node()* > *shrink_node_memcgs()* > *shrink_lruvec()*

```
2425 static void shrink_lruvec(struct lruvec *lruvec, struct scan_control *sc)
2426 {
2427     unsigned long nr[NR_LRU_LISTS];
2428     unsigned long targets[NR_LRU_LISTS];
2429     unsigned long nr_to_scan;
2430     enum lru_list lru;
2431     unsigned long nr_reclaimed = 0;
2432     unsigned long nr_to_reclaim = sc->nr_to_reclaim;
2433     struct blk_plug plug;
2434     bool scan_adjusted;
2435
2436     get_scan_count(lruvec, sc, nr);
2437
2438     /* Record the original scan target for proportional adjustments later */
2439     memcpy(targets, nr, sizeof(nr));
2440
2441     /*
2442      * Global reclaiming within direct reclaim at DEF_PRIORITY is a normal
2443      * event that can occur when there is little memory pressure e.g.
2444      * multiple streaming readers/writers. Hence, we do not abort scanning
2445      * when the requested number of pages are reclaimed when scanning at
2446      * DEF_PRIORITY on the assumption that the fact we are direct
2447      * reclaiming implies that kswapd is not keeping up and it is best to
2448      * do a batch of work at once. For memcg reclaim one check is made to
2449      * abort proportional reclaim if either the file or anon lru has already
2450      * dropped to zero at the first pass.
2451      */
2452     scan_adjusted = (!cgroup_reclaim(sc) && !current_is_kswapd() &&
2453              sc->priority == DEF_PRIORITY);
2454
2455     blk_start_plug(&plug);
2456     while (nr[LRU_INACTIVE_ANON] || nr[LRU_ACTIVE_FILE] ||
2457                     nr[LRU_INACTIVE_FILE]) {
2458         unsigned long nr_anon, nr_file, percentage;
2459         unsigned long nr_scanned;
2460
2461         for_each_evictable_lru(lru) {
2462             if (nr[lru]) {
2463                 nr_to_scan = min(nr[lru], SWAP_CLUSTER_MAX);
2464                 nr[lru] -= nr_to_scan;
2465
2466                 nr_reclaimed += shrink_list(lru, nr_to_scan,
2467                                 lruvec, sc);
2468             }
2469         }
2470
2471         cond_resched();
2472
2473         if (nr_reclaimed < nr_to_reclaim || scan_adjusted)
2474             continue;
2475
2476         /*
2477          * For kswapd and memcg, reclaim at least the number of pages
2478          * requested. Ensure that the anon and file LRUs are scanned
2479          * proportionally what was requested by get_scan_count(). We
2480          * stop reclaiming one LRU and reduce the amount scanning
2481          * proportional to the original scan target.
2482          */
2483         nr_file = nr[LRU_INACTIVE_FILE] + nr[LRU_ACTIVE_FILE];
2484         nr_anon = nr[LRU_INACTIVE_ANON] + nr[LRU_ACTIVE_ANON];
```

```
2485
2486            /*
2487             * It's just vindictive to attack the larger once the smaller
2488             * has gone to zero.  And given the way we stop scanning the
2489             * smaller below, this makes sure that we only make one nudge
2490             * towards proportionality once we've got nr_to_reclaim.
2491             */
2492            if (!nr_file || !nr_anon)
2493                break;
2494
2495            if (nr_file > nr_anon) {
2496                unsigned long scan_target = targets[LRU_INACTIVE_ANON] +
2497                            targets[LRU_ACTIVE_ANON] + 1;
2498                lru = LRU_BASE;
2499                percentage = nr_anon * 100 / scan_target;
2500            } else {
2501                unsigned long scan_target = targets[LRU_INACTIVE_FILE] +
2502                            targets[LRU_ACTIVE_FILE] + 1;
2503                lru = LRU_FILE;
2504                percentage = nr_file * 100 / scan_target;
2505            }
2506
2507            /* Stop scanning the smaller of the LRU */
2508            nr[lru] = 0;
2509            nr[lru + LRU_ACTIVE] = 0;
2510
2511            /*
2512             * Recalculate the other LRU scan count based on its original
2513             * scan target and the percentage scanning already complete
2514             */
2515            lru = (lru == LRU_FILE) ? LRU_BASE : LRU_FILE;
2516            nr_scanned = targets[lru] - nr[lru];
2517            nr[lru] = targets[lru] * (100 - percentage) / 100;
2518            nr[lru] -= min(nr[lru], nr_scanned);
2519
2520            lru += LRU_ACTIVE;
2521            nr_scanned = targets[lru] - nr[lru];
2522            nr[lru] = targets[lru] * (100 - percentage) / 100;
2523            nr[lru] -= min(nr[lru], nr_scanned);
2524
2525            scan_adjusted = true;
2526        }
2527        blk_finish_plug(&plug);
2528        sc->nr_reclaimed += nr_reclaimed;
2529
2530        /*
2531         * Even if we did not try to evict anon pages at all, we want to
2532         * rebalance the anon lru active/inactive ratio.
2533         */
2534        if (total_swap_pages && inactive_is_low(lruvec, LRU_INACTIVE_ANON))
2535            shrink_active_list(SWAP_CLUSTER_MAX, lruvec,
2536                       sc, LRU_ACTIVE_ANON);
2537    }
```

对 lruvec 中的 lru 链表进行内存回收，此 lruvec 有可能属于一个 memcg，也可能是属于一个内存 node 节点。

- 2436 针对目标 lruvec 中的每个 lru 链表, 计算它们本次 scan 应该扫描的页框数量。把计算好的各个 lru 链表需要扫描的页框数量保存在 nr 数组中。
- 2452-2453 判断是否满足下列条件：

  (1) 不是在 kswapd 内核线程中。

(2) 不是快速回收的执行路径 (快速回收的优先级不是 DEF_PRIORITY)。

直接内存回收以 DEF_PRIORITY默认优先级进行的全局回收，在以默认优先级扫描时，此时系统尚未面临严重的内存压力。但既然当前进入了直接回收，就说明 kswapd 后台回收不足以应对当前这种轻微的内存压力，跟不上需求。此时即使已回收了所需的请求数量的页面，内核也不会中止扫描，最好一次性完成一批回收工作。

此处会判断当前是否处于直接内存回收且处于默认回收优先级，若是，则会调整扫描策略。在这种策略下，内核会忽略 struct scan_control->nr_to_reclaim 设定的回收页目标数量的限制，进行循环扫描，直到遍历完由 nr 指定的所有页面。

默认优先级 DEF_PRIORITY（硬编码为 12），会使得 LRU 链表中仅有 1/4096 的页面被扫描。

此处还会考虑是否是 mem cgroup 的回收，如果是，则不进行扫描策略的调整，只优先考虑回收目标数量的页。因为 mem cgroup 通常会配置内存限额或上限。

- 2455 blk_start_plug() 给当前进程” 插上塞子”，把接下来要写磁盘、读磁盘的 IO 暂存起来，攒成一批，最后再调用 blk_finish_plug()(2527 行) 打开” 塞子” 一次性发给磁盘驱动。这么做的目的是为了减少小 IO 次数，大幅提升磁盘效率，避免卡顿。
- 2456-2526 在非活动匿名页、活动文件页和非活动文件页 3 个链表之一仍有待扫描页时持续循环。注意，nr[x] 给出的是各个链表上的带扫描页的数量。
- 2461 遍历每个 LRU 链表（包含活动匿名页）。
- 2462-2463 只要目标 lru 链表还有页待扫描回收，那么每次最多回收 32(SWAP_CLUSTER_MAX) 个页面。这里获取剩下的待扫描的页数量 nr[lru] 与 32 的最小值来作为本次回收页的计划数量。
- 2464 因为可能很多页面被引用、锁定导致无法回收，所以 shrink_list() 可能无法回收目标数目的页，但是即使未能回收目标数量，还是会在链表的每次遍历中减去每次计划的数量。
- 2466 shrink_list() 函数对目标 lru 链表进行回收，计划回收 nr_to_scan 个页面，但实际回收数目不一定能达到该值。本次实际回收的页框数目会累加到 nr_reclaimed 中。
- 2473-2474 如果已经扫描到足够的空闲页，并且无需全部扫描 nr 中的页面，则停止扫描。没有回收到足够页框，或者需要忽略需要回收的页框数量尽可能多的回收，也就是宽松扫描模式，则继续进行回收。
- 2483-2523 这段代码维持由 get_scan_count() 施加的各 LRU 向量之间的扫描比例。由于每次最多批量处理 SWAP_CLUSTER_MAX 个页面，那么在达到或超过回收目标的同时，可能没有维持由 swappiness 所设定的文件页与匿名页之间的比例。
- 2483-2494 获取文件页和匿名页中哪一种还剩更多待扫描页面，这一数值分别存储在 nr_file 和 nr_anon 中。如果其中一个的剩余页数值为零就中止扫描。因为继续扫描可能在后续处理中会将其他链表的扫描目标数也降至零，从而导致不必要的回收停顿。
- 2495-2505 判断文件页和匿名页中哪一种在本次迭代回收中还剩更多未扫描页面。
- 2496-2499 如果文件页剩更多未扫描页面，也就是说扫描次数少。那么计算匿名页剩余未扫描页面数量占匿名页两个 LRU 链表上待扫描页的总数的百分比。

- 2500-2505 见 2496-2499 的解释。
- 2508-2509 如果哪一类剩余待扫描页面相对更多，也就意味着被扫描的页更少。那么我们会清空另一类（被扫描页数相对更多的链表）的剩余扫描目标数量，下次只从剩余待扫描页数更多的链表中进行回收。
- 2515-2518
  - 2516 获取剩余待扫描页较多那类页的已扫描的页数目。
  - 2517 对于剩余待扫描页较多那类页，先计算剩余待扫描页较少的页的剩余未扫描页的百分比，然后计算补集，也就是得到扫描比例。这样做的目的是利用这个比例，确保扫描次数较少（剩余待扫描页较多）的链表的页面扫描比例的目标是这个（补集）。再乘以链表上原先页的总数，得到按比例应扫描的页数量。

    换言之，A 类型页剩余待扫描页较多，意味着 A 页的扫描比例比较低。相应的 B 类型的扫描比例高，那么计算 B 的未扫描的比例，(1 - B 的未扫描比例)= B 的已扫描比例。A 就以该比例 (B 的已扫描比例) 为目标获得应该扫描的 A 的页数。
  - 2518 按比例应该扫描的页减去已扫描的页（取已扫描页和应扫描页中的最小值）后剩下应扫描页的数量。

假设 scan_control->nr_to_reclaim = 128, 也就是计划回收 128 页。并且 get_scan_count 将匿名页的总扫描目标设置为 128 页，active 和 inactive 链表各占 64。文件页的总扫描目标设置为 256 页，同样 active 和 inactive 链表各占 128 页。(为啥扫描页大于计划回收页) 如下：

LRU	计划扫描页数	计划扫描页占比
active 匿名页	*64*	*10%*
inactive 匿名页	*64*	*10%*
active 文件页	*256*	*40%*
inactive 文件页	*256*	*40%*

这里页面回收比例是 80%/20%, 其中文件页占比重大。在对每个 LRU 链表遍历一遍后，假设所有页面都被回收，那么就能达到回收->nr_to_reclaim 个页的目标了。

然而，这完全没有按比例扫描。实际上只是对所有链表进行了平均比例的扫描，如下：

LRU	剩余待扫描页数	已扫描页数	已扫描页占比	已扫描页/计划扫描
active 匿名页	*32*	*32*	*25%*	*50%*
inactive 匿名页	*32*	*32*	*25%*	*50%*
active 文件页	*224*	*32*	*25%*	*12.5%*
inactive 文件页	*224*	*32*	*25%*	*12.5%*

上表中举例的数据说明虽然达到了回收目标，但实际扫描的比例，与 get_scan_count() 所指定的比例 80%/20%(从再前一个表得到,$(256+256)/(64+64)$) 完全不相符。

此外，可以看到，从匿名页的角度来看，我们几乎没有调整的余地—因为再多一批 SWAP_CLUSTER_MAX(32) 个页面，就会耗尽匿名 LRU 链表，这显然是不合理的，因为我们的本意是要更多地扫描文件页。因此，为了调整比例继续扫描时，只扫描那些迭代过程中扫描次数较少的页面类型（也就是还有大量页面待扫描的页面类型）就很合理性了。

可以看到目前还有 448 个可扫描的文件页，而可扫描匿名页只有 64 个。因此，通过增加对文件页的扫描，就能更好地纠正这种失衡状况。在这个例子中，我们可以看到匿名页已扫描页的百分比，等于 32 除以目标值 64，也就是 50%。它的补集也是 50%=(1-50%)。又因为文件页目标值是 256，那么对于 active 和 inactive 文件链表而言，它们的 50% 都是 128 页。减去已经扫描过的 32 页后，每个文件链表还需要扫描 96 页。总共 192 页 (active 文件页 +inactive 文件页)，这就是仅针对文件 LRU 链表调整的宽松扫描模式。

假设我们按此比例成功完成扫描，最终的汇总结果将如下表所示。

LRU	已扫描页数	已扫描页占比	已扫描页/计划扫描
active 匿名页	*32*	*10%*	*50%*
inactive 匿名页	*32*	*10%*	*50%*
active 文件页	*128*	*40%*	*50%*
inactive 文件页	*128*	*40%*	*50%*

可以在表中看到每个 LRU 向量的最终扫描页数：从匿名页链表扫描的原始 32 页，以及文件页 128 页（在原始 32 页基础上又增加了 96 页）。我们总共扫描（并可能回收）了 320 页，这是回收目标页数 (目标回收页数是->nr_to_reclaim=128) 的 2.5 倍—但我们维持了所需的比例 (80% 文件页：20% 匿名页，也就是原始计划扫描页的比例：64：256)。

这也是上面反复说明的：A 类型页剩余未扫描页少，就以A 的已扫描页/A 页回收目标数=(1-(A 的剩余未扫描页/A 页回收目标数)) 作为 B 回收页的百分比. 这样相当于 A 类型和 B 类型页等比例缩放，这样就保证了 A:B 分别在回收的页和计划回收的页上能保持同样的比例。

同时我们达到了 get_scan_count() 计划目标的 50%(匿名页和文件页计划每个链表分别是 64 和 256. 实际回收 32 和 128)，但在间接回收或低于默认优先级的直接回收场景下，我们并不打算达到这些目标，而是回收达到或超过请求回收的页数即可。

- 2520-2523 同 2515-2518.
- 2524 调整扫描模式，确保该逻辑只执行一次。
- 2527 见 2455 行的解释。
- 2534-2536 如果可以 swap，并且 inactive LRU 链表上匿名页数量过少，那么就通过 shrink_active_list() 函数对 active 匿名页降级来平衡链表。因为在” 修剪” 模式下会标记 active 匿名页不应被回收，此时需要通过降级 active 链表上的页来补充 inactive 链表，确保有足够可回收页。

5.1.10.1 get_scan_count() 以及工作页集

shrink_lruvec() 函数 2436 行调用 get_scan_count() 先按比例算出匿名页和文件页分别要扫描的目标页数目，再把 sc->priority 当除数或比例因子，算出调整后的实际扫描数, 也就是再按比例缩小。

如果按 sc->priority 算出的页数目比实际所需的页少，那么会把 priority-1, 再扫描，直到扫描到所需的页数目或 priority 为 0。之所以这么实现的目的是为了快。不够时再慢慢加大力度。避免一开始就全量扫描导致系统卡顿。这就是渐进式扫描的目的。

do_try_to_free_pages() ··· ··· shrink_node_memcgs() shrink_lruvec() get_scan_count()

```
2237 static void get_scan_count(struct lruvec *lruvec, struct scan_control *sc,
2238                unsigned long *nr)
2239 {
2240     struct mem_cgroup *memcg = lruvec_memcg(lruvec);
2241     unsigned long anon_cost, file_cost, total_cost;
2242     int swappiness = mem_cgroup_swappiness(memcg);
2243     u64 fraction[ANON_AND_FILE];
2244     u64 denominator = 0;    /* gcc */
2245     enum scan_balance scan_balance;
2246     unsigned long ap, fp;
2247     enum lru_list lru;
2248 
2249     /* If we have no swap space, do not bother scanning anon pages. */
2250     if (!sc->may_swap || mem_cgroup_get_nr_swap_pages(memcg) <= 0) {
2251         scan_balance = SCAN_FILE;
2252         goto out;
2253     }
2254 
2255     /*
2256      * Global reclaim will swap to prevent OOM even with no
2257      * swappiness, but memcg users want to use this knob to
2258      * disable swapping for individual groups completely when
2259      * using the memory controller's swap limit feature would be
2260      * too expensive.
2261      */
2262     if (cgroup_reclaim(sc) && !swappiness) {
2263         scan_balance = SCAN_FILE;
2264         goto out;
2265     }
2266 
2267     /*
2268      * Do not apply any pressure balancing cleverness when the
2269      * system is close to OOM, scan both anon and file equally
2270      * (unless the swappiness setting disagrees with swapping).
2271      */
2272     if (!sc->priority && swappiness) {
2273         scan_balance = SCAN_EQUAL;
2274         goto out;
2275     }
2276 
2277     /*
2278      * If the system is almost out of file pages, force-scan anon.
2279      */
2280     if (sc->file_is_tiny) {
2281         scan_balance = SCAN_ANON;
2282         goto out;
2283     }
2284 
2285     /*
2286      * If there is enough inactive page cache, we do not reclaim
2287      * anything from the anonymous working right now.
2288      */
2289     if (sc->cache_trim_mode) {
2290         scan_balance = SCAN_FILE;
2291         goto out;
2292     }
2293 
2294     scan_balance = SCAN_FRACT;
2295     /*
2296      * Calculate the pressure balance between anon and file pages.
2297      *
2298      * The amount of pressure we put on each LRU is inversely
2299      * proportional to the cost of reclaiming each list, as
2300      * determined by the share of pages that are refaulting, times
```

```
2301      * the relative IO cost of bringing back a swapped out
2302      * anonymous page vs reloading a filesystem page (swappiness).
2303      *
2304      * Although we limit that influence to ensure no list gets
2305      * left behind completely: at least a third of the pressure is
2306      * applied, before swappiness.
2307      *
2308      * With swappiness at 100, anon and file have equal IO cost.
2309      */
2310     total_cost = sc->anon_cost + sc->file_cost;
2311     anon_cost = total_cost + sc->anon_cost;
2312     file_cost = total_cost + sc->file_cost;
2313     total_cost = anon_cost + file_cost;
2314
2315     ap = swappiness * (total_cost + 1);
2316     ap /= anon_cost + 1;
2317
2318     fp = (200 - swappiness) * (total_cost + 1);
2319     fp /= file_cost + 1;
2320
2321     fraction[0] = ap;
2322     fraction[1] = fp;
2323     denominator = ap + fp;
2324 out:
2325    for_each_evictable_lru(lru) {
2326         int file = is_file_lru(lru);
2327         unsigned long lruvec_size;
2328         unsigned long scan;
2329         unsigned long protection;
2330
2331         lruvec_size = lruvec_lru_size(lruvec, lru, sc->reclaim_idx);
2332         protection = mem_cgroup_protection(sc->target_mem_cgroup,
2333                            memcg,
2334                            sc->memcg_low_reclaim);
2335
2336         if (protection) {
2337             /*
2338              * Scale a cgroup's reclaim pressure by proportioning
2339              * its current usage to its memory.low or memory.min
2340              * setting.
2341              *
2342              * This is important, as otherwise scanning aggression
2343              * becomes extremely binary -- from nothing as we
2344              * approach the memory protection threshold, to totally
2345              * nominal as we exceed it.  This results in requiring
2346              * setting extremely liberal protection thresholds. It
2347              * also means we simply get no protection at all if we
2348              * set it too low, which is not ideal.
2349              *
2350              * If there is any protection in place, we reduce scan
2351              * pressure by how much of the total memory used is
2352              * within protection thresholds.
2353              *
2354              * There is one special case: in the first reclaim pass,
2355              * we skip over all groups that are within their low
2356              * protection. If that fails to reclaim enough pages to
2357              * satisfy the reclaim goal, we come back and override
2358              * the best-effort low protection. However, we still
2359              * ideally want to honor how well-behaved groups are in
2360              * that case instead of simply punishing them all
2361              * equally. As such, we reclaim them based on how much
2362              * memory they are using, reducing the scan pressure
2363              * again by how much of the total memory used is under
2364              * hard protection.
```

```
2365                 */
2366                unsigned long cgroup_size = mem_cgroup_size(memcg);
2367
2368                /* Avoid TOCTOU with earlier protection check */
2369                cgroup_size = max(cgroup_size, protection);
2370
2371                scan = lruvec_size - lruvec_size * protection /
2372                    cgroup_size;
2373
2374                /*
2375                 * Minimally target SWAP_CLUSTER_MAX pages to keep
2376                 * reclaim moving forwards, avoiding decrementing
2377                 * sc->priority further than desirable.
2378                 */
2379                scan = max(scan, SWAP_CLUSTER_MAX);
2380        } else {
2381                scan = lruvec_size;
2382        }
2383
2384        scan >>= sc->priority;
2385
2386        /*
2387         * If the cgroup's already been deleted, make sure to
2388         * scrape out the remaining cache.
2389         */
2390        if (!scan && !mem_cgroup_online(memcg))
2391                scan = min(lruvec_size, SWAP_CLUSTER_MAX);
2392
2393        switch (scan_balance) {
2394        case SCAN_EQUAL:
2395                /* Scan lists relative to size */
2396                break;
2397        case SCAN_FRACT:
2398                /*
2399                 * Scan types proportional to swappiness and
2400                 * their relative recent reclaim efficiency.
2401                 * Make sure we don't miss the last page on
2402                 * the offlined memory cgroups because of a
2403                 * round-off error.
2404                 */
2405                scan = mem_cgroup_online(memcg) ?
2406                       div64_u64(scan * fraction[file], denominator) :
2407                       DIV64_U64_ROUND_UP(scan * fraction[file],
2408                                          denominator);
2409                break;
2410        case SCAN_FILE:
2411        case SCAN_ANON:
2412                /* Scan one type exclusively */
2413                if ((scan_balance == SCAN_FILE) != file)
2414                        scan = 0;
2415                break;
2416        default:
2417                /* Look ma, no brain */
2418                BUG();
2419        }
2420
2421        nr[lru] = scan;
2422    }
2423 }
```

- 2240 获取目标 lruvec 所属的 memcg。
- 2242 获取目标 memcg 的 swap 倾向参数 (swapiness)。

该值控制在内存紧张时，优先回收匿名页（swapout）还是文件页（pageout）。可以通过节点***/proc/sys/vm/swapiness*** 来设置。值范围 $\in [0, 200]$,60 是默认值，当为 0 时有限回收文件页，200 时有限回收匿名页。

- 2250-2253 如果是以下两种场景之一，那么优先考虑只扫描（回收)文件页。
  - 系统当前禁止使用 swap 空间（如 swap 不可用或已满）。
  - 当前控制组 memcg 未使用任何 swap 空间。

  当 swap 不可用或 swap 空间不足时，只能通过两种方式处理匿名页：
  - 直接丢弃：但匿名页数据无磁盘备份，丢弃后进程再次访问会触发缺页异常 (Page Fault)，而内核无法从磁盘恢复数据，只能导致进程崩溃（触发 SIGSEGV 信号）。
  - 通过 OOM(Out of Memory) 机制杀死进程：这是内核最后的手段，通过 oom_killer 选择占用内存最多的进程强制终止，释放其匿名页。

  因此，当 swap 不可用或 swap 空间不足时，尽可能只回收文件页。
- 2262-2265 针对控制组级别的回收（而非全局回收），在明确禁用 Swap (swappiness==0) 时，只回收文件页。

  对于全局回收，即使 $swappiness = 0$，内核仍会在极端情况下（如物理内存耗尽）使用 swap 避免系统崩溃。

  对于 memcg 回收，若 $swappiness = 0$，则完全禁用该控制组的 swap，优先回收文件页，避免匿名页被换出。

  memcg 回收时希望通过 $swappiness = 0$ 完全禁用特定控制组的 swap，因为使用 memory.swap.max 需要实时跟踪 swap 使用量限制，成本过高。而直接禁用 Swap（通过 $swappiness = 0$）可简化逻辑，降低开销。
- 2272-2274 这段代码块描述了在系统接近内存耗尽 (OOM) 时的紧急内存回收策略。

  sc->priority 是” 对内存回收控制的力度 ”。控制力度越大，回收的优先级越低；反之，控制力度越小，回收优先级越高。

  当->priority 值为 0 时，表示控制力度最小，回收优先级最高，意味着此时系统已进入紧急回收状态 (接近 OOM)。当系统内存极度紧张且允许 swap out 时，会放弃复杂精巧的压力平衡逻辑，用同等扫描力度同时从匿名页和文件页中回收内存，提高总体回收速度。
- 2280-2283 系统中文件页消耗殆尽，强制扫描匿名页。*->file_is_tiny* 变量值在 shrink_node() 函数中赋值。
- 2289-2292 ” 修剪” 模式表示系统中文件页充足，此时不从匿名工作页集合中回收或扫描，而优先从文件页回收。换言之，” 修剪” 模式意味着只从文件页回收。
- 2294 排除前面各种特定场景，默认使用普通的比例模式进行扫描回收。
- 2310-2313 这几行统计”refaulting” 的页。”fault” 表示缺页，”refaulting” 字面意思是” 再次缺页”，表示 swap out 出去的匿名页或 page out 出去的文件页再次加载到内存。但如果是工作集页通过再次缺页加载到内存的话，那么说明之前回收该页是个错误，造成了一定的开销，称为回收损耗。*->anon_cost* 和*->file_cost* 就分别表示目标 memcg 上误回收匿名和文件工作集页各自造成的损耗。

在任意时刻，所有进程正在使用的内存总和被称为工作集。它是正常运行所需的内存范围，即进程当前正在使用的页帧总和。从内存回收的角度来看——要防止工作集内的页帧发生“抖动”(thrashing)。”抖动”指的是：反复回收和使用某些页帧，即刚被回收不久就再次发生缺页，必须从外存储系统中重新读取这些页 (也称为“重缺页”，refaulting)。

当一个页帧第一次被访问时，它被加入到欠活跃链表头，然后在系统页面处理过程中会逐渐往欠活跃链表尾部移动。最终到达链表尾时会被踢出链表，并且释放页面，这个踢出的动作称为 eviction(回收)；如果一个欠活跃链表中的页帧回收之前被第二次访问，那么该页帧会被升级到活跃链表。这个动作称为 activation(升级).

如果一个页帧在第一次被访问时加入到欠活跃链表头 (这个时刻记为 ***t0***), 之后没有被第二次访问。那么它需要在欠活跃链表中移动 $len(inactive)$ 个页 ($len(inactive)$ = 欠活跃链表长度) 才会释放并回收 (eviction)，这个回收的时刻记为 ***t1***。此刻欠活跃链表处理的页帧计数是 ***E***。在该目标页帧被回收后，如果系统需要第二次访问该页 (发生重缺页)，那么该时刻记为 ***t2***。此刻欠活跃链表处理的页帧计数是 ***R***。那么在 ***t2 - t1*** 的这段时间里，欠活跃链表上 eviction(回收) 和 activation(升级) 的页帧的总和为 ***R - E***, 也就是 t1 和 t2 这两个时刻欠活跃链表上处理的页帧数目的快照的差值。那么，第一次访问到第二次访问该页帧，也就是 t0,t2 这两个时刻之间欠活跃链表上处理的页帧数目是:

$$len(inactive) + (R - E)$$

这两个时刻欠活跃链表上处理的页帧快照数目的差值，可以看成两次访问 (缺页) 时刻之间页帧数目的”差距”，也就是”重缺页的距离”。**重缺页的距离 (refault distance)** 是指：一个页帧因为缺页被新加入到欠活跃链表后，到回收再次发生重缺页之时，在 inactive 列表中处理的页帧数量。

$$refaulting\ distance = len(inactive) + (R - E)$$

即,

重缺页的距离 = 欠活跃链表长度 + (重缺页时处理页帧数 − 回收时处理的页帧数)

换个角度思考，如果欠活跃链表再多增加一部分页帧的空间，也就是在 [t1,t2] 这个时间段 eviction(回收) 和 activation(升级) 的页帧总数目作为增加的长度。那么目标页帧就仍会保持在 LRU 链表。但是这个 refault distance 不应该比 $len(inactive) + len(active)$ 还长—参考 5.1.4章节—可知 5.1.4章节的 X 就是此处的 $R - E$, 因此:

$$len(inactive) + X \leq len(inactive) + len(active)$$

即,

$$len(inactive) + (R - E) \leq len(inactive) + len(active)$$

化简得:

$$(R - E) \leq len(active)$$

因为系统根据用途的不同将 page 划分为文件页、匿名页。所以在计算中也要考虑其他页面内存总和数量, 也就是考虑可以将其他类型的页竞争或抢过来作为本类型的工作集页。

$$len(inactive_file) + (R - E) \leq len(inactive_file) + len(active_file) + len(inative_anon) + len(active_anon)$$

$$len(inactive_anon) + (R - E) \leq len(inactive_file) + len(active_file) + len(inative_anon) + len(active_anon)$$

化简后得到:

$$(R - E) \leq len(active_file) + len(inative_anon) + len(active_anon)$$

$$(R - E) \leq len(inactive_file) + len(active_file) + len(active_anon)$$

对应的计算工作集长度的代码在 workingset_reafult() 函数中，下面是截取的代码片段:

```
void workingset_refault(struct page *page, void *shadow)
{
        ... ...
        workingset_size = lruvec_page_state(eviction_lruvec, NR_ACTIVE_FILE);
    if (!file) {
        workingset_size += lruvec_page_state(eviction_lruvec,
                             NR_INACTIVE_FILE);
    }
    if (mem_cgroup_get_nr_swap_pages(memcg) > 0) {
        workingset_size += lruvec_page_state(eviction_lruvec,
                             NR_ACTIVE_ANON);
        if (file) {
            workingset_size += lruvec_page_state(eviction_lruvec,
                             NR_INACTIVE_ANON);
        }
    }
    if (refault_distance > workingset_size)
        goto out;
        ... ...
}
```

在缺页载入时，会调用 *workseting_refault()* 来计算和判断目标页是否是工作集页 (如上)。如果重缺页距离大于工作集，那么对于这种情况系统无能为力，依旧将重缺页而载入的页放到 inactive 链表头。反之，则将” 重缺页” 直接放入 active 链表，防止在下次读该页时，该页已被踢出链表。同时在 *workingset_refault()* 中会判断该页之前是否属于工作集页，如是，则会累加回收损耗。

再回到原来的函数流程。

NOTE

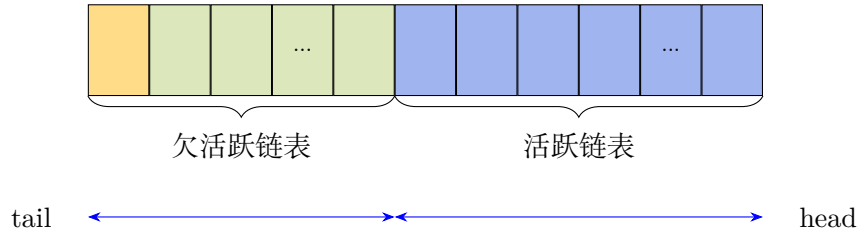
为什么 *LRU* 管理的内存页总数不等于“物理内存总长度”？

因为物理内存中还有:
内核自身占用的内存（如内核代码、数据结构）→ 不参与 *LRU* 回收;
不可回收的页（如锁定页 *mLocked*、内核栈）→ 不在 *LRU* 链表中;
空闲页（*free pages*）→ 未被分配，自然不在 *LRU* 链表中。

因此在理想情况下，目标页帧的平均访问距离大于欠活跃链表，小于LRU 管理的内存页总数。这样就可以保证目标帧在再次访问前不会被踢出 LRU 链表并释放，否则就需要把目标页帧重新加入活跃链表加以保护，以防抖动。因此 refaulting 算法的工作逻辑会通过判断重缺页距离是否 $\in$ (0, LRU 管理的内存页总数] 这个范围，来决定是否将目标页放入活跃链表中，避免下次—第三次访问—访问前该页被踢出 LRU 链表。如果重缺页的距离大于LRU 管理的内存页总数，说明当前系统

内存不足，面对这种状态是无能为力的，这种情况下频繁的换入换出是不可避免的。那么此时不会将再次访问的页放入活跃链表。在这种情况下与第一次 page fault 策略一致将目标页帧直接放置到欠活跃链表的 head 头即可。

通过重缺页距离的判断来避免页访问的抖动，有效的维护 inactive list 、active list 的平衡。



在上图中，系统会对最左侧的页帧 (位于欠活跃列表尾部) 进行回收，该页会被踢出 inactive 链表并且释放页面，这个动作称为 eviction(回收)。

为了避免 active 链表上的其他页帧被不必要地保留，也会从 active 链表中降级一些页帧到 inactive 链表，为了加速回收。

综上而言，我们可以利用这些信息来协调从 active 链表降级页帧和从 inactive 链表回收页帧之间的平衡。

shrink_active_list() 函数会设置 PG_workingset 标志，并且该标志会在页帧的生命周期内持续存在，用于标识该页帧至少曾是活跃 LRU 链表的一部分。workingset_reault() 函数会在缺页发生时被调用，它会检测这个标志的存在，并利用它来计算匿名页和文件页的相对回收代价或相对回收损耗。并把损耗累加存储到 lruvec->anon_cost 和 lruvec->file_cost 成员中。

anon_cost 和 file_cost 分别表示匿名页和文件页各自的页回收损耗。原始的总损耗 total_cost 记为 t:

$$t = a + f$$

这里先将 annon_cost 和 file_cost 分别与原始的总损耗 total_cost 相加，得到新的匿名页损耗 A 和文件页损耗 F。

$$A = t + a = (a + f) + a = 2a + f$$

$$F = t + f = (a + f) + f = a + 2f$$

这里相当于把原来的 a 和 f 加权做了一次” 放大”, 让它们的相对比例更稳定。当 a 和 f 非常小时就不会出现接近 0 的极端值，避免匿名页或文件页在最近的 refaulting 值为 0 时，让对方承担了全部的内存回收压力。因此在假设两者 IO 成本相同的情况下，即使本身的 refaulting 页数目为 0，仍能保证自己一方最低承受 1/3 的内存回收压力。。

再将两个新成本相加，定义为新的总成本 T。

$$T = A + F = 2a + f + a + 2f = 3a + 3f$$

接下来使用 T、A、F 和 swappiness 参数 (swap 倾向度，记为 S) 来计算匿名页和文件页的扫描比例。从 2316 和 2319 行可以看到有”+1” 的动作，这是为了在整数运算中避免出现除 0 或乘数为

0 的情况，因为内核不允许执行浮点运算。不过为了简化，在算式中忽略了偏移 1。如代码所示，ap 和 fp 分别表示匿名页和文件页扫描回收的比重。

swappiness 参数 S 越大，匿名页越容易被回收。而匿名页回收成本 A 越高，回收越难。因此，ap 与 S 成正比，与 A 成反比。

$$ap \propto \frac{S}{A}$$

引入总损耗 T 做为缩放因子。

$$ap = \frac{ST}{A}$$

同理可得 fp:

$$fp = \frac{(200 - S)T}{F}$$

设 D 为匿名页和文件页扫描回收比重的和：

$$D = \frac{ST}{A} + \frac{(200 - S)T}{F}$$

现在可以计算匿名页的比例 (记为 Fr_a) 和文件页的比例 (记为 Fr_f) 分别为:

$$Fr_a = \frac{\frac{ST}{A}}{\frac{ST}{A} + \frac{(200-S)T}{F}}$$

$$Fr_f = \frac{\frac{(200-S)T}{F}}{\frac{ST}{A} + \frac{(200-S)T}{F}}$$

分子父母同除以 T，

$$Fr_a = \frac{\frac{S}{A}}{\frac{S}{A} + \frac{200-S}{F}}$$

$$Fr_f = \frac{\frac{(200-S)}{F}}{\frac{S}{A} + \frac{200-S}{F}}$$

分子分母同乘以 AF, 化简得：

$$Fr_a = \frac{SF}{SF + (200 - S)A}$$

$$Fr_f = \frac{(200 - S)A}{SF + (200 - S)A}$$

由于分母的作用仅在于按分子之和确定每个分数的比例。所以此处重新定义分母 D'，并以 scan_control 中的成本值 a 和 f 来表示方程：

$$Fr_a = \frac{SF}{D'} = \frac{S(a + 2f)}{D'}$$

$$Fr_f = \frac{(200 - S)A}{D'} = \frac{(200 - S)(2a + f)}{D'}$$

假设匿名页和文件页的回收倾向均衡, 交换活跃度 S = 100(中间值), 如果让其中一种回收成本远大于另一种。

假设 $f >> a$(文件页回收成本远大于匿名页):

$$Fr_a \approx \frac{100 * 2f}{100 * 2f + 100 * f} = \frac{200f}{300f} = \frac{2}{3}$$

假设 $a >> f$(匿名页回收成本远大于文件页):

$$Fr_f \approx \frac{100 * 2a}{100 * a + 100 * 2a} = \frac{200a}{300a} = \frac{2}{3}$$

因此，在文件页回收成本远高于匿名页的情况下，匿名页的扫描比例最高可达扫描总量的三分之二；而在匿名页回收成本远高于文件页的情况下，文件页的扫描比例最高也可达扫描总量的三分之二。但是仍保证了自己最低承受 1/3 的内存回收压力。也就是说不允许任何一个链表的回收落后，无论抖动情况或者下文说的 refaulting(重缺页) 情况如何，始终要保持 1/3 的回收压力。

这儿可以看到 anon_cost 和 file_cost 在统计的时候分别都加上了 total_cost，而 2313 的 total_cost 也变成了 2310 行原始值的两倍。这是为了避免匿名页或文件页在最近的 refaulting 值为 0 时，让对方承担了全部的内存回收压力。因此在假设两者 IO 成本相同的情况下，即使本身的 refaulting 页数目为 0，仍能保证自己一方最低承受1/3的内存回收压力。

例如，譬如假设匿名页最近 refaulting 的数量是 a，文件页最近 refaulting 页的数量是 f。那么很显然总数是 (a + f)。所以匿名页所占的份额是：

$$\frac{a}{a+f} \tag{1}$$

现在通过自定义的权重调整方法给匿名页的数量加上原始总和 (a + f)，相应的，总和也要加上 (a + f)。那么，refaulting 的匿名页所占份额变为：

$$\frac{a+(a+f)}{a+f+(a+f)} = \frac{a+(a+f)}{2*(a+f)}$$

化简为：

$$\frac{a+(a+f)}{2*(a+f)} = \frac{2a+f}{2a+2f} = \frac{a+\frac{f}{2}}{a+f} \tag{2}$$

对比 (1) 和 (2) 式，分子是在原有值基础上增加了 $\frac{f}{2}$。对应的 refaulting 的文件页所占份额为:

$$\frac{f+\frac{a}{2}}{a+f} \tag{3}$$

- 2321-2323 ap 和 fp 分别表示匿名页和文件页扫描回收的比重。这样需要扫描的匿名页和文件页的占比分别是 $\frac{ap}{ap+fp}$ 和 $\frac{fp}{ap+fp}$

  而 ap:fp 也就是：

  $$\frac{a+\frac{f}{2}}{a+f} : \frac{f+\frac{a}{2}}{a+f}$$

  如果在 IO 成本相同的情况下，碰巧 refaulting 的匿名页数目为 0，那么上式变成：

  $$\frac{0+\frac{f}{2}}{0+f} : \frac{f+\frac{0}{2}}{0+f}$$

  也就是:

  $$\frac{1}{2} : 1 = 1 : 2$$

  即：

  $$ap : fp = 1 : 2$$

  所以在 refaulting 的匿名页数目为 0 情况下，匿名页承受的内存 (扫描) 压力为：

  $$ap : (ap + fp) = 1 : 3$$

  这也是前面提及的 1/3 内存压力的由来。

- 2315-2319 swappiness 是匿名页回收优先级，或是匿名页的回收倾向度，也可以相对的粗略认为是匿名页 swapping 回收一页的I/O 成本。它的取值范围在 0-200 之间，默认 60。相对的，(200-swappiness) 表示文件页的回收倾向度，也就是文件页和” 匿名页的回收倾向度” 的补值成正比。当取值 100 时，匿名页 swapping 和文件页 caching 会被施加相同的内存压力。更小的值表示更昂贵的 swap IO 成本，意味着不倾向于回收匿名页；更大的值表示 swap 具有更廉价的 IO 成本，也就意味着回收匿名页的倾向度更高。当取 0 值时，内核不会发起 swap，除非该 zone 的水位线低于 high watermark。

  ”refaulting” 这个过程会有一定的资源消耗，所以交换出去的页如果越多，需要再次使用这些页的成本或消耗就越大。这种情况也叫” 抖动”。sc->anon_cost 和 sc->file_cost 分别表示”refaulting” 的匿名页及文件页的数量，也代表回收这两种类型页的成本或代价。所以在某个 LRU 上由于 refaulting 而消耗的成本越多，那么内核在这次计算扫描回收页时，基于压力平衡考虑倾向于给这个 LRU 的内存压力就越小。所以给予的压力权重是（anon_cost / total_cost）的倒数。

  所以 2315-2316 或 2318-2319 的算式踢开避免除以 0 的 +1，可以简化理解如下:

  $$\text{扫描回收匿名页数目的权重} = \text{swap 的倾向度} \times \text{文件页的回收损耗}$$

  和，

  $$\text{扫描回收文件页数目的权重} = (1 - \text{swap 的倾向度}) \times \text{匿名页的回收损耗}$$

  也就是说页的回收数量和它的回收损耗成反比。同时匿名页的回收数量和 $swapiness$ 成正比，文件页的回收数量和 $(200 - swapiness)$ 成正比。

  即

  $$\text{匿名页回收数量} \propto \frac{swapiness}{\text{匿名页回收损耗}}$$

  $$\text{文件页回收数量} \propto \frac{200 - swapiness}{\text{文件页回收损耗}}$$

  又因为 损耗总量 = 匿名页回收损耗 + 文件页回收损耗。所以，和匿名页回收损耗成反比，也就意味着和文件页回收损耗成正比。反之亦然。所以，

  $$\text{匿名页回收数量} \propto (swapiness \cdot \text{文件页回收损耗})$$

  $$\text{文件页回收数量} \propto ((200 - swapiness) \cdot \text{匿名页回收损耗})$$

  化成代码中的形式，

  $$\text{匿名页回收数量} \propto (swapiness \cdot fp)$$

  $$\text{文件页回收数量} \propto ((200 - swapiness) \cdot ap)$$

- 2325 遍历目标 lruvec 上个 lru 链表进行扫描页统计。
- 2331 计算指定 LRU 列表 (匿名页或文件页) 上的页数目。
- 2332-2334 计算目标控制组 memcg 的内存保护阈值 (mem_cgroup_protection() 函数见后)。返回 0 意味着不作保护，正常扫描回收。
- 2366 获取当前 memcg 实际占用的内存是多少页。
- 2369 取实际使用页数和保护水位线页数两者中的较大值。

- 2371-2372 计算扫描页数量。实际公式等价于:

$$\text{扫描页数目} = \text{总可扫描页数} \times \text{允许扫描的比例}$$

- 2379 这里是为了保证扫描的目标页数至少是 32。

  SWAP_CLUSTER_MAX 为 32。之所以这么做是为了避免极端情况 $scan = 0$ 使进程卡住。避免 *sc->priority* 持续优先级提高。

- 2381 对于没有 memcg 保护阈值的场景，扫描页数直接是目标链表上的总页数。

- 2384 利用回收积极度来计算得到修正后的应扫描页数。算式:

$$\text{修正后的扫描页数} = \frac{\text{扫描页数}}{2^{sc->priority}}$$

  每个 lru 链表需要扫描多少页框数与 *sc->priority* 有关，*sc->priority* 越小，那么需要扫描的页框就越多。

- 2390-2391 如果本次扫描页数为 0，并且目标 memcg 已经下线。要把残留的缓存页面清理干净。

- 2393 前面代码已经根据扫描控制参数 (sc) 确定了扫描的平衡策略，也就是控制对匿名页和文件页扫描页数目怎么分配。此处对各个策略处理。

- 2394-2396 按匿名页和文件页 LRU 链表上页的数量作为比例来计算各自扫描页数。

- 2397-2409 按 swapiness、回收损耗计算分配文件和匿名页各自的扫描比例。(详细计算见前面代码)

- 2410-2415 仅扫描文件页或匿名页。

- 2421 通过传出参数 nr 返回文件页和匿名页的 active 和 inactive 链表上要扫描的页的数量。

do_try_to_free_pages() → ··· ··· → *shrink_lruvec()* → *get_scan_count()* → *mem_cgroup_protection()*

```
363 static inline unsigned long mem_cgroup_protection(struct mem_cgroup *root,
364                                                    struct mem_cgroup *memcg,
365                                                    bool in_low_reclaim)
366 {
367     if (mem_cgroup_disabled())
368         return 0;
369 
370     /*
371      * There is no reclaim protection applied to a targeted reclaim.
372      * We are special casing this specific case here because
373      * mem_cgroup_protected calculation is not robust enough to keep
374      * the protection invariant for calculated effective values for
375      * parallel reclaimers with different reclaim target. This is
376      * especially a problem for tail memcgs (as they have pages on LRU)
377      * which would want to have effective values 0 for targeted reclaim
378      * but a different value for external reclaim.
379      *
380      * Example
381      * Let's have global and A's reclaim in parallel:
382      *  |
383      *  A (low=2G, usage = 3G, max = 3G, children_low_usage = 1.5G)
384      *  |\
385      *  | C (low = 1G, usage = 2.5G)
386      *  B (low = 1G, usage = 0.5G)
```

```
387         *
388         * For the global reclaim
389         * A.elow = A.low
390         * B.elow = min(B.usage, B.low) because children_low_usage <= A.elow
391         * C.elow = min(C.usage, C.low)
392         *
393         * With the effective values resetting we have A reclaim
394         * A.elow = 0
395         * B.elow = B.low
396         * C.elow = C.low
397         *
398         * If the global reclaim races with A's reclaim then
399         * B.elow = C.elow = 0 because children_low_usage > A.elow)
400         * is possible and reclaiming B would be violating the protection.
401         *
402         */
403        if (root == memcg)
404            return 0;
405
406        if (in_low_reclaim)
407            return READ_ONCE(memcg->memory.emin);
408
409        return max(READ_ONCE(memcg->memory.emin),
410                   READ_ONCE(memcg->memory.elow));
411    }
```

- 367-368 没有使能 memcg，直接返回。
- 403-404 目标 memcg 就是本次回收的目标 cgroup 本身，不享受任何保护返回 0。

  定向回收也就是 A memcg 自己触发 low 回收，$root = A$。此时 A 本身就是回收的对象，A 自己不能享受保护；但是 A 的子组 B、C 仍然要享有自己的 emin/elow 保护。

  内核的 *emin/elow* 有效值是基于父组配额动态计算出来的。如果全局回收和 A 组内部 low 回收并发运行，两套回收上下文会对同一批子 memcg 算出两套不一样的 elow。如果不特殊处理，全局回收路径上可能错误把 B/C 的保护值算成 0，错误回收 B，违背 B 的 memory.low 保护。

  所以做特殊约定：当被评估 memcg 就是本次回收的 root 目标组，直接返回 0，取消它自身的保护；子 memcg 继续正常走保护逻辑。

  ======== 此处待重新解析
- 406-407 low 回收场景下只尊重 memcg 的 emin 硬保护，elow 软保护直接作废。memcg 只要内存高于 emin，就可以被 low 回收扫描；只有低于 emin，才完全不碰。
- 409-410 该 memcg 受保护内存大小, 回收逻辑尽量不要把内存压到这个数值之下。

  这里使用宏 max() 是防御性编程，防止用户配置颠倒 (即，$memory.min > memory.low$) 导致 $elow < emin$。

5.1.11 shrink_list()

shrink_list() 负责扫描并回收指定链表上的可回收页，释放内存。

do_try_to_free_pages() → ··· ··· → *shrink_lruvec()* → *shrink_list()*

```
2160 static unsigned long shrink_list(enum lru_list lru, unsigned long nr_to_scan,
2161                                   struct lruvec *lruvec, struct scan_control *sc)
2162 {
2163     if (is_active_lru(lru)) {
2164         if (sc->may_deactivate & (1 << is_file_lru(lru)))
2165             shrink_active_list(nr_to_scan, lruvec, sc, lru);
2166         else
2167             sc->skipped_deactivate = 1;
2168         return 0;
2169     }
2170
2171     return shrink_inactive_list(nr_to_scan, lruvec, sc, lru);
2172 }
```

- 2163-2164 检查是否允许对活跃链表上的页进行降级 (deactivate) 操作。

  通过检查scan_control->may_deactivate 字段上的 1 << *is_file_lru*() 位，来判断是否允许对 active 页降级，以加速回收。

  注意: 此处不只对文件页置降级标志。*->may_deactivate* 是一个 2 bit 的变量，*bit[0]==1* 表示允许匿名页的活跃页 deactive；*bit[1]==1* 表示允许文件页的活跃页 deactive。而 *is_file_lru()* 根据目标 lru 判断是文件页则返回 1，匿名页返回 0。

- 2165 若允许降级，则通过 shrink_active_list() 函数对活跃链表进行收缩，即将 LRU 活跃链表中的页降级为 inactive 状态, 放到 inactive 链表。

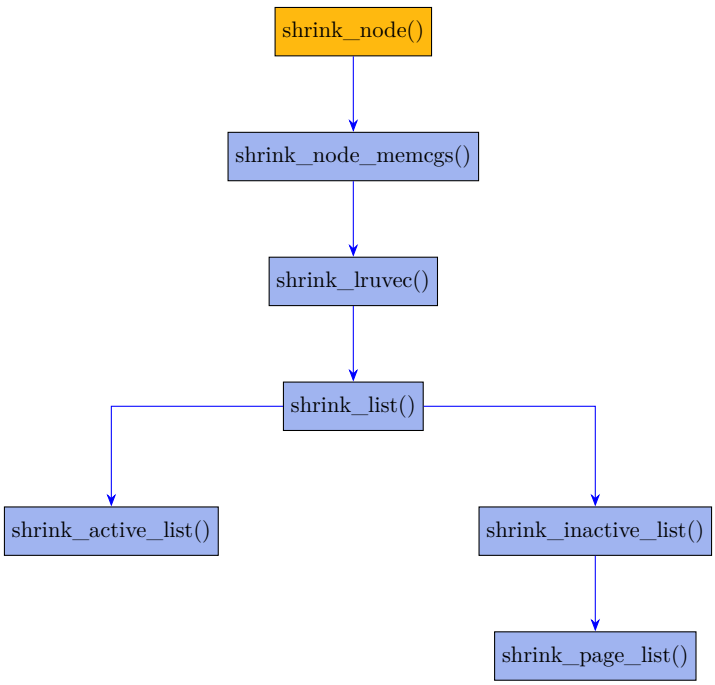
- 2167 若不允许执行降级操作，则会设置 skipped_deactivate 字段.

  NOTE

  如果本次内存回收是由直接内存回收*(direct reclaim)* 触发的，*do_try_to_free_pages()* 函数可根据需要忽略*->may_deactivate* 的设置值，强制执行页降级操作。

- 2171 如果目标链表属于欠活跃 LRU 链表，则对该欠活跃链表执行内存回收

shrink_list() 会根据目标链表的类型和是否允许降级，来对活跃和不活跃链表执行收缩或回收操作如图:



5.1.12 shrink_inactive_list()

下面先看 shrink_inactive_list() 的执行:

mm/vmscan.c :: shrink_inactive_list()

```
1915 static noinline_for_stack unsigned long
1916 shrink_inactive_list(unsigned long nr_to_scan, struct lruvec *lruvec,
1917              struct scan_control *sc, enum lru_list lru)
1918 {
1919     LIST_HEAD(page_list);
1920     unsigned long nr_scanned;
1921     unsigned int nr_reclaimed = 0;
1922     unsigned long nr_taken;
1923     struct reclaim_stat stat;
1924     bool file = is_file_lru(lru);
1925     enum vm_event_item item;
1926     struct pglist_data *pgdat = lruvec_pgdat(lruvec);
1927     bool stalled = false;
1928 
1929     while (unlikely(too_many_isolated(pgdat, file, sc))) {
1930         if (stalled)
1931             return 0;
1932 
1933         /* wait a bit for the reclaimer. */
1934         msleep(100);
1935         stalled = true;
1936 
1937         /* We are about to die and free our memory. Return now. */
1938         if (fatal_signal_pending(current))
1939             return SWAP_CLUSTER_MAX;
1940     }
1941 
1942     lru_add_drain();
1943 
1944     spin_lock_irq(&pgdat->lru_lock);
1945 
1946     nr_taken = isolate_lru_pages(nr_to_scan, lruvec, &page_list,
1947                      &nr_scanned, sc, lru);
1948 
1949     __mod_node_page_state(pgdat, NR_ISOLATED_ANON + file, nr_taken);
1950     item = current_is_kswapd() ? PGSCAN_KSWAPD : PGSCAN_DIRECT;
1951     if (!cgroup_reclaim(sc))
1952         __count_vm_events(item, nr_scanned);
1953     __count_memcg_events(lruvec_memcg(lruvec), item, nr_scanned);
1954     __count_vm_events(PGSCAN_ANON + file, nr_scanned);
1955 
1956     spin_unlock_irq(&pgdat->lru_lock);
1957 
1958     if (nr_taken == 0)
1959         return 0;
1960 
1961     nr_reclaimed = shrink_page_list(&page_list, pgdat, sc, 0,
1962                 &stat, false);
1963 
1964     spin_lock_irq(&pgdat->lru_lock);
1965 
1966     move_pages_to_lru(lruvec, &page_list);
1967 
1968     __mod_node_page_state(pgdat, NR_ISOLATED_ANON + file, -nr_taken);
1969     lru_note_cost(lruvec, file, stat.nr_pageout);
1970     item = current_is_kswapd() ? PGSTEAL_KSWAPD : PGSTEAL_DIRECT;
1971     if (!cgroup_reclaim(sc))
1972         __count_vm_events(item, nr_reclaimed);
```

```
1973        __count_memcg_events(lruvec_memcg(lruvec), item, nr_reclaimed);
1974        __count_vm_events(PGSTEAL_ANON + file, nr_reclaimed);
1975
1976        spin_unlock_irq(&pgdat->lru_lock);
1977
1978        mem_cgroup_uncharge_list(&page_list);
1979        free_unref_page_list(&page_list);
1980
1981       /*
1982         * If dirty pages are scanned that are not queued for IO, it
1983         * implies that flushers are not doing their job. This can
1984         * happen when memory pressure pushes dirty pages to the end of
1985         * the LRU before the dirty limits are breached and the dirty
1986         * data has expired. It can also happen when the proportion of
1987         * dirty pages grows not through writes but through memory
1988         * pressure reclaiming all the clean cache. And in some cases,
1989         * the flushers simply cannot keep up with the allocation
1990         * rate. Nudge the flusher threads in case they are asleep.
1991         */
1992        if (stat.nr_unqueued_dirty == nr_taken)
1993            wakeup_flusher_threads(WB_REASON_VMSCAN);
1994
1995        sc->nr.dirty += stat.nr_dirty;
1996        sc->nr.congested += stat.nr_congested;
1997        sc->nr.unqueued_dirty += stat.nr_unqueued_dirty;
1998        sc->nr.writeback += stat.nr_writeback;
1999        sc->nr.immediate += stat.nr_immediate;
2000        sc->nr.taken += nr_taken;
2001        if (file)
2002            sc->nr.file_taken += nr_taken;
2003
2004        trace_mm_vmscan_lru_shrink_inactive(pgdat->node_id,
2005                nr_scanned, nr_reclaimed, &stat, sc->priority, file);
2006        return nr_reclaimed;
2007    }
```

- 1929-1940 too_many_isolated() 函数 (见后面实现) 逻辑主要防护直接内存回收多进程并发隔离页爆炸的场景，kswapd 单线程后台回收理论上也会产生隔离页，但不会出现隔离页无限疯涨的现象，不会像直接内存回收那样，在极端场景下出现 (隔离页面) 无限增长的情况。

  这段代码主要是处理直接回收时隔离了太多脏页没有加入写回队列的场景，此时会让调用者休眠一会来进行回收限流。换言之，等待异步 IO 拥塞缓解。

  1938-1939 如果在休眠期间进程收到了致命信号，则会直接提前退出。

- 1942 将尚未刷新到 LRU 链表但本应属于 LRU 的页片批量立即放入对应的 LRU 链表中。内核为提高效率，不会立即将这些页面加入 LRU 链表而是通过 pagevec 结构批量暂存。这种延迟机制减少了频繁操作链表带来的锁竞争开销。调用 lru_add_drain() 会强制将所有暂存在 pagevec 中的页面加入对应的 LRU 链表，确保回收逻辑能扫描到最新的页面状态。若未调用此函数，添加的页面未被添加到 active 或 inactive LRU 链表会导致回收逻辑遗漏部分页面或回收延迟。

- 1946 将目标 LRU 链表上的页面隔离出来待回收。

- 1949 本次成功隔离出的页面数目，累加到 node 的隔离页计数器。

  node 维护两个计数器：

  - *NR_ISOLATED_ANON*：本 node 上被隔离的匿名页欠活跃 LRU 链表的页数目。
  - *NR_ISOLATED_FILE*：本 node 上被隔离的文件页欠活跃 LRU 的 file 页数目。

- 1950 若当前是 kswapd 线程则 *item = PGSCAN_KSWAPD*, 当前是普通用户进程则 *item = PGSCAN_DIRECT*。

  PGSCAN_KSWAPD、*PGSCAN_DIRECT* 是 vmstat 事件，含义：

  - *PGSCAN_KSWAPD*：kswapd 扫描了多少页
  - *PGSCAN_DIRECT*: 直接回收 (用户进程) 扫描了多少页

  节点***/proc/vmstat*** 里可以看到这两个字段。通过它们可以方便定位内存压力来源，是后台 kswapd 在干活，还是大量进程在做直接回收。

```
comlee:~ >>>cat /proc/vmstat|grep -Ii "pgscan_"
pgscan_kswapd 0
pgscan_direct 0
```

图 5–4: kswapd 和直接回收显示的扫描页数信息

NOTE

通过判断 current->flags 里是否置位 PF_KSWAPD, 可以判断当前是否 kswapd 线程。

- 1951-1952 不是 memcg 定向回收的时候，才更新全局 node 的 vmstat 事件。把扫描的页数累加到 *PGSCAN_KSWAPD* 或 *PGSCAN_DIRECT* 事件对应的数组.

  全局回收 (非 cgroup_reclaim) 扫描的是整个 node 的 lru，扫描计数计入全局 vmstat (/proc/vmstat)。memcg 定向回收只是扫描 node 上某一部分属于该 memcg 的页面，不是整个 node 的扫描。如果把 memcg 内部回收的扫描量也计入全局 node 的 PGSCAN_*，会造成全局统计被虚高注水。

- 1953 memcg 定向回收，在 memcg 维度 (内部) 记录扫描页数。

- 1954 更新 node 维度 vm 事件计数器，记录扫描到的 PGSCAN_ANON 或 PGSCAN_FILE 的页数。

  需要注意的是要区分两组完全不同的 vmstat 事件，不要搞混：

  - ***PGSCAN_KSWAPD***、***PGSCAN_DIRECT*** 是按回收者身份区分，是谁在扫描 (kswapd 线程/用户进程 (直接回收)).
  - ***PGSCAN_ANON***、***PGSCAN_FILE*** 是按页面类型区分，扫描的是匿名页，还是文件页。

  节点***/proc/vmstat*** 里可以看到这两个字段：

```
comlee:~ >>>cat /proc/vmstat|grep -Ii "pgscan_"
pgscan_kswapd 0
pgscan_direct 0
pgscan_direct_throttle 0
pgscan_anon 0
pgscan_file 0
```

图 5–5: 扫描的文件页和匿名页的数量信息

- 1958-1959 如果未能隔离到任何页，则直接返回失败。

- 1961 在成功隔离页且确认可以继续回收后，调用内存回收机制的核心函数 shrink_page_list() 来执行回收。成功回收的基页数量保存在 nr_reclaimed 中。

  该函数的传入参数 page_list 就是 1946 行隔离出的页挂载的链表。

- 1964-1966 所有未被回收的页会保留在 page_list 中，在持有 struct lruvec->lru_lock 锁的前提下，通过 move_pages_to_lru() 将这些页片还回合适的 LRU 链表。同时把 page_list 链表清 0。

- 1969 记录换出脏页到磁盘/外存的开销。回收页是脏页要产生 I/O 操作，这是昂贵的回收，损耗较大，因此会记录损耗 (cost)。

- 1970-1974 这部分同 1950-1954，只不过记录的是成功回收到的页数。

  需要注意的是，这里 ***PGSTEAL_KSWAPD***、***PGSTEAL_DIRECT***、***PGSTEAL_ANON***、***PGSTEAL_FILE*** 都是用的一个描述词"steal"，意味着：把页面从原有所有者手里抢过来、释放掉，供给新的内存分配使用。

  同样的，这里的 vmstat 事件记录的值同样在*/proc/vmstat* 节点可以看到:

```
comlee:~ >>>cat /proc/vmstat|grep "pgsteal"
pgsteal_kswapd 0
pgsteal_direct 0
pgsteal_anon 0
pgsteal_file 0
```

图 5–6: 成功回收的文件页和匿名页的数量信息

- 1978 批量把一组内存页面还回伙伴系统时，要从对应 mem_cgroup 的资源计数中把这些页面剔除。当内存页面被分配给某个进程或缓存时，内核通过 mem_cgroup_charge() 将该页面内存的使用计入所属的 mem_cgroup 的资源计数器中。反向操作是取消记账 (uncharge) 操作：当页面不再被使用 (释放或迁移等) 时,mem_cgroup_unchange_* 系列函数负责从 usage、stat 计数器中剔除相应的资源页。

- 1979 把解除引用的 page 还回伙伴系统。

- 1992-1993 如果本次扫描拿到的都是脏页，而且都没有被提交给 IO 回写队列，那就意味着: 没有干净页可以回收; 脏页很多。或者说想回收，但是拿到的都是未回写的脏页，都没进入 IO 回写队列。说明回写线程可能在休眠。因此需要强制唤醒内核的脏页回写线程（flusher threads），让它们立刻开始写脏页。

  - *nr_taken* 是本次扫描到的，准备回收的页总数。
  - *nr_unqueued_dirty* 是摘取出的页里面的，尚未提交回写队列的脏页数量。

- 1995-2002 更新 struct scan_control 统计信息。

  需要注意的是，2000 行的 *sc->nr_taken* 是全局**总**计数，是将本次回收的匿名页和文件页总和累加到该变量。而 2002 行统计的仅是属于文件页的数量。

- 2006 返回本次成功回收的基页数量。

```
1794 static int too_many_isolated(struct pglist_data *pgdat, int file,
1795         struct scan_control *sc)
1796 {
1797     unsigned long inactive, isolated;
1798
1799     if (current_is_kswapd())
1800         return 0;
1801
1802     if (!writeback_throttling_sane(sc))
1803         return 0;
1804
1805     if (file) {
1806         inactive = node_page_state(pgdat, NR_INACTIVE_FILE);
1807         isolated = node_page_state(pgdat, NR_ISOLATED_FILE);
1808     } else {
1809         inactive = node_page_state(pgdat, NR_INACTIVE_ANON);
1810         isolated = node_page_state(pgdat, NR_ISOLATED_ANON);
1811     }
1812
1813     /*
1814      * GFP_NOIO/GFP_NOFS callers are allowed to isolate more pages, so they
1815      * won't get blocked by normal direct-reclaimers, forming a circular
1816      * deadlock.
1817      */
1818     if ((sc->gfp_mask & (__GFP_IO | __GFP_FS)) == (__GFP_IO | __GFP_FS))
1819         inactive >>= 3;
1820
1821     return isolated > inactive;
1822 }
```

- 1799-1800 隔离限流是用来保护直接回收的，kswapd 不受这个限制。

  直接回收者一轮最多可以从 LRU 链表隔离 SWAP_CLUSTER_MAX 个页面，随后可能被调度器被切走休眠。大量任务同时进行内存分配时，处于休眠状态的直接回收的上下文在各个 core 不断堆积，会造成 lru 链表被掏空 (隔离)，引起 OoM。

  kswapd 是后台唯一的异步回收线程，如果休眠时隔离页有太多，那么就没有谁能来解决这些隔离页的问题了。被隔离的页需要被处理释放才能回到空闲池，但是负责处理的 kswapd 自己停下来了 (不能依赖直接回收这种方式)，这些页就永远也无法被处理释放，于是可能导致永久阻塞造成死锁。

- 1802-1803 判断是否回收拥塞，如果是，立即退出。把繁重工作给 kswapd 后台线程，避免上下文被卡住。

- 1806-1810 node_page_state() 函数根据传入页的类型索引，调用原子操作来分别读取 pgdate->vm_stat[ ] 数组里 inactive 的文件页或匿名页的数目，以及处于隔离状态的文件页或匿名页的数目。

- 1818-1819 使用 ___GFP_IO 或 ___GFP_FS 的调用者，需要将 inactive 页的数目修正为原数目的 1/8。而 GFP_NOIO,GFP_NOFS 的调用者允许隔离更多页面。因为 NOIO、NOFS 意味着禁止回写，这样能回收的页很少，只能是干净页。所以必须放宽上限，以运行找到更多干净页。否则回收失败容易造成 OoM。

  - ***___GFP_IO*** 分配内存时可以发起磁盘 IO, 但是不经过文件系统，类似dd这种方式。
  - ***___GFP_FS*** 分配内存时可以调用文件系统层代码。

这样隔离页面数超过不活跃页面数的 1/8，就判定为隔离页面过多，增加了返回值为 1 的概率。这样可以在后续处理中（见 shrink_inactive_list() 函数得到该函数返回值后的处理）限制普通直接回收进程对 inactive 页的扫描频率，避免因频繁触发 I/O 或文件系统操作导致系统颠簸。

相对而言，GFP_NOIO/GFP_NOFS 调用返回 1 的可能性远小于 ___GFP_IO/___GFP_FS 的调用，从而允许使用 GFP_NOIO/GFP_NOFS 调用者隔离更多页面，允许隔离页面数量一直增长到等于 inactive 页面总数。避免被常规直接回收者 (进程) 阻塞形成死循环。

- 1821 隔离页面数目大于 inactive 页数目返回 1, 否则返回 0.

  如果隔离页面太多，那么在直接回收调用者调用的 shrink_inactive_list() 函数里休眠一会，进行回收限流。

5.1.13 shrink_page_list()

shrink_inactive_list() > *shrink_page_list()*

```
1072 static unsigned int shrink_page_list(struct list_head *page_list,
1073                       struct pglist_data *pgdat,
1074                       struct scan_control *sc,
1075                       enum ttu_flags ttu_flags,
1076                       struct reclaim_stat *stat,
1077                       bool ignore_references)
1078 {
1079     LIST_HEAD(ret_pages);
1080     LIST_HEAD(free_pages);
1081     unsigned int nr_reclaimed = 0;
1082     unsigned int pgactivate = 0;
1083
1084     memset(stat, 0, sizeof(*stat));
1085     cond_resched();
1086
1087     while (!list_empty(page_list)) {
1088         struct address_space *mapping;
1089         struct page *page;
1090         enum page_references references = PAGEREF_RECLAIM;
1091         bool dirty, writeback, may_enter_fs;
1092         unsigned int nr_pages;
1093
1094         cond_resched();
1095
1096         page = lru_to_page(page_list);
1097         list_del(&page->lru);
1098
1099         if (!trylock_page(page))
1100             goto keep;
1101
1102         VM_BUG_ON_PAGE(PageActive(page), page);
1103
1104         nr_pages = compound_nr(page);
1105
1106         /* Account the number of base pages even though THP */
1107         sc->nr_scanned += nr_pages;
1108
1109         if (unlikely(!page_evictable(page)))
1110             goto activate_locked;
1111
1112         if (!sc->may_unmap && page_mapped(page))
1113             goto keep_locked;
```

```
1114
1115        may_enter_fs = (sc->gfp_mask & __GFP_FS) ||
1116            (PageSwapCache(page) && (sc->gfp_mask & __GFP_IO));
1117
1118        /*
1119         * The number of dirty pages determines if a node is marked
1120         * reclaim_congested which affects wait_iff_congested. kswapd
1121         * will stall and start writing pages if the tail of the LRU
1122         * is all dirty unqueued pages.
1123         */
1124        page_check_dirty_writeback(page, &dirty, &writeback);
1125        if (dirty || writeback)
1126            stat->nr_dirty++;
1127
1128        if (dirty && !writeback)
1129            stat->nr_unqueued_dirty++;
1130
1131        /*
1132         * Treat this page as congested if the underlying BDI is or if
1133         * pages are cycling through the LRU so quickly that the
1134         * pages marked for immediate reclaim are making it to the
1135         * end of the LRU a second time.
1136         */
1137        mapping = page_mapping(page);
1138        if (((dirty || writeback) && mapping &&
1139             inode_write_congested(mapping->host)) ||
1140            (writeback && PageReclaim(page)))
1141            stat->nr_congested++;
1142
1143        /*
1144         * If a page at the tail of the LRU is under writeback, there
1145         * are three cases to consider.
1146         *
1147         * 1) If reclaim is encountering an excessive number of pages
1148         *    under writeback and this page is both under writeback and
1149         *    PageReclaim then it indicates that pages are being queued
1150         *    for IO but are being recycled through the LRU before the
1151         *    IO can complete. Waiting on the page itself risks an
1152         *    indefinite stall if it is impossible to writeback the
1153         *    page due to IO error or disconnected storage so instead
1154         *    note that the LRU is being scanned too quickly and the
1155         *    caller can stall after page list has been processed.
1156         *
1157         * 2) Global or new memcg reclaim encounters a page that is
1158         *    not marked for immediate reclaim, or the caller does not
1159         *    have __GFP_FS (or __GFP_IO if it's simply going to swap,
1160         *    not to fs). In this case mark the page for immediate
1161         *    reclaim and continue scanning.
1162         *
1163         *    Require may_enter_fs because we would wait on fs, which
1164         *    may not have submitted IO yet. And the loop driver might
1165         *    enter reclaim, and deadlock if it waits on a page for
1166         *    which it is needed to do the write (loop masks off
1167         *    __GFP_IO|__GFP_FS for this reason); but more thought
1168         *    would probably show more reasons.
1169         *
1170         * 3) Legacy memcg encounters a page that is already marked
1171         *    PageReclaim. memcg does not have any dirty pages
1172         *    throttling so we could easily OOM just because too many
1173         *    pages are in writeback and there is nothing else to
1174         *    reclaim. Wait for the writeback to complete.
1175         *
1176         * In cases 1) and 2) we activate the pages to get them out of
1177         * the way while we continue scanning for clean pages on the
```

```
1178          * inactive list and refilling from the active list. The
1179          * observation here is that waiting for disk writes is more
1180          * expensive than potentially causing reloads down the line.
1181          * Since they're marked for immediate reclaim, they won't put
1182          * memory pressure on the cache working set any longer than it
1183          * takes to write them to disk.
1184          */
1185         if (PageWriteback(page)) {
1186             /* Case 1 above */
1187             if (current_is_kswapd() &&
1188                 PageReclaim(page) &&
1189                 test_bit(PGDAT_WRITEBACK, &pgdat->flags)) {
1190                 stat->nr_immediate++;
1191                 goto activate_locked;
1192
1193             /* Case 2 above */
1194             } else if (writeback_throttling_sane(sc) ||
1195                 !PageReclaim(page) || !may_enter_fs) {
1196                 /*
1197                  * This is slightly racy - end_page_writeback()
1198                  * might have just cleared PageReclaim, then
1199                  * setting PageReclaim here end up interpreted
1200                  * as PageReadahead - but that does not matter
1201                  * enough to care.  What we do want is for this
1202                  * page to have PageReclaim set next time memcg
1203                  * reclaim reaches the tests above, so it will
1204                  * then wait_on_page_writeback() to avoid OOM;
1205                  * and it's also appropriate in global reclaim.
1206                  */
1207                 SetPageReclaim(page);
1208                 stat->nr_writeback++;
1209                 goto activate_locked;
1210
1211             /* Case 3 above */
1212             } else {
1213                 unlock_page(page);
1214                 wait_on_page_writeback(page);
1215                 /* then go back and try same page again */
1216                 list_add_tail(&page->lru, page_list);
1217                 continue;
1218             }
1219         }
1220
1221                 <..... to be continued.....>
```

shrink_page_list() 函数是页面回收的核心函数，其主要作用是通过一系列检查操作，从目标链表中筛选并回收符合条件的物理页框。

- 1079-1080 先初始化了初始化两个链表，在函数执行过程中临时使用。
  - ret_pages 是返回链表，函数处理过程中无法回收的页会被暂时放入该链表。
  - free_pages 是回收链表，函数执行中可以回收的页被依次放入该链表暂存。并且在退出函数前由 free_unref_page_list() 批量释放 (见后面 1487 行) 这些页。
- 1087 遍历迭代传递进来的被隔离页的链表。
- 1096-1097 从已隔离的目标链表 page_list 中迭代取出链表尾部页，并将该页从 LRU 链表移除，这体现了 LRU 最老的页先回收的策略。

  lru_to_page() 通过 head->prev 获取到 page_list 的尾部节点. 因为 lru 是双向循环链表，所以 page_list 的 head->prev 指向尾部节点，head->next 指向链表头。

========== 待加入图形表示

- 1099-1100 每次循环时，会尝试锁住该页 (通过是否可以成功锁住 page 来判断是否有别的进程/线程在使用该页) ——在存在其他可回收页的情况下，若页面正被其他进程/线程使用，系统不会阻塞等待锁。若无法加锁，则跳至 keep 标签处执行，保留该页，并将其移至无法回收页的暂存 (ret_pages) 链表。
- 1109-1110 如果目标页不可回收，我们会将其激活，跳转并将页面加入到 active 链表，使其避开可回收页的处理流程。

  用户空间可以通过 madvise(MADV_UNEVICTABLE) 建议内核标记内存为不能回收。

  但是要注意的是在用户空间通过系统调用 mlock() 锁住的页面会放到 unevictable 链表，不会被扫描隔离。因此，mlock() 锁住的页不会走到这个判断。
- 1112-1113 若页面当前已被用户空间映射且不允许解除映射，则保留该页。
- 1115-1116 这两行代码的作用是判断是否可以进行文件系统操作。只要满足以下两个条件中的任意一个，may_enter_fs 就会被设置为 true：
  - 当前内存分配操作允许进行文件系统相关的操作 (即设置了 ___GFP_FS 标志), **或**匿名页已经加入 swapcache(匿名页存在 swap 副本)。
  - 允许进行磁盘 (后备存储) I/O (即设置了 ___GFP_IO 标志)。

Table 5–1: ___GFP_FS 和 ___GFP_IO 的区别

___GFP_IO	***___GFP_FS***
允许执行磁盘 *I/O* 操作，处于较底层的块 *I/O* 操作。	允许执行 *VFS* 操作，启动文件系统 *I/O*，处于比磁盘 *I/O* 更高的层级。
主要关注磁盘硬件层面的交互，用于页换出等需磁盘写入的操作。	侧重于文件系统相关操作，涉及文件系统接口调用和元数据操作等。

  - ***PageSwapCache*** 宏用于判断一个页当前是否已经建立 swap 映射。换言之，也就是该页是否已经被换出到 swap 分区了，现在内存中只是缓存的副本。

    系统为了应付应用大量使用内存的场景，会将后备存储 (例如：磁盘、flash、emmc 等) 上的空间当做内存使用。在物理内存紧缺时，将一些暂时不用的数据换出 (swap out) 到交换空间 (swap space)。交换空间细分为若干连续的槽 (slot)，每个槽的长度刚好与系统的一个页帧相同。在大多数处理器上，一个页的长度是 4KB。交换空间也称为交换区，有两个用户空间工具可用于创建和启用交换区，分别是 mkswap(用于 “格式化” 一个交换分区/文件) 和 swapon(用于启用个交换区)。
  - ***swap cache*** 是指已经写入了磁盘或外部存储的 swap 分区，但是在内存中仍然保留副本的匿名页。也就是说内存中仍保留有该页的磁盘映射。
  - ***PageSwapBacked(page)*** 判断的是否匿名页可以被换出到 swap 分区。

- 1124-1129 判断目标页是否为文件页，以及脏页状态 (*PG_dirty*) 和写回 (*PG_writeback*) 状态，并对页面的对应状态更新统计计数。

page_check_dirty_writeback() 用于检查一个文件 lru 链表上的页是否脏页，以及是否处于写回状态。该函数会排除匿名页和 lazyfree 页，只检测文件页。对于匿名页和 lazyfree 页会给传出参数 dirty、writeback 置 false。

lazyfree 是一种特殊页，它是挂在文件页 inactive 链表的匿名页。

stat->nr_dirty 脏页数量将决定节点是否被标记为" 回收拥塞 (reclaim congested)", 拥塞状态会影响是否调用 wait_iff_congested() 来阻塞等待 I/O 完成。

wait_iff_congested() 作用是：有条件的等待后备存储设备解除拥塞或内存节点完成写入操作。当后备存储设备出现拥塞时，此函数将等待最多 timeout jiffies 的时间，直到指定的 sync 队列的后备存储设备解除拥塞，或写入操作完成。若函数睡眠了完整的超时时间，则返回值为 0；否则，返回值为函数退出时剩余的 jiffies 数。若返回值等于 timeout, 则表示函数未进入睡眠状态。

stat->nr_unqueued_dirty 将会影响是否唤醒 flusher 线程对文件页异步回写。

- 1137-1141 统计拥塞页面。

  这里分两种情况, 满足下面之一，则将此页面视为拥塞页面:

  - 脏页或正在回写，mapping 有效, 目标 page 对应的存储设备处于拥塞。简言之，页面要写磁盘，但磁盘设备本身 IO 已经拥堵。
  - 页面正在排队回写 (处在 writeback IO 队列)，目标页上一轮 vmscan 扫描已经标记为待回收 (*PG_reclaim*)。

    PageReclaim() 用于检测目标页 *page->flags* 中的 PG_reclaim 标志。如果置位，表示该页待回收或应立即回收。该标志通常在隔离函数 *isolate_lru_pages()* 中调用 *SetPageReclaim()* 来设置，如是隔离出的页无法回收, 放回原 lru 链表时，不会清除该标志 (=== 待 check 放回的函数)。

    上一次隔离出来的页，因故回收失败被放回 inactive 链表。本次扫描又被隔离出来，

  页面在 LRU 链表中循环速度极快，以至于那些被标记为" 待回收 (reclaim)" 的页面第二次隔离出来,(即该页在此次遍历前已经标记了 reclaim 并处在 writeback IO 队列中了)。

  1137 page_mapping 函数的主要作用是建立物理页和文件/设备映射之间的联系。通过调用 page_mapping 函数，可以获取与某个物理页关联的 address_space 地址空间对象。该函数对于匿名页和 slab 页返回 NULL。所以，此处 if 代码块更新的是文件页的拥塞统计信息。

- 1185 检查目标页是否正在进行回写操作 (*PG_writeback*)。

  回写操作是指将内存中修改过的数据 (脏页) 写回到磁盘或其他持久化存储设备的过程。但是这里的"正在进行" 回写指的是这个页已排入 IO 回写队列。

- 1187-1194 如果本次回收是 kswapd(非直接回收) 后台执行，PG_reclaim 标志上一轮已被设置 (页已经排入回写队列,但尚未完成回写。换言之, 该标志表示立即回收), 且被隔离的页片全都处于回写状态 (判断是否已设置节点标志 ***PGDAT_WRITEBACK***)—设置节点标志 PGDAT_WRITEBACK 的操作此前已在 shrink_node() 中进行—这表明正在回收的隔离页都是处于回写状态的脏页，也意味着本 node 节点处于回写拥塞状态。

  一般正常的逻辑: 页面正在回写，不能回收，一般走 keep 分支，最终放回原链表等待 IO 完成。

  但这里是个例外，这里会将将页面最终放回 active 链表。因为: 整个节点回写拥塞严重，如果此时 kswapd 继续疯狂挑选大量脏页，那么 inactive 链表不断拿到大量正在 writeback 的页，

通过 keep 分支全部放回 inactive，下一轮又会将他们隔离出来。这样 kswapd 就会反复处理同一批 writeback 页，做无用循环。将这些页丢到活跃链表，避免了反复扫描。等后续 IO 回写完成，PGDAT_WRITEBACK 清除，内存压力上升，又会 deactive 到欠活跃链表。

这里除了这个动作外，另一个动作是增加 nr_immediate 计数。一旦内存 node 节点的标志 PGDAT_WRITEBACK 被设置，就会递增 reclaim_state 结构体中的 nr_immediate 字段统计为立即回收 (1190 行) 页，并在后续的 nr_immediate 检查中检测到相关页面时进行等待 (见 shrink_node() 函数 2817-2818，当 kswapd 检测到 reclaim_state->nr_immediate 字段非零时，会触发回收限流操作，等待回写任务完成)。也就是上层来限流。换言之，kswapd 检测到很多页面都要回收，但都在写回操作中。那就说明：磁盘写得太慢。所以必须强制暂停 kswapd，等一等磁盘 IO。(===== 这里的流程和触发可以考虑加副图) 尽管这并不理想，因为无法保证产生脏页的进程会以 balance_dirty_pages() 相同的方式被限流。

如果内存回收过程中遇到节点有大量正在进行回写操作的页面，并且当前目标页面既处于回写状态又被标记为立即回收 (PG_reclaim)，那么这表明页面正在回写队列排队等待进行 I/O 操作，但在 I/O 操作完成之前就通过 LRU 链表被重复扫描再次要被处理。因为 writeback 标志在回写 I/O 完成后会被清 0，因此 LRU 链表在之后回收扫描时又扫描到还没清 0 writeback 标志的页的话，说明了该页 I/O 操作还没完成。

对于直接回收则不做这种延后策略，因为直接回收压力紧急。

- 1194-1211 上面的代码段处理”kswapd 后台回收 + 页面已标记 PG_reclaim + 节点正处回写拥塞”；而这个代码分支处理普通情况 (不是上述大量回写场景)。

  若常规限流机制可用、或正在回写页面无需立即回收 (PG_reclaim)、或当前不可以进行文件系统操作，这三种情况之一，则将页面标记为” 立即回收”，把该页放回活跃链表。

  常规页脏页限流机制在使用传统内存控制组 memcg 时会完全失效。取而代之的是，在 shrink_page_list() 函数中使用直接阻塞的方式来进行限流，也就是在 1214 行调用 wait_on_page_writeback() 函数。不过这种方式缺少自适应等优点。而函数 wb_throttling_sane() 用于判断能否使用常规限流机制。

  全局或新内存控制组的回收过程中遇到一个未被标记为立即回收的页，或调用者不能进行文件系统 (___GFP_FS) 或存储介质 (___GFP_IO) 操作，此时将页标记为立即回收并继续扫描。

  这里存在轻微的竞态条件——end_page_writeback() 函数可能刚刚清除了 PageReclaim 标志，然后在这里重新设置了 PageReclaim 标志——但这并不值得我们过度关注。我们真正希望的是，当下一次内存控制组（memcg）回收操作进行到上述测试环节时，这个页面已经设置了 PageReclaim 标志，这样它就会调用 wait_on_page_writeback() 函数，从而避免出现内存不足 (OOM) 的情况；并且在全局回收操作中，设置该标志也是合适的。

- 1212-1217 传统的内存控制组 memcg 缺乏任何页节流机制，会因过多页面处于回写状态且无其他页面可回收，导致触发内存耗尽 (OoM)。因此扫描到已被标记为” 立即回收” 的页面时，需等待回写完成。然后回写完成的 page 重新加入目标链表，并跳出本次迭代重新扫描。

  另外，如果 kswapd 碰到已标记” 立即回收” 的页，但是当前 node 并没有太多排队等待 writeback 的页的场景，那么无需因为吞吐量考虑脱离当前扫描路径，也会在此处等待回收完成。

shrink_inactive_list() > *shrink_page_list()*

```
1221        if (!ignore_references)
1222            references = page_check_references(page, sc);
1223
1224        switch (references) {
1225        case PAGEREF_ACTIVATE:
1226            goto activate_locked;
1227        case PAGEREF_KEEP:
1228            stat->nr_ref_keep += nr_pages;
1229            goto keep_locked;
1230        case PAGEREF_RECLAIM:
1231        case PAGEREF_RECLAIM_CLEAN:
1232            ; /* try to reclaim the page below */
1233        }
```

- 1221-1222 根据 ignore_references 决定是否需要判断页面冷热度，并根据检查结果决定对目标页面采取何种操作策略，如激活页面、保留页面或尝试回收页面等。

  ignore_references 是传入参数，用于控制是否忽略页面（近期）是否被访问的信息（如 PG_referenced 和 PTE_ACCESSED 标志）。实际上是控制是否调用 page_check_references() 函数。其设置逻辑与具体的内存管理场景密切相关。一般而言，在非强制回收场景下设为 false。此时内核会利用 page_check_references() 评估页面近期是否被访问优化回收策略，保留活跃页以减少性能损耗。它是通过分析页面的 PTE 和 PG_referenced，并返回不同的枚举值来指导后续处理流程。典型场景包括：

  kswapd 后台回收：周期性扫描 LRU 链表时，需根据 PG_referenced 和 PTE_ACCESSED 标志位区分冷热页。

  直接非紧急内存回收：当内存压力未达到临界值时，优先保护活跃页，避免误回收导致的 IO 开销。

  alloc_contig_range() 函数是唯一明确将 ignore_reference 置为 true 的代码路径。其核心目标是强制回收或迁移页面以满足大块连续物理内存的需求。具体场景包括：

  CMA：为设备驱动分配连续内存时（如 DMA），需强制迁移已分配方页面，即使这些页面近期被访问过。

  内存整理：需要连续物理块，绕过引用检查可避免活跃 (active) 页干扰。

  即使 ignore_reference 为 true 的场景，也需要把脏页写回或 swap out 才能进行回收。

  page_check_references() 是内存回收中权衡 CPU 开销与性能的关键环节。它结合 PTE 硬件访问 young 位与 *PG_referenced* 标记评估页面活跃程度，保护热点工作集，避免无谓的 I/O 开销和缓存失效。在普通非强制回收场景下，如果跳过该函数，会造成活跃页面被错误回收，引发大量缺页异常，系统性能明显劣化。内核仅在 ***PG_reclaim*** 标记或内存回收拥塞这类强制回收场景，才会主动短路跳过该检测。例如，在非强制性场景下略过该函数可能会导致如下问题：

  无法区分冷热页，可能将近期访问的页面强制回收，导致后续访问时触发 page fault 和 IO 操作；活跃页被错误移至欠活跃链表或直接释放；频繁回收活跃页会导致磁盘回写和交换操作增多，加剧系统负担，导致性能劣化。

  如果无需动态判断页面引用状态，则继续往下执行进行强制回收。

page->flags 的 ***PG_referenced*** 位是内核软件标记，记录上一轮回收扫描发现该页面有被访问过。进程 PTE 的 young 位会由 MMU 硬件自动置位，CPU 访问页的时候硬件自动置标记，需要内核遍历 rmap 才能读到、软件清除开销大。它们之间简要对比如下:

标志	位置	谁设置	作用
PTE young 位 (accessed 位)	*PTE 表项*	*MMU*	硬件记录进程访问过页面
PG_referenced	*page->flags*	软件接口 *page_check_references()*	记录上一轮页面被访问过

匿名页和文件页对 PTE 的 accessed 位和 page->flags 的 PG_referenced 的使用略有不同:

文件页 第一次被访问时，会置 page->flags 的 PG_referenced 位，并保留页在欠活跃链表。第二次访问该文件页时看到 page->flags 的 PG_referenced 位已经被置位，就会判断这是第二次访问该页，于是将该页激活到活跃链表。

匿名页 只要 PTE 的 accessed 位有记录访问痕迹，直接就提升该页到活跃链表。换言之，匿名页不使用文件页所谓的" 二次机会"。但是需要注意的是，malloc() 发生 page fault 分配页后，匿名页一开始仍挂在欠活跃链表，只有程序读写该页，mmu 才会把进程的 PTE accessed 位置 1，但是该页仍在欠活跃链表。只有内存压力上来了，开始扫描和隔离页面，调用 *page_check_references()->page_referenced()* 来检查到 PTE 的 accessed 位，才把匿名页提升到活跃链表。

- 1224-1232 如果跳转到 activate_locked 标签，表示将 page 移回 active 链表，避免被回收; 如果跳转到 keep_locked 标签，则把页面放回原 lru 链表暂不回收。其中 stat->nr_ref_keep 统计保留页面的数量，用于内核调试和内存回收策略优化。其他分支代码继续执行后续逻辑（如写回脏数据、解除映射等）。switch 中对应分支枚举量含义如下表:

Table 5–2: enum page_references 枚举值含义

枚举值	含义
PAGEREF_RECLAIM	表示该页面可以被回收。当系统内存紧张时，内核会尝试将这个页面所占用的物理内存释放出来，若页面为脏页，可能需先将数据写回磁盘再回收。
PAGEREF_RECLAIM_CLEAN	同样表示页面可被回收，但强调页面是“干净”的，即页面数据与磁盘数据一致。回收时无需先将数据写回磁盘，可直接释放物理内存，提高回收效率。
PAGEREF_KEEP	表示该页面需要暂时保留，不能被回收。可能因为页面正在被回写，或包含重要的数据或状态信息，以保证系统正常运行。
PAGEREF_ACTIVATE	表示需要激活该页面。意味着目标页面要从非活跃状态转换为活跃状态，使页面在后续内存管理中更不易被回收。

继续 shrink_page_list(), 这段代码主要处理内存回收过程中的匿名页和透明大页 (Transparent Huge Page，THP。在传统的内存管理中，操作系统通常以固定大小的小页 (如 4KB) 为单位来管理内存。而透明大页技术允许操作系统将连续的小页合并成更大的页 (常见的是 2MB 或 1GB)，这种合并过程对应用程序是透明的，即应用程序无需进行任何修改就可以受益于大页的使用) 的操作，涉及交换空间分配、页面拆分等操作:

```
1235         /*
1236          * Anonymous process memory has backing store?
1237          * Try to allocate it some swap space here.
1238          * Lazyfree page could be freed directly
1239          */
1240         if (PageAnon(page) && PageSwapBacked(page)) {
1241             if (!PageSwapCache(page)) {
1242                 if (!(sc->gfp_mask & __GFP_IO))
1243                     goto keep_locked;
1244                 if (PageTransHuge(page)) {
1245                     /* cannot split THP, skip it */
1246                     if (!can_split_huge_page(page, NULL))
1247                         goto activate_locked;
1248                     /*
1249                      * Split pages without a PMD map right
1250                      * away. Chances are some or all of the
1251                      * tail pages can be freed without IO.
1252                      */
1253                     if (!compound_mapcount(page) &&
1254                         split_huge_page_to_list(page,
1255                                     page_list))
1256                         goto activate_locked;
1257                 }
1258                 if (!add_to_swap(page)) {
1259                     if (!PageTransHuge(page))
1260                         goto activate_locked_split;
1261                     /* Fallback to swap normal pages */
1262                     if (split_huge_page_to_list(page,
1263                                     page_list))
1264                         goto activate_locked;
1265 #ifdef CONFIG_TRANSPARENT_HUGEPAGE
1266                     count_vm_event(THP_SWPOUT_FALLBACK);
1267 #endif
1268                     if (!add_to_swap(page))
1269                         goto activate_locked_split;
1270                 }
1271 
1272                 may_enter_fs = true;
1273 
1274                 /* Adding to swap updated mapping */
1275                 mapping = page_mapping(page);
1276             }
1277         } else if (unlikely(PageTransHuge(page))) {
1278             /* Split file THP */
1279             if (split_huge_page_to_list(page, page_list))
1280                 goto keep_locked;
1281         }
1282 
1283         /*
1284          * THP may get split above, need minus tail pages and update
1285          * nr_pages to avoid accounting tail pages twice.
1286          *
1287          * The tail pages that are added into swap cache successfully
1288          * reach here.
1289          */
1290         if ((nr_pages > 1) && !PageTransHuge(page)) {
1291             sc->nr_scanned -= (nr_pages - 1);
1292             nr_pages = 1;
1293         }
1294 
1295                 <... ... to be continued ... ...>
```

- 1240 排除文件页和 lazyfree 页。

这里检查页面是否为匿名页，且是否支持交换 (即: 有交换空间 (swap space) 作为后备存储)。如果是匿名页但不支持交换到 swap space, 那么这种页是 lazyfree 页。

lazyfree 页是挂在文件页欠活跃链表上的匿名页，无需 swap out 到后备存储。用户空间通过 madvise(MADV_FREE)) 告诉内核：数据丢了也没关系, 不需要 swapout, 回收时直接丢弃即可。

匿名页 (如进程堆、栈内存) 的脏页需换出到交换空间 (swap space 或称为 swap 分区) 后才能回收脏页所在的物理内存。而文件页的脏页必须先将修改内容写回原始文件，才能回收内存。tmpfs 页是披着文件系统外衣的匿名页，回收手段等价与匿名页 (需要 swap)，但 LRU 冷热策略完全是文件页。

对于匿名页而言，如果没有配置 CONFIG_SWAP, 那么就永远不能换出。这样的话进程堆、栈之类的就会一直占据物理内存。下面是文件页和匿名页回收特性方面的对比。

Table 5–3: 文件页与匿名页的回收对比

回收特性	文件页	匿名页
目的位置	写回原始文件	换出到交换空间
后备存储依赖	无需交换空间	必须分配交换空间/槽
回写机制	文件系统->*writepage()*	交换子系统 *swap_writepage()*
干净页处理	直接丢弃	需保留交换项（可能复用）
失败风险	文件系统故障或只读	交换空间不足

关于 swapcache、swap 或 swap 分区, 以及 swap entry 的概念可以参考第 8 章的解释说明。

- 1241 目标页是否已 swap out 到交换分区。

  判断目标匿名页是否 swapcache 页，是通过 struct page 中 flags 字段是否已被标记 ***PG_swapcache*** 标志。此标志表明该页已被换出到交换空间 (swap)，其元数据（如交换槽索引）正由 swap entry 管理。分配 swap cache ≈ 分配交换槽 (两者绑定)。所以，此行也可以理解为：判断是否分配了交换空间(换言之，是否已经换出到后备磁盘或后备存储的 swap 分区)；

  这里和上一行代码的区别是，上一行代码是判断：是否匿名页且是否可以使用交换空间。

  需要注意的是: 正如 8的说明，只有已经换出到 swap，并且物理内存中仍在的匿名页页，才是 swapcache 页, 才会置 *PG_swapcache* 标志。

  swapcache 实际的源数据页面位于磁盘交换区或后备存储空间。

对于被多个进程共享的页面，当进程 A 的访问触发了这个页面的” 换入 (swap in)” 后，其他进程是不知道的，之后当进程 B 也试图访问这个页面时，因为它的 PTE 里存放的还是 swap space 中的 slot 号，如果它也去从 swap area（交换区，位于磁盘）中 swap in 这个页面，将造成无谓的时间消耗 (访问磁盘很慢) 和内存占用（同一份数据在内存中有多个拷贝)。解决这个问题的办法就是：两个进程共享一页被 swapout 时，内核通过 rmap 遍历清空所有 PTE, 并把 PTE 写成 swap entry，把数据写入 swap 并释放物理页。第一个进程访问时触发缺页，从 swap 读回一页并把自己的 PTE 恢复为有效映射。第二个进程访问时同样缺页，但发现页已在内存，直接把自己的 PTE 映射到该页，无需再次读 swap，最终两进程重新共享同一物理页。

swapcache 与物理内存的交互关系如下：

Table 5–4: 物理内存与 swap cache 交互机制

操作类型	数据存储位置	*swap entry* 的作用
页面换出	数据写入磁盘交换区	记录交换槽索引，避免重复分配
页面换入	数据从磁盘加载到物理内存	提供交换槽索引，协调加载流程
二次换出（未修改）	复用原交换槽的磁盘数据	减少磁盘 *I/O*，提升性能

通俗的理解 swapcache 页的意义，就是已经写入磁盘或后备存储的 swap 分区，并同时在物理内存中存有副本的匿名页。

- 1242-1243 当前回收上下文不允许磁盘或后备存储的 I/O 操作，则跳过回收转到 keep_locked 保留页面。
- 1244-1257 这个代码段主要执行透明大页相关操作。整体回收大页效率低，分割为普通页后可能无需进行 I/O 操作即可单独回收部分子页。

  1246-1247 如果不能拆分，则跳转并准备将该页面提升到活跃链表。

  1253-1256 如果可以，先判断复合页 (如透明大页) 的页表映射次数 。换言之，即是否有多个进程或线程使用此页。如果没有，表示可以安全拆分。如果有进程使用，则准备提升页面至活跃链表。

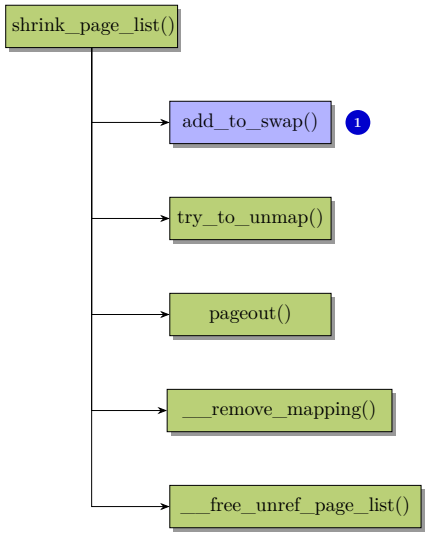
  在内核的内存管理中，为了将虚拟地址映射到物理地址，使用了多级页表结构。页中间目录 (Page Middle Directory, PMD) 是多级页表中的一级。当一个页面有 PMD 映射时，意味着它在页表中有对应的条目，用于记录该页面的映射信息；如果没有页表项，则意味着该页没有被当前进程使用。

- 1258 如果可以正常将页 swapout 到交换空间，那么就通过 add_to_swap() 函数完成该操作 (参见 5.1.13.1的解析)。

  ~~对于匿名脏页，必须将其换出到 swap 空间后才能被回收~~。很遗憾，这句话不对。匿名页没有后备存储，所以无论干净还是脏页都要 swap out 到交换分区才能回收。匿名页的 swap 映射并非在分配内存页时确立，而是在物理内存不足页面换出 (swap out) 时才建立 swap 映射。

  这里 add_to_swap() 函数所做的就是建立 swap 映射的工作: 试着分配交换空间 slot，将页面添加到后备存储的交换空间。并且会通过调用 add_to_swap_cache() 将页面的元数据 (如交换槽索引) 插入 swap entry 结构 ，并递增 nr_pages 计数器以跟踪当前交换页数量 。

  匿名页换出 (swapout) 是内存回收流程的一部分，主要流程就是在该函数 (*shrink_page_list()*) 中执行。关键执行函数如下 (省略了和 swapout 无关的执行函数等):



说明:
1. 文件页不执行 *add_to_swap()* 这一步。

- *add_to_swap()* 分配 swap 槽位,将匿名页加入 swapcache,增加引用计数,设置 *PG_swapcache* 标志。把 *swp_entry_t* 赋值给 *struct page->private*，标记页面为脏 (参见 5.1.13.1章节)。
- *try_to_unmap()* 解除目标页的所有映射。首先遍历目标页面的全部映射，然后对每一个映射递减引用计数，将页表项 (PTE) 清除 present 位，在 PTE 中存入编码 *swp_entry_t*。
- *pageout()* 对页发起异步回写，将页写入磁盘 (参见 5.1.13.8章节)。

  该函数本身的执行并不区分是文件页还是匿名页。

  在 *shrink_page_list()* 流程中，对普通文件脏页, 如果不是 kswapd (!current_is_kswapd()) 中也不会调用 *pageout()*。而是把页重新放回 lru, 由上层 *shrink_node()* 在发现大量脏页时调用 *wakeup_flusher_threads()* 让后台 flusher 去写回 (见 *shrink_page_list()*: 1325-1338 行)。

  即使是 kswapd，也还要满足：页面已带 *PG_reclaim* 标记（说明第二次扫到它），节点设置了 *PGDAT_DIRTY*, 否则同样不走到 pageout()，而是回转 lru 等 flusher 处理。
- *free_unref_page_list()* 批量释放引用计数为 0 的页，释放物理内存。

文件页的回收流程和匿名页在流程上的主要差异在于: 文件页不需要调用 *add_to_swap()*。

add_to_swap() 同时完成内存和磁盘的映射。如果失败 (如交换空间耗尽等):

- 普通页 (1259-1260): 跳转到 activate_locked_split 激活页面
- 透明大页 (1262-1269): 拆分后再次尝试分配和加入交换空间。若仍失败，激活页面。

这里激活页面是为了使其 swapout 失败时避开可回收页的回收流程。若操作成功，则可以继续执行后续步骤。

这里仅传递 head page 给 add_to_swap。当 THP 被成功拆分（1254 行）后，原透明巨页的 head page 会被降级为普通页， 所有子页（包括 head page）会被加入 page_list 链表 。内存回收操作（shrink_page_list 函数里）会逐个处理 page_list 中的页面。拆分后的 head page 会作为当前迭代的 page 参数首先传递给后续操作（add_to_swap 函数），而其他子页会在后续循环迭代中被单独处理 。

透明巨页（THP）拆分条件与映射状态的关系:

- **常规场景**

  当透明巨页存在有效映射（即 compound_mapcount > 0）时， 默认不允许直接拆分 。内核会优先保留映射关系以保证性能 。

- **强制拆分的场景**

  当系统内存严重不足时，内核会绕过映射计数检查，强制拆分 THP 以释放内存 。

1258 行可能由于是" 有效映射大于 0" 的原因进入,而又因为是" 透明巨页" 而导致 add_to_swap 执行失败。这样就会在 1261 行强制拆分，在 1269 行再次尝试换出。

- 1277 如果当前页是透明大页，则进入以下 else if 代码块的处理流程。
- 这个代码段的本义是只处理文件页，把文件页的 THP 拆成普通页，进而跳转到 keep_locked 保留。但是代码这里有个*bug*：被 1240 行 if 代码段排除在外的 lazyfree 透明匿名大页，会通过 if else 代码段执行大页拆分，从而 keep 在目标链表上。而 lazyfree 匿名页理论上是可以直接 free 的。
- 1279-1280 尝试将该透明大页拆分为小页。如果拆分成功，则跳转到 keep_locked 标签处，保留该页面。

shrink_inactive_list() ▸ *shrink_page_list()*

```
1295        /*
1296         * The page is mapped into the page tables of one or more
1297         * processes. Try to unmap it here.
1298         */
1299        if (page_mapped(page)) {
1300            enum ttu_flags flags = ttu_flags | TTU_BATCH_FLUSH;
1301            bool was_swapbacked = PageSwapBacked(page);
1302
1303            if (unlikely(PageTransHuge(page)))
1304                flags |= TTU_SPLIT_HUGE_PMD;
1305
1306            if (!try_to_unmap(page, flags)) {
1307                stat->nr_unmap_fail += nr_pages;
1308                if (!was_swapbacked && PageSwapBacked(page))
1309                    stat->nr_lazyfree_fail += nr_pages;
1310                goto activate_locked;
1311            }
1312        }
1313
1314                <... ... to be continued ... ...>
```

- 1299 如果页存在用户空间映射，那么在执行回收前必须彻底解除映射。此处检查页面是否被映射到至少一个进程的页表 (PTE) 中。它是通过检查 page->_mapcount 字段来判断的，这个字段用来记录页表映射次数。_mapcount >= 0，表示至少有一个进程的页表项 (PTE) 引用该页。

  page_mapped(page) 只判断是否有进程映射该页，该函数一丝一毫都不能判断目标页是文件页还是匿名页。另一个与此名称相似的函数 (宏)**page_mapping()** 很容易与 **page_mapped()** 混淆。

  page_mapping() 获取与文件页面 (struct page) 关联的地址空间 (address_space)，若属于交换缓存 swapcache，返回 swap space 的 swapper_spaces。如果是匿名页或 slab 页则返回 NULL。

page->**count** 和 page->**__mapcount** 看上去也很相似，它们的区别是：前者表示页面的总引用计数，后者页表项 (PTE) 对页面的映射次数。

- 1306 利用函数 *try_to_unmap()* 遍历所有映射该页片的 struct vm_area_struct（VMA）对象，并解除它们与页面的映射关系。

  在 swapout 场景下，*try_to_unmap()* 会向 PTE 中填入 swap_entry_t, 以此告知内核: 后续如果访问这些映射触发缺页异常时，从 swap 分区执行 swapin(换入) 操作。

- 成功解除映射后，页面是否仍为脏页取决于是否设置了强制写回的标志TTU_FLUSH。这里保持脏页原状态，后续再处理 (见后面 1314 行判断脏页，1352 对脏页 flush)。而TTU_BATCH_FLUSH标志仅优化 TLB 刷新逻辑，不涉及脏页的写回或状态修改。

- 1307-1310 如果调用 try_to_unmap() 解除映射操作失败，系统会更新对应统计信息，激活该页，使其避开回收流程。

shrink_inactive_list() ▸ *shrink_page_list()*

```
1314        if (PageDirty(page)) {
1315            /*
1316             * Only kswapd can writeback filesystem pages
1317             * to avoid risk of stack overflow. But avoid
1318             * injecting inefficient single-page IO into
1319             * flusher writeback as much as possible: only
1320             * write pages when we've encountered many
1321             * dirty pages, and when we've already scanned
1322             * the rest of the LRU for clean pages and see
1323             * the same dirty pages again (PageReclaim).
1324             */
1325            if (page_is_file_lru(page) &&
1326                (!current_is_kswapd() || !PageReclaim(page) ||
1327                 !test_bit(PGDAT_DIRTY, &pgdat->flags))) {
1328                /*
1329                 * Immediately reclaim when written back.
1330                 * Similar in principal to deactivate_page()
1331                 * except we already have the page isolated
1332                 * and know it's dirty
1333                 */
1334                inc_node_page_state(page, NR_VMSCAN_IMMEDIATE);
1335                SetPageReclaim(page);
1336
1337                goto activate_locked;
1338            }
1339
1340            if (references == PAGEREF_RECLAIM_CLEAN)
1341                goto keep_locked;
1342            if (!may_enter_fs)
1343                goto keep_locked;
1344            if (!sc->may_writepage)
1345                goto keep_locked;
1346
1347            /*
1348             * Page is dirty. Flush the TLB if a writable entry
1349             * potentially exists to avoid CPU writes after IO
1350             * starts and then write it out here.
1351             */
1352            try_to_unmap_flush_dirty();
1353            switch (pageout(page, mapping)) {
```

```
1354            case PAGE_KEEP:
1355                goto keep_locked;
1356            case PAGE_ACTIVATE:
1357                goto activate_locked;
1358            case PAGE_SUCCESS:
1359                stat->nr_pageout += thp_nr_pages(page);
1360
1361                if (PageWriteback(page))
1362                    goto keep;
1363                if (PageDirty(page))
1364                    goto keep;
1365
1366                /*
1367                 * A synchronous write - probably a ramdisk.  Go
1368                 * ahead and try to reclaim the page.
1369                 */
1370                if (!trylock_page(page))
1371                    goto keep;
1372                if (PageDirty(page) || PageWriteback(page))
1373                    goto keep_locked;
1374                mapping = page_mapping(page);
1375            case PAGE_CLEAN:
1376                ; /* try to free the page below */
1377            }
1378        }
1379
1380                <... ... to be continued ......>
```

- 1325-1327 。

  内核中存在两种不同的触发执行 writeback 的方式。一种是内核线程 kswapd 在后台执行 writeback，也就是异步方式。另一种 writeback 以“直接回收”的形式执行，也就是同步方式。

  所谓“直接回收”就是在分配内存时发现空闲页数量不足，内核会尝试直接在请求内存的进程的上下文中回收页面。这种从分配内存转变为对内存进行清理的活动称为“直接回收”。一旦释放了足够数量的内存，分配器将再次尝试寻找空闲内存进行分配 (当然前提是它刚刚释放的页框没有被其他人又占用)。

  直接回收可以在内核中的几乎任何地方发生，因此很有可能在回收开始之前内核栈已经被用得差不多了。如果回收还涉及对文件系统执行 writeback 的 I/O，则该动作本身的函数调用栈就会很深，最终有可能导致内核栈溢出。除此之外，由于直接回收会尽其可能对页框进行回收，往往会在 I/O 上进行频繁的写，从而损害整个系统的 I/O 性能。

  因此，此处代码逻辑是：首先，如果“脏”页是匿名页，则 writeback 的处理还是和以前一样。原因是因为大部分的匿名页将被写回交换区，这比映射文件的物理页的处理逻辑简单；

  如果“脏”页是文件页，在直接回收处理过程中将不执行直接写回到磁盘的操作。而是将符合条件的页框标记为 *PG_reclaim* 并放在活跃 LRU 中留给 kswapd 来回收。

  间接回收 kswapd 才会处理脏的文件页，目的是避免内核栈过深导致栈溢出。由页面分配引发的直接回收过程中，我们无法确定当前内核栈的剩余大小；而 kswapd 内核进程的执行层次则是明确的。只允许 kswapd 来进行文件页的回写来可以有效避免栈溢出，因为 kswapd 有自己单独的内核栈，并且在处理 writeback 之前它的内核栈基本上是空的。

  此外，除了目标页是文件脏页，并且当前是 kswapd 线程外。在文件脏页满足以下条件之一时，也会执行脏文件页的回写 (1326-1327)：

该页尚未被标记 ***PG_reclaim***(即此前并未跳过该页)。

节点标志 ***PGDAT_DIRTY*** 已被设置，表示本次扫描到的所有页都是脏页，且没有任何一个干净页，内存节点存在显著脏页压力。

否则，会给文件页设置 PG_reclaim，确保在回写完成后将其调度回收。

- 1328-1337 通常情况下内核在检测到目标页面“再一次”被访问时，会立即将该页面移到 active 链表，从而更高效地将“一次性使用”页面保留在欠活跃 (inactive) 链表中。一般情况下这一设计合理且高效。但是在如下场景就会产生问题：

  当大部分被缓存的页面会被再次访问 (refaults)(如反复读取中间计算结果) 移到 active 链表，同时流式写入带来了大量”一次性使用” 的缓存页面。这样就会导致 inactive 链表中大部分是脏页，回收效率低下 (脏页需回写后才能释放)。只要 inactive 链表中存在一次性使用的脏页面，系统就不会主动回收 active 链表中的干净缓存。

  但，为避免回收那些可能被“再次访问”的干净 (clean) 缓存而延迟回收它们，从而优先处理需要写回的脏页，并且等待写回操作完成，这是一种糟糕的权衡。即使确实会发生“再次访问”，读取操作的速度也远快于写回操作。在陷入写回阻塞之前，内存回收机制应优先考虑系统的所有干净缓存——包括 active 链表中的干净缓存。

  为此，当脏页（dirty pages）或正在回写（under writeback）的页面到达 inactive LRU 链表的末尾时，激活这些页面到活跃链表，并标记为立即回收，通过设置页面标志位（如 PG_active）将页面从 inactive 列表移动到 active 列表，使其暂时避开回收流程，内核通常避免直接回收 active 列表中的页面，但在极端内存压力下会强制扫描并补充到非活跃列表，这意味着它们一旦完成回写并变为可回收状态，将立即被移回欠活跃链表的尾部。在此过程中，通过将 inactive 链表精简为仅包含可直接回收的页面，我们允许内存扫描器将 active 链表尾部的干净缓存页降级（deactivate）到 inactive 链表中，从而保证内存回收的持续执行。

  综上，代码会如下处理：

  - 限制普通进程直接触发文件页回写。利用 kswapd 线程执行文件系统回写操作，防止栈溢出风险（文件系统回写可能触发复杂 I/O 路径）。
  - 避免低效的单页回写，仅在满足以下条件时允许回写：

    页面已被多次扫描（PageReclaim(page) 标记）

    内存节点存在显著脏页压力（PGDAT_DIRTY 标志）。

- 1337 将 1222 行返回的指示为 PAGEREF_RECLAIM_CLEAN(回写后可回收) 的页面放到 active 链表。延迟回写，等待后续扫描时可能积累更多脏页了再批量回写。

- 1340-1344 回写权限检查。

  - 1340 干净页直接保留。
  - 1342 此时上下文如果禁止访问文件系统 (如原子上下文)，保留页。
  - 1344 回收策略禁止写回 (如快速回收模式)。

- 1352-1377 这个代码段处理脏页的换出操作。

  对于匿名页，脏页的来源有两种情况：

- 该匿名页曾被写入，*try_to_unmap()* 会置位 PTE 脏位以及 page 自身的脏标志。
- *add_to_swap()* 将页面标记为脏。
- 1352 刷新 TLB，确保所有 CPU 核看到页面的最新状态。try_to_unmap_flush_dirty() 确保在开始执行 I/O 写回之前根据需要刷新 TLB，避免因 TLB 残留映射导致页被不必要地标记为脏。。

  try_to_unmap 和 try_to_unmap_flush_dirty 的区别在于，前者遍历所有映射到目标物理页的进程页表，清除对应的 PTE(页表项), 标记页面为" 未映射", 但不立即刷新 TLB, 而是通过批量刷新机制 (如 flush_tlb_batched_pending) 在后续统一处理，减少频繁刷新的性能开销。后者在解除映射后同步触发 TLB 刷新 (如调用 flush_tlb_all 或 flush_tlb_mm_range), 确保所有 CPU 核的 TLB 立即 invalid。
- 1353 将脏页内容写出到磁盘或后备存储。其返回值决定后续操作。
- 1354 ***PAGE_KEEP*** 回写尚未完成（页仍处于锁定状态)。保留页面, 将页轮转至当前 LRU 链表尾部。
- 1356 ***PAGE_ACTIVATE*** 需要将页面标记为 active。应将页移至 active LRU 链表 (页仍处于锁定状态)。
- 1358 ***PAGE_SUCCESS*** 表示页片已提交回写 (大概率是异步回写)，还需要做一些额外处理。在尝试加锁并在锁保护下再次检查之前，我们会再次确认 *PG_writeback* 和 *PG_dirty* 已被清除。如果页是脏页、正在回写，或者无法加锁，就将页轮转 (即加到当前 LRU 链表尾部).

  1359 更新统计计数器。

  1361-1363 检查页面是否脏页或在写回队列，若是则保留。

  1370 尝试重新锁定页面，若失败则保留。

  1372 若页面已干净且无写回，更新 mapping 信息，继续后续回收流程。

  大部分情况下, pageout() 不会主动释放页锁。但是也有例外情况，譬如在 pageout() 的 I/O 完成回调中 (例如 end_page_writeback()) 会发生锁的释放。ramdisk 的同步回写就是这种场景。

  1361，1363 行和 1372 行执行的函数相同。这里 1361 和 1363 行是为了 early return, 如果执行失败会保留页面。但是如果执行成功，有可能是在释放了锁的情况下 page 状态发生改变，所以会在 1370 行申请了锁的情况下，再次执行相同的判断操作，进行 double check。
- 1375 ***PAGE_CLEAN*** 页面已是干净页 (无需写出)，直接进入后续释放流程。

shrink_inactive_list() ▸ *shrink_page_list()*

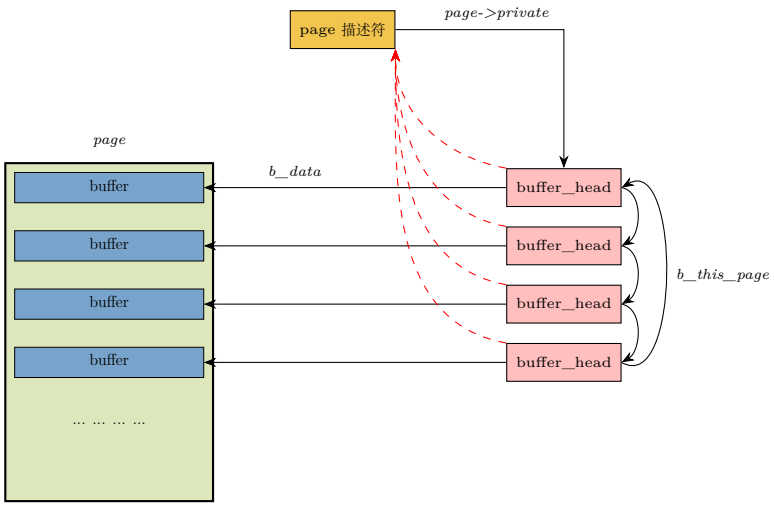
```
1380         /*
1381          * If the page has buffers, try to free the buffer mappings
1382          * associated with this page. If we succeed we try to free
1383          * the page as well.
1384          *
1385          * We do this even if the page is PageDirty().
1386          * try_to_release_page() does not perform I/O, but it is
1387          * possible for a page to have PageDirty set, but it is actually
1388          * clean (all its buffers are clean).  This happens if the
1389          * buffers were written out directly, with submit_bh(). ext3
1390          * will do this, as well as the blockdev mapping.
1391          * try_to_release_page() will discover that cleanness and will
1392          * drop the buffers and mark the page clean - it can be freed.
1393          *
```

```
1394            * Rarely, pages can have buffers and no ->mapping.  These are
1395            * the pages which were not successfully invalidated in
1396            * truncate_complete_page().  We try to drop those buffers here
1397            * and if that worked, and the page is no longer mapped into
1398            * process address space (page_count == 1) it can be freed.
1399            * Otherwise, leave the page on the LRU so it is swappable.
1400            */
1401           if (page_has_private(page)) {
1402               if (!try_to_release_page(page, sc->gfp_mask))
1403                   goto activate_locked;
1404               if (!mapping && page_count(page) == 1) {
1405                   unlock_page(page);
1406                   if (put_page_testzero(page))
1407                       goto free_it;
1408                   else {
1409                       /*
1410                        * rare race with speculative reference.
1411                        * the speculative reference will free
1412                        * this page shortly, so we may
1413                        * increment nr_reclaimed here (and
1414                        * leave it off the LRU).
1415                        */
1416                       nr_reclaimed++;
1417                       continue;
1418                   }
1419               }
1420           }
1421
1422                   <... ... to be continued ... ...>
```

linux 为了加速读写速度，采用了 page cache 机制，在内存中按 page 来缓存磁盘或后备存储上的数据，但是又因为文件系统是按“块”为单位来操作磁盘的，所以内核又把页按块来管理，页中的每一“块”大小的数据被称为 buffer(缓冲区)。换言之，此处的”buffer” 是专有名词，特指磁盘或后备存储上一”块”小数据在内存中的表示。

内核使用 buffer_head 这个数据结构来描述“块 (block)”和“buffer”的关系，buffer_head 是连接 page 和磁盘块的关键。一个 page 可能会管理多个 buffer_head，一个 buffer_head 对应一个磁盘块，buffer_head 中会保存对应的磁盘号。

整体示意图如下:



可以看到上图中 page 描述符 (struct page) 并没有成员指针直接指向物理页框。内核中通过

struct page 找到对应的物理页，本质上是通过计算页帧号 (PFN) 和物理地址的映射关系实现的。

linux 中物理内存被划分为固定大小的页 (通常大小是 4KB), 每个物理页的唯一编号即为 PFN。通过 $PFN << PAGE_SHIFT$ (即 $PFN \times 4096$) 可得到物理页的首地址。

(1) 内核提供了以下宏完成 struct page 到 PFN 的转换:

- *page_to_pfn(struct page *page)*: 将 struct page 换算成对应的 PFN。
- *pfn_to_page(unsigned long pfn)*: 反向操作，从 PFN 找到对应的 struct page。

转换的原理如下:

- **平坦内存模型**: 所有 struct page 存储在全局数组 mem_map 中，通过计算 page 指针与 mem_map 的偏移量得到 PFN:

$$PFN = \frac{addr(page) - addr(mem_map)}{sizeof(mem_map[0])}$$

即,

$$PFN = \frac{\text{page 结构体距 mem_map 数组首地址的距离}}{\text{一个数组成员的 size}}$$

- **稀疏内存模型**: 如果物理内存被划分为多个 mem_section, 那么每个 mem_section 管理一段连续物理页的 struct page 数组。需要先定位 page 所属的 mem_section 再计算偏移。

(2) 从 PFN 到物理地址的转换可以通过以下宏:

```
#define __pfn_to_phys(pfn)  PFN_PHYS(pfn)
#define PFN_PHYS(x) ((phys_addr_t)(x) << PAGE_SHIFT)
```

(3) 对特殊情况的处理:

- **复合页**:

若 struct page 是复合页的尾部页 (tail page), 需通过compound_head()获取头部页 (head page) 后再计算 PFN。

- **设备内存 *(ZONE_DEVICE)***:

设备内存的 struct page 可能指向 struct dev_pagemap, 需要通过 pgmap 字段获取设备内存的物理地址偏移。

- 1401 通过***PG_private***标志位，判断目标 page 是否持有附属资源。有私有附属资源，不能直接回收，必须先释放附属资源。没有这个标记，直接跳过整块逻辑。

private 是 struct page 中的一个类型字段，用于存储与当前页面状态相关的私有数据。其具体含义由页面的 page_type(page->page_type) 决定。

但是 ***PG_private***(*page->flags*) 标志被置位，才代表 private 内容合法有效。没有该标志，*page->private* 无意义，不能读取。虽然 *page->page_type* 和 *page->private* 是 struct page 中的不同成员，但是可以统一通过接口宏 PageXxx(page) 来检测标志位或类型。譬如 PageBuddy(page)。

struct page 的 private 字段在不同场景下的意义列表如下:

Table 5–5: `struct page->private` 的用途

场景	*->page_type*	*->private* 存储的内容	说明
文件页	*PG_private*	*struct buffer_head*	页面挂载 *buffer_head,ext4* 等块文件使用
swap cache 页	*PG_private*	*swap_entry_t(*编码进 *unsigned long)*	匿名页换出，*->private* 保存 *swap slot*。未分配 *swap slot* 时不置位

- 1402 尝试释放目标 page 绑定的附属资源。

  可能会出现以下情况: 尽管页面被标记为脏，但实际上可能已经是干净页面 (所有数据均已被写入磁盘)。这种情况在某些文件系统（如 ext3）或块设备映射中发生，它们直接通过 submit_bh() 提交缓冲区写回操作而未更新页面的脏标记。try_to_release_page() 会检测到 clean 状态，丢弃缓冲区并将页面标记为 clean，此时可以释放该页面。

  在极少数情况下，页面可能仍有缓冲区但未被任何进程或文件系统映射 (->mapping 为空)。这通常是 truncate_complete_page() 未能成功 invalid 页面，该函数负责将文件中超过新长度部分的对应页面从内核缓存 (页面缓存，buffer 缓存) 中失效 (invalid)，并回收相关资源, 将目标页面从文件的->mapping（映射关系）中移除, 清除页面脏标记，释放与页面关联的缓冲区，最终将页面标记为可回收。但如果在 truncate 执行过程中，其它进程正在读取/写入该页面，会导致页面的引用计数无法降为 0，从而 truncate_complete_page() 无法彻底 invalid。

  内核中 *page->mapping* 是 struct page 中的一个关键字段，可以用于标识页所属的地址空间类型。通过检查 *page->mapping* 的 bit[0] 的值，可以间接判断该页是文件页还是匿名页。内核会通过指针复用和掩码的技巧将不同类型的地址空间信息编码到这一字段中。在 linux 内核中通常地址是 4 字节，无论 32bit 还是 64bit 的 arm 平台上指针值的最低两位为 0，因此 bit[1:0] 可以挪作它用。换言之，page->mapping 的值的最低位 bit 0 可以用来表示页的类型。

  - 如果 *page->mapping*的最低位为 0，则代表它是一个合法的 struct address_space* 指针，表示该页面受 address_space 管理，可以是普通文件 pagecache 页，也可以是 shmem/tmpfs 页面。如果是 pagecache 页，此时 *page->private*指向 buffer_head。并由 *PG_private* 标志指示有效。tmpfs/shmem 是内存文件系统，没有后端块设备，数据存物理内存，回写直接落到 swap 分区，不走 block layer，完全不需要 buffer_head 这套块缓冲结构。
  - 如果 page->mapping 的最低位为 1，则表示该页是匿名页。且掩码处理最低位后 mapping 指向 struct anon_vma。

- 1404 若页面没有映射且仅剩自身引用 (page_count==1, 无其他用户持有), 则可直接释放页面。否则把页面保留在 lru 列表中：通过 swap 逐步回收，避免因强制释放导致数据不一致。
- 1405 先解锁 page 锁。page 锁保护页的状态，释放前要解锁。
- 1406-1418 递减 page 引用计数。

  如果递减之后计数 == 0，则返回 true，跳转 free_it 把 page 释放回 buddy 系统，回收成功; 如果递减之后引用计数不为 0，进入 else 分支, 代表发生了竞争。

接下来处理释放 lazy-free 页的场景，将页从其所属的映射中移除。

```
1422        if (PageAnon(page) && !PageSwapBacked(page)) {
1423            /* follow __remove_mapping for reference */
1424            if (!page_ref_freeze(page, 1))
1425                goto keep_locked;
1426            if (PageDirty(page)) {
1427                page_ref_unfreeze(page, 1);
1428                goto keep_locked;
1429            }
1430
1431            count_vm_event(PGLAZYFREED);
1432            count_memcg_page_event(page, PGLAZYFREED);
1433        } else if (!mapping || !__remove_mapping(mapping, page, true,
1434                                                 sc->target_mem_cgroup))
1435            goto keep_locked;
1436
1437                <... ... to be continued ... ...>
```

- 1422 满足该行条件的页面是 lazyfree 匿名页。其本质是用户态通过 *madvise(MADV_FREE)* 标记的匿名页。用户层调用 madvise(MADV_FREE) 后，内核会调用到 *mark_page_lazyfree()* 来清除 PG_swapbacked 标志。此时页面仍保留物理内存，但被移除 active LRU 文件链表，进入欠活跃文件页链表。

  如果目标 page 的 flags 字段未设置 PG_swapbacked 标志，代表不支持使用 swap 空间做为后备内存，所以内存紧张时不会被交换到磁盘，而是直接回收物理内存。但若内存充足则保留物理页。下面是各种页 swap 行为的对比：

Table 5–6: lazyfree 页、匿名页和 shmem 页的 swap 行为对比

特性	匿名页	*shmem* 页	*lazyfree* 页
swap 行为	可被交换到 *swap cache*	可被交换到 *swap cache*	内存紧张时回收，不 *swap*
PageSwapBacked	是	是	否
PageAnon	是	否	是
典型场景	普通堆/栈内存分配	共享内存/*tmpfs*	*MADV_FREE* 标记的内存

- 1424-1425 冻结 (freeze)该页，也就是原子地将其预期的引用计数从 1 替换为 0。这样，当在后续代码释放该页时，无需执行任何其他操作即可直接释放。
- 1433 不涉及 lazyfree, 则检查该页是否有映射。如果存在，通过 ___remove_mapping() 函数将该页映射移除。如果移除映射操作失败，或是 lazyfree 的冻结操作失败，保留该页片并将其轮转至链表尾部即可。

shrink_inactive_list() > *shrink_page_list()*

```
1437        unlock_page(page);
1438free_it:
1439        /*
1440         * THP may get swapped out in a whole, need account
1441         * all base pages.
1442         */
1443        nr_reclaimed += nr_pages;
```

```
1444
1445            /*
1446             * Is there need to periodically free_page_list? It would
1447             * appear not as the counts should be low
1448             */
1449            if (unlikely(PageTransHuge(page)))
1450                destroy_compound_page(page);
1451            else
1452                list_add(&page->lru, &free_pages);
1453            continue;
1454
1455    activate_locked_split:
1456            /*
1457             * The tail pages that are failed to add into swap cache
1458             * reach here.  Fixup nr_scanned and nr_pages.
1459             */
1460            if (nr_pages > 1) {
1461                sc->nr_scanned -= (nr_pages - 1);
1462                nr_pages = 1;
1463            }
1464    activate_locked:
1465            /* Not a candidate for swapping, so reclaim swap space. */
1466            if (PageSwapCache(page) && (mem_cgroup_swap_full(page) ||
1467                            PageMlocked(page)))
1468                try_to_free_swap(page);
1469            VM_BUG_ON_PAGE(PageActive(page), page);
1470            if (!PageMlocked(page)) {
1471                int type = page_is_file_lru(page);
1472                SetPageActive(page);
1473                stat->nr_activate[type] += nr_pages;
1474                count_memcg_page_event(page, PGACTIVATE);
1475            }
1476    keep_locked:
1477            unlock_page(page);
1478    keep:
1479            list_add(&page->lru, &ret_pages);
1480            VM_BUG_ON_PAGE(PageLRU(page) || PageUnevictable(page), page);
1481        }
1482
1483        pgactivate = stat->nr_activate[0] + stat->nr_activate[1];
1484
1485        mem_cgroup_uncharge_list(&free_pages);
1486        try_to_unmap_flush();
1487        free_unref_page_list(&free_pages);
1488
1489        list_splice(&ret_pages, page_list);
1490        count_vm_events(PGACTIVATE, pgactivate);
1491
1492        return nr_reclaimed;
1493    }
```

- 1437-1453 处理已满足回收条件、可以释放页的成功场景。首先将页面解锁，增加已回收计数 nr_reclaimed 返回给调用者，然后将该页添加到 free_list 链表中以便批量释放，之后继续循环处理下一页。

  1443 增加可回收页面的统计计数。普通页的 nr_pages 为 1。如果是透明大页 (THP) 则需要统计所有单页 (head page + tail pages) 的数目。

  1449-1450 如果是透明大页，则直接通过 destroy_conpound_page() 处理整个复合页的释放。

  1452 普通页添加到 free_pages 链表暂存，待对目标 LRU 链表的循环遍历结束后再统一批量释放。

- 1455-1463 当大页成功拆分, 但由于 swap space 空间不足、并发操作冲突等种种原因, 导致 head page 未能成功加入 (交换) 到 space cache 时 (见 1260-1269 行), 控制流跳转到此标签。对已扫描页数和当前扫描页数进行修正, 确保后续逻辑正确。内核预期扫描 nr_pages 个页面, 在已成功拆分的前提下各单页将加入到目标链表。所以失败的情况下将只处理 head page, 其余 tail pages 将在后续循环继续处理。

  其实, 成功拆分后的 THP 页的 head page 无论是否成功换出到 space cache, 都会进行上面的统计数的修正。这是因为, 拆分后的处理都是针对目标单页的。成功交换出 head page 后的类似修正行为见 1290-1293 行代码。

- 1464-1475 这段代码主要将因故未能回收的 page 加到活跃 lru 链表。

  我们会检查以下两种情况:

  该页片位于交换缓存中, 且交换空间已满 (通过 mem_cgroup_swap_full() 判断, 即使不使用 cgroup, 该函数也能正确判断交换空间是否耗尽);

  该页片被 mlock() 锁定, 因此不允许换出。

  只要满足任一情况, 我们就会通过 try_to_free_swap() 释放相关的交换缓存空间

  - 1466-1467 检查页面是否在交换缓存, 也就是 swap slot 中。并且满足以下条件之一: 内存控制组的 Swap 空间已满, 或页面被 mlock 锁定不可换出。

    页面在交换空间 (space space) 同时又锁定在内存不可换出, 这两种看似矛盾的条件同时存在的场景是: 页面在 mlock 调用前已被换出到交换空间, 后续调用 mlock 时, 内核将页面换入内存并锁定, 但未立即清理交换缓存中的残留条目。

    此处无需考虑 page 是否脏页的情况, 因为在前面 if(PageDirty(page)) 开始的代码段已经处理了脏页情况。

    此处 try_to_free_swap() 仅释放交换槽 (可能伴随交换缓存的清除), 不涉及物理内存的换出和释放, 物理内存中的页面会保留且数据完整; 形式与此相似的函数 try_to_swap_out() 则会将物理内存换出的交换分区, 分配交换槽并释放物理内存。

  - 1468 释放页面。对于有 swap slot 的页面会清理 swap 条目对应的后备存储, 而不会影响 mlock 的内存页面。

  - 1470-1474 再次检查确认页片未被 mlock 锁定 (例如页面在解锁期间被设置了锁定标记), 如果已被锁定, 激活操作就没有实际意义。若页面未被锁定, 就设置 PG_active 标志。并且无论它是匿名页还是文件页, 都会被放入 active LRU 链表。

- 1476-1479 无论是保留还是激活一个已锁定的页, 都会先将其解锁再继续执行; 通常情况下, 无论激活还是保留页面, 都会将其添加到 ret_pages 链表中。至此, 循环执行完毕, 输入的 page_list 链表会变为空。释放 page, 并将页加到临时的不可回收链表。

- 1486 调用 try_to_unmap_flush 执行批量 TLB 刷新, 然后再释放这些页。这样做主要是为了避免文件页上发生任何可能导致其被不必要地重新标记为脏的写入操作。

- 1487 将 free_list 上引用计数为 0 的页片批量释放给物理内存分配器.

- 1489 把之前存储在 ret_pages 中的所有需要保留 (即轮转) 或激活的页放回作为输入参数的 page_list 中, 以便上层调用函数将这些页重新放回到对应的 LRU 链表中。

- 1492 返回本次操作中回收的基页 (order==0) 的数量

5.1.13.1 add_to_swap()

add_to_swap() 函数给目标页分配 swap 空间。

shrink_inactive_list() > *shrink_page_list()* > *add_to_swap()*

```
216 int add_to_swap(struct page *page)
217 {
218     swp_entry_t entry;
219     int err;
220 
221     VM_BUG_ON_PAGE(!PageLocked(page), page);
222     VM_BUG_ON_PAGE(!PageUptodate(page), page);
223 
224     entry = get_swap_page(page);
225     if (!entry.val)
226         return 0;
227 
228     /*
229      * XArray node allocations from PF_MEMALLOC contexts could
230      * completely exhaust the page allocator. __GFP_NOMEMALLOC
231      * stops emergency reserves from being allocated.
232      *
233      * TODO: this could cause a theoretical memory reclaim
234      * deadlock in the swap out path.
235      */
236     /*
237      * Add it to the swap cache.
238      */
239     err = add_to_swap_cache(page, entry,
240             __GFP_HIGH|__GFP_NOMEMALLOC|__GFP_NOWARN, NULL);
241     if (err)
242         /*
243          * add_to_swap_cache() doesn't return -EEXIST, so we can safely
244          * clear SWAP_HAS_CACHE flag.
245          */
246         goto fail;
247     /*
248      * Normally the page will be dirtied in unmap because its pte should be
249      * dirty. A special case is MADV_FREE page. The page's pte could have
250      * dirty bit cleared but the page's SwapBacked bit is still set because
251      * clearing the dirty bit and SwapBacked bit has no lock protected. For
252      * such page, unmap will not set dirty bit for it, so page reclaim will
253      * not write the page out. This can cause data corruption when the page
254      * is swap in later. Always setting the dirty bit for the page solves
255      * the problem.
256      */
257     set_page_dirty(page);
258 
259     return 1;
260 
261 fail:
262     put_swap_page(page, entry);
263     return 0;
264 }
```

- 221 检查页面的 page->flags 是否置位了 PG_locked 位。也就是检查目标 page 是否已上锁，否，就直接 crash，这是为了防止对目标页的并发操作。进入当前代码位置时，前置条件是：该 page 必须已经被锁住。
- 222 同上，检查 PG_uptodate 位是否置位。确保页面数据是完整可用的。

  PG_uptodate 是 struct page->flags 中的比特位，代表页缓存的数据有效、与后端存储（磁盘）

同步完成 (页内每一字节，至少不旧于磁盘上对应位置的数据。)，可以直接读取，不需要再发起读 IO。它和 *PG_dirty* 的区别如下:

宏	意义
PG_uptodate	内存页的数据由磁盘正确加载完，可以读。脏页、干净页都可以置这个位。
PG_dirty	内存页被修改，内存版本比磁盘新，需要回写磁盘。

脏页可以同时置 *PG_uptodate* 位。从磁盘读到 page(置 *PG_uptodate*)，然后 write 修改页，标记 *PG_dirty*。此时：内存既有完整有效数据 (uptodate)，又比磁盘新 (dirty)。注意: 写操作不会清 *PG_uptodate* 标记。

- 224-226 从 swap 分区 (swap 文件) 里分配一个空闲的 swap slot(**页槽**)，并返回对应的 swap entry(swp_entry_t)。它会编码出对应的 swap 文件编号以及目标 entry 在 swap 介质内的偏移。如果失败，返回 0。

  该函数会优先使用全局的 per-CPU swap slot 缓存数组，优先从该缓存对应的预分配的 swap slot 数组中分配，避免直接从底层 swap 介质 (swap 文件或 swap 分区) 分配 swap slot(编码成 swp_entry_t)。

- 239-246 把目标匿名页和一个已分配的 swap slot(槽) 绑定，失败则直接跳转 fail 处处理。这里传入的内存分配标识意味着:

  - ***___GFP_HIGH***：可以用紧急预留内存，保证 swap 路径能拿到内存。

    正常分配时，空闲页 $<$ *watermark*[$WMARK_MIN$] 分配直接失败，不能碰紧急预留内存。带上 *___GFP_HIGH* 后，空闲页可以下探到低于 *WMARK_MIN*，消耗系统那一小片应急保留页。

    ___GFP_HIGH 另一功能是指示分配失败时：禁止直接回收、禁止 OoM killer，直接返回 NULL。

  - ***___GFP_NOMEMALLOC***：禁用 PF_MEMALLOC 全局 reserve 内存（防止死锁)。

    每个 zone 都有水位：high → low → min。低于 min 水位的空闲内存就是全局紧急 reserve (系统最后保命内存)，大小由 */proc/sys/vm/min_free_kbytes* 决定。

  - ***___GFP_NOWARN***：失败不打印

- 257 匿名页刚分配出来时，还没写数据，*PTE dirty* $= 0$，该匿名页之前不是 swap cache 页，现在第一次给它分配了 swap 页槽位。所以内存里该页肯定比 swap 分区里的内容新。所以对 swap 来说目标匿名页天然是脏页。因此置脏页标志 *PTE dirty* $= 1$。

- 262 释放前面 *get_swap_page()* 分配出来的 swap 槽位，归还到 swap 空闲槽池。

  get_swap_page() 把目标页加入 swap cache，并将 swap entry 记录到 page->index(把 type + offset 的完整编码存在 index 里)，完成物理页与交换槽 (slot) 的绑定。

5.1.13.2 get_swap_page()

get_swap_page() 会分配一个 swap 条目 (swp_entry_t)。该函数会使用全局的 per-cpu 交换槽缓存，优先尝试从静态变量 *swp_slots* 维护的 swap-slot 缓存数组中分配槽位，而不是直接从底层的交换介质（文件或分区）中实际分配交换条目。如果分配成功，返回的 swp_entry_t 值会对具体的交换文件及其内部偏移量进行编码。如果没有足够的空闲空间分配交换空间，则返回 false。

当交换槽缓存需要填充时，会调用 *get_swap_pages()*，从可用的 swap 文件/分区中获取 swap slot 再编码成 swap_entry_t。

shrink_inactive_list() > *shrink_page_list()* > *add_to_swap()* > *get_swap_page()*

```
306 swp_entry_t get_swap_page(struct page *page)
307 {
308     swp_entry_t entry;
309     struct swap_slots_cache *cache;
310 
311     entry.val = 0;
312 
313     if (PageTransHuge(page)) {
314         if (IS_ENABLED(CONFIG_THP_SWAP))
315             get_swap_pages(1, &entry, HPAGE_PMD_NR);
316         goto out;
317     }
318 
319     /*
320      * Preemption is allowed here, because we may sleep
321      * in refill_swap_slots_cache().  But it is safe, because
322      * accesses to the per-CPU data structure are protected by the
323      * mutex cache->alloc_lock.
324      *
325      * The alloc path here does not touch cache->slots_ret
326      * so cache->free_lock is not taken.
327      */
328     cache = raw_cpu_ptr(&swp_slots);
329 
330     if (likely(check_cache_active() && cache->slots)) {
331         mutex_lock(&cache->alloc_lock);
332         if (cache->slots) {
333 repeat:
334             if (cache->nr) {
335                 entry = cache->slots[cache->cur];
336                 cache->slots[cache->cur++].val = 0;
337                 cache->nr--;
338             } else if (refill_swap_slots_cache(cache)) {
339                 goto repeat;
340             }
341         }
342         mutex_unlock(&cache->alloc_lock);
343         if (entry.val)
344             goto out;
345     }
346 
347     get_swap_pages(1, &entry, 1);
348 out:
349     if (mem_cgroup_try_charge_swap(page, entry)) {
350         put_swap_page(page, entry);
351         entry.val = 0;
352     }
353     return entry;
354 }
```

- 311 *get_swap_page()* 为一个待换出的页面分配 swap 槽位，返回 *swap_entry_t*。无可用 swap 槽

时，返回 *entry.val == 0* 代表分配失败。此处初始化返回值。

- 313-317 透明巨页 swap 槽的分配。

  对于透明大页，如果开启了 swap 支持 (*CONFIG_THP_SWAP*=y), 调用 *get_swap_pages()* 一次性分配多个连续 swap 槽（一个大页对应多个普通 swap 槽）。

  不管分配成功失败，直接 goto out，不走下面 per-cpu 缓存的普通页分配逻辑。

- 328 获取当前 cpu 的 per-cpu swap_slots 缓存实例, 优先尝试从 per-cpu 的 slot(交换槽) 缓存中分配槽位。而不是直接从 swap 分区 (或 swap 文件) 中实际查找空闲 slot 分配给交换条目。避免每次分配都直接去磁盘 swap 分区，从而极大降低锁竞争、提高 swap 分配性能。

  per-cpu 的 swap slot 高速缓存是每个 CPU 单独维护的一批预分配好、可快速取用的 swap 槽 (swp_entry_t)，slot 最终要和 swap 文件的页做映射。swap_slots_cache 解决的是“怎么高效拿到这个映射号”。

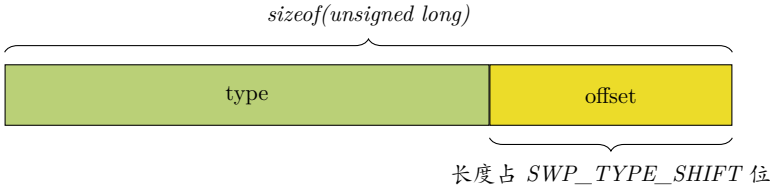
  slot 高速缓存和 struct swap_slots_cache 的定义如下, 里面的 slots 指针指向一组提前映射好的 swap 槽 (页)：

```
static DEFINE_PER_CPU(struct swap_slots_cache, swp_slots);
... ... ... ...
struct swap_slots_cache {
    bool        lock_initialized;
    struct mutex    alloc_lock; /* protects slots, nr, cur */
    swp_entry_t *slots;
    int     nr;
    int     cur;
    spinlock_t  free_lock;  /* protects slots_ret, n_ret */
    swp_entry_t *slots_ret;
    int     n_ret;
};
```

struct swap_slots_cache 结构体成员的作用意义如下:

- ***lock_initialized*** 标记本 CPU 的缓存锁是否已经初始化完成。
- ***allo_lock*** 保护数据的互斥锁。
- ***slots*** 预映射好的 swap 槽缓存数组。这是一个数组指针，里面存了一批提前申请好的 swap slot 槽号, 每个元素是一个 swp_entry_t (对应磁盘 swap 上的一个 slot(页))，分配时直接从这里取比较快。

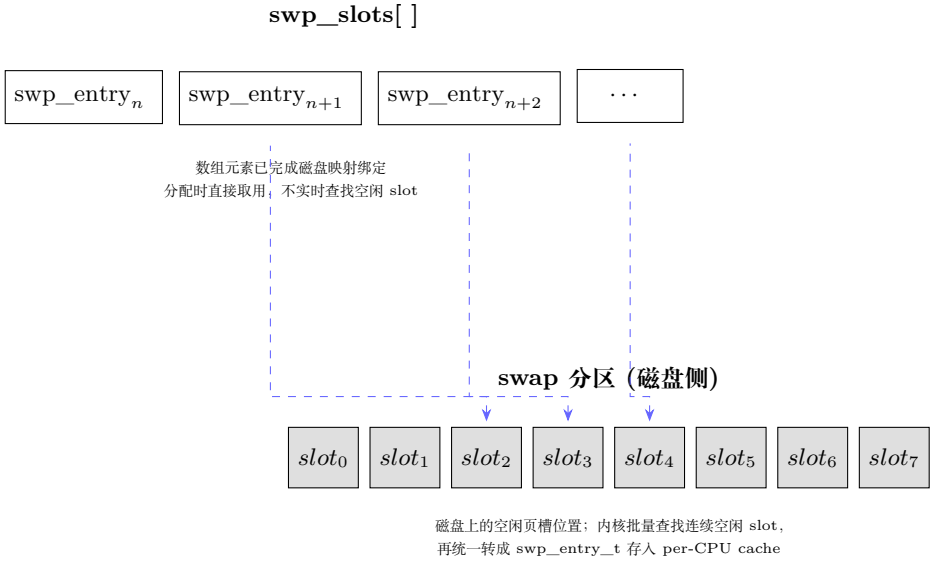
  *swp_entry_t*是一个联合体，用来把 $type+offset$ 打包到一个 64 位 (32 位系统 32 位) 整数，存放在页表项 (PTE) 里面。当页面被换出，pte 不再指向物理 page，而是填入 ***swp_entry_t***。这里的 type 是内核代码中的名称，其实更确切的名称应该是 index(swap 分区索引号)。



- ***nr*** 当前缓存里还有多少个可用的空闲 swap 槽。
- ***cur*** 下一次要从 slots 数组的第几个下标取 slot。

- ***free_lock*** 保护“释放”数据的自旋锁。
- ***slots_ret*** 待归还的 swap 槽数组。不用一个个还回全局管理器，先攒一批，满了再批量归还大幅提高释放效率。
- ***nr_ret*** slots_ret 数组里当前有多少个待归还的 slot 槽, 到达阈值后一次性全部还给 swap 管理器。

批量填充时, 内核去 swap 分区一次性找 N 个连续空闲 slot，一次性建立 N 个 (type, offset) 映射，转成 swp_entry_t，存成 slots 数组。分配时直接从 slots 数组拿映射早已建好的 swp_entry_t 直接用。所以 per-cpu 的 swap slot 高速缓存里的每个 swp_entry_t，都是已经和磁盘 slot 页槽绑定好的有效编号。缓存只是暂存这些编号，避免每次去磁盘里轮询查找 swap 的空闲 slot 再映射 (如下图)。



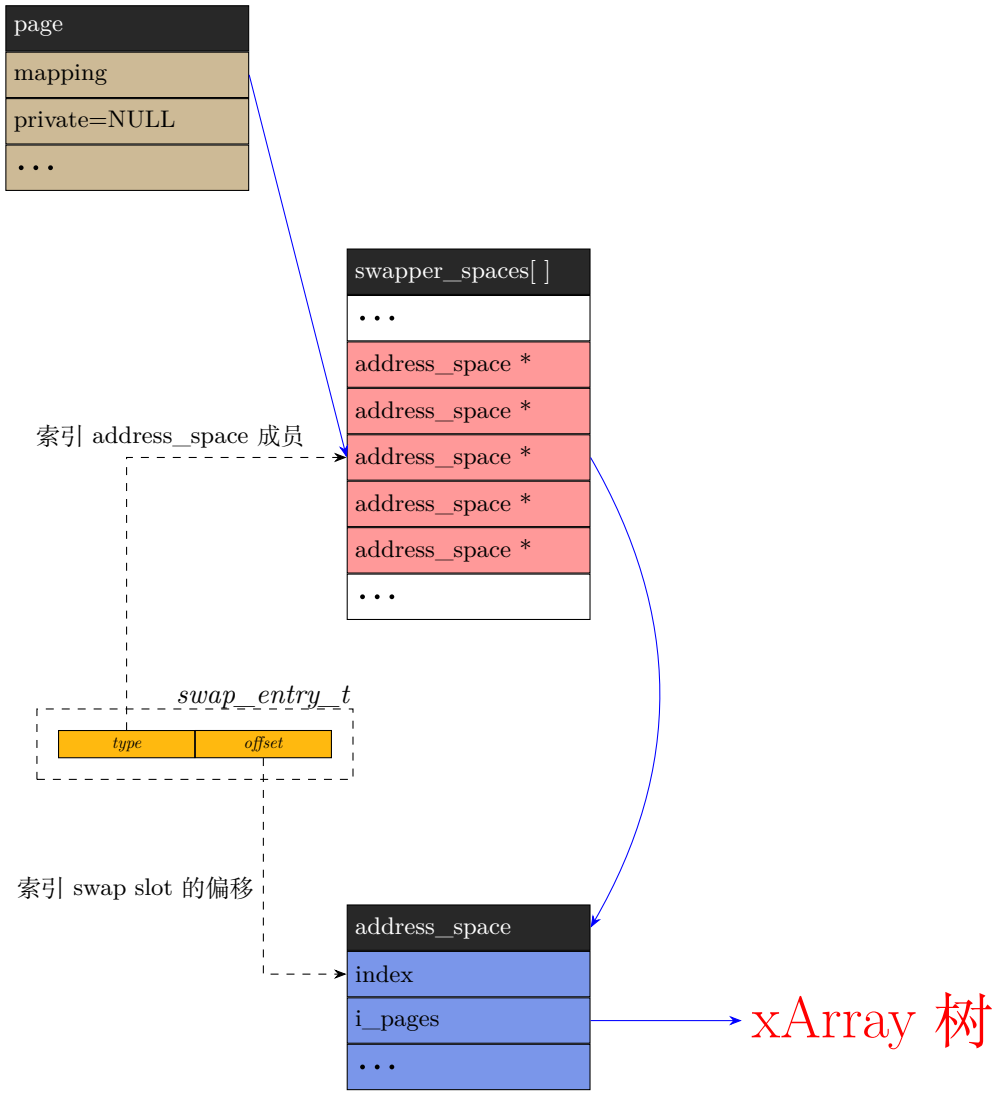
swapper_spaces[] 是一个全局数组，数组中每项对应一个真实的 swap 设备或 swap 分区。每个数组项存放的是给各个 swap 分区伪造的 struct address_space 的指针。

address_space->i_pages 是 xArray, 用来存放对应 swap 设备 (分区) 全部 swapcache 的页指针。而整个 swap 分区的 address_space 指针会赋值给目标页的 page->mapping。

当进程页表接触映射，PTE 的 present 位置 0，其余位直接编码 swp_entry_t。这是一个 unsigned long 型值，由 $type + offset$ 组成，type 实际就是目标 swap 分区对应 address_space 指针在 swapper_spaces[] 数组里的位置索引。而 offse 则是目标页在 swap 分区里的偏移。

一个标准匿名页占据一个 swap slot, 不会拆分。对于 THP 页，会拆成基页存到 swap slot。

如图:



- 330-344 如果 slot 页槽缓存可用，就走快速路径。
  - 330 检查 per-CPU swap 缓存功能是否开启，再检查当前 CPU 的 slot 缓存数组已经有已经映射好的 slot(非 NULL)。
  - 331 加锁，防止多线程并发冲突。
  - 332 加锁后再确认缓存有效, 防止加锁期间缓存被释放。
  - 333 用于缓存空了重新填充后再试一次。
  - 334-337 当前 slot 缓存数组里还有可用的 swap 槽，就直接从 slot 缓存数组里取出一个预映射好的 swap slot 页槽并清 swap entry。然后 cur++，以便下次取下一个。另外, 缓存剩余页槽的数量 −1。
- 338-340 如果 slot 缓存数组已经空了（nr=0）。则调用 refill_swap_slots_cache(cache) 一次性批量申请映射 64(SWAP_SLOTS_CACHE_SIZE) 个 swap slot 槽, 并跳转到 repeat 标签处重新取 slot。
- 343-344 判断是否成功从 slot 缓存拿到了 swap slot 槽。成功则跳转到 out 处。

- 347 如果 slot 页槽缓存数组为空，并且批量申请失败。则走慢速路径，直接从全局 swap 管理器申请。但是速度慢，有全局锁竞争。
- 349 对这个 page 所属 memcg 做 swap 记账。
- 350 记账失败必须把刚刚分配到手的 swap slot 归还回去。

5.1.13.3 add_to_swap_cache()

如果成功分配了 swap entry(交换条目)，就通过 add_to_swap_cache() 将其添加到对应的 struct address_space(如下)。

shrink_inactive_list() > shrink_page_list() > add_to_swap() > add_to_swap_cache()

```
128 int add_to_swap_cache(struct page *page, swp_entry_t entry,
129             gfp_t gfp, void **shadowp)
130 {
131     struct address_space *address_space = swap_address_space(entry);
132     pgoff_t idx = swp_offset(entry);
133     XA_STATE_ORDER(xas, &address_space->i_pages, idx, compound_order(page));
134     unsigned long i, nr = thp_nr_pages(page);
135     void *old;
136 
137     VM_BUG_ON_PAGE(!PageLocked(page), page);
138     VM_BUG_ON_PAGE(PageSwapCache(page), page);
139     VM_BUG_ON_PAGE(!PageSwapBacked(page), page);
140 
141     page_ref_add(page, nr);
142     SetPageSwapCache(page);
143 
144     do {
145         unsigned long nr_shadows = 0;
146 
147         xas_lock_irq(&xas);
148         xas_create_range(&xas);
149         if (xas_error(&xas))
150             goto unlock;
151         for (i = 0; i < nr; i++) {
152             VM_BUG_ON_PAGE(xas.xa_index != idx + i, page);
153             old = xas_load(&xas);
154             if (xa_is_value(old)) {
155                 nr_shadows++;
156                 if (shadowp)
157                     *shadowp = old;
158             }
159             set_page_private(page + i, entry.val + i);
160             xas_store(&xas, page);
161             xas_next(&xas);
162         }
163         address_space->nrexceptional -= nr_shadows;
164         address_space->nrpages += nr;
165         __mod_node_page_state(page_pgdat(page), NR_FILE_PAGES, nr);
166         ADD_CACHE_INFO(add_total, nr);
167 unlock:
168         xas_unlock_irq(&xas);
169     } while (xas_nomem(&xas, gfp));
170 
171     if (!xas_error(&xas))
172         return 0;
173 
```

```
174     ClearPageSwapCache(page);
175     page_ref_sub(page, nr);
176     return xas_error(&xas);
177 }
```

- 131 根据 *swp_entry_t* 的 swap type(swap 分区索引)，拿到该 swap 对应的全局 swap address_space。

  每个 swap 分区对应一个独立 *address_space*，它的 *i_pages* 是 XArray，用来管理 swap cache。

- 132 取出 swap entry 里的 slot 偏移 (offset)，也就是这个页在这个 swap 分区上的 slot 下标，作为 xarray 的索引。

  THP 场景：entry 是起始 slot，后面占用连续 *HPAGE_PMD_NR* 个 slot：*idx, idx+1, idx+2 ...*

- 134 *thp_nr_pages()* 返回目标页的 (基) 页数。普通 4K 页返回 1，THP 页返回 *HPAGE_PMD_NR*。nr 就是要插入 swap cache 的 page 数量。

- 135 保存 xarray 中原位置的旧条目，用来检测是否存在 workingset 的 shadow 条目。

- 137 page 必须持有 page lock，swap cache 插入期间不能被并发修改。

- 138 检测目标 page 是否已经在 swap cache，防止重复加入 swap cache。

- 139 必须是支持后备存储的页，只有支持后备存储的页面才允许进入 swap cache。

- 141 增加 page 引用计数。一旦加入 swap cache，swap cache 本身持有 page 引用，防止 page 被释放掉。

  THP 时，给整个大页增加 nr 个引用。普通页 +1。

- 142 设置 ***PG_swapcache*** 标志位。

- 144-169

  - 145 变量 *nr_shadows* 用于统计本次插入范围内遇到多少个 *shadow entry*

    swap cache 中 shadow entry 代表 slot 已经分配，但 page 不在内存 (已经被回收)，XArray 里面存的不是 *struct page**，而是编码的 value。

  - 146 拿 XArray 的锁，同时关闭本地中断。防止并发修改 *address_space->i_pages* XArray 树结构。

  - 147 预分配 XArray 树需要的中间节点，为连续一段索引 $[idx, idx + nr\text{-}1]$ 建好树节点。如果内存不足，xas 会打上错误标记 ***XAS_ERR_NOMEM***。

  - 148-149 如果节点分配失败，直接跳到 unlock，不做插入。

  - 151 普通页 $nr = 1$ 循环 1 次。THP 巨页 $nr = HPAGE_PMD_NR$，循环遍历每个子页。

  - 152 校验 XAS 当前遍历索引，必须等于预期的 swap offset $idx + i$。顺序错乱直接 BUG，用于排查 XArray 迭代逻辑 bug。

  - 153 读取 XArray 当前索引位置上现存的条目。

  - 154 判断条目是 shadow entry。

    shadow entry 语义:swap slot 已经分配，但 page 不在内存，XArray 存 value 编码占位。当我们重新把 page 载入 swap cache，要把 shadow 替换成真实 page 指针。

- 155 统计遇到多少 shadow。
- 156-157 如果调用方传入了 *shadowp* 非 NULL，把 shadow value 回写给调用者。

- 159 给每个基页设置 *page->private* , 存入 *swp_entry_t.val*。

  page->private 是 *unsigned long*;*swp_entry_t.val* 也是 *unsigned long*，二者类型完全匹配，可以直接赋值存储。

  swp_entry_t 是一个位打包的联合体，把两部分信息压缩到一个 *unsigned long* 整数:

  swap type(swap 分区索引): 标记属于哪一个 swap 分区，对应全局数组 *swap_info[]* 的下标。

  swap offset: 该 slot 在这个 swap 分区 (swap type) 内部的 slot 编号。
- 160 把 *struct page *page* 存入 XArray 当前索引位置。
- 161 XAS 迭代器移动到下一个索引。
- 164 swap cache 里面 page 计数，加上本次插入的页数 nr。
- 165 swap cache 页面会统计计入文件页。

  匿名页 (堆、栈等) 没有文件，要换出时，需要分配 *swap slot*。内核伪造一组 address_space 模拟 “文件缓存”，存放在全局数组 *swapper_spaces[]*；它们没有真实 inode、没有磁盘文件，只是假装是文件页缓存。swap cache 页虽然复用了 page cache 的基础管理框架，但内核通过 ***PG_swapcache*** 标志做了专属标记，不会和普通文件页混淆统计。它是匿名页 swap 流程的中间态产物。

  swap cache 属于 swapper_space，内核统计上归类为非匿名页(non-anonymous pages) 或者更准确地说，是所有 拥有后备存储且通过 address_space 管理的页 。swapper_space 是一个特殊的 address_space 对象。内核在底层统计时，将所有挂在 address_space 下的页面统称为 “file pages” (文件页), 计入了 ***NR_FILE_PAGES*** 中。

  - ***buffers、buffer cache、block device pages*** 缓存 原始块设备 (raw block device) 读写操作的内存区域, 也称之为” 块缓冲页”, 它可能是:

    裸设备数据 如果直接 *dd if=/dev/sda ...* 读取磁盘原始数据，这些数据会被缓存在 buffers 中。

    文件系统元数据 文件系统 (如 ext4, XFS) 的超级块 (superblock)、inode 表、位图 (bitmap) 等元数据，在磁盘上是以 “块” 的形式存在的。内核在读写这些元数据时，通常使用 buffer head 机制，因此它们传统上被计入 buffers。文件实际内容 (file data) 不通过 buffer_head 索引，而是直接由 page cache 管理。

    NOTE

    文件页、匿名页都是用户空间在使用。而块缓冲页是内核使用，所以被单独统计在 *Buffers* 里。这也是块缓冲 (文件) 页和普通文件页的区别之一。

  - ***cached、page cache*** 缓存文件的 逻辑内容。
  - ***swapcache*** 源自匿名页，但暂时有了磁盘备份。内核统计 *NR_FILE_PAGES* 时包含它，是因为它有了 “后备存储”。但在用户视角，它 不属于 “读取的文件”，而是 “进程的数据”。

    /proc/meminfo 中的:

$$cached = total_filed_backed - block_device_pages - swap_cache_pages$$

即,

用户态文件页缓存 = NR_FILE_PAGES − 块设备缓冲页的数目 − $swapcache$

所以用户看到的 cached(page cache) 不包含它，但它确实在底层的 NR_FILE_PAGES 计数里 (详见:13.1.1节)。

- 174-176 xarray 操作失败, 执行错误回滚。
  - 174 清除 $PG_swapcache$ 标志;
  - 175 减掉之前增加的 page 引用计数;
  - 176 返回 xarray 错误码给上层。
- 1043 通过传入的 swap_entry_t 类型的值找到对应的”伪”页缓存 address_space(swapper_spaces[type][j])。之所以叫”伪 (pseudo)”页缓存，原因是：它是模仿普通文件页缓存 (pagecache) 的接口和结构，但背后没有真实磁盘文件，而是 swap 分区/文件。匿名页 (堆、栈等) 没有文件，要换出时，需要分配 swap slot。内核用一组 address_space 来模拟“文件”，每个 address_space 管理 swap 里的一段连续空间；这些 address_space 放在全局数组 swapper_spaces[] 里；它们没有真实 inode、没有磁盘文件，只是假装是文件页缓存。

5.1.13.4 XA_STATE_ORDER()

XA_STATE_ORDER() 调用 ___XA_STATE 构造一个 struct xa_state 实例。这个结构体主要用于执行复杂的、多步骤的 XArray 操作 (如遍历、批量插入等)，它充当了一个”游标”，记录了当前在树中的位置，从而避免每次操作都从根节点重新开始查找。

shrink_inactive_list() › ··· › ··· › *add_to_swap_cache()* › *XA_STATE_ORDER()*

```
1040 __XA_STATE
1041
1042 XA_STATE
1043
1044 XA_STATE_ORDER
1045 ..............................
```

该宏调用 ___XA_STATE() 来填充 xa_state 结构体的关键字段。

- 直接透传 struct xarray * 指针，赋值给 xa_state.xa，指向要操作的 XArray (树) 根。
- 将传入的 index 向下对齐到 2^{order} 的边界。
- 计算当前层级在树结构中对应的位移量 (shift)，用于确定从哪一层开始遍历。

  XA_CHUNK_SHIFT 通常为 6，意味着每个节点有 $2^6 = 64$ 个页槽 (slots)
- 计算在当前树层级（每个节点 64 个页槽）中，该条目占用了多少个连续的页槽 (slot)。

==========> 上面解析未完

5.1.13.5 page_check_dirty_writeback()

前面在 shrink_page_list() 函数中调用 page_check_dirty_writeback() 时已经提过，page_check_dirty_writeback() 用于检查目标页是否脏页，以及是否处于 writeback 状态。

shrink_inactive_list() > *shrink_page_list()* > *page_check_dirty_writeback()*

```
1040 static void page_check_dirty_writeback(struct page *page,
1041                                        bool *dirty, bool *writeback)
1042 {
1043     struct address_space *mapping;
1044
1045     /*
1046      * Anonymous pages are not handled by flushers and must be written
1047      * from reclaim context. Do not stall reclaim based on them
1048      */
1049     if (!page_is_file_lru(page) ||
1050         (PageAnon(page) && !PageSwapBacked(page))) {
1051         *dirty = false;
1052         *writeback = false;
1053         return;
1054     }
1055
1056     /* By default assume that the page flags are accurate */
1057     *dirty = PageDirty(page);
1058     *writeback = PageWriteback(page);
1059
1060     /* Verify dirty/writeback state if the filesystem supports it */
1061     if (!page_has_private(page))
1062         return;
1063
1064     mapping = page_mapping(page);
1065     if (mapping && mapping->a_ops->is_dirty_writeback)
1066         mapping->a_ops->is_dirty_writeback(page, dirty, writeback);
1067 }
```

- 1049 检查目标页是否在文件页 lru 链表。如果不是，则立即返回。
- 1050 在上一行检测出来的基础上继续检查。也就是检查处于文件 lru 链表上的目标页是否匿名页 (通过 page->mapping 的 bit 0 位判断)，并且不支持 swap space 作为后备存储来缓存。换言之，就是判断目标页是否 lazyfree 页。如果是，也立即返回。

  文件脏页的回写可以交给文件系统通用回写线程flusher 异步回写。而匿名页 swapout 只发生在 vmscan 回收上下文 (kswapd/直接回收的进程上下文)，不会交给文件系统通用回写线程 flusher。

  匿名页 (如进程堆、栈) 无文件后备存储，其回写需通过交换分区 (swap) 实现。其回收必须在回收上下文中直接解除页表映射，把所有进程的 PTE 改写成 swap entry。也就是把原来指向物理内存的 PTE 改写成一个特殊编码，用来记录这个页的数据存放在 swap 分区哪个槽位。并且 PTE 的 present 位置 0。$present = 1$ 表示 PTE 里存有物理页帧号，MMU 可以直接访问物理内存。$present = 0$ 表示虚拟地址没有映射到物理页帧，访问会触发 page fault。后续访问时，因为 $present = 0$ 触发 page fault 流程，内核 *do_swap_page()* 会解析 PTE 里保存的 swap entry，找到磁盘上对应 swap 分区的槽位，把 swap 分区里的数据重新读回物理内存，并将 PTE 指向新的物理页，同时 $present = 1$。

  文件页的回写机制通过 bdi_writeback 线程 (flusher)异步回写脏页，而内存回收流程需与其协同，其中通过 page_check_dirty_writeback() 间接计数的，未进入回写队列的脏页数目是唤醒异步线程 flusher 的判断条件。

  当脏页比例超过阈值 (由 dirty_background_radio 或 dirty_expire_centisecs 控制)，触发回写线程批量处理脏页。当回收过程发现脏页回写拥塞，可能直接调用同步回写 (如 wait_iff_congested),

导致回收线程阻塞等待 I/O 完成。page_check_dirty_writeback() 在此过程中充当状态同步的检查点，确保回收操作不会破坏数据一致性。

1049-1053 行的意义在于，匿名页不会提交给文件系统通用回写线程回写，所以立即返回。

linux 内存管理中有个很特别的设计，lazyfree 页会被移动到文件页的 lru，但是他们逻辑上仍然是匿名页。它的 page->mapping 的 bit 0 位为 1，代表它仍然是匿名页。

核心原因在于回收算法的差异:

- 匿名页回收成本高 标准匿名页如果没有 swap 分区支持, 则无法回收 (即使匿名页没有被写过)。即使有 swap, 也需要昂贵的 swap 磁盘 IO 操作。
- lazyfree 页回收成本低 对匿名页执行 *MADV_FREE* 后，内核清除 PTE dirty 位，并且清除 ***PG_swapbacked*** 标记。意味着回收它们无需写回磁盘，直接释放即可。

  但是如果在回收之前应用对其进行**写**访问，则退化成普通匿名页，需要 swapout 后才能回收。

  需要注意的是，*madvice(...,MADV_FREE)* 不会清除 page 本身的 ***PG_dirty*** 位 (*page->flags*), 只会清除进程 PTE 里的 ***PG_dirty*** 位。

  PTE dirty 和 page->flags 的 PG_dirty 位的对比

项	*PTE dirty(页表硬件位)*	*page->flags(PG_dirty(软件标志))*
位置	进程页表 *PTE* 项，每个进程一份	*struct page*，物理页全局唯一。
置 *1*	写操作 *MMU* 自动置位	调用 *SetPageDirty(page)* 置位
清 *0*	*set_pte_at(),flush_tlb_page()*	调用 *ClearPageDirty()* 清 *0*
MADV_FREE 行为	清 *0 PTE dirty* 位	完全不动 *PG_dirty* 位

- 文件页的回收成本低 因为文件页有磁盘副本，干净页可以直接丢弃，代价低。而匿名页没有原始磁盘副本，要回收必须先 swapout。

应用层调用 *madvise(...,MADV_FREE)*, 会陷入内核最终调用 *lru_lazyfree_fn()* 将页面从匿名页链表移动到文件 lru 链表。

因为文件页对应有磁盘副本，因此文件页链表增加，通常应该涉及 IO 读写操作。但是, 大量使用 *MADV_FREE* 后，从*/proc/vmstat* 可以看到在没有文件 IO 的情况下，*nr_inactive_file* 会增加。

- 1057-1058 直接用 *page->flags* 来获取目标页的脏页状况 (*PG_dirty*)，以及是否在 IO 队列中排队回写 (*PG_writeback*)。
- 1061-1062 在前面已排除匿名页的情况下，进一步排除一些特殊文件系统的文件页，如 ramfs 等. 这些文件系统不支持脏页和回写操作，它们的 flag 中没有置 PAGE_FLAGS_PRIVATE 标志位。
- 1065-1066 文件系统通过实现 is_dirty_writeback 可以自定义如何判断页面脏状态和回写状态。若文件系统未实现此回调，内核会使用默认的脏页检查逻辑 (如检查 PG_dirty 和 pg_writeback 标志位)。

前面 1058-1059 行已经获取了 page 的脏页和回写状态 (PG_dirty 和 PG_writeback)。但 *struct page->flags* 和 page 内部的 *buffer_head* 的脏位可能不一致。

文件系统的块设备页会挂载 buffer_head, 它存放于 *page->private*。一个 buffer_head 代表块设备上的一个逻辑块，因为磁盘块的大小通常 <一个 PAGE_SIZE。

假设 *PAGE_SIZE*= $4K$，一个磁盘块大小 = $1K$。那么一页可以对应 4 个磁盘块，对应四个 *buffer_head*。*page->private* 就是用来挂载这一组 (4 个)*buffer_head*。判断 *page->private* 是否为 NULL，就可以判断是否挂载了 *buffer_head*。只修改其中一个块，那么会对这个块的 *buffer_head* 的 ***BH_dirty*** 位置位或清 0。但某些场景下 *page->flags* 和这个 page 里的 buffer_head 的 *BH_dirty* 会出现不同步。

5.1.13.6 文件页链表上的匿名页: lazyfree 页

lru_lazyfree_fn() 函数把满足条件的匿名页转换为 lazyfree 页，从匿名 lru 迁移到欠活跃文件页 lru 链表，清除 *PG_swapbacked*。

lru_lazyfree_fn()

```
594 static void lru_lazyfree_fn(struct page *page, struct lruvec *lruvec,
595                 void *arg)
596 {
597     if (PageLRU(page) && PageAnon(page) && PageSwapBacked(page) &&
598         !PageSwapCache(page) && !PageUnevictable(page)) {
599         bool active = PageActive(page);
600         int nr_pages = thp_nr_pages(page);
601 
602         del_page_from_lru_list(page, lruvec,
603                        LRU_INACTIVE_ANON + active);
604         ClearPageActive(page);
605         ClearPageReferenced(page);
606         /*
607          * Lazyfree pages are clean anonymous pages.  They have
608          * PG_swapbacked flag cleared, to distinguish them from normal
609          * anonymous pages
610          */
611         ClearPageSwapBacked(page);
612         add_page_to_lru_list(page, lruvec, LRU_INACTIVE_FILE);
613 
614         __count_vm_events(PGLAZYFREE, nr_pages);
615         __count_memcg_events(lruvec_memcg(lruvec), PGLAZYFREE,
616                      nr_pages);
617     }
618 }
```

- 597-598 下面 5 个条件必须全部满足，才做 lazyfree 转换:
  - *PageLRU(page)* 页面挂在 LRU 链表上，不处于隔离 (isolated) 状态。
  - *PageAnon(page)* 匿名页。
  - *PageSwapBacked(page)* 该页支持 swap 来做后备存储。
  - *!PageSwapCache(page)* 该页不是 swapcache 页，也就是没有 swap 缓存数据映射。
  - *!PageUnevictable(page)* 页面可回收。

  只处理普通可回收匿名页，已经进入 swapcache 的页不能走 lazyfree 的处理。

- 599 读取页面当前的活跃标记 *PG_active*，判断该页原来在活跃匿名页还是欠活跃匿名页链表。
- 600 获取目标 page 的 (基) 页数量。

- 602-603 从原来的匿名 lru 链表摘除该页，同时修改 lruvec 的计数。
- 604 清除 ***PG_active*** 标记，lazyfree 页面一律视作非活跃页。
- 605 清除 ***PG_referenced*** 引用标记。
- 611 清除 ***PG_swapbacked***。该页的内容已经逻辑释放，不再需要 swap 后备存储支持。
- 612 把这个原本的匿名页，加入到欠活跃的文件 LRU 链表。
- 614 更新全局 vmstat 统计:*PGLAZYFREE*。从节点*/proc/vmstat* 打印出来的*pglazyfree*项可以看到该统计值。



图 5–7: *cat /proc/vmstat* 显示的 lazyfree 页数量信息

- 615-616 当前页面属于哪个 cgroup，就给这个 cgroup 的统计计数器，累加本次 lazyfree 转换的页数。

5.1.13.7 page_check_references()

当页移动至 inactive 链表的末尾附近，内核会检查该页关联的所有页表项（PTE）中的页表访问位是否被置位。用于检查的函数是 page_check_references()(如下)，此函数的返回值决定了后续处理方式:

- ***PAGEREF_ACTIVATE***：表示该页应被移入 active 链表。
- ***PAGEREF_KEEP***：表示应将其放回 inactive 链表的头部，留待下一轮扫描再判断；
- ***PAGEREF_RECLAIM***：表示应对该页执行回收操作。
- ***PAGEREF_RECLAIM_CLEAN***: 表示如果文件页是干净的，则回收。否则要写回 (write back) 到磁盘或外存后才回收。

shrink_inactive_list() > *shrink_page_list()* > *page_check_references()*

```
986 static enum page_references page_check_references(struct page *page,
987                         struct scan_control *sc)
988 {
989     int referenced_ptes, referenced_page;
990     unsigned long vm_flags;
991
992     referenced_ptes = page_referenced(page, 1, sc->target_mem_cgroup,
993                       &vm_flags);
994     referenced_page = TestClearPageReferenced(page);
995
996     /*
997      * Mlock lost the isolation race with us.  Let try_to_unmap()
998      * move the page to the unevictable list.
999      */
1000    if (vm_flags & VM_LOCKED)
1001        return PAGEREF_RECLAIM;
1002
```

```
1003     if (referenced_ptes) {
1004         /*
1005          * All mapped pages start out with page table
1006          * references from the instantiating fault, so we need
1007          * to look twice if a mapped file page is used more
1008          * than once.
1009          *
1010          * Mark it and spare it for another trip around the
1011          * inactive list.  Another page table reference will
1012          * lead to its activation.
1013          *
1014          * Note: the mark is set for activated pages as well
1015          * so that recently deactivated but used pages are
1016          * quickly recovered.
1017          */
1018         SetPageReferenced(page);
1019 
1020         if (referenced_page || referenced_ptes > 1)
1021             return PAGEREF_ACTIVATE;
1022 
1023         /*
1024          * Activate file-backed executable pages after first usage.
1025          */
1026         if ((vm_flags & VM_EXEC) && !PageSwapBacked(page))
1027             return PAGEREF_ACTIVATE;
1028 
1029         return PAGEREF_KEEP;
1030     }
1031 
1032     /* Reclaim if clean, defer dirty pages to writeback */
1033     if (referenced_page && !PageSwapBacked(page))
1034         return PAGEREF_RECLAIM_CLEAN;
1035 
1036     return PAGEREF_RECLAIM;
1037 }
```

- 992 扫描该 page 所有映射的页表 PTE，统计 ***PTE accessed*** 硬件位。统计映射该物理页帧的进程 PTE 页表项中，有多少个被标记”young”，即 accessed 标志位是否置位。即从上次检查该标志位后，是否被访问；该函数在检测后清除这些 PTE 的 accessed 标志位，迎接下一次检查。

  变量 referenced_ptes 用于存储这个扫描周期里映射的目标页帧被访问的次数。换言之，也就是物理页面被进程映射的次数 (或映射该页的 PTE 的数目)。

  同一物理页帧可以被多个进程的页表同时映射，所以一个物理页可以有多个 PTE。每一个 PTE 都有自己独立的 accessed 位 (”young” 标志位).

  page_referenced() 根据页的逆向映射遍历找到所有映射该页的虚拟地址区域 vma, 然后遍历它们各自的页表，直到抵达 PTE 级，再查看页表项 PTE 的 accessed 位是否置位。找到某个 PTE, 如果检测到 accessed 位置位了 (表示目标物理页被对应进程映射), 计数 +1, 将 PTE 的 accessed 位原子的清零, 继续找下一个 PTE。最终返回引用总数给调用函数。

  处理大页时会检查更高层级的页表项 (而非仅 PTE)，以适配不同大小页。x86-64 中：

  - 4KB 普通页：映射到 PTE 层级，检查 PTE 的 _PAGE_ACCESSED；
  - 2MB 大页：映射到 PMD 层级，检查 PMD 的访问位；
  - 1GB 大页：映射到 PUD 层级，检查 PUD 的访问位；

- 994 读并且原子清除 *page->flags* 里的 *PG_referenced* 软件标记.

  变量 referenced_page 存储用的是 flag PG_referenced 的值。PG_referenced 用于记录页面在最近一次 LRU 扫描周期内是否被访问过，用于记录页面历史活跃性，只标识是否被访问，而不是被访问的次数，因此它的值只是 0 或 1。

  TestClearPageReferenced() 是原子操作，用于获取 PG_referenced 的当前值返回并清零。以上两个局部变量的作用是用来辅助确定对目标页的回收策略 (如触发页面激活或保留)。

  PG_referenced 标志保存在 struct page 的 flags 字段中，所以每个物理页只有唯一一个。每次在检测该位后都立即清空它 (可以理解成同时操作)，这样只有在这次检查后页又被重新访问/映射时，该标志才会被再次置位。

- 1000-1001 内存回收过程中，内核将目标页从 LRU 链表隔离之后、还没做回收判断之前，另一个用户线程调用 *mlock()* 锁住了这片 VMA，页面不能回收, 它本应位于 unevictable list, 但因某些竞态条件 (如 mlock 未完成或锁冲突), 该页可能错误地出现在可回收链表 (如 inactive LRU)。=======待 double check

  这种情况下返回 ***PAGEREF_RECLAIM*** 会促使上层调用者 (如 shrink_page_list) 执行 try_to_unmap(), 在 try_to_unmap() 中内核会将页面移至 unevictable 链表，确保后续回收流程不再处理该页。

- 1003-1030 如果目标页在上次检查后到本次检查这个时间段中被访问过，则进入此代码块

  - 1018 此次检查周期中该页被映射它的某个进程访问过 (1003 行的判断)，因此设置目标页 page 结构体中的 *PG_referenced* 标志位。
  - 1020-1021 若该页帧在本次检查前就已置位 page->flags 中的 *PG_referenced* 标志位，或有超过 1 个 PTE 映射了该页帧，则将该页帧激活 (即晋升至活跃 LRU 链表)。

    也就是说，发现目标页在此次检查周期已经是二次引用，或过多个 PTE 映射了该页，那么将通过返回值建议把该页从 inactive LRU 链表提升到 active 链表。

    PG_referenced 保存在 struct page->flags 中, 它的生命周期是这样的 (此处忽略映射多个 PTE 的情况):

    - 初始时页的 PG_referenced = 0
    - 页面被访问后，PTE.accessed = 1
    - page_referenced() 扫描

      发现 accessed == 1

      清零, accessed = 0

      返回值 > 0
    - 返回值 > 0, 代表页面被访问，内核设置 (page->flags) PG_referenced = 1
    - 下一个周期扫描

      重新扫描 PTE.accessed

      检测 *PG_referenced* 值，判断页是否被访问过。清零,PG_referenced = 0。之后再进行上述 page_referenced() 扫描和之后的流程。

    通过上面的流程可知，如果检测到页被引用过 (PTE.accessed 被置位了)，但并未设置 page->flags 的 PG_referenced 标志，那么表示该页在此检测周期是第一次被引用。

- 1026-1027 但目标页属于文件页且映射的是可执行代码（由 VM_EXEC VMA 标志指示），

  文件映射可执行页（程序代码段），第一次访问就直接激活，不需要二次检查。直接通过返回 ***PAGEREF_ACTIVATE*** 表示应立即激活该页。

- 1029 若不满足上述条件，则保留（keep）该页帧，把它放回非活跃链表的头部，留待下一轮扫描再做判断。

- 1033-1036 如果自上次检查以来目标页未被引用过，说明它已符合回收条件。如果是文件页，返回 PAGEREF_RECLAIM_CLEAN，表示仅当页面干净时才允许回收，脏页则交给后台回写处理。否则，返回 PAGEREF_RECLAIM，表示该页允许进入回收流程。

5.1.13.8 pageout()

shrink_page_list() 调用的 pageout() 是内存回收时的“脏页写回入口函数”。它把脏页写回磁盘或后备存储，让页面变成干净页才能释放它。源码如下:

shrink_inactive_list() > *shrink_page_list()* > *pageout()*

```
780 static pageout_t pageout(struct page *page, struct address_space *mapping)
781 {
782     /*
783      * If the page is dirty, only perform writeback if that write
784      * will be non-blocking.  To prevent this allocation from being
785      * stalled by pagecache activity.  But note that there may be
786      * stalls if we need to run get_block().  We could test
787      * PagePrivate for that.
788      *
789      * If this process is currently in __generic_file_write_iter() against
790      * this page's queue, we can perform writeback even if that
791      * will block.
792      *
793      * If the page is swapcache, write it back even if that would
794      * block, for some throttling. This happens by accident, because
795      * swap_backing_dev_info is bust: it doesn't reflect the
796      * congestion state of the swapdevs.  Easy to fix, if needed.
797      */
798     if (!is_page_cache_freeable(page))
799         return PAGE_KEEP;
800     if (!mapping) {
801         /*
802          * Some data journaling orphaned pages can have
803          * page->mapping == NULL while being dirty with clean buffers.
804          */
805         if (page_has_private(page)) {
806             if (try_to_free_buffers(page)) {
807                 ClearPageDirty(page);
808                 pr_info("%s: orphaned page\n", __func__);
809                 return PAGE_CLEAN;
810             }
811         }
812         return PAGE_KEEP;
813     }
814     if (mapping->a_ops->writepage == NULL)
815         return PAGE_ACTIVATE;
816     if (!may_write_to_inode(mapping->host))
817         return PAGE_KEEP;
818
819     if (clear_page_dirty_for_io(page)) {
```

```
820         int res;
821         struct writeback_control wbc = {
822             .sync_mode = WB_SYNC_NONE,
823             .nr_to_write = SWAP_CLUSTER_MAX,
824             .range_start = 0,
825             .range_end = LLONG_MAX,
826             .for_reclaim = 1,
827         };
828 
829         SetPageReclaim(page);
830         res = mapping->a_ops->writepage(page, &wbc);
831         if (res < 0)
832             handle_write_error(mapping, page, res);
833         if (res == AOP_WRITEPAGE_ACTIVATE) {
834             ClearPageReclaim(page);
835             return PAGE_ACTIVATE;
836         }
837 
838         if (!PageWriteback(page)) {
839             /* synchronous write or broken a_ops? */
840             ClearPageReclaim(page);
841         }
842         trace_mm_vmscan_writepage(page);
843         inc_node_page_state(page, NR_VMSCAN_WRITE);
844         return PAGE_SUCCESS;
845     }
846 
847     return PAGE_CLEAN;
848 }
```

- 798-799 is_page_cache_freeable() 用来检查目标页是否可以安全释放。如果目前页面当前不能释放，那就轮转 (下次再检测) 保留。
- 800-812 page->mapping 是一个共享的字段，对于文件页，匿名页分别含义如下:
  - 文件页：page->mapping = 文件的 address_space(inode->i_mapping)
  - 匿名页: page->mapping = NULL
  - swapcache 页：page->mapping = swapper_spaces, 也就是指向 swapper_space[] 中伪造的地址空间。

  匿名页 mapping==NULL 时，代表还没进入 swap cache，页面不存在有效的地址空间映射，不能 swapout, 因此只能轮转。
- 814-815 如果这个地址空间不支持 writepage 回写，就不回收，将该页激活到活跃链表。

  shmem/tmpfs 的 address_space(swapper space) 的 *a_ops->writepage=NULL*, 这类页脏了不会写普通磁盘，只有 swapout 才会写 swap 设备。
- 816-817 判断是否允许向目标 inode 执行写回操作。如果不允许，则目标页保持隔离状态，既不回收也不激活到活跃页链表。本次回收放弃处理，留给下一轮。

  不允许写 inode 的场景:
  - inode 已经设置为 S_IMMUTABLE(不可变文件)。
  - inode 是挂载成只读的磁盘上的文件。
  - inode 正在删除、已经被销毁 (I_FREEING)。

  - 文件系统冻结，禁止 IO.

- 819-829 clear_page_dirty_for_io 执行页面回写，该函数会检查确保能够执行回写，然后清除脏页标志，将页面标记为干净 (同时也会处理相关的映射，不过在当前场景下，映射应当已经被清除)。

  如果可以执行回写，系统会一次性回写 SWAP_CLUSTER_MAX (32) 个页面，并设置 PG_reclaim 标志。

- 830-836 检查是否存在虚拟文件系统的回调钩子，即通过 address_space 结构体中 address_space_operations 里的 writepage 方法，将单个页面写回磁盘。如果不存在该钩子，就无法将页面写回；但由于页面是脏页，我们需要将其移出回收流程，因此标记为激活页面。

- 838-840 检查是否发生了同步 I/O — 即 page 的 *PG_writeback* (在 IO 队列排队写回) 标志是否已被清除。如果是，说明回写已真正完成，立即清除 *PG_reclaim* 标志。在这种情况下，返回 “已成功回收页面”。

- 847 如果 clear_page_dirty_for_io 执行失败，说明页面是干净的，因此返回 *PAGE_CLEAN*。

is_page_cache_freeable() 为了使目标页能安全释放会确认检查：这个页面除了内核自己在用以外，没有任何用户空间进程在用。

如果是下面三种引用，那该页可以安全释放：

- **页面回收扫描组件**隔离该页面的调用者，也就是正在做页面回收的内核线程 (kswapd /direct reclaim)。

  当回收流程开始时，内核从 LRU 链表把目标页面取下来隔离页面。临时占用页面，为了防止别人动，会将页面blue引用计数 +1

- **文件页缓存本身**当页面被用作文件页缓存时，表示文件页的缓存持有这个页了，此时会将页面blue引用计数 +1。

  页分配时的引用是所有页面的基础引用为 1，文件页 (page cache) 会额外多加一个引用。也就是说，文件页刚创建好时，引用计数就是 2。因为:

  alloc_page() → refcount = 1 (页分配)

  add_to_page_cache() → refcount +=1 (用作文件页缓存)

- ***buffer head*** 有些文件页 (ext2/ext3/ext4) 会用 buffer head 管理磁盘块。如果页面有 buffer head，那么会额外占一个引用。页面blue引用计数 +1。
  subitem 它不是进程引用，只是文件系统内部用。

只要是这 3 种文件页就可以释放。如果有用户进程在用，该页则不能释放。

NOTE

***page->_refcount* 和 *page_referenced()* 返回值的区别**

page->_refcount: 表示页面被多少个地方“持有”(引用计数)。*get_page* 会使计数 +1,*put_page()* 会使计数 −1。
另外 *page->_refcount* 不能直接使用，要使用原子变量的访问宏 *page_count()* 来读取。

page_referenced (): 页面最近一个检测周期内多少次被”访问/使用”过。返回的是映射该页面的 *PTE*, 其中有多少个置了硬件访问 *accessed* 位。是进程页表目标页 *PTE* 硬件标记的统计。它不代表映射总个数。
譬如*:* 一个 *page* 被 *10* 个进程映射，但是这个检测周期中全部没有访问，那么 *page_referenced()* 返回 *0*。

---

is_page_cache_freeable() 源码如下：

shrink_inactive_list() > *shrink_page_list()* > *pageout()* > *is_page_cache_freeable()*

```
721 static inline int is_page_cache_freeable(struct page *page)
722 {
723     /*
724      * A freeable page cache page is referenced only by the caller
725      * that isolated the page, the page cache and optional buffer
726      * heads at page->private.
727      */
728     int page_cache_pins = thp_nr_pages(page);
729     return page_count(page) - page_has_private(page) == 1 + page_cache_pins;
730 }
```

- 728 获取目标页的基页数目。如果是普通页则返回 1，如果是透明大页返回 512。
- 729 判断是否除了前面提到的三种内核的引用外还有进程在使用目标页。

  page_has_private(page) 检查 page->private 里是不是放了 buffer head，只返回 0 或 1。而 page_count() 则是上面提到的引用计数 page->_refcount。

5.1.14 shrink_active_list()

内核维护活跃、欠活跃 lru 链表, 新的 fault 页面通常进入欠活跃链表。vmscan() 扫描欠活跃链表，读取 PTE 的 accessed 位判断页面访问情况：

- 文件页: 第一轮访问仅打软件 *PG_referenced* 标记, 需要两轮扫描都看到访问，才提升到活跃链表。
- 匿名页: 本轮扫描发现被访问，则直接提升到活跃链表。
- 无访问痕迹: 留在不活跃链表，允许回收。

活跃链表不能直接回收，必须先老化降级到欠活跃链表，才能成为回收候选。

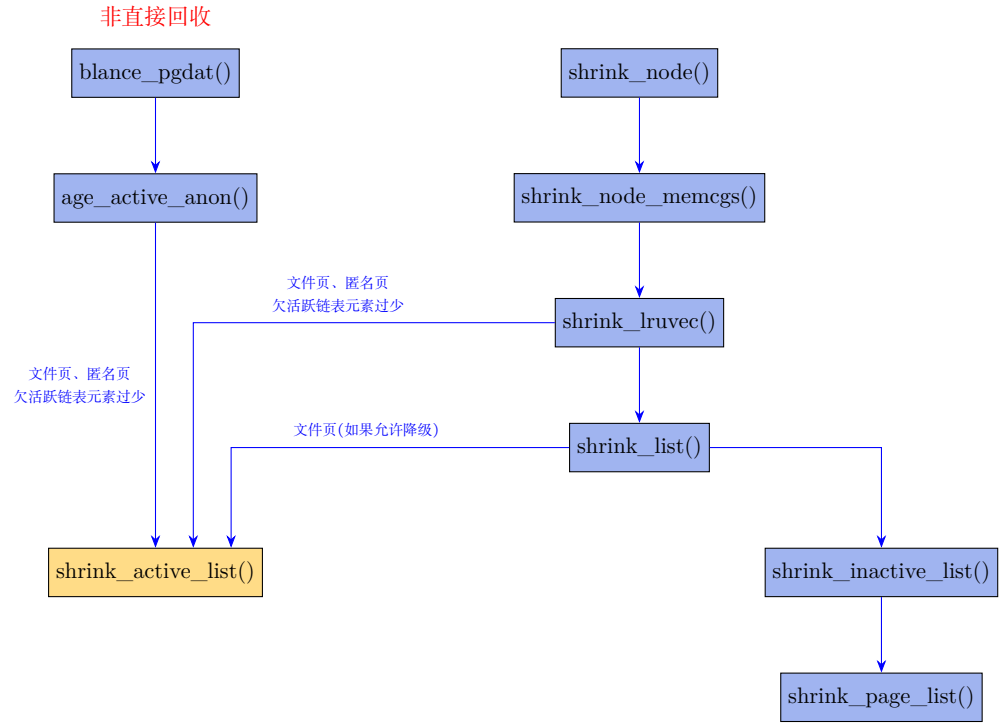
从活跃链表降级通常发生在以下场景：系统发生抖动（thrashing），或inactive 链表容量过小，此时需要将页从活跃链表转移至不活跃链表以维持平衡。

通常情况下，内存回收机制会更倾向于频繁地扫描文件页，因此会设置 struct scan_control->may_deactivate 为 DEACTIVATE_FILE 标志后，并调用 shrink_list()。

然而，匿名页不会被类似文件页般高频次地处理，因此必须额外确保其能被稳定地降级。所以在间接内存回收和执行 LRU 向量收缩时，系统会主动将匿名页从活跃链表迁移至不活跃链表执行降级操作。

在所有场景下，降级操作仅在满足以下条件时触发：

- 系统发生抖动；
- 或不活跃链表过小，无法维持理想的 active/inactive 链表平衡 (由 inactive_is_low() 函数判断)



将页从活跃链表迁移至不活跃链表的函数是 shrink_active_list()，如下。

mm/vmscan.c :: shrink_inactive_list()

```
2009 static void shrink_active_list(unsigned long nr_to_scan,
2010                   struct lruvec *lruvec,
2011                   struct scan_control *sc,
2012                   enum lru_list lru)
2013 {
2014     unsigned long nr_taken;
2015     unsigned long nr_scanned;
2016     unsigned long vm_flags;
2017     LIST_HEAD(l_hold);  /* The pages which were snipped off */
2018     LIST_HEAD(l_active);
2019     LIST_HEAD(l_inactive);
2020     struct page *page;
2021     unsigned nr_deactivate, nr_activate;
2022     unsigned nr_rotated = 0;
2023     int file = is_file_lru(lru);
2024     struct pglist_data *pgdat = lruvec_pgdat(lruvec);
2025
2026     lru_add_drain();
2027
2028     spin_lock_irq(&pgdat->lru_lock);
2029
2030     nr_taken = isolate_lru_pages(nr_to_scan, lruvec, &l_hold,
2031                  &nr_scanned, sc, lru);
2032
2033     __mod_node_page_state(pgdat, NR_ISOLATED_ANON + file, nr_taken);
2034
2035     if (!cgroup_reclaim(sc))
2036         __count_vm_events(PGREFILL, nr_scanned);
2037     __count_memcg_events(lruvec_memcg(lruvec), PGREFILL, nr_scanned);
```

```
2038
2039    spin_unlock_irq(&pgdat->lru_lock);
2040
2041    while (!list_empty(&l_hold)) {
2042        cond_resched();
2043        page = lru_to_page(&l_hold);
2044        list_del(&page->lru);
2045
2046        if (unlikely(!page_evictable(page))) {
2047            putback_lru_page(page);
2048            continue;
2049        }
2050
2051        if (unlikely(buffer_heads_over_limit)) {
2052            if (page_has_private(page) && trylock_page(page)) {
2053                if (page_has_private(page))
2054                    try_to_release_page(page, 0);
2055                unlock_page(page);
2056            }
2057        }
2058
2059        if (page_referenced(page, 0, sc->target_mem_cgroup,
2060                    &vm_flags)) {
2061            /*
2062             * Identify referenced, file-backed active pages and
2063             * give them one more trip around the active list. So
2064             * that executable code get better chances to stay in
2065             * memory under moderate memory pressure.  Anon pages
2066             * are not likely to be evicted by use-once streaming
2067             * IO, plus JVM can create lots of anon VM_EXEC pages,
2068             * so we ignore them here.
2069             */
2070            if ((vm_flags & VM_EXEC) && page_is_file_lru(page)) {
2071                nr_rotated += thp_nr_pages(page);
2072                list_add(&page->lru, &l_active);
2073                continue;
2074            }
2075        }
2076
2077        ClearPageActive(page);  /* we are de-activating */
2078        SetPageWorkingset(page);
2079        list_add(&page->lru, &l_inactive);
2080    }
2081
2082    /*
2083     * Move pages back to the lru list.
2084     */
2085    spin_lock_irq(&pgdat->lru_lock);
2086
2087    nr_activate = move_pages_to_lru(lruvec, &l_active);
2088    nr_deactivate = move_pages_to_lru(lruvec, &l_inactive);
2089    /* Keep all free pages in l_active list */
2090    list_splice(&l_inactive, &l_active);
2091
2092    __count_vm_events(PGDEACTIVATE, nr_deactivate);
2093    __count_memcg_events(lruvec_memcg(lruvec), PGDEACTIVATE, nr_deactivate);
2094
2095    __mod_node_page_state(pgdat, NR_ISOLATED_ANON + file, -nr_taken);
2096    spin_unlock_irq(&pgdat->lru_lock);
2097
2098    mem_cgroup_uncharge_list(&l_active);
2099    free_unref_page_list(&l_active);
2100    trace_mm_vmscan_lru_shrink_active(pgdat->node_id, nr_taken, nr_activate,
2101            nr_deactivate, nr_rotated, sc->priority, file);
```

```
2102 }
```

- 2017-2019 该函数维护着三个链表：

  l_hold：存放从 LRU 链表中隔离出来的页；

  l_active：存放需要放回活跃链表的页；

  l_inactive：存放需要被降级、放入欠活跃链表尾部的页。

- 2026 排空所有已缓存、等待批量加入 LRU 向量的页。

- 2028 获取 struct lruvec 相关的锁。

  struct lruvec->lru_lock 锁的竞争非常激烈，因此内核竭尽全力缩短持有该锁的时间。

- 2030 持有 LRU 锁后，将 LRU 链表中的页隔离出来并放入 l_hold 链表。尝试扫描 nr_to_scan 个页面，并将实际扫描的页数存入 nr_scanned。这一操作由 *isolate_lru_pages()* 函数完成。该函数同时会增加每个被隔离页的引用计数，将其固定在内存中。函数返回被隔离页所包含的 (基) 页数，将其保存在 nr_taken 中。

- 2041-2043 完成页隔离后，立即释放 2028 行获取的锁，并开始遍历被隔离页的链表 l_hold 。内核反向遍历该链表，从链表尾部获取页。通过 lru_to_page() 得到对应的页。

- 2044 所有加入隔离链表的页，通过 *struct page->lru* 链表节点链接入 l_hold。内核通过 list_del() 将该页从隔离链表中删除，准备将其放入合适的目标链表。

- 2046-2047 检查目标页是否被 mlock() 锁定，或被标记为不可换出 (unevictable)。若满足该条件，则终止操作，并通过 putback_lru_page() 将目标页放回对应的 LRU 链表，同时会降低之前 2030 行增加的引用计数。

- 2059-2075 若通过硬件页表访问标志 (PTE 的 accessed 位) 判断，自上次检测后该目标页被访问过 (由函数 page_referenced() 判断)。且该页是映射了可执行代码的文件页，则将其保留在活跃链表中，因此放入 l_active 链表。

  除了可执行文件页 (VM_EXEC) 这一特殊场景外，页会被无差别地从活跃链表降级到欠活跃链表，无需额外决策。一旦活跃链表需要收缩，其中的页会移入欠活跃链表，成为内存回收的候选对象。

- 2077-2079 在其他所有场景下，清除 PG_active 标志，设置 PG_workingset 标志 (标记该页曾属于活跃链表)，并将该页加入 l_inactive 链表。

NOTE

常用的页标志和意义

页标志	含义	置位和清除的位置
PG_lru	页是否在 *LRU* 链表上	加入 *LRU* 时置位，移出时清除
PG_active	页是否在 *active LRU* 上	加入 *active LRU* 时置位，移出时清除
PG_referenced	页近期是否被访问过	页面被访问时置位，老化时清除
PG_workingset	页之前是否位于 *active* 链表	*shrink_active_list()* 回收时置位,*refault* 后清除

- 2085 遍历循环隔离链表结束后，因又需要修改 LRU 向量，内核再次获取 struct lruvec->lru_lock 锁。

- 2087-2088 将所有页迁移到对应的目标链表 (欠活跃页与活跃页分别处理)。
- 2090,2099 将两个待释放链表合并到 l_active，再提交给 free_unref_page_list() 函数将这些页释放回物理内存分配器。

5.1.14.1 isolate_lru_pages()

lru 链表用于管理内存页的回收与活跃状态的切换。全局的 pgdata->lru_lock 会保护这些链表的完整性，但所有内存操作 (如页分配、回收、迁移、交换等) 均需获取此锁。当系统内存规模较大或并发操作频繁时，会因严重的全局锁竞争而造成性能瓶颈。通过批量隔离出部分页面到临时链表，从而在 LRU 锁 (pgdata->lru_lock) 之外来处理这部分页面可以显著提升性能。当然，获取批量页面时还是需要锁来保护的。

LRU 链表收缩的操作分为两个阶段：一是将页面从链表中隔离出来，并确保在隔离过程中这些页的状态保持稳定、不会被并发修改；二是对这些页执行真正的收缩回收处理。

LRU 链表中的页隔离操作由 isolate_lru_pages() 函数完成。

shrink_active_list() ▸ *isolate_lru_pages()*

```
1649 static unsigned long isolate_lru_pages(unsigned long nr_to_scan,
1650         struct lruvec *lruvec, struct list_head *dst,
1651         unsigned long *nr_scanned, struct scan_control *sc,
1652         enum lru_list lru)
1653 {
1654     struct list_head *src = &lruvec->lists[lru];
1655     unsigned long nr_taken = 0;
1656     unsigned long nr_zone_taken[MAX_NR_ZONES] = { 0 };
1657     unsigned long nr_skipped[MAX_NR_ZONES] = { 0, };
1658     unsigned long skipped = 0;
1659     unsigned long scan, total_scan, nr_pages;
1660     LIST_HEAD(pages_skipped);
1661     isolate_mode_t mode = (sc->may_unmap ? 0 : ISOLATE_UNMAPPED);
1662
1663     total_scan = 0;
1664     scan = 0;
1665     while (scan < nr_to_scan && !list_empty(src)) {
1666         struct page *page;
1667
1668         page = lru_to_page(src);
1669         prefetchw_prev_lru_page(page, src, flags);
1670
1671         VM_BUG_ON_PAGE(!PageLRU(page), page);
1672
1673         nr_pages = compound_nr(page);
1674         total_scan += nr_pages;
1675
1676         if (page_zonenum(page) > sc->reclaim_idx) {
1677             list_move(&page->lru, &pages_skipped);
1678             nr_skipped[page_zonenum(page)] += nr_pages;
1679             continue;
1680         }
1681
1682         /*
1683          * Do not count skipped pages because that makes the function
1684          * return with no isolated pages if the LRU mostly contains
```

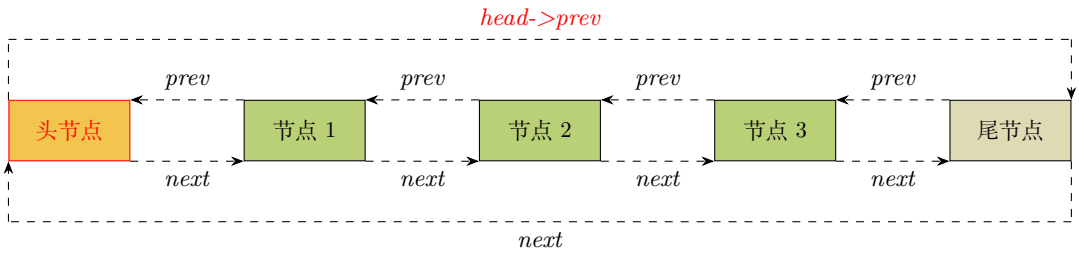
```
1685          * ineligible pages.  This causes the VM to not reclaim any
1686          * pages, triggering a premature OOM.
1687          *
1688          * Account all tail pages of THP.  This would not cause
1689          * premature OOM since __isolate_lru_page() returns -EBUSY
1690          * only when the page is being freed somewhere else.
1691          */
1692         scan += nr_pages;
1693         switch (__isolate_lru_page(page, mode)) {
1694         case 0:
1695             nr_taken += nr_pages;
1696             nr_zone_taken[page_zonenum(page)] += nr_pages;
1697             list_move(&page->lru, dst);
1698             break;
1699
1700         case -EBUSY:
1701             /* else it is being freed elsewhere */
1702             list_move(&page->lru, src);
1703             continue;
1704
1705         default:
1706             BUG();
1707         }
1708     }
1709
1710     /*
1711      * Splice any skipped pages to the start of the LRU list. Note that
1712      * this disrupts the LRU order when reclaiming for lower zones but
1713      * we cannot splice to the tail. If we did then the SWAP_CLUSTER_MAX
1714      * scanning would soon rescan the same pages to skip and put the
1715      * system at risk of premature OOM.
1716      */
1717     if (!list_empty(&pages_skipped)) {
1718         int zid;
1719
1720         list_splice(&pages_skipped, src);
1721         for (zid = 0; zid < MAX_NR_ZONES; zid++) {
1722             if (!nr_skipped[zid])
1723                 continue;
1724
1725             __count_zid_vm_events(PGSCAN_SKIP, zid, nr_skipped[zid]);
1726             skipped += nr_skipped[zid];
1727         }
1728     }
1729     *nr_scanned = total_scan;
1730     trace_mm_vmscan_lru_isolate(sc->reclaim_idx, sc->order, nr_to_scan,
1731                     total_scan, skipped, nr_taken, mode, lru);
1732     update_lru_sizes(lruvec, lru, nr_zone_taken);
1733     return nr_taken;
1734 }
```

该函数将扫描指定的 LRU 链表并尝试隔离 nr_to_scan 个页面到临时链表，并返回实际成功隔离的页面数。

- 1660 定义一个临时链表，该链表用于存储因各种原因选择跳过的页。
- 1665 扫描 LRU 链表。
- 1668 从链表末尾逆向往链表头依次循环遍历目标 LRU 链表中的页，通过 lru_to_page() 取出链表尾部的项。lru_to_page() 宏如下：

```
#define lru_to_page(head) (list_entry((head)->prev, struct page, lru))
```

从 list_head 角度看是从链表尾向链表头依次扫描。如图，首先获取 head->prev 节点，也就是尾节点 node(图中 tail 节点)，然后将处理过的尾节点通过 list_move 移走，使 head->prev 迭代指向新的尾部 node，如此方式向链表头逆向扫描：



页是从头部加入 LRU 链表的，而内存回收时则从尾部读取。

- 1669 预取页面数据以提高缓存命中率。
- 1673-1674 获取目标页 (或复合页) 包含的 (基) 页数，将其累加到已扫描页数的总统计量 total_scan 中。

  我们用 scan 统计未被跳过的页数量，并在循环中与 nr_to_scan 比较，避免扫描大量不符合条件的页却最终什么都没隔离出来。
- 1676-1679 如果某个页所在的内存 zone 索引高于触发本次回收所能申请的最高 zone 索引，则该页会被跳过。

  page_zonenum(page) 返回目标页所在内存 zone 的类型编号，这个编号是该内存 zone 在系统中的位置索引。在 NUMA 系统中，每个内存 node 都有自己的内存 zone 集合，page_zonenum 返回的是 zone 的类型编号。如果要唯一缺点一个 zone，需要 $node\ ID + zone$类型编号。

  ->reclaim_idx 表示可以用于页面回收隔离的最高内存区域 (zone) 的索引。如果目标页面所在 zone 的索引号高于当前允许回收的最高 zone 索引号 (sc->reclaim_idx)，则跳过该页面。

  1677 1676 行被跳过的页面暂时被移到 pages_skipped 链表。等后续一起放到 LRU 链表头部。
- 1693 *___isolate_lru_page()* 是实际进行隔离操作的函数。具体见后。
- 1694-1698 如果成功隔离，则更新计数，并将目标页面移到临时链表。
- 1700-1702 将因为-EBUSY 而未隔离的页移到 head 之后 (即 head->next)。如果不做此操作，后续扫描会反复处理同一页面，从而无效循环导致系统误判” 内存不足”。将跳过的页面置于头部虽会短暂破坏 LRU 链表顺序，但可避免无效扫描和 OOM 风险。这是内核在” 回收效率” 和” 系统稳定性” 之间的折中策略。

  NOTE

  扫描到但未能成功隔离的情况，不算作被跳过 *(skipped)* 的页；这类页只记为”已扫描但未隔离”。
  目标页所在 *zone* 索引高于触发本次回收所能申请的最高 *zone* 索引，则该页会被记为”跳过”
- 1717-1720 如果” 暂时跳过” 链表中存在页，将把这些页一起并回源 LRU 链表头部。

  NOTE

  要保证链表安全必须持有锁，该函数内默认我们已经持有 *struct lruvec->lru_lock* 锁。

- 1721-1727 更新各个 zone 的 VM 事件统计。
- 1729 更新总扫描页面数 (包括跳过的)。
- 1730 记录 trace 事件。
- 1732 在隔离页面后需要更新 LRU 列表大小。
- 1733 返回已成功隔离的页数。

iso_lru_pages() 函数会调用到 ___isolate_lru_page() 尝试移隔离处在合适状态的页面,函数如下:

shrink_active_list > *iso_lru_pages()* > *__isolate_lru_page()*

```
1544 int __isolate_lru_page(struct page *page, isolate_mode_t mode)
1545 {
1546     int ret = -EINVAL;
1547
1548     /* Only take pages on the LRU. */
1549     if (!PageLRU(page))
1550         return ret;
1551
1552     /* Compaction should not handle unevictable pages but CMA can do so */
1553     if (PageUnevictable(page) && !(mode & ISOLATE_UNEVICTABLE))
1554         return ret;
1555
1556     ret = -EBUSY;
1557
1558     /*
1559      * To minimise LRU disruption, the caller can indicate that it only
1560      * wants to isolate pages it will be able to operate on without
1561      * blocking - clean pages for the most part.
1562      *
1563      * ISOLATE_ASYNC_MIGRATE is used to indicate that it only wants to pages
1564      * that it is possible to migrate without blocking
1565      */
1566     if (mode & ISOLATE_ASYNC_MIGRATE) {
1567         /* All the caller can do on PageWriteback is block */
1568         if (PageWriteback(page))
1569             return ret;
1570
1571         if (PageDirty(page)) {
1572             struct address_space *mapping;
1573             bool migrate_dirty;
1574
1575             /*
1576              * Only pages without mappings or that have a
1577              * ->migratepage callback are possible to migrate
1578              * without blocking. However, we can be racing with
1579              * truncation so it's necessary to lock the page
1580              * to stabilise the mapping as truncation holds
1581              * the page lock until after the page is removed
1582              * from the page cache.
1583              */
1584             if (!trylock_page(page))
1585                 return ret;
1586
1587             mapping = page_mapping(page);
1588             migrate_dirty = !mapping || mapping->a_ops->migratepage;
1589             unlock_page(page);
1590             if (!migrate_dirty)
1591                 return ret;
1592         }
```

```
1593     }
1594
1595     if ((mode & ISOLATE_UNMAPPED) && page_mapped(page))
1596         return ret;
1597
1598     if (likely(get_page_unless_zero(page))) {
1599         /*
1600          * Be careful not to clear PageLRU until after we're
1601          * sure the page is not being freed elsewhere -- the
1602          * page release code relies on it.
1603          */
1604         ClearPageLRU(page);
1605         ret = 0;
1606     }
1607
1608     return ret;
1609 }
```

- 1549-1550 目标页面必须在 LRU 链表上。
- 1553-1554 默认不操作不可回收页。连续内存分配器 (CMA) 在需要分配大块连续内存时，可以强制回收不可回收页，但需要通过 mode 参数传递 ***ISOLATE_UNEVICTABLE*** 标志允许此类操作。

  参数 mode 其它可能的选择：

  - ***ISOLATE_INACTIVE*** 隔离欠活动页
  - ***ISOLATE_ACTIVE*** 隔离活动页
  - ***ISOLATE_BOTH*** 活动页和欠活动页都隔离
- 1566 为了尽量减少对 LRU 链表的干扰，可以指定只隔离那些无需阻塞即可操作的页面—-大多数情况下指干净页。
- 1568-1569 ***ISOLATE_ASYNC_MIGRATE*** 标志被用来标示只获取那些可以无需阻塞的可隔离的页面。当页面处于写回状态 (位于 writeback 队列等待 IO 操作) 时，调用者会阻塞。这意味着在 ISOLATE_ASYNC_MIGRATE 标志下遇到 writeback 页时，当前操作立即返回。
- 1588 只有满足以下条件的页面无需阻塞即可迁移：
  - 导致 1587 行 mapping 为 NULL 的页面。此处 mapping 的值由函数 page_mapping() 获得，满足条件的页是: 匿名页和 slab 页。

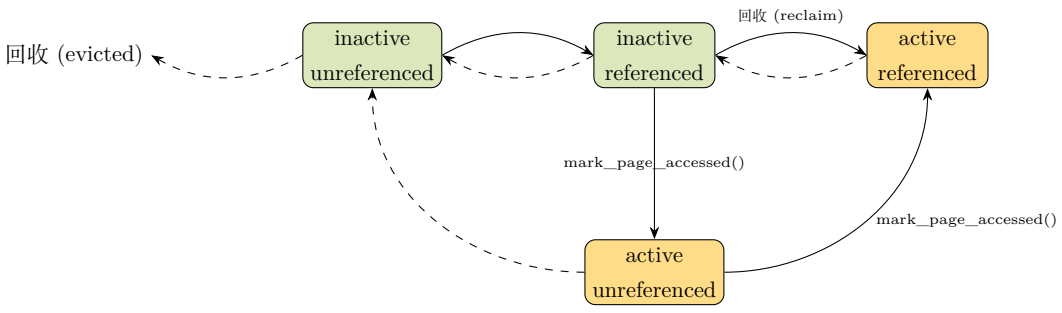
    slab 页已经被排除。因为 slab 以 obj(而非完整页面) 为单位管理内存，脏状态由 obj 自身逻辑处理（如引用计数），而非依赖页级 PG_dirty 标志。当 slab 页被释放时，内存直接归还给伙伴系统，无需触发脏页回写流程。PG_dirty 主要用于标记回写磁盘的文件缓存页或需交换到磁盘的匿名页。slab 页的 struct page 中，标志位 PG_slab 已被用于标识其属于 slab 分配器，与 PG_dirty 的语义互斥，因此早在 1571 行通过”if(PageDirty(page))” 行就会将 slab 页剔除。
  - 拥有->migratepage 回调函数的页面。如果 mapping 不为 NULL，那必然是在文件页 lru 链表 (可能会也是 lazyfree 页)，返回对应的 address_space 指针。若文件页所属的文件系统未实现->migratepage，意味着可能需要等待脏页回写 (同步 I/O)，违反异步迁移 (ISOLATE_ASYNC_MIGRATE) 的设计目标。因为 migratepage 函数的实现会要求：如果入参 migrate mode 是 MIGRATE SAYNC, 则执行的操作必须是非阻塞的。

简言之，这段代码在目标页是脏页的情况下，把 slab 页和没有实现异步迁移回调函数的文件页排除在可隔离页之外。

- 1595-1596 如果是” 隔离未映射页面” 的 ISOLATE_UNMAPPED 模式，则会在目标页面处于被进程映射的状态下立即返回。此处 page_mapped() 的返回值只能是 0 或 1。
- 1598 增加页面的引用计数，该函数通过原子操作检查页面的引用计数 (page->_refcount) 是否为 0. 若不为 0，则增加引用计数并返回成功；若为 0，说明页面已被释放或正在释放流程中，此时不执行操作并返回失败。
- 1604 在 1598 行的检测保证了” 页面没有被释放，并且也不在释放流程中” 后，就可以将目标页从 LRU 链表移除。

再回到 isolate_lru_pages()。早期的碎片缓解策略是: 在内存回收时，尝试连带回收目标页的相邻页面，以形成连续的高阶内存块 (也就是所谓的” 块状回收”)。这块原先的处理是在 isolate_lru_pages() 函数中。但这个方法的缺陷在于：可能误回收高频使用的” 热页”, 导致性能波动。而替代方案就是现在的内存整理技术，通过迁移可移动页实现大阶内存连续性。

发生页帧被访问的事件时 (例如对文件进行读/写操作，或页帧被换入内存 (swap in))，内核会通过 mark_page_accessed() 函数置位 *page->flags* 的 *PG_referenced* 位, 标记该页帧为已访问。如下 (实线代表页帧升级 (promotion), 虚线代表页帧的降级 (demotion))。



mm/swap.c :: mark_page_accessed()

```
422 void mark_page_accessed(struct page *page)
423 {
424     page = compound_head(page);
425
426     if (!PageReferenced(page)) {
```

```
427         SetPageReferenced(page);
428     } else if (PageUnevictable(page)) {
429         /*
430          * Unevictable pages are on the "LRU_UNEVICTABLE" list. But,
431          * this list is never rotated or maintained, so marking an
432          * evictable page accessed has no effect.
433          */
434     } else if (!PageActive(page)) {
435         /*
436          * If the page is on the LRU, queue it for activation via
437          * lru_pvecs.activate_page. Otherwise, assume the page is on a
438          * pagevec, mark it active and it'll be moved to the active
439          * LRU on the next drain.
440          */
441         if (PageLRU(page))
442             activate_page(page);
443         else
444             __lru_cache_activate_page(page);
445         ClearPageReferenced(page);
446         workingset_activation(page);
447     }
448     if (page_is_idle(page))
449         clear_page_idle(page);
450 }
451 EXPORT_SYMBOL(mark_page_accessed);
```

- 426-427 标记页被” 访问 (referenced)”。即，置位 *pages->flags* 的 *PG_referenced* 位。
- 428-433 不可回收页 (unevictable pages) 在 LRU_UNEVICTABLE 链表中。但该链表从不参与轮转或维护操作，对不可回收页标记” 已访问” 没有任何效果——直接跳过此操作。
- 441-444 页提升到活跃链表分两种方式：单基页 (*order* == 0) 激活用 *activate_page()*，批量激活用 *___lru_cache_activate_page()*(需遍历 lruvec 定位页帧)。
- 445 PG_referenced 是一次性的访问凭证，在提升页到活跃链表后清零该标志位，才能让内核准确统计激活到活跃链表后的新访问。
- 448-449 内核会记录” 工作集” 激活（workingset activation）事件，页被激活说明它重新被进程使用，不再是 “闲置页”。因此若目标页帧此前被标记为” 闲置（idle)” 状态，则清除该标志位。

5.2 kswapd 非直接回收

非直接内存回收是作为后台任务执行的回收操作，它通过 per-node 的内核线程，也就是说每个 node 节点对应有一个该任务线程。该线程会等待需要时被唤醒 (例如，当某次内存分配导致某个 zone 的水位在 ___alloc_pages_slowpath() 中低于低水位线时)。

这些非直接回收进程是 **kswapd**，根据其负责回收的 node 节点编号进行命名 (例如，负责 node 0 回收的 kswapd 线程会显示为 [*kswapd*0])

kswapd()

```
3862 static int kswapd(void *p)
3863 {
3864     unsigned int alloc_order, reclaim_order;
```

```
3865     unsigned int highest_zoneidx = MAX_NR_ZONES - 1;
3866     pg_data_t *pgdat = (pg_data_t*)p;
3867     struct task_struct *tsk = current;
3868     const struct cpumask *cpumask = cpumask_of_node(pgdat->node_id);
3869 
3870     if (!cpumask_empty(cpumask))
3871         set_cpus_allowed_ptr(tsk, cpumask);
3872 
3873     /*
3874      * Tell the memory management that we're a "memory allocator",
3875      * and that if we need more memory we should get access to it
3876      * regardless (see "__alloc_pages()"). "kswapd" should
3877      * never get caught in the normal page freeing logic.
3878      *
3879      * (Kswapd normally doesn't need memory anyway, but sometimes
3880      * you need a small amount of memory in order to be able to
3881      * page out something else, and this flag essentially protects
3882      * us from recursively trying to free more memory as we're
3883      * trying to free the first piece of memory in the first place).
3884      */
3885     tsk->flags |= PF_MEMALLOC | PF_SWAPWRITE | PF_KSWAPD;
3886     set_freezable();
3887 
3888     WRITE_ONCE(pgdat->kswapd_order, 0);
3889     WRITE_ONCE(pgdat->kswapd_highest_zoneidx, MAX_NR_ZONES);
3890     for ( ; ; ) {
3891         bool ret;
3892 
3893         alloc_order = reclaim_order = READ_ONCE(pgdat->kswapd_order);
3894         highest_zoneidx = kswapd_highest_zoneidx(pgdat,
3895                                 highest_zoneidx);
3896 
3897 kswapd_try_sleep:
3898         kswapd_try_to_sleep(pgdat, alloc_order, reclaim_order,
3899                     highest_zoneidx);
3900 
3901         /* Read the new order and highest_zoneidx */
3902         alloc_order = reclaim_order = READ_ONCE(pgdat->kswapd_order);
3903         highest_zoneidx = kswapd_highest_zoneidx(pgdat,
3904                                 highest_zoneidx);
3905         WRITE_ONCE(pgdat->kswapd_order, 0);
3906         WRITE_ONCE(pgdat->kswapd_highest_zoneidx, MAX_NR_ZONES);
3907 
3908         ret = try_to_freeze();
3909         if (kthread_should_stop())
3910             break;
3911 
3912         /*
3913          * We can speed up thawing tasks if we don't call balance_pgdat
3914          * after returning from the refrigerator
3915          */
3916         if (ret)
3917             continue;
3918 
3919         /*
3920          * Reclaim begins at the requested order but if a high-order
3921          * reclaim fails then kswapd falls back to reclaiming for
3922          * order-0. If that happens, kswapd will consider sleeping
3923          * for the order it finished reclaiming at (reclaim_order)
3924          * but kcompactd is woken to compact for the original
3925          * request (alloc_order).
3926          */
3927         trace_mm_vmscan_kswapd_wake(pgdat->node_id, highest_zoneidx,
3928                         alloc_order);
```

```
3929            reclaim_order = balance_pgdat(pgdat, alloc_order,
3930                                highest_zoneidx);
3931            if (reclaim_order < alloc_order)
3932                goto kswapd_try_sleep;
3933        }
3934
3935        tsk->flags &= ~(PF_MEMALLOC | PF_SWAPWRITE | PF_KSWAPD);
3936
3937        return 0;
3938    }
```

- 3870-3871 将 kswapd 内核线程绑定到对应的 CPU 集上。
- 3885 为 kswapd 设置特定的内存分配标志——PF_MEMALLOC 和 PF_KSWAPD。

  PF_MEMALLOC: 1. 禁止该线程再次进入内存回收 (防止递归造成死锁)；2. 允许其忽略水位线使用所有的内存预留储备；

  PF_KSWAPD: 将该线程标记为 kswapd 线程，供内核各处通过 current_is_kswapd() 函数进行判断检查。
- 3888-3889 重置与 kswapd 相关的各种节点字段：

  ->**kswapd_order**，用于存储触发当前非直接回收的分配阶(order), 初始化为 0。

  在非直接回收过程中分配阶有可能根据实际情况发生改变，而这个字段会保存触发非直接回收的当前请求阶的原始值。

  ->**kswapd_highest_zoneidx**，用于存储当前 node 节点最高 zone 区域索引。

  直接回收线程调用 wakeup_kswapd() 唤醒 kswapd 之前，有可能会重新设置这个字段。此处 MAX_NR_ZONES 是一个无效值, 告诉系统重新获取当前回收开始的最高 zone 索引，不要用残留的旧值。
- 3890 进入主循环。
- 3893-3894 重置 kswapd 的分配阶为 0，以及最高 zone 区域索引设为无效值 MAX_NR_ZONES。

  这里重置的值是 0 和 MAX_NR_ZONES，如果是在 for 循环里二次重置，那么其实是在下面的 3905-3906 行赋值的。如果是初始化首次进入，则是 3888-3889 行置的初始值。

  需要注意的是: 在 3902-3903 行读取到的这两个字段值和此处读到的可能会不同。因为 kswapd 在 3898-3899 休眠，可能会被某个直接回收线程唤醒，而该直接回收线程会根据自身情况在唤醒 kswap 进行回收前，可能重新给这两个字段设置不同值。
- 3898-3899 让 kswapd 进入等待状态，直到被唤醒来执行回收。

  这里传递的参数reclaim_order、highest_zoneidx决定了能否使 kswapd 进入休眠。kswapd_try_to_sleep() 调用的 prepare_kswapd_sleep() 会在如下条件之一成立时，使 kswapd 进入休眠：

  – kswap 对内存回收无能为力 (失败次数超限)。

  – 对应 node 节点已经在目标请求 order(即这里的入参 reclaim_order) 下达到了内存平衡。

  需要注意的是，reclaim_order 是每次迭代时的回收阶，在循环中调用 balance_pgdat() 后可能被改写。

- 3902-3903 读取->kswapd_order、->kswapd_highest_zoneidx 这些字段的值放入临时变量。
- 3905-3906 立即将上面两个值重置。

- 3908 检查当前线程是否要冻结，如果是，就主动停下来，进入“冻结”状态。(该函数解析见后面)

  如果进入冻结, 线程会直接停在这个函数里直到系统解冻，它才会从函数里醒来，然后返回。它是内核线程冻结的必备机制.

  返回 false 表示没冻结。返回 true 表示刚才被冻住了，现在刚解冻。
- 3909-3910 检查内核线程是否需要终止。
- 3916-3917 从冻结恢复返回后不立即调用 balance_pgdat，直接 continue。这是为了加快任务的解冻恢复速度。

- 3929-3930 调用 balance_pgdat() 函数执行真正的内存回收。该函数会返回回收完成时的分配阶 (该值可能已被置为比请求阶低的值，例如 0 阶)。

  回收会从请求的分配阶开始，但如果大阶分配的请求回收失败，kswapd 会把阶降级为 order-0 进行回收。一旦发生这种情况，kswapd 会根据实际完成回收时的阶数 (reclaim_order) 决定是否休眠，同时会唤醒 kcompactd 进程，按照原始请求的分配阶数 (alloc_order) 执行内存碎片整理。
- 3931-3932 如果返回的分配阶确实已经被重新置值，那么不会直接回到循环开头重置分配阶，而是直接跳转到调用 kswapd_try_sleep 标签处执行。

  这会让内存整理在原始请求阶上执行；并且，如果上一次完成回收的分配阶高于刚刚设置的值，会考虑以更高阶执行回收（具体判断逻辑见 kswapd_try_to_sleep())
- 3935 如果循环终止，则清除任务的 PF_MEMALLOC 和 PF_KSWAPD 标志，因为该线程不再用于 kswapd 回收工作，即将被终止。

kswapd() > *kswapd_try_to_sleep()*

```
3765 static void kswapd_try_to_sleep(pg_data_t *pgdat, int alloc_order, int reclaim_order,
3766                 unsigned int highest_zoneidx)
3767 {
3768     long remaining = 0;
3769     DEFINE_WAIT(wait);
3770
3771     if (freezing(current) || kthread_should_stop())
3772         return;
3773
3774     prepare_to_wait(&pgdat->kswapd_wait, &wait, TASK_INTERRUPTIBLE);
3775
3776     /*
3777      * Try to sleep for a short interval. Note that kcompactd will only be
3778      * woken if it is possible to sleep for a short interval. This is
3779      * deliberate on the assumption that if reclaim cannot keep an
3780      * eligible zone balanced that it's also unlikely that compaction will
3781      * succeed.
3782      */
```

```
3783    if (prepare_kswapd_sleep(pgdat, reclaim_order, highest_zoneidx)) {
3784        /*
3785         * Compaction records what page blocks it recently failed to
3786         * isolate pages from and skips them in the future scanning.
3787         * When kswapd is going to sleep, it is reasonable to assume
3788         * that pages and compaction may succeed so reset the cache.
3789         */
3790        reset_isolation_suitable(pgdat);
3791
3792        /*
3793         * We have freed the memory, now we should compact it to make
3794         * allocation of the requested order possible.
3795         */
3796        wakeup_kcompactd(pgdat, alloc_order, highest_zoneidx);
3797
3798        remaining = schedule_timeout(HZ/10);
3799
3800        /*
3801         * If woken prematurely then reset kswapd_highest_zoneidx and
3802         * order. The values will either be from a wakeup request or
3803         * the previous request that slept prematurely.
3804         */
3805        if (remaining) {
3806            WRITE_ONCE(pgdat->kswapd_highest_zoneidx,
3807                    kswapd_highest_zoneidx(pgdat,
3808                            highest_zoneidx));
3809
3810            if (READ_ONCE(pgdat->kswapd_order) < reclaim_order)
3811                WRITE_ONCE(pgdat->kswapd_order, reclaim_order);
3812        }
3813
3814        finish_wait(&pgdat->kswapd_wait, &wait);
3815        prepare_to_wait(&pgdat->kswapd_wait, &wait, TASK_INTERRUPTIBLE);
3816    }
3817
3818    /*
3819     * After a short sleep, check if it was a premature sleep. If not, then
3820     * go fully to sleep until explicitly woken up.
3821     */
3822    if (!remaining &&
3823        prepare_kswapd_sleep(pgdat, reclaim_order, highest_zoneidx)) {
3824        trace_mm_vmscan_kswapd_sleep(pgdat->node_id);
3825
3826        /*
3827         * vmstat counters are not perfectly accurate and the estimated
3828         * value for counters such as NR_FREE_PAGES can deviate from the
3829         * true value by nr_online_cpus * threshold. To avoid the zone
3830         * watermarks being breached while under pressure, we reduce the
3831         * per-cpu vmstat threshold while kswapd is awake and restore
3832         * them before going back to sleep.
3833         */
3834        set_pgdat_percpu_threshold(pgdat, calculate_normal_threshold);
3835
3836        if (!kthread_should_stop())
3837            schedule();
3838
3839        set_pgdat_percpu_threshold(pgdat, calculate_pressure_threshold);
3840    } else {
3841        if (remaining)
3842            count_vm_event(KSWAPD_LOW_WMARK_HIT_QUICKLY);
3843        else
3844            count_vm_event(KSWAPD_HIGH_WMARK_HIT_QUICKLY);
3845    }
3846    finish_wait(&pgdat->kswapd_wait, &wait);
```

```
3847 }
```

- 3769 通过 DEFINE_WAIT() 声明一个等待队列项，也就是相当于给 kswapd_try_to_sleep() 函数注册一个银行” 排队号”, 然后去睡觉，等待被叫到这个” 排队的号码” 来唤醒。
- 3771-3772 检查任务是否正在被冻结，或是否需要终止——在这两种情况下，都必须提前退出。

  freezing() 作用是检查当前线程是否正在进入冻结流程，实际上也就是检查 task_struct 里有没有 PF_FROZEN 标志。
- 3774 之前声明的等待队列项会被添加到等待队列 struct pglist_data->kswapd_wait 中，以便在需要执行内存回收时被唤醒。
- 3783 通过 prepare_kswapd_sleep() 函数判断是否需要休眠。如果 kswapd 可以休眠，则返回 true。

  前面已经提过，两种情况可以休眠:1. kswapd 判断对内存回收已无能为力。2. 目标节点达到内存平衡。
- 3790-3796 如果需要休眠，首先通过 reset_isolation_suitable() 重置内存整理状态（释放部分内存），并尝试唤醒内存整理内核线程 kcompactd，以释放更多内存。
- 3798 通过 schedule_timeout() 进行休眠，指定休眠时间为 0.1 秒（以 jiffies 为单位，jiffies 是内核内部的时间度量单位）。使用该函数意味着内核指定了超时时间，因此可以判断是否是休眠时间到而被唤醒的。
- 3805-3812 如果被提前唤醒，意味着我们可能因为大阶页请求导致的回收失败后，无能为力而过早地进入了休眠。内核会选择上一次也就是休眠前的回收阶数与新设置阶数中的较大值。换言之，避免过早降低尝试回收的页块阶数。
- 3814-3815 完成这次初始的短暂休眠后，将等待项从等待队列中移除，并再次重置状态，准备下一次休眠。
- 3822 只有在上一次短睡眠没有被提前唤醒时，我们才会进行长睡眠——这通过 remaining 等于 0 来判断，即上一次休眠没有被提前唤醒。

  3798 行的超时唤醒是测试是否会被提前叫醒。那如果睡满了 0.1 秒（没有被提前叫醒），说明内存不紧张，就准备正式长休眠。
- 3834,3839 set_pgdat_percpu_threshold() 待补充说明
- 3837 schedule() 函数会让调度器应用设置的 TASK_INTERRUPTIBLE 状态，即进入无限期的可中断休眠，直到通过等待队列被唤醒。这意味着 kswapd 会一直休眠，直到有事件唤醒它。
- 3839 被唤醒后，节点的统计阈值会被恢复。
- 3846 清理等待队列。

prepare_kswapd_sleep() 函数的任务是判断 kswapd 是否已经无事可做，或者它对当前的内存压力无能为力。

kswapd() > *kswapd_try_to_sleep()* > *prepare_kswapd_sleep()*

```
3455 static bool prepare_kswapd_sleep(pg_data_t *pgdat, int order,
3456                 int highest_zoneidx)
3457 {
3458     /*
3459      * The throttled processes are normally woken up in balance_pgdat() as
3460      * soon as allow_direct_reclaim() is true. But there is a potential
3461      * race between when kswapd checks the watermarks and a process gets
3462      * throttled. There is also a potential race if processes get
3463      * throttled, kswapd wakes, a large process exits thereby balancing the
3464      * zones, which causes kswapd to exit balance_pgdat() before reaching
3465      * the wake up checks. If kswapd is going to sleep, no process should
3466      * be sleeping on pfmemalloc_wait, so wake them now if necessary. If
3467      * the wake up is premature, processes will wake kswapd and get
3468      * throttled again. The difference from wake ups in balance_pgdat() is
3469      * that here we are under prepare_to_wait().
3470      */
3471     if (waitqueue_active(&pgdat->pfmemalloc_wait))
3472         wake_up_all(&pgdat->pfmemalloc_wait);
3473
3474     /* Hopeless node, leave it to direct reclaim */
3475     if (pgdat->kswapd_failures >= MAX_RECLAIM_RETRIES)
3476         return true;
3477
3478     if (pgdat_balanced(pgdat, order, highest_zoneidx)) {
3479         clear_pgdat_congested(pgdat);
3480         return true;
3481     }
3482
3483     return false;
3484 }
```

- 3471-3472 首先确保没有直接回收进程因为"回收限流" 在等待 kswapd 改善内存状况。若有，则唤醒这些线程。并通过 3483 行返回 false 不进入休眠 (如果接下来几行不提前返回 true 的话)。
- 3475-3476 检查 node 节点的 kswapd 失败次数计数器 struct pglist_data->kswapd_failures 是否达到或超过了最大重试次数 MAX_RECLAIM_RETRIES。如果是，则代表 kswapd 已无能为力，因此可以休眠，返回 true 将工作交给直接回收处理。
- 3478 判断 node 节点是否已达到内存平衡. 打到内存平衡也就意味着 kswapd 已经没有工作可做，可以休眠。
- 3479-3480 清除标识该 node 节点回收拥塞的标志位——因为节点已平衡且无需回收，就意味着不可能存在拥塞。
- 3483 如果 node 节点未达到平衡状态，则 kswapd 不允许休眠，返回 false 并开始执行间接回收。

5.2.1 balance_pgdat

非直接回收 (kswapd) 通过该函数达到 node 节点内存均衡,

按照高端内存区 → 普通内存区 → DMA 区的顺序扫描各个内存区域。它会跳过空闲页面数高于该区域高水位线的区域；但一旦发现某个区域的空闲页面数低于或等于其高水位线，则该区域以及所有优先级更低的区域中的所有页面都可被回收，直到至少有一个可用区域恢复平衡。

kswapd() *balance_pgdat()*

```
3542 static int balance_pgdat(pg_data_t *pgdat, int order, int highest_zoneidx)
3543 {
3544     int i;
3545     unsigned long nr_soft_reclaimed;
3546     unsigned long nr_soft_scanned;
3547     unsigned long pflags;
3548     unsigned long nr_boost_reclaim;
3549     unsigned long zone_boosts[MAX_NR_ZONES] = { 0, };
3550     bool boosted;
3551     struct zone *zone;
3552     struct scan_control sc = {
3553         .gfp_mask = GFP_KERNEL,
3554         .order = order,
3555         .may_unmap = 1,
3556     };
3557 
3558     set_task_reclaim_state(current, &sc.reclaim_state);
3559     psi_memstall_enter(&pflags);
3560     __fs_reclaim_acquire();
3561 
3562     count_vm_event(PAGEOUTRUN);
3563 
3564     /*
3565      * Account for the reclaim boost. Note that the zone boost is left in
3566      * place so that parallel allocations that are near the watermark will
3567      * stall or direct reclaim until kswapd is finished.
3568      */
3569     nr_boost_reclaim = 0;
3570     for (i = 0; i <= highest_zoneidx; i++) {
3571         zone = pgdat->node_zones + i;
3572         if (!managed_zone(zone))
3573             continue;
3574 
3575         nr_boost_reclaim += zone->watermark_boost;
3576         zone_boosts[i] = zone->watermark_boost;
3577     }
3578     boosted = nr_boost_reclaim;
3579 
3580 restart:
3581     sc.priority = DEF_PRIORITY;
3582     do {
3583         unsigned long nr_reclaimed = sc.nr_reclaimed;
3584         bool raise_priority = true;
3585         bool balanced;
3586         bool ret;
3587 
3588         sc.reclaim_idx = highest_zoneidx;
3589 
3590         /*
3591          * If the number of buffer_heads exceeds the maximum allowed
3592          * then consider reclaiming from all zones. This has a dual
3593          * purpose -- on 64-bit systems it is expected that
3594          * buffer_heads are stripped during active rotation. On 32-bit
3595          * systems, highmem pages can pin lowmem memory and shrinking
3596          * buffers can relieve lowmem pressure. Reclaim may still not
3597          * go ahead if all eligible zones for the original allocation
3598          * request are balanced to avoid excessive reclaim from kswapd.
3599          */
3600         if (buffer_heads_over_limit) {
3601             for (i = MAX_NR_ZONES - 1; i >= 0; i--) {
3602                 zone = pgdat->node_zones + i;
3603                 if (!managed_zone(zone))
3604                     continue;
3605 
```

```
3606                sc.reclaim_idx = i;
3607                break;
3608            }
3609        }
3610
3611        /*
3612         * If the pgdat is imbalanced then ignore boosting and preserve
3613         * the watermarks for a later time and restart. Note that the
3614         * zone watermarks will be still reset at the end of balancing
3615         * on the grounds that the normal reclaim should be enough to
3616         * re-evaluate if boosting is required when kswapd next wakes.
3617         */
3618        balanced = pgdat_balanced(pgdat, sc.order, highest_zoneidx);
3619        if (!balanced && nr_boost_reclaim) {
3620            nr_boost_reclaim = 0;
3621            goto restart;
3622        }
3623
3624        /*
3625         * If boosting is not active then only reclaim if there are no
3626         * eligible zones. Note that sc.reclaim_idx is not used as
3627         * buffer_heads_over_limit may have adjusted it.
3628         */
3629        if (!nr_boost_reclaim && balanced)
3630            goto out;
3631
3632        /* Limit the priority of boosting to avoid reclaim writeback */
3633        if (nr_boost_reclaim && sc.priority == DEF_PRIORITY - 2)
3634            raise_priority = false;
3635
3636        /*
3637         * Do not writeback or swap pages for boosted reclaim. The
3638         * intent is to relieve pressure not issue sub-optimal IO
3639         * from reclaim context. If no pages are reclaimed, the
3640         * reclaim will be aborted.
3641         */
3642        sc.may_writepage = !laptop_mode && !nr_boost_reclaim;
3643        sc.may_swap = !nr_boost_reclaim;
3644
3645        /*
3646         * Do some background aging of the anon list, to give
3647         * pages a chance to be referenced before reclaiming. All
3648         * pages are rotated regardless of classzone as this is
3649         * about consistent aging.
3650         */
3651        age_active_anon(pgdat, &sc);
3652
3653        /*
3654         * If we're getting trouble reclaiming, start doing writepage
3655         * even in laptop mode.
3656         */
3657        if (sc.priority < DEF_PRIORITY - 2)
3658            sc.may_writepage = 1;
3659
3660        /* Call soft limit reclaim before calling shrink_node. */
3661        sc.nr_scanned = 0;
3662        nr_soft_scanned = 0;
3663        nr_soft_reclaimed = mem_cgroup_soft_limit_reclaim(pgdat, sc.order,
3664                                sc.gfp_mask, &nr_soft_scanned);
3665        sc.nr_reclaimed += nr_soft_reclaimed;
3666
3667        /*
3668         * There should be no need to raise the scanning priority if
3669         * enough pages are already being scanned that that high
```

```
3670          * watermark would be met at 100% efficiency.
3671          */
3672         if (kswapd_shrink_node(pgdat, &sc))
3673             raise_priority = false;
3674 
3675         /*
3676          * If the low watermark is met there is no need for processes
3677          * to be throttled on pfmemalloc_wait as they should not be
3678          * able to safely make forward progress. Wake them
3679          */
3680         if (waitqueue_active(&pgdat->pfmemalloc_wait) &&
3681                 allow_direct_reclaim(pgdat))
3682             wake_up_all(&pgdat->pfmemalloc_wait);
3683 
3684         /* Check if kswapd should be suspending */
3685         __fs_reclaim_release();
3686         ret = try_to_freeze();
3687         __fs_reclaim_acquire();
3688         if (ret || kthread_should_stop())
3689             break;
3690 
3691         /*
3692          * Raise priority if scanning rate is too low or there was no
3693          * progress in reclaiming pages
3694          */
3695         nr_reclaimed = sc.nr_reclaimed - nr_reclaimed;
3696         nr_boost_reclaim -= min(nr_boost_reclaim, nr_reclaimed);
3697 
3698         /*
3699          * If reclaim made no progress for a boost, stop reclaim as
3700          * IO cannot be queued and it could be an infinite loop in
3701          * extreme circumstances.
3702          */
3703         if (nr_boost_reclaim && !nr_reclaimed)
3704             break;
3705 
3706         if (raise_priority || !nr_reclaimed)
3707             sc.priority--;
3708     } while (sc.priority >= 1);
3709 
3710     if (!sc.nr_reclaimed)
3711         pgdat->kswapd_failures++;
3712 
3713 out:
3714     /* If reclaim was boosted, account for the reclaim done in this pass */
3715     if (boosted) {
3716         unsigned long flags;
3717 
3718         for (i = 0; i <= highest_zoneidx; i++) {
3719             if (!zone_boosts[i])
3720                 continue;
3721 
3722             /* Increments are under the zone lock */
3723             zone = pgdat->node_zones + i;
3724             spin_lock_irqsave(&zone->lock, flags);
3725             zone->watermark_boost -= min(zone->watermark_boost, zone_boosts[i]);
3726             spin_unlock_irqrestore(&zone->lock, flags);
3727         }
3728 
3729         /*
3730          * As there is now likely space, wakeup kcompact to defragment
3731          * pageblocks.
3732          */
3733         wakeup_kcompactd(pgdat, pageblock_order, highest_zoneidx);
```

```
3734        }
3735
3736        snapshot_refaults(NULL, pgdat);
3737        __fs_reclaim_release();
3738        psi_memstall_leave(&pflags);
3739        set_task_reclaim_state(current, NULL);
3740
3741        /*
3742         * Return the order kswapd stopped reclaiming at as
3743         * prepare_kswapd_sleep() takes it into account. If another caller
3744         * entered the allocator slow path while kswapd was awake, order will
3745         * remain at the higher level.
3746         */
3747        return sc.order;
3748    }
```

- 3558 将 slab 组件完成的回收量赋值给当前进程 struct task_struct->rs 字段指向的 struct reclaim_state 对象。
- 3562 统计正在非直接回收的事件，PAGEOUTRUN 标识表示当前正在进行非直接回收.
- 3570-3577 迭代检索目标 node 中的 zone 区域，将因抬高水位线而导致 (水位线) 增加的页面数汇总到 nr_boost_reclaim 中，将各 zone 区域的各自抬高的页面数记录在本地的 zone_boosts 数组中。

  nr_boost_reclaim 可以认为是：为了修复碎片，需要额外回收的页数.

  NOTE

  zone 水位线抬高（*zone boosting*）机制仅在出现迁移类型碎片时触发，它通常会为目标 *node* 节点中每个水位线增加一个 *pageblock* 大小的 *(基)* 页数 *(同时会受 watermark_boost_factor* 因子影响和上限限制*)*。目的在于更早的触发内存回收。

  迁移类型是物理页的一种属性，用于决定该内存存储的内容是否可移动 *(迁移)*。这是为了将可迁移内存与不可迁移内存（即用于内核分配的内存）分开管理，从而避免因可迁移内存与不可迁移内存交错排布而导致内存碎片，从而无法释放出大阶页。

  在无法找到所需迁移类型的内存时，内核会从其他迁移类型的内存块中“窃取”内存 *(这里的内存块是特指物理地址连续的，阶数为 pageblock_order* 内存块*)*。发生这种情况时，内核就会通过 *boost_watermark()* 抬高水位线设定页面数量。这样在物理页分配函数 *rmqueue()* 中会因水位线抬高而提前为该节点唤醒 *kswapd* 来回收。

  NOTE

  zone 水位线抬高机制有两个核心作用

  1. 在 *rmqueue()* 中提前唤醒 *kswapd()* 回收内存. 目的*:* 提前干预，防止碎片越来越严重。

  2. 在内存回收时，即使节点已经平衡（*balanced*），也要额外多回收指定页数 *(*平均每个 *zone* 至少多一个 *pageblock)*。目的*:* 回收大块内存，下次分配就不用再去“偷”其他类型的内存块了，根治碎片。

  因为内存碎片是顽疾，等节点不平衡了再回收就太晚了。所以内核搞了 *boost* 机制提前回收，保持大块连续内存

- 3618-3622 如果内存节点（pgdat）处于不平衡状态，则忽略 zone 水位线抬高（boosting）机制。跳转，重新按照提高前水位线进行回收。

  node 节点不平衡说明内存不够用了，因此这时候得先把碎片问题放一放，先保证内存够用，不搞碎片优化。内核会把抬高的水位线 Δ(增量) 清零。下次 kswapd 醒来重新判断要不要再抬高，不会一直保留这个临时调整。

  zone 水位线抬高在效果上被视为一种独立的非直接接回收模式——仅当节点已经平衡时，我们才执行 zone 水位线相关的逻辑。否则将 nr_boost_reclaim 重置为 0，重新开始循环。

- 3629-3630 如果未启用抬高机制，仅在无任何可用 zone 时才继续执行回收。换言之，也就是该 node 节点处于内存不平衡状态就继续回收。否则，达成目标退出。

  当节点内存已经平衡，但开启了 zone 水位线临时抬高机制时：循环不会因为“节点平衡”而退出，而是达到下面任一条件才退出：

  因抬高水位线提高的页回收增量回收完成（nr_boost_reclaim 达到目标，也就是为了修复了碎片，额外需要回收的页数也回收到了）

  回收优先级已经到 0 还没完成，放弃

- 3633-3634 先按默认优先级 DEF_PRIORITY 进行回收。这里的优先级其实是一个因子，譬如需要扫描的页数量是 n，那么考虑到这个优先级的话，实际要回收的页数量是:$\frac{1}{priority} * n$.

  进入核心回收循环会反复尝试执行回收；除非回收的页数已经达到要求，或者 raise_priority 被清除，否则逐步提高回收优先级 (对应 sc.priority 值逐步变小)

  如果当前处于抬高水位线模式—换言之，在回收页数量到达因抬高水位线造成的页增量之前—我们确保回收优先级不会低于 DEF_PRIORITY - 2（注意：优先级数值越小，意味着扫描更多的页片），因此在这种情况下禁止继续提高优先级（即禁止每次循环递减 struct scan_control->priority）。我们通过设置 struct scan_control->may_writepage 来决定回收时是否允许回写脏页：当系统不在笔记本模式（该模式通过避免回收时执行回写，防止旋转式硬盘启动），且不在 zone 水位线抬高模式下才允许回写——因为 boost 模式下，回写操作只会延缓快速释放内存，不利于避免页块碎片。优先级越高 prio 值就越低，prio 越低扫描页就越多，扫描越多页越容易写磁盘，就越慢。这违背了 boost 快速回收的目的.

- 3643 禁止执行磁盘交换. 说明见 3658 行禁止写回。

- 3651 通过 age_active_anon() 对活跃链表上的匿名页进行“老化”，如果 inactive_is_low() 显示非活跃匿名链表太小，就通过 shrink_active_list() 将这些页迁移到非活跃链表。内核会格外谨慎地对活跃匿名页 LRU 链表执行后台老化操作，以保证非活跃链表中始终保有足够数量的页片，使得匿名页在被换出之前能够有机会被再次访问。此外，相比于文件缓存页，我们通常更不愿意换出匿名页（可以类比页缓存权衡机制），因此也必须保证匿名页能够被持续、稳定地老化处理

- 3657 如果回收进展不顺利（表现为回收优先级降至第 3 级及更低），我们会强制忽略笔记本模式，允许在回收过程中回写脏页（注意该逻辑不影响区域提升模式，因为我们已经明确限制了该模式下的优先级无法降至这一水平）

- 3672 kswapd_shrink_node() 真正执行内存回收。该函数的返回值是布尔值用于标识是否已经至少回收了请求数量的页面：

  如果是，则无需提高回收优先级（下次检查节点平衡状态时即可直接退出循环）；反之则需要提高优先级 (即降低 sc.priority)。

- 3680-3682 既然已经已达到低水位线，就没有必要让进程在 pfmemalloc_wait 等待队列上继续被限流阻塞；此时它们已经可以安全地继续推进执行，于是唤醒这些线程正在 struct pglist_data->pfmemalloc_wait 等待队列上休眠。因此我们需要检查是否有等待线程，并通过 allow_direct_reclaim() 函数判断是否已释放足够内存，预留出充足空间以唤醒被阻塞的直接回收线程。如果满足条件，我们就唤醒所有休眠线程，让它们恢复执行直接回收。这个和回写限流或 IOD 限流上等待的线程不是一回事，区分，加表格

等待在 pfmemalloc_wait 上的线程是正在做直接内存回收的线程，它因为内存严重不足，为了保护预留的紧急内存（PF_MEMALLOC）而被阻塞. 它们等 kswapd 回收出足够内存，危险解除，才能被唤醒继续运行 这些 wait 点在哪些代码，待添加

- 3685-3687 进程冻结时不能持有文件系统回收锁，否则会卡死系统。因此暂时不做文件系统相关回收。让 kswapd 可以安全地被“冻结”（休眠）。解冻 / 醒来后，再把锁加回来把状态恢复回去，继续正常回收内存。这是通用内核机制，不是内存管理策略
- 3688-3689 检查内核线程是否已被终止，并执行相应处理。
- 3695 计算本次回收的页面：即 kswapd_shrink_node() 执行回收后最终设置在 struct scan_control->nr_reclaimed 中的总回收页数，与循环开始时保存在 nr_reclaimed 中的上一轮计数值之差。
- 3696 在抬高水位线模式下，我们的目标是至少回收与水位线抬高后页面总数相等的页面数量，因此将本次实际回收的页数从 nr_boost_reclaim 中扣除，该值用于判断是否需要继续维持区域提升回收模式
- 3703-3704 如果抬高水位线没有取得任何回收进展，则停止回收；因为此时无法将 I/O 请求排队，并且在极端情况下可能会导致无限循环。
- 3706-3707 鉴于在当前优先级下回收效果良好，只需重试回收即可，而非扫描超出必要数量的页片。除非本次回收页数量为 0 或明确要提升优先级才会把优先级提高。

  在因抬高水位线造成的页回收增量 $\Delta == 0$ 之前，优先级的值只能最多降到 (DEF_PRIORITY - 2)。
- 3708 循环会在 struct scan_control->priority 大于等于 1 时持续执行，这意味着我们最多扫描可用页片的一半 $(\frac{1}{2} * n)$。

- 3725 重置目标节点的各个 zone 的->watermark_boost 为 0
- 3733 唤醒 kcompacted 进行内存整理，清除内存碎片。
- 3736 读取保存的页面重载 (refaults) 的快照统计。

  snapshot_refaults() 函数用来读取统计快照，相应的写或保存页面重载次数或快照的函数是 workingset_refault()。workingset_refault() 在发生重缺页（refault）时被调用。

  文件页缺页处理（page fault）的最后一步：add_to_page_cache_lru() 会把新页面加入页缓存 + LRU 链表。在该函数的执行中会判断新页是否重载 (之前已经被加载到内存，但是后来该页被回收了) 的页 (通过影子项). 如果是，并且是”读”操作，那么会调用 workingset_refault() 保存 (增加) 此时的重载计数。

  同样的，对于匿名页的”读”操作，如果之前目标页也曾存在于内存，也会调用 workingset_refault() 来更新重载计数。

NOTE

为什么”写”导致的缺页重新载入没有触发工作集重载统计？

重缺页逻辑只统计命中 *shadow* 的读缺页。写缺页被排除，因为它们不代表对之前缓存数据的重读。内核社区认为写缺页不依赖旧缓存，它们是新写入，不是对之前缓存数据的重读，不能证明之前回收错了，计入会扭曲工作集检测。

- 3747 返回停止回收时的分配阶。prepare_kswapd_sleep() 函数会将该值纳入判断依据。

函数 snapshot_refaults() 会更新与目标节点关联的 struct lruvec 结构中的成员。

5.2.2 snapshot_refaults()

kswapd() > *balance_pgdat()* > *snapshot_refaults()*

```
2984 static void snapshot_refaults(struct mem_cgroup *target_memcg, pg_data_t *pgdat)
2985 {
2986     struct lruvec *target_lruvec;
2987     unsigned long refaults;
2988
2989     target_lruvec = mem_cgroup_lruvec(target_memcg, pgdat);
2990     refaults = lruvec_page_state(target_lruvec, WORKINGSET_ACTIVATE_ANON);
2991     target_lruvec->refaults[0] = refaults;
2992     refaults = lruvec_page_state(target_lruvec, WORKINGSET_ACTIVATE_FILE);
2993     target_lruvec->refaults[1] = refaults;
2994 }
```

- 2989 获取节点中目标 cgroup 的 lruvec 向量。
- 2990-2991 读取因发生重缺页 (refault), 而且 reafult distance<workingset_size, 从而而被判定为工作集，直接放入活跃 LRU 链表的匿名页的计数快照，存入 struct lruvec->refaults[0]。
- 2992-2993 同理对文件页计数的操作，见 2990-2991 的说明。

5.2.3 pgdat_balanced

非直接接回收的最终目的，是让目标 node 节点内存达到平衡状态。简单来说就是：在考虑了低端内存预留的前提下，至少有一个内存 zone，其空闲页面数达到或超过高水位线，可用于当前请求阶 (order) 的分配。该检查在 pgdat_balanced() 函数中完成

5.2.4 kswapd_shrink_node()

该函数源码如下:

kswapd() > *balance_pgdat()* > *kswapd_shrink_node()*

```
3494 static bool kswapd_shrink_node(pg_data_t *pgdat,
3495                 struct scan_control *sc)
3496 {
3497     struct zone *zone;
3498     int z;
3499
3500     /* Reclaim a number of pages proportional to the number of zones */
3501     sc->nr_to_reclaim = 0;
3502     for (z = 0; z <= sc->reclaim_idx; z++) {
3503         zone = pgdat->node_zones + z;
3504         if (!managed_zone(zone))
3505             continue;
3506
3507         sc->nr_to_reclaim += max(high_wmark_pages(zone), SWAP_CLUSTER_MAX);
3508     }
3509
3510     /*
3511      * Historically care was taken to put equal pressure on all zones but
3512      * now pressure is applied based on node LRU order.
3513      */
3514     shrink_node(pgdat, sc);
3515
```

```
3516     /*
3517      * Fragmentation may mean that the system cannot be rebalanced for
3518      * high-order allocations. If twice the allocation size has been
3519      * reclaimed then recheck watermarks only at order-0 to prevent
3520      * excessive reclaim. Assume that a process requested a high-order
3521      * can direct reclaim/compact.
3522      */
3523     if (sc->order && sc->nr_reclaimed >= compact_gap(sc->order))
3524         sc->order = 0;
3525 
3526     return sc->nr_scanned >= sc->nr_to_reclaim;
3527 }
```

- 3502-3508 对当前页面失衡的目标 node 节点中，可用于分配的最高索引的 zone 及以下的 zone 区域内的页面进行迭代。

  3507 确定总共应当尝试回收的页面数量。它针对每个可能用于分配的区域，选择回收等于该区域高水位线的页面数量。这里内核采取的是一种保守回收策略，不精确计算，只做粗粒度回收。

  内核认为 high_wmark 水位线是目标 zone 健康运行需要的最小空闲内存，内核直接把目标 node 各个 zone 的这个 “安全需求量” 之和作为本次后台回收应该回收的页数。它没有精确计算安全量-当前剩余内存的差值作为回收数量，而是保守的用安全值作为目标 node 的总回收目标。

  在每种场景下，将 SWAP_CLUSTER_MAX 作为尝试回收页面数量的最低阈值，这与它作为整个回收流程中操作的最小粒度保持一致。

- 3514 调用回收机制的入口函数 shrink_node() 执行实际的回收操作。

- 3523-3524 内存碎片可能导致系统无法以大阶页为基页单元实现页的负载均衡。因此在假定请求大阶页的进程可以执行直接内存回收与内存整理的情况下，如果已经回收了请求阶 (order) 两倍大小的内存 (通过 compact_gap() 函数估算内存整理可能需要的页面数 — 该值等于目标阶数对应页面数量的两倍)，此时已经可以满足请求大阶页的进程进行隔离回收需要的裕量了。那么则恢复以 0 阶页面为基页来检查水位线，可以安全地将回收阶数重置为 0 阶，以避免过度回收。

  唯一调用 kswapd_shrink_node() 的地方是 balancd_pgdat()。该函数会将 sc.order(有可能如上所示被 kswapd_shrink_node() 改成 0 了) 作为 return 值返回给更上一层调用者 kswapd()。从上面代码和说明可以看出，返回的 sc.order 只可能是两个值之一：一是请求的阶 (order); 二是 0。

  kswapd() 中会判断返回的 sc.order 是否比原生请求阶小，如果是，那么说明大阶页的回收失败。返回了 0。这时 kswapd() 会考虑休眠。

  在 kswapd() 中会有两次休眠，第一次是尝试短暂定时休眠，第二次是长休眠直到被唤醒。短暂休眠被唤醒后会检查是超时唤醒还是提前被唤醒。

  如果是提前唤醒，那么是有新的紧急大阶页需求没得到满足需要回收。如果新的需求阶大于休眠时的回收阶，那么直接用新的需求阶 (order) 作为回收阶。

  如果是超时唤醒，说明系统内存暂时还够用，因此进行长休眠直到被唤醒。唤醒后直接以休眠时的需求阶回收。

  此处添加图

- 3526 如果 kswapd 已经至少扫描了请求回收的页面数量，则返回 true。该函数的返回值被调用者用于判断是否需要提升扫描优先级。

5.3 try_to_freeze()

kswapd() try_to_freeze()

```
63 static inline bool try_to_freeze(void)
64 {
65     if (!(current->flags & PF_NOFREEZE))
66         debug_check_no_locks_held();
67     return try_to_freeze_unsafe();
68 }
```

该函数是内核线程在业务循环中调用，用于检测系统是否要求冻结本任务。用户进程不显式调用该接口，靠信号处理路径 get_signal() 里面自动调用该接口。

- 65 检测当前任务是否允许冻结。

  PF_NOFREEZE 表示禁止冻结当前 task。

  如果当前任务允许冻结，那么必须在冻结之前释放全部锁，带着锁休眠会造成死锁。*debug_check_no_locks_held()* 函数主要进行锁的检查。如果持有锁，会进行栈回朔告警。该函数要在打开 *CONFIG_LOCKDEP* 后才生效，否则是空函数。

- 67 *try_to_freeze_unsafe()* 进行冻结判断。

  此处的 *_unsafe* 代表: 调用者必须保证不持有锁，该函数内不会去释放锁。如果调用者没有释放锁，那么该函数不保证安全。

kswapd() try_to_freeze() try_to_freeze_unsafe()

```
55 static inline bool try_to_freeze_unsafe(void)
56 {
57     might_sleep();
58     if (likely(!freezing(current)))
59         return false;
60     return __refrigerator(false);
61 }
```

- 57 检测这个点是否可以睡眠，如果在原子上下文中会告警。
- 58 判断是否需要冻结任务。
- 60 进入冻结。

kswapd() try_to_freeze() try_to_freeze_unsafe() __refrigerator()

```
56 bool __refrigerator(bool check_kthr_stop)
57 {
58     /* Hmm, should we be allowed to suspend when there are realtime
59        processes around? */
60     bool was_frozen = false;
61     long save = current->state;
62
63     pr_debug("%s entered refrigerator\n", current->comm);
64
65     for (;;) {
66         set_current_state(TASK_UNINTERRUPTIBLE);
67
68         spin_lock_irq(&freezer_lock);
69         current->flags |= PF_FROZEN;
```

```
70          if (!freezing(current) ||
71              (check_kthr_stop && kthread_should_stop()))
72              current->flags &= ~PF_FROZEN;
73          spin_unlock_irq(&freezer_lock);
74
75          if (!(current->flags & PF_FROZEN))
76              break;
77          was_frozen = true;
78          schedule();
79      }
80
81      pr_debug("%s left refrigerator\n", current->comm);
82
83      /*
84       * Restore saved task state before returning.  The mb'd version
85       * needs to be used; otherwise, it might silently break
86       * synchronization which depends on ordered task state change.
87       */
88      set_current_state(save);
89
90      return was_frozen;
91  }
92  EXPORT_SYMBOL(__refrigerator);
```

该函数循环让出 cpu 直到解冻结。

- 66 将 task 运行状态置为" 不可打断的休眠 (*TASK_UNINTERRUPTIBLE*)"，也就是说普通信号不能唤醒本线程。只能内核主动 wakeup。
- 69 给当前线程置" 冻结" 标记**PF_FROZEN**。
- 70 检查全局冻结条件是否已经撤销，判断是否需要继续冻结。
- 71 判断是否收到停止的请求。若是，要退出冻结允许目标线程退出。
- 72 清除 task_struct->flags 的冻结标记位 *PF_FROZEN*。
- 75-76 *PF_FROZEN* 被清除，则退出冻结。
- 78 主动让出调度，休眠。

kswapd() > *try_to_freeze()* > *try_to_freeze_unsafe()* > *__refrigerator()* > *freezing()*

```
35  static inline bool freezing(struct task_struct *p)
36  {
37      if (likely(!atomic_read(&system_freezing_cnt)))
38          return false;
39      return freezing_slow_path(p);
40  }
```

- 37 全局冻结计数为 0。系统没有发起任何冻结请求，直接返回。

  这是热路径优化。因为内核线程会高频调用 try_to_freeze(), 但绝大多数时候不需要冻结。为了避免每次都进入慢速路径拿锁、判断。

- 39 确实有冻结发生时进入慢速路径。

kswapd() > *...* > *__refrigerator()* > *freezing()* > *freezing_slow_path()*

```
37 bool freezing_slow_path(struct task_struct *p)
38 {
39     if (p->flags & (PF_NOFREEZE | PF_SUSPEND_TASK))
40         return false;
41
42     if (test_tsk_thread_flag(p, TIF_MEMDIE))
43         return false;
44
45     if (pm_nosig_freezing || cgroup_freezing(p))
46         return true;
47
48     if (pm_freezing && !(p->flags & PF_KTHREAD))
49         return true;
50
51     return false;
52 }
53 EXPORT_SYMBOL(freezing_slow_path);
```

- 39-40 当前 task 禁止被冻结，或者当前线程就是执行 suspend 冻结的线程，则不冻结。

  PF_SUSPEND_TASK 是专门给 suspend/resume 内核线程自身的标记。

  譬如 suspend 完整流程:

  freeze_processes() 设置全局 pm_freezing/pm_nosig_freezing,system_freezing_cnt++, 冻结系统上几乎所有用户进程、普通内核线程。冻结完所有其他任务后，该线程继续设备的 suspend，之后会 resume。如果该线程冻结，则系统彻底死锁。

- 42-43 Oom 被 kill 的线程不参与冻结。

- 45-46 suspend 冻结线程分两步

  - 阶段 1: $pm_freezing = true$

    只冻结用户态进程。靠伪造信号 fake_signal_wake_up() 唤醒进程。进程返回用户空间时，信号路径触发 try_to_freeze() 进入冻结。内核线程在此阶段不受影响，进行执行。

  - 阶段 2: $pm_nosig_freezing = true$

    用户进程已经全部冻结。此时要冻结可冻结的普通内核线程 + 可冻结的工作队列。

    内核线程不返回用户态，不能靠信号冻结。只能靠内核线程自己在业务循环中主动调用 try_to_freeze() 检测状态。此处的 *_nosig_* 也就是 *no_signal* 的意思。

  这里满足两个条件之一就冻结:

  $pm_nosig_freezing = true$ 意味着冻结进入第二阶段，需要冻结普通内核线程。

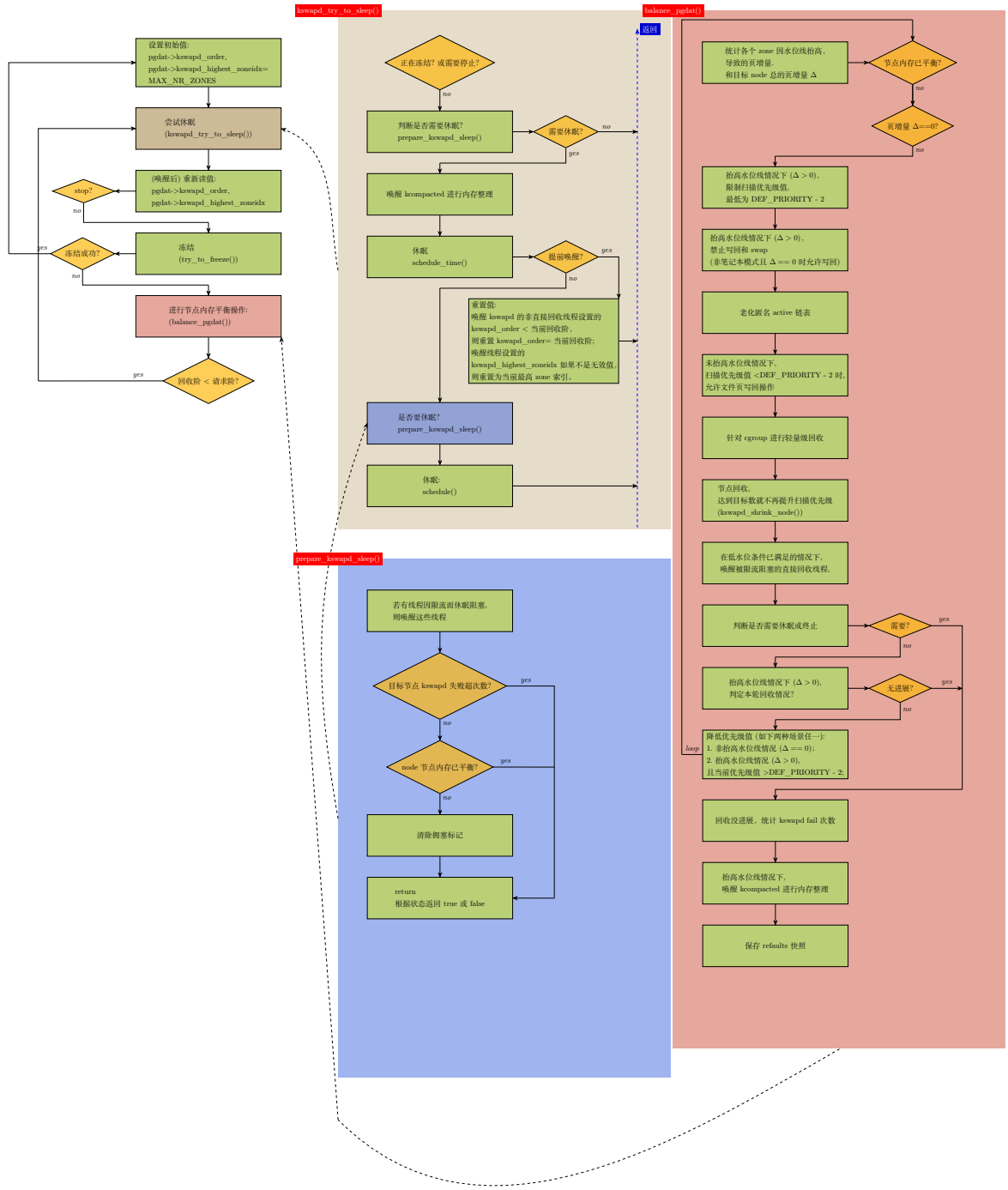
  目标任务所属 cgroup 设置了冻结。cgroup 用户不受用户/内核线程区分。

- 48-49 只冻结用户态进程，不冻结内核线程。

  这里是冻结第一阶段。PF_KTHREAD 用于判断是否内核线程。

综上，这里返回给调用者 true 意味着要冻结，返回 false 意味着无需冻结。

整个 kswapd 的代码大致流程如下图 (省略了一些分支，特别是 balance_pgdat() 函数中):



5.4 备忘

5.4.1 结构体 scan_control 成员说明

struct scan_control 是 Linux 内核页面回收子系统的核心控制结构，用于封装单 (每) 次页面回收操作的目标、范围、策略、优先级与运行状态。内核在执行内存回收时，会根据该结构体的配置决定回收强度、扫描范围、允许的操作类型，并在回收过程中更新统计信息与执行状态。该结构体贯穿整个回收流程，是协调内存回收策略与机制的关键数据结构。

每次页面回收都会创建一个 scan_control。它相当于每次回收任务的”任务单”。内核回收函数会用

它作为入参，按照它的配置执行回收 (当然，就它的成员参数而言，有可能在回收执行流程中发生参数值的改变)。

```
66  struct scan_control {
67      /* How many pages shrink_list() should reclaim */
68      unsigned long nr_to_reclaim;
69
70      /*
71       * Nodemask of nodes allowed by the caller. If NULL, all nodes
72       * are scanned.
73       */
74      nodemask_t  *nodemask;
75
76      /*
77       * The memory cgroup that hit its limit and as a result is the
78       * primary target of this reclaim invocation.
79       */
80      struct mem_cgroup *target_mem_cgroup;
81
82      /*
83       * Scan pressure balancing between anon and file LRUs
84       */
85      unsigned long   anon_cost;
86      unsigned long   file_cost;
87
88      /* Can active pages be deactivated as part of reclaim? */
89  #define DEACTIVATE_ANON 1
90  #define DEACTIVATE_FILE 2
91      unsigned int may_deactivate:2;
92      unsigned int force_deactivate:1;
93      unsigned int skipped_deactivate:1;
94
95      /* Writepage batching in laptop mode; RECLAIM_WRITE */
96      unsigned int may_writepage:1;
97
98      /* Can mapped pages be reclaimed? */
99      unsigned int may_unmap:1;
100
101     /* Can pages be swapped as part of reclaim? */
102     unsigned int may_swap:1;
103
104     /*
105      * Cgroups are not reclaimed below their configured memory.low,
106      * unless we threaten to OOM. If any cgroups are skipped due to
107      * memory.low and nothing was reclaimed, go back for memory.low.
108      */
109     unsigned int memcg_low_reclaim:1;
110     unsigned int memcg_low_skipped:1;
111
112     unsigned int hibernation_mode:1;
113
114     /* One of the zones is ready for compaction */
115     unsigned int compaction_ready:1;
116
117     /* There is easily reclaimable cold cache in the current node */
118     unsigned int cache_trim_mode:1;
119
120     /* The file pages on the current node are dangerously low */
121     unsigned int file_is_tiny:1;
122
123     /* Allocation order */
124     s8 order;
125
126     /* Scan (total_size >> priority) pages at once */
```

```
127     s8 priority;
128
129     /* The highest zone to isolate pages for reclaim from */
130     s8 reclaim_idx;
131
132     /* This context's GFP mask */
133     gfp_t gfp_mask;
134
135     /* Incremented by the number of inactive pages that were scanned */
136     unsigned long nr_scanned;
137
138     /* Number of pages freed so far during a call to shrink_zones() */
139     unsigned long nr_reclaimed;
140
141     struct {
142         unsigned int dirty;
143         unsigned int unqueued_dirty;
144         unsigned int congested;
145         unsigned int writeback;
146         unsigned int immediate;
147         unsigned int file_taken;
148         unsigned int taken;
149     } nr;
150
151     /* for recording the reclaimed slab by now */
152     struct reclaim_state reclaim_state;
153 };
```

它的各成员说明如下:

- 68 ***nr__to__reclaim*** 指定本次尝试回收的基页数量。
- 74 ***nodemask*** 指定参与本次回收的内存节点的集合。
- 85 ***anon__cost*** 回收匿名页的开销，该值由 workingset 逻辑决定；该逻辑会检测之前被回收的页是否很快又重新载入内存 (refault)。匿名页与文件页的开销均保存在 struct lruvec 的对应字段中，由 lru_note_cost() 计算得出；这一过程由 workingset_refault() 中的工作集 (workingset) 逻辑触发。shrink_node() 只根据 struct lruvec 中的值来设置 anon_cost 与 file_cost 字段。
- 86 ***file__cost*** 由 shrink_node () 函数设置——表示回收文件页的开销，含义参看 anon_cost 项。
- 91 ***may__deactivate*** 由 shrink_node() 设置——决定内存回收时是否可以将页面从活跃链表降级到欠活跃链表；换言之，该标志决定 shrink_list() 是否可以调用 shrink_active_list()。针对该成员位内核分别为匿名页与文件页定义了独立的标志：DEACTIVATE_ANON 和 DEACTIVATE_FILE。
- 92 ***force__deactivate*** 由 do_try_to_free_pages() 设置，属于直接回收流程的一部分——强制将文件页与匿名页全部从活跃链表降级。该标志会在某次页面降级被跳过之后设置，以确保后续回收流程一定会执行页面降级操作。
- 93 ***skipped__deactivate*** 由 shrink_node() 设置——表示 force_deactivate 已被设置，shrink_list() 没有调用 shrink_active_list()，即跳过了活跃页面降级流程。
- 96 ***may__writepage*** 用户自定义——指定在回收过程中是否允许将脏页回写到磁盘。该标志在 shrink_page_list() 中被检查。
- 102 ***may__swap*** 用户自定义——指定在回收过程中是否允许将页面 swap out 到磁盘。

- 115 ***compaction_ready*** 用于直接回收。表示内存整理的执行优先于内存回收。该标志在 shrink_zones() 中设置，由 compaction_ready() 判断是否生效；do_try_to_free_pages() 会检查该标志，若已设置则中止回收流程，优先执行内存整理。
- 118 ***cache_trim_mode*** 由 shrink_node() 设置，表示当前处于缓存修剪模式：当前系统禁止降级文件页、且 LRU 链表中存在大量欠活跃文件页，此时仅允许回收文件页。换言之，也就是修剪模式顾名思义，只修剪去掉枯枝 (欠活跃文件 LRU)。
- 121 ***file_is_tiny*** 由 shrink_node() 设置——表示系统满足以下场景：存在大量可回收匿名页、禁止匿名页降级，并且即使释放全部文件页也无法满足间接回收需求；即：可回收文件页与空闲文件页的总页数小于节点内所有内存区域的水位线之和。此时，回收扫描仅限定在匿名页范围内。

- 124 ***order*** 指定触发本次内存回收的内存请求的阶数 (order)。
- 127 ***priority*** 字面意思是:优先级。实际用处是作为一个因子来指定本轮回收操作中应当扫描的页面数量。例如，如果该值是 n，则意味着仅扫描 LRU 向量中的 $\frac{1}{1<<n} = \frac{1}{2^n}$ 页面。
- 130 ***reclaim_idx*** 指定可分配的最大内存 zone 区域索引。
- 133 ***gfp_mask*** 指定分配时的 GFP(get frame page) 掩码，代表内存分配所受的限制条件。
- 136 ***nr_scanned*** 由回收机制设置——记录回收过程中扫描（检查）的基页总数。
- 139 ***nr_reclaimed*** 由回收机制设置——记录回收过程中成功回收并释放的基页总数。

- 142 ***nr.dirty*** 由回收机制设置——整个回收过程中，产生的 nr_dirty 数量全部总和。即扫描过程中处于脏 (PG_dirty 标识) 或回写状态 (PG_writeback 标识) 的基页数量。
- 143 ***nr.unqueued_dirty*** 由回收机制设置——整个回收过程中，产生的 nr_unqueued_dirty 计数总和，即已标记为脏页 (PG_dirty 标识) 但尚未提交回写 (未设置 PG_writeback 标志) 的基页数量。
- 144 ***nr.congested*** 由回收机制设置——整个回收过程中，产生的 nr_congested 计数总和，即同时设置了 PG_writeback 和 PG_reclaim 标志的页数量，意味着这些页面未完成回写又再次进入回收流程，反映了回收页的回写拥塞状况。
- 145 ***nr.writeback*** 由回收机制设置——整个回收过程中，产生的 nr_writeback 计数总和，即回收时正在执行回写 (PG_writeback 标识) 的基页数目，且不属于 nr.immediate 定义的立即处理页面。
- 146 ***nr.immediate*** 由回收机制设置——整个回收过程中，产生的 nr_immediate 计数总和——指正在回写 (已设置 PG_writeback 标志)、由 kswapd 扫描且同时标记了 PG_reclaim 标志的文件页数量 (以基页为单位)。这表明这些页面正在循环进入回收流程，且系统已判定当前发生了过量回写，具体体现为内存 node 节点已设置 PGDAT_WRITEBACK 标志。

  系统判定当前回写太多、过载了，需要优先、立即处理这些页面。用来触发紧急策略，让回收机制优先处理这些页面。

  和 nr.congested 区别在于 node 节点设置了 PGDAT_WRITEBACK 标志，要紧急处理这些页。
- 147 ***nr.file_taken*** 由回收机制设置——为执行回收而从 LRU 链表中隔离出来的文件页总数 (以基页为单位)。隔离操作由 isolate_lru_pages() 函数完成。

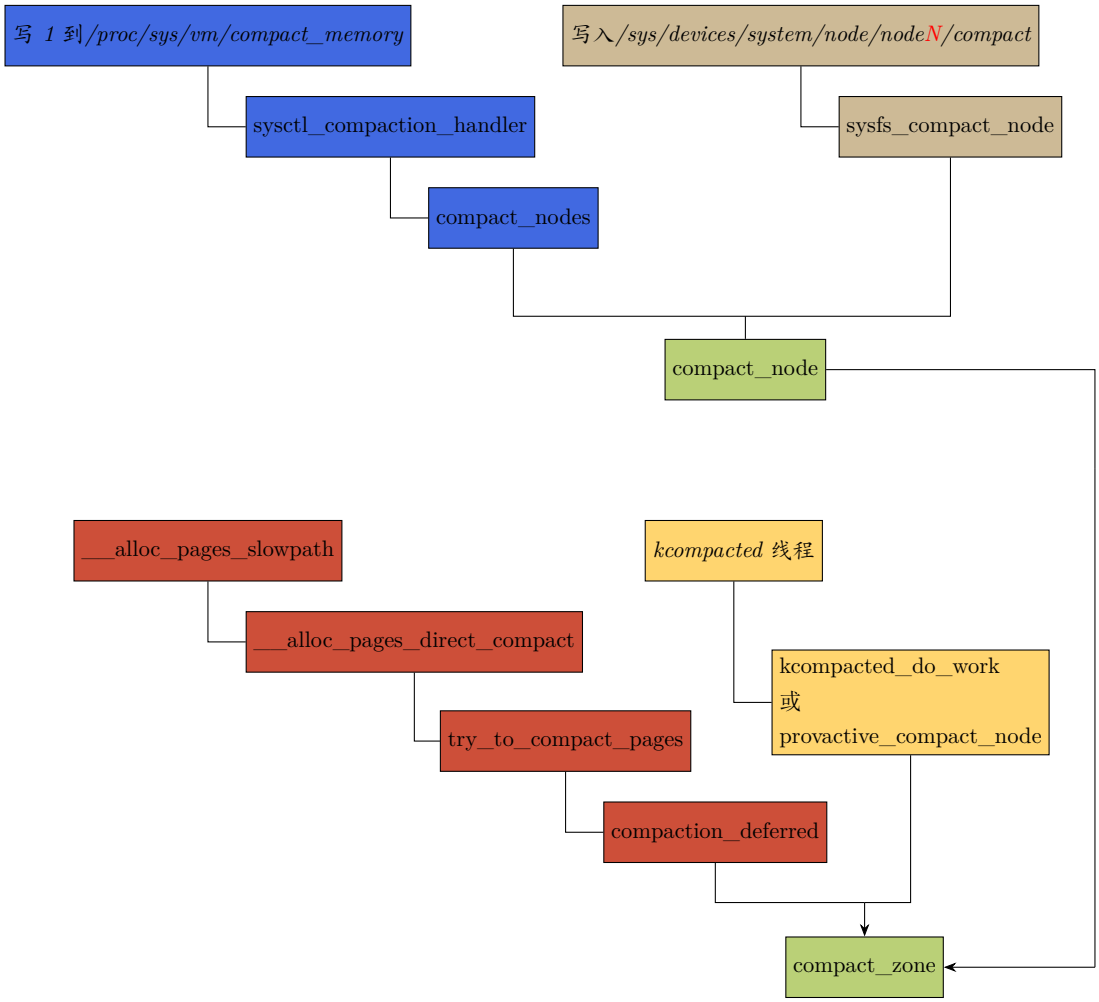
- 148 ***nr.taken*** 由回收机制设置——为执行回收而从 LRU 链表中隔离出来的所有页总数 (以基页为单位)，包含文件页与匿名页。隔离操作由 isolate_lru_pages() 函数完成。
- 152 ***reclaim_state*** 由回收机制设置——特定于 Slab 分配器 (组件) 的状态信息，在 Slab 分配器 (组件) 完成内存释放后进行更新. 只用于 slab 分配器回收，和页面回收无关.

此处列出了 *struct scan_control* 各个成员变量的作用，这些成员元素关系着由它所管控的内存回收 *shrink_node* 流程中的回收配置。

6

内存整理

不但内存分配的 slowpath 路径会调用 compact_zone() 进行内存整理，应用层用户空间也可以显式触发内存整理。各调用路径如下:



下面是 compact_zone() 源码:

6.1 compact_zone()

mm/compaction.c :: compact_zone()

```
2193 static enum compact_result
2194 compact_zone(struct compact_control *cc, struct capture_control *capc)
2195 {
2196     enum compact_result ret;
2197     unsigned long start_pfn = cc->zone->zone_start_pfn;
2198     unsigned long end_pfn = zone_end_pfn(cc->zone);
2199     unsigned long last_migrated_pfn;
2200     const bool sync = cc->mode != MIGRATE_ASYNC;
2201     bool update_cached;
2202
2203     /*
2204      * These counters track activities during zone compaction.  Initialize
2205      * them before compacting a new zone.
2206      */
2207     cc->total_migrate_scanned = 0;
2208     cc->total_free_scanned = 0;
2209     cc->nr_migratepages = 0;
2210     cc->nr_freepages = 0;
2211     INIT_LIST_HEAD(&cc->freepages);
2212     INIT_LIST_HEAD(&cc->migratepages);
2213
2214     cc->migratetype = gfp_migratetype(cc->gfp_mask);
2215     ret = compaction_suitable(cc->zone, cc->order, cc->alloc_flags,
2216                               cc->highest_zoneidx);
2217     /* Compaction is likely to fail */
2218     if (ret == COMPACT_SUCCESS || ret == COMPACT_SKIPPED)
2219         return ret;
2220
2221     /* huh, compaction_suitable is returning something unexpected */
2222     VM_BUG_ON(ret != COMPACT_CONTINUE);
2223
2224     /*
2225      * Clear pageblock skip if there were failures recently and compaction
2226      * is about to be retried after being deferred.
2227      */
2228     if (compaction_restarting(cc->zone, cc->order))
2229         __reset_isolation_suitable(cc->zone);
```

- 2197-2198 分别是目标 zone 的起始页帧号和 zone 的结束页帧号。

  需要注意的是结束页帧号不是有效页帧号，换言之，zone 的有效页帧范围 $\in [start_pfn, end_pfn)$。

- 2199 临时记录上一次成功完成页面迁移的物理页帧号的变量。

- 2200 标识本次整理是同步还是异步模式。

  需要注意的是: 在内存整理中的异步指不阻塞。

  $MIGRATE_ASYNC + (skip_on_failure = true)$ 是异步直接整理。也就是在直接整理过程中的异步 (不阻塞)。只有直接整理时才能打开 $skip_on_failure = true$。

  $MIGRATE_ASYNC + (direct_compaction = false)$ 是 kcompacted 后台整理的异步。可以阻塞。

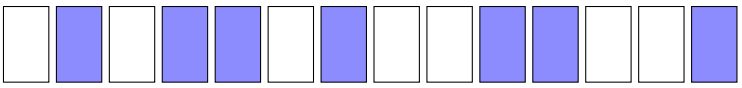
  综上，只有**异步直接整理** ($MIGRATE_ASYNC + cc- > direct_compaction$) 才启用 $skip_on_failure$; 单纯 kcompactd 后台异步整理不开启。所有同步模式 $skip_on_failure$=$false$。

- 2201 zone 会缓存上一次整理停下的物理页帧号，下次整理不用从 zone_start_pfn 开始进行断点续扫。该变量决定是否更新该缓存。
- 2207 该成员表示迁移扫描器总共扫过了多少页。
- 2208 该变量表示空闲扫描器总共扫过了多少页。
- 2209 本次隔离出来待迁移的页的数量。

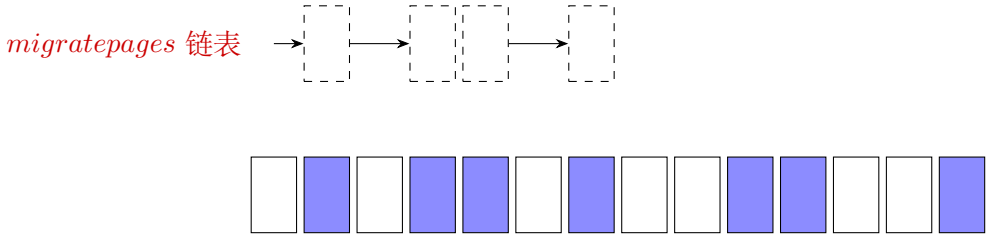
  每轮小迭代 *isolate_migratepages()* 会把可迁移页摘取出来放到 *cc->migratepages* 链表。
- 2211 初始化链表头，该链表存放隔离出来的空闲页，作为迁移的目的位置。
- 2212 初始化链表头。该链表存放隔离出来准备迁走的页面。

  扫描时迁移扫描器 (migrate scanner) 从低 PFN 向高 PFN 扫描，找可以被迁移走的页。而空闲扫描器 (free scanner) 从高 PFN 向低 PFN 扫描，找空闲页当迁移的目的地。直到这两个扫描器相遇时终止扫描。需要注意的是这里的相遇指的是迁移扫描器扫描到的 PFN ≥ 空闲扫描器扫描到的 PFN 时。

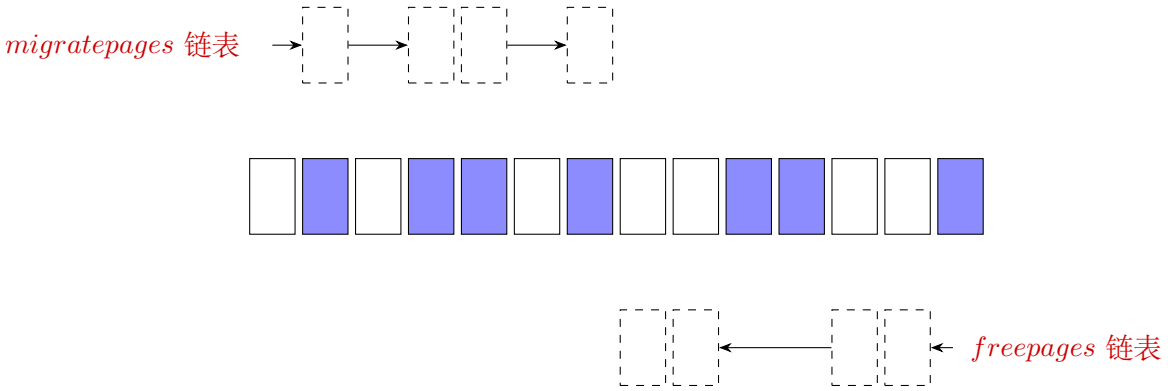
  假设 zone 区域初始状态如图所示，每个蓝色框代表一个被使用页，白色框代表空闲页。



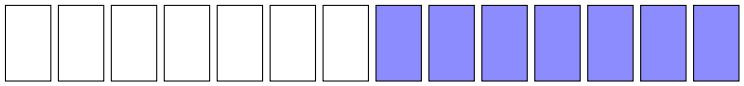
迁移扫描器 (migrate scanner) 从低 PFN 向高 PFN 扫描，把扫描到的可迁移页放到 $cc->migratepages$ 链表。



空闲页扫描器 (free scanner) 从尾部 (高 PFN) 向低 PFN 扫描，把扫描到的空闲页放到 $cc->freepages$ 链表。



将 $cc->migratepages$ 链表中的页数据放到 $cc->freepages$ 链表中的空闲页里，最终相当于可迁移页和空闲页交换了位置，最终 zone 中的页如下图所示。



这套机制称为 *memory compaction*，”compaction” 中文直译过来的意思是: 紧缩、压实，这里空闲页就像是气泡一样，迁移的过程相当于把这些气泡挤压了出来，把真正有数据的页压实了。同时，这也可以看做是一个归纳整理的过程，为了方便起见，本笔记中统一称之为**内存整理**。

- 2214 从 gfp 解析出本次分配希望从伙伴系统的哪条迁移类型的 freelist 取空闲块。
- 2215 判断对应 zone 是否适合进行本次 order 阶的内存碎片整理。也就是估算通过整理是否能凑出目标阶的连续页。
- 2218 如果不需要进行内存整理 (COMPACT_SUCCESS)，或当前 zone 不适合内存整理 (COMPACT_SKIPPED)，那么退出。不对 zone 执行整理。
- 2222 检测非法返回值。
- 2228 检查是否需要重新启动内存整理。

  内核有整理退避 (compaction defer) 机制，多次尝试整理某个 order 都失败。内核会给该 zone 打一个延迟标记，短期内不再对该 zone 进行反复整理。*compaction_restarting()* 返回 true 代表延时到期后，第一次启动整理。

  如果内存整理失败，那之后如果内存再次请求整理的 order 大于或等于失败那次，就会忽略 (skip) 这次的请求。在忽略 (skip) 行为达到一定次数后将会重新尝试整理，此时需要清除 pageblock 的” 忽略 (skip)” 标记。可以通过设置 *compact_blockship_flush = true* 来清除 PB_migrate_skip 位。

6.1.1 compaction_restarting()

compaction_restarting() 的实现如下。

compact_zone() > *compaction_restarting()*

```
212 bool compaction_restarting(struct zone *zone, int order)
213 {
214     if (order < zone->compact_order_failed)
215         return false;
216
217     return zone->compact_defer_shift == COMPACT_MAX_DEFER_SHIFT &&
218         zone->compact_considered >= 1UL << zone->compact_defer_shift;
219 }
```

- 214 compact_order_failed 记录了内存整理可能会失败的最小 order 数。例如，若某次尝试分配 order=3 (8 页连续内存) 失败，则该值会被设为 3. 也意味着如果请求比该 order 大的内存整理操作，默认就会失败。
- 217 在内存整理失败后，系统将忽略至少 ($1 << COMPACT_MAX_DEFER_SHIFT$) 次内存整理的请求后才会再次尝试。

  COMPACT_MAX_DEFER_SHIFT 默认为 6。

自上次失败以来内存整理请求的 skip 次数通过->compact_considered 跟踪。但是需要说明的是，skip 计数针对的是 order 大于或等于上次失败请求 order 的整理请求次数，而不是最近一次 fail 后所有内存整理请求的次数。

6.1.2 compaction_suitable()

回到 compact_zone() 函数 2215 行调用的函数 compaction_suitable()。该函数用于检测目标 zone 区域是否适合执行内存整理。

compact_zone() *compaction_suitable()*

```
2123 enum compact_result compaction_suitable(struct zone *zone, int order,
2124                     unsigned int alloc_flags,
2125                     int highest_zoneidx)
2126 {
2127     enum compact_result ret;
2128     int fragindex;
2129
2130     ret = __compaction_suitable(zone, order, alloc_flags, highest_zoneidx,
2131                     zone_page_state(zone, NR_FREE_PAGES));
2132     /*
2133      * fragmentation index determines if allocation failures are due to
2134      * low memory or external fragmentation
2135      *
2136      * index of -1000 would imply allocations might succeed depending on
2137      * watermarks, but we already failed the high-order watermark check
2138      * index towards 0 implies failure is due to lack of memory
2139      * index towards 1000 implies failure is due to fragmentation
2140      *
2141      * Only compact if a failure would be due to fragmentation. Also
2142      * ignore fragindex for non-costly orders where the alternative to
2143      * a successful reclaim/compaction is OOM. Fragindex and the
2144      * vm.extfrag_threshold sysctl is meant as a heuristic to prevent
2145      * excessive compaction for costly orders, but it should not be at the
2146      * expense of system stability.
2147      */
2148     if (ret == COMPACT_CONTINUE && (order > PAGE_ALLOC_COSTLY_ORDER)) {
2149         fragindex = fragmentation_index(zone, order);
2150         if (fragindex >= 0 && fragindex <= sysctl_extfrag_threshold)
2151             ret = COMPACT_NOT_SUITABLE_ZONE;
2152     }
2153
2154     trace_mm_compaction_suitable(zone, order, ret);
2155     if (ret == COMPACT_NOT_SUITABLE_ZONE)
2156         ret = COMPACT_SKIPPED;
2157
2158     return ret;
2159 }
```

- 2130 调用 ___compaction_suitable() 函数进行水位线检查。该函数的细节见后面。
- 2148 COMPACT_CONTINUE 意味着目标 zone 中的空闲单页满足 low 或 min(order>=PAGE_ALLOC_COSTLY_ORDER 时) 水位线的要求，但是要满足请求阶的分配还需要进行内存整理。此处就是这种情况下做进一步考虑。

请求阶是否过大 ($order > PAGE_ALLOC_COSTLY_ORDER$ 时), 请求阶过大会导致内存的分配花费较大的代价。

$PAGE_ALLOC_COSTLY_ORDER(3)$, > 该分配阶内核认为是高代价的内存分配，因为 > 该阶的页申请，只靠 回收 + 伙伴系统的自然合并 成功率很低，往往要执行代价昂贵的内存迁移或整理，因此叫高代价分配。

- 2149 获取内存碎片指数。

  碎片指数仅在请求大小的内存分配失败时才有意义。若请求分配失败，该指数可表明问题根源是由于内存不足还是外部碎片造成的。其值可用于决定请求失败后接下来应该进行页面回收还是内存整理。碎片指数可能值的范围是:$[-1000, 1000]$。

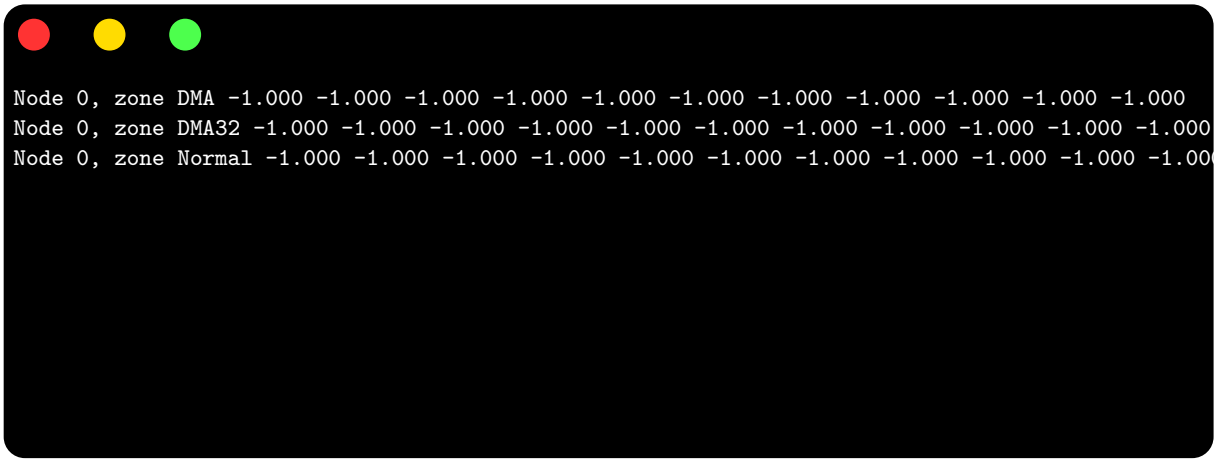
  - 指数**-1000** 意味着分配可能成功, 但之前在大阶 (high-order) 页水位线检查中失败了。
  - 指数趋于 **0** 意味着失败是由于内存不足。
  - 指数趋于 **1000** 意味着失败是由于碎片引起的。

  仅在预期失败是由于碎片问题引起的才进行内存整理。

  在开启 ***CONFIG_DEBUG_FS*** 和 ***CONFIG_COMPACTION*** 后，并且通过:

  mount -t debugfs none /sys/kernel/debug

  挂载文件系统后,可以通过*/sys/kenrel/debug/extfrag/extfrag_index*在用户空间查看碎片指数(如图)。



数值的每一列依次对应 order-0、order-1...order-10。另外，输出的是除以 1000 之后的浮点数，范围在 $[-1.000, 1.000]$。而内核代码返回的范围是 $[-1000, 1000]$。

另外，对于非高代价分配阶的情况忽略碎片指数，因为在这种场景下，相较于成功的回收/整理，唯一替代方案就是 OoM(内存耗尽)。

- 2150 碎片化指数在 [0, sysctl_extfrag_threshold] 范围之内，说明碎片化不严重，不适合整理，置值 COMPACT_NOT_SUITABLE_ZONE。sysctl_extfrag_threshold 是通过属性节点*/proc/sys/vm/extfrag_threshold* 预设的值, 其值的范围是 0 ~ 1000，只有超过这个值才进行内存碎片整理。默认值是 500

- 2151 如果因为前面的执行判断碎片化不严重，则返回 COMPACT_SKIPPED 告知调用者不执行整理。

6.1.2.1 fragmentation_index()

fragmentation_index() 用于获取内存碎片指数。

compact_zone() → compaction_suitable() → fragmentation_index()

```
1118 int fragmentation_index(struct zone *zone, unsigned int order)
1119 {
1120     struct contig_page_info info;
1121
1122     fill_contig_page_info(zone, order, &info);
1123     return __fragmentation_index(order, &info);
1124 }
```

- 1122 填充用于获取碎片指数的连续页的信息。

compact_zone() → compaction_suitable() → fragmentation_index() → fill_contig_page_info()

```
1042 static void fill_contig_page_info(struct zone *zone,
1043                 unsigned int suitable_order,
1044                 struct contig_page_info *info)
1045 {
1046     unsigned int order;
1047
1048     info->free_pages = 0;
1049     info->free_blocks_total = 0;
1050     info->free_blocks_suitable = 0;
1051
1052     for (order = 0; order < MAX_ORDER; order++) {
1053         unsigned long blocks;
1054
1055         /* Count number of free blocks */
1056         blocks = zone->free_area[order].nr_free;
1057         info->free_blocks_total += blocks;
1058
1059         /* Count free base pages */
1060         info->free_pages += blocks << order;
1061
1062         /* Count the suitable free blocks */
1063         if (order >= suitable_order)
1064             info->free_blocks_suitable += blocks <<
1065                         (order - suitable_order);
1066     }
1067 }
```

- 1052 从最小阶 (order=0, 单页) 到最大阶 (MAX_ORDER-1) 进行循环, 每次处理一个阶的空闲块。计算目标内存区域 zone 中的空闲页帧数量、连续空闲页帧的数量，以及满足目标大小分配请求的足够大 (连续) 的空闲块数量。
- 1056 获取目标 zone 当前迭代阶的空闲块的数量。nr_free 表示的是 2^order 个连续物理页组成的空闲块的数量。例如:

  order=0: 1 页为一个块，nr_free 表示 order=0 的单页的页数量。

order=3: 8 页 (2^3) 为一个块，nr_free=5, 表示有 5 个 8 页的连续块，即 40 页空闲。

order=10: 1024 页 (2^10) 为一个块，nr_free=2 表示有 2 个 1024 页的连续块 (共 2048 个空闲页)

- 1057 对目标 zone 所有阶总空闲块数量不断循环累计。
- 1060 目标 zone 当前阶空闲块的数量换算成单页 ($order = 0$) 数量，并在循环中累计到总的空闲 (单) 页的统计量中。
- 1063 大阶块可拆分为多个目标阶块。计算当前大阶的空闲块可以拆分成多少个目标阶块。
- 1064 将上述计算结果累加到->free_blocks_suitable, 它表示能满足所申请大小的内存块的总数量。

回到 fragmentation_index() 函数 1123 行的调用:

compact_zone() > *compaction_suitable()* > *fragmentation_index()* > *__fragmentation_index()*

```
1076 static int __fragmentation_index(unsigned int order, struct contig_page_info *info)
1077 {
1078     unsigned long requested = 1UL << order;
1079
1080     if (WARN_ON_ONCE(order >= MAX_ORDER))
1081         return 0;
1082
1083     if (!info->free_blocks_total)
1084         return 0;
1085
1086     /* Fragmentation index only makes sense when a request would fail */
1087     if (info->free_blocks_suitable)
1088         return -1000;
1089
1090     /*
1091      * Index is between 0 and 1 so return within 3 decimal places
1092      *
1093      * 0 => allocation would fail due to lack of memory
1094      * 1 => allocation would fail due to fragmentation
1095      */
1096     return 1000 - div_u64( (1000+(div_u64(info->free_pages * 1000ULL, requested))), info
         ->free_blocks_total);
1097 }
```

- 1080 若 order 超过伙伴系统支持的最大阶数 (MAX_ORDER, 通常为 11), 触发内核警告并返回 0.
- 1086 若内存区域无任何空闲块，从 order-0 到 order-MAX_ORDER 阶没有任何空闲块，系统内存严重不足，无法评估碎片。
- 1087 存在满足请求大小的空闲块，返回-1000。表示分配不会失败，意味着此时碎片指数无意义。
- 1096 最终的碎片化指数需要在 [0,1000] 之间，是一个千分比的数值。整个公式如下:

$$fragmentation_index = 1000 - \left(\frac{1000 + \frac{1000 \cdot free_pages}{request_blk_pages}}{free_blocks_total} \right)$$

公式可变形:

$$
\begin{aligned}
fragmentation_index &= 1000 - \left(\frac{1000 + \frac{1000 \cdot free_pages}{request_blk_pages}}{free_blocks_total} \right) \\
&= 1000 - 1000 \times \left(\frac{1 + \frac{1 \times free_pages}{request_blk_pages}}{free_blocks_total} \right) \\
&= 1000 - 1000 \times \left(\frac{1 + \frac{1 \times free_pages}{request_blk_pages}}{1} \right) \cdot \frac{1}{free_blocks_total} \\
&= 1000 - 1000 \times \left(\frac{1 + \frac{free_pages}{request_blk_pages}}{1} \right) \cdot \frac{1}{free_blocks_total}
\end{aligned}
$$

上式中"+1" 是为了以单位 1 为基准后向对齐，在 $\frac{free_pages}{request_blk_pages}$ 计算后，如果结果是 < 1 的小数，真正得到的整数除法结果会是 0。换言之，也就是前向对齐得到的值。这时结果在 [0, 0.999] 这个区间，无论主导因素是" 内存耗尽" 还是" 碎片化严重", 在不后向对齐的情况下，都会是 0。此时无法正确区分内存不足还是碎片化。"+1" 后相当于后向对齐，"÷1" 得到的结果会是 1. 这样避免了特殊值 0 的情况。

又因为整型变量的除法会丢失小数点后的精度，为了得到小数点后三位，所以这里整体 ×1000.

先把 1000 剥离，上式最终可变形为:

$$
fragmentation_index = 1 - \frac{1 + \frac{free_pages}{request_blk_pages}}{free_blocks_total}
$$

即，

$$
\text{碎片指数} = 1 - \frac{1 + \frac{\text{空闲页总数}}{\text{请求阶对应的基页数目}}}{\text{各个阶的空闲块总和}}
$$

需要注意的是: 这个公式是在 1087 行判断为 fail 才有意义。换言之，也就是不存在任何 ≥ 请求阶大小的空闲连续页块时，这个碎片指数才有意义。所以修正后的表示如下:

$$
\text{碎片指数} = 1 - \frac{1 + \frac{\text{空闲页数}}{\text{请求阶对应的基页数目}}}{\text{各个阶的空闲块总和}} \quad , \quad \text{(空闲块的大小} < \text{请求阶对应的大小)}
$$

从公式可以看出:

(1) 当空闲页 (free_pages) 很小，即使没有内存碎片也凑不出一个请求的页阶 (request_blk_pages), 此时 $\frac{\text{空闲页总数}}{\text{请求阶对应的基页数}}$ 接近于 0，那么此时碎片指数也就接近于 0。

(2) 如果空闲页充足，但内存极度碎片化，那么 $free_blocks_total$ 极度大。$\frac{1+\frac{\text{空闲页数}}{\text{请求阶对应的基页数目}}}{\text{各个阶的空闲块总和}}$ 接近于 0，此时碎片指数接近于 1(或 1000)。

$\div free_blocks_total$ 的引入是为了将理论可请求的连续块的能力评价到每个实际的空闲块上。通过空闲块总数将理论可请求连续块的能力分散化，来衡量连续化程度。就像物理系统中能力均匀分布在各个粒子上，导致熵增。也就是碎片化无序程度增加。当系统频繁 alloc/free 内存，大内存被不断切割。最终内存空间中不再有集中的大段连续区域，而是充满大小不一的碎片，空闲的连续大内存分布趋于均匀 (小块增多), 通过 $\div free_blocks_total$ 量化和衡量了这种" 总量充足但连续性不足" 的场景。

free_clocks_total 本质上是对熵的一种简化度量:

空闲块数少 -> 资源集中在少数大块中 -> (有序) -> 熵低 -> 碎片化程度低

空闲块数多 -> 资源集中在大量小块中 -> (无序) -> 熵高 -> 碎片化程度高

6.1.2.2 ___compaction_suitable()

compact_zone() *compaction_suitable()* *___compaction_suitable()*

```
2080 static enum compact_result __compaction_suitable(struct zone *zone, int order,
2081                     unsigned int alloc_flags,
2082                     int highest_zoneidx,
2083                     unsigned long wmark_target)
2084 {
2085     unsigned long watermark;
2086
2087     if (is_via_compact_memory(order))
2088         return COMPACT_CONTINUE;
2089
2090     watermark = wmark_pages(zone, alloc_flags & ALLOC_WMARK_MASK);
2091     /*
2092      * If watermarks for high-order allocation are already met, there
2093      * should be no need for compaction at all.
2094      */
2095     if (zone_watermark_ok(zone, order, watermark, highest_zoneidx,
2096                           alloc_flags))
2097         return COMPACT_SUCCESS;
2098
2099     /*
2100      * Watermarks for order-0 must be met for compaction to be able to
2101      * isolate free pages for migration targets. This means that the
2102      * watermark and alloc_flags have to match, or be more pessimistic than
2103      * the check in __isolate_free_page(). We don't use the direct
2104      * compactor's alloc_flags, as they are not relevant for freepage
2105      * isolation. We however do use the direct compactor's highest_zoneidx
2106      * to skip over zones where lowmem reserves would prevent allocation
2107      * even if compaction succeeds.
2108      * For costly orders, we require low watermark instead of min for
2109      * compaction to proceed to increase its chances.
2110      * ALLOC_CMA is used, as pages in CMA pageblocks are considered
2111      * suitable migration targets
2112      */
2113     watermark = (order > PAGE_ALLOC_COSTLY_ORDER) ?
2114                 low_wmark_pages(zone) : min_wmark_pages(zone);
2115     watermark += compact_gap(order);
2116     if (!__zone_watermark_ok(zone, 0, watermark, highest_zoneidx,
2117                              ALLOC_CMA, wmark_target))
2118         return COMPACT_SKIPPED;
2119
2120     return COMPACT_CONTINUE;
2121 }
```

compaction_suitable() 对 zone 是否适合执行内存碎片整理的操作进行判断, 实际调用的函数是___compaction_suitable()。

该函数有三个返回值:

- *COMPACT_SKIPPED* 表示该 zone 几乎没有空闲页可以用来执行碎片整理。忽略掉该 zone。
- *COMPACT_SUCCESS* 表示该 zone 不需要内存碎片整理即可进行分配。

- *COMPACT_CONTINUE* 表示函数返回后，应该对目标 zone 执行碎片整理的操作。
- 2087-2088 碎片整理指令如果是通过节点*/proc/sys/vm/compact_memory* 下达的，那么可以不受条件限制直接对目标 zone 执行内存整理操作。因此返回 COMPACT_CONTINUE。
- 2090 获取当前 zone 的水位线值。具体是 min、low、high 三条水位线中的哪一条水位线，则根据入参 alloc_flags 的枚举值指示。
- 2095-2097 该行最终调用 ___zone_watermark_ok()。通过目标 zone 的水位线判断是否具有分配 order 阶连续空闲页的能力。若有，则返回 COMPACT_SUCCESS 来表示不需要执行内存整理。
- 2113 对于代价昂贵的大阶内存分配，需满足低水位线 (low) 而非最小水位线 (min) 的条件才能执行内存整理。这是为了提高整理成功几率。大阶分配需要更多连续内存，若仅依赖 min 水位线，此时整理成功率极低。低水位线时空闲内存相对 min 水位线而言尚多，整理后更可能形成连续大页块，满足大阶分配需求。
- 2115 compact_gap() 的实现如下:

  用于在给定水位线之上再保留一定数量的空闲单页，以确保内存整理有足够空间迁移目标页。假设当前请求的阶数为 order, 则理论上内存整理时需要把 $1 << order$ 个页面 (例如 order=3 时需 8 页) 隔离。换言之，就是冻结部分现有资源。

  这是为了确保内存整理后有足够的连续空闲页供分配使用。但是，如果该过程中可能需拆分一个 (order-1) 阶的空闲页 (如 order=3 时拆分 4 页), 就会导致临时的额外内存占用。因此预留 $1 << order$ 个页面不足，更安全的方法是要求两倍该数量。预留理论值的两倍 (例如 order=3 时，冻结 16($2*2^16$) 页), 确保有足够资源完成页面迁移。

___zone_watermark() 会计算在分配所需的页面后，如果空闲页数目仍高于水位线，则返回 true。

如果至少有一个合适大小 (order-n 阶) 的空闲页面能分配，并且分配后，按 order-0 阶 (单页) 统计出的空闲页面数仍高于水位线，那么对大阶目标阶 $order - n$ 的检查返回 true。

在这里 check 是为了防止没有空闲页的情况下，在程序的分配路径中获取 zone lock 再去 check。

compact_zone() > *compaction_suitable()* > *___compaction_suitable()* > *___zone_watermark_ok()*

```
3593 ool __zone_watermark_ok(struct zone *z, unsigned int order, unsigned long mark,
3594             int highest_zoneidx, unsigned int alloc_flags,
3595             long free_pages)
3596 {
3597     long min = mark;
3598     int o;
3599     const bool alloc_harder = (alloc_flags & (ALLOC_HARDER|ALLOC_OOM));
3600 
3601     /* free_pages may go negative - that's OK */
3602     free_pages -= __zone_watermark_unusable_free(z, order, alloc_flags);
3603 
3604     if (alloc_flags & ALLOC_HIGH)
3605         min -= min / 2;
3606 
3607     if (unlikely(alloc_harder)) {
3608         /*
3609          * OOM victims can try even harder than normal ALLOC_HARDER
3610          * users on the grounds that it's definitely going to be in
3611          * the exit path shortly and free memory. Any allocation it
3612          * makes during the free path will be small and short-lived.
3613          */
```

```
3614            if (alloc_flags & ALLOC_OOM)
3615                min -= min / 2;
3616            else
3617                min -= min / 4;
3618        }
3619
3620        /*
3621         * Check watermarks for an order-0 allocation request. If these
3622         * are not met, then a high-order request also cannot go ahead
3623         * even if a suitable page happened to be free.
3624         */
3625        if (free_pages <= min + z->lowmem_reserve[highest_zoneidx])
3626            return false;
3627
3628        /* If this is an order-0 request then the watermark is fine */
3629        if (!order)
3630            return true;
3631
3632        /* For a high-order request, check at least one suitable page is free */
3633        for (o = order; o < MAX_ORDER; o++) {
3634            struct free_area *area = &z->free_area[o];
3635            int mt;
3636
3637            if (!area->nr_free)
3638                continue;
3639
3640            for (mt = 0; mt < MIGRATE_PCPTYPES; mt++) {
3641                if (!free_area_empty(area, mt))
3642                    return true;
3643            }
3644
3645    #ifdef CONFIG_CMA
3646            if ((alloc_flags & ALLOC_CMA) &&
3647                !free_area_empty(area, MIGRATE_CMA)) {
3648                return true;
3649            }
3650    #endif
3651            if (alloc_harder && !free_area_empty(area, MIGRATE_HIGHATOMIC))
3652                return true;
3653        }
3654        return false;
3655    }
```

- 3602 计算获取, 扣除不能被本次分配使用的页后剩余的空闲页数量。
- 3604-3605 如果设置了 ALLOC_HIGH 标志, 表明这是一个高优先级的分配。因此，为了满足水位线要求顺利分配，把水位线减半。
- 3607-3618 如果进入这几行代码块，表示调用者要求更努力的分配。为了能满足水位线要求，把水位线降低。

  对于 ALLOC_OOM 标志，把水位线减去 1/2; 如果设置了 ALLOC_HARDER 标志，把水位线减去 1/4。
- 3625-3626 空闲内存如果 ≤ (水位线 + 低端内存保留页面数)(此概念见前) 则不能从该区域分配。

  这里检查分配单页 (order-0) 是否能成功。如果单页分配都不能满足，即使有一个合适的 order-n 连续页碰巧被释放，大阶 (order-n) 页分配也不能成功。满足水位线要求的前提是：分配后剩余页数目高于水位线。这里的 free_pages 值已经是请求分配单页后的值 (对 free_pages 减 1 的操作在 3602 行的 ___zone_watermark_unusable_free() 函数中已经进行)。

- 3633-3643 对于大阶请求，从请求的阶数依次枚举到最大阶，检查是否有合适的连续页。
- 3646-3649 如果调用者指定了可以从 CMA 区域分配，那么在前面分配请求未能成功的前提下，可以从 CMA 区域分配。

compact_zone() › ··· › ··· › *__zone_watermark_ok()* › *__zone_watermark_unusable_free()*

```
3564 static inline long __zone_watermark_unusable_free(struct zone *z,
3565                 unsigned int order, unsigned int alloc_flags)
3566 {
3567     const bool alloc_harder = (alloc_flags & (ALLOC_HARDER|ALLOC_OOM));
3568     long unusable_free = (1 << order) - 1;
3569 
3570     /*
3571      * If the caller does not have rights to ALLOC_HARDER then subtract
3572      * the high-atomic reserves. This will over-estimate the size of the
3573      * atomic reserve but it avoids a search.
3574      */
3575     if (likely(!alloc_harder))
3576         unusable_free += z->nr_reserved_highatomic;
3577 
3578 #ifdef CONFIG_CMA
3579     /* If allocation can't use CMA areas don't use free CMA pages */
3580     if (!(alloc_flags & ALLOC_CMA))
3581         unusable_free += zone_page_state(z, NR_FREE_CMA_PAGES);
3582 #endif
3583 
3584     return unusable_free;
3585 }
```

- 3575-3576
  - **ALLOC_HARDER** 表示需要采用更激进的策略来满足内存分配请求。
  - **ALLOC_OOM** 这个分配掩码通常用于内存耗尽被系统终止的进程，其获取内存的尝试甚至需要比使用 ALLOC_HARDER 标志的进程更加努力。被迫终止进程退出的过程中，它所进行的任何内存分配都将是少量且短暂的，所以置这个标志来尽量满足它的内存分配请求。原因在于，它很快就会进入退出流程并释放内存。

  如果调用者没有设置 ALLOC_HARDER 则意味着不从保留的 highatomic 内存分配。换言之，这块空闲内存不可使用。当然，这里夸大了 highatomic 内存的大小，因为这块区域可能被其它调用者分配了，这里没有查找计算精确值是为了避免开销。
- 3580-3581 如果调用者没有设明确置 ALLOC_CMA flag, 则也意味着不从 CMA 区域分配。
- 3584 返回预留内存大小。

6.1.3 ___reset_isolation_suitable()

compact_zone() 函数 2229 行，重置内存区域的状态，为下一次内存整理做准备。

函数 ___reset_isolation_suitable() 代码如下:

compact_zone() › *___reset_isolation_suitable()*

```
334 static void __reset_isolation_suitable(struct zone *zone)
335 {
336     unsigned long migrate_pfn = zone->zone_start_pfn;
337     unsigned long free_pfn = zone_end_pfn(zone) - 1;
338     unsigned long reset_migrate = free_pfn;
339     unsigned long reset_free = migrate_pfn;
340     bool source_set = false;
341     bool free_set = false;
342 
343     if (!zone->compact_blockskip_flush)
344         return;
345 
346     zone->compact_blockskip_flush = false;
347 
348     /*
349      * Walk the zone and update pageblock skip information. Source looks
350      * for PageLRU while target looks for PageBuddy. When the scanner
351      * is found, both PageBuddy and PageLRU are checked as the pageblock
352      * is suitable as both source and target.
353      */
354     for (; migrate_pfn < free_pfn; migrate_pfn += pageblock_nr_pages,
355                     free_pfn -= pageblock_nr_pages) {
356         cond_resched();
357 
358         /* Update the migrate PFN */
359         if (__reset_isolation_pfn(zone, migrate_pfn, true, source_set) &&
360             migrate_pfn < reset_migrate) {
361             source_set = true;
362             reset_migrate = migrate_pfn;
363             zone->compact_init_migrate_pfn = reset_migrate;
364             zone->compact_cached_migrate_pfn[0] = reset_migrate;
365             zone->compact_cached_migrate_pfn[1] = reset_migrate;
366         }
367 
368        /* Update the free PFN */
369         if (__reset_isolation_pfn(zone, free_pfn, free_set, true) &&
370             free_pfn > reset_free) {
371             free_set = true;
372             reset_free = free_pfn;
373             zone->compact_init_free_pfn = reset_free;
374             zone->compact_cached_free_pfn = reset_free;
375         }
376     }
377 
378     /* Leave no distance if no suitable block was reset */
379     if (reset_migrate >= reset_free) {
380         zone->compact_cached_migrate_pfn[0] = migrate_pfn;
381         zone->compact_cached_migrate_pfn[1] = migrate_pfn;
382         zone->compact_cached_free_pfn = free_pfn;
383     }
384 }
```

- 346 置 false, 防止后续再次进入清除 PB_migrate_skip 的代码路径 (仅在需要时，由其它路径重新激活该标志)。这里的目的是防止重复触发。至于 PB_migrate_skip 的清零重置在后面的 ___reset_isolation_pfn() 函数中。

- 354 以一个 pageblock 所包含的页为步进间隔，以目标 zone 的起始页帧 migrate_pfn, 和尾页帧 free_pfn 作为起点分别向前和向后扫描。

  其中，会在 ___reset_isolation_pfn 中调用 clear_pageblock_skip 清零重置 PG_migrate_skip 标记。

- 359 ___reset_isolation_pfn() 函数的实现见后面。
- 363 ->compact_init_migrate_pfn 表示内存整理过程中迁移扫描的初始页帧号。在整理过程中，若遇到无法移动的页块 (如被标记为 skip 的 pageblock)，内核会更新此值以跳过无效区域，确保下次整理从新的位置开始。compact_init_migrate_pfn 的更新取决于 ___test_isolation_pfn 函数的返回值。___reset_isolation_pfn 返回 true(表示需要更新迁移扫描的起始位置以跳过当前扫描的 pageblock)，且当前 migrate_pfn 小于已记录的 reset_migrate，则会更新 zone->compact_init_migrate_pfn 到新的 migrate_pfn 位置。并非每次循环都会修改此值。更新仅发生在需要跳过无效 pageblock 的情况下。
- 364-365 分别对应异步和同步整理模式下的缓存迁移扫描位置。
- 379 reset_migrate >= reset_free 表示迁移端起始位置已超过空闲端终止位置，说明扫描过程未找到合适的块。
- 380-382 此时需要强制重置缓存中的 PFN 值，避免后续扫描整理操作使用无效的缓存位置。将缓存的迁移 PFN(compact_cached_migrate_pfn) 和空闲 PFN（compact_cached_free_pfn）重置为循环开始前的初始值 (migrate_pfn 和 free_pfn)。若缓存值未重置，后续可能误认为存在有效块，导致错误操作。

6.1.3.1 ___reset_isolation_pfn()

compact_zone() → *___reset_isolation_suitable()* → *___reset_isolation_pfn()*

```
257 static bool
258 __reset_isolation_pfn(struct zone *zone, unsigned long pfn, bool check_source,
259                           bool check_target)
260 {
261     struct page *page = pfn_to_online_page(pfn);
262     struct page *block_page;
263     struct page *end_page;
264     unsigned long block_pfn;
265
266     if (!page)
267         return false;
268     if (zone != page_zone(page))
269         return false;
270     if (pageblock_skip_persistent(page))
271         return false;
272
273     /*
274      * If skip is already cleared do no further checking once the
275      * restart points have been set.
276      */
277     if (check_source && check_target && !get_pageblock_skip(page))
278         return true;
279
280     /*
281      * If clearing skip for the target scanner, do not select a
282      * non-movable pageblock as the starting point.
283      */
284     if (!check_source && check_target &&
285         get_pageblock_migratetype(page) != MIGRATE_MOVABLE)
286         return false;
287
```

```
288     /* Ensure the start of the pageblock or zone is online and valid */
289     block_pfn = pageblock_start_pfn(pfn);
290     block_pfn = max(block_pfn, zone->zone_start_pfn);
291     block_page = pfn_to_online_page(block_pfn);
292     if (block_page) {
293         page = block_page;
294         pfn = block_pfn;
295     }
296 
297     /* Ensure the end of the pageblock or zone is online and valid */
298     block_pfn = pageblock_end_pfn(pfn) - 1;
299     block_pfn = min(block_pfn, zone_end_pfn(zone) - 1);
300     end_page = pfn_to_online_page(block_pfn);
301     if (!end_page)
302         return false;
303 
304     /*
305      * Only clear the hint if a sample indicates there is either a
306      * free page or an LRU page in the block. One or other condition
307      * is necessary for the block to be a migration source/target.
308      */
309     do {
310         if (pfn_valid_within(pfn)) {
311             if (check_source && PageLRU(page)) {
312                 clear_pageblock_skip(page);
313                 return true;
314             }
315 
316             if (check_target && PageBuddy(page)) {
317                 clear_pageblock_skip(page);
318                 return true;
319             }
320         }
321 
322         page += (1 << PAGE_ALLOC_COSTLY_ORDER);
323         pfn += (1 << PAGE_ALLOC_COSTLY_ORDER);
324     } while (page <= end_page);
325 
326     return false;
327 }
```

- 270 order 大于等于 pageblock_order 的复合页应始终被跳过，直到其被释放。譬如，若 pageblock_order 设置为 9(对应 2MB 的页块大小)，而需要分配一个 3MB 的复合页，由于内存对齐规则，3MB 的复合页无法完全容纳在单个 pageblock 中，必须跨越两个相邻的 pageblock。

  第一个 pageblock: 完全被复合页占据 (2MB).

  第二个 pageblock：仅占用前 1MB. 但是未使用的 1MB 无法分配给其它请求，导致内部碎片。

- 277 如果 PB_migrate_skip 标记已清除，意味着完成设置” 重启内存整理”，无需进行进一步检查直接返回。

- 284 若当前页不适合作为迁移源或目标 (例如无有效页面)，快速跳过一个高阶块 (PAGE_ALLOC_COSTLY_ORDER，默认为 3) 的步长，减少冗余扫描，避免逐页检查开销。这种避免处理碎片化小内存块开销的方法是性能与利用率之间的一种权衡。

6.2 compact_zone()(续)

继续回到 compact_zone() 函数:

compact_zone()

```
2231     /*
2232      * Setup to move all movable pages to the end of the zone. Used cached
2233      * information on where the scanners should start (unless we explicitly
2234      * want to compact the whole zone), but check that it is initialised
2235      * by ensuring the values are within zone boundaries.
2236      */
2237     cc->fast_start_pfn = 0;
2238     if (cc->whole_zone) {
2239         cc->migrate_pfn = start_pfn;
2240         cc->free_pfn = pageblock_start_pfn(end_pfn - 1);
2241     } else {
2242         cc->migrate_pfn = cc->zone->compact_cached_migrate_pfn[sync];
2243         cc->free_pfn = cc->zone->compact_cached_free_pfn;
2244         if (cc->free_pfn < start_pfn || cc->free_pfn >= end_pfn) {
2245             cc->free_pfn = pageblock_start_pfn(end_pfn - 1);
2246             cc->zone->compact_cached_free_pfn = cc->free_pfn;
2247         }
2248         if (cc->migrate_pfn < start_pfn || cc->migrate_pfn >= end_pfn) {
2249             cc->migrate_pfn = start_pfn;
2250             cc->zone->compact_cached_migrate_pfn[0] = cc->migrate_pfn;
2251             cc->zone->compact_cached_migrate_pfn[1] = cc->migrate_pfn;
2252         }
2253 
2254         if (cc->migrate_pfn <= cc->zone->compact_init_migrate_pfn)
2255             cc->whole_zone = true;
2256     }
2257 
2258     last_migrated_pfn = 0;
2259 
2260    /*
2261      * Migrate has separate cached PFNs for ASYNC and SYNC* migration on
2262      * the basis that some migrations will fail in ASYNC mode. However,
2263      * if the cached PFNs match and pageblocks are skipped due to having
2264      * no isolation candidates, then the sync state does not matter.
2265      * Until a pageblock with isolation candidates is found, keep the
2266      * cached PFNs in sync to avoid revisiting the same blocks.
2267      */
2268     update_cached = !sync &&
2269         cc->zone->compact_cached_migrate_pfn[0] == cc->zone->compact_cached_migrate_pfn
             [1];
2270 
2271     trace_mm_compaction_begin(start_pfn, cc->migrate_pfn,
2272                 cc->free_pfn, end_pfn, sync);
2273 
2274     migrate_prep_local();
2275 
2276     while ((ret = compact_finished(cc)) == COMPACT_CONTINUE) {
2277         int err;
2278         unsigned long start_pfn = cc->migrate_pfn;
2279 
2280         /*
2281          * Avoid multiple rescans which can happen if a page cannot be
2282          * isolated (dirty/writeback in async mode) or if the migrated
2283          * pages are being allocated before the pageblock is cleared.
2284          * The first rescan will capture the entire pageblock for
2285          * migration. If it fails, it'll be marked skip and scanning
2286          * will proceed as normal.
2287          */
2288         cc->rescan = false;
2289         if (pageblock_start_pfn(last_migrated_pfn) ==
```

```
2290            pageblock_start_pfn(start_pfn)) {
2291            cc->rescan = true;
2292        }
2293
2294        switch (isolate_migratepages(cc)) {
2295        case ISOLATE_ABORT:
2296            ret = COMPACT_CONTENDED;
2297            putback_movable_pages(&cc->migratepages);
2298            cc->nr_migratepages = 0;
2299            goto out;
2300        case ISOLATE_NONE:
2301            if (update_cached) {
2302                cc->zone->compact_cached_migrate_pfn[1] =
2303                    cc->zone->compact_cached_migrate_pfn[0];
2304            }
2305
2306            /*
2307             * We haven't isolated and migrated anything, but
2308             * there might still be unflushed migrations from
2309             * previous cc->order aligned block.
2310             */
2311            goto check_drain;
2312        case ISOLATE_SUCCESS:
2313            update_cached = false;
2314            last_migrated_pfn = start_pfn;
2315            ;
2316        }
2317
2318        err = migrate_pages(&cc->migratepages, compaction_alloc,
2319                compaction_free, (unsigned long)cc, cc->mode,
2320                MR_COMPACTION);
2321
2322        trace_mm_compaction_migratepages(cc->nr_migratepages, err,
2323                                &cc->migratepages);
2324
2325        /* All pages were either migrated or will be released */
2326        cc->nr_migratepages = 0;
2327        if (err) {
2328            putback_movable_pages(&cc->migratepages);
2329            /*
2330             * migrate_pages() may return -ENOMEM when scanners meet
2331             * and we want compact_finished() to detect it
2332             */
2333            if (err == -ENOMEM && !compact_scanners_met(cc)) {
2334                ret = COMPACT_CONTENDED;
2335                goto out;
2336            }
2337            /*
2338             * We failed to migrate at least one page in the current
2339             * order-aligned block, so skip the rest of it.
2340             */
2341            if (cc->direct_compaction &&
2342                        (cc->mode == MIGRATE_ASYNC)) {
2343                cc->migrate_pfn = block_end_pfn(
2344                        cc->migrate_pfn - 1, cc->order);
2345                /* Draining pcplists is useless in this case */
2346                last_migrated_pfn = 0;
2347            }
2348        }
2349
2350 check_drain:
2351        /*
2352         * Has the migration scanner moved away from the previous
2353         * cc->order aligned block where we migrated from? If yes,
```

```
2354            * flush the pages that were freed, so that they can merge and
2355            * compact_finished() can detect immediately if allocation
2356            * would succeed.
2357            */
2358           if (cc->order > 0 && last_migrated_pfn) {
2359               unsigned long current_block_start =
2360                   block_start_pfn(cc->migrate_pfn, cc->order);
2361
2362               if (last_migrated_pfn < current_block_start) {
2363                   lru_add_drain_cpu_zone(cc->zone);
2364                   /* No more flushing until we migrate again */
2365                   last_migrated_pfn = 0;
2366               }
2367           }
2368
2369           /* Stop if a page has been captured */
2370           if (capc && capc->page) {
2371               ret = COMPACT_SUCCESS;
2372               break;
2373           }
2374       }
2375
2376   out:
2377       /*
2378        * Release free pages and update where the free scanner should restart,
2379        * so we don't leave any returned pages behind in the next attempt.
2380        */
2381       if (cc->nr_freepages > 0) {
2382           unsigned long free_pfn = release_freepages(&cc->freepages);
2383
2384           cc->nr_freepages = 0;
2385           VM_BUG_ON(free_pfn == 0);
2386           /* The cached pfn is always the first in a pageblock */
2387           free_pfn = pageblock_start_pfn(free_pfn);
2388           /*
2389            * Only go back, not forward. The cached pfn might have been
2390            * already reset to zone end in compact_finished()
2391            */
2392           if (free_pfn > cc->zone->compact_cached_free_pfn)
2393               cc->zone->compact_cached_free_pfn = free_pfn;
2394       }
2395
2396       count_compact_events(COMPACTMIGRATE_SCANNED, cc->total_migrate_scanned);
2397       count_compact_events(COMPACTFREE_SCANNED, cc->total_free_scanned);
2398
2399       trace_mm_compaction_end(start_pfn, cc->migrate_pfn,
2400                   cc->free_pfn, end_pfn, sync, ret);
2401
2402       return ret;
2403   }
```

- 2237 关闭快速路径，走标准完整扫描。
- 2238-2256 准备整理将可移动页搬移到目标 zone 的尾部。整理过程中会使用缓存信息确定迁移扫描器和空闲扫描器的起始位置 (除非明确要求对整个 zone 进行整理)，需要校验这些缓存值确保 PFN 落在 zone 的地址范围之内。

  cc->migrate_pfn 和 *cc->free_pfn* 使用的就是这些缓存信息。

  - 2238-2240 这里需要整理整个 zone, 不使用断点缓存，从头扫到尾。

    end_pfn-1 是 zone 内的最末尾的有效 PFN。*pageblock_start_pfn(end_pfn-1)* 是把该最末尾的有效 PFN 对齐到它所属的 *pageblock*, 得到 zone 最后一个 *pageblock* 的起始 PFN。

- 2242 获取缓存的 zone 保存的上一次整理停下来时的 migrate_pfn。

  compact_cached_migrate_pfn[2] 数组有两个成员:0 对应异步模式缓存的 migrate_pfn。1 对应同步模式缓存的 migrate_pfn。

- 2243 获取空闲扫描器缓存的 pfn。这个不区分同步和异步，只有一份。
- 2244-2347 如果缓存的 free_pfn 不在 zone 的有效 pfn 区间 $[start_pfn, end_pfn)$, 说明缓存失效。此时将 free_pfn 重置为最末一个 pageblock 的其实 pfn, 同时更新到缓存变量里。
- 2248-2252 检测迁移扫描器缓存的 pfn 是否 $\in [start_pfn, end_pfn)$, 如果不是则重置为 zone 的起始 pfn. 同时将两种模式的缓存也更新，防止另一种模式也拿到非法值。
- 2254-2255

  zone->compact_init_migrate_pfn 是上一次做完整全 zone 整理时的迁移扫描器起始 PFN。只有当迁移扫描器和空闲扫描器真正跑完一轮并相遇，内核才会把该轮迁移扫描器的起始 PFN 保存到这个缓存变量。

  需要注意的是-*>compact_init_migrate_pfn* 和 zone 最开头的 start_pfn 不一定是同一个物理页帧。因为有可能 zone 的第一块 pageblock 不可迁移，那么迁移扫描器会直接跳到下一个 pageblock 开始。

  compact_cached_migrate_pfn 保存断点，下一次整理从这里继续往后扫 (pfn 增大方向)，避免从头扫整个 zone。这一次如果是完整全 zone 扫描，那么已经完整检查过 $[zone_start_pfn, compact_init_migrate_pfn\text{-}1]$ 这一片区域。如果迁移扫描起点，跑到了 *compact_init_migrate_pfn* 左边, 如果还用增量逻辑，内核会错误地跳过一大片区域，漏掉很多 pageblock。所以强制设置全 zone 扫描模式。

- 2258 清零上一次迁移 pfn。
- 2268-2269 在还没有碰到有可迁移候选页的 pageblock 之前，同步、异步缓存要保持一致，避免重复扫描相同 pageblock。

  只要 pageblock 里面存在可迁移候选页，同步模式就会尽力隔离、阻塞等待完成迁移，不会随便跳过。同步路径很少出现 “连续一大段 pageblock 全部被无条件跳过”。所以 “避免重复扫描空块” 这个问题，在同步路径上几乎不存在，没有动机去改写异步 *cache[0]*。

- 2276-2348 这段代码块负责持续迁移页面直到整理完成。
  - 2276 判断整理是否可以结束。
  - 2278 获取本轮迭代 migrate 扫描器的起始迁移页帧号 pfn。
  - 2288-2292 上一轮迁移和本轮迁移在同一个 pageblock 时，置 rescan=true。
  - 2294 migrate 扫描器向前推进，扫描 pageblock，把可移动页面隔离出来，挂入 *cc->migratepages* 链表。返回值的枚举值如下:

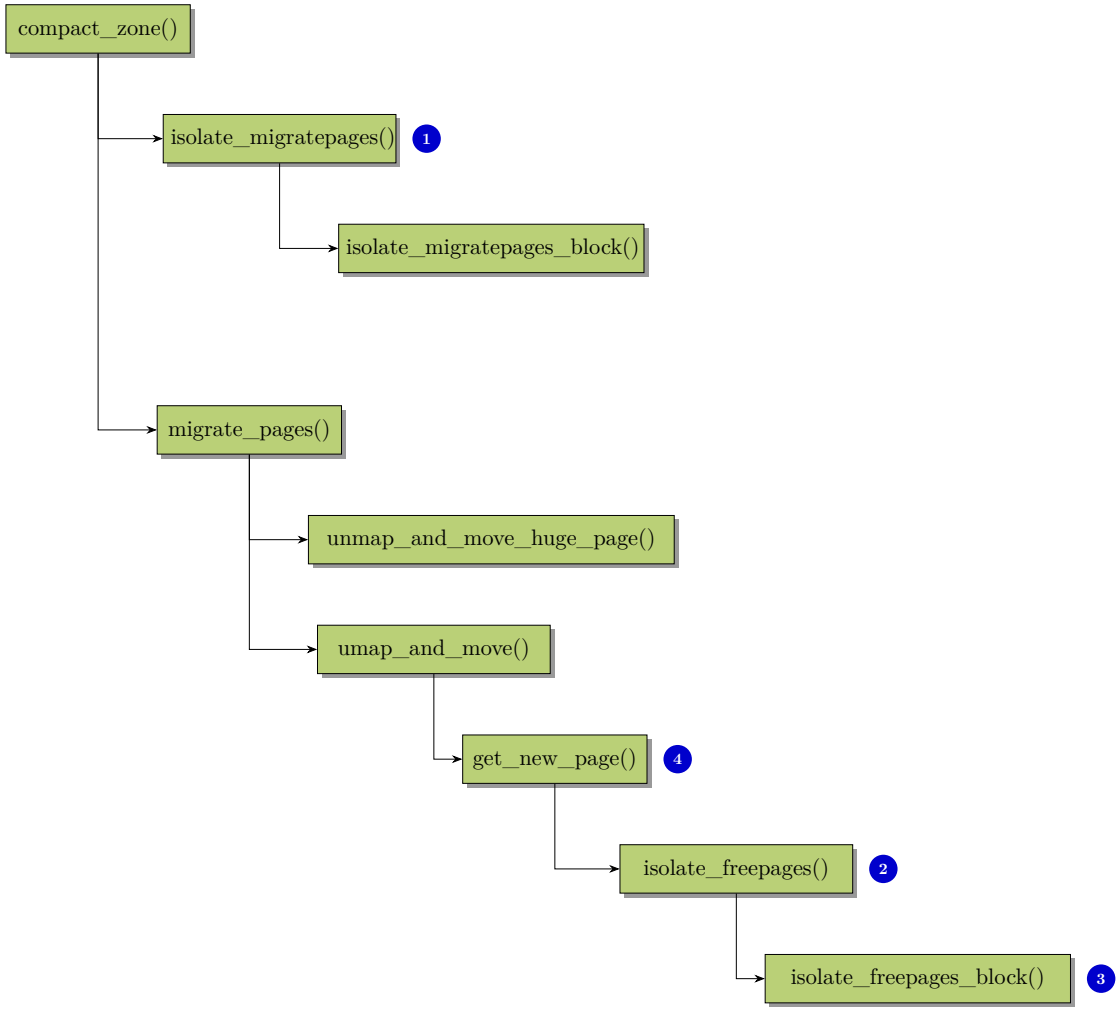
    ISOLATE_ABORT 隔离被中断 (锁竞争、抢占)。把已经隔离出来的页面放回 lru，退出整理。

    ISOLATE_NONE 本 pageblock 没有隔离到任何可迁移页面。如果 $update_cached = true$，把 async 的缓存同步赋值给 sync 缓存，保持两套缓存一致。跳转做 pcplist 冲刷。

    ISOLATE_SUCCESS 成功隔离到一批页面。关闭 *update_cached*，不再强制同步两套缓存, 记录本轮起始 pfn 为 $last_migrated_pfn$。

- 2318-2320 核心迁移整理函数。

  compact_zone() 中进行扫描迁移整理的几个核心函数的流程图如下:



  说明:

  1. isolate_migratepages() 是迁移扫描器入口。
  2. isolate_freepages() 是空闲页扫描器入口, 在 zone 中扫描 pageblock。
  3. isolate_freepages_block() 扫描 pageblock 内部的块。
  4. get_new_page 实际是函数指针，实际函数是 compaction_alloc()。

  - 2322-2323 ftrace 埋点，记录迁移统计。
  - 2327-2328 把链表上没有迁移成功的页面放回 lru 链表。
  - 2333-2336 分配迁移目标页失败。如果迁移扫描器和空闲扫描器还没有相遇，说明不是扫描结束，是资源竞争，标记 $COMPACT_CONTENDED$ 退出。
  - 2341-2347 直接路径（用户态分配触发同步压缩，不是 kswapd），async 模式下迁移失败。直接把 migrate 扫描器跳到当前 order 对齐块的末尾，跳过这个块不再尝试，减少反复失败；清零 $last_migrated_pfn$，不需要 drain pcplist。

- 2358-2366 当 migrate 扫描器跨出上一次迁移所在的 order 对齐块时，把 pcplist 里的页真正释放回 buddy，让页合并生效。目的：让 $compact_finished()$ 能够立刻看到合并后的高阶空闲页，避免 pcplist 缓存掩盖真实 buddy 状态，造成误判。

- 2381-2382 释放本次压缩收集的空闲页链表 freepages，归还到 buddy。

- 2387 free 扫描器缓存必须对齐 pageblock。
- 2396-2397 更新 vmstat 整理事件计数，*/proc/vmstat* 可以看到这两个统计项。

6.2.1 isolate_migratepages()

isolate_migratepages() 以 *pageblock* 为单位遍历 zone，筛选合格 pageblock，然后调用 *isolate_migratepages_block()* 进入块内部逐页隔离页面。

compact_zone() → *isolate_migratepages()*

```
1773 static isolate_migrate_t isolate_migratepages(struct compact_control *cc)
1774 {
1775     unsigned long block_start_pfn;
1776     unsigned long block_end_pfn;
1777     unsigned long low_pfn;
1778     struct page *page;
1779     const isolate_mode_t isolate_mode =
1780         (sysctl_compact_unevictable_allowed ? ISOLATE_UNEVICTABLE : 0) |
1781         (cc->mode != MIGRATE_SYNC ? ISOLATE_ASYNC_MIGRATE : 0);
1782     bool fast_find_block;
1783
1784     /*
1785      * Start at where we last stopped, or beginning of the zone as
1786      * initialized by compact_zone(). The first failure will use
1787      * the lowest PFN as the starting point for linear scanning.
1788      */
1789     low_pfn = fast_find_migrateblock(cc);
1790     block_start_pfn = pageblock_start_pfn(low_pfn);
1791     if (block_start_pfn < cc->zone->zone_start_pfn)
1792         block_start_pfn = cc->zone->zone_start_pfn;
1793
1794     /*
1795      * fast_find_migrateblock marks a pageblock skipped so to avoid
1796      * the isolation_suitable check below, check whether the fast
1797      * search was successful.
1798      */
1799     fast_find_block = low_pfn != cc->migrate_pfn && !cc->fast_search_fail;
1800
1801     /* Only scan within a pageblock boundary */
1802     block_end_pfn = pageblock_end_pfn(low_pfn);
1803
1804     /*
1805      * Iterate over whole pageblocks until we find the first suitable.
1806      * Do not cross the free scanner.
1807      */
1808     for (; block_end_pfn <= cc->free_pfn;
1809             fast_find_block = false,
1810             low_pfn = block_end_pfn,
1811             block_start_pfn = block_end_pfn,
1812             block_end_pfn += pageblock_nr_pages) {
1813
1814         /*
1815          * This can potentially iterate a massively long zone with
1816          * many pageblocks unsuitable, so periodically check if we
1817          * need to schedule.
1818          */
1819         if (!(low_pfn % (SWAP_CLUSTER_MAX * pageblock_nr_pages)))
1820             cond_resched();
1821
1822         page = pageblock_pfn_to_page(block_start_pfn,
```

```
1823                        block_end_pfn, cc->zone);
1824        if (!page)
1825            continue;
1826
1827        /*
1828         * If isolation recently failed, do not retry. Only check the
1829         * pageblock once. COMPACT_CLUSTER_MAX causes a pageblock
1830         * to be visited multiple times. Assume skip was checked
1831         * before making it "skip" so other compaction instances do
1832         * not scan the same block.
1833         */
1834        if (IS_ALIGNED(low_pfn, pageblock_nr_pages) &&
1835            !fast_find_block && !isolation_suitable(cc, page))
1836            continue;isolate_migratepages_block
1837
1838        /*
1839         * For async compaction, also only scan in MOVABLE blocks
1840         * without huge pages. Async compaction is optimistic to see
1841         * if the minimum amount of work satisfies the allocation.
1842         * The cached PFN is updated as it's possible that all
1843         * remaining blocks between source and target are unsuitable
1844         * and the compaction scanners fail to meet.
1845         */
1846        if (!suitable_migration_source(cc, page)) {
1847            update_cached_migrate(cc, block_end_pfn);
1848            continue;
1849        }
1850
1851        /* Perform the isolation */
1852        low_pfn = isolate_migratepages_block(cc, low_pfn,
1853                        block_end_pfn, isolate_mode);
1854
1855       if (!low_pfn)
1856            return ISOLATE_ABORT;
1857
1858        /*
1859         * Either we isolated something and proceed with migration. Or
1860         * we failed and compact_zone should decide if we should
1861         * continue or not.
1862         */
1863        break;
1864    }
1865
1866    /* Record where migration scanner will be restarted. */
1867    cc->migrate_pfn = low_pfn;
1868
1869    return cc->nr_migratepages ? ISOLATE_SUCCESS : ISOLATE_NONE;
1870 }
```

- 1779-1782 获取页面隔离模式。

  1780: 内核全局变量 *sysctl_compact_unevictable_allowed* 控制是否允许隔离不可回收 (unevictable) 页面, 也就是是否允许不可回收页迁移。

  该值可以通过写 proc 节点*/proc/sys/vm/compact_unevictable_allowed*，或执行命令 *sysctl vm.compact_unevictable_allowed* 来控制。

  1781: 如果上下文允许异步整理，置 *ISOLATE_ASYNC_MIGRATE*。

  这里的异步不是指后台执行，而是特指是否允许阻塞。

- 1789 调用快速查找函数尝试利用缓存 hint，快速跳过一批标记为 *PB_migrate_skip* 的 pageblock，直接跳到一个候选 pageblock；失败的话就返回 *cc->migrate_pfn*，退化成普通线性扫描。

- 1790-1792 把 migrate 扫描器起始 PFN 向下对齐到 pageblock 起始 pfn, 不能小于 zone 的最小 pfn。
- 1799 判断 fast 查找是否真正成功。
- 1802 算出当前 pageblock 的结束 pfn(开区间)。
- 1808-1812 以 pageblock 为粒度将迁移扫描器往前推进。结束条件是:$migrate_pfn >= free_pfn$。需要注意的是: 这个循环不会遍历所有 pageblock 然后全部隔离完，而是找到第一个合格 pageblock，调用 *isolate_migratepages_block()* 返回之后立刻 break，跳出 for 循环返回上一层调用者。
- 1819-1820 zone 可能非常大，大量 pageblock 都不可用。所以每隔若干 pageblock 主动让出 CPU，避免内核线程长时间霸占 CPU。
- 1822-1825 根据 pageblock 的 pfn 范围获取该 pageblock 第一个 pfn 对应的 *struct page*; 如果该 pfn 范围无效，直接 continue，跳到下一个 pageblock。

  pageblock 的 migratetype、PB_migrate_skip hint 全部存在 pageblock 第一个 page 的 pageblock_flags 位图; 块内其他 page 不存这份信息。所以只要拿到 pageblock 首页 struct page 就可以做全部块级判断，不需要遍历块内所有 page。
- 1834-1836 检查 pageblock 的 *PB_migrate_skip hint* 标记, 判断是否需要跳过本 pageblock。

  IS_ALIGNED() 在这里等价于判断 low_pfn 是不是 pageblock 的起始 PFN。只有 pageblock 首页的 struct page 里的 pageblock_flags 位图才有 PB_migrate_skip hint 信息。

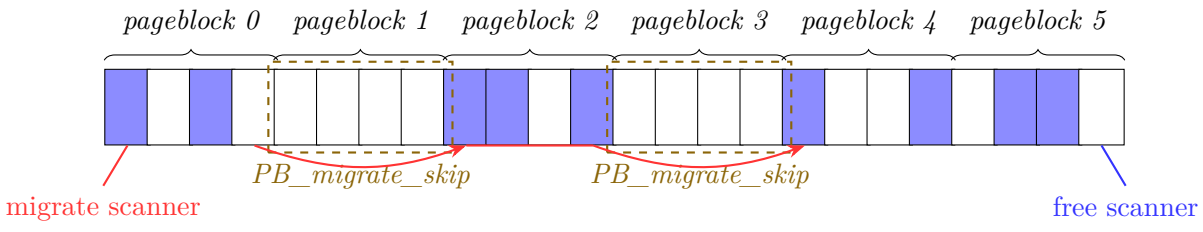
  不是 fast 查找路径 (fast 已经校验过 hint)。
- 1846-1848 读取 pageblock 的迁移类型。

  pageblock 为 *MIGRATE_UNMOVABLE/MIGRATE_ISOLATE* 时，返回 false。

  异步模式下，*MIGRATE_RECLAIMABLE* 也返回 false，异步整理不处理可回收块。
- 1852-1853 调用块内扫描函数。进入目标 pageblock 内部，逐页循环，隔离可迁移页面到 $cc->migratepages$ 链表。返回扫描结束后的 pfn(本 pageblock 内扫描到哪里)。
- 1863 只要成功调用完一次 1852 行的 *isolate_migratepages_block()*，就直接跳出外层 for 循环。返回上层调用者 *compact_zone()*，上层去执行 *migrate_pages()* 做页面迁移, 迁移完之后会再次调用 *isolate_migratepages()*，继续下一个 pageblock。不是一次性把 zone 所有 pageblock 全部扫完！扫描一块、迁移一块，再回来扫描下一块。

  如下图，pageblock1、pageblock3 在扫描之后被跳过 (图中每个小框代表一个页帧)。

规则：遇到 PB_migrate_skip 整个 pageblock 直接跳过，不扫描内部 page



migrate 扫描器行为：

- 从低 PFN 向高 PFN（向右）遍历 pageblock；
- pageblock 开头检查 `PB_migrate_skip` hint；标记为 skip 的块**整块跳过，不进入块内逐页扫描**；
- 只有非 skip 的 pageblock，才进入 `isolate_migratepages_block` 做内部页面隔离；
- migrate 扫描器不允许越过 free 扫描器，两者相遇则本轮整理结束。

  空闲页扫描器的行为与之类似，只是从高 PFN 往低 PFN 扫描。

- 1867 保存扫描游标，记录本次 migrate 扫描器停止位置，下一次调用 *isolate_migratepages()* 就从这个 pfn 继续往后扫描。
- 1869 如果成功隔离到至少 1 个可迁移页，那么上层调用者接下来执行页迁移。

6.2.1.1 suitable_migration_source()

suitable_migration_source() 判断目标 page，是否适合迁移。

compact_zone() → *isolate_migratepages()* → *suitable_migration_source()*

```
1149 static bool suitable_migration_source(struct compact_control *cc,
1150                                        struct page *page)
1151 {
1152     int block_mt;
1153
1154     if (pageblock_skip_persistent(page))
1155         return false;
1156
1157     if ((cc->mode != MIGRATE_ASYNC) || !cc->direct_compaction)
1158         return true;
1159
1160     block_mt = get_pageblock_migratetype(page);
1161
1162     if (cc->migratetype == MIGRATE_MOVABLE)
1163         return is_migrate_movable(block_mt);
1164     else
1165         return block_mt == cc->migratetype;
1166 }
```

- 1154-1155 运行时检查目标 page 是否属于一个尺寸 ≥ 完整 pageblock 的复合巨页；只要这个巨页未释放，该 pageblock 内所有页面都持续跳过，不要做迁移源。巨页释放后自动失效。名字里 *persistent* 是指“只要巨页存在就持续跳过”。
- 1157-1158 只要满足下面任意一条，就不做迁移类型严格校验，全部允许作为迁移源。
  - 当前是同步直接整理模式 (direct compaction 阻塞等待)。
  - 不是直接整理，是后台异步整理。
- 1160 获取目标 page 所属 pageblock 的迁移类型。

  一个 pageblock 内所有页面共用同一个迁移类型，实际实现是内部会向下对齐到 pageblock 首 page 再读取它的 flags。

- 1162-1163 异步直接整理才会跑到此处。

- 这里的异步模式是:*MIGRATE_ASYNC*。仅表示整理迁移的时候不阻塞当前进程，并不是指后台执行方式。遇到锁、遇到等待就直接放弃迁移，不阻塞、不等待。

  本次分配需要的类型如果是 *MIGRATE_MOVABLE*(可移动类型)。调用 *is_migrate_movable()* 判断该 pageblock 类型是否允许迁移:

  - *MIGRATE_MOVABLE*、*MIGRATE_RECLAIMABLE* 返回 true，可以迁移;
  - *MIGRATE_UNMOVABLE*、*MIGRATE_UNMOVABLERECLAIM* 返回 false，不可迁移。

  异步直接整理时要找 *MIGRATE_MOVABLE* 的大块，只允许从可迁移的 pageblock 拿页。

- 1164-1165 源 pageblock 的迁移类型必须和本次 cc->migratetype 完全相等，才允许拿来迁移。

  在异步 (不阻塞) 直接整理场景下，若本次分配目标迁移类型为 *MIGRATE_MOVABLE*，if 分支采用宽松规则，允许 *MIGRATE_MOVABLE* 和 *MIGRATE_RECLAIMABLE* 两类 pageblock 都作为迁移源。else 分支针对其余迁移类型，执行严格校验，仅允许源 pageblock 类型与目标类型完全一致才可用作迁移源，以此避免异步 (不阻塞) 直接整理下跨类型迁移加剧内存碎片。

6.2.1.2 isolate_migratepages_block()

该函数在一个 PFN 区间 $[low_pfn, end_pfn)$ 内扫描，隔离可迁移页面，把成功隔离的 page 挂入 *cc->migratepages* 链表。

compact_zone() > *isolate_migratepages()* > *isolate_migratepages_block()*

```
800 static unsigned long
801 isolate_migratepages_block(struct compact_control *cc, unsigned long low_pfn,
802             unsigned long end_pfn, isolate_mode_t isolate_mode)
803 {
804     pg_data_t *pgdat = cc->zone->zone_pgdat;
805     unsigned long nr_scanned = 0, nr_isolated = 0;
806     struct lruvec *lruvec;
807     unsigned long flags = 0;
808     bool locked = false;
809     struct page *page = NULL, *valid_page = NULL;
810     unsigned long start_pfn = low_pfn;
811     bool skip_on_failure = false;
812     unsigned long next_skip_pfn = 0;
813     bool skip_updated = false;
814
815     /*
816      * Ensure that there are not too many pages isolated from the LRU
817      * list by either parallel reclaimers or compaction. If there are,
818      * delay for some time until fewer pages are isolated
819      */
820     while (unlikely(too_many_isolated(pgdat))) {
821         /* stop isolation if there are still pages not migrated */
822         if (cc->nr_migratepages)
823             return 0;
824
825         /* async migration should just abort */
826         if (cc->mode == MIGRATE_ASYNC)
827             return 0;
828
829         congestion_wait(BLK_RW_ASYNC, HZ/10);
830
831         if (fatal_signal_pending(current))
```

```
832             return 0;
833     }
834
835     cond_resched();
836
837     if (cc->direct_compaction && (cc->mode == MIGRATE_ASYNC)) {
838         skip_on_failure = true;
839         next_skip_pfn = block_end_pfn(low_pfn, cc->order);
840     }
841
842    /* Time to isolate some pages for migration */
843     for (; low_pfn < end_pfn; low_pfn++) {
844
845         if (skip_on_failure && low_pfn >= next_skip_pfn) {
846             /*
847              * We have isolated all migration candidates in the
848              * previous order-aligned block, and did not skip it due
849              * to failure. We should migrate the pages now and
850              * hopefully succeed compaction.
851              */
852             if (nr_isolated)
853                 break;
854
855             /*
856              * We failed to isolate in the previous order-aligned
857              * block. Set the new boundary to the end of the
858              * current block. Note we can't simply increase
859              * next_skip_pfn by 1 << order, as low_pfn might have
860              * been incremented by a higher number due to skipping
861              * a compound or a high-order buddy page in the
862              * previous loop iteration.
863              */
864             next_skip_pfn = block_end_pfn(low_pfn, cc->order);
865         }
866
867         /*
868          * Periodically drop the lock (if held) regardless of its
869          * contention, to give chance to IRQs. Abort completely if
870          * a fatal signal is pending.
871          */
872         if (!(low_pfn % SWAP_CLUSTER_MAX)
873             && compact_unlock_should_abort(&pgdat->lru_lock,
874                         flags, &locked, cc)) {
875             low_pfn = 0;
876             goto fatal_pending;
877         }
878
879         if (!pfn_valid_within(low_pfn))
880             goto isolate_fail;
881         nr_scanned++;
882
883         page = pfn_to_page(low_pfn);
884
885         /*
886          * Check if the pageblock has already been marked skipped.
887          * Only the aligned PFN is checked as the caller isolates
888          * COMPACT_CLUSTER_MAX at a time so the second call must
889          * not falsely conclude that the block should be skipped.
890          */
891         if (!valid_page && IS_ALIGNED(low_pfn, pageblock_nr_pages)) {
892             if (!cc->ignore_skip_hint && get_pageblock_skip(page)) {
893                 low_pfn = end_pfn;
894                 goto isolate_abort;
895             }
```

```
896             valid_page = page;
897         }
898 
899         /*
900          * Skip if free. We read page order here without zone lock
901          * which is generally unsafe, but the race window is small and
902          * the worst thing that can happen is that we skip some
903          * potential isolation targets.
904          */
905         if (PageBuddy(page)) {
906             unsigned long freepage_order = buddy_order_unsafe(page);
907 
908             /*
909              * Without lock, we cannot be sure that what we got is
910              * a valid page order. Consider only values in the
911              * valid order range to prevent low_pfn overflow.
912              */
913             if (freepage_order > 0 && freepage_order < MAX_ORDER)
914                 low_pfn += (1UL << freepage_order) - 1;
915             continue;
916         }
917 
918         /*
919          * Regardless of being on LRU, compound pages such as THP and
920          * hugetlbfs are not to be compacted unless we are attempting
921          * an allocation much larger than the huge page size (eg CMA).
922          * We can potentially save a lot of iterations if we skip them
923          * at once. The check is racy, but we can consider only valid
924          * values and the only danger is skipping too much.
925          */
926         if (PageCompound(page) && !cc->alloc_contig) {
927             const unsigned int order = compound_order(page);
928 
929            if (likely(order < MAX_ORDER))
930                 low_pfn += (1UL << order) - 1;
931             goto isolate_fail;
932         }
933 
934         /*
935          * Check may be lockless but that's ok as we recheck later.
936          * It's possible to migrate LRU and non-lru movable pages.
937          * Skip any other type of page
938          */
939         if (!PageLRU(page)) {
940             /*
941              * __PageMovable can return false positive so we need
942              * to verify it under page_lock.
943              */
944             if (unlikely(__PageMovable(page)) &&
945                     !PageIsolated(page)) {
946                 if (locked) {
947                     spin_unlock_irqrestore(&pgdat->lru_lock,
948                                            flags);
949                     locked = false;
950                 }
951 
952                 if (!isolate_movable_page(page, isolate_mode))
953                     goto isolate_success;
954             }
955 
956             goto isolate_fail;
957         }
958 
959         /*
```

```
960             * Migration will fail if an anonymous page is pinned in memory,
961             * so avoid taking lru_lock and isolating it unnecessarily in an
962             * admittedly racy check.
963             */
964            if (!page_mapping(page) &&
965                page_count(page) > page_mapcount(page))
966                goto isolate_fail;
967
968            /*
969             * Only allow to migrate anonymous pages in GFP_NOFS context
970             * because those do not depend on fs locks.
971             */
972            if (!(cc->gfp_mask & __GFP_FS) && page_mapping(page))
973                goto isolate_fail;
974
975            /* If we already hold the lock, we can skip some rechecking */
976            if (!locked) {
977                locked = compact_lock_irqsave(&pgdat->lru_lock,
978                                              &flags, cc);
979
980                /* Try get exclusive access under lock */
981                if (!skip_updated) {
982                    skip_updated = true;
983                    if (test_and_set_skip(cc, page, low_pfn))
984                        goto isolate_abort;
985                }
986
987                /* Recheck PageLRU and PageCompound under lock */
988                if (!PageLRU(page))
989                    goto isolate_fail;
990
991                /*
992                 * Page become compound since the non-locked check,
993                 * and it's on LRU. It can only be a THP so the order
994                 * is safe to read and it's 0 for tail pages.
995                 */
996                if (unlikely(PageCompound(page) && !cc->alloc_contig)) {
997                    low_pfn += compound_nr(page) - 1;
998                    goto isolate_fail;
999                }
1000           }
1001
1002           lruvec = mem_cgroup_page_lruvec(page, pgdat);
1003
1004           /* Try isolate the page */
1005           if (__isolate_lru_page(page, isolate_mode) != 0)
1006               goto isolate_fail;
1007
1008           /* The whole page is taken off the LRU; skip the tail pages. */
1009           if (PageCompound(page))
1010               low_pfn += compound_nr(page) - 1;
1011
1012          /* Successfully isolated */
1013           del_page_from_lru_list(page, lruvec, page_lru(page));
1014           mod_node_page_state(page_pgdat(page),
1015                   NR_ISOLATED_ANON + page_is_file_lru(page),
1016                   thp_nr_pages(page));
1017
1018   isolate_success:
1019           list_add(&page->lru, &cc->migratepages);
1020           cc->nr_migratepages += compound_nr(page);
1021           nr_isolated += compound_nr(page);
1022
1023           /*
```

```
1024            * Avoid isolating too much unless this block is being
1025            * rescanned (e.g. dirty/writeback pages, parallel allocation)
1026            * or a lock is contended. For contention, isolate quickly to
1027            * potentially remove one source of contention.
1028            */
1029           if (cc->nr_migratepages >= COMPACT_CLUSTER_MAX &&
1030               !cc->rescan && !cc->contended) {
1031               ++low_pfn;
1032               break;
1033           }
1034
1035           continue;
1036   isolate_fail:
1037           if (!skip_on_failure)
1038               continue;
1039
1040           /*
1041            * We have isolated some pages, but then failed. Release them
1042            * instead of migrating, as we cannot form the cc->order buddy
1043            * page anyway.
1044            */
1045           if (nr_isolated) {
1046               if (locked) {
1047                   spin_unlock_irqrestore(&pgdat->lru_lock, flags);
1048                   locked = false;
1049               }
1050               putback_movable_pages(&cc->migratepages);
1051               cc->nr_migratepages = 0;
1052               nr_isolated = 0;
1053           }
1054
1055           if (low_pfn < next_skip_pfn) {
1056               low_pfn = next_skip_pfn - 1;
1057               /*
1058                * The check near the loop beginning would have updated
1059                * next_skip_pfn too, but this is a bit simpler.
1060                */
1061               next_skip_pfn += 1UL << cc->order;
1062           }
1063       }
1064
1065       /*
1066        * The PageBuddy() check could have potentially brought us outside
1067        * the range to be scanned.
1068        */
1069       if (unlikely(low_pfn > end_pfn))
1070           low_pfn = end_pfn;
1071
1072   isolate_abort:
1073       if (locked)
1074           spin_unlock_irqrestore(&pgdat->lru_lock, flags);
1075
1076       /*
1077        * Updated the cached scanner pfn once the pageblock has been scanned
1078        * Pages will either be migrated in which case there is no point
1079        * scanning in the near future or migration failed in which case the
1080        * failure reason may persist. The block is marked for skipping if
1081        * there were no pages isolated in the block or if the block is
1082        * rescanned twice in a row.
1083        */
1084       if (low_pfn == end_pfn && (!nr_isolated || cc->rescan)) {
1085           if (valid_page && !skip_updated)
1086               set_pageblock_skip(valid_page);
1087           update_cached_migrate(cc, low_pfn);
```

```
1088        }
1089
1090        trace_mm_compaction_isolate_migratepages(start_pfn, low_pfn,
1091                                nr_scanned, nr_isolated);
1092
1093    fatal_pending:
1094        cc->total_migrate_scanned += nr_scanned;
1095        if (nr_isolated)
1096            count_compact_events(COMPACTISOLATED, nr_isolated);
1097
1098        return low_pfn;
1099    }
```

- 804 获取 zone 所属的 NUMA 节点。
- 810 获取扫描区间起始 pfn。
- 820 检查目标节点 lru 上被隔离的页数量是否超限。

  多个并发回收/整理会把大量页面从 lru 隔离，隔离太多会破坏 lru 逻辑，需要等待隔离计数降下来。
- 822-823 如果当前已经隔离了一批还未迁移的页面，直接返回 0，不再继续隔离，优先把已经隔离的页迁移完毕。
- 826-827 异步模式不阻塞，遇到隔离数超限直接终止隔离。此处的异步仍然指不阻塞，并非是后台执行。
- 829 等待设备解除拥塞，休眠 *HZ/10* 时间让出 CPU，等待其他线程完成页面回写、归还隔离页。
- 831-832 检测到关键信号，直接终止隔离。
- 835 如果有更高优先级任务，主动让出 CPU，避免内核线程占死 CPU。
- 837-840 进程上下文进行非阻塞 (异步) 直接整理；尝试收集连续 $2^{o}rder$ 页；如果块内隔离中途失败，就不再挨个扫描 pfn，直接跳过整个 order 块，减少开销。

  839 这里得到的 *next_skip_pfn* 是 low_pfn 所在的，按 *cc->order* 对齐的块的末尾的 pfn。它是开区间 $[low_pfn, next_skip_pfn)$ 的尾部边界，属于下一个 order 块。

  需要注意的是：这里是块内隔离时的跳过，是 *cc->order* 的粒度。和 pageblock 因为 *PB_migrate_skip* 的 hint 而跳过的的粒度不同。

  ========== 检测过程的插入图
- 843 主循环，逐个 pfn 扫描。
- 845 如果开启了快速跳过 (异步直接整理)，且 pfn 走到了上一个 order 块的末尾。换言之，已经完整遍历完一个 $1 << cc- > order$ 大小的对齐块。
- 852-853 若在刚才这一段 *cc->order* 对齐块里，确实隔离到了可迁移页面 ($nr_isolated > 0$)，停止扫描，返回上层去迁移已经隔离好的页面，不继续扫描更多页。

  刚刚完整扫完一个 order 粒度的对齐块，并且在这块里面隔离到了可迁移页。那就不再继续扫描本 pageblock 剩下的页面了，直接停止扫描。上层调用者 (*compact_zone()*) 拿着已经隔离好的页面链表去执行 *migrate_pages()* 做迁移，寄希望于迁移完就能得到目标 *cc->order* 的连续空闲内存，让整理成功。如果这段一块可迁移页都没隔离到，则继续扫描下一个 order 对齐块。

同步内存整理不会走这个提前退出逻辑，会完整扫描整个 pageblock(隔离计数达到 *COMPACT_CLUSTER_MAX* 时，也会提前终止 pageblock 扫描)。

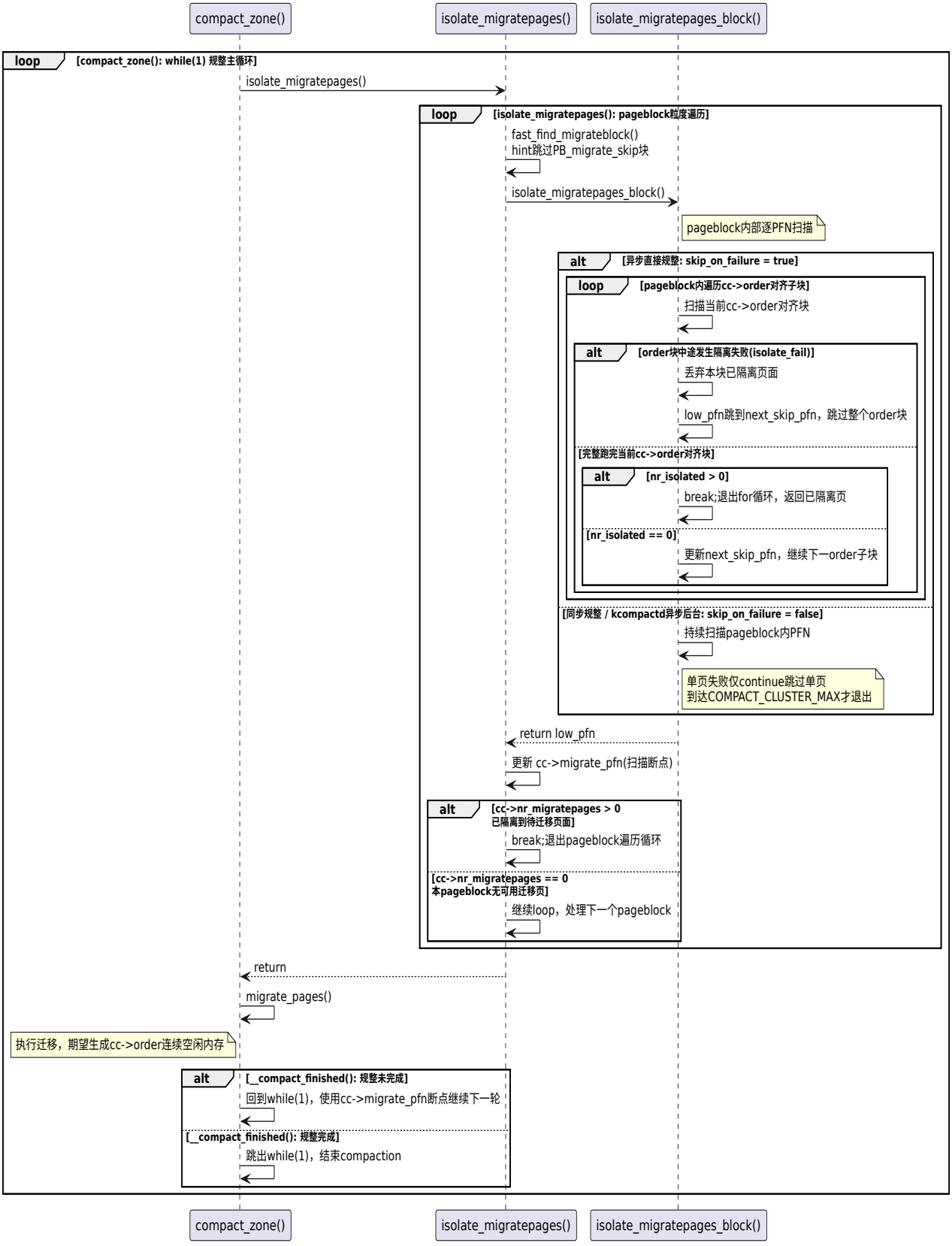


图 6–1: compact_zone() 扫描迁移块时主要动作的序列图

- 864 上一个 order 块隔离失败, 直接取该块末尾 pfn 作为新的跳过边界。计算 order 对齐的下一个 pfn。

  上一轮我们未能隔离出按阶对齐的内存块。将新的边界设置为当前内存块的末尾。不能简单地将 *next_skip_pfn* 增加 $1 << order$, 因为在上一轮循环迭代中，由于跳过复合页或者高阶伙伴页，*low_pfn* 的步进值可能已经大于该阶对应的页数量。此时再粗暴 $next_skip_pfn += 1 << order$ 会产生 pfn 错位，所以直接取当前块的末尾 pfn 作为新的跳过边界。

- 872-877 周期性释放锁。如果有紧急信号要处理，则完全终止本次整理。

  每次扫描完 $SWAP_CLUSTER_MAX(32)$ 个页面触发一次检查，检查下面两件事:

  - 如果持有 *pgdat->lru_lock* 则临时解锁。
  - 检查当前进程是否有紧急信号 (如 *SIGKILL*)pending 等待处理。

- 875-876 一旦检测到致命信号，就置 *low_pfn*, 并返回上层。

  直接内存整理实在用户进程上下文执行，进程收到 SIGKILL 信号，整理的动作必须立即停止不能继续循环扫描内存。

- 879-880 判断目标 pfn 在目标 zone 内是否有效物理页帧。若是空洞等则返回 fail。

- 883 获取目标页帧对应的 struct page 结构体。后语操作基于 struct page。

- 891 对全新的 pageblock 的第一页，这些 hint 检查。pageblock 内部其他 pfn 跳过此分支。

  valid_page 是目标 pageblock 的起始 page 指针，初始为 NULL。一旦进入过本 pageblock 扫描，*valid_page* 就会被赋值。

  IS_ALGIN() 此处等价于判断 *low_pfn* 是否是一个 pageblock 的起始 pfn。

- 892 在无需忽略 hint 的场景下，读取 pageblock 的 page_flags, 判断是否打上了 *PB_migrate_skip*。

  - *MIGRATE_SYNC* 同步整理，意味着可以阻塞等待。通常强制忽略 hint，不跳过。$ignore_skip_hint = true$。
  - *MIGRATE_ASYNC* 异步整理，这里的异步不是指后台执行，而是意味着不阻塞等待。$ignore_skip_hint = false$。

- 893-894 如果上面代码判断需要跳过目标 pageblock, 将 pfn 游标直接设置为该 pageblock 的开区间末尾的 pfn。不再扫描该 pageblock。

- 896 进入全新的 pageblock, 且无需忽略该 pageblock，那么把 valid_page 置为首 page。这样之后多次进入该 pageblock 内存扫描时无需再检测 hint。

- 904-915 如果目标库属于伙伴系统的空闲块，则跳过。

  - 906 读取目标块的 order。

    这里是无锁操作，正常应该持有 zone 的对应锁。正常来讲这是不安全的，但是竞争窗口很小，最坏的结果就是跳过了一部分本可以被隔离的候选页。

  - 913-914 因为是无锁操作，无法保证读到的 order 一定是合法的。这里只接收合法范围的 order 值，并计算出末尾边界的 pfn。

- 926-932 非 CMA 这种大尺寸连续内存的分配场景下，遇到复合大页。直接跳过大页不整理，当做页面隔离失败处理，除非 *skip_in_failure* 机制。

  929-930 同 913 行，在无锁操作下把 order 限制在合法范围。

- 939-954 非 lru 页的处理。

  - 944-945 未被隔离的非 lru 可移动页。
  - 952-953 进行隔离。
  - 956 其余非 lru 页跳转作隔离失败处理。

- 964-966 如果目标页是匿名页，且匿名页的引用计数 > 进程映射计数，说明内核层面有额外的 pin(固定住) 引用，隔离失败。

  匿名页的 *page->mapping = NULL*。

- 972-973 如果目标页是文件页，但当前上下文不允许文件系统操作，返回隔离失败。

- 976-1000 加锁后重新校验。

- 1005 给目标页标记 *PG_isolated*。

- 1009-1010 如果是 THP 复合页，head 隔离成功，整个 THP 就一起被隔离，tail 无需扫描处理。

  这里游标直接跳到 THP 最末一个 tail 子页的 PFN。

- 1013 将目标页从对应 lru 链表摘除，页面正式进入隔离状态。

- 1014-1016 增加 node 的隔离页统计。THP 按基页数量的多少进行统计。

- 1019 将隔离成功的页加入迁移链表 (*cc->migratepages*)。

- 1020-1021 分别累加迁移页的数量，和隔离成功的页的数量。

- 1029-1033 正常情况下，一次最多隔离 *COMPACT_CLUSTER_MAX* 个页，分批处理 pageblock，防止单次隔离过多页面，长时间占用 lru_lock。

  cc->rescan 表示重扫模式，代表 pageblock 内有大量脏页/回写页，需要多隔离，不受 32 页限制。

  cc->contended 表示是否有锁竞争，如果有，就不受 32 页上限的约束，要多隔离一些页，减少反复拿锁的次数。

  如果满足 if 的条件，则结束循环退出。若不满足，则进行走 1035 行，处理下一个 PFN。

- 1037-1038 同步模式 (MIGRATE_SYNC) 遇到单页隔离失败，继续处理下一个 PFN。异步模式 (MIGRATE_ASYNC) 则往下走要跳过整个 *cc->order* 块。

- 1045-1050 已经成功隔离若干页，但是块内出现不可迁移页，凑不出目标 order，因此将迁移链表上已经隔离成功的页放回原 lru 链表，将这批页退回，不再参与本次迁移。

- 1051-1052 清 0 隔离计数器。

- 1056 将 low_pfn 直接跳到待跳过块的最末一个 PFN。

- 1061 把下一个要跳过块的起点往后挪 1 << *cc− > order*。

  异步直接整理才会走到这里，同步模式直接继续一页一页扫，不做块跳过。

- 1069-1070 末尾边界保护，防止跳到 end_pfn 之外。
- 1084 完整扫完 pageblock, 但是一页都没隔离成功，或者当前处于重扫模式。
- 1085-1086 给目标 pageblock 打上 *PB_migrate_skip* 标记。这是临时标记，下一轮整理会重新评估。

6.2.2 compact_finished()

在 2276 行的 compact_finished() 最终会调用 ___compact_finished(), 如下:

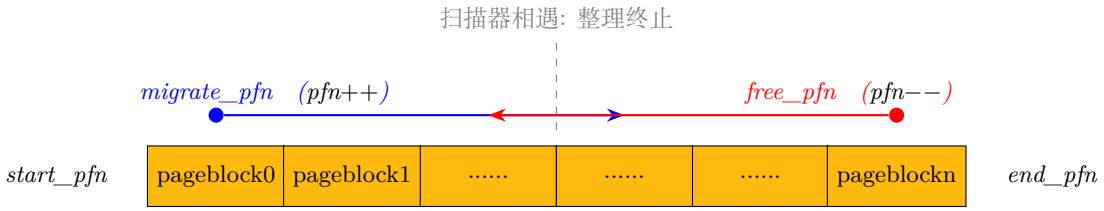
6.2.2.1 ___compact_finished()

compact_zone() > *compact_finished()* > *___compact_finished()*

```
1951 static enum compact_result __compact_finished(struct compact_control *cc)
1952 {
1953     unsigned int order;
1954     const int migratetype = cc->migratetype;
1955     int ret;
1956
1957     /* Compaction run completes if the migrate and free scanner meet */
1958     if (compact_scanners_met(cc)) {
1959         /* Let the next compaction start anew. */
1960         reset_cached_positions(cc->zone);
1961
1962         /*
1963          * Mark that the PG_migrate_skip information should be cleared
1964          * by kswapd when it goes to sleep. kcompactd does not set the
1965          * flag itself as the decision to be clear should be directly
1966          * based on an allocation request.
1967          */
1968         if (cc->direct_compaction)
1969             cc->zone->compact_blockskip_flush = true;
1970
1971         if (cc->whole_zone)
1972             return COMPACT_COMPLETE;
1973         else
1974             return COMPACT_PARTIAL_SKIPPED;
1975     }
1976
1977     if (cc->proactive_compaction) {
1978         int score, wmark_low;
1979         pg_data_t *pgdat;
1980
1981         pgdat = cc->zone->zone_pgdat;
1982         if (kswapd_is_running(pgdat))
1983             return COMPACT_PARTIAL_SKIPPED;
1984
1985         score = fragmentation_score_zone(cc->zone);
1986         wmark_low = fragmentation_score_wmark(pgdat, true);
1987
1988         if (score > wmark_low)
1989             ret = COMPACT_CONTINUE;
1990         else
1991             ret = COMPACT_SUCCESS;
```

```
1992
1993            goto out;
1994        }
```

- 1958 检查迁移扫描器 (migrate scanner) 和空闲扫描器 (free scanner) 是否相遇。



→ migrate scanner：pfn 从小到大扫描
← free scanner：pfn 从大到小扫描

内存整理过程中，迁移扫描器从 zone 头部向后扫描可移动页，空闲扫描器从 zone 尾部向前扫描空闲页。相遇表示当前扫描范围已处理完毕。

- 1960 为下一次整理操作提供干净的扫描起点。扫描器相遇，本次整理区间走完，下一次整理不要复用旧断点，重新开始。

- 1968 检查当前是否为直接内存整理 (direct compaction)，目的是通过同步整理立即解决高阶连续内存分配问题。

- 1969 若当前是直接内存整理，则将 *compact_blockskip_flush* 标记为 true。此标记的作用是通知内核，在后续流程中需要清除 *PG_migrate_skip* 信息。

  PG_migrate_skip 标志用于标记某些页块 (pageblock) 是否应跳过迁移扫描。直接内存整理是同步且高优先级的操作，当直接内存整理过程中发现某个页块无法迁移 (如包含不可移到页或锁竞争) 时，内核会设置此标志，避免短期内的重复无效扫描。

  但长期保留会导致内存整理逻辑僵化，因此需在内存压力缓解后 (kswapd 睡眠意味着内存压力缓解) 重置状态。内核在下一次 kswapd 睡眠时检查到 *compact_blockskip_flush* 标记为 true，就会清除 *PG_migrate_skip*，强制后续整理重新扫描这些块。

  后台整理线程 kcompacted 仅处理异步整理，其决策不依赖内存分配请求，因此不直接管理此标志。

- 1971 检查当前整理是否覆盖整个 zone。

  返回值 *COMPACT_COMPLETE*(整个 zone 已整理完毕)。*COMPACT_PARTIAL_SKIPPED* 表示仅部分 zone 完成整理，只是增量片段扫完，还有部分 zone 没有触碰。上层调用者会区分这两个返回值。

- 1977 检查当前是否处于主动内存整理 (proactive compaction) 模式。

  主动整理由内核线程 kcompactd 周期性设置->proactive_compaction 来触发 (细节待添加: 对->proactive_compaction 的赋值)，目的是在内存压力出现前提前主动整理碎片。

  proactive_compaction = true：代表本次是 ***kcompactd*** 后台预整理路径，不是分配触发的**直接整理**。

- 1982 检查当前节点的内存回收线程 kswapd 是否正在进行。若是，则返回 *COMPACT_PARTIAL_SKIPPED*, 跳过主动整理。避免 kcompactd 与 kswapd 同时操作内存。
- 1985 计算当前 zone 的碎片化分值。这个分值是就大页 (*COMPACTION_HPAGE_ORDER* 阶的页) 而言的外部碎片的评分，范围是 [0,100]

$$\text{碎片贡献分数} = \frac{\text{当前 zone 内实际存在的物理总页数}}{\text{整个 node 的物理总页数}} \times \text{当前 zone 针对巨页的外部碎片指数}$$

COMPACTION_HPAGE_ORDER: 表示大页分配所需的内存阶数。通常对应内核的默认大页尺寸。通过把 *COMPACTION_HPAGE_ORDER* 作为碎片评分的基准阶数，为了优先保障大页 (Huge Page) 分配所需的连续内存，从而驱动内存整理的触发和执行。

- 1986 获取主动内存整理的低阈值 (代码见后)。

  这是碎片贡献分数的水位, 碎片贡献分数降到低阈值以下，就认为本 zone 的巨页碎片已经达标，kcompactd 不用再折腾这个 zone。

- 1988-1991
  - *zone*加权碎片贡献分数 *wmark_low*: 这个 zone 碎片贡献还大，继续整理。
  - *zone*加权碎片贡献分数 $\leq$ *wmark_low*: 这个 zone 已经合格，跳过不再整理；

  vm.compaction_proactiveness(0-100)，主动整理积极性越大，*wmark_low* 数值就越小。

extfrag_for_order() 用来计算一个 zone 针对指定 order 的外部碎片指数。也就是目标 zone 内存区域的空闲页里，$< 2^{order}$ 的页数占全部空闲页的百分比。返回值范围 $\in [0, 100]$。

外部碎片含义: 总空闲页数够，但是空闲块都太小，凑不出一块连续 2^{order} 页。

compact_zone() › *compact_finished()* › ··· › *fragmentation_score_zone()* › *extfrag_for_order()*

```
1104 unsigned int extfrag_for_order(struct zone *zone, unsigned int order)
1105 {
1106     struct contig_page_info info;
1107
1108     fill_contig_page_info(zone, order, &info);
1109     if (info.free_pages == 0)
1110         return 0;
1111
1112     return div_u64((info.free_pages -
1113             (info.free_blocks_suitable << order)) * 100,
1114             info.free_pages);
1115 }
```

- 1108 遍历 zone 的空闲链表, 累加所有空闲页到 *info.free_pages*, 并统计有多少个满足 $\geq$ *order* 的空闲页块。
- 1109-1110 如果空闲页为 0 就直接返回。
- 1112-1114 这几行的公式如下:

$$exfrag = \frac{\left(free_pages - free_blocks_suitable \times 2^{order}\right) \times 100}{free_pages}$$

$free_blocks_suitable << order$ 等价 $free_blocks_suitable \times 2^{order}$。可以满足 order 分配的空闲块，它们加起来一共占的空闲页数。

那么 $\left(free_pages - free_blocks_suitable \times 2^{order}\right)$，代表总空闲页 − “合格大块里面的空闲页”，得到的差就是碎片页。

所以前面的公式也就是:

$$\begin{aligned}\text{外部碎片指数} &= \frac{\left(\text{空闲页总数} - \text{合格的块数量} \times 2^{order}\right) \times 100}{\text{空闲页总数}} \\ &= \frac{(\text{空闲页总数} - \text{合格的块对应的页数}) \times 100}{\text{空闲页总数}} \\ &= \frac{(\text{碎片页数量}) \times 100}{\text{空闲页总数}} \\ &= \frac{\text{碎片页数量}}{\text{空闲页总数}}\%\end{aligned}$$

这里很明显可以看出: 大页块满足目标 order 之外的富余部分会被错误算成碎片。但它是主动预先整理(proactive compaction) 的启发式打分，够用即可，不追求数学上完美精确的计算外部碎片。

compact_zone() → *compact_finished()* → *__compact_finished()* → *fragmentation_score_zone()*

```
1896 static unsigned int fragmentation_score_zone(struct zone *zone)
1897 {
1898     unsigned long score;
1899
1900     score = zone->present_pages *
1901             extfrag_for_order(zone, COMPACTION_HPAGE_ORDER);
1902     return div64_ul(score, zone->zone_pgdat->node_present_pages + 1);
1903 }
```

函数 *extfrag_for_order()* 只反映 zone 本身针对巨页级 (*COMPACTION_HPAGE_ORDER*) 块的外部碎片指数 ($\in [0, 100]$); 本函数做加权，压制小 zone，防止小 zone 碎片高就频繁触发主动整理。

- 1900-1901 利用公式:

  $$score = \text{当前 zone 内实际存在的物理总页数} \times \text{当前 zone 正对巨页的外部碎片指数}$$

  将碎片指数按 zone 内存大小做加权放大或缩小。

  COMPACTION_HPAGE_ORDER: 巨页对应的 order，如 arm64 2M 巨页 order=9。

- 1902 将 score÷ 整个节点实际存在物理页数。也就是:

  $$\begin{aligned}\text{碎片贡献值} &= \frac{\text{当前 zone 内实际存在的物理总页数} \times \text{当前 zone 针对巨页的外部碎片指数}}{\text{整个 node 的物理总页数}} \\ &= \frac{\text{当前 zone 内实际存在的物理总页数}}{\text{整个 node 的物理总页数}} \times \text{当前 zone 针对巨页的外部碎片指数}\end{aligned}$$

  返回目标 zone 对整个 NUMA 节点巨页外部碎片问题的贡献分数: 同等碎片指数下，zone 占 node 内存比重越大，贡献分数越高；小 zone 贡献被自然压低，避免小特殊 zone 无谓触发主动内存规整。

  代码这里的 +1 是为了防止除零产生 crash。

compact_zone() > compact_finished() > __compact_finished() > fragmentation_score_wmark()

```
1927 static unsigned int fragmentation_score_wmark(pg_data_t *pgdat, bool low)
1928 {
1929     unsigned int wmark_low;
1930
1931     /*
1932      * Cap the low watermak to avoid excessive compaction
1933      * activity in case a user sets the proactivess tunable
1934      * close to 100 (maximum).
1935      */
1936     wmark_low = max(100U - sysctl_compaction_proactiveness, 5U);
1937     return low ? wmark_low : min(wmark_low + 10, 100U);
1938 }
```

- 1936 sysctl_compaction_proactiveness 是内核全局变量 (默认是 20)，可以在用户空间通过 sysctl 命令或节点来重新设置值。表示内存整理的积极程度。该值越大” 碎片阈值” 就越低，从而内核对碎片的容忍度就越低，因此就越倾向于更早触发整理。

  通过 max(...,5U) 确保 wmark_low 最低为 5%, 防止因用户设置过高的 proactiveness 导致频繁整理。该行返回的是低阈值 (wmark_low)。

- 1937 通过入参 low 为 false(0) 还是 true(1)，决定返回高阈值 (wmark_high) 还是低阈值 (wmark_low)。高阈值 == 低阈值 $\times$ (1 + 10%), 高阈值最大不超过 100。

很多应用程序都能从大页的使用中显著受益。但在内存碎片化的环境下，大页分配往往会产生很高的延迟，甚至直接分配失败。当每个 NUMA 节点专属的 *kcompactd* 线程被唤醒时，它只会做刚好足够的整理工作，产出一个目标阶数的大页，一旦得到该大页，*kcompactd* 线程就重新进入睡眠。这种工作模式会导致高阶内存分配延迟很高，对于需要突发批量分配大量大页的业务负载，会严重损害性能。

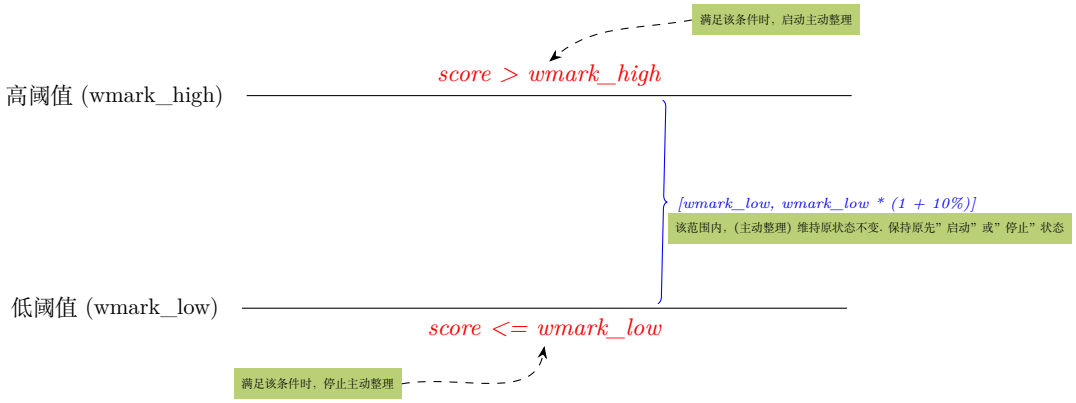
主动内存整理 (***proactive compaction***) 通过在后台执行内存整理，可以为这类问题提供有效的解决方案。

/proc/sys/vm/compaction_proactiveness，该参数控制后台执行内存整理的激进程度，取值范围 [0, 100]，默认值为 20。每个 NUMA 节点已有的 *kcompactd* 内核线程完成后台整理的工作。每个线程会周期性计算本节点的节点碎片化分数 (表征内存碎片化程度)，并与阈值做对比。

每个节点 (per-node) 的 *kcompactd* 线程周期性检查节点的碎片化评分值。若节点的碎片化分值超过**高阈值**，则主动启动 *kcompactd* 线程整理 (置位->proactive_compaction), 直至分值降至**低阈值**以下才停止整理。

在物理地址空间大量散布**不可移动页面**。在这种内存状态下，内存整理几乎无法产生实际收益，只会空耗 CPU。所以检测一轮整理完成后，节点碎片化分数没有下降，一旦判定该情况，就会将后续的**主动**整理任务延后一段时间执行。

这里对于是否执行碎片主动整理的逻辑，和之前基于 high、low 水位线唤醒和休眠 kswapd 的方式很类似。示意如下：



下面回到 ___compact_finished() 函数 1986 行之后的代码。

compact_zone() > compact_finished() > ___compact_finished()

```
1985            score = fragmentation_score_zone(cc->zone);
1986            wmark_low = fragmentation_score_wmark(pgdat, true);
1987
1988            if (score > wmark_low)
1989                ret = COMPACT_CONTINUE;
1990            else
1991                ret = COMPACT_SUCCESS;
1992
1993            goto out;
1994        }
1995
1996        if (is_via_compact_memory(cc->order))
1997            return COMPACT_CONTINUE;
1998
1999        /*
2000         * Always finish scanning a pageblock to reduce the possibility of
2001         * fallbacks in the future. This is particularly important when
2002         * migration source is unmovable/reclaimable but it's not worth
2003         * special casing.
2004         */
2005        if (!IS_ALIGNED(cc->migrate_pfn, pageblock_nr_pages))
2006            return COMPACT_CONTINUE;
2007
2008        /* Direct compactor: Is a suitable page free? */
2009        ret = COMPACT_NO_SUITABLE_PAGE;
2010        for (order = cc->order; order < MAX_ORDER; order++) {
2011            struct free_area *area = &cc->zone->free_area[order];
2012            bool can_steal;
2013
2014            /* Job done if page is free of the right migratetype */
2015            if (!free_area_empty(area, migratetype))
2016                return COMPACT_SUCCESS;
2017
2018    #ifdef CONFIG_CMA
2019            /* MIGRATE_MOVABLE can fallback on MIGRATE_CMA */
2020            if (migratetype == MIGRATE_MOVABLE &&
2021                !free_area_empty(area, MIGRATE_CMA))
2022                return COMPACT_SUCCESS;
2023    #endif
2024            /*
2025             * Job done if allocation would steal freepages from
2026             * other migratetype buddy lists.
2027             */
2028            if (find_suitable_fallback(area, order, migratetype,
2029                                       true, &can_steal) != -1) {
2030
```

```
2031            /* movable pages are OK in any pageblock */
2032            if (migratetype == MIGRATE_MOVABLE)
2033                return COMPACT_SUCCESS;
2034
2035            /*
2036             * We are stealing for a non-movable allocation. Make
2037             * sure we finish compacting the current pageblock
2038             * first so it is as free as possible and we won't
2039             * have to steal another one soon. This only applies
2040             * to sync compaction, as async compaction operates
2041             * on pageblocks of the same migratetype.
2042             */
2043            if (cc->mode == MIGRATE_ASYNC ||
2044                    IS_ALIGNED(cc->migrate_pfn,
2045                            pageblock_nr_pages)) {
2046                return COMPACT_SUCCESS;
2047            }
2048
2049            ret = COMPACT_CONTINUE;
2050            break;
2051        }
2052    }
2053
2054 out:
2055    if (cc->contended || fatal_signal_pending(current))
2056        ret = COMPACT_CONTENDED;
2057
2058    return ret;
2059 }
```

- 1988-1991 如果 score 大于该值意味着碎片化程度高，需要继续整理; 如果 score 小于该值，说明碎片化程度低，整理可结束。
- 1993 直接跳到末尾，不再走后面直接整理的空闲页判断逻辑。
- 1996-1997 判断本次是否用户空间通过*/proc/sys/vm/compact_memory* 触发的手动强制整理。如果是手动触发的整理，那么返回 *COMPACT_CONTINUE*，不提前退出，会做完整的全局 zone 整理。此时 order 参数被设为-1. 这种场景下内存整理会扫描所有内存 zone。
- 2005-2006 判断 *migrate_pfn* 是否对齐到 pageblock 边界。

  必须把当前 pageblock 处理完，不能半路从 pageblock 中间退出。
- 2010 从本次需要的 *cc->order* 一直遍历到 *MAX_ORDER-1*, 逐个检查每个 order 的 free_area 空闲链表。
- 2015-2016 当前迭代 order 的 free_area 链表中本迁移类型已经存在空闲块。内存整理目标达成，直接返回 *COMPACT_SUCCESS* 退出整理。
- 2018-2023 *CONFIG_CMA* 开启时：*MIGRATE_MOVABLE* 分配，可以备选使用 CMA 类型空闲页; 如果该 order 存在 CMA 空闲块，同样直接返回 *COMPACT_SUCCESS* 结束整理。
- 2028-2029 查找是否可以从其他迁移类型“偷”空闲块来满足本次分配。
- 2032-2033 如果本次申请分配的是 *MIGRATE_MOVABLE*：只要有可偷的备选块，直接成功退出。可迁移类型的页可以放在任意 pageblock，不需要额外约束。

- 2043-2046 非可移动类型分配，如果要偷别的类型块，尽量把当前 pageblock 榨干释放，避免短时间内又要偷第二个 pageblock。

  两个满足其一就直接 *COMPACT_SUCCESS* 退出:

  - 异步内存整理。
  - 同步直接内存整理，本 pageblock 已处理完。

- 2049 同步模式，且 migrate_pfn 没有到达 pageblock 边界，即使找到备选也不能退出：设置 *COMPACT_CONTINUE* 继续.

- 2055-2056 内存整理发生竞争，比如 page 锁争用。或当前进程收到致命信号 (SIGKILL)。只要满足任意一个，强制把返回值改写为 *COMPACT_CONTENDED*，表示因竞争/信号终止内存整理。

6.2.3 migrate_pages()

回到 compact_zone() 函数 2318 行的 migrate_pages():

compact_zone() › *migrate_pages()*

```
1416 int migrate_pages(struct list_head *from, new_page_t get_new_page,
1417         free_page_t put_new_page, unsigned long private,
1418         enum migrate_mode mode, int reason)
1419 {
1420     int retry = 1;
1421     int thp_retry = 1;
1422     int nr_failed = 0;
1423     int nr_succeeded = 0;
1424     int nr_thp_succeeded = 0;
1425     int nr_thp_failed = 0;
1426     int nr_thp_split = 0;
1427     int pass = 0;
1428     bool is_thp = false;
1429     struct page *page;
1430     struct page *page2;
1431     int swapwrite = current->flags & PF_SWAPWRITE;
1432     int rc, nr_subpages;
1433
1434     if (!swapwrite)
1435         current->flags |= PF_SWAPWRITE;
1436
1437     for (pass = 0; pass < 10 && (retry || thp_retry); pass++) {
1438         retry = 0;
1439         thp_retry = 0;
1440
1441         list_for_each_entry_safe(page, page2, from, lru) {
1442 retry:
1443             /*
1444              * THP statistics is based on the source huge page.
1445              * Capture required information that might get lost
1446              * during migration.
1447              */
1448             is_thp = PageTransHuge(page) && !PageHuge(page);
1449             nr_subpages = thp_nr_pages(page);
1450             cond_resched();
1451
1452             if (PageHuge(page))
1453                 rc = unmap_and_move_huge_page(get_new_page,
1454                         put_new_page, private, page,
```

```
                        pass > 2, mode, reason);
            else
                rc = unmap_and_move(get_new_page, put_new_page,
                        private, page, pass > 2, mode,
                        reason);

            switch(rc) {
            case -ENOMEM:
                /*
                 * THP migration might be unsupported or the
                 * allocation could've failed so we should
                 * retry on the same page with the THP split
                 * to base pages.
                 *
                 * Head page is retried immediately and tail
                 * pages are added to the tail of the list so
                 * we encounter them after the rest of the list
                 * is processed.
                 */
                if (is_thp) {
                    lock_page(page);
                    rc = split_huge_page_to_list(page, from);
                    unlock_page(page);
                    if (!rc) {
                        list_safe_reset_next(page, page2, lru);
                        nr_thp_split++;
                        goto retry;
                    }

                    nr_thp_failed++;
                    nr_failed += nr_subpages;
                    goto out;
                }
                nr_failed++;
                goto out;
            case -EAGAIN:
                if (is_thp) {
                    thp_retry++;
                    break;
                }
                retry++;
                break;
            case MIGRATEPAGE_SUCCESS:
                if (is_thp) {
                    nr_thp_succeeded++;
                    nr_succeeded += nr_subpages;
                    break;
                }
                nr_succeeded++;
                break;
            default:
                /*
                 * Permanent failure (-EBUSY, -ENOSYS, etc.):
                 * unlike -EAGAIN case, the failed page is
                 * removed from migration page list and not
                 * retried in the next outer loop.
                 */
                if (is_thp) {
                    nr_thp_failed++;
                    nr_failed += nr_subpages;
                    break;
                }
                nr_failed++;
                break;
```

```
1519                }
1520            }
1521        }
1522        nr_failed += retry + thp_retry;
1523        nr_thp_failed += thp_retry;
1524        rc = nr_failed;
1525    out:
1526        count_vm_events(PGMIGRATE_SUCCESS, nr_succeeded);
1527        count_vm_events(PGMIGRATE_FAIL, nr_failed);
1528        count_vm_events(THP_MIGRATION_SUCCESS, nr_thp_succeeded);
1529        count_vm_events(THP_MIGRATION_FAIL, nr_thp_failed);
1530        count_vm_events(THP_MIGRATION_SPLIT, nr_thp_split);
1531        trace_mm_migrate_pages(nr_succeeded, nr_failed, nr_thp_succeeded,
1532                               nr_thp_failed, nr_thp_split, mode, reason);
1533
1534        if (!swapwrite)
1535            current->flags &= ~PF_SWAPWRITE;
1536
1537        return rc;
1538    }
```

- 1431 读取当前进程 task_struct->flags 的 *PF_SWAPWRITE* 位。该位决定了是否允许执行 swap out。
- 1434-1435 检查当前进程是否具备 swap 写入权限，若无，则临时启用该权限 。页面迁移可能涉及匿名页的换出操作, 若未启用此权限, 内核无法将匿名页写入 swap 分区，可能引发内存分配阻塞或 OoM。
- 1437 普通页 (retry) 或透明大页 (thp_retry) 最多可以尝试 10 次迁移。
- 1441 遍历待迁移页面链表。list_for_each_entry_safe 遍历链表时，用临时指针存储下一个数据结构的地址，所以在遍历链表的时候如果要删除链表中的当前项, 可以安全的删除, 而不会影响接下来的遍历过程 (用临时指针可以继续完成接下来的遍历, 而 list_for_each_entry 则无法继续遍历)
- 1442 goto 标签。专门用于 THP 拆分之后，原地重新尝试迁移同一个 head page。
- 1448 判断是否透明大页 (参见 14.1章节)。

  PageTransHuge(page) 判断是否透明巨页的 head page。

  !PageHuge(page) 用于排除 hugetlb 静态巨页。

对比维度	***HugeTLB* 静态大页**	***THP* 透明巨页**
物理内存来源	启动时预分配 *hugetlb* 私有池，不属于普通 *buddy* 空闲链表	普通 *buddy* 系统高阶页 *(order-9 2M)*，完全由 *buddy* 管理
生命周期	释放后归还 *hugetlb* 私有池，不释放回 *buddy*	用完可以 *split_huge_page()* 拆为 *512* 个 *4K* 小页，直接归还 *buddy*；可被回收整理
swap 交换支持	不支持 *swap*，不能换出到 *swap* 设备	支持 *swapout*
内存回收支持	不会被 *kswap* 回收，不受 *vm* 水位控制	可被 *kswap* 回收；可被页面老化逻辑扫描回收
内存整理的支持	已预留的 *hugetlb* 页不参与内存整理迁移扫描	强依赖内存整理，分配不到高阶页会触发直接内存整理
内存浪费风险	需要预先预估用量；预留内存普通进程无法使用，容易闲置浪费	内存动态复用，资源利用率高

- 1449 获取透明巨页包含的子页数。
- 1452-1459
  - 1452-1455 如果是 hugetlb 静态大页，则调用 unmap_and_move_huge_page() 进行处理 (待分析)。
  - 1456-1459 对普通页调用 unmap_and_move 进行迁移。
- 1461-1519 对迁移操作的结果进行处理。
- 1462-1489 因内存不足原因迁移失败后的处理。

  大页迁移可能不受支持，或者分配内存失败。因此失败后将对同一页面重试，把透明大页拆分成基页。首页会立即重试迁移，尾页会被添加到迁移列表尾部，以便在处理完列表其余部分后再处理它们。
  - 1476 拆分大页到迁移链表，每个子页作为普通页处理。(====== 待详细解析)
  - 1479-1480 拆分成功的情况下，存储当前节点的下一个节点的位置，并统计拆分次数。

    list_safe_reset_next 会重新计算下一个节点的地址。当链表在被遍历期间被并发修改时, next 可能指向已失效的节点, 原因如下: 譬如并发删除非当前节点。假设迭代顺序为 A→B→C, 当前处理 B。释放锁后, 其他线程删除 C 并添加 D。重新获取锁时, next 仍指向原 C (已删除), 而实际下一个节点应为 D。因此必须重新计算: 通过 list_safe_reset_next 重新获取正确的下一个节点, 避免指向失效节点。
  - 1481 跳转，重新尝试迁移透明巨页拆分后的 head page。tail 子页会在本轮循环后面遍历到。
  - 1484-1485 统计大页拆分失败次数，以及迁移失败的页数目，子页需要统计计入。
- 1490-1496 临时性失败 (如可能是页被锁定), 下一轮循环会重试。
  - 1492,1495 区分大页和普通页，设置重试标志.
- 1497-1504 迁移成功, 统计成功迁移成功计数。如果是大页，要将子页数目页累加。
- 1505-1518 永久性失败 (-EBUSY、-ENOSYS 等错误码场景) 与-EAGAIN 情况不同，此种失败的页会从迁移页表中移除，且不会在后续外层循环中重试。更新统计信息后跳出。
- 1522 累加未完成的重试页面数
- 1523 累加大页重试失败数
- 1526-1530 记录普通页/大页迁移成功、失败、拆分次数，用于性能分析和调试。
- 1531-1532 通过 tracepoint 记录迁移事件，供 ftrace 等工具追踪内核行为。
- 1534-1535 如果之前临时性允许了当前进程 swap 写入权限，那么移除进程的 PF_SWAPWRITE 标志，恢复原先的权限。

6.2.3.1 split_huge_page_to_list()

compact_zone() > migrate_pages() > split_huge_page_to_list()

```
2617 int split_huge_page_to_list(struct page *page, struct list_head *list)
2618 {
2619     struct page *head = compound_head(page);
2620     struct pglist_data *pgdata = NODE_DATA(page_to_nid(head));
2621     struct deferred_split *ds_queue = get_deferred_split_queue(head);
2622     struct anon_vma *anon_vma = NULL;
2623     struct address_space *mapping = NULL;
2624     int count, mapcount, extra_pins, ret;
2625     unsigned long flags;
2626     pgoff_t end;
2627 
2628     VM_BUG_ON_PAGE(is_huge_zero_page(head), head);
2629     VM_BUG_ON_PAGE(!PageLocked(head), head);
2630     VM_BUG_ON_PAGE(!PageCompound(head), head);
2631 
2632     if (PageWriteback(head))
2633         return -EBUSY;
2634 
2635     if (PageAnon(head)) {
2636         /*
2637          * The caller does not necessarily hold an mmap_lock that would
2638          * prevent the anon_vma disappearing so we first we take a
2639          * reference to it and then lock the anon_vma for write. This
2640          * is similar to page_lock_anon_vma_read except the write lock
2641          * is taken to serialise against parallel split or collapse
2642          * operations.
2643          */
2644         anon_vma = page_get_anon_vma(head);
2645         if (!anon_vma) {
2646             ret = -EBUSY;
2647             goto out;
2648         }
2649         end = -1;
2650         mapping = NULL;
2651         anon_vma_lock_write(anon_vma);
2652     } else {
2653         mapping = head->mapping;
2654 
2655         /* Truncated ? */
2656         if (!mapping) {
2657             ret = -EBUSY;
2658             goto out;
2659         }
2660 
2661         anon_vma = NULL;
2662         i_mmap_lock_read(mapping);
2663 
2664         /*
2665          *__split_huge_page() may need to trim off pages beyond EOF:
2666          * but on 32-bit, i_size_read() takes an irq-unsafe seqlock,
2667          * which cannot be nested inside the page tree lock. So note
2668          * end now: i_size itself may be changed at any moment, but
2669          * head page lock is good enough to serialize the trimming.
2670          */
2671         end = DIV_ROUND_UP(i_size_read(mapping->host), PAGE_SIZE);
2672     }
2673 
2674     /*
2675      * Racy check if we can split the page, before unmap_page() will
2676      * split PMDs
2677      */
```

```
2678     if (!can_split_huge_page(head, &extra_pins)) {
2679         ret = -EBUSY;
2680         goto out_unlock;
2681     }
2682
2683     unmap_page(head);
2684     VM_BUG_ON_PAGE(compound_mapcount(head), head);
2685
2686     /* prevent PageLRU to go away from under us, and freeze lru stats */
2687     spin_lock_irqsave(&pgdata->lru_lock, flags);
2688
2689     ... ... ... ...
2690 }
```

- 2621 获取一个内存节点或 memcg 的用于“延迟拆分 (deferred split)”的队列，这是内核中处理巨页拆分的一种优化机制。巨页 (如 2MB 或 1GB 大小) 的拆分是一个开销较大的操作，可能涉及修改多级页表，刷新 TLB，同步多个 cpu 核心的缓存。为避免在关键路径上执行这些耗时操作，内核引入了延迟拆分机制。

  当需要拆分巨型页时，先标记页为“待拆分”状态。在合适的时机 (如 CPU 空闲时) 再实际执行拆分操作。

- 2628-2630 检查页面状态是否符合预期，如果条件不满足，会触发内核 bug 并打印错误信息。

  - 2628 巨型零页 (huge zero page) 是内核预分配的特殊只读页，用于高效共享零内容。此处检查是否为巨型零页，如果是巨型零页，则触发 bug。

    系统启动时，内核会预分配一定数量的巨型零页，以便后续通过 mmap 请求初始化内存时，若系统支持大页且满足条件，内核会尝试分配巨型零页。巨型零页是系统中唯一的、全局共享的只读页，用于高效提供全零内存区域，内核明确禁止对其进行拆分操作.

    当对巨型零页执行“写入内存”的操作，触发 page fault，陷入内核态。内核 do_page_fault() 函数检测到写错误，并确认当前页为巨型零页 (page == huge_zero_page) 后，检查权限（“写入只读页”（vm_flags & MAP_WRITE），于是内核调用 alloc_huge_page 分配一个新的巨型页（或普通页，取决于配置），并初始化为全零（与巨型零页内容一致）。若分配巨型页失败（如系统大页资源不足），内核会降级为分配普通 4KB 页（拆分映射）。页表更新完成后，内核从异常处理中返回用户态，允许进程重新执行写入操作。此时写入将直接作用于新分配的私有页，而原巨型零页不受影响。当巨型零页因写入操作触发写时复制（COW）后，新分配的私有页不再被称为“巨型零页”，而是一个普通的私有匿名页。然后让进程恢复写入操作，而此时共享巨型零页的进程不受影响。对于写入进程而言它获得了私有页，写入操作仅修改新分配的私有页，确保了进程间内存隔离。

  - 2629 确保后续操作在页锁定保护下进行，避免并发访问导致的数据不一致。如果复合页的 head 页未被锁定，则触发 bug。
  - 2630 确保代码仅对复合页执行相关操作。如果不是复合页，则触发 bug。

- 2632-2633 页内容已提交到 IO 写队列正在写磁盘，此时拆分可能导致数据不一致，因此返回 -EBUSY 错误。确保拆分操作不会与页回写操作并发执行。

- 2644 *anon_vma* 是管理匿名页映射的关键数据结构。page_get_anon_vma() 函数会增加 anon_vma 的引用计数，防止其在拆分过程中被释放

  如果映射到用户虚拟内存区域的匿名页上，则 *page->mapping* 指向其 anon_vma，而非 struct address_space. 且会设置 PAGE_MAPPING_ANON 位以区别。

- 2651 用锁确保同一时间只有一个线程能拆分或合并该匿名页，防止并发操作导致的数据结构损坏。
- 2653 获取文件映射信息，此处 *page->mapping* 指向 address_space 结构，包含文件与页的映射关系
- 2671 计算获取文件末尾位置,end 表示文件大小对应的页数 (向上取整)。拆分时需确保不保留超出文件末尾的页，避免映射到无效数据。
- 2678-2680 检查一个巨页是否可以被安全地拆分为普通页。
- 2683 将指定的页从所有进程的虚拟地址空间中解除映射，确保后续操作 (如释放页) 的安全性。unmap_page() 函数见后。
- 2684 确保传入的页是复合页的头部页。复合页由多个连续物理页组成，只有头部页可用于解除映射操作。

can_split_huge_page() 用于检查一个大页是否满足拆分条件，主要通过比较引用计数来判断。

compact_zone() > *migrate_pages()* > *split_huge_page_to_list()* > *can_split_huge_page()*

```
2584 bool can_split_huge_page(struct page *page, int *pextra_pins)
2585 {
2586     int extra_pins;
2587
2588     /* Additional pins from page cache */
2589     if (PageAnon(page))
2590         extra_pins = PageSwapCache(page) ? thp_nr_pages(page) : 0;
2591     else
2592         extra_pins = thp_nr_pages(page);
2593     if (pextra_pins)
2594         *pextra_pins = extra_pins;
2595     return total_mapcount(page) == page_count(page) - extra_pins - 1;
2596 }
```

- 2589-2592 获取匿名 THP 透明巨页和文件 THP 额外引用计数。

  匿名页若已在交换缓存中 (已建立 swap 分区的映射)，则设置额外引用计数的值 extra_pins 为大页包含的普通单页的数目。否则，额外引用计数为 0。swapcache 巨页的每个 4K 页对应一个 swap 分区上的 swap slot, 拆成基页后相当于每个 slot 持有对应 cache 的一个引用。在复合页时，xArray 里只存头页的指针，拆分后子页也要存入 xArray, 所以要计入子页的引用。

  文件映射页的额外引用计数直接设为大页包含的普通页数目。

  NOTE

  swapcache 的准确含义是：swap 磁盘分区的内容在 RAM 里的 cache(缓存)。
  匿名页 swapout 到磁盘后，仍留在内存 RAM 里的页就是 swapcache。

  同理，磁盘文件在内存 RAM 里的缓存，就是 pagecache。

- 2593-2594 将页的额外引用计数赋值给传出参数，这样上层调用函数可以得到该值。
- 2595 映射计数恰好等于引用计数减去额外引用数再减 1，来判断是否满足拆分条件。

  mapcount 初始值是 -1 表示未被任何进程映射。当用户空间只有唯一的映射者时，拆分大页，内核可安全地将其拆分为小页并重新分配，而不会影响其他进程。而 total_mapcount == 0 意味着用户空间只有唯一的映射。

这里 total_mapcount 不可能是大于 0 的值, 因为此处的调用隐含保证了 total_mapcount == 0。内核规定一个透明大页 (THP) 在系统中只允许被一个进程映射，同一进程对同一大页的多次映射视为同一份映射，不可能出现 THP 的 *total_mapcount* > 0 的情况。而 migrate_pages() 函数调用 split_huge_page_to_list() -> can_split_huge_page() 之前，已经通过 “if (is_thp)” 限定了只处理透明大页。

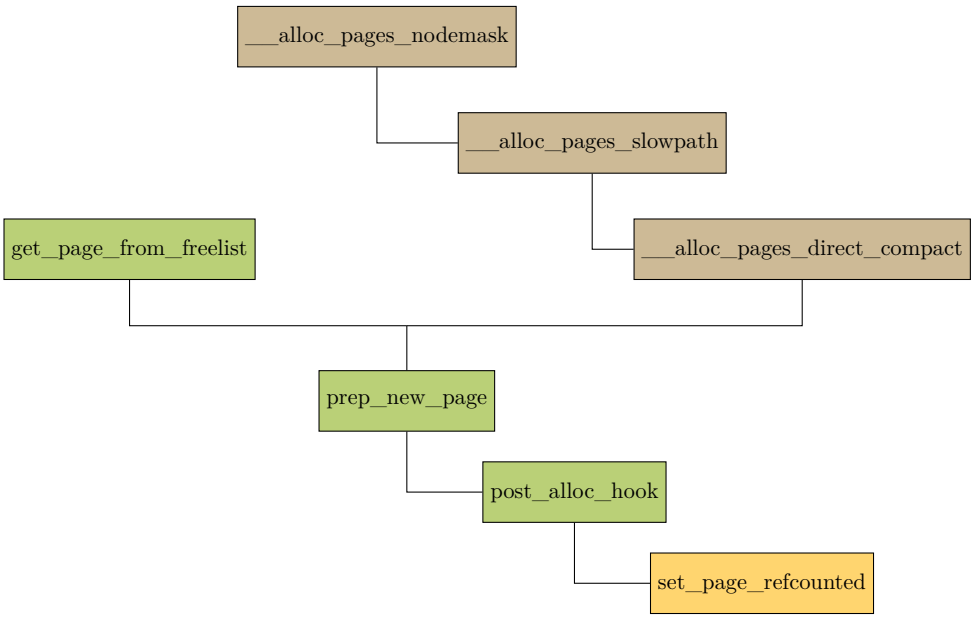
- 系统通过 ->***__refcount*** 管理内核自身对页面的持有 (如页缓存、swap 缓存、内核模块直接引用) 计数。
- 通过 ->***mapcount*** 管理用户空间映射次数。

最终释放条件均需 count == 0 且 mapcount == -1，保证 “内核无引用、用户无映射” 时才回收物理页。

页的引用计数和映射计数变化的场景有以下几处:

- **分配页时**

  引用计数会置成 1，代表内核内部对页的基础引用 (内部持有)。而映射计数会初始化为-1, 表示无映射。分配页设置引用计数的调用链如下:



  其中 set_page_refcounted() 函数用来对->_refcount 置 1。

- **当匿名页被映射到交换分区 *(swap cache)* 时**

  匿名页交换到 swap space 时会获取页面引用, 调用 page_ref_add() 使->_refcount 增加 1, 因为未被用户空间映射，->mapcount 保持 -1。

- **文件页通过 *page cache* 关联磁盘文件时**

  文件页会调用 add_to_page_cache_lru 加入页缓存时增加引用计数。

- **当匿名页通过 *mmap()* 映射到用户空间时**

  page->mapcount 增加 1(每映射一次加 +1)。page->_refcount 不变 (用户空间映射不直接影响引用计数)。

- **从 *swap cache* 移除时**

  调用 delete_from_swap_cache() 减少引用计数

- **用户空间解除映射时**

  每次调用 munmap() 减少 mapcount.

阶段/类型	匿名页	文件页
初始分配后	*count=1*，*mapcount=-1* （内核分配，仅内核基础持有，无用户映射）	*count=1*，*mapcount=-1* （内核分配，未关联页缓存，无用户映射）
进入缓存体系	加入 *Swap Cache* → *count=2*，*mapcount=-1*	加入 *Page Cache* → *count=2*，*mapcount=-1*
用户空间映射 *N* 次	*count=2*，*mapcount=N-1* （每次映射 *mapcount+1*，内核仍通过 *Swap Cache* 持有，*count* 不变）	*count=2*，*mapcount=N-1* （每次映射 *mapcount+1*，内核仍通过 *Page Cache* 持有，*count* 不变）
离开缓存体系	离开 *Swap Cache* → *count=1*，*mapcount=N-1* （*Swap* 缓存释放引用，仅内核基础持有）	离开 *Page Cache* → *count=1*，*mapcount=N-1* （*Page Cache* 释放引用，仅内核基础持有）
用户空间解除映射 *N* 次	*count=1*，*mapcount=-1*（可释放） （所有用户映射解除，*mapcount* 归 *-1*；若后续 *count* 归 *0* 则触发物理内存释放）	*count=1*，*mapcount=-1*（可释放） （所有用户映射解除，*mapcount* 归 *-1*；若后续 *count* 归 *0* 则触发物理内存释放）

尽管匿名页和文件页的计数逻辑不同，但两者最终通过 page_count() 和 total_mapcount() 提供统一的计数查询接口，便于内核通用代码处理。

2595 行的写法是非常诡异和反直觉的，后来 6.x 的内核废弃了这里的写法。从 2590 和 2592 行可以看到，当 extra_pins 不为 0 时，$extra_pins = thp_nr_pages(page)$, 对于 2M 的 THP 来说，也就是 512。

因此对于代码的作者而言，2595 行的本意其实是 -1, 而不是 -512(如下):

$$total_mapcount(page) == page_count(page) - (extra_pins ? 1 : 0) - 1$$

但是作者没有这么写，而是使用了一个 tricks, 直接把 extra_pins 扔进式子里，靠等式右边很小的整数 -512(假设是 2M 的 THP)，通过 unsigned 减法产生溢出得到了用来返回的 false。

当然，如果 $extra_pins = 0$ 的话，此处没有溢出就是正常合理的式子。

回到 split_huge_page_to_list()2683 行的 unmap_page()。

compact_zone() → migrate_pages() → split_huge_page_to_list() → unmap_page()

```
2322 static void unmap_page(struct page *page)
2323 {
2324     enum ttu_flags ttu_flags = TTU_IGNORE_MLOCK | TTU_IGNORE_ACCESS |
2325         TTU_RMAP_LOCKED | TTU_SPLIT_HUGE_PMD;
2326     bool unmap_success;
2327
2328     VM_BUG_ON_PAGE(!PageHead(page), page);
2329
2330     if (PageAnon(page))
2331         ttu_flags |= TTU_SPLIT_FREEZE;
2332
2333     unmap_success = try_to_unmap(page, ttu_flags);
2334     VM_BUG_ON_PAGE(!unmap_success, page);
2335 }
```

- 2324-2325 这几个宏的含义如下:
  - *TTU_IGNORE_MLOCK*: 忽略页是否被 mlock() 锁定，强制解除映射。

  - *TTU_SYNC*: 同步解除映射操作，确保所有 CPU 上的 TLB 刷新完成。
  - *TTU_RMAP_LOCKED*: 调用者已持有反向映射 (rmap) 锁，避免重复获取。
  - *TTU_SPLIT_HUGE_PMD*: 若遇到巨型页表项 (Huge PMD)，自动拆分。
- 2328 确保传入的页是复合页的头部页 (head)。复合页由多个连续物理页组成，只有头部页可用于解除映射操作。
- 2330-2331 针对匿名页 (如堆、栈内存)，在拆分巨型页时冻结页表更新，防止并发修改。确保拆分过程中不会有新的映射建立。
- 2333 执行解除映射操作：对巨型页实现，遍历所有映射了该页的进程页表。从每个进程的页表中删除该页的映射 (置为无效)，自动拆分为普通页表项（因设置了 TTU_SPLIT_HUGE_PMD)。
- 2334 检查页是否仍存在有效映射。若仍有映射，打印一次警告。

回到 split_huge_page_to_list() 函数。

compact_zone() > *migrate_pages()* > *split_huge_page_to_list()*

```
2689    if (mapping) {
2690        XA_STATE(xas, &mapping->i_pages, page_index(head));
2691
2692        /*
2693         * Check if the head page is present in page cache.
2694         * We assume all tail are present too, if head is there.
2695         */
2696        xa_lock(&mapping->i_pages);
2697        if (xas_load(&xas) != head)
2698            goto fail;
2699    }
2700
2701    /* Prevent deferred_split_scan() touching ->_refcount */
2702    spin_lock(&ds_queue->split_queue_lock);
2703    count = page_count(head);
2704    mapcount = total_mapcount(head);
2705    if (!mapcount && page_ref_freeze(head, 1 + extra_pins)) {
2706        if (!list_empty(page_deferred_list(head))) {
2707            ds_queue->split_queue_len--;
2708            list_del(page_deferred_list(head));
2709        }
2710        spin_unlock(&ds_queue->split_queue_lock);
2711        if (mapping) {
2712            if (PageSwapBacked(head))
2713                __dec_node_page_state(head, NR_SHMEM_THPS);
2714            else
2715                __dec_node_page_state(head, NR_FILE_THPS);
2716        }
2717
2718        __split_huge_page(page, list, end, flags);
2719        ret = 0;
2720    } else {
2721        if (IS_ENABLED(CONFIG_DEBUG_VM) && mapcount) {
2722            pr_alert("total_mapcount: %u, page_count(): %u\n",
2723                    mapcount, count);
2724            if (PageTail(page))
2725                dump_page(head, NULL);
2726            dump_page(page, "total_mapcount(head) > 0");
2727            BUG();
2728        }
```

```
2729            spin_unlock(&ds_queue->split_queue_lock);
2730    fail:       if (mapping)
2731                xa_unlock(&mapping->i_pages);
2732            spin_unlock_irqrestore(&pgdata->lru_lock, flags);
2733            remap_page(head, thp_nr_pages(head));
2734            ret = -EBUSY;
2735        }
2736
2737    out_unlock:
2738        if (anon_vma) {
2739            anon_vma_unlock_write(anon_vma);
2740            put_anon_vma(anon_vma);
2741        }
2742        if (mapping)
2743            i_mmap_unlock_read(mapping);
2744    out:
2745        count_vm_event(!ret ? THP_SPLIT_PAGE : THP_SPLIT_PAGE_FAILED);
2746        return ret;
2747    }
```

- 2689-2699 巨型页拆分需要确保其在于页缓存中。若页已被其他线程回收（如内存压力导致的页换出），则拆分操作无意义，甚至会引发错误。

  为什么假设头部页存在则尾部页也存在?

  巨型页的所有组成页在物理上连续，且在页缓存中作为一个整体管理。若头部页存在，说明整个巨型页结构尚未被破坏。

- 2702 现在要操作 head page 的引用计数，必须拿这个锁，避免和后台延迟拆分队列产生竞争。

- 2703 读取 head page 的 *_refcount* 引用计数。

- 2704 该复合页全部的映射计数总和，代表还有多少页表映射指向这个巨页。

- 2705 在调用 page_ref_freeze() 前，内核会确保页已从所有进程的页表中解除映射 (映射计数 = -1)，页的引用计数通常为 1（仅保留分配者的基础引用）。page_ref_freeze() 函数见后。

  映射计数反映页被映射的次数，控制页是否可被换出。引用计数反映页的使用次数，控制页是否可被释放。

- 2706 若页存在于延迟拆分队列中，将其移除。页的拆分任务将同步执行（通过 ___split_huge_page()），而非异步执行（见 ___split_huge_page）

- 2711-2716 减少系统中巨型页的统计计数。

  共享内存巨型页（NR_SHMEM_THPS）：若页属于共享内存（如 shmget）。

  文件映射巨型页（NR_FILE_THPS）：若页属于文件映射（如 mmap 映射文件）。

- 2718 执行实际拆分操作，从延迟队列移除后，页的拆分任务将同步执行（通过 ___split_huge_page()），而非异步执行

- 2710-2735 页无法冻结（page_ref_freeze 返回 false）或页已经不在 page cache 中。进行恢复操作: 解锁各种锁，重新映射页（remap_page), 若拆分失败，确保页仍可被访问。

```
2413 static void __split_huge_page(struct page *page, struct list_head *list,
2414         pgoff_t end, unsigned long flags)
2415 {
2416     struct page *head = compound_head(page);
2417     pg_data_t *pgdat = page_pgdat(head);
2418     struct lruvec *lruvec;
2419     struct address_space *swap_cache = NULL;
2420     unsigned long offset = 0;
2421     unsigned int nr = thp_nr_pages(head);
2422     int i;
2423 
2424     lruvec = mem_cgroup_page_lruvec(head, pgdat);
2425 
2426     /* complete memcg works before add pages to LRU */
2427     mem_cgroup_split_huge_fixup(head);
2428 
2429     if (PageAnon(head) && PageSwapCache(head)) {
2430         swp_entry_t entry = { .val = page_private(head) };
2431 
2432         offset = swp_offset(entry);
2433         swap_cache = swap_address_space(entry);
2434         xa_lock(&swap_cache->i_pages);
2435     }
2436 
2437     for (i = nr - 1; i >= 1; i--) {
2438         __split_huge_page_tail(head, i, lruvec, list);
2439         /* Some pages can be beyond i_size: drop them from page cache */
2440         if (head[i].index >= end) {
2441             ClearPageDirty(head + i);
2442             __delete_from_page_cache(head + i, NULL);
2443             if (IS_ENABLED(CONFIG_SHMEM) && PageSwapBacked(head))
2444                 shmem_uncharge(head->mapping->host, 1);
2445             put_page(head + i);
2446         } else if (!PageAnon(page)) {
2447             __xa_store(&head->mapping->i_pages, head[i].index,
2448                     head + i, 0);
2449         } else if (swap_cache) {
2450             __xa_store(&swap_cache->i_pages, offset + i,
2451                     head + i, 0);
2452         }
2453    }
2454 
2455     ClearPageCompound(head);
2456 
2457     split_page_owner(head, nr);
2458 
2459     /* See comment in __split_huge_page_tail() */
2460     if (PageAnon(head)) {
2461         /* Additional pin to swap cache */
2462         if (PageSwapCache(head)) {
2463             page_ref_add(head, 2);
2464             xa_unlock(&swap_cache->i_pages);
2465         } else {
2466             page_ref_inc(head);
2467         }
2468     } else {
2469         /* Additional pin to page cache */
2470         page_ref_add(head, 2);
2471         xa_unlock(&head->mapping->i_pages);
2472     }
2473 
2474     spin_unlock_irqrestore(&pgdat->lru_lock, flags);
2475 
2476     remap_page(head, nr);
```

```
2477
2478        if (PageSwapCache(head)) {
2479            swp_entry_t entry = { .val = page_private(head) };
2480
2481            split_swap_cluster(entry);
2482        }
2483
2484        for (i = 0; i < nr; i++) {
2485            struct page *subpage = head + i;
2486            if (subpage == page)
2487                continue;
2488            unlock_page(subpage);
2489
2490           /*
2491             * Subpages may be freed if there wasn't any mapping
2492             * like if add_to_swap() is running on a lru page that
2493             * had its mapping zapped. And freeing these pages
2494             * requires taking the lru_lock so we do the put_page
2495             * of the tail pages after the split is complete.
2496             */
2497            put_page(subpage);
2498        }
2499    }
```

- 2424 mem_cgroup_page_lruvec() 见后。
- 2429-2435 这段代码处理的是匿名页并且已被换出到交换空间的巨型页在拆分前的准备工作。其核心目的是锁定交换缓存 (swap cache)，确保拆分过程中页的 swap 状态不会被并发修改。
- 2430 获取页的私有数据字段，对于已换出的页该字段存储 swap entry 项，该项表示页在 swap space 中的位置。
- 2432 从 swap entry 项中提取页在 swap 设备中的偏移量 (以页为单位)。后续拆分时，需为每个子页计算在交换空间 (swap space) 中的新位置，原始偏移量是计算的基础。
- 2433 返回管理目标 swap 项的 address_space 对象。

  address_space 是内核管理文件或交换空间映射的核心结构，包含:

  页树 (i_pages): 存储已换出页的缓存 (XA 树结构)

  操作函数集：定义如何与底层存储交互（如读/写交换空间）。

  匿名页没有关联文件，其换出内容必须存储在交换空间中。拆分为巨型页时，需要为每个子页重新分配 swap entry 项，并更新交换缓存。
- 2437-2453 这段代码处理的是巨型页拆分后的子页管理，包括子页的缓存更新、地址处理以及交换空间映射维护
- 2438 核心拆分函数，将复合页的尾部页从头部页解关联。初始化子页的元数据（如清除复合页标志）。将子页加入指定的 LRU 链表 (通过 lruvec) 或临时链表 (list)。

```
2438 static void __split_huge_page_tail(struct page *head, int tail,
2439         struct lruvec *lruvec, struct list_head *list)
2440 {
2441     struct page *page_tail = head + tail;
2442 
2443     VM_BUG_ON_PAGE(atomic_read(&page_tail->_mapcount) != -1, page_tail);
2444 
2445     /*
2446      * Clone page flags before unfreezing refcount.
2447      *
2448      * After successful get_page_unless_zero() might follow flags change,
2449      * for exmaple lock_page() which set PG_waiters.
2450      */
2451     page_tail->flags &= ~PAGE_FLAGS_CHECK_AT_PREP;
2452     page_tail->flags |= (head->flags &
2453             ((1L << PG_referenced) |
2454              (1L << PG_swapbacked) |
2455              (1L << PG_swapcache) |
2456              (1L << PG_mlocked) |
2457              (1L << PG_uptodate) |
2458              (1L << PG_active) |
2459              (1L << PG_workingset) |
2460              (1L << PG_locked) |
2461              (1L << PG_unevictable) |
2462 #ifdef CONFIG_64BIT
2463              (1L << PG_arch_2) |
2464 #endif
2465              (1L << PG_dirty)));
2466 
2467     /* ->mapping in first tail page is compound_mapcount */
2468     VM_BUG_ON_PAGE(tail > 2 && page_tail->mapping != TAIL_MAPPING,
2469             page_tail);
2470     page_tail->mapping = head->mapping;
2471     page_tail->index = head->index + tail;
2472 
2473     /* Page flags must be visible before we make the page non-compound. */
2474     smp_wmb();
2475 
2476     /*
2477      * Clear PageTail before unfreezing page refcount.
2478      *
2479      * After successful get_page_unless_zero() might follow put_page()
2480      * which needs correct compound_head().
2481      */
2482     clear_compound_head(page_tail);
2483 
2484     /* Finally unfreeze refcount. Additional reference from page cache. */
2485     page_ref_unfreeze(page_tail, 1 + (!PageAnon(head) ||
2486                       PageSwapCache(head)));
2487 
2488     if (page_is_young(head))
2489         set_page_young(page_tail);
2490     if (page_is_idle(head))
2491         set_page_idle(page_tail);
2492 
2493     page_cpupid_xchg_last(page_tail, page_cpupid_last(head));
2494 
2495     /*
2496      * always add to the tail because some iterators expect new
2497      * pages to show after the currently processed elements - e.g.
2498      * migrate_pages
2499      */
2500     lru_add_page_tail(head, page_tail, lruvec, list);
2501 }
```

- 2441 计算巨型页中第 tail 个普通页的地址。巨型页由多个连续的普通页组成，head 是巨型页的起始地址，page_tail 指向将拆分的某个尾页
- 2443 检查尾部页面的映射计数（mapcount）是否为 -1，若不为 -1，触发内核 BUG。

  巨型页拆分的核心条件是：整个巨型页只能被一个进程映射（即头部页的.mapcount == 0，表示被映射一次）

  巨型页的映射状态由头部页统一管理，尾部页的->mapcount 恒为 -1

  头部页（head page）：管理整个巨型页的元数据，包括映射计数（mapcount）、引用计数（count）等。

  尾部页（tail pages）：仅存储数据，共享头部页的元数据。尾部页的.mapcount 固定为 -1，表示“不单独维护映射计数”。
- 2451-2465 将巨型页头部的 flags 标志位复制到尾部页面。

  PAGE_FLAGS_CHECK_AT_PREP：清除需要特殊检查的标志位。

  复制的标志位包括：引用状态（PG_referenced）、交换状态（PG_swapbacked）、缓存状态（PG_swapcache）等。这些标志反映了页面的使用状态和属性。
- 2468 TAIL_MAPPING 是一个预定义的特殊指针，用于标记复合页的尾部页 (tail)。

  内核通过 mapping 字段区分头部页和尾部页。对于巨型页的 > 2 的尾部页，mapping 通常被设置为 TAIL_MAPPING，直到拆分时才会被更新为实际映射。

  在从伙伴系统进行内存分配时，有一个 GFP_COMP 分配掩码，该 flag 表示从伙伴系统分配的连续页帧为一个复合页。复合页就是将多个页帧进行组合，视作一个更大 size 的页。所有尾部页 page 的 first_page 字段都存放指向 head 页的指针。

  第一个尾页成员 struct.compound_mapcount 表示复合页的映射计数，即多少个虚拟页映射到这个物理页，初始值是 - 1。这个成员和成员 mapping 组成一个联合体，占用相同的位置。

  第二个尾页为巨型页引入独立的 page_pinned_refcount 字段，专门记录被 pin 的次数，避免头部页的->_refcount 字段基础计数与 pin 计数叠加导致溢出。在巨型页压力测试中，未使用 page_pinned_refcount 时出现频繁的引用计数溢出。该字段被用于对页面进行“固定”操作 (如避免被换出、迁移等场景)，它是一个固定的数值（通常为 1024，即 1 << 10)。在用户态内存操作相关场景中，如调用 PIN_COUNTING_BIAS 作为一个偏移值，是指在 Linux 内核内存管理中，它是一个固定的数值 (通常为 1024，即 1 << 10)。在用户态内存操作相关场景中，如调用 get_user_pages () 等函数来 pin 住物理页面的操作，会将页面的引用计数增加（如 PIN_COUNTING_BIAS），这使得被 pin 住的页面的引用计数比正常情况高出这个固定的数值。从而当普通页面在引用计数上区分开来，这样内核在进行页面管理（如页面迁移、释放等操作）时，通过判断引用计数是否包含这个偏移值，具体来说，当使用 put_page 操作，从而与普通页的 put 操作上区分开来，这样内核在进行页面管理（如页面迁移、释放等操作）时，通过判断引用计数是否包含这个偏移值，就能知道该页面是否处于被 pin 住的状态，进而防止该页面被用户态 DMA 等操作过程中被意外处理，保证相关操作能够正确访问内存。而在解除 pin 操作时，会将引用计数减去该偏移值，从而恢复正常的引用计数。该偏移值的设计，虽然解决了页面在被 pin 时的计数问题，但也带来了一个潜在的问题：当页面被频繁 pin 和 unpin 时，引用计数可能会因为多次叠加和减去这个偏移值而出现溢出的风险。

  为了解决这个问题，引入了 page_pinned_refcount 字段，专门用于记录页面被 pin 的次数，这样就将 pin 操作相关的计数从页面的基础引用计数（->_refcount）中分离出来，避免了基础引用计数的溢出。

现在，当对页面进行 pin 操作时，会增加 page_pinned_refcount 的值；当解除 pin 操作时，会减少该值。而页面的基础引用计数（->_refcount）则主要用于跟踪页面的正常引用情况（如被进程的页表映射等），这样就有效地降低了引用计数溢出的可能性，提升了内核内存管理的稳定性和可靠性。

这里的流程示意如下:

- 将尾部页的 mapping 设置为一个污化的地址。(====== 原页布局照片待添加)
- page_tail->mapping = head->mapping;
- 验证尾部页状态（mapping == TAIL_MAPPING）。
- 将所有尾部页的 mapping 更新为头部页的映射（head->mapping）。
- 将每个尾部页的 index 设置为连续值（head->index + tail）

compact_zone() ··· split_huge_page_to_list() __split_huge_page() mem_cgroup_page_lruvec()

```
1376 struct lruvec *mem_cgroup_page_lruvec(struct page *page, struct pglist_data *pgdat)
1377 {
1378     struct mem_cgroup_per_node *mz;
1379     struct mem_cgroup *memcg;
1380     struct lruvec *lruvec;
1381
1382     if (mem_cgroup_disabled()) {
1383         lruvec = &pgdat->__lruvec;
1384         goto out;
1385     }
1386
1387     memcg = page->mem_cgroup;
1388     /*
1389      * Swapcache readahead pages are added to the LRU - and
1390      * possibly migrated - before they are charged.
1391      */
1392     if (!memcg)
1393         memcg = root_mem_cgroup;
1394
1395     mz = mem_cgroup_page_nodeinfo(memcg, page);
1396     lruvec = &mz->lruvec;
1397 out:
1398     /*
1399      * Since a node can be onlined after the mem_cgroup was created,
1400      * we have to be prepared to initialize lruvec->zone here;
1401      * and if offlined then reonlined, we need to reinitialize it.
1402      */
1403     if (unlikely(lruvec->pgdat != pgdat))
1404         lruvec->pgdat = pgdat;
1405     return lruvec;
1406 }
```

- 1382 当 memcg 启用时，内核会为每个 memcg 维护独立的 LRU 链表 (活跃/非活跃页)。当 memcg 禁用时，系统使用全局 LRU 链表 (属于 pg_data)。
- 1383 当前 NUMA 节点（pg_data_t）的全局 LRU 向量。当 memcg 禁用时，所有页的 LRU 管理都通过这个全局结构进行。

- 1387 获取页关联的 memcg（若有）。页被换出时（如换入到内存的页），会被加入到 LRU 链表中，可能在此过程中被迁移。这些预读页被添加到 LRU 或迁移时，尚未加入到内存控制组（mem_cgroup）来管理（charge）。对于没有 MemCG（root_mem_cgroup）管理的页，默认使用根 MemCG（root_mem_cgroup）来进行资源管控。

  预读的典型场景: 页面换入 (page in) 时的预读取，若检测到连续的页面换入请求 (如顺序访问大数组)，内核会预读取相邻的页面。

  预读取的触发条件:

  当进程访问一个已被换出的页面时，触发页 page fault，内核调用 do_swap_page() 处理换入。页面不在 swapcache 中，从 swapcache 读取并可能调用 swapin_readahead 预读取相邻页面。预读取的页面被分配并添加到 LRU 链表，但此时尚未进行 mem_cgroup 纳入内存控制组的资源管控范围。调用 add_to_swap() 将页面添加到 swapcache，调用 swap_writepage() 将页面写入 swapcache

- 1395 返回 mem_cgroup 在页面所在的特定内存 node 节点上的信息结构体 mem_cgroup_node。
- 1396 获取目标 cmem group 的 lru 向量。lru 向量用于管理不同内存 zone 区域的 lru 链表。
- 1403-1404 确保 lruvec 的 pgdat 始终指向当前有效的 node 节点，避免因节点状态变化导致内存管理错误 (如 lru 链表归属错误)。

  NUMA 架构中，node 节点可能在 mem_cgroup 创建后才被动态激活 online, 或经历 offline(离线) 后重新 online。这会导致 mem_cgroup 关联的 lruvec 的 pgdat 失效，需重新初始化。

compact_zone() › *migrate_pages()* › *split_huge_page_to_list()* › *page_ref_freeze()*

```
173 static inline int page_ref_freeze(struct page *page, int count)
174 {
175     int ret = likely(atomic_cmpxchg(&page->_refcount, count, 0) == count);
176
177     if (page_ref_tracepoint_active(page_ref_freeze))
178         __page_ref_freeze(page, count, ret);
179     return ret;
180 }
```

- 175 这行代码是冻结操作的核心。atomic_cmpxchg() 宏比较 page->_refcount 的当前值与 count，若相等，则将 _refcount 设置为 0，并返回旧值（即 count）。若不相等，不做任何修改，直接返回当前值。

  对页的冻结的实现：

  当 atomic_cmpxchg() 成功 (返回值等于 count) 时, ->_refcount 被设置为 0，此时页会被冻结，在内核中，_refcount 为 0 或负数时表示页被 “冻结”，不会被回收。冻结状态会阻止页被释放，直到解冻结操作恢复其引用计数。引用计数为正数表示页处于活跃状态。。

  容易引起混淆的是，页的释放发生在以下条件满足时：引用计数降为 0。通过 put_page() 或类似函数减少引用计数，当结果为 0 时触发页的释放。这是一个动态判断的过程。若直接置 0，则不满足这个动态判断的过程，不会触发页的释放，仅表示冻结页。未被冻结页的 _refcount 必须为正数（或特殊状态 PAGE_FLAGS_ACCOUNT 未被设置）。

6.3 end_page_writeback()

end_page_writeback()

```
1473 void end_page_writeback(struct page *page)
1474 {
1475     /*
1476      * TestClearPageReclaim could be used here but it is an atomic
1477      * operation and overkill in this particular case. Failing to
1478      * shuffle a page marked for immediate reclaim is too mild to
1479      * justify taking an atomic operation penalty at the end of
1480      * ever page writeback.
1481      */
1482     if (PageReclaim(page)) {
1483         ClearPageReclaim(page);
1484         rotate_reclaimable_page(page);
1485     }
1486 
1487     /*
1488      * Writeback does not hold a page reference of its own, relying
1489      * on truncation to wait for the clearing of PG_writeback.
1490      * But here we must make sure that the page is not freed and
1491      * reused before the wake_up_page().
1492      */
1493     get_page(page);
1494     if (!test_clear_page_writeback(page))
1495         BUG();
1496 
1497     smp_mb__after_atomic();
1498     wake_up_page(page, PG_writeback);
1499     put_page(page);
1500 }
1501 EXPORT_SYMBOL(end_page_writeback);
```

- 1482 检查目标页面是否被标记” 尽快回收”，这个标记是回收流程 (shrink_page_list) 提前打上的.
- 1483-1484 因为脏页回写完成才会调用当前函数，所以如果检测到 *PG_reclaim* 标记，此处清除该” 尽快回收” 标记, 并把目标页放到欠活跃 (inactive)lru 链表尾部，以便下次扫描就回收。

  这里本可以用原子操作，但原子操作开销太重了，没必要。就算偶尔没把**要回收**的页面放到回收链表，影响也很小，不值得在每次写回结束都付出原子操作的代价。
- 1493 增加页的引用计数，确保 *struct page* 和物理页面在接下来的操作中不会被释放。

  因为在之后第 1498 行会唤醒等待目标页面写回完成的进程 (譬如，直接内存回收，fsync /msync 等进程)，这些进程可能会执行页面引用计数减一的操作，导致页面被释放。此处引用计数 +1，后面退出之前 1499 行再-1，可以保证对此页操作的完整执行。

  在函数入口到 *get_page()* 增加计数之前，依靠 *PG_writeback* 而不是 *refconut* 计数来保证页面有效。只要 *PG_writeback* 置位，其他譬如 truncate/invalid 的线程就会 *wait_on_page_writeback()* 阻塞，不会把页面摘除释放 *struct page*
- 1494 原子清除 *PG_writeback* 标志, 表示写回结束。

  如果页面没有置写回标记 (*PG_writeback*)，但是却执行到此处 *end_page_writeback()* 的代码，说明内核出错，直接触发 crash。
- 1497 内存屏障, 保证所有 CPU core 能立刻看到 *PG_writeback* 标记已清除。防止之后的执行语句因为 cpu 指令的乱序执行提前运行，从而看到了 *PG_writeback* 未被清除。
- 1498 唤醒所有等待这个页面写回完成而阻塞休眠的线程。

导致线程休眠等待页面回写 (writeback) 结束的函数是 wait_on_page_writeback(), 它的内部实现是 wait_on_page_bit(page, PG_writeback)

NOTE

和 wait_on_page_writeback() 类似，等待页面解锁的函数是 wait_on_page_locked()。它的内部实现同样是 wait_on_page_bit(...,PG_locked), 只不过参数是 PG_locked。

- 1499 见 1493 行说明。

7

脏页限流

脏页限流只针对 file-backed 文件页。

在执行脏页回写的过程中，必须严格控制回写请求量级，避免请求量过大致使系统运行失稳。

另外，若脏页数量堆积过多，页缓存 (pagecache) 与磁盘实际存储的数据状态出现严重不一致，倘若不能及时通过回写操作抹平差异，会大幅提升数据损坏丢失的风险。

因此，当脏页数量累积达到一定规模后，内核必须采取措施进行干预。

内核主要依靠两组可调参数实现流量管控：

- *vm.dirty_background_ratio*(等效配置字节：vm.dirty_background_bytes)
- *vm.dirty_ratio*(等效配置字节：vm.dirty_bytes)。

这两组参数划定了系统脏页堆积的上限阈值，内核会在脏页达到对应阈值后执行对应管控动作：

后台脏页阈值 vm.dirty_background_ratio(或字节方式 vm.dirty_background_bytes) 用于定义触发脏页回写的脏页水位。当脏页 > 脏页后台阈值 (vm.dirty_background_bytes) 内核会启动后台回写，该参数通常默认值为可用脏内存总量的 10%。

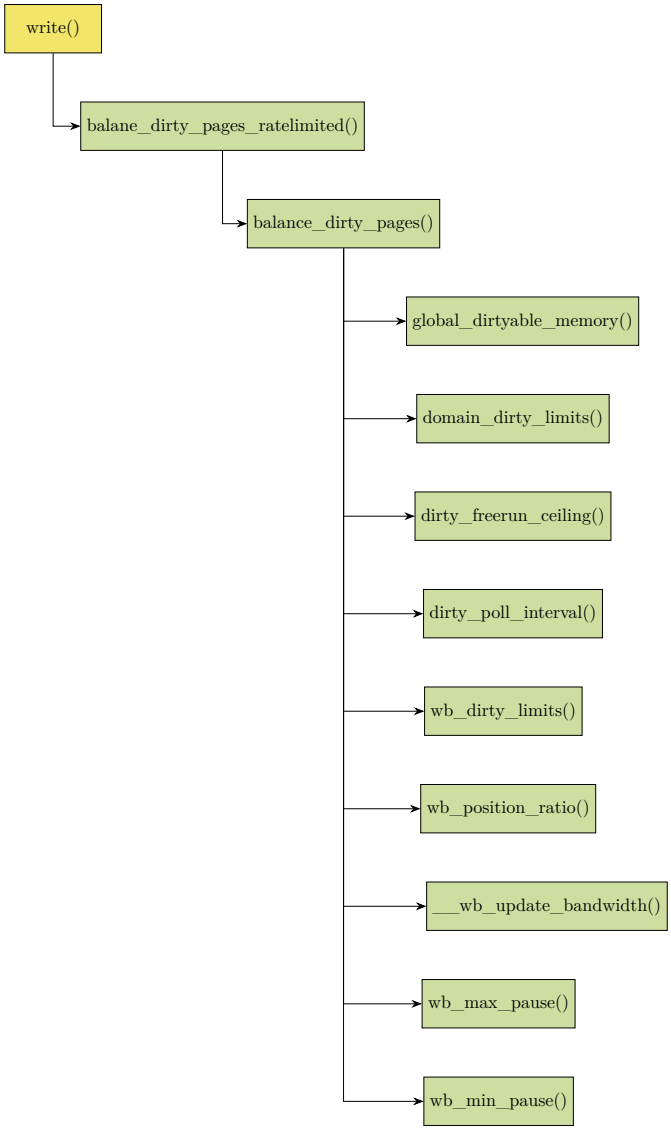
前台脏页硬阈值 vm.dirty_ratio(或配置字节:vm.dirty_bytes) 是硬性阻塞上限。一旦脏页占比触及该值 (或对应字节数)，所有向块设备写入数据、产生脏页的进程都会被阻塞休眠，直至回写进程把脏页占比压回阈值以下才能恢复运行。该参数默认值为可用脏内存总量的 20%。

这两套阈值规则并非完整的管控逻辑。在后台阈值与前台硬阈值之间的区间内，内核还会依据单个块设备 (单磁盘) 的实际 IO 带宽占用情况，对写脏页缓存 (pagecache) 行为进行速率限流，主动规避全局脏页量到达前台硬阈值的极端风险。

7.1 balance_dirty_pages_ratelimited()

仅当后台回写无法跟上脏页生成速度时，才对进程进行限流。换言之，不轻易把写进程卡住休眠，先放任脏页增长；只有发现后台回写已经追不上脏页堆积了，才限流阻塞进程。

应用程序写文件会调用 write() 系统调用，陷入内核后会调用 balance_dirty_pages_ratelimited() 函数。该函数是内核用来限流 + 触发脏页平衡的入口函数，防止脏页平衡操作被频繁调用导致性能开销。



7.2 进程脏页阈值和 per-cpu 的脏页阈值

整个调用的层次如上，下面从入口函数看起。

balance_dirty_pages_ratelimited()

```
1877 void balance_dirty_pages_ratelimited(struct address_space *mapping)
1878 {
1879     struct inode *inode = mapping->host;
1880     struct backing_dev_info *bdi = inode_to_bdi(inode);
1881     struct bdi_writeback *wb = NULL;
1882     int ratelimit;
1883     int *p;
1884
1885     if (!(bdi->capabilities & BDI_CAP_WRITEBACK))
1886         return;
1887
```

```
1888     if (inode_cgwb_enabled(inode))
1889         wb = wb_get_create_current(bdi, GFP_KERNEL);
1890     if (!wb)
1891         wb = &bdi->wb;
1892 
1893     ratelimit = current->nr_dirtied_pause;
1894     if (wb->dirty_exceeded)
1895         ratelimit = min(ratelimit, 32 >> (PAGE_SHIFT - 10));
1896 
1897     preempt_disable();
1898     /*
1899      * This prevents one CPU to accumulate too many dirtied pages without
1900      * calling into balance_dirty_pages(), which can happen when there are
1901      * 1000+ tasks, all of them start dirtying pages at exactly the same
1902      * time, hence all honoured too large initial task->nr_dirtied_pause.
1903      */
1904     p =  this_cpu_ptr(&bdp_ratelimits);
1905     if (unlikely(current->nr_dirtied >= ratelimit))
1906         *p = 0;
1907     else if (unlikely(*p >= ratelimit_pages)) {
1908         *p = 0;
1909         ratelimit = 0;
1910     }
1911     /*
1912      * Pick up the dirtied pages by the exited tasks. This avoids lots of
1913      * short-lived tasks (eg. gcc invocations in a kernel build) escaping
1914      * the dirty throttling and livelock other long-run dirtiers.
1915      */
1916     p = this_cpu_ptr(&dirty_throttle_leaks);
1917     if (*p > 0 && current->nr_dirtied < ratelimit) {
1918         unsigned long nr_pages_dirtied;
1919         nr_pages_dirtied = min(*p, ratelimit - current->nr_dirtied);
1920         *p -= nr_pages_dirtied;
1921         current->nr_dirtied += nr_pages_dirtied;
1922     }
1923     preempt_enable();
1924 
1925     if (unlikely(current->nr_dirtied >= ratelimit))
1926         balance_dirty_pages(wb, current->nr_dirtied);
1927 
1928     wb_put(wb);
1929 }
1930 EXPORT_SYMBOL(balance_dirty_pages_ratelimited);
```

- 1879 从地址空间 mapping 获取对应的索引节点 inode。inode 可用于找到文件对应的目标磁盘的相关信息。

  一个文件本体 (元数据 + 存储) 对应一个 inode。一个 inode 不可能同时代表两份不同的数据。

  下面是硬链接和软链接场景下容易混淆的地方:

  - **硬链接** *ln a b*

    不创建新 inode, b 的 dentry(目录项) 直接指向 a 的 inode。a 和 b 是同一个文件本体。

  - **软链接** *ln -s a c*

    此时会创建全新的 c 的 inode,c 的 inode 里存放的是 a 的目标路径字符串。

- 1880 从 inode 获取磁盘设备的回写信息结构 bdi。bdi 管理这个磁盘的脏页回写策略 (inode_to_bdi() 函数的解析见后面)。

- 1881 定义回写上下文 wb，用于后续脏页控制。

- 1885-1886 检查设备是否支持脏页回写。不支持脏页回写的设备 (或文件系统) 直接退出，不做脏页平衡。

  ramfs 直接把页缓存作为文件系统，没有任何后端存储 (磁盘/闪存)。传统 /dev/ramX(块设备级 RAM disk) 用内存块模拟块设备，写入即完成，不存在“延迟刷回”的概念。这些设备的 BDI_CAP_WRITEBACK=0。

  需要注意是，tmpfs 支持脏页回写。它是内存 + swap 后端, 有回写/换出逻辑。它的 pagecache 缓存页会被标记为脏，并参与全局脏页统计。内存紧张时可换出到 swap 分区。而对于 ramfs, 通常用于嵌入式系统启动前的极简环境 (此时 swap 尚未启用)。

特性	*ramfs*	*tmpfs*	*/dev/ramX(传统 RAM disk)*
底层实现	直接绑定页缓存 *(page cache)*	直接绑定页缓存 *(page cache)*	模拟块设备，使用内存块作为存储介质
是否支持 *writeback*	不支持	支持，脏页可换出到 *swap*	不支持 *(无需支持，本身就是模拟的后备存储设备)*
是否参与脏页统计	不参与 *(无写回机制)*	参与	不参与 *(无写回概念)*
是否需要格式化	无需	无需	需要 *(如 ext2/ext4)*
性能	最高	极高	较低 *(多层开销)*
适用场景	*initramfs*、内核启动阶段	*/tmp*、*/run*、临时缓存	仅用于历史兼容或特殊嵌入式场景

- 1888-1889 在开启了 cgroup 的情况下，获取当前 cgroup 的 wb。不同 cgroup 脏页隔离，互不干扰。

  wb 是回写上下文对象，负责把脏页刷到磁盘。wb->dwork 是工作队列，交给全局 bdi_wq 执行回调 wb_workfn(), 就是 flusher 线程逻辑。

- 1890-1891 没创建 cgroup 回写就用设备默认 wb。

- 1893 获取进程在普通 write 写路径下被允许产生脏页的上限配额 (->nr_dirtied_pause)。这是一个进程在触发下一次脏页平衡检测 (*balance_dirty_pages()*)，允许弄脏的最大页面数阈值。换言之，也就是该进程脏页超过此限制值将被” 暂停”。

  进程每弄脏一个页面，会增加自己的计数器 (current->nr_dirtied)。当 current->nr_dirtied 超过阈值 (也就是 nr_dirtied_pause) 时，就意味着进程脏页超限了，会调用脏页平衡逻辑 (*blance_dirty_pages()*) 可能会休眠一会儿。*balance_dirty_pages()* 会计算该进程需要休眠多久，随后让进程休眠。并且在休眠完成准备返回时，调用 *dirty_poll_interval()* 计算下一次的配额 (->nr_dirtied_pause), 同时清 0->*nr_dirtied*。

  需要注意的是：无论是进程的脏页阈值还是 per-cpu 的脏页阈值都不会决定是否回写暂停 (休眠)，它们只是决定了是否调用 *balance_dirty_pages()* 的时机。而 *balance_dirty_page()* 内部会通过判断回写域的 *bg_thresh*、*thresh* 水位来决策是否休眠。

  nr_dirtied $\geq$ *nr_dirtied_pause* 的检测是在 write 写文件弄脏页，调用本函数 (*balance_dirty_pages_ratelimited()*)，在本函数内部检测判断。

- 1894-1895 如果设备脏页已经超标，就强制把限流阈值强制降到极小值 (32 Kbytes)，加快触发回写确保系统快速恢复可控状态。这里不是全局脏页压力，而是单个设备维度的脏页压力过载。

  1895 行的常量 32 的单位是 KB(千字节)。此处是保证阈值极小值的上限为 32K 字节。

假设此处 PAGE_SHIFT 为 12, 那么

$$PAGE_SHIFT - 10$$

表示在目标平台上 1 页的大小是

$$2^{PAGE_SHIFT-10}(KB)$$

那么，

$$32 >> (PAGE_SHIFT - 10)$$

也就是,

$$32(KB)/(2^{PAGE_SHIFT-10})(KB)$$

即,

$$\frac{32}{2^{PAGE_SHIFT-10}}$$

得到的结果表示阈值是多少页。

程序中 $32 >> (PAGE_SHIFT - 10)$ 是跨架构兼容的写法。不同硬件平台内核支持不同页大小，此处用位移计算，无论得到的页数目是何值，大小永远保证是 32K 字节。

- 1904 获取当前 CPU 的脏页计数值。每个 CPU 独立计数。
- 1905-1906 如果当前进程脏页已达阈值，清空计数值。准备强制立刻触发脏页平衡检测操作 (注意: 只决定是否检测，即是否调用 *balance_dirty_pages()*)。
- 1907-1909 如果当前 CPU 累计脏页太多，则也准备强制触发平衡操作。此举是为了防止大量进程一起写造成瞬间脏页爆炸。这一机制可以避免单个 CPU 堆积过多脏页却不触发脏页平衡校验。特别是当上千个进程同时开始写入脏页，并且各进程初始脏页暂停阈值偏大时，容易出现这类问题。因此设定上限，无论多少进程，只要总脏页数达到目标 cpu 的总脏页阈值均必须触发脏页限流平衡的决策。

  若该情况发生，则将**限流值**设为 0，这意味着后续 1925 行 ->nr_dirtied 与**限流值**的比较将一定触发脏页平衡检测。

  当前 cpu 的脏页阈值 *ratelimit_pages* 也是动态计算出的。

变量	作用	更新点
->nr_dirtied_pause	进程 *task* 粒度脏页阈值。	*balance_dirty_pages()* 退出前调用 *dirty_poll_interval()* 算出，每个进程独立
ratelimit_pages	*per-cpu* 的脏页阈值。	*sysctl* 脏页参数修改、内存热插拔、内核初始化 *(待确认)*

- 1916-1922 把已经退出的进程遗留的脏页算到当前进程上。

  正常长生命期进程会一直写文件,脏页不断累积达到阈值后触发”脏页平衡”操作,从而”写脏页”动作被限流等待回写。而短生命周期的进程只产生一点脏页，还没累积到阈值 (struct task_struct->*nr_dirtied_pause*),进程就结束直接退出,进程一结束,计数就丢了。这就叫:脏页泄漏(dirty throttle leaks)。但成千上万个这种进程一起写会造成瞬间爆脏页，长进程被卡死。因此不能忽略脏页泄漏的影响。

所以进程通过 do_exit() 退出时，会把它的脏页数加到 per-CPU 的 ***dirty_throttle_leaks*** 这个" 桶" 里 (这个桶是 per-cpu 的)。内核接着把这些漏网脏页强行加到当前正在运行的进程头上，从而避免短生命周期进程产生的脏页逃脱限流。

- 1925-1926 达到脏页平衡处理阈值时触发脏页平衡操作 (源码见下)。

  这里的 ratelimit 就是 1893 行的-*>nr_dirtied_pause*, 该值是判断是否接下来调用脏页平衡检测的一个阈值。

  该函数会定期检查系统的脏页状态，如果需要，就会发起磁盘回写。但是，在大型机器 (多进程并发写) 上，获取回写状态的成本很高，所以在脏页不超限时要避免频繁调用 (减小检测频率)。但是，一旦超过脏页限制，又需要加大检查频率，避免每个进程都超出限制很多。

  通俗一点就是: 未超脏页阈值时刻意减少调用 *balance_dirty_pages()*，即减少高频检测，降低性能开销; 超出阈值：大幅限流、拉高检查频次，遏制单个进程过量生成脏页。这个算法的实现在对进程的 task_struct->nr_dirtied_pause 的赋值上，而这个值会在 dirty_poll_interval() 中计算获取 (见该函数的解析)。dirty_poll_interval() 中实现成若脏页 < 脏页上限阈值，则 $\sqrt{thresh - dirted}$ 作为间隔。否则脏页 > 脏页上限阈值时，则以 1(页) 作为间隔.

balance_dirty_pages_ratelimited() > *inode_to_bdi()*

```
136 static inline struct backing_dev_info *inode_to_bdi(struct inode *inode)
137 {
138     struct super_block *sb;
139
140     if (!inode)
141         return &noop_backing_dev_info;
142
143     sb = inode->i_sb;
144 #ifdef CONFIG_BLOCK
145     if (sb_is_blkdev_sb(sb))
146         return I_BDEV(inode)->bd_bdi;
147 #endif
148     return sb->s_bdi;
149 }
```

- 138 临时变量保存目标 inode 所属的超级块 super_block 指针。
- 140-141 如果传入 NULL 的 inode, 就返回不会产生 oops 的空操作对象 bdi。
- 143 获取 inode 所属的超级块指针。

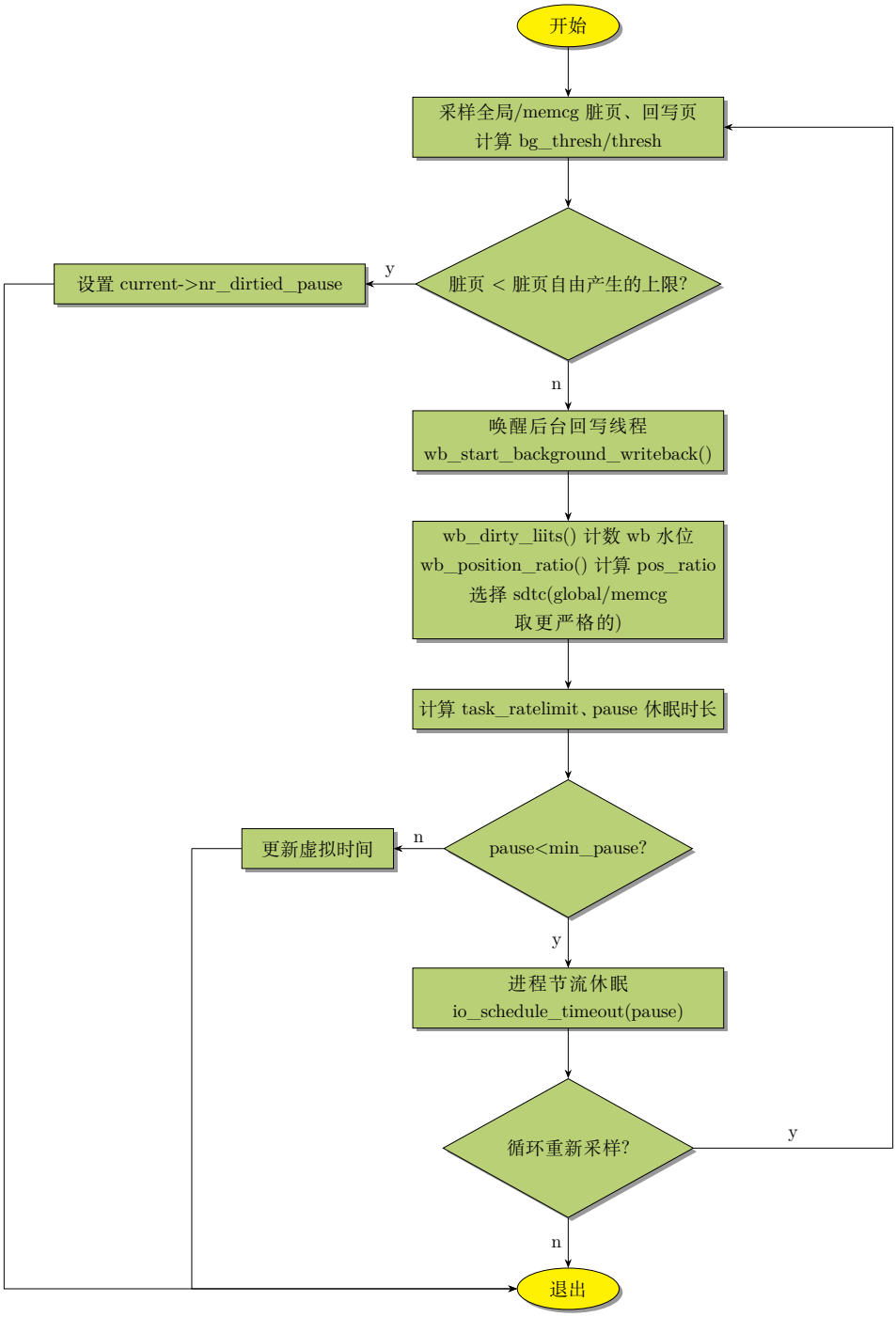
  对于普通文件系统 (ext4，nfs 等)，每执行一次 mount, 内核就分配一个全新 struct super_block。每个挂载示例拥有自己独立的 super_block。该挂载下面所有 inode 的-*>i_sb* 都指向这同一个 super_block。

  对于裸 (块) 设备 inode(*/dev/sda,/dev/nvme0n1* 等)，所有裸设备 inode 全部共用同一个 bdev_sb伪超级块, 也就是*/dev/sda,/dev/sdb,/dev/nvme0n1* 的 *inode->i_sb* = &*bdev_sb*。
- 145 这里判断是不是裸设备的伪超级块。
- 146 所有裸 (块) 设备 inode 共享一个伪超级块，但是每个硬件设备有独立的 *struct block_device* 实例。

从裸块设备的 inode 里取出关联的 *struct block_device *dev*, 然后返回 bdev->bd_bdi, 拿到该磁盘真正的 bdi。

7.2.1 balance_dirty_pages()

让写脏页 (page cache) 的进程休眠一会 (脏页限流) 的核心操作是在 balance_dirty_pages() 中进行的。



该函数中会无限循环检查脏页水位，脏页不降到上限安全阈值以下，就反复休眠，唤醒检查。直到脏页降到安全值后才放行进程 (主要流程见上图，图中省略了一些 break 见下面详细解析)。

balance_dirty_pages_ratelimited() → *balance_dirty_pages()*

```
1339 static void balance_dirty_pages(struct bdi_writeback *wb,
1340                 unsigned long pages_dirtied)
1341 {
1342     struct dirty_throttle_control gdtc_stor = { GDTC_INIT(wb) };
1343     struct dirty_throttle_control mdtc_stor = { MDTC_INIT(wb, &gdtc_stor) };
1344     struct dirty_throttle_control * const gdtc = &gdtc_stor;
1345     struct dirty_throttle_control * const mdtc = mdtc_valid(&mdtc_stor) ?
1346                             &mdtc_stor : NULL;
1347     struct dirty_throttle_control *sdtc;
1348     unsigned long nr_reclaimable;   /* = file_dirty */
1349     long period;
1350     long pause;
1351     long max_pause;
1352     long min_pause;
1353     int nr_dirtied_pause;
1354     bool dirty_exceeded = false;
1355     unsigned long task_ratelimit;
1356     unsigned long dirty_ratelimit;
1357     struct backing_dev_info *bdi = wb->bdi;
1358     bool strictlimit = bdi->capabilities & BDI_CAP_STRICTLIMIT;
1359     unsigned long start_time = jiffies;
1360
1361     for (;;) {
1362         unsigned long now = jiffies;
1363         unsigned long dirty, thresh, bg_thresh;
1364         unsigned long m_dirty = 0;  /* stop bogus uninit warnings */
1365         unsigned long m_thresh = 0;
1366         unsigned long m_bg_thresh = 0;
1367
1368         nr_reclaimable = global_node_page_state(NR_FILE_DIRTY);
1369         gdtc->avail = global_dirtyable_memory();
1370         gdtc->dirty = nr_reclaimable + global_node_page_state(NR_WRITEBACK);
1371
1372         domain_dirty_limits(gdtc);
1373
1374         if (unlikely(strictlimit)) {
1375             wb_dirty_limits(gdtc);
1376
1377             dirty = gdtc->wb_dirty;
1378             thresh = gdtc->wb_thresh;
1379             bg_thresh = gdtc->wb_bg_thresh;
1380         } else {
1381             dirty = gdtc->dirty;
1382             thresh = gdtc->thresh;
1383             bg_thresh = gdtc->bg_thresh;
1384         }
1385
1386         if (mdtc) {
1387             unsigned long filepages, headroom, writeback;
1388
1389             /*
1390              * If @wb belongs to !root memcg, repeat the same
1391              * basic calculations for the memcg domain.
1392              */
1393             mem_cgroup_wb_stats(wb, &filepages, &headroom,
1394                         &mdtc->dirty, &writeback);
1395             mdtc->dirty += writeback;
1396             mdtc_calc_avail(mdtc, filepages, headroom);
1397
1398             domain_dirty_limits(mdtc);
```

```
1399
1400            if (unlikely(strictlimit)) {
1401                wb_dirty_limits(mdtc);
1402                m_dirty = mdtc->wb_dirty;
1403                m_thresh = mdtc->wb_thresh;
1404                m_bg_thresh = mdtc->wb_bg_thresh;
1405            } else {
1406                m_dirty = mdtc->dirty;
1407                m_thresh = mdtc->thresh;
1408                m_bg_thresh = mdtc->bg_thresh;
1409            }
1410        }
1411
1412        /*
1413         * Throttle it only when the background writeback cannot
1414         * catch-up. This avoids (excessively) small writeouts
1415         * when the wb limits are ramping up in case of !strictlimit.
1416         *
1417         * In strictlimit case make decision based on the wb counters
1418         * and limits. Small writeouts when the wb limits are ramping
1419         * up are the price we consciously pay for strictlimit-ing.
1420         *
1421         * If memcg domain is in effect, @dirty should be under
1422         * both global and memcg freerun ceilings.
1423         */
1424        if (dirty <= dirty_freerun_ceiling(thresh, bg_thresh) &&
1425            (!mdtc ||
1426             m_dirty <= dirty_freerun_ceiling(m_thresh, m_bg_thresh))) {
1427            unsigned long intv;
1428            unsigned long m_intv;
1429
1430free_running:
1431            intv = dirty_poll_interval(dirty, thresh);
1432            m_intv = ULONG_MAX;
1433
1434            current->dirty_paused_when = now;
1435            current->nr_dirtied = 0;
1436            if (mdtc)
1437                m_intv = dirty_poll_interval(m_dirty, m_thresh);
1438            current->nr_dirtied_pause = min(intv, m_intv);
1439            break;
1440        }
1441
1442        if (unlikely(!writeback_in_progress(wb)))
1443            wb_start_background_writeback(wb);
1444
1445        mem_cgroup_flush_foreign(wb);
1446
1447        /*
1448         * Calculate global domain's pos_ratio and select the
1449         * global dtc by default.
1450         */
1451        if (!strictlimit) {
1452            wb_dirty_limits(gdtc);
1453
1454            if ((current->flags & PF_LOCAL_THROTTLE) &&
1455                gdtc->wb_dirty <
1456                dirty_freerun_ceiling(gdtc->wb_thresh,
1457                              gdtc->wb_bg_thresh))
1458                /*
1459                 * LOCAL_THROTTLE tasks must not be throttled
1460                 * when below the per-wb freerun ceiling.
1461                 */
1462                goto free_running;
```

```
1463        }
1464
1465        dirty_exceeded = (gdtc->wb_dirty > gdtc->wb_thresh) &&
1466            ((gdtc->dirty > gdtc->thresh) || strictlimit);
1467
1468        wb_position_ratio(gdtc);
1469        sdtc = gdtc;
1470
1471        if (mdtc) {
1472            /*
1473             * If memcg domain is in effect, calculate its
1474             * pos_ratio.  @wb should satisfy constraints from
1475             * both global and memcg domains.  Choose the one
1476             * w/ lower pos_ratio.
1477             */
1478            if (!strictlimit) {
1479                wb_dirty_limits(mdtc);
1480
1481                if ((current->flags & PF_LOCAL_THROTTLE) &&
1482                    mdtc->wb_dirty <
1483                    dirty_freerun_ceiling(mdtc->wb_thresh,
1484                                mdtc->wb_bg_thresh))
1485                    /*
1486                     * LOCAL_THROTTLE tasks must not be
1487                     * throttled when below the per-wb
1488                     * freerun ceiling.
1489                     */
1490                    goto free_running;
1491            }
1492            dirty_exceeded |= (mdtc->wb_dirty > mdtc->wb_thresh) &&
1493                ((mdtc->dirty > mdtc->thresh) || strictlimit);
1494
1495            wb_position_ratio(mdtc);
1496            if (mdtc->pos_ratio < gdtc->pos_ratio)
1497                sdtc = mdtc;
1498        }
1499
1500        if (dirty_exceeded && !wb->dirty_exceeded)
1501            wb->dirty_exceeded = 1;
1502
1503        if (time_is_before_jiffies(wb->bw_time_stamp +
1504                       BANDWIDTH_INTERVAL)) {
1505            spin_lock(&wb->list_lock);
1506            __wb_update_bandwidth(gdtc, mdtc, start_time, true);
1507            spin_unlock(&wb->list_lock);
1508        }
1509
1510        /* throttle according to the chosen dtc */
1511        dirty_ratelimit = wb->dirty_ratelimit;
1512        task_ratelimit = ((u64)dirty_ratelimit * sdtc->pos_ratio) >>
1513                            RATELIMIT_CALC_SHIFT;
1514        max_pause = wb_max_pause(wb, sdtc->wb_dirty);
1515        min_pause = wb_min_pause(wb, max_pause,
1516                     task_ratelimit, dirty_ratelimit,
1517                     &nr_dirtied_pause);
1518
1519        if (unlikely(task_ratelimit == 0)) {
1520            period = max_pause;
1521            pause = max_pause;
1522            goto pause;
1523        }
1524        period = HZ * pages_dirtied / task_ratelimit;
1525        pause = period;
1526        if (current->dirty_paused_when)
```

```
1527            pause -= now - current->dirty_paused_when;
1528        /*
1529         * For less than 1s think time (ext3/4 may block the dirtier
1530         * for up to 800ms from time to time on 1-HDD; so does xfs,
1531         * however at much less frequency), try to compensate it in
1532         * future periods by updating the virtual time; otherwise just
1533         * do a reset, as it may be a light dirtier.
1534         */
1535        if (pause < min_pause) {
1536            trace_balance_dirty_pages(wb,
1537                          sdtc->thresh,
1538                          sdtc->bg_thresh,
1539                          sdtc->dirty,
1540                          sdtc->wb_thresh,
1541                          sdtc->wb_dirty,
1542                          dirty_ratelimit,
1543                          task_ratelimit,
1544                          pages_dirtied,
1545                          period,
1546                          min(pause, 0L),
1547                          start_time);
1548            if (pause < -HZ) {
1549                current->dirty_paused_when = now;
1550                current->nr_dirtied = 0;
1551            } else if (period) {
1552                current->dirty_paused_when += period;
1553                current->nr_dirtied = 0;
1554            } else if (current->nr_dirtied_pause <= pages_dirtied)
1555                current->nr_dirtied_pause += pages_dirtied;
1556            break;
1557        }
1558        if (unlikely(pause > max_pause)) {
1559            /* for occasional dropped task_ratelimit */
1560            now += min(pause - max_pause, max_pause);
1561            pause = max_pause;
1562        }
1563
1564pause:
1565        trace_balance_dirty_pages(wb,
1566                      sdtc->thresh,
1567                      sdtc->bg_thresh,
1568                      sdtc->dirty,
1569                      sdtc->wb_thresh,
1570                      sdtc->wb_dirty,
1571                      dirty_ratelimit,
1572                      task_ratelimit,
1573                      pages_dirtied,
1574                      period,
1575                      pause,
1576                      start_time);
1577        __set_current_state(TASK_KILLABLE);
1578        wb->dirty_sleep = now;
1579        io_schedule_timeout(pause);
1580
1581        current->dirty_paused_when = now + pause;
1582        current->nr_dirtied = 0;
1583        current->nr_dirtied_pause = nr_dirtied_pause;
1584
1585        /*
1586         * This is typically equal to (dirty < thresh) and can also
1587         * keep "1000+ dd on a slow USB stick" under control.
1588         */
1589        if (task_ratelimit)
1590            break;
```

```
1591
1592            /*
1593             * In the case of an unresponding NFS server and the NFS dirty
1594             * pages exceeds dirty_thresh, give the other good wb's a pipe
1595             * to go through, so that tasks on them still remain responsive.
1596             *
1597             * In theory 1 page is enough to keep the consumer-producer
1598             * pipe going: the flusher cleans 1 page => the task dirties 1
1599             * more page. However wb_dirty has accounting errors.  So use
1600             * the larger and more IO friendly wb_stat_error.
1601             */
1602            if (sdtc->wb_dirty <= wb_stat_error())
1603                break;
1604
1605            if (fatal_signal_pending(current))
1606                break;
1607        }
1608
1609        if (!dirty_exceeded && wb->dirty_exceeded)
1610            wb->dirty_exceeded = 0;
1611
1612        if (writeback_in_progress(wb))
1613            return;
1614
1615        /*
1616         * In laptop mode, we wait until hitting the higher threshold before
1617         * starting background writeout, and then write out all the way down
1618         * to the lower threshold.  So slow writers cause minimal disk activity.
1619         *
1620         * In normal mode, we start background writeout at the lower
1621         * background_thresh, to keep the amount of dirty memory low.
1622         */
1623        if (laptop_mode)
1624            return;
1625
1626        if (nr_reclaimable > gdtc->bg_thresh)
1627            wb_start_background_writeback(wb);
1628    }
```

- 1342-1347 dirty_throttle_control 结构体对象用于在脏页限流处理过程中传递状态信息。

  可以看出，这个结构体就接口之间的作用来说和之前用到的内存回收、整理过程中的 *scan_control*、*compact_control* 结构体很类似，用于在接口间传递控制信息。

  gdtc_stor 是栈上临时对象，它的内部指针指向全局静态对象 *global_wb_domain*。*mdtc_stor* 内部指向 memcg 维度的 wb_domain，用于 memcg 内存组脏页限制。同理，*sdtc* 用于单个块 (bdi 设备维度) 的脏页水位限制。

- 1361 这是脏页限流的主循环，直接采样脏页统计，计算阈值。如果脏页压力不大就直接退出。如果脏页超限，则计算暂停 (休眠) 时间进行休眠。睡醒之后继续从头获取脏页采样统计数据 (脏页有可能被后台 flusher 回写下降)，直到压力解除才 break。

- 1368 获取当前系统中文件脏页的总数。

  global_node_page_state() 会通过原子方式读 *vm_node_stat[]* 数组项。vm_node_stat 数组是全局的统计数组，会对各种类型的页的数目进行计数统计。包括但不限于脏页、写回页、active/inactive 文件或匿名页……。该数组的索引就是各类型页的枚举值。此处是读索引为NR_FILE_DIRTY的元素的值，即文件脏页的计数。

- 1369 获取系统里能用来当文件缓存、存脏页的全局总内存的大小 (单位: 页)。函数见后面，该函数也会调用 global_node_page_state() 来原子的读取值。
- 1370 把系统中所有脏页与回写页的总和存入 dirty 字段—区别于 1368 行—这才是系统中**真正**所有脏文件页的总数。

  NR_FILE_DIRTY(文件脏页) 和 ***NR_WRITEBACK***(回写页) 的统计是互斥的、不会重复统计。正在回写的页已经从 NR_FILE_DIRTY 里移除，划入 NR_WRITEBACK，所以必须相加才是完整脏页总量。
- 1372 通过 domain_dirty_limits() 函数计算获取回写域的脏页强制限流阈值与后台回写启动阈值 (解析见后面)。

  获取回写域的两个阈值，也就是 $->thresh$(脏页强制限流值) 和 $->bg_thresh$(后台回写启动阈值)。

  当脏页合计 ≥ 脏页限流阈值 (->thresh) 进程会被暂停脏页写回触发休眠。当脏页 ≥ 后台回写启动阈值，会唤醒 flusher 后台线程开始回写。

  而前面的 *current->nr_dirtied_pause* 和 *dirty_throttle_leaks* 是用于判断是否要执行脏页均衡操作的判断的阈值。换言之，也就是判断是否要进行脏页均衡的决策，而不是判断是否要执行脏页均衡操作。
- 1374-1384 分别获取的脏页数目、限流阈值 和 后台回写阈值的数值，赋值给本地临时变量。
  - 1374-1379 如果开启了严格限制模式(strictlimit), 则直接依据当前回写设备的脏页统计值与限制阈值做决策。也就是不用系统全局内存计算阈值，只用回写后端的限制计算阈值。

    严格限制模式 (strictlimit) 就是每个磁盘或后备存储独立限流每个 BDI，每个磁盘各自统计自己的脏页阈值，互不干扰。

    通常 **FUSE** 会使用严格限制模式。FUSE 是用户空间文件系统，写页缓存 (page cache) 速度极快，但用户态 FUSE 进程后端的 **回写**很慢 (网盘、远程挂载)，导致脏页一直积压，吃光系统全局 dirty 配额；如果用全局脏页限流，FUSE 能瞬间占满整个系统内存，引发 OOM、阻塞所有本地磁盘 IO。所以必须用 strictlimit 来进行单设备隔离限速。FUSE 是一个 “虚拟的独立回写设备”，不绑定某个 /dev/sda 物理盘，每个 FUSE 单独创建一个独立 BDI，wb_dirty_limits() 专门按内存比例单独给 FUSE 算一套专属阈值。

    其中 1375 行用于计算当前磁盘的阈值，具体源码见后。
  - 1381-1383 否则 (正常情况), 就用全局的脏页统计，全局阈值，全局后台回写阈值。
- 1386 当前回写域属于有效的 memcg, 需要计算 cgroup 维度的脏页水位。
- 1393-1394 读取 cgroup 内文件脏页数和正在回写的脏页数目。
- 1395 $dirty$ = 脏页数目 + 正在回写的页数目。
- 1396 目标 memcg 可以用来做脏页的可用内存。
- 1398 计算 memcg 的两个阈值。*thresh/bg_thresh*
- 1400-1409 和全局脏页阈值类似，memcg 也分严格限制模式和普通模式。
- 1424-1426

- 1424 脏页数量低于**脏页不限流的上限数量**时，不用限流。

  脏页不限流的上限数量是一个安全水位线，在脏页阈值和后台回写阈值的中间:

  $$freerun_ceiling = \frac{thresh + bg_thresh}{2}$$

  即，

  $$\text{脏页不限流的上限数量} = \frac{\text{脏页阈值} + \text{后台回写脏页阈值}}{2}$$

- 1425-1426 如果启用了内存控制组机制，则脏页用量必须同时高于全局与内存组各自的不限流脏页上限才进行限流。

- 1430-1431 这里往下到 1439 处是不限流 (安全区) 的处理。

  根据全局脏页和全局阈值计算：还要写多少页，才需要再次检查脏页。(如果是严格限制模式，此处应该根据设备 (回写域) 的脏页数和设备的脏页阈值计算。为了方便叙述，后面只针对一般情况而论，只考虑全局阈值情况。)。

  具体算法见 dirty_poll_interval(), 此处直接引用结论, 返回的页数如下:

  $$\sqrt{thresh - dirty}$$

  1431 或 1437 行这里不直接返回线性差值是因为: 线性差值会导致检查策略太呆板。这里利用了平方根的特性：剩余空间 (页)$(thresh - dirty)$ 很大时，增长 ($\sqrt{\Delta_{thresh-dirty}}$ 的变化) 缓慢。这样检查间隔大，不用频繁检查。剩余空间很小时，下降极快（检查瞬间变密集）. 这样离阈值远就可以写很多页再检查；离阈值近，写少量页就必须检查。实现了越临近阈值检查越频繁的平滑控制，避免不必要的开销。

- 1432 对于 mem 控制组，检查间隔默认预设为 $ULONG_MAX$。

- 1434 记录本次限流检查的时间 (单位:jiffies(节拍))。

- 1435 当前线程累计的脏页计数清零 (已经完成一轮检查重新计算)。

  一个线程对脏页总量的贡献程度，等于它自上一次暂停以来生成的脏页数量。内核利用该值调整每个线程的限流强度。为了实现该调整，需要追踪每个线程生成的脏页数量。该功能通过 struct task_struct->nr_dirtied 实现，该结构体成员记录了线程自上一次被 balance_dirty_pages() 检查以来生成的脏页数量。同时上次检查时间记录在 struct task_struct->dirty_paused_when 中 (见 1434 行)。

  这里之所以线程的脏页贡献是" 自上一次暂停以来生成的脏页数量"，是因为: 上一次暂停是由于线程脏页写入太多，对此内核已经" 惩罚" 了该线程 (让它暂停)。从那之后的 (过量) 写入是它新犯下的" 过错"。所以这个新" 过错" 是从上次暂停之后统计。当然，线程必须在暂停而后唤醒运行才能再次写脏页。

  1434 和 1435 行是在 free_running 的 tag 下运行的，也就是此时不考虑限速。因此每次检查时都立即把时间和脏页清零 (时间置为 now)。

- 1436-1437 使用内存控制组，就按 memcg 的脏页算检查间隔。

- 1438 取全局和 memcg 中更小的那个间隔。意味着再写这么多页，就必须调用 blance_dirty_pages() 来检查脏页是否超阈值。

- 1442-1443 如果当前没有 flusher 回写在进行，那么立即启动后台回写。

  这里隐含了脏页 > 全局或回写域的**不限流脏页上限**这个条件。因为脏页小于阈值的场景在 1424-1440 行已经处理，并且在 脏页 ≤ 不限流脏页上限 时会 break 出 for 循环。所以程序走到这里意味着脏页在阈值 *bg_thresh* 之上。而 不限流脏页上限 > *bg_thresh*。脏页数量 > *bg_thresh*，所以启动后台回写。

  需要注意的是: 这里只是唤醒 flusher 线程，并不阻塞当前线程。也就意味着此处还没开始进行阻塞限流。

- 1445 待补充

- 1451-1452 在非严格模式下，利用全局脏页阈值计算本 domain 的脏页水位阈值。

- 1453-1463

  严格限制模式就是磁盘各自限流，不受全局脏页约束。

  非严格限制模式就是受全局脏页约束。

  在非严格限制模式时，进程本应受全局脏页数量的约束，但是如果标记了 ***PF_LOCAL_THROTTLE***，就成为该模式下的例外，只受本地磁盘的限流阈值的约束来判断是否限流。

  - 1452 计算当前磁盘 (wb) 的脏页数量、脏页阈值、后台脏页阈值。
  - 1454 当前进程是不是标记为"PF_LOCAL_THROTTLE" 进程。***PF_LOCAL_THROTTLE*** 标记的进程是本地磁盘限流的。不受全局脏页限流的约束。

    换言之，没有标记该标志的进程必须走全局脏页检查，不能脏页不受限(free running)。也就是在全局脏页超标情况下，不管本地磁盘空不空闲都必须限流。如果普通进程也进这个分支，无视全局爆掉的脏页，系统会失控。

    但是，对于 flusher 回写线程而言，回写线程本身不能被脏页限流阻塞。否则可能陷入死锁。所以对于设置了该 flag 的 fluser 会不继续执行后面的 sleep，直接退出 for 循环。
  - 1455-1456 如果当前磁盘的脏页数量 < 自由运行安全线，意味着这块后备存储的 IO 还很空闲，回写跟得上，不用限流。
  - 1462 直接跳去自由运行逻辑：不限流、不睡眠、继续写。

- 1465-1466 dirty_exceeded 是整个脏页限流的核心判断条件，表示是否要阻塞、对写进程限流。如果当前磁盘 (wb: 回写域) 的脏页数量 > 该磁盘本身的限流阈值,并且满足下面任一条件，那么就需要**限流**:

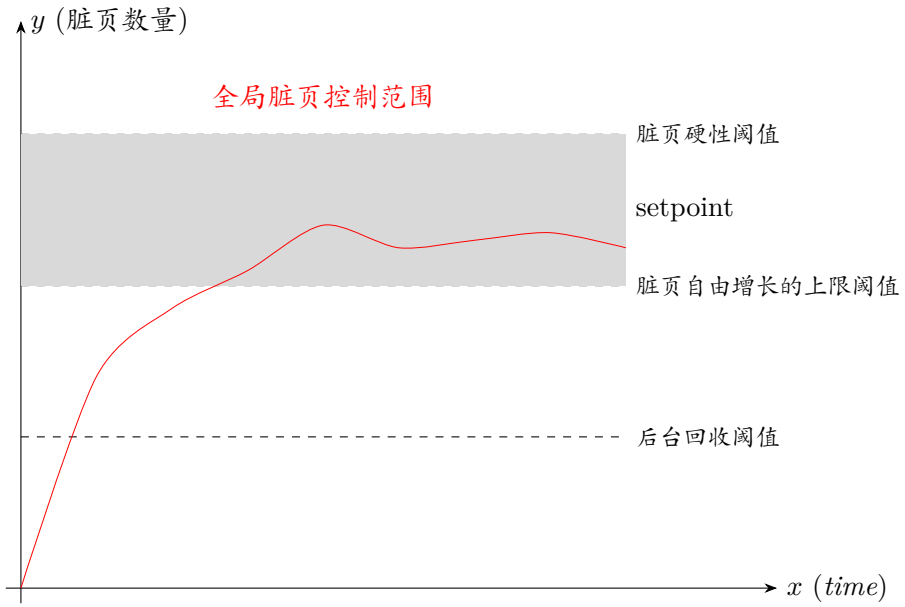
  - 全局脏页 > 全局限流阈值。
  - 当前是严格限制模式 (如:FUSE)。

    严格限制模式 (*BDI_CAP_STRICTLIMIT*) 用于不信任的文件系统，譬如，防止 FUSE 自身 bdi 脏页已经满了，但是全局脏页还没到不限流的脏页上限数(free running)。这种场景下 FUSE 可能堆积大量脏页，吃光系统内存。

  换言之，磁盘脏页超出阈值并且全局脏页超出阈值则限流；或者，磁盘脏页超出阈值并且当前是严格限制模式，也限流。

- 1468 计算获取调速系数。

  pos_ratio 用来对 dirty_ratelimit 做缩放，脏页数越靠近硬阈值，pos_ratio 就越小，进程运行写脏页速率就越低。



- 1469 接下来的限流计算，默认使用刚才算好的全局数据。
- 1478-1491 如果是 memcg 回写域的非严格限制模式场景，同样对 ***PF_LOCAL_THROTTLE*** 线程不做限流处理。
- 1492-1493 只要全局 domain 和 memcg 域任一脏页超限，就置 $dirty_exceed = true$。也就是说，即使全局脏页没超，但是 memcg 超了，同样要触发限流。
- 1495-1497 取全局和 memcg 中极小的 pos_ratio 进行赋值。
- 1500-1501 脏页超标需要限流，如果当前回写设备 wb 还没有标记，那么标记" 超标限流"。
- 1503-1508 以 $BANDWIDTH_INTERVAL$(200 毫秒) 周期定时调用 ___wb_update_bandwidth() 更新带宽值。
  - 1503-1504 把 上一次成功更新带宽的时间点 + 200毫秒($BANDWIDTH_INTERVAL$) 得到的时刻，和当前时间点进行比较。换言之，判断自上次成功更新带宽到此刻，是否超过 200 毫秒。
  - 1506 更新带宽值。包括更新写带宽 (wb->write_bandwidth)、平均写带宽 (wb->avg_write_bandwidth)，以及 wb->dirty_ratelimit。

    需要注意的是: 调用 ___wb_update_bandwidth() 时最后一个参数传入 true 时，才会更新 wb->dirty_ratelimit。整个内核传入 true 给 *___wb_update_bandwidth()* 的地方只此一处。因此从这个 if 代码段可以看出，每次调用 balance_dirty_pages() 后只有在更新带宽的时间戳后 (可以休眠或 for 循环) 的间隔 >200ms(BANDWIDTH_INTERVAL) 时才会更新 wb->dirty_ratelimit。

NOTE

回写域带宽更新的时间间隔是 *200ms*。
更新的内容: 写带宽 *(wb->write_bandwidth)*、平均写带宽 *(wb->avg_write_bandwidth)*

- 1511-1513 根据限流控制结构，开始计算限速。
  - 1511 获取取当前磁盘的基准脏页限速值，也就是设备基准的脏页生成速度的上限。是内核设置的设备维度的基准限速，不是单进程的。
  - 1512-1513 此处计算得到单进程的脏页产生速率上限值，公式是:

$$\text{基准脏页限速值} \times \text{调速系数} = \text{目标限速值}$$

这里 *RATELIMIT_CALC_SHIFT* 值为 10，所以意味着要将上式的结果再除以 2^{10}, 得到真正的限速值。这是因为调速系数 $pos_ratio \in [0, 1]$ 之间，但是内核不能做浮点数计算，所以，在之前计算 pos_ratio(调速系数) 时将其放大了 1024。因此此处右移 10 位 (除以 2^{10}) 将结果还原。

- 1514-1517 分别计算线程的最大和最小暂停时间。
- 1519-1523 如果进程写入速度 (进程脏页产生速率, 这里的**写**是指写内存而不是磁盘) 被限制为 0，不允许写，那就直接阻塞! 这种情况很少出现，一般是脏页已经严重超标，此时内核需要完全堵住脏页写进程. 需要等回写线程把脏页刷下去才能醒。

NOTE

脏页写进程和脏页回写进程是不同的东西。

脏页写进程是往 *pagecache* 里写，写的是内存。而脏页回写进程写的是磁盘或后备存储设备。

- 1524 利用上面的目标限速值(进程脏页产生速率) 计算产生数量为 pages_dirtie 的脏页 (传入的参数)，理论上要消耗多长时间 (假设这个时间段是 t_a)。将结果乘以 HZ，把时间单位从秒转换成内核的节拍数 (jiffies)。这段时间也可理解成时间配额。

  其中 pages_dirtied 取自当前线程的 struct task_struct->nr_dirtied，即线程自上一次退出 *balance_dirty_pages()*(freerunning 或休眠返回) 后产生的脏页数量。换言之，就是两次 *balance_dirty_pages()* 调用之间产生的脏页数。

- 1526-1527 ->dirty_paused_when 是当前线程上次离开 *balance_dirty_pages()* 的时刻 (jiffies 时间戳)。

  那么以 当前时刻 − 进程上次退出*balance_dirty_pages*的时刻, 得到一个时间段 t_b, 这个时长就是线程两次调用 *balance_dirty_pages()* 的真实间隔时长。用这个时间段和 1524 计算出的 t_a 比较。(====== 待插图)。也就是 $t_a - t_b$，最终得到一个 Δ_t。这个 Δ_t 就是:

$$\text{理论上按限速值产生这批脏页本来要花的时间} - \text{生成这批脏页进程实际运行时间} = \Delta_t$$

这个差值 Δ_t 就是进程需要休眠把脏页产生速率压到 *task_ratelimit* 的时间。也就是补偿时间。

从数值上看这个 Δ_t 有可能大于 0 或小于 0.

- $\Delta_t > 0$, 表示配额时间 > 真实时间间隔。

- $\Delta_t \leq 0$, 表示配额时间 $\leq$ 真实时间间隔。

- 1533-1557 如果 $\Delta_t <$ 最小休眠阈值，就不调用 *io_schedule_timeout()* 进行休眠或时间补偿。因为太短的休眠唤醒造成的调度器的开销远大于休眠收益，容易造成调度抖动。

  - 1548-1550 $\Delta_t < -HZ$ 表示进程时间生成这么多脏页时间比理论时间多耗出 1s。

    说明进程远低于脏页限速在运行，这种情况不补偿，并且过去的限速基线时间没有太大意义，直接把虚拟时间重置为当前时间，脏页计数清 0。

  - 1551-1553 $-HZ \leq \Delta_t < 0 || 0 \leq \Delta_t < min_pause$ 这种情况会进行真正的虚拟时间补偿，不管 $\Delta > 0$ 还是 < 0, 都把虚拟时间基准往后推period。因为 Δ_t 很小，为了规避调度开销不进行真正休眠，只进行时间戳记账。

  - 1554-1555 $current- > nr_dirtied_pause \leq pages_dirtied$ 该情况意味原有检测阈值过小。如不调整会造成频繁调用 *balance_dirty_pages()*, 所以稍微提高下次触发检查的阈值页数 (->nr_dirtied_pause)，避免频繁检测浪费 CPU。

    这里隐含了条件 $period == 0$, 因为如果 $perioid \neq 0$ 的话会落在前一个分支里。既然 $period == 0$ 说明 1524 行 task_ratelimit 很大，脏页生成速率高。

- 1558-1562 这段代码主要是应对极少出现的限速值骤降的场景。算出来如果要休眠太久 ($\Delta > max_pause$)，就强制只睡最大允许时间。

- 1564 以下这段是对进程脏页生成速率进行限速休眠的处理。

- 1577 把当前进程设置为可 kill 的睡眠状态。进程睡着了也能响应 SIGKILL 信号 (不会卡死、杀不掉), 这是一种安全的睡眠状态。

- 1578 记录当前这个写脏页的进程即将开始休眠的时间点。注意: 不是回写设备!

- 1579 进程主动放弃 CPU 进入睡眠停止产生脏页，睡眠时间 $= pause$(前面算出来的休眠时长)。

- 1589-1590 进程从休眠醒来之后 check 系统内存状态，判断是否可以停止限流、恢复正常写入。

  写脏页限速不为 0，说明系统已经恢复正常，直接退出循环不再限流。

  这里的退出机制是为了这两个场景:

  - **正常情况**，脏页 $<$ 阈值，此时 task_ratelimit 就会 > 0,break 放行进程。
  - **极端情况**，这才是内核真正想强调的超级关键点。

    譬如插了一个超慢速 USB U 盘，同时跑 1000 个 dd 进程疯狂写数据。由于磁盘写回太慢，这样全局脏页永远降不下来 (dirty 永远 > thresh)。如果只判断 $dirty < thresh$，所有进程会一起死等，导致系统卡死。

    对此，内核的解决办法是：不判断全局脏页，而是判断每个进程自己的 task_ratelimit。只要这个进程已经被限速“罚够了”($task_ratelimit > 0$)，脏页没降下来也能 break 放行。这样才能控制上千个进程往慢 U 盘写数据，同时又不会所有进程都卡死。

- 1602-1603 如果这个设备上已经没有可写的脏页了，就直接退出限流循环。

- 1605-1606 如果当前进程收到了杀死自己的信号 (比如 kill -9) 立刻退出循环，不再休眠、不再限流，让进程能正常退出。

- 1609-1610 当前判断脏页已经低于脏页阈值，但是 wb 结构里还标记着“超过阈值”，那么就把 wb->dirty_exceeded 清为 0。
- 1612-1613 如果目标设备的回写正在进行，就直接返回，不再启动新一轮回写。
- 1623-1624 笔记本模式下直接返回。如果是**非**笔记本模式，则跳过这两行。

  笔记本的电池为了省电需要减少磁盘频繁启停。因此在笔记本 (laptop) 模式/电池续航模式下，内核会等到脏页量到达较高阈值时，才开始后台回写，并且一次性持续回写，直到脏页量降至较低阈值。这样一来，低速写盘进程相对而言只会产生最少量的磁盘读写动作。也就是笔记本模式脏页很少时会攒一波脏页，尽量 IO 合并再统一回写，把磁盘活动降到最低。否则磁盘起来转一下又停下，反复这样会非常耗电。

  在普通工作模式下，系统只要达到更低的后台阈值，就会启动后台回写，以此始终把脏页内存占用维持在较低水平。
- 1626-1627 因为前面 1623-1624 行显示笔记本模式的场景会立即返回，所以此处处理的仅是在普通模式下,可以被回收的脏页数量超过 后台回写阈值时的场景。在该场景下，立即启动后台回写线程来写磁盘或后备存储。

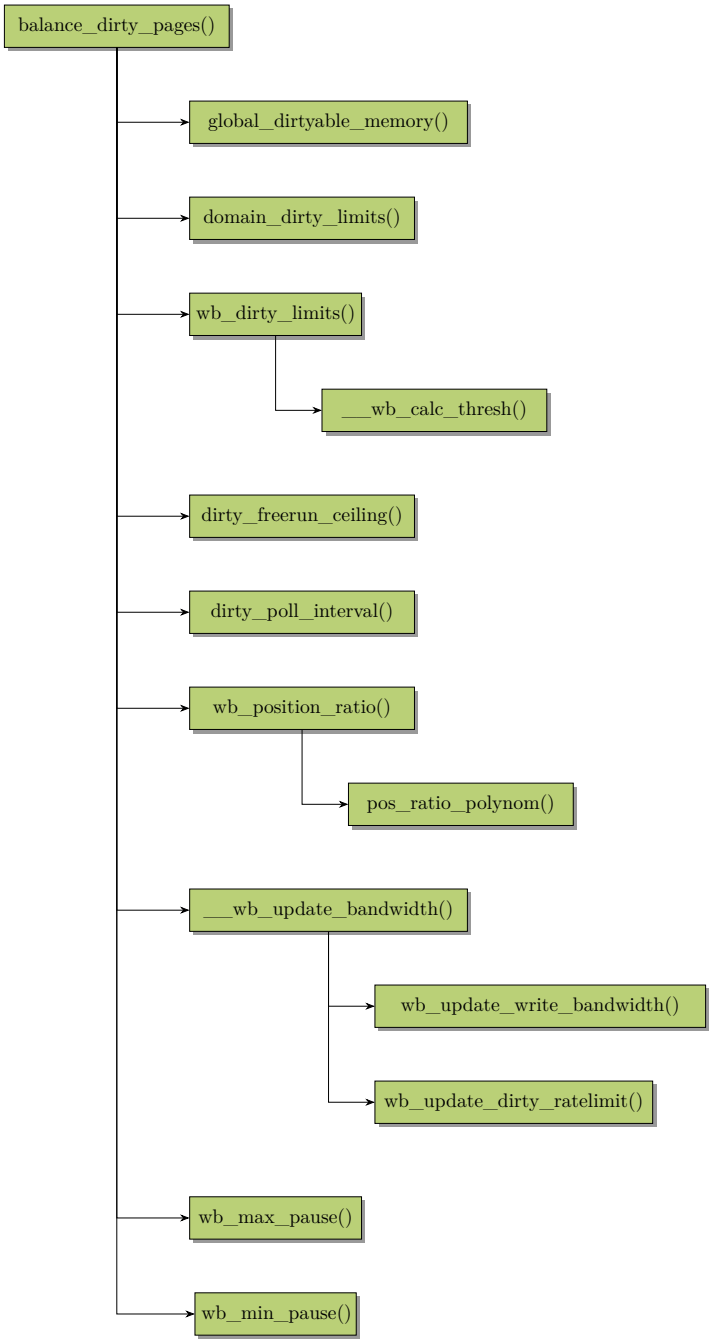
下面是这段代码和后面代码中频繁出现的几个重要变量的备注说明。

- *wb->**write_bandwidth*** 回写域每秒总的写入页面数 (带宽)。该数值由 wb_update_write_bandwidth() 调用 ___wb_update_bandwidth() 完成更新。
- *wb->**avg_write_bandwidth*** 由写入带宽 (上一行的变量) 计算得出的平滑平均值，用于抹平写入带宽数据中的短期波动。
- *wb->**balanced_dirty_ratelimit*** 单线程每秒脏页的限速上限。它是算法计算出的、每个活跃写入线程在理想状态下应该遵守的速度基准。如果有 N 个线程，那么系统总写入速度应为 $N \cdot balanced_dirty_ratelimit$（当然还要受限于 write_bw）。单线程每秒可生成脏页的上限。

  它由 ___wb_update_bandwidth() 调用 wb_update_dirty_ratelimit() 完成更新；关键前提是必须传入 update_ratelimit 参数才会执行更新。周期性带宽更新流程并不会触发它，仅在 balance_dirty_pages() 即将使用该速率阈值做脏页限流判断前，才会主动调用更新。该数值会结合位置比例系数做缩放运算，以此实现合理的脏页限流控制。
- *wb->**dirty_ratelimit*** 平滑后的单线程基准限速。它是 balanced_dirty_ratelimit 经过指数平滑（低通滤波）后的值，逻辑上与 balanced_dirty_ratelimit 等价，但该值会小幅步进更新，得到的是波动更小、经过平滑平均的脏页速率限制值。
- ***task_ratelimit*** 针对当前线程的微调值。内核会以 wb->dirty_ratelimit 为基础，根据当前线程的情况，微调出这个线程专用的限速值。

- *task_struct->**nr_dirtied*** 线程上一次因脏页限流被阻塞后，新增的脏页页面数量。
- *task_struct->**nr_dirtied_pause*** 该数值表示当前线程 (自上次休眠阻塞或清 0 后) 再生成这个变量指定数量的脏页，才会在 balance_dirty_pages() 中触发脏页限流。若线程此前被限流阻塞，该值取最小休眠时长对应的阈值；若线程处于无限制运行状态 (freerun)，则由 dirty_poll_interval() 函数计算得出

balance_dirty_pages() 函数中调用的重要子函数的层次如图：



下面分别解析这些函数。

7.3 全局脏页上限阈值

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *global_dirtyable_memory()*

```
362 static unsigned long global_dirtyable_memory(void)
363 {
364     unsigned long x;
365 
366     x = global_zone_page_state(NR_FREE_PAGES);
367     /*
```

```
368         * Pages reserved for the kernel should not be considered
369         * dirtyable, to prevent a situation where reclaim has to
370         * clean pages in order to balance the zones.
371         */
372        x -= min(x, totalreserve_pages);
373
374        x += global_node_page_state(NR_INACTIVE_FILE);
375        x += global_node_page_state(NR_ACTIVE_FILE);
376
377        if (!vm_highmem_is_dirtyable)
378            x -= highmem_dirtyable_memory(x);
379
380        return x + 1;    /* Ensure that we never return 0 */
381    }
```

- 366 获取系统全局空闲物理页数量。
- 372 从空闲页里减去内核保留 (reserve) 页。内核保留页不能算作可弄脏内存, 内核保留内存是给紧急、底层、不可被抢占的线程使用的。
- 374 inactive 文件页可以被重新弄脏，所以计入可脏内存。
- 375 正在使用的 active 文件页同样可被改写，所以也计入可脏内存。
- 377-378 如果关闭 vm_highmem_is_dirtyable, 则减去 highmem 空间。

  32 位系统才有 HIGHMEM。

  所以:

  可脏内存 = 可分配空闲页 + 活跃文件页 + 非活跃文件页 − 内核保留页 − (如果禁止 highmem 脏页) highmem 页
- 380 避免后续 $/x$ 出现除零。

7.4 允许不限速产生脏页的上限

dirty_freerun_ceiling() 函数计算允许目标进程自由 (不限速) 产生脏页的数量的上限值。换言之，脏页数量 ≤ 这个值时进程可以随便写，不限速。

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *dirty_freerun_ceiling()*

```
710 static unsigned long dirty_freerun_ceiling(unsigned long thresh,
711                         unsigned long bg_thresh)
712 {
713     return (thresh + bg_thresh) / 2;
714 }
```

- 713 目标进程写脏页不限速时，允许的脏页上限 = (脏页阈值 + 后台回写阈值) /2。这个水位也是从自由运行区间切换到全局脏页控制区间的临界点。该临界点分隔了两种不同的脏页生成区间：一种是自由运行区间，该区间内生成脏页的线程不需要脏页限流；另一种是全局脏页控制区间，该区间内会让线程休眠适当的时间，以此限流脏页生成。

7.5 回写带宽的更新

___wb_update_bandwidth() 函数，这是脏页限流、带宽估算的核心函数，专门用来计算磁盘回写带宽和动态调整限流速度。

7.5.1 ___wb_update_bandwidth()

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *___wb_update_bandwidth()*

```
1339 static void __wb_update_bandwidth(struct dirty_throttle_control *gdtc,
1340                                    struct dirty_throttle_control *mdtc,
1341                                    unsigned long start_time,
1342                                    bool update_ratelimit)
1343 {
1344     struct bdi_writeback *wb = gdtc->wb;
1345     unsigned long now = jiffies;
1346     unsigned long elapsed = now - wb->bw_time_stamp;
1347     unsigned long dirtied;
1348     unsigned long written;
1349
1350     lockdep_assert_held(&wb->list_lock);
1351
1352     /*
1353      * rate-limit, only update once every 200ms.
1354      */
1355     if (elapsed < BANDWIDTH_INTERVAL)
1356         return;
1357
1358     dirtied = percpu_counter_read(&wb->stat[WB_DIRTIED]);
1359     written = percpu_counter_read(&wb->stat[WB_WRITTEN]);
1360
1361     /*
1362      * Skip quiet periods when disk bandwidth is under-utilized.
1363      * (at least 1s idle time between two flusher runs)
1364      */
1365     if (elapsed > HZ && time_before(wb->bw_time_stamp, start_time))
1366         goto snapshot;
1367
1368     if (update_ratelimit) {
1369         domain_update_bandwidth(gdtc, now);
1370         wb_update_dirty_ratelimit(gdtc, dirtied, elapsed);
1371
1372         /*
1373          * @mdtc is always NULL if !CGROUP_WRITEBACK but the
1374          * compiler has no way to figure that out.  Help it.
1375          */
1376         if (IS_ENABLED(CONFIG_CGROUP_WRITEBACK) && mdtc) {
1377             domain_update_bandwidth(mdtc, now);
1378             wb_update_dirty_ratelimit(mdtc, dirtied, elapsed);
1379         }
1380     }
1381     wb_update_write_bandwidth(wb, elapsed, written);
1382
1383 snapshot:
1384     wb->dirtied_stamp = dirtied;
1385     wb->written_stamp = written;
1386     wb->bw_time_stamp = now;
1387 }
```

- 1344 获取当前设备的回写上下文 wb, 所有带宽、脏页统计都存在这里。
- 1345-1346 计算距离上一次更新带宽过了多久 (用 jiffies 表示)。

- 1355 带宽更新间隔 200ms 执行一次，避免太频繁。
- 1358-1359 分别获取目标 cpu 的计数: 总共产生了多少脏页，写回了多少页。
- 1365-1366 如果距离上次更新超过 1 秒，且这段时间完全空闲，那么跳转到 1383 行 tag 处，只更新时间戳，不计算带宽。因为空闲时间的带宽没有意义。

  -> bw_time_stamp 是上一次更新带宽的时间，start_time 是本次回写线程也就是 flusher 线程启动时间。time_before() 大于 0 意味着：上次更新带宽的时间，比本次回写线程启动时间要早。

  wb->bw_time_stamp 这个 “带宽更新时间戳”，只在 ___wb_update_bandwidth() 函数 1386 行被更新.time_before() 大于 0 也就表示，自从上次调用本函数后，直到本次回写线程开始前，再也没有调用过本函数。

  ___wb_update_bandwidth() 函数会在两个地方被调用来更新带宽时间戳:

  用户进程 write() 时, 进程写文件、产生脏页时。

  flusher 回写线程,start_time 就是该线程启动时间。

  所以，传入的参数 start_time 不是函数调用时间，而是本次 flusher 线程开始运行的时间。

  从上次更新带宽时间戳到 flusher 启动之间，没有再次更新过带宽时间戳。也就是这段时间系统空闲、没有回写。换言之，就是两次 flusher 线程启动之间，完全没有写文件、没有任何调用来更新带宽时间戳。
- 1368 判断是否更新脏页阈值。balance_dirty_pages() 调用该函数时, 会置位 update_ratelimit 标志位，如此则意味着需要更新脏页阈值。
- 1369 更新全局域的带宽统计信息.
- 1370 根据脏页数量和时间，重新计算更新脏页限速值。
- 1376 在开启了 cgroup 回写且 mdtc 非空的情况下继续 1377 和 1378 行。
- 1377-1378 更新 cgroup 域带宽和 cgroup 脏页限速值。
- 1381 更新回写带宽统计数据。
- 1384 记录本次脏页计数的快照
- 1385 记录本次写入页数的快照
- 1386 记录当前时间戳。

balance_dirty_pages() → *___wb_update_bandwidth()* → *wb_update_write_bandwidth()*

```
1077 static void wb_update_write_bandwidth(struct bdi_writeback *wb,
1078                                        unsigned long elapsed,
1079                                        unsigned long written)
1080 {
1081     const unsigned long period = roundup_pow_of_two(3 * HZ);
1082     unsigned long avg = wb->avg_write_bandwidth;
1083     unsigned long old = wb->write_bandwidth;
1084     u64 bw;
```

```
1085
1086        /*
1087         * bw = written * HZ / elapsed
1088         *
1089         *                    bw * elapsed + write_bandwidth * (period - elapsed)
1090         * write_bandwidth = ---------------------------------------------------
1091         *                                          period
1092         *
1093         * @written may have decreased due to account_page_redirty().
1094         * Avoid underflowing @bw calculation.
1095         */
1096        bw = written - min(written, wb->written_stamp);
1097        bw *= HZ;
1098        if (unlikely(elapsed > period)) {
1099            bw = div64_ul(bw, elapsed);
1100            avg = bw;
1101            goto out;
1102        }
1103        bw += (u64)wb->write_bandwidth * (period - elapsed);
1104        bw >>= ilog2(period);
1105
1106        /*
1107         * one more level of smoothing, for filtering out sudden spikes
1108         */
1109        if (avg > old && old >= (unsigned long)bw)
1110            avg -= (avg - old) >> 3;
1111
1112        if (avg < old && old <= (unsigned long)bw)
1113            avg += (old - avg) >> 3;
1114
1115    out:
1116        /* keep avg > 0 to guarantee that tot > 0 if there are dirty wbs */
1117        avg = max(avg, 1LU);
1118        if (wb_has_dirty_io(wb)) {
1119            long delta = avg - wb->avg_write_bandwidth;
1120            WARN_ON_ONCE(atomic_long_add_return(delta,
1121                        &wb->bdi->tot_write_bandwidth) <= 0);
1122        }
1123        wb->write_bandwidth = bw;
1124        wb->avg_write_bandwidth = avg;
1125    }
```

- 1081 将 3 秒换算成内核内部时间单位 jiffies，并且向上取整为 2 的幂次方，方便位运算。

  脏页写回的瞬时值忽高忽低，直接使用会导致带宽波动很大，系统有时会使劲写回、有时会阻塞卡死。因此内核尝试在固定的时间窗口内计算滑动平均值 (这个固定的时间窗固定为 3 秒)。简单说：滑动平均是为了不让数值突变，用“过去一段时间”的数据慢慢平滑过渡。它会把最近一小段时间内的数据加权算平均，让结果曲线更平稳。

  所谓加权的意思是给不同的数据不同的话语权，也就是不同的重要性，这就是所谓的权重。用数学观点看就是给不同的数据赋以不同的系数 (因子)。

- 1082 上次更新时平滑计算得到的平均带宽。

- 1083 上次更新时的瞬时带宽。

- 1096 计算真正写入的页数.

  wb->written_stamp 是上次更新时刻的总写入页数。written 是到此刻累计写入磁盘的总页数。正常情况下，这个值只会增加，不会减少。但有时候，内核会重标记脏页，有些已经提交 IO 的页被

算成 written（写入成功），结果又被改成脏页 PG_dirty，重新排队回写。这时候内核会把已经成功写入磁盘的页数往回减！于是计数造成了后面时刻比前面时候的 written 小的情况。

也就是 account_page_redirty() 函数的影响，导致 written 的值可能会出现减小。待补充: 变小代码的实现?????

为了避免 bw 计算过程中出现无符号数小减大，得到的计算结果变成巨大的正数. 换言之，也就是出现减数>被减数的情况，导致计算崩溃。于是这里通过 min() 宏对 written 和 wb -> written_stamp 取最小值。保证了计算结果最小为 0 而不是负数。

- 1098-1102 该代码段是：本次调用更新距离上次更新带宽的间隔超过 3 秒时带宽的计算。
  - 1098 period 即是 1081 行获取的，对应 3 秒的 jiffies 值 (向上取整成 2 的幂)。此处即是判断更新带宽数据的时间间隔是否大于 3 秒。
  - 1099 若更新时间大于 3s。则带宽 = 间隔时间内的写入页数 / 耗时 (此处耗时是以 jiffies 为单位)。也就是，

$$bw = \frac{\Delta_{written} \cdot HZ}{elapsed}$$

而耗时 elapsed 是两个采样时刻的差值，以 jiffies 为单位。即，$elapsed = \Delta_{jiffies}$。所以，

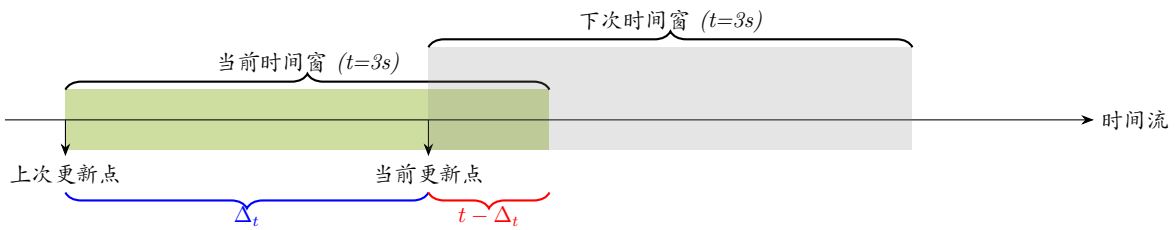
$$\begin{aligned} bw &= \frac{\Delta_{written} \cdot HZ}{elapsed} \\ &= \frac{\Delta_{written} \cdot HZ}{\Delta_{jiffies}} \\ &= \frac{\Delta_{written}}{\frac{\Delta_{jiffies}}{HZ}} \end{aligned}$$

又，$\Delta_t = \frac{\Delta_{jiffies}}{HZ}$, 所以，上式即：

$$bw = \frac{\Delta_{written}}{\Delta_t}$$

因此，可以看出 bw 也就是每秒的速率, 也就是每秒写多少页.

  - 1100 更新平均带宽值。
  - 在两次更新带宽的间隔 >3s 的场景下，计算出新的平均带宽之后。跳转到收尾的处理，准备退出函数。
- 1103-1104 在两次更新间隔不超过 3 秒的情况下 (如下图)，通过对 3s 的时间窗口进行加权平均来计算带宽。将距离上次更新的时间间隔 (Δ_t) 占的 3s 的比重做为 $\Delta_{written}$ 的加权因子，剩下的时间 ($t - \Delta_t$) 作为旧带宽的加权因子。



由图可以看出，如果本次更新带宽的时刻距离上次更新带宽的时刻越久 (就 3s 的时间窗而言), 旧带宽的话语权就越小.(这里的时间仍然是以 jiffies 为单位)。公式如下：

$$\begin{aligned} bw &= \Delta_{written} \cdot \frac{\Delta_t}{t} + bw_{old} \cdot \frac{t - \Delta_t}{t} \\ &= \frac{\Delta_{written} \cdot \Delta_t + bw_{old} \cdot (t - \Delta_t)}{t} \end{aligned}$$

另外，从图中可看出: 本次更新的时刻点是下次滑动时间窗 (灰色的矩形色块) 的起始时刻。

- 1109-1113 二次平滑。如果历史平均带宽值比旧带宽值高、新测量得到的带宽值比上次更新的带宽值小。也就是: 历史平均带宽 > 上次带宽 ≥ 当前写带宽，就让平均值下降，为了避免突变和瞬间波动，只让平均值慢慢减 1/8(即 >> 3), 平滑下降。同理，1374-1375 行将平均值平滑上升。
- 1117 保证平均带宽始终大于 0，确保存在脏页回写设备时，总带宽值一定大于 0。
- 1118-1122 如果这个磁盘还有脏页要回写, 才更新总带宽。如果没有脏页了，意味着回写停了，那么带宽就不用计算了。换言之，有脏页正在写时带宽才有效。
  - 1119 计算平均带宽变化了多少。
  - 1120-1121 原子操作把当前算出的平均带宽变化量 Δ 加到目标磁盘或后备存储的所有活跃脏页 wb 的带宽权重和。如果总带宽 0，对于这种不应该发生的情况，内核会报警。

    1 个磁盘 (bdi) 包含多个回写单元 (wb)。bdi->tot_write_bandwidth 是各个 cgroup/wb 各自统计的历史平滑带宽的和。
- 1123-1124 分别保存当前平滑带宽和平均带宽，后者真正用于限速。

7.6 控制脏页产生速率的调速系数

该系数用于缩放脏页写入速率上限。在回写压力场景下，确保每一个独立写入线程的脏页产生速率，逐步收敛至系统预设的理想阈值；该系数取值被限制在 0-2 之间：

- 当脏页生成速率过低时，脏页限速最大可放宽至原本的两倍；
- 当脏页生成速率过高、即将压垮块设备 IO 负载时，该系数会收紧，转为硬性限流阈值。

7.6.1 wb_position_ratio()

balance_dirty_pages_ratelimited() → *balance_dirty_pages()* → *wb_position_ratio()*

```
898 static void wb_position_ratio(struct dirty_throttle_control *dtc)
899 {
900     struct bdi_writeback *wb = dtc->wb;
901     unsigned long write_bw = wb->avg_write_bandwidth;
902     unsigned long freerun = dirty_freerun_ceiling(dtc->thresh, dtc->bg_thresh);
903     unsigned long limit = hard_dirty_limit(dtc_dom(dtc), dtc->thresh);
904     unsigned long wb_thresh = dtc->wb_thresh;
905     unsigned long x_intercept;
906     unsigned long setpoint;     /* dirty pages' target balance point */
907     unsigned long wb_setpoint;
908     unsigned long span;
```

```
909     long long pos_ratio;        /* for scaling up/down the rate limit */
910     long x;
911
912     dtc->pos_ratio = 0;
913
914     if (unlikely(dtc->dirty >= limit))
915         return;
916
917     /*
918      * global setpoint
919      *
920      * See comment for pos_ratio_polynom().
921      */
922     setpoint = (freerun + limit) / 2;
923     pos_ratio = pos_ratio_polynom(setpoint, dtc->dirty, limit);
924
925    /*
926      * The strictlimit feature is a tool preventing mistrusted filesystems
927      * from growing a large number of dirty pages before throttling. For
928      * such filesystems balance_dirty_pages always checks wb counters
929      * against wb limits. Even if global "nr_dirty" is under "freerun".
930      * This is especially important for fuse which sets bdi->max_ratio to
931      * 1% by default. Without strictlimit feature, fuse writeback may
932      * consume arbitrary amount of RAM because it is accounted in
933      * NR_WRITEBACK_TEMP which is not involved in calculating "nr_dirty".
934      *
935      * Here, in wb_position_ratio(), we calculate pos_ratio based on
936      * two values: wb_dirty and wb_thresh. Let's consider an example:
937      * total amount of RAM is 16GB, bdi->max_ratio is equal to 1%, global
938      * limits are set by default to 10% and 20% (background and throttle).
939      * Then wb_thresh is 1% of 20% of 16GB. This amounts to ~8K pages.
940      * wb_calc_thresh(wb, bg_thresh) is about ~4K pages. wb_setpoint is
941      * about ~6K pages (as the average of background and throttle wb
942      * limits). The 3rd order polynomial will provide positive feedback if
943      * wb_dirty is under wb_setpoint and vice versa.
944      *
945      * Note, that we cannot use global counters in these calculations
946      * because we want to throttle process writing to a strictlimit wb
947      * much earlier than global "freerun" is reached (~23MB vs. ~2.3GB
948      * in the example above).
949      */
950     if (unlikely(wb->bdi->capabilities & BDI_CAP_STRICTLIMIT)) {
951         long long wb_pos_ratio;
952
953         if (dtc->wb_dirty < 8) {
954             dtc->pos_ratio = min_t(long long, pos_ratio * 2,
955                        2 << RATELIMIT_CALC_SHIFT);
956             return;
957         }
958
959         if (dtc->wb_dirty >= wb_thresh)
960             return;
961
962         wb_setpoint = dirty_freerun_ceiling(wb_thresh,
963                             dtc->wb_bg_thresh);
964
965         if (wb_setpoint == 0 || wb_setpoint == wb_thresh)
966             return;
967
968         wb_pos_ratio = pos_ratio_polynom(wb_setpoint, dtc->wb_dirty,
969                          wb_thresh);
970
971         /*
972          * Typically, for strictlimit case, wb_setpoint << setpoint
```

```
         * and pos_ratio >> wb_pos_ratio. In the other words global
         * state ("dirty") is not limiting factor and we have to
         * make decision based on wb counters. But there is an
         * important case when global pos_ratio should get precedence:
         * global limits are exceeded (e.g. due to activities on other
         * wb's) while given strictlimit wb is below limit.
         *
         * "pos_ratio * wb_pos_ratio" would work for the case above,
         * but it would look too non-natural for the case of all
         * activity in the system coming from a single strictlimit wb
         * with bdi->max_ratio == 100%.
         *
         * Note that min() below somewhat changes the dynamics of the
         * control system. Normally, pos_ratio value can be well over 3
         * (when globally we are at freerun and wb is well below wb
         * setpoint). Now the maximum pos_ratio in the same situation
         * is 2. We might want to tweak this if we observe the control
         * system is too slow to adapt.
         */
        dtc->pos_ratio = min(pos_ratio, wb_pos_ratio);
        return;
    }

    /*
     * We have computed basic pos_ratio above based on global situation. If
     * the wb is over/under its share of dirty pages, we want to scale
     * pos_ratio further down/up. That is done by the following mechanism.
     */

    /*
     * wb setpoint
     *
     *        f(wb_dirty) := 1.0 + k * (wb_dirty - wb_setpoint)
     *
     *                         x_intercept - wb_dirty
     *                     := --------------------------
     *                         x_intercept - wb_setpoint
     *
     * The main wb control line is a linear function that subjects to
     *
     * (1) f(wb_setpoint) = 1.0
     * (2) k = - 1 / (8 * write_bw)  (in single wb case)
     *     or equally: x_intercept = wb_setpoint + 8 * write_bw
     *
     * For single wb case, the dirty pages are observed to fluctuate
     * regularly within range
     *        [wb_setpoint - write_bw/2, wb_setpoint + write_bw/2]
     * for various filesystems, where (2) can yield in a reasonable 12.5%
     * fluctuation range for pos_ratio.
     *
     * For JBOD case, wb_thresh (not wb_dirty!) could fluctuate up to its
     * own size, so move the slope over accordingly and choose a slope that
     * yields 100% pos_ratio fluctuation on suddenly doubled wb_thresh.
     */
    if (unlikely(wb_thresh > dtc->thresh))
        wb_thresh = dtc->thresh;
    /*
     * It's very possible that wb_thresh is close to 0 not because the
     * device is slow, but that it has remained inactive for long time.
     * Honour such devices a reasonable good (hopefully IO efficient)
     * threshold, so that the occasional writes won't be blocked and active
     * writes can rampup the threshold quickly.
     */
    wb_thresh = max(wb_thresh, (limit - dtc->dirty) / 8);
```

```
1037     /*
1038      * scale global setpoint to wb's:
1039      *  wb_setpoint = setpoint * wb_thresh / thresh
1040      */
1041     x = div_u64((u64)wb_thresh << 16, dtc->thresh | 1);
1042     wb_setpoint = setpoint * (u64)x >> 16;
1043     /*
1044      * Use span=(8*write_bw) in single wb case as indicated by
1045      * (thresh - wb_thresh ~= 0) and transit to wb_thresh in JBOD case.
1046      *
1047      *        wb_thresh                    thresh - wb_thresh
1048      * span = --------- * (8 * write_bw) + ------------------ * wb_thresh
1049      *         thresh                           thresh
1050      */
1051     span = (dtc->thresh - wb_thresh + 8 * write_bw) * (u64)x >> 16;
1052     x_intercept = wb_setpoint + span;
1053
1054     if (dtc->wb_dirty < x_intercept - span / 4) {
1055         pos_ratio = div64_u64(pos_ratio * (x_intercept - dtc->wb_dirty),
1056                               (x_intercept - wb_setpoint) | 1);
1057     } else
1058         pos_ratio /= 4;
1059
1060     /*
1061      * wb reserve area, safeguard against dirty pool underrun and disk idle
1062      * It may push the desired control point of global dirty pages higher
1063      * than setpoint.
1064      */
1065     x_intercept = wb_thresh / 2;
1066     if (dtc->wb_dirty < x_intercept) {
1067         if (dtc->wb_dirty > x_intercept / 8)
1068             pos_ratio = div_u64(pos_ratio * x_intercept,
1069                                 dtc->wb_dirty);
1070         else
1071             pos_ratio *= 8;
1072     }
1073
1074     dtc->pos_ratio = pos_ratio;
1075 }
```

- 914-915 如果全局脏页已经超上限，直接返回 0，进行完全限速。换言之，不然产生新脏页了。
- 922 计算理想脏页平衡点，也就是内核期望的最佳脏页数量。它是自由运行水位 (freerun) 和硬限速水位 (limit) 的中点。

  =======> 待插入 bg_limit、limit、freeruning、setpount 几者水位线关系图

- 923 调用多项式函数，算出全局脏页调速系数 pos_ratio。
- 950-994 如果磁盘开启了严格限制模式 (STRICTLIMIT) 则进入此代码块。

  strictlimit 机制是一种防护手段，用于防止不可信文件系统在脏页限流生效前，累积大量脏页。对于这类文件系统，balance_dirty_pages 会始终将回写 (wb) 计数器和 wb 回写阈值做对比判断；即便全局脏页计数 (nr_dirty) 尚处于自由增长（freerun）的安全区间，该限制依然强制生效。

  该机制对 FUSE 文件系统尤为关键。因为 FUSE 是用户态文件系统，不可信、不可控。所以内核默认给 FUSE 极小的脏页配额: 将 bdi->max_ratio 配置为 1%(它表示: 这个设备 /文件系统，最多只能占用 “全局脏页总容量” 的 1%)。倘若没有 strictlimit 机制，FUSE 的 max_ratio 就形同虚设。FUSE 回写流程可能无节制占用内存，原因是 FUSE 文件系统的回写脏页会被统计至

NR_WRITEBACK_TEMP，而这部分内存不纳入全局脏页 (nr_dirty) 的统计，天然绕过了全局限流。

绝大多数普通文件系统（ext4/xfs）不用” 严格限制模式”。只有 FUSE 等少数文件系统会使用，所以标记 unlikely.

在本函数中，脏页生成的调速系数 pos_ratio 仅依据两个本地指标计算：目标回写域的当前脏页量（wb_dirty）与回写域的限流阈值（wb_thresh）。

需要注意的是：开启 strictlimit 的回写域 (wb) 后，不使用全局内存计数器参与调速计算。内核需要对这类 wb 回写提前限流、提前拦截，不能等到全局脏页触及运行上限才管控。

- 953-956 如果当前磁盘的脏页数量特别少 (少于 8 页)，那就直接允许 “双倍速度” 写入！把限速系数直接 ×2 放大（最多放大到 2048）。然后返回。
- 959-960 目标磁盘脏页已大于脏页限制天花板，直接返回;
- 962-963 计算本磁盘无需考虑脏页限速的 freerun 水位。
- 965-966 如果平衡点是 0，或者平衡点等于上限阈值，那就直接返回，不计算三次多项式进行调速。也就是平衡点必须在 0 和上限阈值之间。如果不满足，三次函数公式会失效。
- 968-969 计算本磁盘的调速系数 pos_ratio。
- 992 取全局调速系数和本磁盘的调速系数两者中的最小值。简言之，谁限制更严，用谁。

- 1027-1028 本磁盘脏页上限不能超过全局脏页上限.
- 1036 给磁盘一个保底脏页阈值，避免完全卡死。确保 wb_thresh 不会变得太小、太接近 0，给脏页控制保留一个 “最小安全阈值”，防止调速曲线变得极端、导致系统抖动/卡死。就算计算出来的 wb_thresh 很小，也不能小于 “剩余脏页额度的 1/8”。因为 wb_thresh 脏页数值趋近于 0，极有可能并非因设备读写性能低下，而是该设备已长期处于空闲静默状态。针对这类设备，需要赋予一个合理且优化 IO 效率的阈值，从而避免零星偶然的写入操作被阻塞，同时保证高频连续写入场景下，能够快速拉高回写阈值。
- 1041-1042 按本地脏页值和全局脏页值的比例算出本地目标磁盘的平衡点。

  计算公式如下：

$$wb_setpoint = setpoint \cdot \frac{wb_thresh}{thresh}$$

  这里的$dtc->thresh|1$操作是为了避免产生除以 0 的操作。

- 1051 计算 span。它的公式如下:

$$span = \frac{wb_thresh}{thresh} \cdot (8 \cdot write_bw) + \frac{thresh - wb_thresh}{thresh} \cdot wb_thresh \qquad (*)$$

  span 代表了设备脏页从” 理想工作点” 到” 极限点” 之间的缓冲空间。就单磁盘而言，在时间维度 span 代表了内核允许脏页在内存中停留的最长时间窗口:span= 设备带宽 * 时间窗口 (默认 8 秒). 但是对于 JBOD 场景，在内存紧张时可能分配过多缓存。所以引入了系统剩余资源的考量，并加入了公平性比例系数。所谓公平性比例系数是: 当前设备的脏页阈值占全局阈值的比例 (但至少要达到全局阈值的 1/8, 见 1036 行)。

NOTE

内核回写框架里存在两种回写场景：

单回写域场景 对应：单块物理磁盘或单个存储设备。只有一个 *wb* 回写实例、一套脏页阈值、*IO* 带宽稳定，脏页波动小，只用固定带宽系数 *8 * write_bw* 控制即可。
多盘拼接场景 *(JBOD)* 对应：多块磁盘组合成 *JBOD* 逻辑盘。每块子盘都有独立的单盘回写阈值 *(wb_thresh)*。整体全局阈值是所有盘阈值之和。

(*) 式变形可得:

$$span = \frac{wb_thresh}{thresh} \cdot [(8 \cdot write_bw) + (thresh - wb_thresh)]$$

- ***thresh-wb_thresh*** 是系统剩余资源。thresh 是全局允许的最大脏页数量；wb_thresh 是分配给当前设备的脏页阈值。因此，这项代表当前系统还有多少脏页可以分配给新的写入操作。这是为了确保 span 的大小受限于系统整体资源。
- ***8 * write_bw*** 是设备保底能力。write_bw 是设备的带宽，单位是" 页/秒"。系数 8 是个经验值。这是为了保证 span 有个合理最小值维持设备能持续工作。从另一角度看，也就是内核允许目标设备的脏页在内存停留的时间窗是 8 秒。

  所以，目标设备的 span 为:

$$span = (\text{剩余资源} + 8 \times \text{设备带宽}) \times \text{比例系数}$$

- 1052 x_intercept 顾名思义就是 x 轴上的截距。它等于当前设备的理想点 (wb_setpoint) 加上 span 得到的和 (见下图)。紧接着下面我们将以此建立一个在全局 pos_ratio 基础上微调设备新 pos_ratio 的线性方程。

关于 pos_ratio 值，前面代码中利用三次函数算出的是基础的全局限速调整系数，所有磁盘 (后备存储) 设备一起遵守。若是某个后备存储自己的脏页数超标了，就要单独惩罚；反之则进行奖励。换言之，回写设备的脏页数量高于或低于其应占的份额，我们需要进一步下调或上调 pos_ratio。。此处即通过线性函数对单设备修正微调。

首先，我们设计一个函数，这个函数能根据当前设备脏页数量 wb_dirty 动态计算**新**的限速系数 pos_ratio。该函数满足两个核心需求:

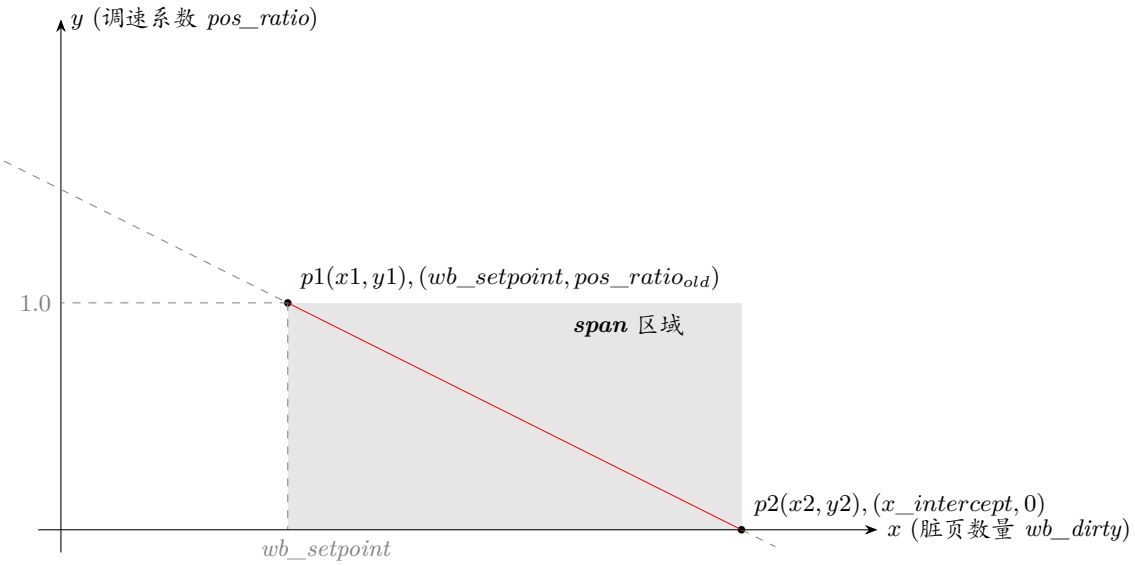
- 当前设备脏页数量等于理想值时 ($wb_dirty = wb_setpoint$), 限速系数保持不变, 即,

$$pos_ratio_{new} = pos_ratio_{old}$$

- 当设备脏页数量达到上限时 ($wb_dirty = x_intercept$), 限速系数变为 0，即,

$$pos_ratio_{new} = 0$$

这次是在全局系数基础上进行微调，所以只需要一个简单的线性模型。我们将这两个需求转化为坐标系中的两个点:



如图，

$$p1(x1, y1) = (wb_setpoint, pos_ratio_{old})$$
$$p2(x2, y2) = (x_intercept, 0)$$

已知两点 p1(x1,y1) 和 p2(x2,y2), 则可得直线两点式方程为:

$$\frac{y - y1}{x - x1} = \frac{y2 - y1}{x2 - x1}$$

将 p1,p2 的坐标代入，

$$\frac{y - pos_ratio_{old}}{x - wb_setpoint} = \frac{0 - pos_ratio_{old}}{x_intercept - wb_setpoint}$$

即，

$$\frac{pos_ratio_{new} - pos_ratio_{old}}{wb_dirty - wb_setpoint} = \frac{0 - pos_ratio_{old}}{x_intercept - wb_setpoint}$$

将 pos_ratio_{new} 单独放在等式左边，

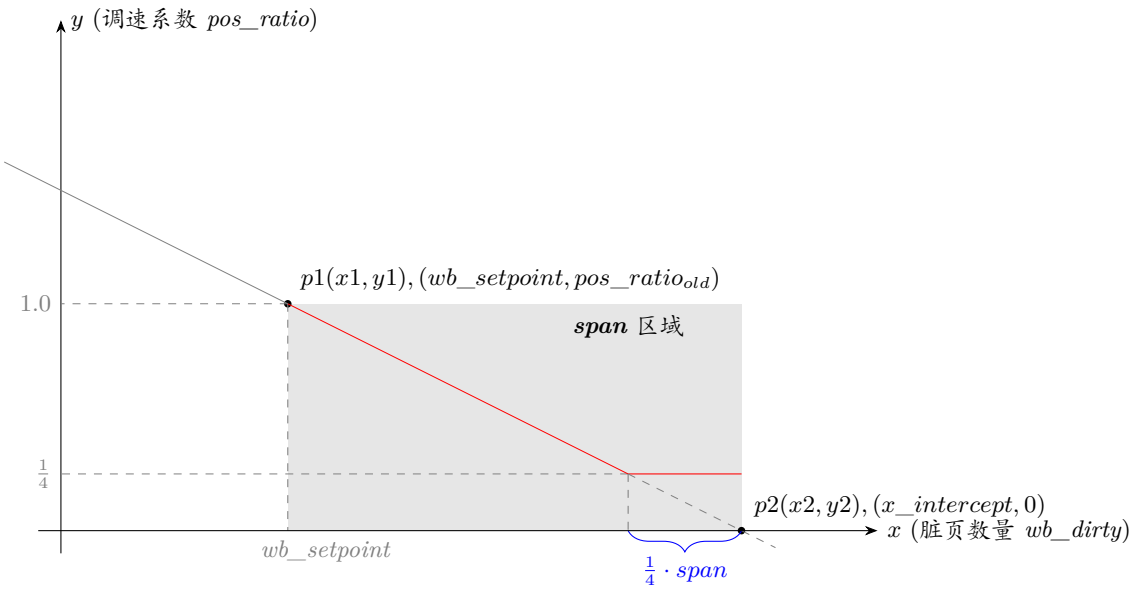
$$pos_ratio_{new} = \frac{0 - pos_ratio_{old}}{x_intercept - wb_setpoint} \cdot (wb_dirty - wb_setpoint) + pos_ratio_{old}$$

整理得:

$$\begin{aligned}
pos_ratio_{new} &= pos_ratio_{old} \cdot \left[1 + \frac{-1}{x_intercept - wb_setpoint} \cdot (wb_dirty - wb_setpoint)\right] \\
&= pos_ratio_{old} \cdot \left(1 + \frac{wb_setpoint - wb_dirty}{x_intercept - wb_setpoint}\right) \\
&= pos_ratio_{old} \cdot \left[\frac{(x_intercept - wb_setpoint) + (wb_setpoint - wb_dirty)}{x_intercept - wb_setpoint}\right] \\
&= pos_ratio_{old} \cdot \left(\frac{x_intercept - wb_dirty}{x_intercept - wb_setpoint}\right)
\end{aligned}$$

- 1054 检测当前磁盘或后备存储设备的脏页是否小于限速线 (截距:x_intercept)−安全裕量 ($\frac{span}{4}$)。若是脏页数量较少 (处于 span 控制区间的前 3/4 部分)，则系统脏页水平还安全，可以正常线性调速。反之系统脏页水平已经很危险！直接砍速 pos_ratio 成原值到 $\frac{1}{4}$, 降低脏页写入速度。

  所以设备的 pos_ratio 在前图的基础上还要再次修正:



如上图所示，当脏页数量距 span 尾部 $\frac{1}{4}$ 时，直接 pos_ratio 变成 $\frac{1}{4} \cdot pos_ratio_{old}$。

- 1055 代码的计算公式即前面的式子 ("|1" 操作是为了避免出现除 0 操作，暂时忽略):

$$pos_ratio = pos_ratio \cdot \frac{x_intercept - wb_dirty}{x_intercept - wb_setpoint}$$

这是一个线性递减函数。当 wb_dirty 等于 wb_setpoint 时，分母分子相等，比例不变；当 wb_dirty 接近 x_intercept 时，分子趋近 0，比例趋近 0，从而降低写入速度或增加回写压力。

- 1065 重新赋值 x 轴上的截距的值。这里实际是借用变量名 x_intercept 表示一个低水位的阈值。
- 1066-1072 和之前代码有区别的是这个代码块在目标设备脏页数量非常低的情况下进入。当前设备的脏页数量 < 当前设备脏页阈值的 $\frac{1}{2}$ 时才进入这段代码块。这段代码负责在目标磁盘或设备脏页数量非常低的情况时，调整脏页调速系数，避免磁盘空闲。
  - 1067 当前设备的脏页数量 > 当前设备脏页阈值的 $\frac{1}{16}$
  - 1068-1069 如果满足上一项的判断条件，则，

$$\begin{aligned} pos_ratio &= \frac{pos_ratio \cdot \frac{wb_thresh}{2}}{wb_dirty} \\ &= pos_ratio \cdot \frac{\frac{wb_thresh}{2}}{wb_dirty} \end{aligned}$$

- 1071 若当前设备的脏页数量 ≤ 当前设备脏页阈值的 $\frac{1}{16}$, 则:

$$pos_ratio = 8 \cdot pos_ratio$$

脏页极少，直接将调速因子放大 8 倍，最大程度地允许写入。

下表将脏页较少时当前设备脏页数、当前设备脏页阈值、脏页调速系数的变化列出了。

脏页数 *vs* 脏页阈值	脏页调速系数的变化
$\frac{1}{16} \cdot wb_thresh < wb_dirty < \frac{1}{2} \cdot wb_thresh$	$pss_ratio = pos_ratio \cdot \frac{wb_thresh}{wb_dirty} \cdot \frac{1}{2}$
$0 < wb_dirty \leq \frac{1}{16} \cdot wb_thresh$	$pos_ratio = 8 \cdot pos_ratio$

7.6.1.1 pos_ratio_polynom()

为了对进程的脏页产生速度进行动态限速，923 行调用的 pos_ratio_polynom() 会计算出脏页回写动态调速系数。它根据当前系统脏页数量，没有用复杂 PID(比例-积分-微分控制器)，只用一个极简三次多项式的数学曲线，硬写出来一个简化版 PID 控制器计算脏页写入调速系数。用极低 CPU 开销实现了”大偏差快修正、小偏差稳得住” 的同款负反馈控制效果。偏差越大，负反馈就越反向纠正它。目的是把脏页数量锁在平衡点 (setpoint) 附近.

balance_dirty_pages_ratelimited() → *balance_dirty_pages()* → *wb_position_ratio()* → *pos_ratio_polynom()*

```
806 static long long pos_ratio_polynom(unsigned long setpoint,
807                       unsigned long dirty,
808                       unsigned long limit)
809 {
810     long long pos_ratio;
811     long x;
812
813     x = div64_s64(((s64)setpoint - (s64)dirty) << RATELIMIT_CALC_SHIFT,
814               (limit - setpoint) | 1);
815     pos_ratio = x;
816     pos_ratio = pos_ratio * x >> RATELIMIT_CALC_SHIFT;
817     pos_ratio = pos_ratio * x >> RATELIMIT_CALC_SHIFT;
818     pos_ratio += 1 << RATELIMIT_CALC_SHIFT;
819
820     return clamp(pos_ratio, 0LL, 2LL << RATELIMIT_CALC_SHIFT);
821 }
```

内核空间不能执行浮点运算, 因而使用整型完成整套计算。由于该系数本质是线程脏页生成速率的浮点缩放因子，为规避浮点运算限制，所有计算会先通过宏 RATELIMIT_CALC_SHIFT(10) 进行左移定点放大，也就是相当于乘以 2^{10}。另外，因为后续计算也要用整数，所以 pos_ratio 以乘以 2^{10}的形式存在并直接返回放大后的值。

该函数的核心计算公式为:

$$ratio = 1 + \left(\frac{setpoint - dirty}{limit - setpoint}\right)^3$$

式中:

- ***dirty*** 当前全局脏页总数量

- ***limit*** 系统脏页的上限阈值
- ***setpoint*** 脏页平衡点。换言之，就是脏页的理想目标数，是内核希望系统维持稳定保持的脏页数。

下面表格是写入速度、脏页数、平衡点等和调速系数的对应关系:

脏页和平衡点的关系	脏页状态	三次项	调速系数	写入速度的调整
$dirty < setpoint$	脏页较少	$\left(\frac{setpoint-dirty}{limit-setpoint}\right)^3 > 0$	$ratio > 1$	加速页写入
$dirty == setpoint$	脏页数较理想	$\left(\frac{setpoint-dirty}{limit-setpoint}\right)^3 == 0$	$ratio == 1$	写入速度不变
$dirty > setpoint$	脏页太多	$\left(\frac{setpoint-dirty}{limit-setpoint}\right)^3 < 0$	$ratio < 1$	限流，减速页写入

从上式可知调速系数 ratio 是关于 dirty 的函数，函数也可写为:

$$\begin{aligned} f(dirty) &= 1 + \left(\frac{setpoint - dirty}{limit - setpoint}\right)^3 \\ &= 1 + \left(\frac{1}{limit - setpoint}\right)^3 \cdot (setpoint - dirty)^3 \end{aligned}$$

在此多项式中 limit、setpoint 是已知值，所以上面等式等价与下面的三次等式:

$$\begin{aligned} f(x) &= 1 + \left(\frac{1}{limit - setpoint}\right)^3 \cdot (setpoint - x)^3 \\ &= 1 + a \cdot (b - x)^3 \end{aligned} \tag{1}$$

- $a = \left(\frac{1}{limit-setpoint}\right)^3$
- $b = setpoint$

- 813-814 给 x 赋值的两行代码的执行如下:

$$x = \frac{(setpoint - dirty) \cdot 1024}{limit - setpoint}$$

设:

$$A = \frac{setpoint - dirty}{limit - setpoint} \tag{2}$$

则,

$$x = A \cdot 1024$$

- 816 该行得到的计算结果如下,

$$\begin{aligned} tmp &= \frac{x \cdot x}{1024} \\ &= \frac{(A \cdot 1024) \cdot (A \cdot 1024)}{1024} \\ &= A^2 \cdot 1024 \end{aligned}$$

- 817 继续计算，得到

$$\begin{aligned} tmp &= \frac{(A^2 \cdot 1024) \cdot x}{1024} \\ &= \frac{(A^2 \cdot 1024) \cdot (A \cdot 1024)}{1024} \\ &= \frac{A^3 \cdot 1024^2}{1024} \\ &= A^3 \cdot 1024 \end{aligned}$$

- 818 最终得到计算结果，

$$\begin{aligned} pos_ratio &= tmp + (1 \ll 10) \\ &= A^3 \cdot 1024 + 1024 \\ &= 1024 + A^3 \cdot 1024 \\ &= 1024 \cdot (1 + A^3) \end{aligned}$$

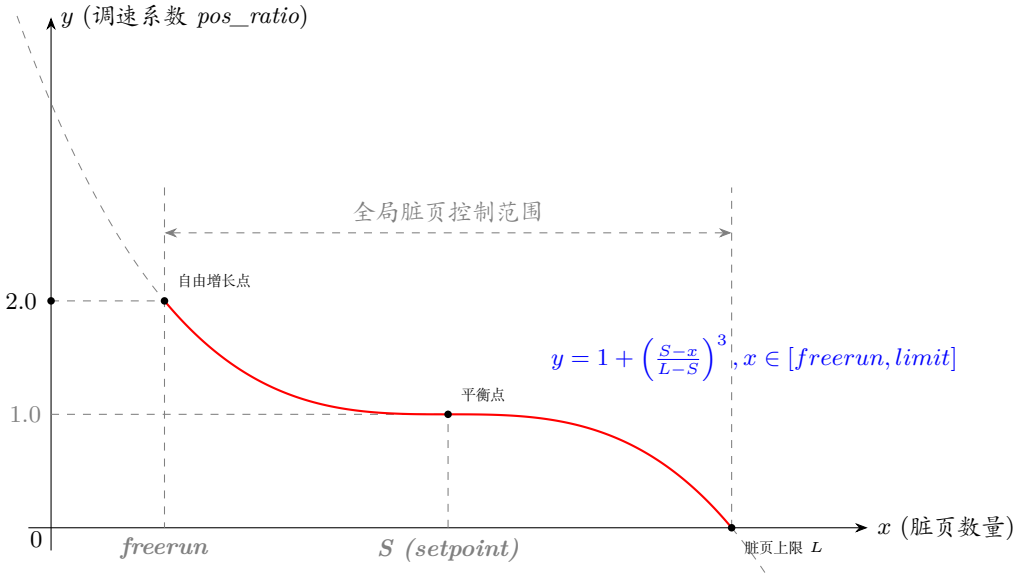
将上面 (2) 处式子代入，也就是:

$$pos_ratio = 1024 \cdot (1 + A^3) = 1024 \cdot \left[1 + \left(\frac{setpoint - dirty}{limit - setpoint}\right)^3\right]$$

- 820 得到的最终结果钳位到 $[0 \times 1024, 2 \times 1024]$.

所以，最终得到的 pos_ratio 在 $[0, 2.0]$ 之间，用整数表示是在 $0 \times 1024 \le pos_ratio \le 2 \times 1024$ 之间。

从代码可以看出，返回值是已经扩大 1024 后的值。



函数曲线如上，调速系数钳位在 $[0, 2]$ 之间，超出该区间的三次曲线部分 (灰色虚线表示) 是无意义的。换言之，只红色部分有效。

当脏页越多时，调速系数会越小。另外，越靠近平衡点 S 时，调速系数的变化越来越平缓; 越远离平衡点 S，调速系数变化就越剧烈。当脏页数量小于平衡点时，调速系数 >1, 加速脏页生成。当脏页数量大于平衡点时，调速系数 <1, 限制脏页生成。直到脏页达到限制点时，直接停止新脏页生

成。脏页数量小于 freerun 点的数值时，调速系数可以自由增长 (即，忽略三次曲线灰色虚线部分的轨迹)。

7.7 脏页阈值和后台回写阈值

回写域 (wb) 的脏页阈值和后台回写阈值在 domain_dirty_limits() 函数中计算。

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *domain_dirty_limits()*

```
392 static void domain_dirty_limits(struct dirty_throttle_control *dtc)
393 {
394     const unsigned long available_memory = dtc->avail;
395     struct dirty_throttle_control *gdtc = mdtc_gdtc(dtc);
396     unsigned long bytes = vm_dirty_bytes;
397     unsigned long bg_bytes = dirty_background_bytes;
398     /* convert ratios to per-PAGE_SIZE for higher precision */
399     unsigned long ratio = (vm_dirty_ratio * PAGE_SIZE) / 100;
400     unsigned long bg_ratio = (dirty_background_ratio * PAGE_SIZE) / 100;
401     unsigned long thresh;
402     unsigned long bg_thresh;
403     struct task_struct *tsk;
404
405     /* gdtc is !NULL iff @dtc is for memcg domain */
406     if (gdtc) {
407         unsigned long global_avail = gdtc->avail;
408
409         /*
410          * The byte settings can't be applied directly to memcg
411          * domains.  Convert them to ratios by scaling against
412          * globally available memory.  As the ratios are in
413          * per-PAGE_SIZE, they can be obtained by dividing bytes by
414          * number of pages.
415          */
416         if (bytes)
417             ratio = min(DIV_ROUND_UP(bytes, global_avail),
418                     PAGE_SIZE);
419         if (bg_bytes)
420             bg_ratio = min(DIV_ROUND_UP(bg_bytes, global_avail),
421                       PAGE_SIZE);
422         bytes = bg_bytes = 0;
423     }
424
425     if (bytes)
426         thresh = DIV_ROUND_UP(bytes, PAGE_SIZE);
427     else
428         thresh = (ratio * available_memory) / PAGE_SIZE;
429
430     if (bg_bytes)
431         bg_thresh = DIV_ROUND_UP(bg_bytes, PAGE_SIZE);
432     else
433         bg_thresh = (bg_ratio * available_memory) / PAGE_SIZE;
434
435     if (bg_thresh >= thresh)
436         bg_thresh = thresh / 2;
437     tsk = current;
438     if (rt_task(tsk)) {
439         bg_thresh += bg_thresh / 4 + global_wb_domain.dirty_limit / 32;
440         thresh += thresh / 4 + global_wb_domain.dirty_limit / 32;
441     }
442     dtc->thresh = thresh;
443     dtc->bg_thresh = bg_thresh;
```

```
444
445     /* we should eventually report the domain in the TP */
446     if (!gdtc)
447         trace_global_dirty_state(bg_thresh, thresh);
448 }
```

- 394 获取当前域脏页可用内存的大小，单位是: 页。
- 395 获取全局脏页控制结构 gdtc。
- 396 读取系统参数 vm.dirty_bytes。这是用户配置的脏页上限。
- 397 读取 vm.dirty_background_bytes。这是用户配置的后台回写阈值。脏页数量达到后台回写阈值后，内核会启动后台异步回写。
- 399 计算每页内存允许产生多少” 脏页配额”，单位是字节。

  vm_dirty_ratio 是脏页占系统可用内存的百分比阈值，控制脏页占可用内存达到多少时，写进程直接阻塞并强制刷盘。$\frac{vm_dirty_ratio}{100}$ 即表示 vm_dirty_ratio%(所以这里要除以 100)。
- 400 同上，把后台回写脏页阈值占可用内存的比例，换算成每页内存的回写阈值” 配额”。
- 406 gdtc 不为空时，当前脏页控制结构 dtc 隶属于内存组（memcg）域。这是为了区分全局脏页管控和cgroup 脏页管控两套逻辑。
- 407 获取目标域脏页可用内存大小。
- 416-417 目标 memcg 控制组可用脏页树木和全局可用脏页数是不相等的，因此重新计算每页内存允许产生多少”(memcg 控制组的) 脏页配额”。

  bytes 是用户配置的脏页上限 (单位: 字节)。DIV_ROUND_UP (a, b) 表示:a 除以 b，得到的结果向上取整。所以，DIV_ROUND_UP(bytes , global_avail) 代表: 脏页总字节上限 / 脏页总可占用页数 = 每页可允许脏字节数。因为一页要么干净，要么全脏，不可能出现：一页里脏字节 >4096。所以如果这个值 >PAGE_SIZE, 那么取 PAGE_SIZE(4096)。
- 419-421 同上。
- 425-428 如果用户配置了 vm.dirty_bytes, 那么直接用：字节数 / 页大小并按页面大小对齐，获取脏页上限阈值。若是用户没有配置，那么用” 比例 / 页 × 可用内存页的数目”，得到脏页阈值。
- 430-433 同上。同样的方法获取后台回写阈值。
- 435-436 处理异常情况：后台回写阈值被错误地设置得高于强制脏页阈值，这种情况是不允许的。此时强制后台回写阈值 = $\frac{1}{2}$ 上限阈值。
- 437-441 给实时任务 (RT 任务) 适当放宽脏页阈值，不让它们被磁盘写回卡住。实时任务不能被卡顿, 因此上限阈值和后台回写阈值提高 25%。再额外加一点点全局阈值的 1/32。保证实时进程流畅不阻塞
- 442-443 把最终计算的上限阈值和后台回写阈值分别保存起来, 给之后的脏页限流逻辑使用。
- 446-447 如果是全局域，就打印阈值日志用于调试的观测。

7.8 单设备的脏页阈值

每个 bdi_writeback 结构体专属的设备级脏页阈值，就是单个磁盘（单个设备）专属的脏页阈值在 wb_dirty_limits() 函数中计算。单个磁盘的脏页阈值不是硬限制！不像全局脏页阈值那样超过就卡死，超过单磁盘脏页阈值后会用温和限速慢慢降脏页。

7.8.1 wb_dirty_limits()

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *wb_dirty_limits()*

```
1507 static inline void wb_dirty_limits(struct dirty_throttle_control *dtc)
1508 {
1509     struct bdi_writeback *wb = dtc->wb;
1510     unsigned long wb_reclaimable;
1511
1512     /*
1513      * wb_thresh is not treated as some limiting factor as
1514      * dirty_thresh, due to reasons
1515      * - in JBOD setup, wb_thresh can fluctuate a lot
1516      * - in a system with HDD and USB key, the USB key may somehow
1517      *   go into state (wb_dirty >> wb_thresh) either because
1518      *   wb_dirty starts high, or because wb_thresh drops low.
1519      *   In this case we don't want to hard throttle the USB key
1520      *   dirtiers for 100 seconds until wb_dirty drops under
1521      *   wb_thresh. Instead the auxiliary wb control line in
1522      *   wb_position_ratio() will let the dirtier task progress
1523      *   at some rate <= (write_bw / 2) for bringing down wb_dirty.
1524      */
1525     dtc->wb_thresh = __wb_calc_thresh(dtc);
1526     dtc->wb_bg_thresh = dtc->thresh ?
1527         div_u64((u64)dtc->wb_thresh * dtc->bg_thresh, dtc->thresh) : 0;
1528
1529     /*
1530      * In order to avoid the stacked BDI deadlock we need
1531      * to ensure we accurately count the 'dirty' pages when
1532      * the threshold is low.
1533      *
1534      * Otherwise it would be possible to get thresh+n pages
1535      * reported dirty, even though there are thresh-m pages
1536      * actually dirty; with m+n sitting in the percpu
1537      * deltas.
1538      */
1539     if (dtc->wb_thresh < 2 * wb_stat_error()) {
1540         wb_reclaimable = wb_stat_sum(wb, WB_RECLAIMABLE);
1541         dtc->wb_dirty = wb_reclaimable + wb_stat_sum(wb, WB_WRITEBACK);
1542     } else {
1543         wb_reclaimable = wb_stat(wb, WB_RECLAIMABLE);
1544         dtc->wb_dirty = wb_reclaimable + wb_stat(wb, WB_WRITEBACK);
1545     }
1546 }
```

- 1509 获取当前磁盘的 bdi_writeback，后面所有计算只针对该磁盘。
- 1525 调用函数计算目标磁盘的脏页上限。系统全局有全局脏页阈值，每个磁盘有各自自己的阈值，由全局阈值按比例分配。
- 1526-1527 全局后台回写阈值也会按比例（单磁盘脏页阈值占全局脏页阈值的比例）分给单个磁盘。公式如下：

$$dtc->wb_bg_thresh = \frac{dtc->wb_thresh}{dtc->thresh} \cdot (dtc->thresh)$$

- 1539 如果单个磁盘的脏页上限阈值太低，小到比“统计误差”的 2 倍还小。进入高精度统计模式。会使用 wb_stat_sum() 而非 wb_stat()，强制汇总所有 CPU 的统计值，而不是使用常规的批量更新值。
- 1540-1542 高精度统计模式下，计算当前设备的总脏页数。当前设备的总脏页数 = 等待写回的脏页 + 正在写的脏页。

  wb_stat_sum() 会将所有 CPU 上 per CPU 的值累加统计，统计“可回收的脏页（等待写，WB_RECLAIMABLE)”和”正在写回页 (WB_WRITEBACK)”，但这种方式极准但慢。
- 1543-1544 快速模式下，计算当前设备的总脏页数。

  wb_stat() 读当前 CPU 上缓存的值，读取速度快但有误差。内核为了快，每个 CPU 会缓存脏页数量。

7.8.1.1 ___wb_calc_thresh()

___wb_calc_thresh() 函数计算每个单磁盘应该分到的脏页阈值的比例。把全局脏页阈值按磁盘 IO 的快慢（权重）分配给各个单磁盘。磁盘越快分到的脏页阈值越多，磁盘越慢分到脏页阈值越少。

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *wb_dirty_limits()* > *___wb_calc_thresh()*

```
758 static unsigned long __wb_calc_thresh(struct dirty_throttle_control *dtc)
759 {
760     struct wb_domain *dom = dtc_dom(dtc);
761     unsigned long thresh = dtc->thresh;
762     u64 wb_thresh;
763     unsigned long numerator, denominator;
764     unsigned long wb_min_ratio, wb_max_ratio;
765
766     /*
767      * Calculate this BDI's share of the thresh ratio.
768      */
769     fprop_fraction_percpu(&dom->completions, dtc->wb_completions,
770                   &numerator, &denominator);
771
772     wb_thresh = (thresh * (100 - bdi_min_ratio)) / 100;
773     wb_thresh *= numerator;
774     wb_thresh = div64_ul(wb_thresh, denominator);
775
776     wb_min_max_ratio(dtc->wb, &wb_min_ratio, &wb_max_ratio);
777
778     wb_thresh += (thresh * wb_min_ratio) / 100;
779     if (wb_thresh > (thresh * wb_max_ratio) / 100)
780         wb_thresh = thresh * wb_max_ratio / 100;
781
782     return wb_thresh;
783 }
```

- 760 这里暂时忽略 cgroup 相关逻辑，因此这里假设 dtc_dom() 返回的是代表整个系统统计数据的 global_wb_domain（全局回写域）。
- 769-770 计算目标磁盘脏页上限应该占全局脏页阈值的多少份额。numerator 和 denominator 是传出参数。
  - ***numerator*** 本磁盘的 IO 完成次数
  - ***denominator*** 所有磁盘总 IO 完成次数

- 772 预留一小部分 (bdi_min_ratio) 脏页阈值, 平均分给所有磁盘做保底脏页额度，剩下的再分配给各个单磁盘。换言之，先保底均分，再按硬盘性能差异化分配剩余额度。
- 773-774 按 IO 权重把全局脏页阈值分配给单磁盘。分配规则如下:

$$\text{单盘脏页阈值} = \text{全局脏页阈值} \times \frac{\text{目标磁盘 IO 权重}}{\text{全部磁盘总权重}}$$

- 776 获取这个磁盘的最小/最大脏页比例限制。防止一个盘脏页占太多，或太少。这些比例是可通过 sysfs 配置的参数。最小比例和最大比例的默认值分别为 0% 和 100%，因此如果没有手动修改这些参数，这部分调整逻辑不会产生实际效果。
- 778 加上最小保证比例, 保证每个磁盘至少有一点脏页空间。
- 779-780 限制单盘最大脏页上限, 不让某个硬盘占用全部全局阈值。

7.9 脏页轮询的间隔

dirty_poll_interval() 函数控制脏页回写的轮询间隔，决定内核多久检查一次脏页。

但这个返回的间隔不是实际时间，而是按”剩余可写页数”的平方根进行的动态调整。表示线程每累计脏化多少页，就要触发一次脏页平衡检查。

7.9.1 dirty_poll_interval()

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *dirty_poll_interval()*

```
1404 static unsigned long dirty_poll_interval(unsigned long dirty,
1405                       unsigned long thresh)
1406 {
1407     if (thresh > dirty)
1408         return 1UL << (ilog2(thresh - dirty) >> 1);
1409
1410     return 1;
1411 }
```

- 1407-1408 这个分支是在脏页小于上限阈值的情况下的处理。

  balance_dirty_pages() 会周期性检查是否需要启动脏页限流 (dirty throttling)。如果 dirty_poll_interval 的返回值设置得过低，大型 NUMA 机器会开销很大。因此，将该值按近似平方根的比例，与安全裕量 (safety margin)挂钩进行缩放。(安全裕量：指在不超过脏页限制的前提下，我们还能弄脏的页面数量)。

  1408 行代码运算如下:

$$1 \ll [(\log_2 (thresh - dirty)) \gg 1]$$

  也就是,

$$1 \ll \left[(\log_2 (thresh - dirty)) \cdot \frac{1}{2}\right]$$

等价于，

$$1 \ll \left[\log_2 (thresh - dirty)^{\frac{1}{2}}\right]$$

也就是，

$$2^{\left[\log_2 (thresh-dirty)^{\frac{1}{2}}\right]}$$

等价于，

$$2^{\log_2 (thresh-dirty)^{\frac{1}{2}}}$$

即，

$$(thresh - dirty)^{\frac{1}{2}}$$

即，

$$\sqrt{thresh - dirty}$$

由此可知，以上算法是对安全裕量开平方，作为脏页检查间隔。这样可以在裕量不足时动态伸缩，迅速缩小检测间隔，从而在越短间隔得知脏页情况，从而快速触发脏页限流。反之，在余量充足时，检测间隔变小会很平缓。

- 1410 脏页超出上限阈值，则将返回结果 1，以此促使脏页限流机制立即触发。

7.10 脏页速率限制值

7.10.1 wb_update_dirty_ratelimit()

该函数维护 wb->dirty_ratelimit，即基础脏页限流速率。

balance_dirty_pages_ratelimited() ⋯ *__wb_update_bandwidth()* *wb_update_dirty_ratelimit()*

```
1181 static void wb_update_dirty_ratelimit(struct dirty_throttle_control *dtc,
1182                                        unsigned long dirtied,
1183                                        unsigned long elapsed)
1184 {
1185     struct bdi_writeback *wb = dtc->wb;
1186     unsigned long dirty = dtc->dirty;
1187     unsigned long freerun = dirty_freerun_ceiling(dtc->thresh, dtc->bg_thresh);
1188     unsigned long limit = hard_dirty_limit(dtc_dom(dtc), dtc->thresh);
1189     unsigned long setpoint = (freerun + limit) / 2;
1190     unsigned long write_bw = wb->avg_write_bandwidth;
1191     unsigned long dirty_ratelimit = wb->dirty_ratelimit;
1192     unsigned long dirty_rate;
1193     unsigned long task_ratelimit;
1194     unsigned long balanced_dirty_ratelimit;
1195     unsigned long step;
1196     unsigned long x;
1197     unsigned long shift;
1198
1199     /*
1200      * The dirty rate will match the writeout rate in long term, except
1201      * when dirty pages are truncated by userspace or re-dirtied by FS.
1202      */
```

```
1203     dirty_rate = (dirtied - wb->dirtied_stamp) * HZ / elapsed;
1204
1205     /*
1206      * task_ratelimit reflects each dd's dirty rate for the past 200ms.
1207      */
1208     task_ratelimit = (u64)dirty_ratelimit *
1209                      dtc->pos_ratio >> RATELIMIT_CALC_SHIFT;
1210     task_ratelimit++; /* it helps rampup dirty_ratelimit from tiny values */
1211
1212     /*
1213      * A linear estimation of the "balanced" throttle rate. The theory is,
1214      * if there are N dd tasks, each throttled at task_ratelimit, the wb's
1215      * dirty_rate will be measured to be (N * task_ratelimit). So the below
1216      * formula will yield the balanced rate limit (write_bw / N).
1217      *
1218      * Note that the expanded form is not a pure rate feedback:
1219      *  rate_(i+1) = rate_(i) * (write_bw / dirty_rate)          (1)
1220      * but also takes pos_ratio into account:
1221      *  rate_(i+1) = rate_(i) * (write_bw / dirty_rate) * pos_ratio  (2)
1222      *
1223      * (1) is not realistic because pos_ratio also takes part in balancing
1224      * the dirty rate.  Consider the state
1225      *  pos_ratio = 0.5                            (3)
1226      *  rate = 2 * (write_bw / N)                      (4)
1227      * If (1) is used, it will stuck in that state! Because each dd will
1228      * be throttled at
1229      *  task_ratelimit = pos_ratio * rate = (write_bw / N)       (5)
1230      * yielding
1231      *  dirty_rate = N * task_ratelimit = write_bw           (6)
1232      * put (6) into (1) we get
1233      *  rate_(i+1) = rate_(i)                        (7)
1234      *
1235      * So we end up using (2) to always keep
1236      *  rate_(i+1) ~= (write_bw / N)                      (8)
1237      * regardless of the value of pos_ratio. As long as (8) is satisfied,
1238      * pos_ratio is able to drive itself to 1.0, which is not only where
1239      * the dirty count meet the setpoint, but also where the slope of
1240      * pos_ratio is most flat and hence task_ratelimit is least fluctuated.
1241      */
1242     balanced_dirty_ratelimit = div_u64((u64)task_ratelimit * write_bw,
1243                        dirty_rate | 1);
1244     /*
1245      * balanced_dirty_ratelimit ~= (write_bw / N) <= write_bw
1246      */
1247     if (unlikely(balanced_dirty_ratelimit > write_bw))
1248         balanced_dirty_ratelimit = write_bw;
1249
1250     /*
1251      * We could safely do this and return immediately:
1252      *
1253      *  wb->dirty_ratelimit = balanced_dirty_ratelimit;
1254      *
1255      * However to get a more stable dirty_ratelimit, the below elaborated
1256      * code makes use of task_ratelimit to filter out singular points and
1257      * limit the step size.
1258      *
1259      * The below code essentially only uses the relative value of
1260      *
1261      *  task_ratelimit - dirty_ratelimit
1262      *  = (pos_ratio - 1) * dirty_ratelimit
1263      *
1264      * which reflects the direction and size of dirty position error.
1265      */
1266
```

```
1267     /*
1268      * dirty_ratelimit will follow balanced_dirty_ratelimit iff
1269      * task_ratelimit is on the same side of dirty_ratelimit, too.
1270      * For example, when
1271      * - dirty_ratelimit > balanced_dirty_ratelimit
1272      * - dirty_ratelimit > task_ratelimit (dirty pages are above setpoint)
1273      * lowering dirty_ratelimit will help meet both the position and rate
1274      * control targets. Otherwise, don't update dirty_ratelimit if it will
1275      * only help meet the rate target. After all, what the users ultimately
1276      * feel and care are stable dirty rate and small position error.
1277      *
1278      * |task_ratelimit - dirty_ratelimit| is used to limit the step size
1279      * and filter out the singular points of balanced_dirty_ratelimit. Which
1280      * keeps jumping around randomly and can even leap far away at times
1281      * due to the small 200ms estimation period of dirty_rate (we want to
1282      * keep that period small to reduce time lags).
1283      */
1284     step = 0;
1285
1286     /*
1287      * For strictlimit case, calculations above were based on wb counters
1288      * and limits (starting from pos_ratio = wb_position_ratio() and up to
1289      * balanced_dirty_ratelimit = task_ratelimit * write_bw / dirty_rate).
1290      * Hence, to calculate "step" properly, we have to use wb_dirty as
1291      * "dirty" and wb_setpoint as "setpoint".
1292      *
1293      * We rampup dirty_ratelimit forcibly if wb_dirty is low because
1294      * it's possible that wb_thresh is close to zero due to inactivity
1295      * of backing device.
1296      */
1297     if (unlikely(wb->bdi->capabilities & BDI_CAP_STRICTLIMIT)) {
1298         dirty = dtc->wb_dirty;
1299         if (dtc->wb_dirty < 8)
1300             setpoint = dtc->wb_dirty + 1;
1301         else
1302             setpoint = (dtc->wb_thresh + dtc->wb_bg_thresh) / 2;
1303     }
1304
1305     if (dirty < setpoint) {
1306         x = min3(wb->balanced_dirty_ratelimit,
1307              balanced_dirty_ratelimit, task_ratelimit);
1308         if (dirty_ratelimit < x)
1309             step = x - dirty_ratelimit;
1310     } else {
1311         x = max3(wb->balanced_dirty_ratelimit,
1312              balanced_dirty_ratelimit, task_ratelimit);
1313         if (dirty_ratelimit > x)
1314             step = dirty_ratelimit - x;
1315     }
1316
1317     /*
1318      * Don't pursue 100% rate matching. It's impossible since the balanced
1319      * rate itself is constantly fluctuating. So decrease the track speed
1320      * when it gets close to the target. Helps eliminate pointless tremors.
1321      */
1322     shift = dirty_ratelimit / (2 * step + 1);
1323     if (shift < BITS_PER_LONG)
1324         step = DIV_ROUND_UP(step >> shift, 8);
1325     else
1326         step = 0;
1327
1328     if (dirty_ratelimit < balanced_dirty_ratelimit)
1329         dirty_ratelimit += step;
1330     else
```

```
1331            dirty_ratelimit -= step;
1332
1333        wb->dirty_ratelimit = max(dirty_ratelimit, 1UL);
1334        wb->balanced_dirty_ratelimit = balanced_dirty_ratelimit;
1335
1336        trace_bdi_dirty_ratelimit(wb, dirty_rate, task_ratelimit);
1337    }
```

- 1186 当前系统的脏页总数。
- 1187 计算获取自由写脏页的上限。脏页总数低于这个值进程可以随便写，不限速。
- 1188 从全局和回写域的脏页上限阈值中获取最大值来作为脏页硬上限。
- 1189 计算获取目标稳定点的脏页数量。
- 1190 获取磁盘平均写带宽。
- 1203 计算回写域的实际脏页生成速度。

  $dirtied - wb- > dirtied_stamp$ 得到的是这段时间 (elapsed) 新增的脏页数目。公式可以如下变形:

$$
\begin{aligned}
dirty_rate &= (dirtied - wb- > dirtied_stamp) \times HZ \div elapsed \\
&= \frac{(dirtied - wb- > dirtied_stamp) \times HZ}{elapsed} \\
&= \frac{dirtied - wb- > dirtied_stamp}{\frac{elapsed}{HZ}} \\
&= \frac{dirtied - wb- > dirtied_stamp}{\Delta_t} \\
&= \frac{\Delta_{pages}}{\Delta_t}
\end{aligned}
$$

  因此得到每秒产生脏页的数量 (即速率)。

- 1208 计算单个进程脏页限流速率 task_ratelimit。

  此处用基础限速值 × 调速系数得到单个进程的脏页限速值。

  和前面提到的类似: 这里 RATELIMIT_CALC_SHIFT 值为 10，所以意味着要将上式的结果再除以 2^{10} ，得到真正的限速值。这是因为调速系数 $pos_ratio \in [0,1]$ 之间，但是内核不能做浮点数计算，所以，在之前计算 pos_ratio(调速系数) 时将其放大了 1024。因此此处右移 10 位 (除以 2^{10}) 将结果还原。

- 1210 单个进程的脏页限速值 +1, 这是为了防止 0 值, 导致后面除以 0 导致崩溃。
- 1242-1243 考虑当前磁盘写入带宽，修正计算回写域的理想限速值。**回写域的理想限速值**的计算公式为:

  单个进程的限速值 × (**磁盘总带宽** / **实际脏页产生速度**)

即，

$$\begin{aligned} balanced_dirty_ratelimit &= \frac{task_ratelimit \cdot write_bw}{dirty_rate} \\ &= task_ratelimit \cdot \frac{write_bw}{dirty_rate} \end{aligned} \tag{1}$$

设 B=$write_bw$,D=$dirty_rate$

那么，

$$balanced_dirty_ratelimit = task_ratelimit \cdot \frac{B}{D}$$

- $D > B$: 当前实际脏页产生速度超过磁盘写入能力 (带宽)。$\frac{B}{D} < 1$, 倍率小于 1, 降速，对写 page cache 限流，避免脏页持续堆积爆满。
- $D < B$: 写脏页任务负载偏轻，磁盘带宽空闲富余。$\frac{B}{D} > 1$, 倍率大于 1, 对写 page cache 提速。
- $D = B$: 实测速率刚好匹配磁盘带宽 $\frac{B}{D} = 1$，倍率为 1，速度维持不变，处于流量均衡。

假定系统里有 N 个进程同时写文件。每个进程被限制为：task_ratelimit. 那么总写速度为：

$$dirty_rate = N \cdot task_ratelimit \tag{2}$$

(2) 代入 (1) 式，得:

$$\begin{aligned} balanced_dirty_ratelimit &= task_ratelimit \cdot \frac{write_bw}{dirty_rate} \\ &= task_ratelimit \cdot \frac{write_bw}{N \cdot task_ratelimit} \\ &= \frac{write_bw}{N} \end{aligned}$$

所以，这里算出一个理想限速，让 N 个进程平分磁盘带宽。通过计算过程可以看出，这里得到的**回写域的理想限速值**更精确的意思是回写域中**每个进程**的理想限速或基准限速值。

1243 行 $dirty_rate|1$ 是为了避免 $dirty_rate = 0$ 时，除以 0 产生崩溃。

- 1247-1248 理论上，计算出来的平衡 (理想) 限速 balanced_dirty_ratelimit 约等于磁盘总带宽 / 进程数 N, 而且它一定不会超过磁盘总带宽 write_bw。如果算出来的限速竟然超过了磁盘能承受的最大速度，那就强制把它限制为磁盘最大写入带宽，不允许超过。
- 1284 前面算出了理论最优值。本可以直接执行下面这行代码并立刻返回：

$$wb->dirty_ratelimit = balanced_dirty_ratelimit;$$

但是实测脏页速率 dirty_rate 的采样窗口短 (200ms)，波动大，直接硬改会让限速剧烈跳变，引发 IO 抖动。于是引入分步平滑逼近，核心就是算 step 调整步长。这里默认初始无调整量 (step=0)。

- 1297-1303 对于严格限制模式 (strictlimit)，所有计算都是基于 wb (设备) 自己的统计值和限制值 (从 wb_position_ratio 开始，一直算到 balanced_dirty_ratelimit)。

因此，为了正确计算 step (调整步长)，必须使用 wb_dirty 当作当前脏页数，使用 wb_setpoint 当作目标脏页数。换言之，保证在这种模式下，脏页统计、目标值都用设备自己的。

  - 1299-1230 如果脏页数特别少, 把目标值 setpoint 也设低。目的是防止设备闲置太久变卡顿。同理，1300 行这里 +1 是为了防止 setpoint 为 0，造成除以 0 的场景。
  - 1301-1302 否则正常计算目标值, 取两个阈值的中间值作为目标。
- 1305-1310 如果脏页 < 目标值, 脏页太少, 因此可以加快写入速度。
  - 1306-1307 取最保守、最小的值, 防止提速太快导致抖动。
  - 1308-1309 如果当前速度比 x 小, 计算提速步长。
- 1310-1315 如果脏页 ≥ 目标值, 需要降速。
  - 1311-1312 取最大、最安全的值。降速不要太猛，避免卡顿
  - 1313-1314 如果当前速度比 x 大，计算降速步长。
- 1322 计算一个衰减系数 shift, 决定接下来的调整幅度要缩小多少倍。不追求速率做到百分百精准匹配，当数值越接近目标值，就越降低追踪调整的幅度，以此消除无意义的小幅震荡波动。
- 1323-1324 如果计算获取的 shift 的值在合理范围内 (不会溢出), 那么步长除以 2^shift, 再除以 8。这个算法导致的结果就是平滑调节:

  回写域的基准限速值离理想速度值远时，调整幅度就会大一点；反之，离目标点近，调整幅度会极小极小。这样就防止了速度剧烈抖动。
- 1325-1326 非常接近目标了，直接不调整。
- 1328-1331 如果当前基准限速阈值 > 之前的基准限速阈值, 那么提高之前的基准阈值; 如果当前基准限速阈值 < 旧的基准阈值, 降低旧的基准阈值。

  如前所讨论，**基准脏页限速阈值**是在当前回写域有 N 个进程写脏页，考虑回写域的磁盘带宽限制，计算得到的每个进程/线程的理想限速值。
- 1333 确保基准限速阈值最小是 1，绝对不会变成 0。
- 1334 把这一轮算出的理想平衡阈值存回 wb 结构体, 下一次调整时，可以继续用它做参考。这样就形成闭环控制，让阈值一步步逼近最优状态。

7.11 写 page cache 线程的暂停间隔

为最大化磁盘 IO 吞吐量，内核在处理文件写入请求时，并不会直接将数据同步写入后备存储，而是会先将数据暂存在页缓存（page cache）中，形成临时的脏页集合；待脏页总量到达预设的内存占用阈值、或脏页在内存中的停留时长超过预设的落盘延迟后，才会批量将脏页回写至磁盘。

这一机制的关键，是精准平衡“用户进程的写操作速度”与“磁盘的实际回写速度”——如果用户进程产生脏页的速度，持续超过磁盘的回写能力上限，将导致脏页量持续积压、最终突破内存安全阈值；此时内核会被迫进入强制回写流程，直接阻塞用户进程的写入操作，甚至引发整系统的 IO 吞吐量衰减。

wb_min_pause() 和 wb_max_pause() 函数的核心作用，正是在这一场景下，为用户进程计算出合理的暂停时长区间——通过让造脏速度过快的进程暂停片刻，将全局造脏速率，精准压制到与磁盘实际回写能力匹配的水平线上。

7.11.1 wb_min_pause()

balance_dirty_pages_ratelimited() > balance_dirty_pages() > wb_min_pause()

```
1432 static long wb_min_pause(struct bdi_writeback *wb,
1433              long max_pause,
1434              unsigned long task_ratelimit,
1435              unsigned long dirty_ratelimit,
1436              int *nr_dirtied_pause)
1437 {
1438     long hi = ilog2(wb->avg_write_bandwidth);
1439     long lo = ilog2(wb->dirty_ratelimit);
1440     long t;     /* target pause */
1441     long pause; /* estimated next pause */
1442     int pages;  /* target nr_dirtied_pause */
1443 
1444     /* target for 10ms pause on 1-dd case */
1445     t = max(1, HZ / 100);
1446 
1447     /*
1448      * Scale up pause time for concurrent dirtiers in order to reduce CPU
1449      * overheads.
1450      *
1451      * (N * 10ms) on 2^N concurrent tasks.
1452      */
1453     if (hi > lo)
1454         t += (hi - lo) * (10 * HZ) / 1024;
1455 
1456     /*
1457      * This is a bit convoluted. We try to base the next nr_dirtied_pause
1458      * on the much more stable dirty_ratelimit. However the next pause time
1459      * will be computed based on task_ratelimit and the two rate limits may
1460      * depart considerably at some time. Especially if task_ratelimit goes
1461      * below dirty_ratelimit/2 and the target pause is max_pause, the next
1462      * pause time will be max_pause*2 _trimmed down_ to max_pause.  As a
1463      * result task_ratelimit won't be executed faithfully, which could
1464      * eventually bring down dirty_ratelimit.
1465      *
1466      * We apply two rules to fix it up:
1467      * 1) try to estimate the next pause time and if necessary, use a lower
1468      *    nr_dirtied_pause so as not to exceed max_pause. When this happens,
1469      *    nr_dirtied_pause will be "dancing" with task_ratelimit.
1470      * 2) limit the target pause time to max_pause/2, so that the normal
1471      *    small fluctuations of task_ratelimit won't trigger rule (1) and
1472      *    nr_dirtied_pause will remain as stable as dirty_ratelimit.
1473      */
1474     t = min(t, 1 + max_pause / 2);
1475     pages = dirty_ratelimit * t / roundup_pow_of_two(HZ);
1476 
1477     /*
1478      * Tiny nr_dirtied_pause is found to hurt I/O performance in the test
1479      * case fio-mmap-randwrite-64k, which does 16*{sync read, async write}.
1480      * When the 16 consecutive reads are often interrupted by some dirty
1481      * throttling pause during the async writes, cfq will go into idles
1482      * (deadline is fine). So push nr_dirtied_pause as high as possible
1483      * until reaches DIRTY_POLL_THRESH=32 pages.
1484      */
1485     if (pages < DIRTY_POLL_THRESH) {
1486         t = max_pause;
1487         pages = dirty_ratelimit * t / roundup_pow_of_two(HZ);
```

```
1488            if (pages > DIRTY_POLL_THRESH) {
1489                pages = DIRTY_POLL_THRESH;
1490                t = HZ * DIRTY_POLL_THRESH / dirty_ratelimit;
1491            }
1492        }
1493
1494        pause = HZ * pages / (task_ratelimit + 1);
1495        if (pause > max_pause) {
1496            t = max_pause;
1497            pages = task_ratelimit * t / roundup_pow_of_two(HZ);
1498        }
1499
1500        *nr_dirtied_pause = pages;
1501        /*
1502         * The minimal pause time will normally be half the target pause time.
1503         */
1504        return pages >= DIRTY_POLL_THRESH ? 1 + t / 2 : t;
1505    }
```

- 1438 磁盘回写平均速度 (带宽) 的对数等级。
- 1439 回写域的写脏页限速上限的对数等级。值得注意的是: 这个限速值是该回写域算出的，该回写域中写脏页的各个 task 的限速上限 (基准限速)，而不是限速总和。(参考 7.9.1 wb_update_dirty_ratelimit() 函数)

  这个限速上限是该回写域 (磁盘) 理想状态下平均单个业务进程写脏页速度。是内核的**管控速度**。由 wb_update_dirty_ratelimit() 每 200ms 左右根据磁盘实测带宽，当前脏页水位，活跃进程数对该值进行平滑更新；初始在 wb_init() 中设默认值。
- 1445 设置基准的默认目标暂停时间 (t = 10ms) 对应的节拍。
- 1453-1454 这是为了解决当大量进程同时写入时，单个进程的暂停时间过短导致写进程频繁休眠唤醒,cpu 效率低下的问题。并且无法有效抑制总体的脏页生成速率。因此随并发脏页生产者数量增加而提升最小暂停时间。当存在大量并发的脏页生成者时，balance_dirty_pages() 会被极其频繁地调用，从而消耗大量 CPU 周期。因为多个任务同时产生脏页，每个任务都会在达到一定脏页阈值后触发调用平衡函数 balance_dirty_pages()，导致大量 CPU 周期被消耗在频繁的调度和计算上。通过以下方式修复该问题:
  - 引入一个随任务数的对数进行缩放的最小暂停时间。
  - 每当并发任务数增加 2^N 时，就将最小暂停时间增加一个固定的量子间隔 10 毫秒。

  即,

$$task_num = \frac{avg_write_bandwidth}{drity_ratelimit_{thread}}$$

  也就是,

$$2^N = \frac{avg_write_bandwidth}{drity_ratelimit_{thread}}$$

  内核按上式根据实时的写入负载 (带宽) 动态估算并发任务取对数 (即所谓的 “N” 值)，也就是 hi 比 lo 大多少对数等级 (即 N)，就增加多少个 10ms 的节拍。任务越多，每次暂停的下限就越高，从而强制降低 balance_dirty_pages() 的调用频率，节省 CPU 资源。

体现了其核心逻辑：暂停时间的增长需要与并发任务数相匹配。内核在观察到实际写入带宽后，通过 ilog2 对带宽差距取对数，并将其转化为线性的时间增量，最终叠加到目标暂停时间上。并发越多，每次睡越久，检查频率越低。

变量	意义	行为方向
wb->avg_write_bandwidth	回写域实测平均脏页回写带宽，是脏页实际刷入存储设备的真实平均速率	脏页写存储盘，流出速度
wb->dirty_ratelimit	单磁盘单任务脏页生成速率上限，是该回写域的单进程每秒允许生成脏页的理想速率	脏页生成，单线程理想流入上限
task_ratelimit	分摊给当前进程的脏页生成速率上限	脏页生成，单进程流入上限

- 目标暂停不超过最大暂停的一半，防止波动太大。
- 在时长为 t 的暂停时间 (节拍) 内，按照 dirty_ratelimit 的上限阈值速率，一共会产生多少页脏页。
- DIRTY_POLL_THRESH(这里是 32) 是暂停一次的时间内至少要能产生的理论脏页数。如果低于该值，进程就会不停的写、暂停、写、暂停……，几乎变成高频轮询，造成资源浪费。

  如果理论脏页数 < 理论脏页最小值，那么把暂停时间拉到最大 max_pause。重新计算 page，把理论脏页数也拉大。

  如果修正计算出的 page 数此时 > 理论脏页最小值，那么强制 page= 理论最小值。再反过来通过 page 计算目标暂停时间。
- 单进程限速值 task_ratelimit 算出：在回写域上限速率阈值下暂停时间段产生的理论脏页数 pages，在单进程限速阈值下会暂停多久。
- 如果算出的暂停时间 > 最大暂停，那么强制暂停时间为最大暂停时间。并通过这个暂停时间反向算出单进程限速值下的理论产生的脏页数。
- 输出最终的脏页暂停阈值。
- 返回最小暂停时间。如果理论脏页 ≥ 阈值则返回 t/2，否则就直接返回 t。返回 t/2 是为了平滑限流、防抖动。

7.11.2 wb_max_pause()

balance_dirty_pages_ratelimited() > *balance_dirty_pages()* > *wb_max_pause()*

```
1413 static unsigned long wb_max_pause(struct bdi_writeback *wb,
1414                  unsigned long wb_dirty)
1415 {
1416     unsigned long bw = wb->avg_write_bandwidth;
1417     unsigned long t;
1418
1419     /*
1420      * Limit pause time for small memory systems. If sleeping for too long
1421      * time, a small pool of dirty/writeback pages may go empty and disk go
1422      * idle.
```

```
1423        *
1424        * 8 serves as the safety ratio.
1425        */
1426       t = wb_dirty / (1 + bw / roundup_pow_of_two(1 + HZ / 8));
1427       t++;
1428
1429       return min_t(unsigned long, t, MAX_PAUSE);
1430   }
```

- 1416 通过指数平滑计算得出的设备真实稳态写带宽。区别于瞬时带宽，该值过滤了 IO 抖动，保证限流算法平稳不抖动。
- 1426 计算限制小内存系统的暂停时间。若休眠时间过长，少量的脏页/回写页 (page cache) 池可能很快被写回磁盘或后备存储，导致无脏页写回从而磁盘空闲。此处数字 8 是一个经验值被用作比率系数。目的是动态缩短修正小内存系统的最大暂停时间。

  代码的公式如下 (此处以 jiffies 节拍作为时间的单位):

$$t = \frac{wb_dirty}{1 + \frac{bw}{roundup_pow_of_two\left(1+\frac{HZ}{8}\right)}}$$

  这个公式里两个 +1 都是为了避免分母值在某些情况下为 0, 为了简化理解，现在把它进行简化，得,

$$\begin{aligned} t &= \frac{wb_dirty}{\frac{bw}{\frac{HZ}{8}}} \\ &= \frac{wb_dirty}{bw \cdot \frac{8}{HZ}} \\ &= \frac{wb_dirty}{bw} \cdot \frac{1}{8} \cdot HZ \end{aligned}$$

  理论上 $\frac{wb_dirty}{bw}$(即, 脏页/平均带宽)，得到的结果便是时间 t', 再乘以经验参数 $\frac{1}{8}$, 得到的 t 即是时间。

- 1427 这里 +1 是为了避免 $t == 0$, 从而导致之后除以 0 导致 crash。
- 1429 MAX_PAUSE 固定为 200ms（HZ/5)。无论脏页多少、磁盘多慢，最大休眠绝不超过 200ms。这里限制最大休眠时间不超过 200ms 的核心目的是: 死守系统响应性，避免前台进程长期卡顿、卡死。

8

swapcache

内存过量使用机制与请求分页机制，使得 Linux 允许进程映射的内存总量，超过系统实际的物理内存容量。这就意味着，当内存被大量占用时，内核必须执行内存回收，以响应后续的内存申请请求。交换分区 swap 的作用，就是将匿名页的内存数据写入磁盘，以此完成这类内存的回收。

当需要从页缓存 (pagecache) 中回收内存时，内核可以直接丢弃其中的干净页，快速释放内存；代价是后续再次访问该页面时，会触发主缺页异常。但匿名内存页无法直接丢弃，必须先换出至磁盘，待后续访问时再重新换入内存。

狭义的 swapcache 一般指有 swap 分区副本的物理内存中的匿名页本身。**广义的 swapcache** 指 RAM 中的匿名页 + 管理结构体：swap entry(交换项，记录页面在 swap 磁盘分区的位置)、页表项、哈希链表、页描述符等。**swap 或 swap 空间**指在磁盘上真正“把内存写到硬盘”的那块空间。内核通过交换项 (swap entry) 标记已换出的内存，同时借助交换缓存 (swapcache) 暂存这类可以被换出的匿名数据。

swapcache 和 pagecache 的区别:

- ***pagecache*** 是磁盘上的文件页在内存中的缓存 (cache)，对应的磁盘文件持有该 cache 的引用。
- ***swapcache*** 是 swap 分区或 swap 文件在内存中的缓存 (cache)，对应的 swap 磁盘上的文件持有该 cache 的引用。

一个内存中的匿名页，必须已经成功写入磁盘 swap 文件、磁盘存有该页的完整副本，才会被归入 swapcache。换言之，匿名页只有在首次被换出至 swap(交换空间) 后，才会加入 swapcache(交换缓存)。顾名思义，swapcache 是 swap 分区上的数据在 RAM 中的缓存 (cache)。下面分场景说明:

- **从未换出到磁盘**，普通匿名页 (进程堆/栈等) 仅存于物理内存，磁盘无对应副本。那么该匿名页只是普通匿名页，不属于 swapcache。
- **正在换出**，页面正在往磁盘 swap 写，但写入未完成、磁盘无有效副本。该匿名页仍不算 swapcache。
- **换出完成**，磁盘或后备存储已有该匿名页副本，内存中暂时还保留着该页面，那么该页已经正式成为 swapcache 页。

- **在 swapcache 中被修改**，页面在 swapcache 阶段被修改，磁盘上的旧副本失效。此时该页退出 swapcache，变回普通匿名页；下次 swapout 需要重新写盘 (如果是从 5.1.13跳转参考此章节，那么此处可以跳回)。

NOTE

swap 或 swap 分区不是用来扩充系统内存容量、充当内存备用空间的工具。
它的核心作用只有一个：是为了实现匿名页的回收，进而平衡匿名页与页缓存 (pagecache, 文件页缓存) 之间的内存压力占比。

swapcache(交换缓存)由数组 swapper_spaces 定义。该数组共有 MAX_SWAPFILES 个指针元素，每个指针又指向一个包含 nr_swapper_spaces[**i**] 个 address_space 结构体的数组 (见下图)(其中 **i** 表示是第 **i** 个 swap 文件，同时又是数组中该指针的索引)。swapper_spaces 可以看成是一个二维数组：第一维长度固定，第二维动态分配；对于第 i 个交换文件，其第二维数组长度由 nr_swapper_spaces[i] 的值指定。swapper_spaces 中的每一个指针指向的动态数组对应某一个 swap 交换文件或交换分区的可用空间 (或直接称为 swap)。

struct address_space 是页的“数据源”在内存里的管理员。struct page->mapping 是指向这个管理员的指针。

address_space 是“页缓存管理者”，每一个能被缓存到内存的数据源，都有一个 address_space。

page_mapping()

```
677 struct address_space *page_mapping(struct page *page)
678 {
679     struct address_space *mapping;
680 
681     page = compound_head(page);
682 
683     /* This happens if someone calls flush_dcache_page on slab page */
684     if (unlikely(PageSlab(page)))
685         return NULL;
686 
687     if (unlikely(PageSwapCache(page))) {
688         swp_entry_t entry;
689 
690         entry.val = page_private(page);
691         return swap_address_space(entry);
692     }
693 
694     mapping = page->mapping;
695     if ((unsigned long)mapping & PAGE_MAPPING_ANON)
696         return NULL;
697 
698     return (void *)((unsigned long)mapping & ~PAGE_MAPPING_FLAGS);
699 }
700 EXPORT_SYMBOL(page_mapping);
```

- 681 如果是复合页 (大页)，先找到首页 (head page)。
- 684-685 slab 页是内核自己的“内部小内存池”，不归文件、不归进程、不归交换。设置 *PG_mapping* 标记后，page->mapping 字段不再用于 address_space，该 union 区域会交给 slab 保存 slab 元数据。

下面是几种不同页和 mapping 的关系列表：

页	归属	页的数据源	*struct address_space?*
文件页	用户进程	文件 *(磁盘)*	有,*struct page->mapping=&inode->i_data(指向文件的 address_space)*
匿名页	用户进程	无	无,*struct page->mapping=(void *)1*
swapcache 页	用户进程	*swap(file* 或分区*)*	无,*struct page->mapping=&swapper_space*
slab 页	内核线程	无	无,*page->mapping=NULL*

NOTE

page->mapping 的最低位的意义

page->mapping 的最低位为 *1* 时，表示该页是匿名页，同时无 *swap(*也就是*:* 不是 *swapcache* 页*)*。换言之，*swapcache* 页的 *page->mapping* 最低位是 *0*。

- 687-692 这个代码段针对 swapcache 页进行处理。
  - 690 从 page->private 中取出 swap entry(类型为 swap_entry_t), 通过对该值偏移和掩码可以分别得到: 目标 page 所在 swapfile 在 swapper_space 数组中的索引，和这个索引成员指向的 address_space 数组中成员的偏移。

    这个 swap_entry_t 是 unsigned long 类型表示一个编码值 ($type + offset$):

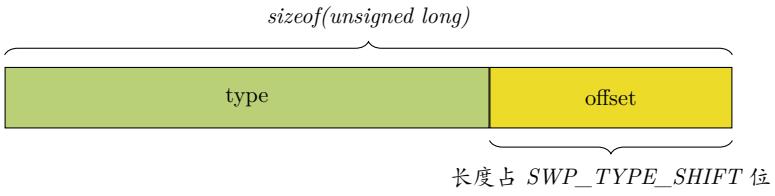
    * **type**: 哪个 swap 分区。这里的 type 是内核代码中的名称, 其实更确切的名称应该是 index。
    * **offset**: 目标分区里的 slot 号，或者可以理解为目标 swap file 的第几页 (页编号).

    swap_entry_t 在 swp_entry() 中进行编码组合, 如下:

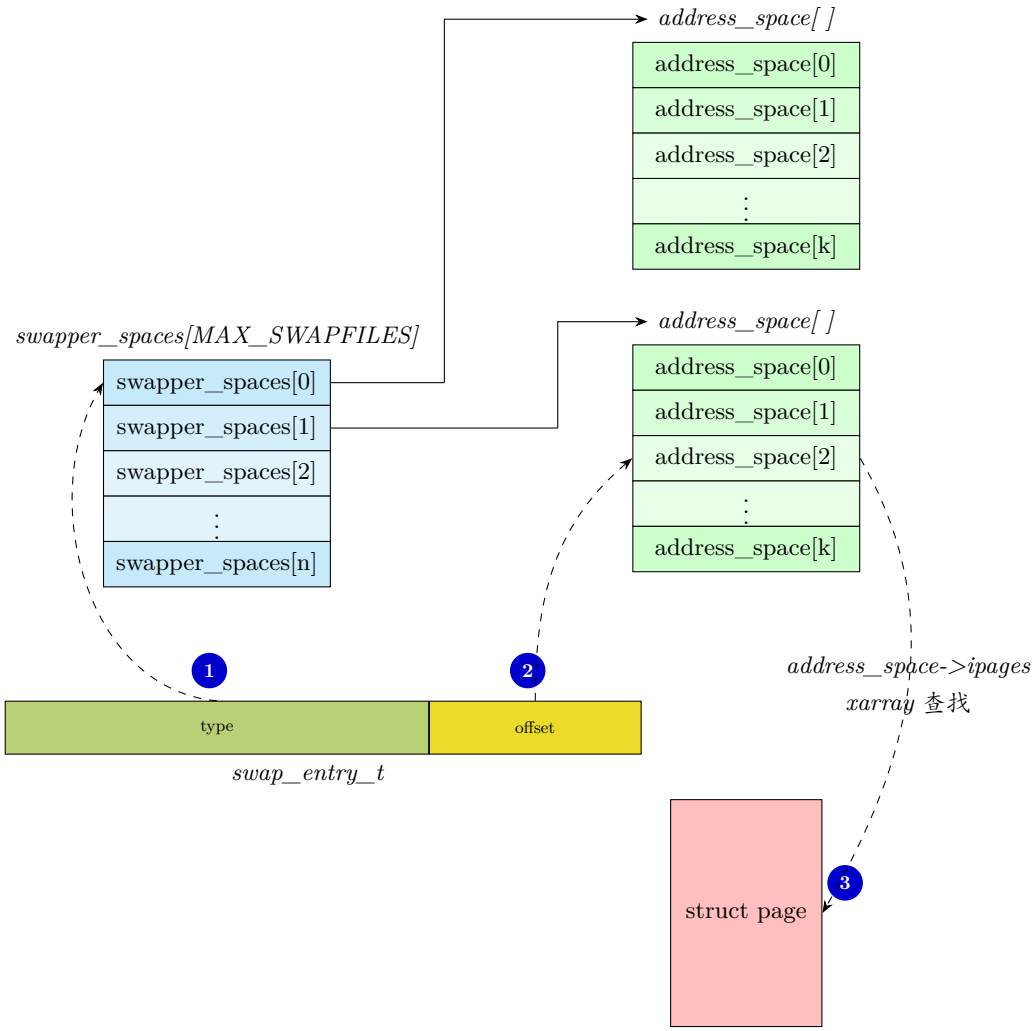
```
static inline swp_entry_t swp_entry(unsigned long type, pgoff_t offset)
{
    swp_entry_t ret;

    ret.val = (type << SWP_TYPE_SHIFT) | (offset & SWP_OFFSET_MASK);
    return ret;
}
```

    该函数比较简单 (解析略)，swap_entry_t 编码格式如下图所示:



利用 swap_entry_t 在全局数组 swapper_spaces 中找到对应的 struct page。address_space 是中间桥梁，用 XArray 存 slot→page 的映射。



说明:
1. 以 type 作为全局数组 swapper_spaces[ ] 的 index。数组中第 type 个成员，指向第 type 个 swap 分区。
2. offset 是目标 slot 在 swap 分区内的编号。
$offset \gg SWAP_ADDRESS_SPACE_SHIFT$(14) 得到目标 swap 分区对应的 address_space 数组 (第二维) 的下标.
每一个数组成员管理 64MB 分片。
3. 通过计算$offset$ & $0x3fff$ 可以得到目标 64M 分片内的页号，从而找到目标页。

另外，需要注意的是: 仅当 page 驻留在内存、位于 swapcache 中，address_space->i_pages(XArray) 才保存 slot 到 struct page 的映射；如果 page 已经被伙伴系统回收，XArray 中该 slot 条目会被擦除。此时 swp_entry_t 会编码保存在进程的 PTE 里，记录数据在 swap 磁盘的 slot 等。缺页异常时需要重新分配新 page, 发起 swapin 重新进行磁盘 IO 读数据。

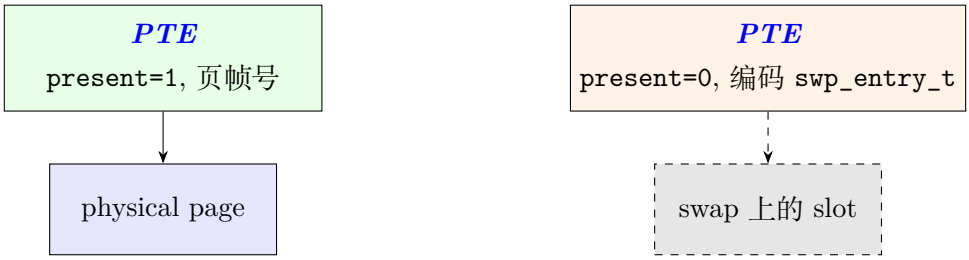
- 695-696 如果是匿名页就没有对应数据源或 address_space，返回 NULL。

  正如前述，address_space 是内存中的页缓存对数据源的管理员。

- 698 排除了前面各种类型的页，现在剩下文件页，把文件页 mapping 里的低位标记位 (flag bits) 去掉返回真正的、干净的 address_space 指针。

内存被换出 (swap out) 时，进程的虚拟地址到物理内存的映射并没有被删掉！只是把页表项 (PTE), 从指向物理页改成了指向 *swap_entry_t*。

swapcache 的 PTE 的核心特征是：其存在位 (present bit) 为 0，但页表项本身并非全 0。也就是说，页表项里存了 swap 信息，但页面不在物理内存中。present bit = 0，意味着页面不在内存里 (页面实际在 swap 分区中); 但 PTE 不是全 0, 里面存了 swp_entry_t(见前面解释)。所以它既不是有效内存页，也不是空页，而是专门表示页面被换出到磁盘的特殊页表项。



说明:
swapout 换出过程中的 PTE 状态变更: 虚拟地址映射关系仍然保留，
PTE 被重写，存入 swp_entry_t，同时清除 present 位。。

下面是不同类型页的 PTE 的比较:

页类型	*PTE* 值的意义	*present* 位	含义
正常内存页	物理页帧号	*1*	页面在内存中
空页	全 *0*	*0*	内存无映射页
swapcache 页	*swap_entry_t* 值	*0*	页面在 *swap* 磁盘分区

从上面可知 swap_entry_t 在 page->private 和 PTE 中都有保存。但他们的用途不一样:

当匿名页被换出到 swap 时：PTE 的位 *present* = 0,PTE 的其他位编码成 swp_entry_t。进程下次访问该虚拟地址, 会产生缺页中断，然后从 PTE 读出 swp_entry_t，利用该编码项去 swap 磁盘读回该页。也就是说 PTE 里的 swp_entry_t 是给 MMU/缺页中断用的。

而 page->private 的 swap_entry_t 是给 swapcache、回收逻辑用的。

9

反向映射

执行内存回收或者内存迁移时，操作粒度是 page(页)。我们需要能够高效查找映射该内存的各个 PTE，但是要找到 PTE 首先必须找到 VMA (*vm_area_struct*，进程虚拟地址区间)。这种从 page 反向查找对应 VMA 的机制，称为**反向映射(reverse mapping)**。

task_struct->mm 指向进程的地址空间描述符 *mm_struct*。*mm_struct->mmap* 是 VMA 单向链表头指针，指向第一个 *vm_area_struct*。各个 vma 通过成员 *vm_next* 串成单链表，按虚拟地址从小到大排序。*mm_struct->task* 反向指回所属 ***task_struct***。其中每一个 vma 成员代表目标进程虚拟地址上的不同区间。

只要物理页被用户进程 PTE 映射 (*_mapcount* ≥ 0)，不管是匿名还是文件页，回收时都必须靠反向映射找到全部 PTE 并解除映射。

内核经常的操作：查找映射了指定页的全部 VMA。譬如：通过 *try_to_unmap()*进行内存回收，把待回收页面从所有引用它的 VMA 中解除映射。因此必须找出该页面被映射到了哪些 VMA 中 (要注意：fork 写时复制机制下，单个匿名页可以被多个进程的 VMA 同时映射)。这就是页的反向映射。简言之，就是通过物理页找到所有映射它的虚拟地址。

NOTE

反向映射为什么内核要先找 *VMA*，而不是直接找 *PTE*？

内核只有物理地址，没有虚拟地址；*PTE* 是隶属于进程 + 虚拟地址的，没有 *VMA* 就找不到 *PTE* 在哪里。
要找到一条 *PTE*，必须知道两个信息：
1. 属于哪一个进程 *(task_struct->mm_struct)*，*VMA(vm_area_struct)* 属于某个进程的 *mm_struct*
2. 对应的虚拟地址 *(vaddr)*
然后页表遍历：*mm* → *pgd* → *pud* → *pmd* → *pte*，才能拿到 *PTE* 项。

与之相对，**正向映射(forward mapping)** 指 *task_struct* 通过虚拟地址拿到对应的 PTE、物理页 (或 struct page)。后面通过 mm_struct->pgd 来查找页表和后面反向映射的步骤一致。

之所以必须找到 VMA(vm_area_struct)。是因为 VMA 提供了地址合法性验证和权限检查，这是保证内存安全和管理正确性的关键步骤。

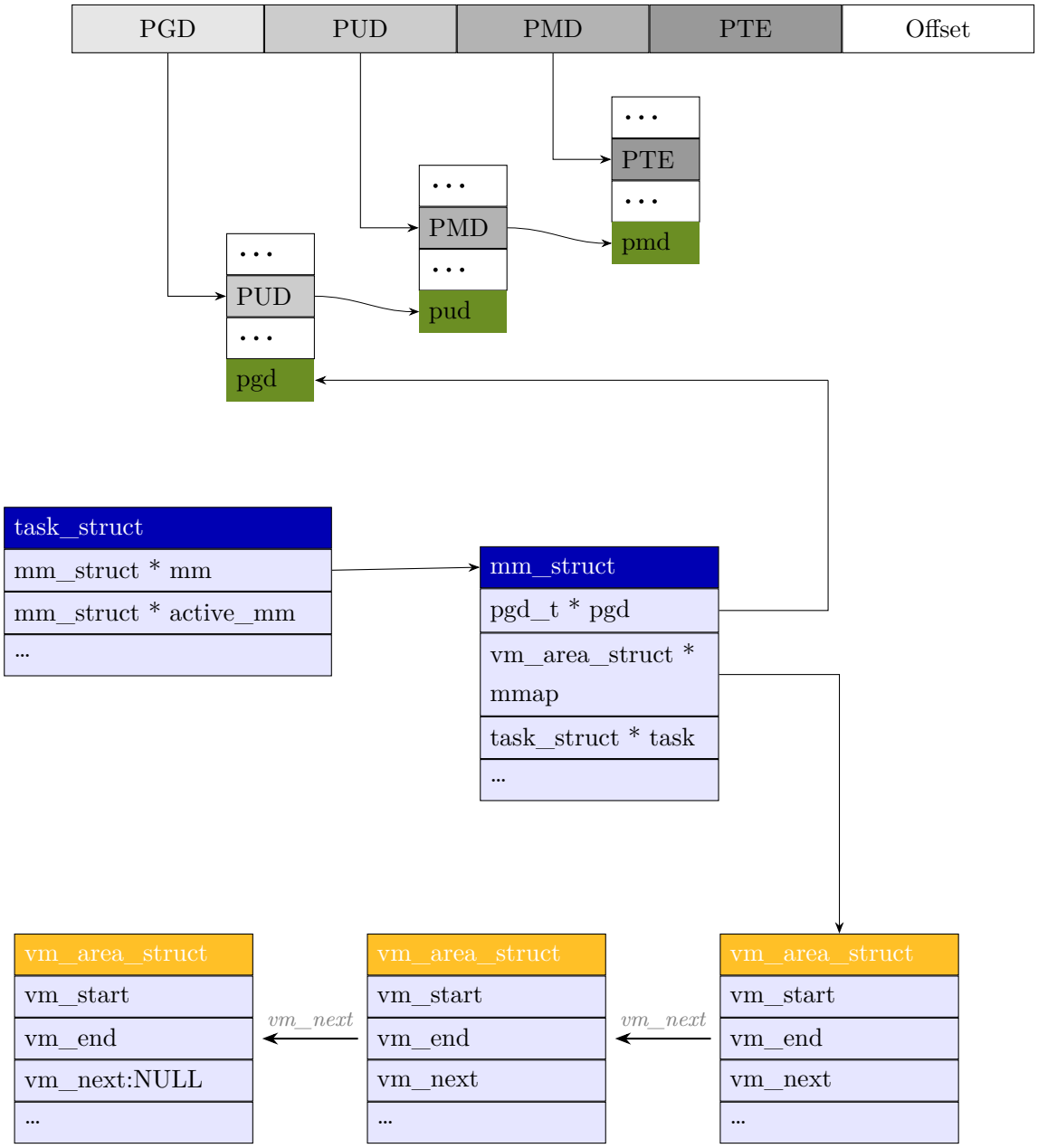
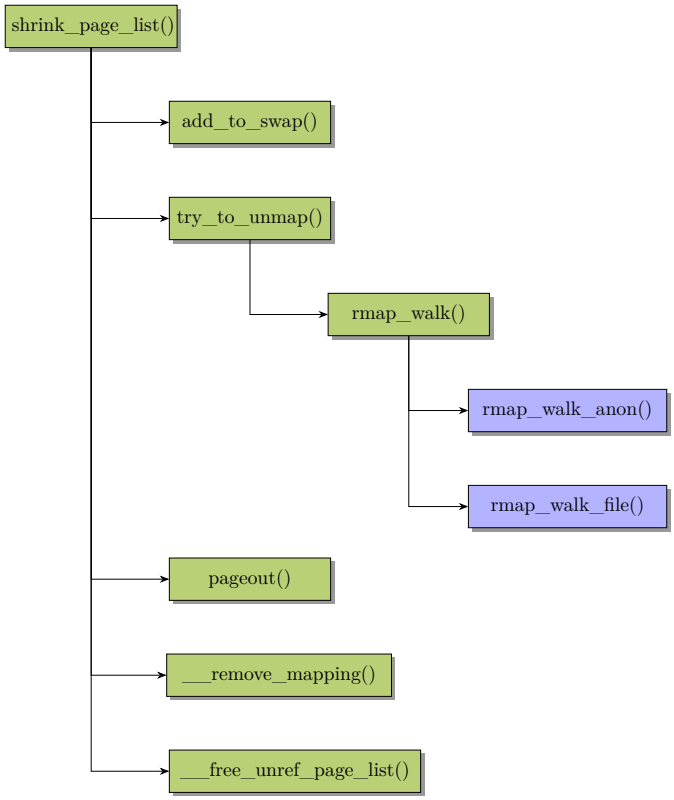


图 9–1: 正向映射: 从 task_struct 查找 mm_struct(mm), 再通过 mm->pgd 和虚拟地址检索页表，定位到 PTE

9.1 匿名页和文件页反向映射的入口函数

反向映射分为匿名页和文件页的反向映射，它们是两套不同实现。在 *try_to_unmap()* 内部分两条路径：***rmap_walk_anon()***(匿名页)、***rmap_walk_file()***(文件页)。

从 *shrink_page_list()* 至此的调用路径如下图:



9.1.1 rmap_walk_anon()

函数 *rmap_walk_anon()* 利用给定的一个匿名页，遍历所有映射该页的 VMA，对每一个 VMA 调用回调 *rwc->rmap_one()*(try_to_unmap_one()，修改 PTE)。如下:

rmap_walk_anon()

```
1847 static void rmap_walk_anon(struct page *page, struct rmap_walk_control *rwc,
1848         bool locked)
1849 {
1850     struct anon_vma *anon_vma;
1851     pgoff_t pgoff_start, pgoff_end;
1852     struct anon_vma_chain *avc;
1853 
1854     if (locked) {
1855         anon_vma = page_anon_vma(page);
1856         /* anon_vma disappear under us? */
1857         VM_BUG_ON_PAGE(!anon_vma, page);
1858     } else {
1859         anon_vma = rmap_walk_anon_lock(page, rwc);
1860     }
1861     if (!anon_vma)
1862         return;
1863 
1864     pgoff_start = page_to_pgoff(page);
1865     pgoff_end = pgoff_start + thp_nr_pages(page) - 1;
1866     anon_vma_interval_tree_foreach(avc, &anon_vma->rb_root,
1867             pgoff_start, pgoff_end) {
1868         struct vm_area_struct *vma = avc->vma;
1869         unsigned long address = vma_address(page, vma);
1870 
1871         cond_resched();
1872 
```

```
1873            if (rwc->invalid_vma && rwc->invalid_vma(vma, rwc->arg))
1874                continue;
1875
1876            if (!rwc->rmap_one(page, vma, address, rwc->arg))
1877                break;
1878            if (rwc->done && rwc->done(page))
1879                break;
1880        }
1881
1882        if (!locked)
1883            anon_vma_unlock_read(anon_vma);
1884    }
```

- 1850 获取匿名反向映射的根对象, 匿名页的 *page->mapping* 低位带有 *PAGE_MAPPING_ANON* 标记，指向 anon_vma。

  如果目标 page 属于页缓存 (pagecache, 一般提到页缓存指的是文件页缓存或 swapcache 页, 它映射到某个文件的一部分，也就是磁盘或 swap 分区文件在 RAM 里的缓存), 则 *struct page->mapping* 字段指向 *struct address_space* 对象，用来描述一个页缓存实体 (通常对应一个文件或 swap 文件)。如果是匿名页，则 *page->mapping* 指向 *struct anon_vma* 对象，用于管理一组匿名 page(如下示意)。

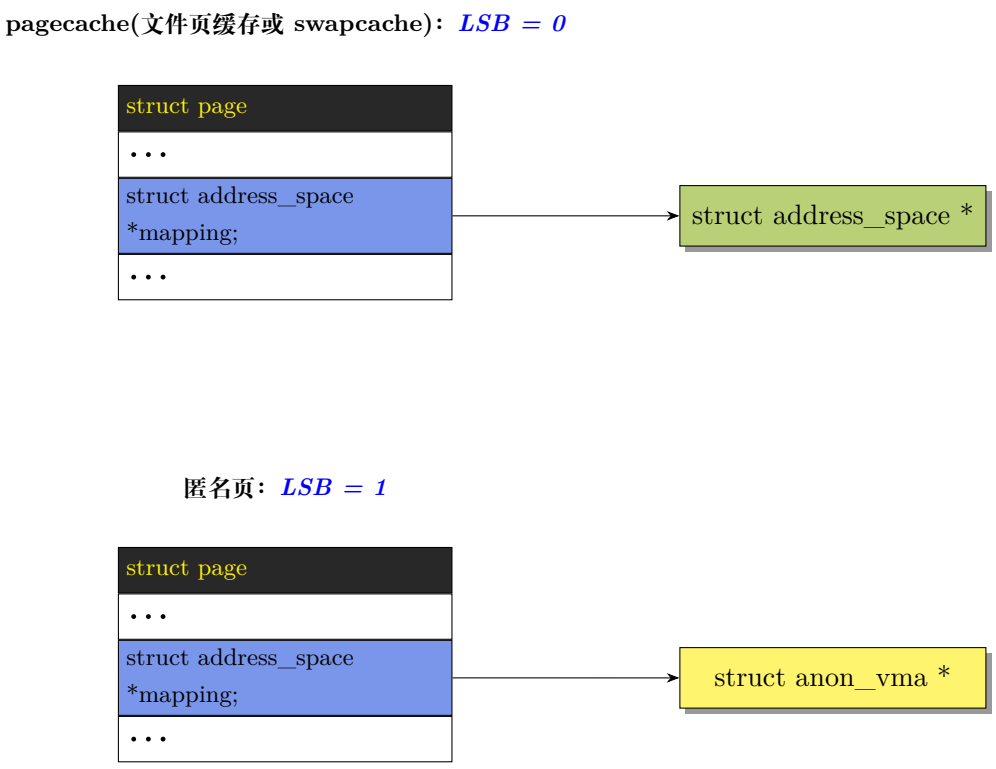


图 9–2: pagecache 和匿名页的 mapping 指向不同对象

- 1854-1857 调用者已经获得了锁，此时直接获取 anon_vma。
- 1859 调用方未上锁, 调用 *rmap_walk_anon_lock()* 内部处理锁。
- 1861-1862 获取 anon_vma 失败, 直接返回，不做遍历。
- 1864 返回 page->index，也就是目标匿名页在该页在 anon_vma 内部的页偏移。
- 1865 目标页/大页的结束页的索引。

对于文件映射，它表示该页数据在 文件中 的第几页 (从文件开头算起的页索引)。例如，如果 index 为 5，表示这是文件的第 5 个 4KB 数据块。

对于匿名映射 ，它通常用于表示该页相对于 $anon_vma$ 对象的全局虚拟页索引，作为 $anon_vma_interval_tree$ 的搜索 key。直接对应 struct page 结构体中的 $->index$ 成员。

- 1866-1879
  - 1866-1867 $anon_vma->rb_root$ 是区间红黑树，树上挂载全部 $anon_vma_chain$(avc)，每一个 avc 代表一个映射此 anon_vma 的 VMA。该宏遍历红黑树上的 avc 节点，筛选出**所有**和 $[pgoff_start, pgoff_end]$ 区间有重叠的 avc 节点。

    循环体内拿到 avc，再通过 $avc->vma$ 得到 VMA。每一个匹配的 avc 代表这个 VMA 映射了目标物理页。
  - 1868 取出对应的用户态 VMA。
  - 1869 计算目标物理页在当前 vma 内部对应的虚拟地址。
  - 1871 反向映射遍历 VMA 数量可能很多，为了防止内核卡死，如果需要调度，让出 cpu。
  - 1873-1874 如果设置了 $invalid_vma$ 回调，回调执行返回 true 代表该 VMA 已经无效，跳过这个 VMA，继续下一个 avc。
  - 1876-1877 调用核心回调函数 $rmap_one$(实际是 $try_to_unmap_one()$), 传入 page、vma、虚拟地址，去找到 PTE，执行 PTE 修改 (匿名页改写为 swap_entry_t, 文件页置无效)。
  - 1878-1879 done 回调钩子, 上层可以设置 (例如：已经找到全部需要的映射，提前终止遍历，不需要走完所有 VMA), 满足条件跳出循环。
- 1882-1883 如果锁是函数内部 $rmap_walk_anon_lock()$ 拿到的 $(locked = false)$，释放 anon_vma 读锁。如果 $locked = true$，锁归调用者负责释放，本函数不解锁。

综上, 内核使用 rmap(反向映射) 机制来从物理页找到 PTE, 对于**匿名页**流程如下：

- 1. 首先通过 $page->mapping$ 获取带标记的指针 (调用 $page_anon_vma()$，掩码清除最低位 $PAGE_MAPPING_ANON$)，得到真正的 anon_vma 对象，见下图1。
- 2. $anon_vma$ 维护一棵区间增强红黑树，树根为 $anon_vma->rb_root$；树上存储的节点类型是 $anon_vma_chain$(AVC)(下图2)。获取该页的 $page->index$(这是相对于 $anon_vma$ 的全局页偏移 Key，不是 VMA 内部偏移)，结合 THP 页数算出待检索区间 $[pgoff_start, pgoff_end]$；遍历区间红黑树，筛选出所有与待检索区间重叠的 AVC 节点；每一个 AVC 节点对应一个 vm_area_struct(下图标号3)。
- 3. 取出 AVC 关联的 VMA($avc->vma$)；通过 $vma->vm_mm$ 获取该 VMA 所属进程的 mm_struct(图中标号4)。fork-COW 场景下，同一个物理匿名页会映射到多个进程，因此会出现多个不同的 mm_struct。
- 4. 基于 $anon_vma$ 全局偏移 ($page->index$) 和当前 VMA 的参数，调用 $vma_address(page, vma)$，计算出目标页在当前 VMA 内对应的虚拟地址。
- 5. 使用 $mm_struct->pgd$ 和上一步得到的虚拟地址，依次调用各级页表遍历宏：$pgd_offset()$、$pud_offset()$、$pmd_offset()$、$pte_offset()$，在该进程地址空间中找到对应的 PTE(图中标号5)。

- 6. 对每一个匹配到的 AVC/VMA，重复步骤 3-5；遍历完全部 AVC 节点，匿名页反向映射查找流程就结束了。

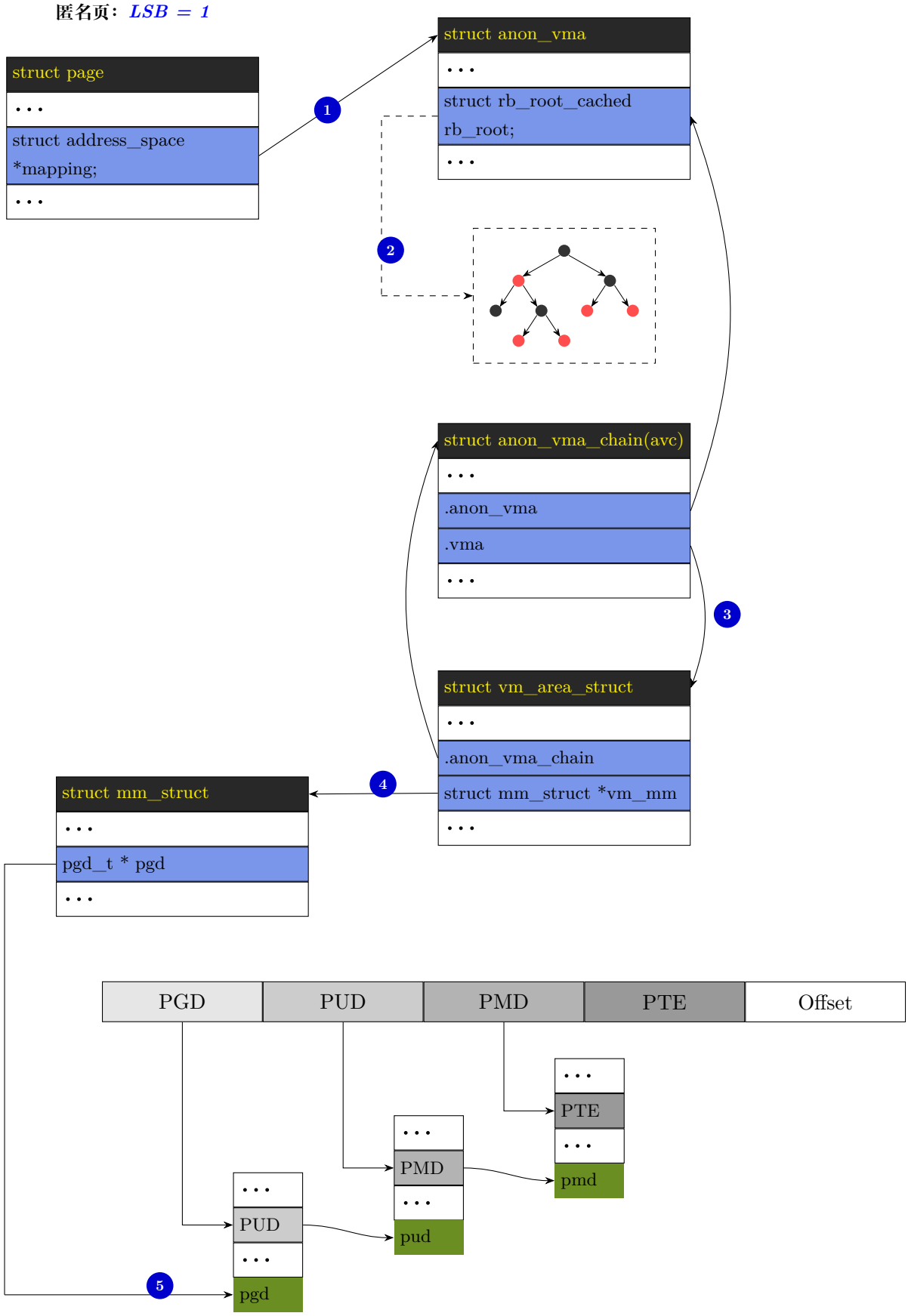


图 9–3: 反向映射: 匿名页查找 PTE 的流程

这里是内核代码从 $mm_struct->pgd$ 的虚拟地址开始检索 PTE。

如果是硬件MMU 找 PTE，x86 的 ***CR3*** 寄存器存放 PGD 物理基址，会从 CR3 寄存器中的物理地址出发遍历页表项。每个进程拥有独立的 PGD 页表，但是全局共享内核页表分支，所以每个**页表项**的高位索引都填同样的值，指向 *swapper_pg_dir*。低位索引每个进程各不相同，指向私有的用户空间页表分支。

arm64 架构使用双基址寄存器: ***TTBR0_EL1*** 用于用户态虚拟地址 (进程私有 mm) 的访问，而 ***TTBR1_EL1*** 固定指向内核页表。每个用户进程拥有独立的 PGD 物理地址，进程上下文切换时，内核先将 `mm->pgd` 转换为物理地址，再更新到 TTBR0_EL1。MMU 自动根据虚拟地址范围判断使用 ***TTBR0_EL1*** 还是 ***TTBR1_EL1***，不需要内核软件控制。**TTBR1_EL1** 是内核态虚拟地址的页表根寄存器。指向全局内核页表 (*swapper_pg_dir*)，进程切换时不会修改。当虚拟地址落在内核地址空间 (高地址范围，高位全 1)，MMU 使用 ***TTBR1_EL1*** 进行地址翻译。

arm64 cpu 访问用户空间时，从 ***TTBR0_EL1*** 获取用户页表的基地址 (用户页表基地址存储在进程的 mm->pgd 中)，而在访问内核空间时会从 ***TTBR1_EL1*** 获取内核页表基地址 (*swapper_pg_dir*)。注意:*mm− > pgd 和 swapper_pg_dir* 都是虚拟地址，更新 TTBR0/1_EL1 时会先转换成物理地址再填充。

下面是不同架构下 PGD 基址寄存器的差异对比。

项	*x86_64*	*arm64(AArach64)*	*arm32(armv7)*	*RISC-V64*
用户根页表物理基址寄存器	*CR3*	*TTBR0_EL1*	*TTBR* (每个进程独立页表基址)	*satp.PPN*
内核根页表物理基址寄存器	*CR3*	*TTBR1_EL1*	*TTBR*, 内核页表条目复制到每个进程的页表	*satp.PPN*
说明	每个进程切换后 *CR3* 里存的是指向该进程 *PGD* 数组的指针。*PGD* 数组的高半部分索引用于内核页表存 *PUD* 指针。也就是 *swapper pgd(内核全局 PGD)* 数组高半部分索引的值原样拷贝过来，低半部分索引存进程用户空间页表的 *PUD* 指针。	*TTBR0_EL1*: 进程独立根页表，只处理用户地址 *TTBR1_EL1*: 全局内核根页表，硬件直接使用，不需要把内核 *PGD* 项拷贝到进程对应的 *PGD*。	和 *x86* 类似，直接拷贝内核 *PGD* 里的条目值，填到本进程 *PGD* 对应位置。用户空间页表各个进程私有。	存根页表物理页帧号而不是物理地址。进程切换时直接把新进程根页表的物理页帧号写入 *satp*. 每个进程根页表内核区间拷贝内核 *PGD* 索引项

对于 *ARMv7*: TTBR0/TTBR1 没有 EL 后缀，阈值由 TTBCR.N 配置。

RISC-V 使用 **satp** 寄存器。其中 PPN 字段保存根页表 (PGD) 物理基址；mode 位开启分页。进程切换改写 satp 的 PPN 部分，同时执行 sfence.vma 刷新 TLB。

另外，arm64 的 ***TTBR1_EL1*** 是唯一的，指向同一个物理地址不变。而 x86_64 不同进程下 CR3 指向的物理基址是不同的，只是 pgd 页表**项**的高位是不变的 (指向 *swapper_pg_dir*)。

9.1.1.1 try_to_unmap_one()

try_to_unmap_one() 是反向映射 rmap 的回调函数。*rmap_walk* 遍历目标 page 所有映射的 VMA，对每一个 VMA 里对应的虚拟地址调用 *try_to_unmap_one()*，把这个 VMA 里指向目标 page 的 PTE 解除映射。

rmap_walk_anon() > *try_to_unmap_one()*

```
1373 static bool try_to_unmap_one(struct page *page, struct vm_area_struct *vma,
1374              unsigned long address, void *arg)
1375 {
1376     struct mm_struct *mm = vma->vm_mm;
1377     struct page_vma_mapped_walk pvmw = {
1378         .page = page,
1379         .vma = vma,
```

```
1380            .address = address,
1381        };
1382        pte_t pteval;
1383        struct page *subpage;
1384        bool ret = true;
1385        struct mmu_notifier_range range;
1386        enum ttu_flags flags = (enum ttu_flags)(long)arg;
1387
1388        /* munlock has nothing to gain from examining un-locked vmas */
1389        if ((flags & TTU_MUNLOCK) && !(vma->vm_flags & VM_LOCKED))
1390            return true;
1391
1392        if (IS_ENABLED(CONFIG_MIGRATION) && (flags & TTU_MIGRATION) &&
1393            is_zone_device_page(page) && !is_device_private_page(page))
1394            return true;
1395
1396        if (flags & TTU_SPLIT_HUGE_PMD) {
1397            split_huge_pmd_address(vma, address,
1398                    flags & TTU_SPLIT_FREEZE, page);
1399        }
1400
1401        /*
1402         * For THP, we have to assume the worse case ie pmd for invalidation.
1403         * For hugetlb, it could be much worse if we need to do pud
1404         * invalidation in the case of pmd sharing.
1405         *
1406         * Note that the page can not be free in this function as call of
1407         * try_to_unmap() must hold a reference on the page.
1408         */
1409        mmu_notifier_range_init(&range, MMU_NOTIFY_CLEAR, 0, vma, vma->vm_mm,
1410                    address,
1411                    min(vma->vm_end, address + page_size(page)));
1412       if (PageHuge(page)) {
1413            /*
1414             * If sharing is possible, start and end will be adjusted
1415             * accordingly.
1416             */
1417            adjust_range_if_pmd_sharing_possible(vma, &range.start,
1418                                 &range.end);
1419        }
1420        mmu_notifier_invalidate_range_start(&range);
1421
1422        while (page_vma_mapped_walk(&pvmw)) {
1423    #ifdef CONFIG_ARCH_ENABLE_THP_MIGRATION
1424            /* PMD-mapped THP migration entry */
1425            if (!pvmw.pte && (flags & TTU_MIGRATION)) {
1426                VM_BUG_ON_PAGE(PageHuge(page) || !PageTransCompound(page), page);
1427
1428                set_pmd_migration_entry(&pvmw, page);
1429                continue;
1430            }
1431    #endif
1432
1433            /*
1434             * If the page is mlock()d, we cannot swap it out.
1435             * If it's recently referenced (perhaps page_referenced
1436             * skipped over this mm) then we should reactivate it.
1437             */
1438            if (!(flags & TTU_IGNORE_MLOCK)) {
1439                if (vma->vm_flags & VM_LOCKED) {
1440                    /* PTE-mapped THP are never mlocked */
1441                    if (!PageTransCompound(page)) {
1442                        /*
1443                         * Holding pte lock, we do *not* need
```

```
                     * mmap_lock here
                     */
                    mlock_vma_page(page);
                }
                ret = false;
                page_vma_mapped_walk_done(&pvmw);
                break;
            }
            if (flags & TTU_MUNLOCK)
                continue;
        }

        /* Unexpected PMD-mapped THP? */
        VM_BUG_ON_PAGE(!pvmw.pte, page);

        subpage = page - page_to_pfn(page) + pte_pfn(*pvmw.pte);
        address = pvmw.address;

        if (PageHuge(page) && !PageAnon(page)) {
            /*
             * To call huge_pmd_unshare, i_mmap_rwsem must be
             * held in write mode.  Caller needs to explicitly
             * do this outside rmap routines.
             */
            VM_BUG_ON(!(flags & TTU_RMAP_LOCKED));
            if (huge_pmd_unshare(mm, vma, &address, pvmw.pte)) {
                /*
                 * huge_pmd_unshare unmapped an entire PMD
                 * page.  There is no way of knowing exactly
                 * which PMDs may be cached for this mm, so
                 * we must flush them all.  start/end were
                 * already adjusted above to cover this range.
                 */
                flush_cache_range(vma, range.start, range.end);
                flush_tlb_range(vma, range.start, range.end);
                mmu_notifier_invalidate_range(mm, range.start,
                                      range.end);

                /*
                 * The ref count of the PMD page was dropped
                 * which is part of the way map counting
                 * is done for shared PMDs.  Return 'true'
                 * here.  When there is no other sharing,
                 * huge_pmd_unshare returns false and we will
                 * unmap the actual page and drop map count
                 * to zero.
                 */
                page_vma_mapped_walk_done(&pvmw);
                break;
            }
        }

         if (IS_ENABLED(CONFIG_MIGRATION) &&
            (flags & TTU_MIGRATION) &&
            is_zone_device_page(page)) {
            swp_entry_t entry;
            pte_t swp_pte;

            pteval = ptep_get_and_clear(mm, pvmw.address, pvmw.pte);

            /*
             * Store the pfn of the page in a special migration
             * pte. do_swap_page() will wait until the migration
             * pte is removed and then restart fault handling.
```

```
1508             */
1509            entry = make_migration_entry(page, 0);
1510            swp_pte = swp_entry_to_pte(entry);
1511
1512            /*
1513             * pteval maps a zone device page and is therefore
1514             * a swap pte.
1515             */
1516            if (pte_swp_soft_dirty(pteval))
1517                swp_pte = pte_swp_mksoft_dirty(swp_pte);
1518            if (pte_swp_uffd_wp(pteval))
1519                swp_pte = pte_swp_mkuffd_wp(swp_pte);
1520            set_pte_at(mm, pvmw.address, pvmw.pte, swp_pte);
1521            /*
1522             * No need to invalidate here it will synchronize on
1523             * against the special swap migration pte.
1524             *
1525             * The assignment to subpage above was computed from a
1526             * swap PTE which results in an invalid pointer.
1527             * Since only PAGE_SIZE pages can currently be
1528             * migrated, just set it to page. This will need to be
1529             * changed when hugepage migrations to device private
1530             * memory are supported.
1531             */
1532            subpage = page;
1533            goto discard;
1534        }
1535
1536        if (!(flags & TTU_IGNORE_ACCESS)) {
1537            if (ptep_clear_flush_young_notify(vma, address,
1538                        pvmw.pte)) {
1539                ret = false;
1540                page_vma_mapped_walk_done(&pvmw);
1541                break;
1542            }
1543        }
1544
1545        /* Nuke the page table entry. */
1546        flush_cache_page(vma, address, pte_pfn(*pvmw.pte));
1547        if (should_defer_flush(mm, flags)) {
1548            /*
1549             * We clear the PTE but do not flush so potentially
1550             * a remote CPU could still be writing to the page.
1551             * If the entry was previously clean then the
1552             * architecture must guarantee that a clear->dirty
1553             * transition on a cached TLB entry is written through
1554             * and traps if the PTE is unmapped.
1555             */
1556            pteval = ptep_get_and_clear(mm, address, pvmw.pte);
1557
1558            set_tlb_ubc_flush_pending(mm, pte_dirty(pteval));
1559        } else {
1560            pteval = ptep_clear_flush(vma, address, pvmw.pte);
1561        }
1562
1563        /* Move the dirty bit to the page. Now the pte is gone. */
1564        if (pte_dirty(pteval))
1565            set_page_dirty(page);
1566
1567        /* Update high watermark before we lower rss */
1568        update_hiwater_rss(mm);
1569
1570        if (PageHWPoison(page) && !(flags & TTU_IGNORE_HWPOISON)) {
1571            pteval = swp_entry_to_pte(make_hwpoison_entry(subpage));
```

```
1572            if (PageHuge(page)) {
1573                hugetlb_count_sub(compound_nr(page), mm);
1574                set_huge_swap_pte_at(mm, address,
1575                                     pvmw.pte, pteval,
1576                                     vma_mmu_pagesize(vma));
1577            } else {
1578                dec_mm_counter(mm, mm_counter(page));
1579                set_pte_at(mm, address, pvmw.pte, pteval);
1580            }
1581
1582        } else if (pte_unused(pteval) && !userfaultfd_armed(vma)) {
1583            /*
1584             * The guest indicated that the page content is of no
1585             * interest anymore. Simply discard the pte, vmscan
1586             * will take care of the rest.
1587             * A future reference will then fault in a new zero
1588             * page. When userfaultfd is active, we must not drop
1589             * this page though, as its main user (postcopy
1590             * migration) will not expect userfaults on already
1591             * copied pages.
1592             */
1593            dec_mm_counter(mm, mm_counter(page));
1594            /* We have to invalidate as we cleared the pte */
1595            mmu_notifier_invalidate_range(mm, address,
1596                                          address + PAGE_SIZE);
1597        } else if (IS_ENABLED(CONFIG_MIGRATION) &&
1598                (flags & (TTU_MIGRATION|TTU_SPLIT_FREEZE))) {
1599            swp_entry_t entry;
1600            pte_t swp_pte;
1601
1602            if (arch_unmap_one(mm, vma, address, pteval) < 0) {
1603                set_pte_at(mm, address, pvmw.pte, pteval);
1604                ret = false;
1605                page_vma_mapped_walk_done(&pvmw);
1606                break;
1607            }
1608
1609            /*
1610             * Store the pfn of the page in a special migration
1611             * pte. do_swap_page() will wait until the migration
1612             * pte is removed and then restart fault handling.
1613             */
1614            entry = make_migration_entry(subpage,
1615                    pte_write(pteval));
1616            swp_pte = swp_entry_to_pte(entry);
1617            if (pte_soft_dirty(pteval))
1618                swp_pte = pte_swp_mksoft_dirty(swp_pte);
1619           if (pte_uffd_wp(pteval))
1620                swp_pte = pte_swp_mkuffd_wp(swp_pte);
1621            set_pte_at(mm, address, pvmw.pte, swp_pte);
1622            /*
1623             * No need to invalidate here it will synchronize on
1624             * against the special swap migration pte.
1625             */
1626        } else if (PageAnon(page)) {
1627            swp_entry_t entry = { .val = page_private(subpage) };
1628            pte_t swp_pte;
1629            /*
1630             * Store the swap location in the pte.
1631             * See handle_pte_fault() ...
1632             */
1633            if (unlikely(PageSwapBacked(page) != PageSwapCache(page))) {
1634                WARN_ON_ONCE(1);
1635                ret = false;
```

```
1636                /* We have to invalidate as we cleared the pte */
1637                mmu_notifier_invalidate_range(mm, address,
1638                            address + PAGE_SIZE);
1639                page_vma_mapped_walk_done(&pvmw);
1640                break;
1641            }
1642
1643            /* MADV_FREE page check */
1644            if (!PageSwapBacked(page)) {
1645                if (!PageDirty(page)) {
1646                    /* Invalidate as we cleared the pte */
1647                    mmu_notifier_invalidate_range(mm,
1648                        address, address + PAGE_SIZE);
1649                    dec_mm_counter(mm, MM_ANONPAGES);
1650                    goto discard;
1651                }
1652
1653                /*
1654                 * If the page was redirtied, it cannot be
1655                 * discarded. Remap the page to page table.
1656                 */
1657                set_pte_at(mm, address, pvmw.pte, pteval);
1658                SetPageSwapBacked(page);
1659                ret = false;
1660                page_vma_mapped_walk_done(&pvmw);
1661                break;
1662            }
1663
1664           if (swap_duplicate(entry) < 0) {
1665                set_pte_at(mm, address, pvmw.pte, pteval);
1666                ret = false;
1667                page_vma_mapped_walk_done(&pvmw);
1668                break;
1669            }
1670            if (arch_unmap_one(mm, vma, address, pteval) < 0) {
1671                set_pte_at(mm, address, pvmw.pte, pteval);
1672                ret = false;
1673                page_vma_mapped_walk_done(&pvmw);
1674                break;
1675            }
1676            if (list_empty(&mm->mmlist)) {
1677                spin_lock(&mmlist_lock);
1678                if (list_empty(&mm->mmlist))
1679                    list_add(&mm->mmlist, &init_mm.mmlist);
1680                spin_unlock(&mmlist_lock);
1681            }
1682            dec_mm_counter(mm, MM_ANONPAGES);
1683            inc_mm_counter(mm, MM_SWAPENTS);
1684            swp_pte = swp_entry_to_pte(entry);
1685            if (pte_soft_dirty(pteval))
1686                swp_pte = pte_swp_mksoft_dirty(swp_pte);
1687            if (pte_uffd_wp(pteval))
1688                swp_pte = pte_swp_mkuffd_wp(swp_pte);
1689            set_pte_at(mm, address, pvmw.pte, swp_pte);
1690            /* Invalidate as we cleared the pte */
1691            mmu_notifier_invalidate_range(mm, address,
1692                                address + PAGE_SIZE);
1693        } else {
1694            /*
1695             * This is a locked file-backed page, thus it cannot
1696             * be removed from the page cache and replaced by a new
1697             * page before mmu_notifier_invalidate_range_end, so no
1698             * concurrent thread might update its page table to
1699             * point at new page while a device still is using this
```

```
1700                 * page.
1701                 *
1702                 * See Documentation/vm/mmu_notifier.rst
1703                 */
1704                dec_mm_counter(mm, mm_counter_file(page));
1705        }
1706 discard:
1707        /*
1708         * No need to call mmu_notifier_invalidate_range() it has be
1709         * done above for all cases requiring it to happen under page
1710         * table lock before mmu_notifier_invalidate_range_end()
1711         *
1712         * See Documentation/vm/mmu_notifier.rst
1713         */
1714        page_remove_rmap(subpage, PageHuge(page));
1715        put_page(page);
1716    }
1717
1718    mmu_notifier_invalidate_range_end(&range);
1719
1720    return ret;
1721 }
```

- 1376 获取 VMA 所属的地址空间 *mm_struct*，即进程地址空间。
- 1377-1381 结构体 *page_vma_mapped_walk* 用于保存迭代遍历映射目标页的各级页表时的上下文。它通过 *page_vma_mapped_walk()* 完成迭代遍历。

  page_vma_mapped_walk 结构体保存当前正在检查的 PTE 相关状态，整个操作过程中，*struct page_vma_mapped_walk− > pte* 会被更新。
- 1389-1390 无需遍历或检查那些未被锁定的 VMA。
- 1392-1394 只有设备私有页能迁移 (搬移)。

  如果内核配置了页面迁移功能，并且当前是迁移操作的执行路径，那么 *MEMORY_DEVICE_FS_DAX*、*MEMORY_DEVICE_GENERIC*、*MEMORY_DEVICE_PCI_P2PDMA*(这些宏的解释见下面) 在迁移场景下直接跳过这个代码段。而**设备私有页**返回值是 true(已处理完), 表示目标页不该在此被 unmap。
  目标页是设备页，但不是设备私有页(device private page)，那么不在这里 unmap。那么跳过这个 VMA，继续下一个。

  设备页是一类特殊内存, 是设备侧的本地内存, 不属于普通 RAM, 归属于 *ZONE_DEVICE* 区域。由设备驱动调用 *devm_memremap_pages()* 把设备侧内存注册进内核 page 体系，生成 *struct page*，供用户态 mmap、内核 mm 子系统 (rmap、迁移、回收) 识别。

  譬如，GPU 板载显存 (VRAM)，可以通过 PCIe 暴露给系统注册到 *ZONE_DEVICE*。还有 AI 加速卡 (NPU/TPU) 的板载内存暴露给 Linux 内核后, 用户态程序可以 mmap 映射这块加速器内存, 用于存放模型权重、特征数据。还有 DPDK 硬件加速网卡自带板载内存用于报文缓冲，也可以注册成 *ZONE_DEVICE*。这些内存是外设自带的设备侧本地内存，物理地址空间属于 PCIe 设备域，不是 CPU 的主存。

  当然，设备侧的设备不一定限定是 PCIe 设备。譬如, 有些 SoC 有内部 NPU，NPU 自带片上 SRAM/本地 DDR，挂在 SoC 内部总线 (AXI)，不是 PCIe 总线。还有使用 CXL 总线的 CXL 设备 (compute Express link) 的本地内存也都可以注册 *ZONE_DEVICE*。驱动可以把 NPU 本地内存注册为 *ZONE_DEVICE*。

代码中的"migrate" 字样在这里就是本意: 数据" 搬移"。

设备侧内存和普通内存的对比如下:

项	普通 *RAM*	设备侧内存
归属	*cpu* 侧	设备侧
struct page	内核伙伴系统自动创建。	驱动显式调用 *devm_memremap_pages()* 创建
访问方式	*cpu* 直接访问, 无需设备驱动介入	两种访问模式, *1.* 私有模式 *(MEMORY_DEVICE_PRIVATE)*: *cpu* 访问触发 *fault*, 驱动处理。*2.generic* 模式 *(MEMORY_DEVICE_GENERIC)*: *cpu* 通过 *MMIO* 直接访问。详见后面"设备页"部分的解析。
页面迁移	可以整理、迁移	受 *page->pgmap->type* 类型的约束, 部分类型支持
回收	伙伴系统回收	由设备驱动 + 内核 *mm* 协同, 不能随意回收

是否有迁移 (搬移) 能力不是由 page 本身决定, 是由这块内存的注册类型 (page->pgmap->type) 来限定约束的。普通系统 RAM 页 $page->pgmap == NULL$, 走标准迁移 (搬移) 逻辑。type 是枚举类型, 如下:

- ***MEMORY_DEVICE_PRIVATE*** 是 HMM 的设备私有内存 (如:GPU 板载内存 VRAM), CPU 不能直接读写, 但可以进行页面迁移 (搬运).

  HMM 迁移 (数据搬运) 是跨两种不同类型内存域的数据搬移, 目的是异构内存按需换入换出, 类似 swap, 但交换目的地是 GPU 显存而不是磁盘 swap 分区。HMM 全称是:Heterogeneous Memory Management, 即异构内存管理。

  迁移 (搬运) 的实际数据拷贝不是内核 mm 做, 而是调用驱动提供的 $page->pgmap->ops->migrate()$ 回调。内核 mm 只负责 rmap、页表拆除、page 元数据; 真正把数据从 GPU 显存搬到系统内存 (或反向) 由设备驱动实现。如果驱动没有实现 migrate 回调, 迁移会失败。

  这种类型的内存对应的是**设备私有页**。页面位于设备显存。CPU PTE 为非驻留 (present) 的特殊条目。也就是说目标页对应的 CPU 硬件页表项 (PTE) 被标记为 "无效/不存在", 所以导致 CPU 无法直接访问该地址, 必须通过软件机制 (缺页异常 + 迁移) 才能获取数据。CPU 访问触发缺页异常, HMM(异构内存管理) 框架介入将数据迁移至系统 RAM。迁移前, 通过 *mmu_interval_notifier* 通知驱动清理硬件映射, 内核随后将 cpu 页表中的 device private entry 替换为特殊的 migration entry(迁移条目)。当 CPU 再次访问该地址时, 会触发缺页异常。内核处理程序检测到这是 migration entry, 会让当前进程阻塞, 直到迁移完成并恢复有效的 PTE。

- ***MEMORY_DEVICE_GENERIC*** 是通用设备内存 (譬如:PCIe BAR), 这种 pgmap->type 类型不支持页面迁移 (搬运)。

  该类型内存对应**通用设备页**。页面位于设备内存, 但 CPU 可直接访问。目标页的 CPU PTE 标记为" 驻留 (present)" 且指向设备内存的物理地址。不涉及数据迁移。虽然不加入匿名页 rmap, 但内核 MM 仍正常管理其 CPU 页表映射。当 VMA 销毁时, 内核正常拆除 PTE。

- ***MEMORY_DEVICE_FS_DAX*** 是持久内存 (PMEM), 掉电数据不丢失。最早是用来替代老的 XIP(execute-in-place, 就地执行)。文件系统 (xfs/ext4) 开启 DAX(*mount o dax*), mmap 时不走 pagecache, 直接建 PTE/PMD 映射指向 PMEM。CPU 可以直接用普通 load/store 指令按字节随机访问这片物理地址。

  DAX(Direct Access) 即直接访问。

  MEMORY_DEVICE_FS_DAX 仅适用于原生硬件可字节可寻址的持久内存 (PMEM/NVDIMM)。普通 NAND/NOR Flash 无法挂载为此类型。**NAND Flash** 不是字节寻址, 只能按

页读、按块擦写。**NOR Flash** 虽然可以字节读 (XIP)，写操作依然必须按页/块擦除，硬件行为和 PMem 完全不同；Linux 内核不会把普通 NOR Flash 暴露成 pmem 设备，不能走 FSDAX。

NOTE

mount -o dax 只是文件系统层面开启 *DAX*。
真正创建 *ZONE_DEVICE*、生成一批 *struct page*、设置 *pgmap->type = MEMORY_DEVICE_FS_DAX* 这件事，发生在 *PMEM* 名字空间初始化阶段，不是 *mount* 的时候。

- ***MEMORY_DEVICE_PCI_P2PDMA*** 是 PCI 点对点 DMA。用于 PCI 设备之间点对点 DMA，不允许内核 mm 做页面迁移。

GPU 显存等设备侧的物理内存是动态发现的，不能像普通的 RAM(ZONE_NORMAL/ZONE_DMA 等) 区域在启动时由 BIOS/UEFI/DTS 告知内核为其建立 struct page。而且这些设备侧物理地址范围可能不连续或不在常规 RAM 范围内。为了能像管理普通内存一样管理这些设备内存 (例如支持 get_user_pages、迁移、引用计数等)，必须为这些物理地址范围创建 struct page。内核通过调用 ***devm_memremap_pages()*** 接口函数来创建对应的 *struct page* 描述符，并将这些页面划分到 ***ZONE_DEVICE*** 区域，在此基础上统一管理这些设备页面。

ZONE_DEVICE 是专门为 PMEM、GPU 显存、CXL 内存等设备物理地址保留的区域，是由设备驱动通过 ***devm_memremap_pages()*** 等接口映射进来的设备内存。

devm_memremap_pages() 更底层的实现是***memremap_pages()***。对于设备私有页和设备托管页同样通过 *devm_memremap_pages()* 注册的 *ZONE_DEVICE* 页的类型是不同的，它们分别是:***MEMORY_DEVICE_PRIVATE***和***MEMORY_DEVICE_GENERIC***。

下面是设备私有页和通用设备页这两种类型设备页的对比:

项	*MEMORY_DEVICE_PRIVATE*	*MEMORY_DEVICE_GENERIC*
CPU 页表 *PTE*	非 *present*，*HMM swap entry*。	*present PTE*，指向设备 *PFN*
CPU 访问行为	触发 *CPU* 缺页，*HMM fault* 处理。	*MMU* 直接访问，无缺页异常
rmap 反向映射	加入 *anon_vma rmap* 链表	不加入 *rmap* 链表
页面迁移 *try_to_unmap*	需要调用，内核拆除 *PTE*	跳过 *try_to_unmap*，内核不拆 *PTE*
映射管理方	内核 *mm* 管理 *CPU* 页表。	驱动 *mmu_interval_notifier* 接管

========== 设备私有页 (*MEMORY_DEVICE_PRIVATE*) 迁移的主要动作和 PTE 的处理

- 1396-1399 把 PMD 级映射降级成 PTE 级。后面的代码路径 (1560 行 ptep_clear_flush() 等) 都是按 PTE 写的，必须先拆。*TTU_SPLIT_FREEZE* 决定拆分 PMD 的同时，把拆出来的每一条 PTE 冻结为 migration entry(迁移项)，修改 PTE，清掉 present 位，写入特殊迁移标记。用户态访问时硬件会因此触发缺页阻塞，停止对该页面的用户态访问，让内核安全地做 THP 拆分。拆分完成，解冻，PTE 恢复 present 指向原有子页。migration entry 属于一类特殊的 swap entry，没有写到磁盘，只是编码在 PTE 里，用来标记 “这个页面正在被迁移/冻结，暂时不可访问”。
- 1409-1420 这段代码通知所有外部设备 (GPU 等): CPU 侧页表马上要清除目标虚拟地址区间的映射，请设备驱动把自己硬件页表/TLB 里对应这段 VA 的映射作废、停止新的设备侧访问，阻塞新的对设备侧内存访问。它只管通知设备侧来进行回调的执行。CPU 侧 PTE 的拆除、写 migration entry 是这行代码之后才做的。

1420 广播通知所有注册 *mmu_notifier* 的驱动 (GPU HMM 等), 触发驱动注册的回调。

- 1422 循环查找目标 page 在这个 VMA 里对应的下一个 PTE/PMD 页表项。返回 true 代表还有映射需要处理，返回 false 代表这个 VMA 下目标 page 的所有页表项遍历完毕。

  pvmw(*struct page_vma_mapped_walk* 是遍历器每次迭代时保存上下文的结构体，保存本次遍历的全部状态。

- 1423-1431 *pvmw.pte == NULL* 表明这是一个 PMD 级映射的 THP(页表项在 PMD 层，没有 PTE)。整块 2M 一次性换成 PMD 级 migration entry，然后继续下一轮.

- 1438-1454

  在 swapout 场景下，*shrink_page_list()* 调用 *try_to_unmap()* 时会给 *ttu_flags*(***ttu***:try to unmap) 置上 ***TTU_BATCH_FLUSH*** 标志，并且不会设置 ***TTU_IGNORE_MLOCK***，所以必须处理 mlock () 带来的边缘情形。

  TTU_BATCH_FLUSH 表示批量 TLB 刷新。每解除一个 PTE 映射，就需要立刻刷新 TLB。在 swapout 回收路径，因为大量页面要解除 PTE 映射，频繁刷 TLB 会造成性能很差。因此会设置 ***TTU_BATCH_FLUSH***: 暂时不立刻刷 TLB，等到一批 PTE 全部处理完成之后，统一进行批量刷新 TLB。这是内存回收 swapout 路径的专属优化标志，*shrink_page_list()* 调用 *try_to_unmap()* 一定会置位该标志。

  TTU_IGNORE_MLOCK 表示忽略 mlock() 锁定约束：就算页面被 *mlock()* 锁定 (*PG_mlocked*)，依旧继续执行 unmap、换出页面。

  NOTE

  什么时候才会置位 *TTU_IGNORE_MLOCK*？

  只有页迁移 *(migration)* 场景下才会置位该标志。
  页迁移允许绕过 *mlock* 限制，页面依旧保持锁定，只是换物理位置。

  swapout 回收场景绝对不打开 *TTU_IGNORE_MLOCK*。
  swapout 是把页面丢弃，写到 *swap*，释放物理内存。

正常情况下 VMA 有标记 ***VM_LOCKED***，page 有标记 ***PG_mlocked***。那么该页不能回收，不做 swapout。但是，会存在一种边缘情况: VMA 有 ***VM_LOCKED***，page 缺少 ***PG_mlocked*** 标记, 此时会调用 *mlock_vma_page()* 修复 page 标记，把 page 移入不可回收 (unevictable) LRU，放弃本次 swapout。

该场景通常是：VMA 设置了 *VM_LOCKED*，但是该虚拟地址还没有发生缺页，没有分配物理 page，那么自然不存在 *PG_mlocked*。

- 1457 走到这段代码说明一定是 PTE 映射，不能是 PMD 巨页。如果还遇到 PMD 直接 BUG(前面 TTU_SPLIT_HUGE_PMD 分支已经把 PMD THP 拆分完成)。

- 1459 计算获取目标 PTE 所指向的页的 struct page *指针。

  对于普通 4K 单页，*subpage == page*。

  对于复合页，*page*：就是 head page 指针 (THP 巨页的头 page)。*page_to_pfn(page)*：THP 的 head page 对应的物理页帧号 PFN。*pte_pfn(*pvmw.pte)*：当前 PTE 里面存的物理页帧号 PFN。

所以该行就等价于公式:

$$\begin{aligned}\text{目标子页 page 指针} &= \text{THP 头页 page 指针} - \text{THP 头页物理页帧号} + \text{目标子页物理页帧号}\\ &= \text{THP 头页 page 指针} + (\text{目标子页物理页帧号} - \text{THP 头页物理页帧号})\\ &= \text{THP 头页 page 指针} + \text{PFN 差值}\end{aligned}$$

- 1462-1494 这个分支专门处理非匿名的共享透明巨页 (file-backed THP)。

  ======== 待解析

- 1496-1534 该代码分支专门处理页面迁移 (搬运) 路径下的 ZONE_DEVICE 区域的设备私有页面 (MEMORY_DEVICE_PRIVATE)。普通 RAM 页面不走这个分支。

  上层调用会通过传入标志 *ttu_flags* 控制行为:

  - ***TTU_UNMAP*** 一般是页面回收、swap out 路径。把有效 PTE 替换成 swap entry:$PTEbit0 = 0(Valid/Present = 0)$, PTE 编码 $swap\ type + offset$, 减少 _mapcount(page 映射计数); TLB invalidate。用户后续访问目标虚拟地址会触发 page fault, 走 *do_swap_entry()*, 从 swap 读回页面。

    设备私有页不支持普通页面回收 (swapout)。

  - ***TTU_MIGRATION*** 内存规整/页面迁移的执行路径。写入 migration entry(迁移项)(一种特殊 swp_entry_t, $bit0 = 0$), 用户访问目标虚拟地址时会进入 fault handler 等待迁移完成, 不会读磁盘 swap。

  ======== 待插入 HMM 驱动的架构

  - 1502 读出当前 PTE, 保存到局部变量 pteval, 把硬件 PTE 清空 (置 0)。旧 PTE 本来就是非 present(驻留内存) 的设备私有页的 swap entry, TLB 里压根没有它的条目, 无需 flush。对比普通页走 1560 行的 ptep_clear_flush()。
  - 1509 构造迁移项 *swp_entry_t*。type 填充原映射的读写属性类型。write=0 时, type 填充 *SWP_MIGRATION_READ*(读类型 migration entry); write=1, type 填充 *SWP_MIGRATION_WRITE*(可写类型迁移 entry)。CPU 访问该 VA 触发缺页, 进入 *migration_entry_wait()* 阻塞进程.

    传入 page, 取 page_to_pfn(page), 把 PFN 存入 swp_entry 的 offset 字段,type 填原映射的属性。

    需要注意的是对于普通 swap 和迁移项的 swp_entry_t 编码的区别如下:

    普通 swap: type = swap 设备编号 (index), offset = 目标页在 swap 文件内的 slot 索引号。

    迁移 entry: type = entry 项的类型 (*SWP_MIGRATION_READ* 或 *SWP_MIGRATION_WRITE*), offset = PFN。

  - 1510 将 1509 行构造的 *swp_entry_t* 编码填入 pte_t, 生成一个非 present(驻留内存) 的 swap 类 PTE。
  - 1516-1517 检查原来旧 PTE 是否带有 soft-dirty 标记。soft-dirty 用于页追踪 (用户空间内存快照 / 内存审计)。如果旧 PTE 标记了 soft dirty, 就在新建的迁移 PTE 上也置上 soft-dirty 位。这样页面的软脏状态不会因为替换迁移项 (PTE) 而丢失。

- 1518-1519 - 如果原来 PTE 开启了 uffd 写保护，迁移后的 migration PTE 保留这个标记。用户态 userfaultfd 逻辑依赖该位，迁移过程不能丢失该状态。
- 1520 把为迁移项 (migration entry) 包装的非驻留 (non-present) PTE 写入硬件页表。
- 1532 5.10 内核的设备私有页只支持 4K，所以 $head == subpage$。1459 行由 swap PTE 算出的是无效指针, 在此修正，指向当前 page。
- 1533 ZONE_DEVICE 页由设备驱动/HMM 管理，不走普通页的 rmap 计数，跳过普通 anon 页的 rmap 处理逻辑 (不执行 page_remove_rmap、减 page->_mapcount 等)。

上面已经走完 *ZONE_DEVICE* 的特殊分支。现在处理普通页面 (anon/file)，当前 PTE 是驻留内存的有效映射。

- 1536-1543 若 PTE 的 young/Accessed 位被置起 (说明最近被访问过，页面回收路径本该跳过它)，则放弃 unmap 并让上层去 activate 这页。

  调用 *try_to_unmap* 的场景不同，flag 会不一样：

  页面迁移、内存压缩不设置 ***TTU_IGNORE_ACCESS***，需要检查 young 位；

  进程退出、munmap 销毁映射：会带上 ***TTU_IGNORE_ACCESS***，不需要关心页面最近有没有被访问，直接拆映射。

  - 1537 *ptep_clear_flush_young_notify()* 会做如下动作:

    1. 原子的读取和清除 PTE 的 Young(访问位)，判断硬件是否标记这个页最近被访问，若是则清除该位。

    2. 如果访问位原本被置 1(页面近期被访问过)：则刷 TLB、触发 mmu_notifier 通知 (通知 GPU 等外部设备 MMU 失效)。

    PTE 存在两处：

    1. 内存的页表里。

    2. CPU TLB 缓存 (硬件缓存 PTE 副本，加速地址翻译)。

    ptep_clear_flush_young_notify() 原子的读取和清 0 PTE 的 Young 位, 内存页表中 PTE 的变成 $Young = 0$, 但是 CPU TLB 里还缓存着旧条目：$Young = 1$。CPU 后续读一下这个 VA，TLB 命中，硬件就会看到缓存副本 Young=1。所以修改 PTE 的标志位 (譬如 young/dirty)，只要 TLB 里可能有副本，就需要 TLB flush，让 CPU invalid 旧缓存条目，下次访问重新从内存页表加载最新 PTE。

    只要页表项 (PTE) 被修改，任何持有该 PTE 缓存的硬件 (CPU TLB、GPU MMU 缓存) 都要失效缓存，保证缓存一致性。

    这段代码不会跑在 ZONE_DEVICE 私有页迁移分支，因为 ZONE_DEVICE 分支命中直接 goto discard 跳过这里。所以这里 *ptep_clear_flush_young_notify()* 内部调用的 mmu_notify 是针对 young 位维护，不是设备私有页迁移时的 mmu_notify()(前面 *mmu_notifier_invalidate_range_start()* 那处)。两处 mmu_notifier 触发场景完全不一样

- 1546 将物理页在 CPU cache 中的脏数据写回物理内存。

  即将移除 PTE 映射, cache 里可能还有未回写的缓存行。在撤销映射前，需要保证内存视图一致。

- 1547 判断是否启用延迟 TLB 刷新。

  TLB flush 是开销昂贵的操作，内核会做优化：不立即刷 TLB，把多个 TLB 失效 (invalid) 请求暂存起来，等到合适时机一次性批量刷。

- 1556 读出当前 present PTE 的值保存到 pteval (保留 dirty 位等标记)。原子的将页表项 pvmw.pte 清零 (PTE 变为 invalid)。

  这里没有刷 TLB，只是修改内存里的页表。CPU TLB 缓存还保留旧的有效映射条目，这就是 1547 行判断 defer(延迟) 的含义。

- 1558 标记当前 mm 地址空间存在待执行的 TLB 刷新任务。内核记住这个 mm 有 TLB 项延后统一失效；等一批 unmap 全部做完之后，再一次性执行 TLB flush，减少 flush 次数，降低开销。

- 1560 该 else 分支不使用批量延迟刷新，同步立即刷 TLB。

  原子的读出并清零内存 PTE。立刻执行 TLB invalid，丢弃 CPU TLB 里缓存的旧 PTE。

  上面两个分支执行完之后，内存中的 PTE 已经被清零 (present=0)。区别只在于 TLB 是立刻失效，还是延后批量失效。

- 1564-1565 检查刚才读出的旧 PTE 的 pteval 里 PTE dirty 位是否置位 (页面曾经被写), 若是，置位 struct page 里的 dirty flag。

- 1568 更新目标进程历史最大 RSS(resident set size, 常驻内存大小)。目前正在摘除 PTE，后续会减少进程 RSS。要在 RSS 下降之前更新 hiwater，先更新再减计数，否则峰值统计可能会偏低。

- 1570-1581 该段专门处理硬件毒页 (HWPoison): 内存硬件检测到物理页损坏，内核标记为 HWPoison。把当前 VA 的 PTE 替换成 hwpoison swap entry(毒页项，non-present)，用户态后续访问该 VA 会触发 hwpoison 缺页，内核发 SIGBUS 杀死进程。

  - 1570 判断 page->flags 是否置了 *PG_hwpoison*，该标准代表这是硬件损坏的物理页 (内存 ECC 报错等场景)。***TTU_IGNORE_HWPOISON***：忽略硬件毒页，正常执行 unmap，不安装 hwpoison 条目。
  - 1571 构造 hwpoison 类型的 swp_entry_t。和 migration entry 类似，属于特殊 swap entry，不是磁盘 swap。编码存入被毒坏的 page 的 PFN。函数会把 $present = 0$。后续 CPU 访问这个 VA，触发缺页，内核识别到 hwpoison entry，向上层进程抛出 ***SIGBUS***。
  - 1572 判断是否 HugeTLB 巨页 (注意: 不是 THP，是静态 Hugetlbfs 巨页)。
  - 1574-1575 给 Hugetlb 巨页写入非 present swap 类 PTE(hwpoison/migration entry)。*set_pte_at()* 不能直接用于巨页 PTE，要用这个巨页专用函数把 hwpoison pteval 写入 PMD/PTE。
  - 1576 下面这个分支处理普通 4K 页。
  - 1578 取出该页归属的 RSS 计数器（anon/file）。原子的减少进程 mm 对应 RSS 计数。因为原来 present PTE 已经被替换成 non-present hwpoison entry，页面不再计入进程常驻内存 RSS。
  - 1579 把 hwpoison 类型 non-present pte 写入页表 PTE 项。

- 1582-1596 页面是从未使用的干净页 + VMA 没有 userfaultfd，走这个分支。

  userfaultfd_armed：VMA 开启 uffd 用于用户态监听缺页事件。如果开启，内核不能直接静默销毁映射，必须留给 uffd 处理。

userfaultfd(简称 uffd) 全称：user fault file descriptor。是 linux 内核把缺页异常的处理权，从内核交给用户态程序 的机制。

正常缺页时:CPU 缺页，内核全权处理 (分配物理页、读磁盘、填 PTE)，用户态完全无感。

uffd 模式缺页：CPU 缺页，内核暂停 pagefault 线程，把缺页事件发给用户态的 uffd 管理线程，由用户态代码决定怎么解决缺页，用户态处理完再通过 ioctl 通知内核唤醒阻塞线程。

只要 VMA 开启 uffd，内核不能私自、静默销毁 PTE 映射。页面的映射销毁、缺页解析，必须交给 uffd 用户态管理器统一管理。内核不能私自破坏 uffd 维护的页表状态。

- 1595-1596 发起 *mmu_notifier* 通知：invalid $[address, address + PAGE_SIZE)$ 这段虚拟地址范围的设备 MMU 缓存。

  这里是普通 unused 页摘除映射的 notify，不是 ZONE_DEVICE 迁移分支的 notify。

- 1597-1598 本次 try_to_unmap 的目的是页面迁移/THP 拆分冻结，进入 migration entry 安装分支。
- 1602-1607 如果架构不允许本次 PTE 摘除/替换操作，则进入这个代码块处理并回滚。
  - 1603 回滚操作。

    前面流程已经调用 *ptep_clear_flush* 把内存 PTE 清零了,pteval 保存原来的 present PTE。如果 *arch_unmap_one* 失败，把原来的 present PTE 写回去，恢复页表，撤销本次 unmap 操作，防止页表损坏。
  - 1605 终止 pvmw 遍历器，释放页表锁，清理遍历上下文，防止锁泄漏。
- 1614-1615 构造迁移条目 *swp_entry_t*, 继承原先的 flag。
- 1617 如果旧 PTE 有软脏标记，则把这个标记复制到新的 migration entry pte 里面保存。迁移过程不能丢失 soft-dirty 状态，迁移完成恢复 PTE 时还原给新页
- 1619-1620 旧 PTE 带有 userfaultfd 写保护标记 (*UFFDIO_REGISTER_MODE_WP*)。如果 VMA 启用 uffd 写保护，把 uffd-wp 标记 ** 复制到 migration entry。页面迁移冻结期间，要保留 uffd 写保护语义；迁移完成恢复映射时，重新还原 uffd wp 状态。
- 1621 把构造好的 migration entry(non-present pte) 写入页表 PTE 项。
- 1626-1692 该代码分支处理匿名页的 swapout 换出逻辑。文件页不会进入这个分支。
  - 1627 从 subpage->private 字段取出预先分配好的 swapentry。匿名页准备 swapout 时，内核提前分配 swap slot，把 swapentry 存进 page->private。entry 编码 swap 设备号 + swap 槽位 offset，就是普通磁盘 swap entry，不是 migration/hwpoison 特殊 entry。
  - 1633 这行是做一致性校验：PageSwapBacked 和 PageSwapCache 两个 flag 必须同真同假 (同时为真或假)，不相等就是 page 元数据损坏。
  - 1644 目标匿名页还没有绑定 swap slot 槽位，典型就是 *MADV_FREE* 页面。

    MADV_FREE：用户态告诉内核，这块内存可以回收；内核不立即回收，等内存压力上来再处理；页面此时没有分配 swap slot。
  - 1649 扣减进程 mm 的匿名页 RSS 计数。
  - 1650 直接跳到函数末尾 discard 标签。页面直接丢弃，不需要写入 swap 文件，物理页直接释放。这是 MADV_FREE 的快速回收优化

- 1657-1661 下面是脏的 MADV_FREE 页面的处理。
- 1658 给 page 打上 ***PG_swapbacked*** 标记，代表这个匿名页现在需要 swap 后端支持
- 1664-1669 一个 swap 槽位可以被多个进程共享 (共享匿名页 COW 场景)。swap_duplicate 做引用计数递增；失败说明 swap 槽不可用，本次 unmap 失败。
- 1670-1674 架构相关 unmap 钩子检查。架构层不允许本次操作, 本次失败，回滚。和前面 migration 分支的 *arch_unmap_one* 是同一个函数。
- 1676-1681 每个进程 *mm_struct* 有 mmlist 链表，用于内存回收遍历。如果当前 mm 还没加入全局 mmlist 链表，挂载到 init_mm.mmlist。保证内存回收扫描能找到这个 mm。
- 1682 扣减进程的匿名页 RSS 计数。
- 1683 增加进程 swap 条目计数：这个 VA 现在指向 swap 槽，计入 swap 占用统计。
- 1684 把普通 swap entry 转换成 non-present pte。
- 1689 把 swap entry 写入 PTE。现在 VA 的 PTE present=0，指向 swap 分区的 slot。后续访问这个 VA 触发缺页，内核从 swap 磁盘读取页面回来。
- 1691 通知所有 *mmu_notifier* 注册的驱动 (HMM/GPU): 该 VA 映射已经换成 swapentry，设备侧 MMU 缓存必须失效，不能继续使用旧 VA→PA 映射。

- 1704 页在 page cache 里、数据在磁盘上，不需要写 swap entry，直接减 RSS 计数即可。

9.2 PTE(页表项)

页表项 (***PTE***) 指页表里面的每一条记录。它保存着虚拟地址到物理地址的映射信息 (*PFN*, 物理页帧号)，同时附带访问权限、内存属性、页状态标记等。

每个虚拟页 (4K)，在页表里面有唯一对应的 *PTE* 项。多个不同虚拟页 (不同进程、同一个进程多个 VMA)，它们各自独立的 *PTE* 可以映射到同一个物理页帧！但是一个虚拟页，最多只有一个 *PTE* (在它所属进程的页表里)

arm64 和 x86_64 的 PTE 位图对比的表格如下图:

bit	x86_64 名称	x86_64 说明	x86_64 宏	ARM64 名称	ARM64 说明	ARM64 宏
0	Present (P)	1= 页面有效；0= 进入 swp entry 解析	`pte_present()`	Valid(V)	1= 页面有效；0= 进入 swp entry 解析	`pte_present()`
1	RW	1= 允许写，0= 只读	`pte_write()`	Page	与 bit0 组合`0b11`代表 L3 页描述符	`PTE_TYPE_PAGE`
2	USER	1= 用户态可访问	`pte_user()`	AttrIndx[0]	MAIR 内存属性索引 bit0	`pte_attr_indx()`
3	PWT	写通缓存策略	—	AttrIndx[1]	MAIR 内存属性索引 bit1	`pte_attr_indx()`
4	PCD	禁止缓存	—	AttrIndx[2]	MAIR 内存属性索引 bit2	`pte_attr_indx()`
5	Accessed (Young)	硬件访问自动置 1；内核软件清零	`pte_young()`/ `pte_old()`	NS	Non-Secure，安全扩展标记	—
6	Dirty(D)	硬件写自动置 1；内核软件清零	`pte_dirty()`	AP[1]	AP 位组：1= 允许 EL0 用户态访问	`pte_user()`
7	PAT	Page Attribute Table，内存类型	—	AP[2]	AP 位组：1= 只读	`pte_write()`
8	Global(G)	全局 TLB，CR3 切换不失效	—	SH[0]	共享域 bit0	—
9	Soft-Dirty	软件标记，硬件忽略；内存追踪	`pte_soft_dirty()`	SH[1]	共享域 bit1	—
10	AVL2	OS 可用保留位	—	AF	Access Flag；HA 开启硬件访问自动置 1	`pte_young()`/ `pte_mkyoung()`
11	AVL3	OS 可用保留位	—	nG	not-Global，私有 TLB 条目（带 ASID）	—
12∼47	PFN(低 36bit)	物理页帧号起始	`pte_pfn()`	PFN	物理页帧号	`pte_pfn()`
48∼51	PFN(高 4bit)	PFN 高位	`pte_pfn()`	Reserved	硬件保留，必须 0	—
51	—	—	—	DBM	ARMv8.2 硬件脏页管理控制位	硬件字段
52	PFN 剩余高位	PFN 剩余高位	`pte_pfn()`	Contiguous	连续页 TLB 合并 hint	`PTE_CONT`
53	Reserved	硬件保留	—	PXN	EL1 禁止执行	`pte_pxn()`
54	Reserved	硬件保留	—	UXN	EL0 禁止执行	`pte_uxn()`
55	Reserved	硬件保留	—	Dirty	HD 开启：硬件写自动置脏	`PTE_DIRTY`/ `pte_dirty()`
56	Reserved	硬件保留	—	Special	VM_PFNMAP 裸物理映射标记	`PTE_SPECIAL`
57	Reserved	硬件保留	—	Write	软件可写标记，配合 DBM	`PTE_WRITE`
58	Reserved	硬件保留	—	DEVMAP	ZONE_DEVICE devmap 页标记	`PTE_DEVMAP`/ `pte_devmap()`
59∼62	Reserved	硬件保留	—	Reserved	内核可用保留位	—
63	NX	NoExecute，禁止执行	`pte_nx()`	Reserved	内核可用保留位	—

Table 9–1: x86_64 vs ARM64 (HA+HD+DBM 开启, Linux 5.10, L3 4K 页 PTE) 对比表

当 $PTE\ bit[0] = 0$ 时，硬件 *MMU* 停止地址翻译硬件直接报 *translation fault*, 整个 64 bit 项被内核

解析为 swp_entry_t，内核依靠此时的 *PTE* 高位的编码区分为三类 swp 项 (注意: 此时和上面表格描述的编码是两套独立编码体系)。*bit*[0] = 1 时才是正常有效物理页 *PTE*，由硬件 *MMU* 解析。

内核中 x86_64 在 PTE 的 *bit*[0] = 0 时，编码 swp 项的类型 type 用的是 *bit*[63 : 59](*arch/x86/include/asm/pgtable_64.h*)。

```
/*
 * Encode and de-code a swap entry
 *
 * |      ...              | 11| 10|  9|8|7|6|5| 4| 3|2| 1|0| <- bit number
 * |      ...              |SW3|SW2|SW1|G|L|D|A|CD|WT|U| W|P| <- bit names
 * | TYPE (59-63) | ~OFFSET (9-58)  |0|0|X|X| X| X|F|SD|0| <- swp entry
 *
 * ...
 */
#define SWP_TYPE_BITS       5
...
/* Extract the high bits for type */
#define __swp_type(x) ((x).val >> (64 - SWP_TYPE_BITS))
```

arm64 在 PTE 的 *bit*[0] = 0 时，编码 swp 项的类型 type 用的是 *bit*[7 : 2](*arch/arm64/include/asm/pgtable.h*)。

```
/*
 * Encode and decode a swap entry:
 *  bits 0-1:   present (must be zero)
 *  bits 2-7:   swap type
 *  bits 8-57:  swap offset
 *  bit  58:    PTE_PROT_NONE (must be zero)
 */
#define __SWP_TYPE_SHIFT    2
#define __SWP_TYPE_BITS     6
```

此处是 5.10.0 内核的情况，内核版本不同 swap type 在 *PTE* 中的编码字段位会有不同，甚至 swap type 的编码位宽也会有差异。

x86_64 和 *arm64* 的 migration/swap/hwpoison entry 位域对比如下：

项	*swp type*	*offset* 内容	说明
普通 *swap entry*	*swap* 设备号	*swap slot* 编号	页面换出至磁盘 *swap*；缺页从磁盘载入
migration entry	*SWP_MIGRATION_READ* / *SWP_MIGRATION_WRITE*	*page* 的 *PFN*	页面正在迁移；访问阻塞等待迁移完成；区分普通内存与设备私有页靠 *is_zone_device_page()*
hwpoison entry	*SWP_HWPOISON*	损坏 *page* 的 *PFN*	物理页硬件损坏；访问触发 *SIGBUS* 终止进程
x86_64	*bits [63:59]*	—	—
arm64	*bits [7:2]*	—	—

Table 9–2: swp_entry_t: migration / swap / hwpoison entry 位域对比 (x86_64/ARM64)

综上，三种 swp entry 的唯一区分依据为 type 域。

9.2.1 PTE 什么时候加载进 TLB

CPU 做虚拟地址访存发生页表遍历 (page walk) 成功找到合法 *PTE* 之后，硬件自动把这条 *PTE* 缓存进 *TLB*。软件不会手动写 *TLB*，软件只能 invalidate(失效)*TLB*。

只有 $present = 1$(驻留内存) 的有效页的 *PTE* 才会被存入 *TLB*。*swp_entry* 不会进 *TLB*，会直接报缺页异常交给内核处理。

CPU 访问虚拟地址的完整动作:

- CPU 发起 load/store，给出虚拟地址 VA.
- *MMU* 先查 *TLB*:
  - *TLB* 命中: 直接拿 *TLB* 缓存的 *PTE*，得到物理地址，跳过页表遍历。
  - *TLB* 未命中(miss): 启动页表遍历，逐级通过页目录查找 *PTE*。
- 硬件读到 *PTE*:
  - 如果 *PTE* $present == 1$，硬件自动把该 PTE 填入 TLB，然后用 PTE 里的物理页地址访问内存。

    x86: 硬件自动置位 *PTE* 的 Accessed (Young)、Dirty 位。

    arm64: 硬件自动置 *AF* 位。
  - 如果 *PTE* $present == 0$(对于 arm64 而言是 valid 位), 不会进 *TLB*，硬件触发缺页异常 (pagefault), 陷入内核。内核解析 swp_entry 做换入/页面迁移/毒页处理。处理完成内核重新写入合法 *PTE* 之后，下一次 CPU 访问 VA(虚拟地址)，页表遍历才会把这个新 *PTE* 缓存进 *TLB*。

    PTE = 全 0，纯粹未分配映射（无任何 swp 编码），走匿名页/文件页缺页分配新页；

    PTE bit0=0，但高位有合法 swp 编码，属于 swap entry/migration entry/hwpoison entry，走 swapin/migration/hwpoison 的处理。

9.2.1.1 TLB 加载和 invalid

下面是填充 *TLB* 和 invalid *TLB* 的一些场景:

- 首次访问某个虚拟地址,*TLB* 中不存在对应 *PTE* 项，*TLB* 不命中，因此会进行页表遍历，硬件自动加载 *PTE* 进 *TLB*。这种场景最常见。
- 内核修改页表项 (譬如清 0 young(access) 位)，执行 *TLB* invalid 后旧 *TLB* 项失效。后续再次访问该虚拟地址 (VA) 时 *TLB* 不命中，硬件重新读 *PTE* 进 *TLB*。
- *TLB* 项被硬件替换 (*TLB* 容量有限，淘汰旧条目), 再次访问对应 VA 虚拟地址时，*TLB* miss(不命中)，重新加载 *PTE* 进 *TLB*。
- 进程切换 (switch_mm), 切换到新进程的 mm(*CR3/TTBR0_EL1* 中的 pgd 更换)。旧 *TLB* 项属于旧地址空间，需要 invalid。新进程访问 VA(虚拟地址) 触发 *TLB* miss，重新填充 *TLB*

- *THP*/大页拆分，invalid 原大页 *TLB* 项。访问拆分后的子页发生 *TLB* miss，加载 4K 基页的 *PTE* 入 *TLB*。

10

异构内存管理

异构内存管理 (*HMM:Heterogeneous Memory Management*) 是 Linux 内核用来统一管理系统主存 (RAM) 和异构设备内存 (GPU 显存、CXL 设备内存等)，让设备内存可以像普通系统内存一样参与内核页表、反向映射、缺页异常、页面迁移的标准 MM 框架。

HMM 异构内存管理的目标是跨设备共享虚拟地址空间, 页面迁移是实现该地址空间透明访问的核心机制。页迁移本身是通用内存管理功能, HMM 复用并扩展了此功能, 增加了设备私有内存作为迁移的一侧。

对于设备私有内存 ***MEMORY_DEVICE_PRIVATE***(如 GPU 显存):

- CPU 不能建立有效 present PTE；CPU 页表里面只能放 device private entry(non-present 的特殊 swp 项)。
- CPU 访问该 VA 触发缺页异常，内核识别这是 device private entry，调用 HMM 驱动回调，把显存页面迁移 (搬运) 到系统 RAM。
- 迁移 (搬运) 完成后，PTE 替换成指向系统 RAM 的有效 PTE，CPU 才可以访问。

VRAM 的 *ZONE_DEVICE* *struct page* 是 (GPU) 设备驱动通过 KPI 接口函数 *devm_memremap_pages()* 注册出来的，代表 *GPU* 显存物理地址；这块 *VRAM* 物理内存是 *GPU* 硬件的显存，不是系统 *RAM*。*HMM* 迁移的本质是**数据拷贝**。

异构内存管理中两种页迁移方向:

- 系统 RAM 页迁移到 GPU 设备私有内存 (迁移到显存)。
- GPU 设备私有内存页迁移到系统 RAM。

10.1 GPU 私有内存迁移到系统 RAM

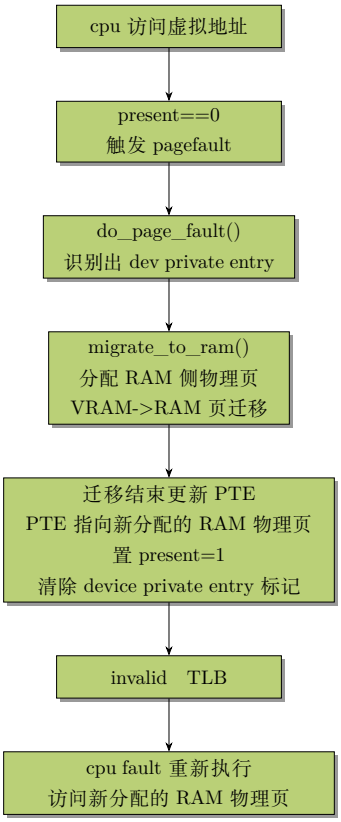
GPU 私有内存迁移到系统 RAM 有两条情况:

- cpu 缺页触发的单页回迁。
- 批量回迁。

10.1.1 单页回迁

cpu 在用户态访问虚拟内存时, 若硬件页表遍历发现 PTE 的 present 位为 0, 就会会触发 pagefault(缺页异常)。异常经由对应架构的异常向量表入口分发，如是用户态虚拟地址产生的 pagefault, 最终会进入内核缺页处理函数 *do_page_fault ()*(下文以 arm64 架构为例展开分析)。

然后再检查 PTE 中的 swap entry 项是否是设备私有页的 device private entry 项，即 type 类型是 SWP_DEVICE_READ 或 SWP_DEVICE_WRITE 之一。若是，则调用 GPU 设备驱动的 *migrate_to_ram()* 回调进行页面从设备私有内存到 cpu 主 RAM 的迁移。主要步骤如下图:



其中 do_page_fault() 内部执行到 GPU 设备驱动回调 $->$ *migrate_to_ram*()(图中标号2) 的调用栈如下:

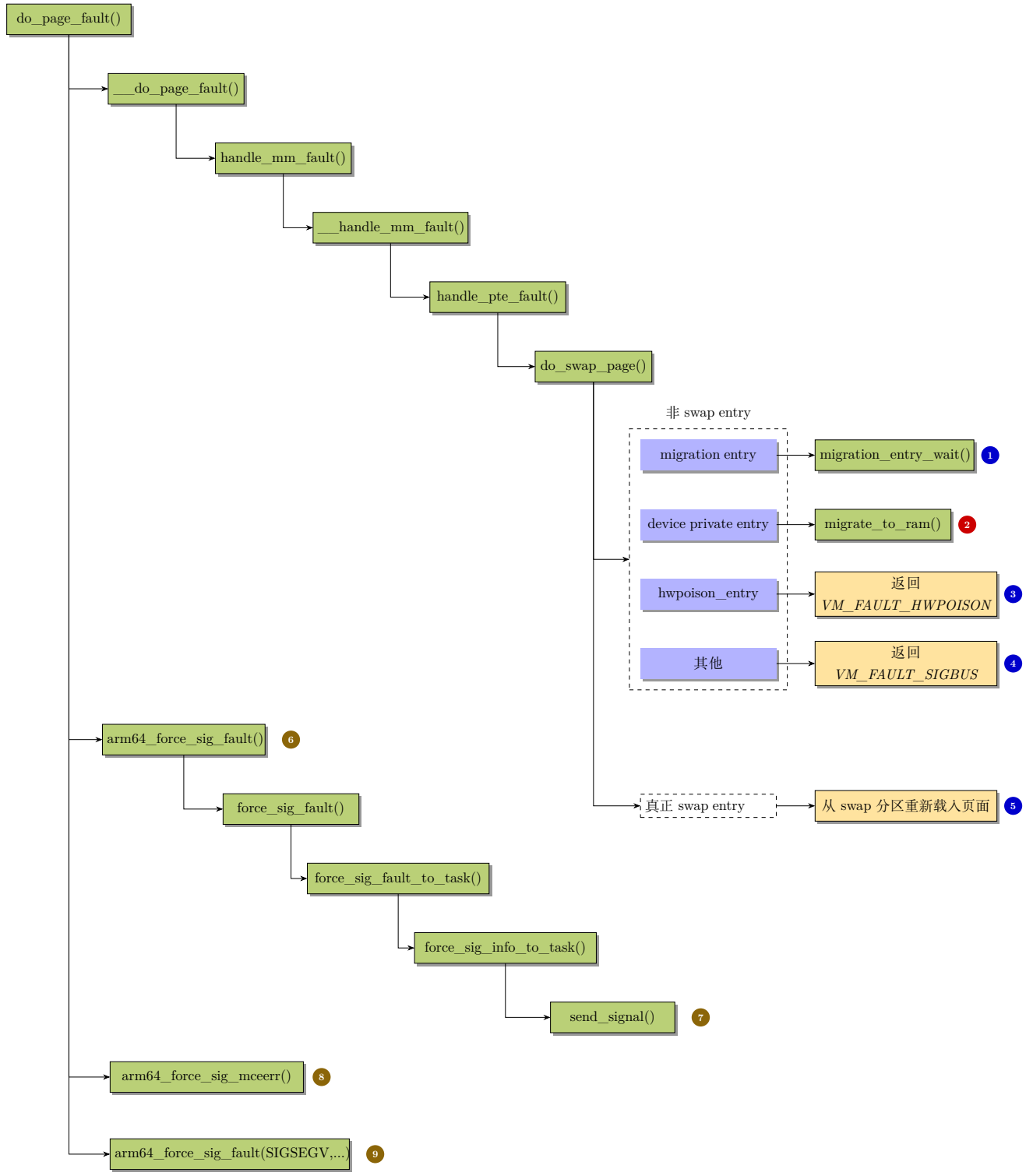


图 10–1: do_page_fault() 的调用栈

说明:

- 1 缺页异常 *do_page_fault()* 解析 PTE，发现是迁移项 (*migration entry*) 时，阻塞当前 fault 进程，等待页面迁移操作完成，迁移完成后被唤醒，重新重试。
- 2 识别到是 device private entry，触发 VRAM– >RAM 的回迁。

- 3、4 两个最终都会给用户态进程发送信号 ***SIGBUS***, 但是 si_code 不同。

  VM_FAULT_HWPOISON 表示物理内存硬件损坏。*VM_FAULT_SIGBUS* 表示内存映射/swap 逻辑错误，硬件本身没问题。

  6、8、9和页迁移无关，主要是处理3、4返回的错误。并给对应的目标进程发送 sig 信号(此图8、9省略了对 *send_signal()* 的调用)。

  其中6判断 *___do_page_fault()* 返回值是 *VM_FAULT_SIGBUS*, 则发送 *SIGBUS*,*si_code=BUS_ADRERR* 给对应进程。8判断返回值是 *VM_FAULT_HWPOISON*, 也发送 *SIGBUS*, 但是 *si_code=BUS_MCEERR_AR*。9是其它错误场景，统一发送信号 *SIGSEG*。

- 5 如果是真正的 swap entry, 就执行 swap in, 把页面从 swap 分区重新载入 RAM，修复 PTE 为 *present* = 1 的有效页表项。

10.1.1.1 和异构内存管理无关但重要的主题 · signal 的组装和发送

上图中的 *send_signal()* 是信号发送的底层调用接口，它是 *___send_signal()* 的包裹函数。会把信号挂入目标进程/线程的 pending 信号队列，标记待处理并尝试唤醒目标线程。而上层调用者会通过 info 这个参数的" 编码" 把意义带进来。

在不同的调用者调用下，*info->si_code* 的意义会有所不同。为了明确 *kernel_siginfo info* 结构体成员的意义，下面看下从 force_sig_fault_to_task() 开始到 send_signal() 的执行路线。虽然这个执行路径和内存管理关系不大，但是为了之后在工作中碰到 crash 时，可以从 dump 文件保存的传递给用户空间的 *siginfo_t* 中找到 crash 的原因，下面详细分析一下该执行路径。

(如果继续跟踪解析异构内存管理可以跳过该小节)

force_sig_fault_to_task()

```
1653 int force_sig_fault_to_task(int sig, int code, void __user *addr
1654     ___ARCH_SI_TRAPNO(int trapno)
1655     ___ARCH_SI_IA64(int imm, unsigned int flags, unsigned long isr)
1656     , struct task_struct *t)
1657 {
1658     struct kernel_siginfo info;
1659
1660     clear_siginfo(&info);
1661     info.si_signo = sig;
1662     info.si_errno = 0;
1663     info.si_code  = code;
1664     info.si_addr  = addr;
1665 #ifdef __ARCH_SI_TRAPNO
1666     info.si_trapno = trapno;
1667 #endif
1668 #ifdef __ia64__
1669     info.si_imm = imm;
1670     info.si_flags = flags;
1671     info.si_isr = isr;
1672 #endif
1673     return force_sig_info_to_task(&info, t);
1674 }
```

- 1658-1660 定义内核态 siginfo 结构体并清 0。
- 1661-1672 *struct kernel_siginfo* 结构体的定义如下:

```
#ifndef __ARCH_HAS_SWAPPED_SIGINFO
#define __SIGINFO           \
struct {                \
    int si_signo;           \
    int si_errno;           \
    int si_code;            \
    union __sifields _sifields; \
}
#else
#define __SIGINFO           \
struct {                \
    int si_signo;           \
    int si_code;            \
    int si_errno;           \
    union __sifields _sifields; \
}
#endif /* __ARCH_HAS_SWAPPED_SIGINFO */
```

这里的 *__ARCH_HAS_SWAPPED_SIGINFO* 是架构宏开关，用来判断当前架构的 *siginfo_t* 是否把 *si_errno* 和 *si_code* 两个字段顺序交换 (swapped)。绝大多数架构 (x86、arm64) 没有定义 *__ARCH_HAS_SWAPPED_SIGINFO*，所以会走 *#ifndef* 下面的代码。而 MIPS 架构是特例，它的 ABI 规定的用户态 siginfo 里 *si_errno* 和 *si_code* 两个成员顺序是互换的。

内核在把内核态构造的 siginfo_t 拷贝到用户空间时:

- 如果架构没有定义 *__ARCH_HAS_SWAPPED_SIGINFO*(x86/arm64)，那么直接按标准顺序拷贝 si_signo, si_errno, si_code。
- 如果架构有定义 *__ARCH_HAS_SWAPPED_SIGINFO*(MIPS)，那么拷贝时交换 si_errno、si_code 的位置来适配 MIPS 用户态 ABI。

- 1661 获取传入的信号编号。
- 1662 填充关联的错误号，内存硬件故障或缺页异常 (当前场景) 固定填 0。

  si_errno 只用于少数系统调用相关信号,SIGBUS/SIGSEGV 这类硬件异常信号不使用此字段，所以置 0。
- 1663 获取调用者传入的故障原因码。

  HWPoison 故障码是 *BUS_MCEERR_AR*。VM_FAULT_SIGBUS 故障码是 *BUS_ADRERR*。对于越界的故障 (SIGSEG)，如果 fault 是 VM_FAULT_BADACCESS，故障码是 *SEGV_ACCERR*，否则是 *SEGV_MAPERR*。
- 1664 获取触发故障的虚拟地址。用户空间信号处理函数可以读取 *siginfo->si_addr*，拿到触发信号 SIGBUS/SIGSEGV 的虚拟地址。

- 1673 向指定线程发送组装好的 *kernel_siginfo* 信号。

force_sig_fault_to_task() ▸ *force_sig_info_to_task()*

```
1309 static int
1310 force_sig_info_to_task(struct kernel_siginfo *info, struct task_struct *t)
1311 {
1312     unsigned long int flags;
1313     int ret, blocked, ignored;
1314     struct k_sigaction *action;
1315     int sig = info->si_signo;
```

```
1316
1317    spin_lock_irqsave(&t->sighand->siglock, flags);
1318    action = &t->sighand->action[sig-1];
1319    ignored = action->sa.sa_handler == SIG_IGN;
1320    blocked = sigismember(&t->blocked, sig);
1321    if (blocked || ignored) {
1322        action->sa.sa_handler = SIG_DFL;
1323        if (blocked) {
1324            sigdelset(&t->blocked, sig);
1325            recalc_sigpending_and_wake(t);
1326        }
1327    }
1328    /*
1329     * Don't clear SIGNAL_UNKILLABLE for traced tasks, users won't expect
1330     * debugging to leave init killable.
1331     */
1332    if (action->sa.sa_handler == SIG_DFL && !t->ptrace)
1333        t->signal->flags &= ~SIGNAL_UNKILLABLE;
1334    ret = send_signal(sig, info, t, PIDTYPE_PID);
1335    spin_unlock_irqrestore(&t->sighand->siglock, flags);
1336
1337    return ret;
1338 }
```

- 1315 获取取要发送的信号编号 (如 *SIGBUS*、*SIGSEGV*)。
- 1317 获取任务的信号锁，关闭本地中断并保存当前中断标志。
- 1318 每个信号都绑定一个处理动作，保存在 `task_struct->sighand->action[]`，更确切的说，是保存在 `action->sa.sa_handler`，一共三类取值 (指针):
  - ***SIG_IGN*** 忽略信号。
  - ***SIG_DFL*** 默认动作。
    SIG_IGN 和 ***SIG_DFL*** 是特殊的指针:

    ```
    typedef __signalfn_t __user *__sighandler_t;
    ... ...
    #define SIG_DFL ((__sighandler_t)0) /* default signal handling */
    #define SIG_IGN ((__sighandler_t)1) /* ignore signal */
    ```

  - **函数指针**用户自定义的信号处理函数。

信号编号从 1 开始 (编号如下图)，而数组下标是从 0 开始，因此 −1 得到信号对应处理句柄的数组索引。

```
comlee:~ >>>kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

- 1319 判断目标信号当前是否被设置为忽略 (SIG_IGN)。
- 1320 检查信号掩码，判断信号是否被屏蔽。
- 1323-1326 绕过对 signal 处理的忽略/屏蔽,强制要求处理。这就是当前函数 *force_sig_info_to_task()* 名称中force的含义。

  如果信号被忽略或者被屏蔽:

  - (无视" 忽略") 把信号处理函数强制重置为 *SIG_DFL*(默认动作)。

    譬如: 如果原本是 *SIG_IGN*, 此处就不再忽略，改用默认行为 (比如 *SIGBUS* 默认行为是杀进程)。
  - 如果是被屏蔽:

    从屏蔽掩码里删掉这个信号，解除屏蔽。如果任务正在睡眠，唤醒任务，让它处理信号。

  这里需要注意的是,1321 行 if 判断只在 `blocked || ignored` 成立时，才会把 handler 改成 SIG_DFL。如果原本是用户自定义处理句柄 (*sa_handler*)，这个分支根本不会进，所以此处不会覆盖用户自定义信号处理函数。
- 1332-1333 清除不可杀标记，允许默认信号把进程杀死。

  被 ptrace 调试的任务，不清除 SIGNAL_UNKILLABLE 标记。init 进程默认不可杀。
- 1335 把构造好的 *kernel_siginfo* 放到目标任务/进程的信号队列。

force_sig_fault_to_task() > *force_sig_info_to_task()* > *send_signal()*

```
1209 static int send_signal(int sig, struct kernel_siginfo *info, struct task_struct *t,
1210             enum pid_type type)
1211 {
1212     /* Should SIGKILL or SIGSTOP be received by a pid namespace init? */
1213     bool force = false;
1214 
1215     if (info == SEND_SIG_NOINFO) {
1216         /* Force if sent from an ancestor pid namespace */
1217         force = !task_pid_nr_ns(current, task_active_pid_ns(t));
1218     } else if (info == SEND_SIG_PRIV) {
1219         /* Don't ignore kernel generated signals */
1220         force = true;
1221     } else if (has_si_pid_and_uid(info)) {
1222         /* SIGKILL and SIGSTOP is special or has ids */
1223         struct user_namespace *t_user_ns;
1224 
1225         rcu_read_lock();
1226         t_user_ns = task_cred_xxx(t, user_ns);
1227         if (current_user_ns() != t_user_ns) {
1228             kuid_t uid = make_kuid(current_user_ns(), info->si_uid);
1229             info->si_uid = from_kuid_munged(t_user_ns, uid);
1230         }
1231         rcu_read_unlock();
1232 
1233         /* A kernel generated signal? */
1234         force = (info->si_code == SI_KERNEL);
1235 
1236         /* From an ancestor pid namespace? */
1237         if (!task_pid_nr_ns(current, task_active_pid_ns(t))) {
1238             info->si_pid = 0;
1239             force = true;
1240         }
```

```
1241     }
1242     return __send_signal(sig, info, t, type, force);
1243 }
```

- 1215-1217 发送不带 siginfo 附加信息的信号 (传统简单的用户进程发送的信号) 会进入此分支。此时入参 info 不是一个真实的 *struct kernel_siginfo* 结构体指针 (结构体地址)，是一个特殊的魔法指针值, 表示不带 siginfo 信息的信号。*SEND_SIG_NOINFO* 定义如下:

```
#define SEND_SIG_NOINFO ((struct kernel_siginfo *) 0)
```

  从上层 pid 命名空间向子命名空间发无附加信息信号，开启 *force=true*，允许杀掉子命名空间的 init 进程。

- 1218-1219 内核产生的信号，不能被 pid 命名空间 init 保护机制拦截, 直接置 $force = true$。

  设备私有页故障产生的 *SIGBUS* 不属于这个 else if，该场景场景传入的是真实 *kernel_siginfo* 的地址，不是 *SEND_SIG_PRIV*。

  SEND_SIG_PRIV 定义如下:

```
#define SEND_SIG_PRIV   ((struct kernel_siginfo *) 1))
```

- 1221 根据信号编号与 *si_code* 得到*siginfo*联合体布局类型。

  判断当前 siginfo 所使用的 union 分支的成员布局是否包含 *si_pid* 与 *si_uid* 成员；返回 true 代表可以安全访问这两个字段，返回 false 代表该布局没有这两个成员，不应读写。对于 *SIGBUS*、*SIGSEGV* 这类硬件 fault 信号，布局类型为 *SIL_FAULT*，所以会跳过该代码块。

- 1226 获取目标进程对应的用户态命名空间。

- 1227 如果信号发送者 task 和接收者 task 是否在同一个用户命名空间，uid 在不同命名空间 ID 号不一样，需要映射。

- 1228 把命名空间里信号发送者的 uid，转为内核全局 kuid。

- 1229 把全局 kuid 翻译成目标 user 命名空间可见的 uid，回填到 `info->si_uid`。这样用户态拿到 siginfo 里的 *si_uid*，看到的是自己用户态命名空间下的 uid 号，而不是外层命名空间的 ID。

- 1234 判断目标信号是否内核生成的信号。

  设备私有页缺页产生的 *SIGBUS*，*si_code* 不是 *SI_KERNEL*，是故障类 *si_code*(比如 *BUS_ADRERR*)，所以 $force = false$。

  不同的信号来源会使 *si_code* 的意义不同。kill/raise/用户态发送信号的 *si_code=SI_USER*。内核主动生成、但不属于硬件故障信号的 *si_code=SI_KERNEL*。硬件/内存故障类 *si_code* 是 BUS_ADRERR/SEGV_MAPERR 等 (SIL_FAULT)。表格对比如下:

项	*SI_USER*	*SI_KERNEL*	*BUS_ADDRERR*
si_code	*SI_USER*	*SI_KERNEL*	*BUS_ADRERR*
siginfo 布局	*SIL_KILL*	*SIL_KILL*	*SIL_FAULT*
union 可用字段	*si_pid, si_uid*	*si_pid, si_uid*	*si_addr, si_trapno*
sig 来源	*用户态 kill() 发送*	*内核 deliver*	*CPU 硬件缺页、内存故障*

- 1237-1240 判断信号发送进程是否处于目标进程的祖先 pid 命名空间。若是，清空信号发送者 pid。子命名空间进程不能看到父命名空间进程的 pid。并且开启 force，允许投递 *SIGKILL/SIGSTOP* 给 pid 命名空间的 init。
- 1242 用底层函数 *___send_signal* 进行真正的信号 pending 队列的插入。

force_sig_fault_to_task() *force_sig_info_to_task()* *send_signal()* *___send_signal()*

```
1071 static int __send_signal(int sig, struct kernel_siginfo *info, struct task_struct *t,
1072             enum pid_type type, bool force)
1073 {
1074     struct sigpending *pending;
1075     struct sigqueue *q;
1076     int override_rlimit;
1077     int ret = 0, result;
1078 
1079     assert_spin_locked(&t->sighand->siglock);
1080 
1081     result = TRACE_SIGNAL_IGNORED;
1082     if (!prepare_signal(sig, t, force))
1083         goto ret;
1084 
1085     pending = (type != PIDTYPE_PID) ? &t->signal->shared_pending : &t->pending;
1086     /*
1087      * Short-circuit ignored signals and support queuing
1088      * exactly one non-rt signal, so that we can get more
1089      * detailed information about the cause of the signal.
1090      */
1091     result = TRACE_SIGNAL_ALREADY_PENDING;
1092     if (legacy_queue(pending, sig))
1093         goto ret;
1094 
1095     result = TRACE_SIGNAL_DELIVERED;
1096     /*
1097      * Skip useless siginfo allocation for SIGKILL and kernel threads.
1098      */
1099     if ((sig == SIGKILL) || (t->flags & PF_KTHREAD))
1100         goto out_set;
1101 
1102     /*
1103      * Real-time signals must be queued if sent by sigqueue, or
1104      * some other real-time mechanism.  It is implementation
1105      * defined whether kill() does so.  We attempt to do so, on
1106      * the principle of least surprise, but since kill is not
1107      * allowed to fail with EAGAIN when low on memory we just
1108      * make sure at least one signal gets delivered and don't
1109      * pass on the info struct.
1110      */
1111     if (sig < SIGRTMIN)
1112         override_rlimit = (is_si_special(info) || info->si_code >= 0);
1113     else
1114         override_rlimit = 0;
1115 
1116     q = __sigqueue_alloc(sig, t, GFP_ATOMIC, override_rlimit);
1117    if (q) {
1118         list_add_tail(&q->list, &pending->list);
1119         switch ((unsigned long) info) {
1120         case (unsigned long) SEND_SIG_NOINFO:
1121             clear_siginfo(&q->info);
1122             q->info.si_signo = sig;
1123             q->info.si_errno = 0;
1124             q->info.si_code = SI_USER;
```

```
1125            q->info.si_pid = task_tgid_nr_ns(current,
1126                                    task_active_pid_ns(t));
1127            rcu_read_lock();
1128            q->info.si_uid =
1129                from_kuid_munged(task_cred_xxx(t, user_ns),
1130                                 current_uid());
1131            rcu_read_unlock();
1132            break;
1133        case (unsigned long) SEND_SIG_PRIV:
1134            clear_siginfo(&q->info);
1135            q->info.si_signo = sig;
1136            q->info.si_errno = 0;
1137            q->info.si_code = SI_KERNEL;
1138            q->info.si_pid = 0;
1139            q->info.si_uid = 0;
1140            break;
1141        default:
1142            copy_siginfo(&q->info, info);
1143            break;
1144        }
1145    } else if (!is_si_special(info) &&
1146           sig >= SIGRTMIN && info->si_code != SI_USER) {
1147        /*
1148         * Queue overflow, abort.  We may abort if the
1149         * signal was rt and sent by user using something
1150         * other than kill().
1151         */
1152        result = TRACE_SIGNAL_OVERFLOW_FAIL;
1153        ret = -EAGAIN;
1154        goto ret;
1155    } else {
1156        /*
1157         * This is a silent loss of information.  We still
1158         * send the signal, but the *info bits are lost.
1159         */
1160        result = TRACE_SIGNAL_LOSE_INFO;
1161    }
1162
1163 out_set:
1164    signalfd_notify(t, sig);
1165    sigaddset(&pending->signal, sig);
1166
1167    /* Let multiprocess signals appear after on-going forks */
1168    if (type > PIDTYPE_TGID) {
1169        struct multiprocess_signals *delayed;
1170        hlist_for_each_entry(delayed, &t->signal->multiprocess, node) {
1171            sigset_t *signal = &delayed->signal;
1172            /* Can't queue both a stop and a continue signal */
1173            if (sig == SIGCONT)
1174                sigdelsetmask(signal, SIG_KERNEL_STOP_MASK);
1175            else if (sig_kernel_stop(sig))
1176                sigdelset(signal, SIGCONT);
1177            sigaddset(signal, sig);
1178        }
1179    }
1180
1181    complete_signal(sig, t, type);
1182 ret:
1183    trace_signal_generate(sig, info, t, type != PIDTYPE_PID, result);
1184    return ret;
1185 }
```

- 1079 进入 *___send_signal()* 必须持有自旋锁, 没有锁会直接触发 panic。

- 1081-1083 信号 deliver 前的校验, 判断进程是否存在、是否处于可接收信号状态等。
- 1085 选择信号 pending 队列。
  - *PIDTYPE_PID* 信号发给单个线程, 使用线程私有的信号 pending 队列 ($->pending$)
  - *PIDTYPE_TGID* 发给整个进程 (线程组), 使用线程组共享的信号 pending 队列 $->signal->shared_pending$。
- 1091-1093 判断目标信号是否是**非实时信号**, 并且该信号已经在 pending 位图中置位。

  非实时信号 ($sig < SIGRTMIN$) pending 位图中同一个信号只允许存在一个标记，不重复排队。实时信号 ($\geq SIGRTMIN$) 不受这个限制，可以多次入队多个 sigqueue。
- 1099-1100 SIGKILL 不需要附加信息, 内核线程不处理用户态信号。二者都不需要分配 sigqueue 和保存 siginfo。直接跳转到 out_set，只置位 pending 位图。

  SIGBUS、用户进程都不进入该分支。
- 1111-1112 针对非实时信号 ($sig < SIGRTMIN$) 满足下面两个条件之一的话:
  - *is_si_special(info)* info 是魔术数值指针 *SEND_SIG_NOINFO/SEND_SIG_PRIV*。是特殊指针: 代表用户进程发信号 (kill 系统调用), 或者是内核内部发出的信号 (SI_KERNEL)。其它非特殊指针直接传入的真实 *siginfo_t*。
  - *info->si_code >=0* 故障类 si_code(BUS_ADRERR，$SEGV_MAPERR$ 等都是 ≥ 0)

  满足任一，则分配 sigqueue 不受 rlimit 限制。
- 1113-1114 实时信号严格遵守信号数量上限。
- 1116-1118 分配新 sigqueue 节点挂到 pending 链表尾部。pending 链表用来保存带详细 siginfo 的信号。
- 1120-1132 siginfo 是特殊的魔术值指针 (`((struct kernel_siginfo *) 0)` 的用户空间进程发送的普通信号。内核自动填充 `si_code=SI_USER`，并填入发送进程 pid、uid。
- 1133-1140 siginfo 是特殊的魔术值指针 (SEND_SIG_PRIV) 代表是内核发送的私有信号。填充 `si_code=SI_KERNEL`，pid/uid 清零。
- 1141-1144 将 `kernel_siginfo` 完整拷贝到 sigqueue 内部的`->info`, 其中包括 `si_addr` 故障地址。*SIGBUS* 等会走这里。
- 1145-1154 sigqueue 分配失败进入该分支。

  如果是实时信号 + 非特殊 info+ $si_code \neq SI_USER$，则 deliver 失败。
- 1155-1160 其余场景 (非实时信号，如 *SIGBUS*) 信号本身仍然 deliver(位图置位)，但是丢失 siginfo 详情 (si_addr 丢失)。
- 1164 有信号到达，通知唤醒阻塞在 signalfd 上的用户态进程。
- 1165 置位 pending 信号位图。

- 1168-1179 ， 仅对 *type* > *PIDTYPE_TGID*(跨进程组广播信号) 多进程延迟信号处理。本次调用 `type=PIDTYPE_PID` 会直接跳过。
- 1181 信号投递收尾函数。做如下处理:
  - 在目标线程组里寻找可以接收信号、可被唤醒的线程。
  - 唤醒睡眠的线程。
  - 设置 ***TIF_SIGPENDING*** 线程标志, 线程从内核态返回用户态时，内核检测这个标志，意味着有 pending 的 signal 待处理，会进入信号处理流程。

force_sig_fault_to_task() › *force_sig_info_to_task()* › *send_signal()* › *has_si_pid_and_uid()*

```
1187 static inline bool has_si_pid_and_uid(struct kernel_siginfo *info)
1188 {
1189     bool ret = false;
1190     switch (siginfo_layout(info->si_signo, info->si_code)) {
1191     case SIL_KILL:
1192     case SIL_CHLD:
1193     case SIL_RT:
1194         ret = true;
1195         break;
1196     case SIL_TIMER:
1197     case SIL_POLL:
1198     case SIL_FAULT:
1199     case SIL_FAULT_MCEERR:
1200     case SIL_FAULT_BNDERR:
1201     case SIL_FAULT_PKUERR:
1202     case SIL_SYS:
1203         ret = false;
1204         break;
1205     }
1206     return ret;
1207 }
```

- 1190 获取 siginfo 联合体的布局类型 (***SIL_xxx***), 进入 switch 分支。
- 1191-1195 下面这几个**信号来源**的 siginfo 的联合体中包含 *si_pid* 和 *si_uid* 成员。
  - *SIL_KILL*　kill 类信号布局 (*SI_USER*、*SI_KERNEL* 的结构体 siginfo 都属于这个内存布局)，联合体 *_kill* 分支。
  - *SIL_CHLD*　SIGCHLD 子进程退出信号，联合体 *_child* 分支。
  - *SIL_RT*　实时信号，联合体 *_rt* 分支。
- 1196-1204 下面几个**信号来源**的 signinfo 的内存布局不包含 *si_pid* 和 *si_uid* 成员。
  - *SIL_TIMER*　定时器信号。
  - *SIL_POLL*　poll/select IO 信号。
  - *SIL_FAULT*　硬件故障类信号 (*SIGBUS*、*SIGSEGV*)，联合体走 *_fault* 分支。
  - *SIL_FAULT_MCEERR*　MCE 硬件内存错误。
  - *SIL_FAULT_BNDERR*　MPX 边界异常。
  - *SIL_FAULT_PKUERR*　PKU 内存保护异常。

- *SIL_SYS* SIGSYS 信号。被禁止的系统调用被 seccomp 规则拦截、或者非法 syscall 号，内核才发送 SIGSYS。

  seccomp 规则指一段 BPF 字节码程序，进程把这段 BPF 被注入内核。此后该进程每一次陷入 syscall，内核都会先跑这个 BPF 程序，根据 BPF 返回的判决，决定要不要执行这个系统调用。

  当 BPF 返回 *SECCOMP_RET_TRAP*, 内核不执行该系统调用, 而是发送信号 ***SIGSYS***, 此时 siginfo 的 *si_code=SYS_SECCOMP*, 并填充被拦截的系统调用号、指令地址与架构信息。

siginfo_layout() 根据信号 sig + si_code，判断 *struct siginfo* 是哪一种内存布局 (*SIL_**)，告诉内核 siginfo 联合体里面哪些字段有效。

force_sig_fault_to_task() ⋯ *send_signal()* *has_si_pid_and_uid()* *siginfo_layout()*

```
3171 enum siginfo_layout siginfo_layout(unsigned sig, int si_code)
3172 {
3173     enum siginfo_layout layout = SIL_KILL;
3174     if ((si_code > SI_USER) && (si_code < SI_KERNEL)) {
3175         if ((sig < ARRAY_SIZE(sig_sicodes)) &&
3176             (si_code <= sig_sicodes[sig].limit)) {
3177             layout = sig_sicodes[sig].layout;
3178             /* Handle the exceptions */
3179             if ((sig == SIGBUS) &&
3180                 (si_code >= BUS_MCEERR_AR) && (si_code <= BUS_MCEERR_AO))
3181                 layout = SIL_FAULT_MCEERR;
3182             else if ((sig == SIGSEGV) && (si_code == SEGV_BNDERR))
3183                 layout = SIL_FAULT_BNDERR;
3184 #ifdef SEGV_PKUERR
3185             else if ((sig == SIGSEGV) && (si_code == SEGV_PKUERR))
3186                 layout = SIL_FAULT_PKUERR;
3187 #endif
3188         }
3189         else if (si_code <= NSIGPOLL)
3190             layout = SIL_POLL;
3191     } else {
3192         if (si_code == SI_TIMER)
3193             layout = SIL_TIMER;
3194         else if (si_code == SI_SIGIO)
3195             layout = SIL_POLL;
3196         else if (si_code < 0)
3197             layout = SIL_RT;
3198     }
3199     return layout;
3200 }
```

- 3173 如果所有分支都不命中，就默认当成普通 kill 类信号。*SIL_KILL* 只保证 *si_pid*、*si_uid* 有效。*si_addr* 无效。*SI_USER*、*SI_KERNEL* 就属于这个布局。
- 3174 当 $si_code \in (SI_USER, SI_KERNEL)$ 是进入此分支，也就是 $si_code \in (0, 0x80)$
- 3175-3176 如果信号值 sig 没有超出数组范围，并且当前 $si_code \leq$ 该信号允许的最大 *si_code* 进入次代码分支。
- 3177 获取目标信号预设的 siginfo 内存布局。
- 3179-3181 $SIGBUS + MCE$ 硬件内存错误, 则 siginfo 内存布局改成 *SIL_FAULT_MCEERR*, 除 *si_addr* 外，*si_bdaddr* 字段有效 (MCE 专用)。

- 3182-3183 $SIGSEGV + SEGV_BNDERR$(Intel MPX 边界检查异常), 则 siginfo 内存布局改成 SIL_FAULT_BNDERR, 会包含 *si_lower*, *si_upper* 边界信息。
- 3184-3187 没有开启 PKU 时这段代码被裁剪。

  PKU(Protection Keys，内存保护密钥) 权限规则是 x86 架构特有的，暂时忽略。
- 3189-3190 信号不在 *sig_sicodes* 表，或者 *si_code* 超出该目标信号 limit，则布局改为 SIL_POLL。
- 3191-3198 当 $si_code \notin (SI_USER, SI_KERNEL)$ 时，进入此分支。
  - 3192-3193 *si_code* 等于 SI_TIMER, 此时定时器相关字段有效。
  - 3194-3195 此时 SIL_POLL 相关字段有效。
  - 3196-3197 *si_code* 为负数是实时信号，siginfo 内存布局是 SIL_RT，*si_value* 有效。
- 3199 返回最终的 layout，系统靠这个知道 siginfo 联合体里哪些成员合法可读。

10.1.1.2 驱动的->migrate_to_ram() 回调

再回到异构内存管理的主题。

图 10–1 (*do_page_fault()* 的调用栈) 中的❷是 GPU 设备驱动实现的回调函数 (`struct dev_pagemap_ops->migrate_to_ram()`)，*VRAM->RAM* 的单页迁移会最终调用它，来进行设备私有内存到 cpu 主 RAM 的页迁移。

驱动实现的回调 migrate_to_ram() 中会执行如下动作:

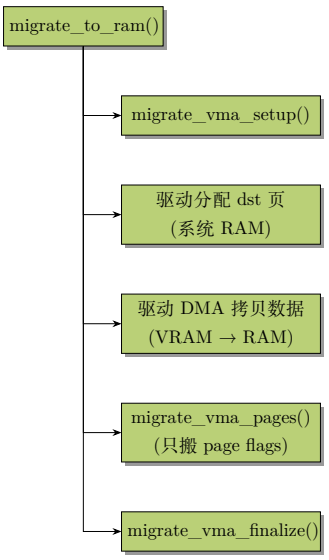


图 10–2: migrate_to_ram() 的部分执行逻辑

上图中可以看到 HMM 的骨架三段式接口 (***migrate_vma_*()*** 形式的三个函数), 不管方向是 RAM→VRAM 还是 VRAM→RAM 都会调用该三个函数，只是方向相反、源/目标角色互换。这几个函数的解析见下面**批量回迁**子章节。

10.1.2 批量回迁

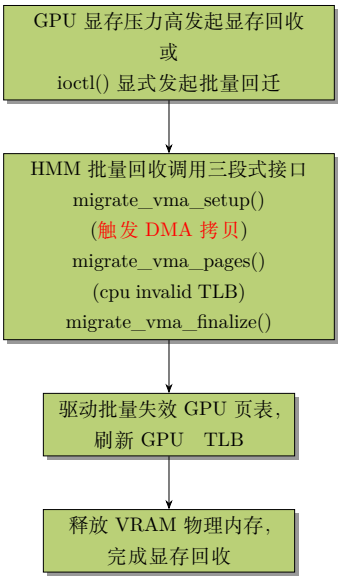
VRAM->RAM 的批量回迁通常发生在:

- GPU 侧显存回收。

  GPU 显存 (VRAM) 空间远小于系统内存，当 GPU 驱动发现显存紧张, 驱动不能直接扔掉页面 (VRAM) 内容，必须先回迁到系统 RAM 再释放 VRAM (把这块显存回收复用)。

- CPU 侧 (用户空间) 调用 *ioctl()* 触发批量回迁。

批量回迁 (VRAM->RAM) 的主要步骤见下图:



GPU 设备驱动主动调用 *migrate_vma_setup()* 启动批量 VMA 范围设备私有页回迁流程。在执行中，内核会调用 *try_to_unmap(TTU_MIGRATION)*，底层经由 *try_to_unmap_one()*，将 CPU PTE 页表 $present = 1$ 替换为 $migration\ entry + present = 0$，防止并发缺页产生竞态场景。

```
int migrate_vma_setup(struct migrate_vma *args)
```

GPU 设备驱动初始化了 *struct migrate_vma* 中的字段，并将该结构体指针传递给 *migrate_vma_setup()*。`args->flags` 字段是用来过滤哪些源页面应该被迁移。例如，设置 ***MIGRATE_VMA_SELECT_SYSTEM*** 是迁移系统内存; 设置 ***MIGRATE_VMA_SELECT_DEVICE_PRIVATE*** 是迁移驻留在设备私有内存中的页面。如果后者被设置，$args->pgmap_owner$ 字段被用来识别驱动所拥有的设备私有页。

所以 *migrate_vma_setup()* 本身不分方向, 内存数据迁移方向由调用者传进去的 flags 和 dst_pfns(迁移到哪里) 决定。

10.1.2.1 migrate_vma_setup()

迁移某段范围虚拟地址里的页数据时,*migrate_vma_setup()* 会收集该虚拟地址区间背后对应的所有物理页，保存到 src 源数组中。然后锁住这些页面，并解除 CPU 页表对这些页的映射。

页面锁定 + 解除映射完成后，检查每一页是否被 pin(固定)。未被 pin 住的页面，会在 src 数组对应项上被设置 *MIGRATE_PFN_MIGRATE* 标记。对于被 pin 住的页面，会恢复映射、解锁页面。

migrate_vma_setup()

```
2800 int migrate_vma_setup(struct migrate_vma *args)
2801 {
2802     long nr_pages = (args->end - args->start) >> PAGE_SHIFT;
2803
2804     args->start &= PAGE_MASK;
2805     args->end &= PAGE_MASK;
2806     if (!args->vma || is_vm_hugetlb_page(args->vma) ||
2807         (args->vma->vm_flags & VM_SPECIAL) || vma_is_dax(args->vma))
2808         return -EINVAL;
2809     if (nr_pages <= 0)
2810         return -EINVAL;
2811     if (args->start < args->vma->vm_start ||
2812         args->start >= args->vma->vm_end)
2813         return -EINVAL;
2814     if (args->end <= args->vma->vm_start || args->end > args->vma->vm_end)
2815         return -EINVAL;
2816     if (!args->src || !args->dst)
2817         return -EINVAL;
2818
2819     memset(args->src, 0, sizeof(*args->src) * nr_pages);
2820     args->cpages = 0;
2821     args->npages = 0;
2822
2823     migrate_vma_collect(args);
2824
2825     if (args->cpages)
2826         migrate_vma_prepare(args);
2827     if (args->cpages)
2828         migrate_vma_unmap(args);
2829
2830     /*
2831      * At this point pages are locked and unmapped, and thus they have
2832      * stable content and can safely be copied to destination memory that
2833      * is allocated by the drivers.
2834      */
2835     return 0;
2836
2837 }
2838 EXPORT_SYMBOL(migrate_vma_setup);
```

- 2802 计算本次迁移区间的总页数。虚拟地址差值右移 *PAGE_SHIFT*，得到页面数量。也就是:

  $$\text{页面数量} = \frac{\text{以字节数表示的范围大小}}{\text{一页的字节数}}$$

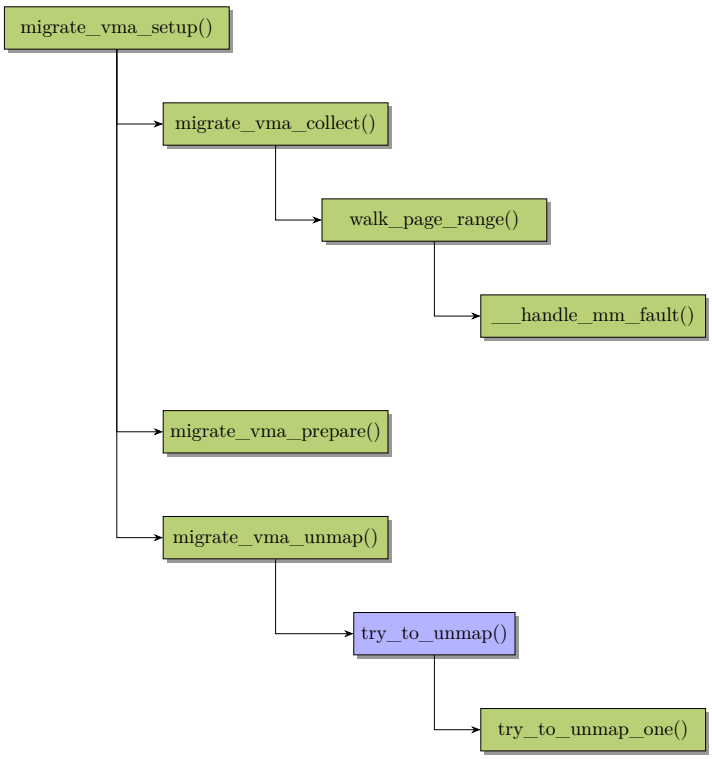
- 2804-2805 把传入的起止虚拟地址页对齐，掉保证操作以整页为单位。
- 2806-2808 合法性校验，不支持下面场景:
  - vma 为空。
  - 巨页 (hugetlb)。
  - VMA 标记为 *VM_SPECIAL* 的特殊区域。

    - 直接访问设备内存区域。也就是绕过操作系统页缓存，让用户态程序直接对持久内存 (PMEM，如 Intel Optane) 或者特定设备内存进行裸访问。

- 2809-2810 页数 ≤ 0, 返回错误。
- 2811-2815 迁移起止 VA 必须落在 VMA 的地址范围之内，超出 VMA 直接报错。防止越界操作其他 VMA。
- 2816-2817 src、dst 数组不能为空指针。
- 2819 清零 src 数组.
- 2820-2821 初始化两个计数器：
    - *cpages* 收集到的有效页面数量。
    - *npages* 本次迁移总候选页面数。
- 2823 遍历 $[start, end)$ 虚拟地址范围，逐个遍历页收集每一页对应的源 pfn，填充 *args->src* 数组, 尝试锁定页面，标记 *MIGRATE_PFN_** 系列标记，排除被 pin(固定) 住的页面。
- 2825-2826 如果收集到有效页面，那么准备反向映射、页引用计数检查，为 unmap 做准备。
- 2827-2828 解除 CPU 页表映射。调用 *try_to_unmap_one*(见下图)，清理 PTE，TLB 后续会被刷新。

    执行到这里页面已经被 lock，CPU 侧 PTE 已经 unmap，CPU 不能直接访问，可以安全 DMA 拷贝。但是目标函数自身不做数据拷贝和页元数据迁移，它只上锁、解除 CPU 页表映射。

如上所述 *try_to_unmap()*(代码解析见第 9章) 会在 *migrate_vma_setup()* 的执行中调用的，流程如下:



10.1.2.2 migrate_vma_collect()

函数 *migrate_vma_collect()* 会遍历 VMA 中对应的页表，筛选符合迁移条件的页面，并将符合条件页的 PTE 改为 migration entry(迁移项)、记录 PFN、增加 page 引用计数，标记待迁移，为 *migrate_vma_prepare()* 做准备。

migrate_vma_setup() → *migrate_vma_collect()*

```
2496 static void migrate_vma_collect(struct migrate_vma *migrate)
2497 {
2498     struct mmu_notifier_range range;
2499
2500     /*
2501      * Note that the pgmap_owner is passed to the mmu notifier callback so
2502      * that the registered device driver can skip invalidating device
2503      * private page mappings that won't be migrated.
2504      */
2505     mmu_notifier_range_init_migrate(&range, 0, migrate->vma,
2506         migrate->vma->vm_mm, migrate->start, migrate->end,
2507         migrate->pgmap_owner);
2508     mmu_notifier_invalidate_range_start(&range);
2509
2510     walk_page_range(migrate->vma->vm_mm, migrate->start, migrate->end,
2511             &migrate_vma_walk_ops, migrate);
2512
2513     mmu_notifier_invalidate_range_end(&range);
2514     migrate->end = migrate->start + (migrate->npages << PAGE_SHIFT);
2515 }
```

- 2505-2507 初始化 *mmu_notifier_range* 结构体变量 range，该函数专门用于迁移场景。

  该函数操作本质是: 把*这是一次 HMM 页面迁移，地址范围 A∼B，owner=XXX*等信息打包放到 mmu_notifier_range 结构体里。

- 2508 发送 MMU 通知: 开始对该虚拟区间做失效 (invalid) 准备。

  遍历所有注册在目标进程 mm 上的 mmu_notifier（如 AMDGPU SVM），调用它们的 *invalidate_range_start()* 回调来通知 GPU 驱动: 内核马上要修改这段 VMA 的 CPU 页表，准备迁移页面。

  HMM 迁移的场景: 同一段虚拟地址 VA，CPU 有一套页表，GPU 硬件也持有一套 SVM(shared VM) 页表 + GPU TLB。当内核要修改 CPU 侧 PTE(换成 migration entry) 时，要通知 GPU。

  驱动收到通知后，禁止新增映射、会对比 pgmap_owner，如果目标页:

  - 属于本次迁移的设备私有页 (即,`dev_pagemap->owner == range->pgmap_owner`)，此时跳过 invalid 操作 (迁移完成后再进行失效操作)。

    内核准备修改 CPU 页表，把 PTE 替换成 migration entry，物理页还没拷贝、物理地址还没变。

    对于 $GPU VRAM -> RAM$ 的迁移，页面此时还安静躺在 GPU VRAM 里面，物理地址不变。此刻不需要销毁 GPU PTE、不需要冲刷 GPU TLB。只有在迁移完成，数据已经拷贝到主机 RAM。GPU 侧 (VRAM)PTE 才直接 unmap，把这个 VA 在 GPU 页表里面删掉 + 无效化 GPU 映射 (invalid GPU TLB)。

    对于 $RAM -> GPU VRAM$ 的迁移，要等驱动执行拷贝 (通常 DMA 方式) 把时间从 RAM 拷贝到 VRAM 后，才会新建 GPU PTE+invalid TLB。

- 不属于本次 owner 的设备映射，主动失效 GPU 侧 TLB/PTE，防止设备继续访问即将被改动的页表。

  owner 不匹配场景: 一个进程的某个 VMA 虚拟地址区间里混合了属于其他设备/驱动/GPU 的 ZONE_DEVICE 页, 本次 HMM 迁移只迁移 page A，但是通知的 VA 范围把属于另外 GPU 的 page B 也包进去了。本次迁移流水线从头到尾不会处理 page B，没有后续 finalize 流程来更新或清理它的 GPU 映射。如果现在不把它的 GPU PTE/TLB 干掉，GPU 会一直拿着旧 TLB 条目，在内核修改 CPU 页表的临界窗口并发访问，引发竞态 bug。

  所以这类跨 owner 的 VRAM 页，本次迁移后没有任何代码去清理它的 GPU 映射, 必须提前失效。

  这一步还没有真正修改 CPU 页表，只是通知设备**准备失效**。

- 2510-2511 页表遍历, 并调用 *migrate_vma_walk_ops* 的回调来检查页面是否满足迁移条件、增加 page 引用计数 pin 住页面、替换 PTE 为 migration entry、填充 $migrate->src[]$ 数组打上 *MIGRATE_PFN_MIGRATE* 标记。

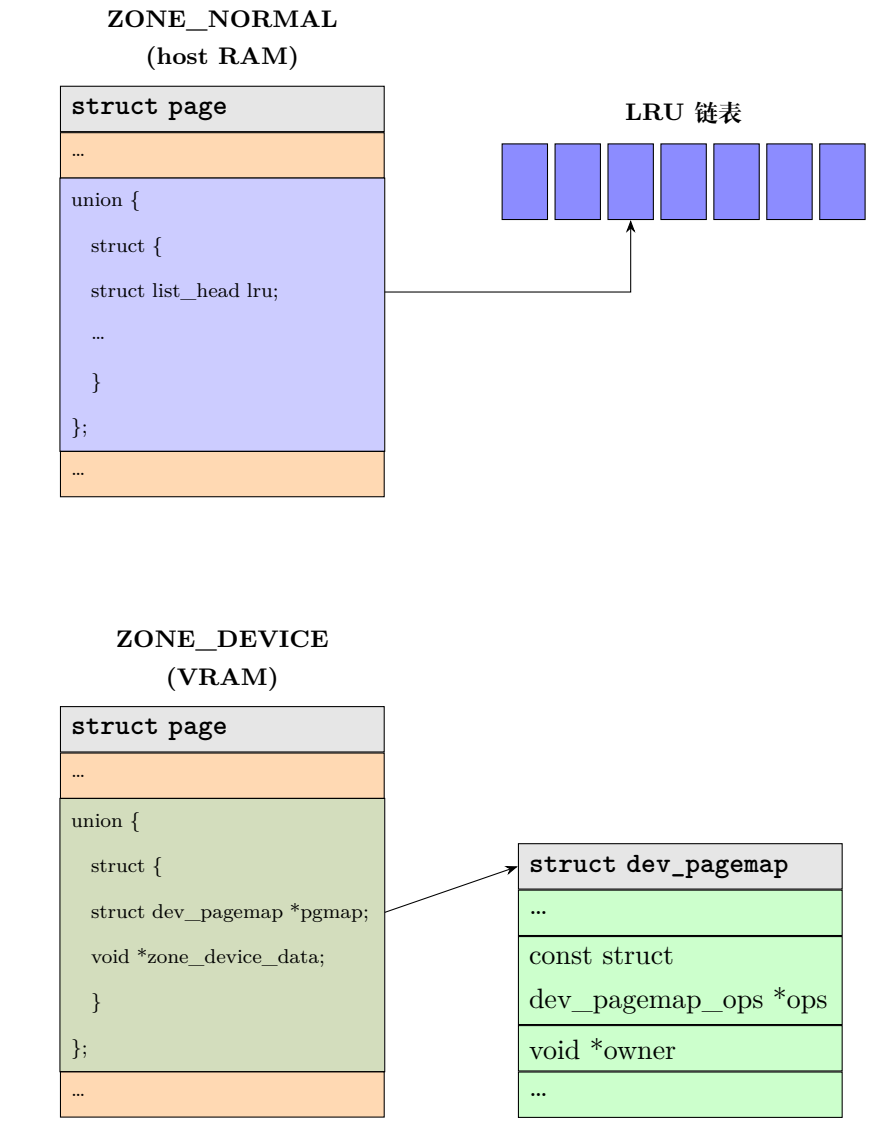
- 2513

10.1.2.3 migrate_vma_prepare()

migrate_vma_setup() 中会调用 *migrate_vma_prepare()* 来将页面从 LRU 链表隔离, 过滤掉不能迁移的页面。

但是普通内存页会从 LRU 隔离出来。而 GPU 的 VRAM 设备私有页 (*ZONE_DEVICE*) 不会进入 LRU，因此，*MEMORY_DEVICE_PRIVATE* 类型页会跳过 LRU 隔离的操作。普通内存页的标记 *PG_lru* 表示目标 page 当前挂在内核 LRU 链表上。

ZONE_DEVICE 的 struct page 只是描述 GPU 显存物理页的页描述符, 不是主机 RAM，永远不会进入系统空闲页池。*ZONE_DEVICE* 的 *page->lru*(其实是 union 结构体中的 *page->pgmap*) 被复用来指向这块内存的 ZONE_DEVICE 映射描述体，而不再用于做为加入 LRU 链表的节点。



说明:
两个 struct page 中的浅蓝色和暗绿色部分是同一个 union,
但是在普通 RAM 页和设备私有页时分别复用表示不同的成员元素。

图 10–3: struct page 的 union 中 lru 和 pgmap 的复用

结构体 *struct page* 中关于这部分的定义片段如下:

```
struct page {
                ... ...
    union {
      struct {    /* Page cache and anonymous pages */
           /**
            * @lru: Pageout list, eg. active_list protected by
            * pgdat->lru_lock.  Sometimes used as a generic list
            * by the page owner.
            */
           struct list_head lru;
                      ... ...
               };
      ... ...
      struct {    /* ZONE_DEVICE pages */
```

```
            /** @pgmap: Points to the hosting device page map. */
            struct dev_pagemap *pgmap;
            void *zone_device_data;
            /*
             * ZONE_DEVICE private pages are counted as being
             * mapped so the next 3 words hold the mapping, index,
             * and private fields from the source anonymous or
             * page cache page while the page is migrated to device
             * private memory.
             * ZONE_DEVICE MEMORY_DEVICE_FS_DAX pages also
             * use the mapping, index, and private fields when
             * pmem backed DAX files are mapped.
             */
        };

        /** @rcu_head: You can use this to free a page by RCU. */
        struct rcu_head rcu_head;
    };
    ... ...
} _struct_page_alignment;
```

其中，对于设备私有页，*page->pgmap* 指向 *struct dev_pagemap*，*page->zone_device_data* 是驱动私有数据指针。

migrate_vma_setup() → *migrate_vma_prepare()*

```
2579 static void migrate_vma_prepare(struct migrate_vma *migrate)
2580 {
2581     const unsigned long npages = migrate->npages;
2582     const unsigned long start = migrate->start;
2583     unsigned long addr, i, restore = 0;
2584     bool allow_drain = true;
2585
2586     lru_add_drain();
2587
2588     for (i = 0; (i < npages) && migrate->cpages; i++) {
2589         struct page *page = migrate_pfn_to_page(migrate->src[i]);
2590         bool remap = true;
2591
2592         if (!page)
2593             continue;
2594
2595         if (!(migrate->src[i] & MIGRATE_PFN_LOCKED)) {
2596             /*
2597              * Because we are migrating several pages there can be
2598              * a deadlock between 2 concurrent migration where each
2599              * are waiting on each other page lock.
2600              *
2601              * Make migrate_vma() a best effort thing and backoff
2602              * for any page we can not lock right away.
2603              */
2604             if (!trylock_page(page)) {
2605                 migrate->src[i] = 0;
2606                 migrate->cpages--;
2607                 put_page(page);
2608                 continue;
2609             }
2610             remap = false;
2611             migrate->src[i] |= MIGRATE_PFN_LOCKED;
2612         }
2613
2614         /* ZONE_DEVICE pages are not on LRU */
2615         if (!is_zone_device_page(page)) {
```

```
2616            if (!PageLRU(page) && allow_drain) {
2617                /* Drain CPU's pagevec */
2618                lru_add_drain_all();
2619                allow_drain = false;
2620            }
2621
2622            if (isolate_lru_page(page)) {
2623                if (remap) {
2624                    migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
2625                    migrate->cpages--;
2626                    restore++;
2627                } else {
2628                    migrate->src[i] = 0;
2629                    unlock_page(page);
2630                    migrate->cpages--;
2631                    put_page(page);
2632                }
2633                continue;
2634            }
2635
2636            /* Drop the reference we took in collect */
2637            put_page(page);
2638        }
2639
2640        if (!migrate_vma_check_page(page)) {
2641            if (remap) {
2642                migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
2643                migrate->cpages--;
2644                restore++;
2645
2646                if (!is_zone_device_page(page)) {
2647                    get_page(page);
2648                    putback_lru_page(page);
2649                }
2650            } else {
2651                migrate->src[i] = 0;
2652                unlock_page(page);
2653                migrate->cpages--;
2654
2655                if (!is_zone_device_page(page))
2656                    putback_lru_page(page);
2657                else
2658                    put_page(page);
2659            }
2660        }
2661    }
2662
2663    for (i = 0, addr = start; i < npages && restore; i++, addr += PAGE_SIZE) {
2664        struct page *page = migrate_pfn_to_page(migrate->src[i]);
2665
2666        if (!page || (migrate->src[i] & MIGRATE_PFN_MIGRATE))
2667            continue;
2668
2669        remove_migration_pte(page, migrate->vma, addr, page);
2670
2671        migrate->src[i] = 0;
2672        unlock_page(page);
2673        put_page(page);
2674        restore--;
2675    }
2676}
```

- 2581 本次迁移候选页面总数量。

- 2582 待迁移 VMA 的起始虚拟地址。
- 2586 把当前 CPU 的 pagevec 页缓存批量暂存数组里缓存的页面，刷入 LRU 链表。

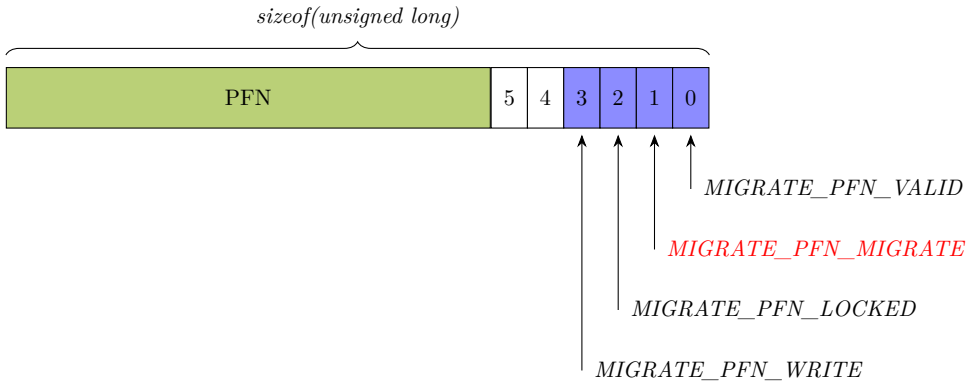
  内核不直接把回收的 page 挂 LRU，而是放到 pagevec 延迟加入。prepare 阶段要操作 LRU 时，会先把本 CPU 的 pagevec 刷入 LRU 链表。
- 2588 遍历所有候选页面。

  migrate->cpages 是当前有效待迁移页面计数。
- 2589 从待迁移的源页 src 数组项获取 PFN 来得到对应的 *struct page* 结构体指针。

  数组项的每一项 (`struct migrate_vma->src[i]`) 不是单纯 PFN 整数，是带标志位的复合值:

  $$PFN + 迁移控制位图 flag$$

  `struct migrate_vma->src[i]` 数组项是 unsigned long 型指针，指向 unsigned long 型的内存变量。数组项中的复合成员占位表示的意义如下:



  其中 bit[2] 置 1 (*MIGRATE_PFN_MIGRATE*) 时表示允许目标页执行本次迁移。

- 2592-2593 pfn 无效，无对应的 *struct page*，跳过。
- 2595-2612 判断目标页面是否已经获取了页面锁 page lock。若没有获取锁则进入下面代码段。
  - 2604-2609 尝试获取页面锁。失败则直接返回，不会阻塞等待。

    因为是一次性迁移多个页面，两个并发迁移任务之间可能互相等待对方页面锁，产生死锁。migrate_vma() 是尽力而为的机制，遇到无法立刻上锁的页面直接退让、放弃该页面。

    trylock 尝试获取锁失败，则如下处理：将目标候选页从待迁移数组清除、减少有效待迁移页的计数 (*cpages* − −)、目标页引用计数 −1、处理下一页。
  - 2610-2611 获取锁成功，目标页面打上 *MIGRATE_PFN_LOCKED* 标记。

- 2615-2638 **非设备私有页**才进入此代码段处理。该代码段主要是进行 LRU 链表上页的隔离。

  ZONE_DEVICE 页面 (VRAM 设备私有页) 不挂在 LRU 链表，没有 *PG_lru* 标记，直接跳过 LRU 隔离逻辑，所以此段主要是处理普通 RAM 页。

- 2616-2619 页面不在 LRU，并且还没执行过全局冲刷，则调用 *lru_add_drain_all()* 冲刷所有 CPU 的 pagevec，把挂在 pagevec 还没进 LRU 的页面刷入 LRU。

  全局冲刷开销很大，*allow_drain* 在循环 (2588 行的 for 循环) 外部默认初始值为 true，此处置 *false*，保证了在整个循环中最多只执行一次。

  页不在 LRU 有几类不同的场景:

  (1) *ZONE_DEVICE* 设备内存页 (前面的 if 判断已排除)。

  (2) *hugetlbfs* 巨页 完全独立内存池，不使用 LRU 链表。

  (3) *slab/slub* 内核缓存页 用于内核对象分配，不走 LRU 框架。

  (4) 正在执行内存迁移的页面 已经将页面移出 LRU 进入隔离链表。

  (5) 页面刚 fault 分配出来，挂在 per-cpu pagevec 临时缓存里，还没刷进全局 LRU 链表，*PG_lru* 标志没置位。内核为了减少全局 LRU 锁的争抢，每个 CPU 有 pagevec 缓存，缺页异常 (pagefault) 新分配的页，先放进当前 cpu 的 pagevec ，不直接链入全局 LRU，此时不设置 *PG_lru*。待 pagevec 满了、或者主动调用 *lru_add_drain()/lru_add_drain_all()*，才会批量把 pagevec 里的页面挪进 LRU 链表，同时置上 *PG_lru*。

  2618 行主要是针对场景 (5)，因为其它场景是永久不进入 LRU 的 (除了 (4) 可能会回滚进入)。

  NOTE

  free 的页面走的是 *per-cpu* 的 *pcp list.*
  而缺页新分配 *anon/pagecache* 页，是 *lru_add pagevec(LRU* 延迟加入缓存*)*。

  二者是两套完全独立的缓存，不要混在一起。

- 2622 把页面从 LRU 链表摘出来隔离。

- 2623-2626 如果 collect 阶段已经提前替换 PTE 为 migration entry，清除 *MIGRATE_PFN_MIGRATE* 标记 (不再迁移此页)，有效页面计数减一，标记这个页面后面需要回滚 PTE。

  collect 阶段已经把用户页表 PTE 替换成 migration entry(迁移项)(目的是阻止页表并发修改)。但到 prepare 阶段，发现这个页面隔离失败不满足迁移条件，也就是此处的场景。那么清除 *MIGRATE_PFN_MIGRATE* 标记 (从本次迁移任务的候选页数组里剔除该 page)。此时由于 migration entry 已经写进页表了，所以后续必须用 *remove_migration_pte()*(2669 行) 把 PTE 回滚恢复回去。

  remap=true 意味着 *migrate_vma_collect()* 的时候，就已经将 VMA 对应 PTE 替换成 migration entry。此后 CPU(进程) 访问这个虚拟地址就会触发缺页异常 (pagefault)，在异常处理路径中检查到 PTE 是 migration entry 时，内核就 block 住该进程等待迁移结束再重试 pagefault。

  和下面代码段对比可以发现，此处页面锁没有解锁，页面引用没有释放。因为后面循环结束后，内核会利用 restore 计数，把这个页的 migration entry 回滚回原始 PTE，页面仍然 lock，保证回滚页表期间安全。

- 2628-2631 PTE 没有被替换，不需要恢复页表。

因为隔离失败,src 数组里的目标页清除，不再参与迁移。待迁移页计数 −1,page 引用计数 −1。不需要恢复 PTE，所以直接释放 page 引用。

- 2637 LRU 隔离成功：释放 *migrate_vma_collect()* 拿到的页面引用。

- 2640 检查页面状态，判断页面是否允许迁移。例如是否页面被锁住、正在写回、是特殊不可迁移页面等。
- 2641-2649 校验失败，同样和前一个代码段一样要 remap 处理。remap: 也就是回滚 PTE。

  remap=true(collect 已经把 PTE 修改成 migration PTE), 则取消 MIGRATE 标记，$cpages--$，$restore++$(准备后面要回滚 PTE)。普通内存页:get_page + putback_lru_page，页面放回 LRU 链表。ZONE_DEVICE 设备私有页不走这里。
- 2651-2659 如果没有替换 PTE，普通页:*putback_lru_page()* 放回 LRU; 设备页：*put_page* 释放引用。

  在循环结束时，变量 restore 最终记录的数值是：预先将 PTE 设置成 migration PTE，但最终无法迁移，需要进行回滚 PTE 操作的页面数量。
- 2663-2675 该代码段专门进行回滚 PTE 的操作。所以只在 $restore > 0$ 时执行。遍历所有候选页面，对那些 PTE 已经填充成 migration pte，但无法迁移的页面进行 PTE 回滚。
  - 2666-2667 跳过无效 page 和仍然保留 MIGRATE 标记 (要正常迁移，不需要回滚) 的页面。
  - 2669 撤销 migration entry，把页表 PTE 恢复成原来的正常页表项。
  - 2671-2674 清理工作：将不需要迁移的页在迁移页源数组中对应位置 (src[i]) 清除，解锁页面，释放 page 引用, 待回滚页数量 (即计数 restore)−1。

10.1.2.4 migrate_vma_pages()

该函数并不真正执行 VMA 页面迁移，它只遍历预先收集好的 src/dst 页数组，提交元数据，不拷贝页数据。迁移分两类场景:

- 普通内存页的迁移 ($RAM->RAM$)。
- HMM 设备私有页的迁移。

  $RAM->VRAM$ 单页/批量迁移会调用 `migrate_vma_pages()->migrate_vma_insert_page()`。

  $VRAM->RAM$ 批量回迁会调用 `migrate_vma_pages()->migrate_page()`。

设备私有页的迁移除了上面列出的，还有其它: 譬如，$VRAM->RAM$ 单页迁移会从 cpu 侧的 pagefault 进入。RAM 和 VRAM 之间元数据迁移最终调用接口汇总如下表:

迁移方向	触发方式	规模	元数据迁移接口	说明
RAM->VRAM	驱动主动迁出	批量	*migrate_vma_insert_page()*	在 *migrate_vma_pages()* 循环内逐页调用元数据迁移接口。
RAM->VRAM	驱动主动迁出	单页	*migrate_vma_insert_page()*	同样走 *migrate_vma* 体系，最终调用本元数据迁移接口。
VRAM->RAM	驱动主动回迁	批量	*migrate_page()*	在 *migrate_vma_pages()* 循环内逐页调用元数据迁移接口。
VRAM->RAM	*CPU pagefault* 被动回迁	批量	*migrate_to_ram()*	单页 *fault* 路径，最终通过迁移接口进入 *migrate_vma* 框架。

migrate_vma_pages() 源码 (下面主要围绕设备私有页解析该函数) 如下:

migrate_vma_pages()

```
2981 void migrate_vma_pages(struct migrate_vma *migrate)
2982 {
2983     const unsigned long npages = migrate->npages;
2984     const unsigned long start = migrate->start;
2985     struct mmu_notifier_range range;
2986     unsigned long addr, i;
2987     bool notified = false;
2988
2989     for (i = 0, addr = start; i < npages; addr += PAGE_SIZE, i++) {
2990         struct page *newpage = migrate_pfn_to_page(migrate->dst[i]);
2991         struct page *page = migrate_pfn_to_page(migrate->src[i]);
2992         struct address_space *mapping;
2993         int r;
2994
2995         if (!newpage) {
2996             migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
2997             continue;
2998         }
2999
3000         if (!page) {
3001             if (!(migrate->src[i] & MIGRATE_PFN_MIGRATE))
3002                 continue;
3003             if (!notified) {
3004                 notified = true;
3005
3006                 mmu_notifier_range_init(&range,
3007                             MMU_NOTIFY_CLEAR, 0,
3008                             NULL,
3009                             migrate->vma->vm_mm,
3010                             addr, migrate->end);
3011                 mmu_notifier_invalidate_range_start(&range);
3012             }
3013             migrate_vma_insert_page(migrate, addr, newpage,
3014                         &migrate->src[i],
3015                         &migrate->dst[i]);
3016             continue;
3017         }
3018
3019         mapping = page_mapping(page);
3020
3021         if (is_zone_device_page(newpage)) {
3022             if (is_device_private_page(newpage)) {
3023                 /*
3024                  * For now only support private anonymous when
3025                  * migrating to un-addressable device memory.
3026                  */
3027                 if (mapping) {
3028                     migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
3029                     continue;
```

```
3030                }
3031            } else {
3032                /*
3033                 * Other types of ZONE_DEVICE page are not
3034                 * supported.
3035                 */
3036                migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
3037                continue;
3038            }
3039        }
3040
3041        r = migrate_page(mapping, newpage, page, MIGRATE_SYNC_NO_COPY);
3042        if (r != MIGRATEPAGE_SUCCESS)
3043            migrate->src[i] &= ~MIGRATE_PFN_MIGRATE;
3044    }
3045
3046    /*
3047     * No need to double call mmu_notifier->invalidate_range() callback as
3048     * the above ptep_clear_flush_notify() inside migrate_vma_insert_page()
3049     * did already call it.
3050     */
3051    if (notified)
3052        mmu_notifier_invalidate_range_only_end(&range);
3053}
3054EXPORT_SYMBOL(migrate_vma_pages);
```

- 2983-2984 获取本次迁移总页面数, 和待迁移虚拟地址区间起始 VA。
- 2989 循环逐页变量待迁移的页。
- 2990 从 dst 数组的 PFN 拿到该目标页对应的 *struct page* 结构体指针。
- 2991 从 src 数组 PFN 拿到该源页对应的 *struct page* 结构体指针。
- 2995-2997 目标页不存在 (譬如目标内存分配失败)，当前页面无法迁移。清除 src 数组对应项的 *MIGRATE_PFN_MIGRATE* 标志，标记该页放弃迁移。
- 3000-3002 源 page 为 NULL，RAM→VRAM 迁出分支。源主机 RAM 页已经被 unmap、refcount 释放。判断 src 项是否有 *MIGRATE_PFN_MIGRATE* 标记，若没有迁移标记，直接跳过。
- 3013-3015 把 CPU 页表 PTE 替换成 device private PTE。把虚拟地址对应的 PTE 修改指向设备私有页 (即，写入特殊 swap entry)，之后 CPU 访问这个地址就会触发 page fault 调用回迁的回调函数。
- 3021-3022 判断目的页是否是 *ZONE_DEVICE*(设备页) 页, 更进一步判断是否是 (设备) **私有页**。

  ZONE_DEVICE 是大类，里面包含多种类型:

  - ***MEMORY_DEVICE_PRIVATE*** 设备私有内存。CPU 不能直接映射访问，必须通过 *migrate_vma* 框架在主机 RAM 和设备内存之间迁移。
  - ***MEMORY_DEVICE_FS_DAX*** 文件系统 DAX(*DAX = Direct Access*) 设备内存 (持久内存，PMEM)。读写不走 pagecache。
  - ***MEMORY_DEVICE_GENERIC*** 通用设备内存。CPU 可以直接映射访问该设备内存，没有 *MEMORY_DEVICE_PRIVATE* 那种 CPU 不可访问的强限制。
  - ***MEMORY_DEVICE_PCI_P2PDMA*** 点对点 DMA 内存。PCIe 设备之间 DMA 用。

- 3027 *mapping* ≠ *NULL* 代表是有文件后备存储 (映射) 的页, 但是内核仅支持匿名页迁移到设备私有内存 (目的页)。
- 3028-3029 若源页是文件映射页，则清除 *MIGRATE_PFN_MIGRATE* 标记，放弃该源页的迁移。否则继续执行。
- 3031-3037 内核不支持源页是 *ZONE_DEVICE* 的其它页类型，若是此类场景也清除 *MIGRATE_PFN_MIGRATE* 标记，放弃该源页的迁移。
- 3041 进行页元数据迁移。

  传入参数 *MIGRATE_SYNC_NO_COPY* 意味着: 内核只处理锁、页表、页描述符 (元数据) 等内核元数据。页面拷贝操作放在 `struct address_space_operationsmigratepage->migratepage()` 回调之外完成，一般由 DMA 引擎执行。
- 3042-3044 若页元数据迁移失败，则清除 *MIGRATE_PFN_MIGRATE* 标记，标记当前页面迁移失败。
- 3052 通知驱动: 整个 VMA 区间的 PTE 失效操作全部完成。

========= 待插入代码解析，以及 migrate_vma_pages->migrate_vma_insert_page->ptep_clear_flush_notify 来 invalid CPU TLB

10.1.2.5 (AMD GPU) 用户空间通过 ioctl() 进行批量迁移的流程

migrate_vma_pages() 中只进行元数据的迁移，并不真正的拷贝数据，真正的数据拷贝在调用此函数之前。

此笔记的源码基于 5.10，但是 5.10 中 AMD GPU 驱动还没合入完整 HMM SVM，所以此处以 6.10 源码中 AMD GPU 的驱动为例看下用户空间 ioctl() 触发批量迁移时，代码执行中 DMA 数据搬运的位置 (下图):

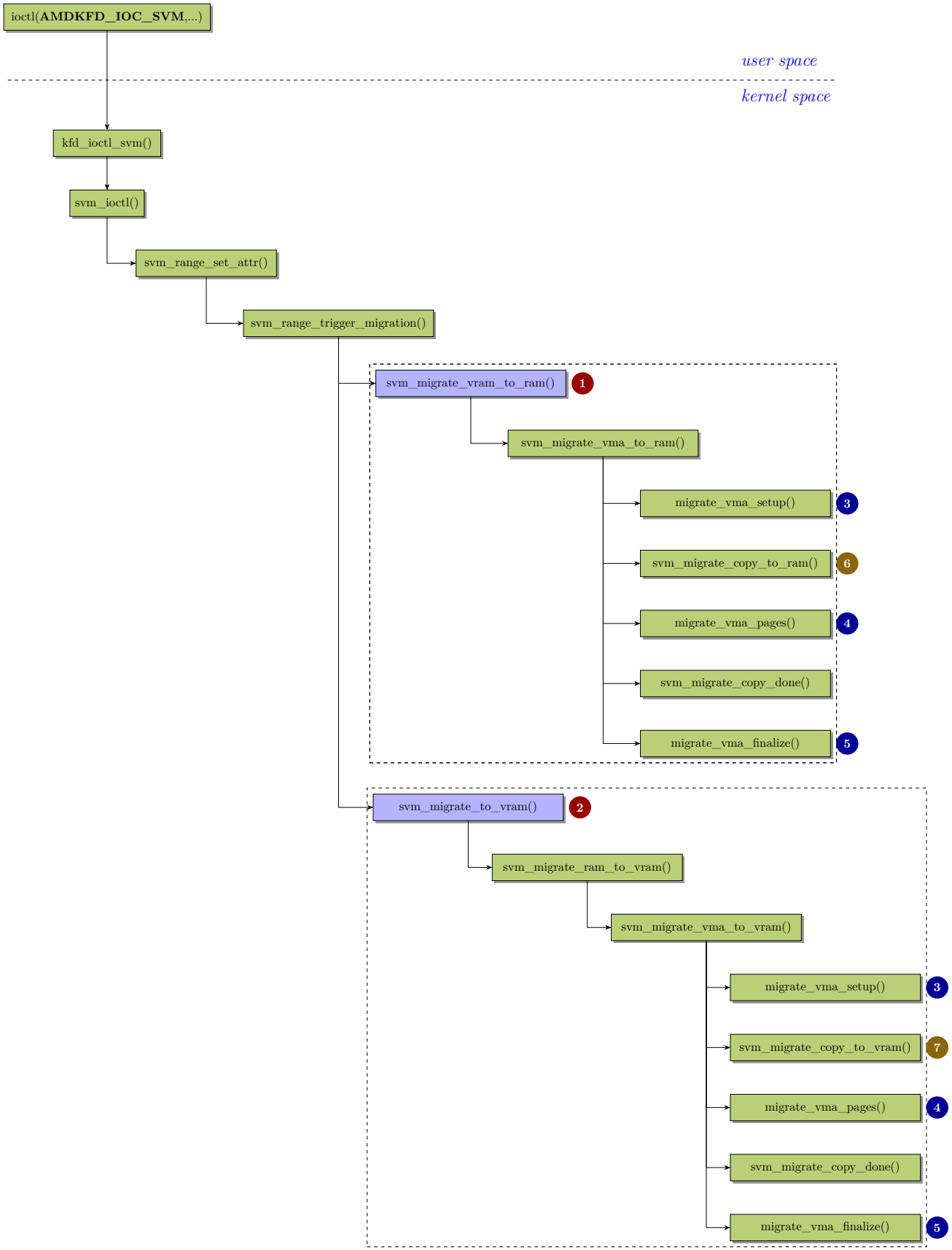


图 10–4: ioctl() 触发的批量迁移的执行流

说明:

- 1 VRAM → RAM 批量迁移的入口函数。
- 2 RAM → VRAM 批量迁移的入口函数。
- 3、4、5 HMM 迁移的三段式函数。
  - 3 遍历 VA 区间的页表、收集源页填入 src[]、在 PTE 中装入 migration entry，把这段地址”锁” 在迁移状态。
  - 4 把 struct page 的元数据 (flags、mapping、rmap 等) 从 src 页搬到 dst 页并提交，数据内容不在这里拷 (数据在6(VRAM → RAM) 或7(RAM → VRAM) 处由 DMA 搬运完)。
  - 5 把 migration entry 换成 dst 新页的 PTE，释放源页迁移结束；若迁移失败则清理回滚原 PTE。
- 6 分配 RAM 目标页，DMA 搬运 VRAM 数据到 RAM。
- 7 DMA 从 RAM 搬运数据到 VRAM。

10.1.2.6 (AMD GPU) 因显存压力内核空间触发批量迁移的流程

显存压力大时 TTM(通用 GPU 内存管理器) 会触发 eviction workqueue 来进行 VRAM → RAM 批量回收。迁移的最终入口和 *ioctl()* 相同是:*svm_migrate_vram_to_ram()*。

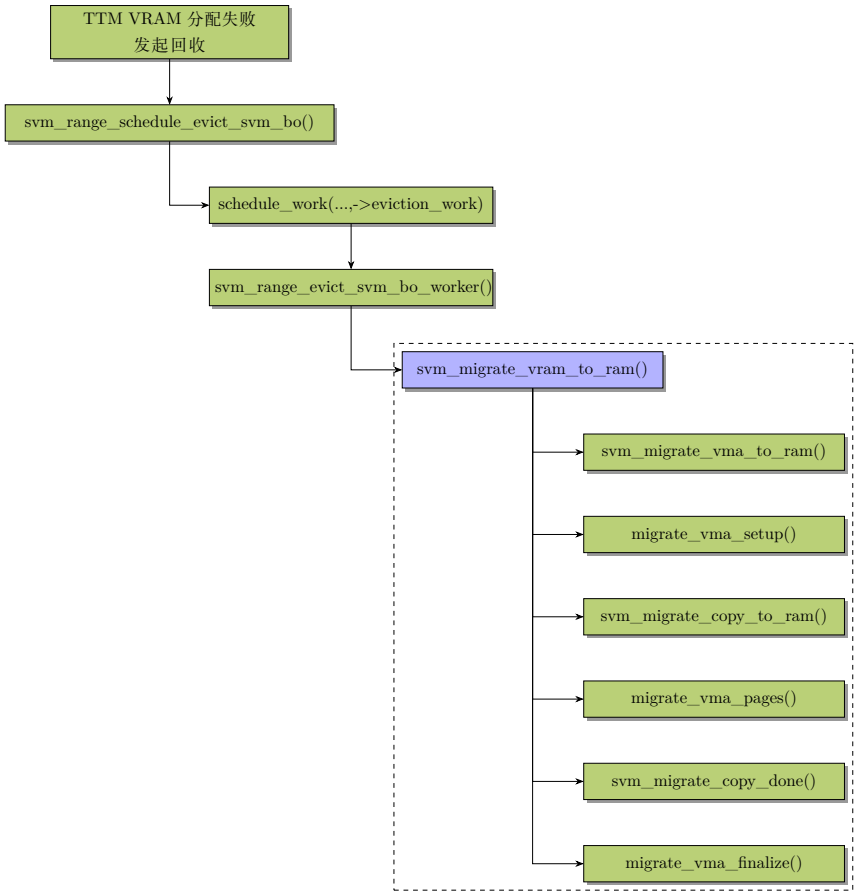


图 10–5: 内存压力触发的 VRAM → RAM 批量迁移的执行流

10.2 RAM 数据迁移到 GPU 私有内存

RAM−>VRAM 页迁移由 GPU 设备驱动主动发起，业务场景多为批量迁移整片 VMA，极少使用单页迁移。

GPU 内存迁移开销大头不在拷贝本身，而是在准备工作:*mmu_notifier*、页表锁、GPU 命令提交等**固定开销**。这些准备/收尾开销都是固定成本，和页数关系不大。如果每次只迁移 1 页，每一页都重复全套锁 + 通知 + fence，开销会很大。所以内核设计优先批量迁移一整个 VMA 区间，一次调用摊销所有固定开销。所以 RAM->VRAM 的单页迁移属于 RAM->VRAM 批量迁移的特例，它们调用同一个接口。

10.2.1 批量 (单页) 迁移

RAM 页往 GPU 私有内存的批量迁移发起方是 GPU 驱动。GPU 驱动主动调用 *migrate_vma_setup()*，属于驱动主动预迁移。批量迁移属于软件判断。

往设备私有内存批量迁移 (RAM->VRAM) 时，*try_to_unmap_one()* 处理原驻留内存 (present) 的 RAM 页面; 往 RAM 批量回迁 (VRAM->RAM) 时,*try_to_unmap_one()* 处理已经是 device private entry 的非驻留 entry(项)，也就此时是 *SWP_DEVICE_READ/SWP_DEVICE_WRITE* 是两个独立的 swp_type 之一。

migrate_vma_setup() 执行会调用 *try_to_unmap(**TTU_MIGRATION**)*，两种场景都会进入 *try_to_unmap_one()*，但依据源 (src) 页类型会走不同子分支:

- `RAM->VRAM`(批量迁出): 源页为系统匿名页, *is_zone_device_page()* 返回 false, 走普通页面迁移分支。旧 PTE 为 present 有效项, 调用 *ptep_clear_flush()* 清除并刷新 TLB。*make_migration_entry(page, pte_write(pteval))*，迁移条目的写位继承自原始 PTE。
- `VRAM->RAM`(批量回迁): 源页为设备私有页，*is_zone_device_page()* 与 *is_device_private_page()* 均为 true，进入设备页专属迁移分支。旧 PTE 是 *SWP_DEVICE_READ/WRITE* 非 present(驻留) 项，使用 *ptep_get_and_clear()* 清除 PTE 并触发 TLB 失效函数的调用。由于 PTE 原本非 present，CPU TLB 中不存在该虚拟地址的有效映射，TLB 刷新操作实际为空操作。*make_migration_entry(page,0)* 将迁移项写位硬编码为 0。迁移完成移除迁移项时，内核重建最终 PTE 的访问权限。

从用户空间 *ioctl()* 触发 `RAM->VRAM` 批量迁移的调用轨迹见图 10–4。整个流程和 `VRAM->RAM` 批量回迁很相似，此处略。

10.3 PTE 状态转换图

RAM、VRAM 之间设备私有页迁移涉及的 *PTE* 状态转换图如下:

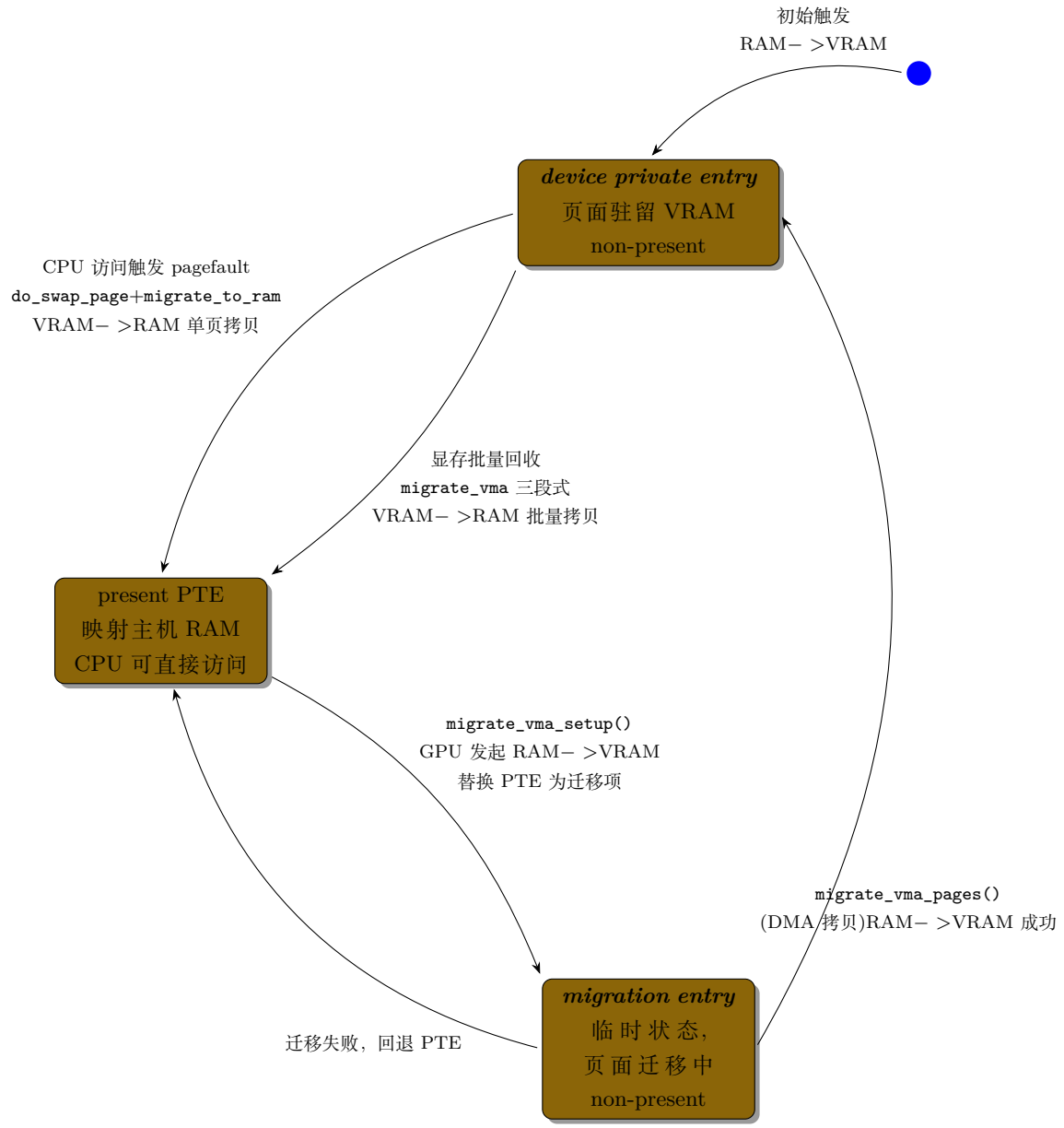


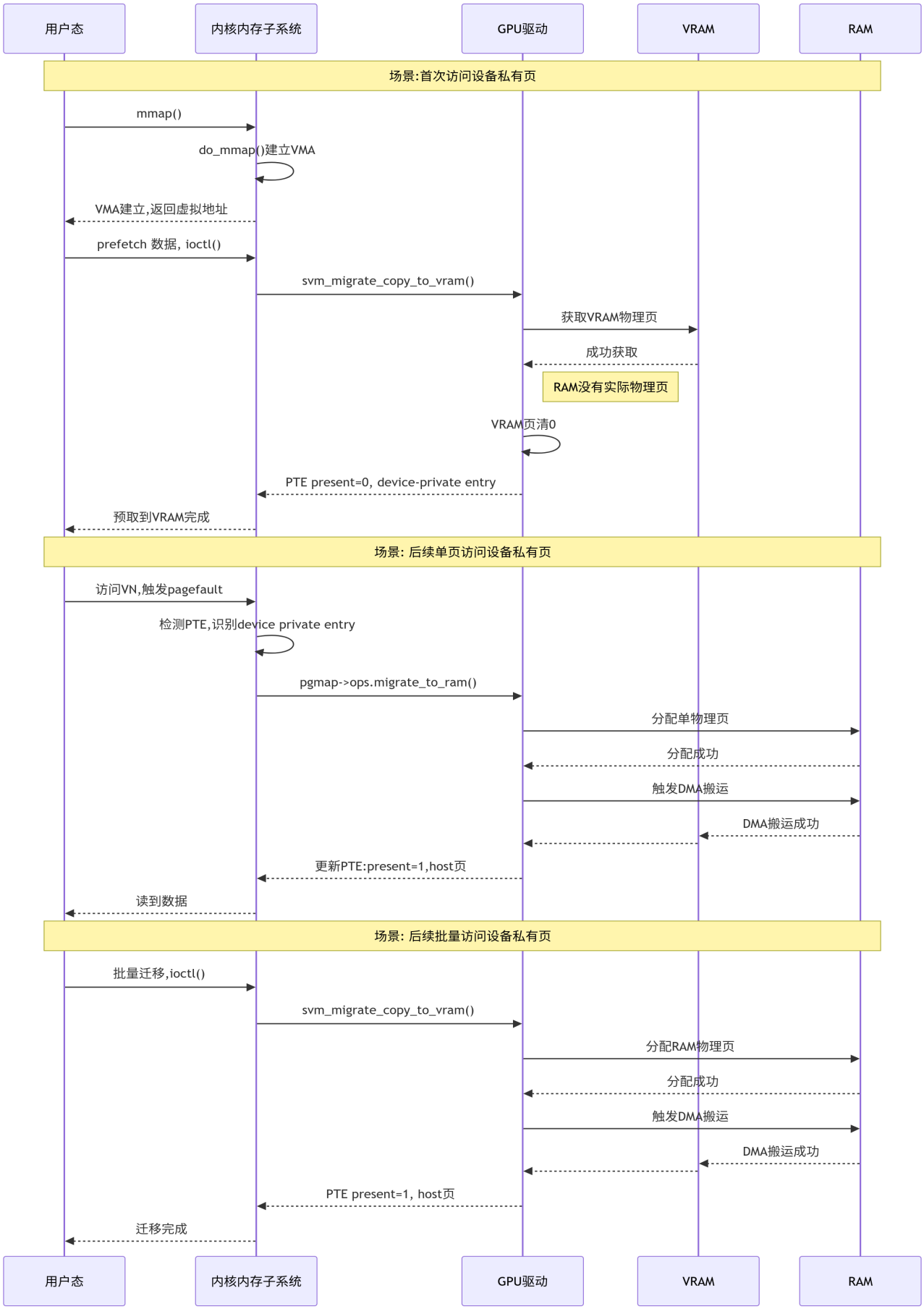
图 10–6: HMM 设备私有页 PTE 状态转换图

需要注意的是: 如果 CPU 从未访问过 *ZONE_DEVICE* 的目标页，那么 CPU" 第一次访问" 只有撞上了一个已经存在的 *device private entry*，然后 pagefault 处理、迁回 RAM 才能读到数据。

因为只有 GPU 主动迁移或 GPU page fault 或 ioctl prefetch 触发初始的一次"RAM → VRAM 迁移"，才会在 *migrate_vma_insert_page()* 里把 *device private entry* 写进 PTE。

10.4 用户空间访问设备私有内存的流程

用户空间访问设备私有内存的流程如下:


用户态
内核内存子系统
GPU驱动
VRAM
RAM
场景:首次访问设备私有页
mmap()
do_mmap()建立VMA
VMA建立,返回虚拟地址
prefetch 数据, ioctl()
svm_migrate_copy_to_vram()
获取VRAM物理页
成功获取
RAM没有实际物理页
VRAM页清0
PTE present=0, device-private entry
预取到VRAM完成
场景: 后续单页访问设备私有页
访问VN,触发pagefault
检测PTE,识别device private entry
pgmap->ops.migrate_to_ram()
分配单物理页
分配成功
触发DMA搬运
DMA搬运成功
更新PTE:present=1,host页
读到数据
场景: 后续批量访问设备私有页
批量迁移,ioctl()
svm_migrate_copy_to_vram()
分配RAM物理页
分配成功
触发DMA搬运
DMA搬运成功
PTE present=1, host页
迁移完成
用户态
内核内存子系统
GPU驱动
VRAM
RAM

10.5 设备私有内存映射

devm_memremap_pages() 是 HMM 异构内存管理的 API，用于为设备物理内存（如 GPU VRAM）创建 *ZONE_DEVICE* 类型的 *struct page*，将设备内存注册到内核内存管理子系统，进行资源托管，设备卸载自动回收。对于 *MEMORY_DEVICE_PRIVATE* 类型设备内存，CPU 无法直接访问，用户态访问时触发缺页异常，内核通过 migration entry 完成页面迁移。

该函数与 *ioremap()* 用途不同：*ioremap()* 用于 MMIO 寄存器的内核虚拟地址映射，不创建 *struct page*。

devm_memremap_pages() 和 *ioremap()* 关键对比如下:

函数	用途	创建对象	内存类型	能否直接访问
ioremap()	*MMIO* 寄存器映射	内核页表，生成 *__iomem* 虚拟地址	*MMIO* 寄存器	*CPU* 通过 *readl/writel* 访问
devm_memremap_pages()	*HMM* 设备内存	创建 *ZONE_DEVICE* 的 *struct page*，注册到 *MM* 子系统	设备私有内存 *ZONE_DEVICE*	*MEMORY_DEVICE_PRIVATE* 时 *CPU* 不可直接访问，访问触发缺页迁移。

10.5.1 devm_memremap_pages()

devm_memremap_pages()(5.10.0) 源码如下:

devm_memremap_pages()

```
410 void *devm_memremap_pages(struct device *dev, struct dev_pagemap *pgmap)
411 {
412     int error;
413     void *ret;
414
415     ret = memremap_pages(pgmap, dev_to_node(dev));
416     if (IS_ERR(ret))
417         return ret;
418
419     error = devm_add_action_or_reset(dev, devm_memremap_pages_release,
420             pgmap);
421     if (error)
422         return ERR_PTR(error);
423     return ret;
424 }
425 EXPORT_SYMBOL_GPL(devm_memremap_pages);
```

- 415-417 根据 pgmap 描述的物理资源，创建对应的 memmap(struct page 实例数组)，把设备内存(VRAM) 接入内核页管理体系，归属 *ZONE_DEVICE*。返回值为内核错误指针则直接向上返回错误。
- 419 向设备的 devm 资源链表增加一个动作。

  如果注册 action/reset 失败. 内核会立刻执行这个 release 回调，自动清理已经由 *memremap_pages()* 创建好的资源。

  如果注册成功，之后设备被卸载时，内核会自动销毁 memmap、释放资源、清理 *pgmap->ref* 引用计数。所有这些清理的动作都会**自动**进行，避免了用户没有显式清理造成资源泄漏。

10.5.2 memremap_pages()

memremap_pages() 将设备物理内存范围注册进内核内存管理，生成对应的 struct page 数组(memmap)。

devm_memremap_pages() → *memremap_pages()*

```
299 void *memremap_pages(struct dev_pagemap *pgmap, int nid)
300 {
301     struct mhp_params params = {
302         .altmap = pgmap_altmap(pgmap),
303         .pgprot = PAGE_KERNEL,
304     };
305     const int nr_range = pgmap->nr_range;
306     int error, i;
307
308     if (WARN_ONCE(!nr_range, "nr_range must be specified\n"))
309         return ERR_PTR(-EINVAL);
310
311     switch (pgmap->type) {
312     case MEMORY_DEVICE_PRIVATE:
313         if (!IS_ENABLED(CONFIG_DEVICE_PRIVATE)) {
314             WARN(1, "Device private memory not supported\n");
315             return ERR_PTR(-EINVAL);
316         }
317         if (!pgmap->ops || !pgmap->ops->migrate_to_ram) {
318             WARN(1, "Missing migrate_to_ram method\n");
319             return ERR_PTR(-EINVAL);
320         }
321         if (!pgmap->ops->page_free) {
322             WARN(1, "Missing page_free method\n");
323             return ERR_PTR(-EINVAL);
324         }
325         if (!pgmap->owner) {
326             WARN(1, "Missing owner\n");
327             return ERR_PTR(-EINVAL);
328         }
329         break;
330     case MEMORY_DEVICE_FS_DAX:
331         if (!IS_ENABLED(CONFIG_ZONE_DEVICE) ||
332             IS_ENABLED(CONFIG_FS_DAX_LIMITED)) {
333             WARN(1, "File system DAX not supported\n");
334             return ERR_PTR(-EINVAL);
335         }
336         break;
337     case MEMORY_DEVICE_GENERIC:
338         break;
339     case MEMORY_DEVICE_PCI_P2PDMA:
340         params.pgprot = pgprot_noncached(params.pgprot);
341         break;
342     default:
343         WARN(1, "Invalid pgmap type %d\n", pgmap->type);
344         break;
345     }
346
347     if (!pgmap->ref) {
348         if (pgmap->ops && (pgmap->ops->kill || pgmap->ops->cleanup))
349             return ERR_PTR(-EINVAL);
350
351         init_completion(&pgmap->done);
352         error = percpu_ref_init(&pgmap->internal_ref,
353                 dev_pagemap_percpu_release, 0, GFP_KERNEL);
354         if (error)
355             return ERR_PTR(error);
356         pgmap->ref = &pgmap->internal_ref;
```

```
357     } else {
358         if (!pgmap->ops || !pgmap->ops->kill || !pgmap->ops->cleanup) {
359             WARN(1, "Missing reference count teardown definition\n");
360             return ERR_PTR(-EINVAL);
361         }
362     }
363 
364     devmap_managed_enable_get(pgmap);
365 
366     /*
367      * Clear the pgmap nr_range as it will be incremented for each
368      * successfully processed range. This communicates how many
369      * regions to unwind in the abort case.
370      */
371     pgmap->nr_range = 0;
372     error = 0;
373     for (i = 0; i < nr_range; i++) {
374         error = pagemap_range(pgmap, &params, i, nid);
375         if (error)
376             break;
377         pgmap->nr_range++;
378     }
379 
380     if (i < nr_range) {
381         memunmap_pages(pgmap);
382         pgmap->nr_range = nr_range;
383         return ERR_PTR(error);
384     }
385 
386     return __va(pgmap->ranges[0].start);
387 }
388 EXPORT_SYMBOL_GPL(memremap_pages);
```

- 301 内存热插拔参数结构体，用于配置热插拔参数。
- 302 取出 pgmap 的备用 page 数组 altmap,altmap 允许把 *struct page* 数组放在设备内存而不是系统 RAM，在大显存场景节省了主机内存。
- 303 默认页保护属性，内核可读写。
- 305 获取目标物理地址区间数量，一个 pgmap 可以挂载多段不连续设备内存。
- 308-309 驱动必须填充 *pgmap->nr_range*。如果区间数量为 0，打印一次性警告，返回非法参数。
- 312-329 如果设备内存类型是 *MEMORY_DEVICE_PRIVATE*, 表示这是设备私有内存，因此进入该代码段。
  - 313-316 如果 *CONFIG_DEVICE_PRIVATE* 没有打开, 则不支持设备私有页，直接报错。
  - 317-320 必须提供 *->migrate_to_ram()* 回调, 没有这个回调，HMM 缺页回迁无法工作。
  - 321-324 必须提供 *->page_free()* 回调, 当设备私有页引用计数归零，内核调用该回调释放对应的显存资源。
  - 325-328 必须提供 *->owner*, 这是设备归属指针 (GPU 设备对象)，用于内核追踪这个设备内存属于哪个驱动实例。
- 330-336 持久内存 (PMEM) 直访文件系统内存，不属于 GPU VRAM 场景。依赖配置 *CONFIG_ZONE_DEVICE*。*CONFIG_FS_DAX_LIMITED* 是精简版 DAX，不支持。

- 337-338 通用设备内存，无回调要求。
- 339-341 PCI P2P 点对点设备内存，修改页属性为**非缓存** (noncached)，CPU 访问不经过 cache，防止跨设备 cache 一致性问题。
- 347-356 驱动没有传入外部引用计数，内核自动创建内部 percpu_ref。
  - 348-349 驱动不提供外部 ref 时，不能设置 kill/cleanup ops。这两个回调是给外部 ref 生命周期管理使用。
  - 351 初始化完成通知链, 用于等待所有页引用释放完毕，销毁 pgmap。
  - 352-355 初始化 percpu 引用计数 (*->internal_ref*)，当 ref 降到 0 时触发 *dev_pagemap_percpu_release()* 来清理 pgmap。
  - 356 把自动创建的内部 ref 挂载到 *pgmap->ref*。
- 357-362 驱动自带 ref，则必须提供 kill、cleanup 回调。
- 364 增加 pgmap 的托管引用计数，标记这个 dev_pagemap 处于启用状态，防止中途被释放。
- 371 清零 *pgmap->nr_range*。循环每成功注册一段内存，就 +1。一旦中途失败，*memunmap_pages* 读取当前 *nr_range*, 回滚成已经成功注册的区间。

  原始 *nr_range* 保存在局部变量 *nr_range*。
- 373-378 逐个处理每一段设备物理地址区间, 为迭代的目标段设备物理内存创建 memmap(*struct page* 数组) 加入 *ZONE_DEVICE*，设置 page 的 page->pgmap 指针。成功一段，*pgmap->nr_range* 自增。一旦一段失败，跳出循环。
- 380-384 循环中途出错, 销毁已经成功创建的 *struct page*，从 *ZONE_DEVICE* 移除，释放资源。恢复 *pgmap->nr_range* 为原始值。
- 386 全部区间注册成功, 则返回第一段设备物理地址对应的**内核**虚拟地址。

10.5.3 pagemap_range()

vm_memremap_pages() > *memremap_pages()* > *pagemap_range()*

```
185 static int pagemap_range(struct dev_pagemap *pgmap, struct mhp_params *params,
186         int range_id, int nid)
187 {
188     struct range *range = &pgmap->ranges[range_id];
189     struct dev_pagemap *conflict_pgmap;
190     int error, is_ram;
191
192     if (WARN_ONCE(pgmap_altmap(pgmap) && range_id > 0,
193                 "altmap not supported for multiple ranges\n"))
194         return -EINVAL;
195
196     conflict_pgmap = get_dev_pagemap(PHYS_PFN(range->start), NULL);
197     if (conflict_pgmap) {
198         WARN(1, "Conflicting mapping in same section\n");
199         put_dev_pagemap(conflict_pgmap);
200         return -ENOMEM;
201     }
202
```

```
    conflict_pgmap = get_dev_pagemap(PHYS_PFN(range->end), NULL);
    if (conflict_pgmap) {
        WARN(1, "Conflicting mapping in same section\n");
        put_dev_pagemap(conflict_pgmap);
        return -ENOMEM;
    }

    is_ram = region_intersects(range->start, range_len(range),
        IORESOURCE_SYSTEM_RAM, IORES_DESC_NONE);

    if (is_ram != REGION_DISJOINT) {
        WARN_ONCE(1, "attempted on %s region %#llx-%#llx\n",
                is_ram == REGION_MIXED ? "mixed" : "ram",
                range->start, range->end);
        return -ENXIO;
    }

    error = xa_err(xa_store_range(&pgmap_array, PHYS_PFN(range->start),
                PHYS_PFN(range->end), pgmap, GFP_KERNEL));
    if (error)
        return error;

    if (nid < 0)
        nid = numa_mem_id();

    error = track_pfn_remap(NULL, &params->pgprot, PHYS_PFN(range->start), 0,
            range_len(range));
    if (error)
        goto err_pfn_remap;

    mem_hotplug_begin();

    /*
     * For device private memory we call add_pages() as we only need to
     * allocate and initialize struct page for the device memory. More-
     * over the device memory is un-accessible thus we do not want to
     * create a linear mapping for the memory like arch_add_memory()
     * would do.
     *
     * For all other device memory types, which are accessible by
     * the CPU, we do want the linear mapping and thus use
     * arch_add_memory().
     */
    if (pgmap->type == MEMORY_DEVICE_PRIVATE) {
        error = add_pages(nid, PHYS_PFN(range->start),
                PHYS_PFN(range_len(range)), params);
    } else {
        error = kasan_add_zero_shadow(__va(range->start), range_len(range));
        if (error) {
            mem_hotplug_done();
            goto err_kasan;
        }

        error = arch_add_memory(nid, range->start, range_len(range),
                    params);
    }

    if (!error) {
        struct zone *zone;

        zone = &NODE_DATA(nid)->node_zones[ZONE_DEVICE];
        move_pfn_range_to_zone(zone, PHYS_PFN(range->start),
                PHYS_PFN(range_len(range)), params->altmap,
                MIGRATE_MOVABLE);
```

```
267     }
268
269     mem_hotplug_done();
270     if (error)
271         goto err_add_memory;
272
273     /*
274      * Initialization of the pages has been deferred until now in order
275      * to allow us to do the work while not holding the hotplug lock.
276      */
277     memmap_init_zone_device(&NODE_DATA(nid)->node_zones[ZONE_DEVICE],
278                 PHYS_PFN(range->start),
279                 PHYS_PFN(range_len(range)), pgmap);
280     percpu_ref_get_many(pgmap->ref, pfn_end(pgmap, range_id)
281             - pfn_first(pgmap, range_id));
282     return 0;
283
284 err_add_memory:
285     kasan_remove_zero_shadow(__va(range->start), range_len(range));
286 err_kasan:
287     untrack_pfn(NULL, PHYS_PFN(range->start), range_len(range));
288 err_pfn_remap:
289     pgmap_array_delete(range);
290     return error;
291 }
```

- 188 获取当前待注册的物理地址区间。

  range_id 表示当前正在处理的是 *pgmap->ranges[]* 里第几个区间。*pgmap* 是设备内存总描述符。

- 192-194 altmap 不支持多段 range。如果启用 altmap，同时 $range_id > 0$(说明当前不是第 0 段区间) 直接报错。原因是：altmap 的 vmemmap 布局是连续的，无法切分成多段不连续物理区间。对应上层调用者 *memremap_pages()* 的循环: 只有 $range_id = 0$ 才能使用 altmap。

- 196-201 防止同一段物理地址两次注册设备内存。

  取区间 (range) 起始物理页号 (PFN)，调用 *get_dev_pagemap()* 查询该 PFN 是否已经被别的 *dev_pagemap* 注册。

  函数 *get_dev_pagemap* 是内核全局 *pgmap_array*(XArray) 查询接口，根据 PFN 找对应的 *dev_pagemap*, 拿到 refcount 引用。

- 203-208 对区间末尾 PFN 再做一次检测，只要首尾任意一点已经注册设备内存，都判定冲突。

- 210-218 设备内存物理区间不能与系统 system RAM 重叠，换言之，设备内存不能占用主机系统 RAM 地址范围。

  region_intersects 查询内核资源树，判断目标物理范围和系统 RAM(*IORESOURCE_SYSTEM_RAM*) 是否存在重叠部分。

- 220-223 把当前区间 (range) 的 PFN 范围存入全局 XArray 数组 *pgmap_array*。

  pgmap_array 是内核全局索引。后续 *get_dev_pagemap(pfn)* 就是查这个 XArray。

  之后内核可以通过任意 PFN 反向找到所属 *dev_pagemap*，HMM 缺页、页迁移都会依赖这个查找。

- 225-226 如果上层传入 nid 无效，回退到默认 NUMA 节点。GPU 场景驱动一般会传入 GPU 归属 nid，不会走这个分支。
- 228-231 用于内核 PFN 跟踪，登记这一段物理页是重映射设备页，记录页保护属性。用于内核 MM、kasan、debug 工具识别设备内存 PFN。失败则跳转做清理。
- 233 获取内存热插拔锁，保护下面 *add_pages/arch_add_memory* 的内存上线流程，防止内存热插拔并发竞争。
- 246-248 对于设备私有内存(即，***MEMORY_DEVICE_PRIVATE***或 GPU VRAM) 只分配 + 初始化这批 PFN 对应的 *struct page*，不建立内核直接线性映射，CPU 不能直接读物理地址。
- 249-258 对其他设备内存 (FS DAX/GENERIC/P2PDMA) 的处理。
  - 250 给这段设备内存创建 **KASAN** 影子内存，用于内核内存越界检测。
  - 256 不同架构实现的相关内存热添加函数: 建立内核线性映射 (*___va()* 可以直接访问物理地址)。

宏	内存域	*CPU* 是否可直接访问	*pagemap_range* 执行分支
MEMORY_DEVICE_PRIVATE	*ZONE_DEVICE*	不可直接访问，访问触发缺页回迁	*add_pages()*，不创建内核线性映射
MEMORY_DEVICE_FS_DAX	*ZONE_DEVICE*	可以直接访问	*arch_add_memory()*，建立内核线性映射
MEMORY_DEVICE_GENERIC	*ZONE_DEVICE*	可以直接访问	*arch_add_memory()*，建立内核线性映射
MEMORY_DEVICE_PCI_P2PDMA	*ZONE_DEVICE*	可以直接访问 *(无缓存)*	*arch_add_memory()*，建立内核线性映射，*pgprot* 改为 *noncached*

如上表所示，这 4 种全部是 *ZONE_DEVICE* 内存。只是分为两大类:
CPU 不可直接访问: 仅 *MEMORY_DEVICE_PRIVATE*(设备私有内存)，走 *add_pages*。
CPU 可直接访问: *GENERIC/FS_DAX/P2PDMA*，统一走 *arch_add_memory()*

需要注意的是:*MEMORY_DEVICE_GENERIC*名字叫 GENERIC，但它不是普通系统 RAM. 而是驱动主动调用 *devm_memremap_pages()* 热插拔注册进 *ZONE_DEVICE* 的，不属于原生系统 RAM。CPU 直接访问物理地址即可，不会触发缺页调用驱动回调。

- 260-267 如果 *add_pages/arch_add_memory* 执行成功, 获取当前 NUMA 节点的 *ZONE_DEVICE* 区域描述符，把这一段 PFN 归属划入 *ZONE_DEVICE*，标记页面迁移类型为 *MIGRATE_MOVABLE*(可迁移页，HMM migrate_vma 框架迁移要求页是 movable 类型)。
- 269 释放内存热插拔锁，热插拔临界区结束。
- 277-279 初始化 *ZONE_DEVICE* 中对应的 *struct page* 元数据。遍历所有 PFN，给每个 *struct page* 设置: $page->pgmap = pgmap$, 绑定到当前 *dev_pagemap*。标记 page 类型为设备页，填充 page flags，设置 HMM 迁移回调关联。

  前面 *add_pages* 只是分配 *struct page* 内存, 这里才真正把 *page* 和 *dev_pagemap* 绑定。
- 280-281 批量增加 pgmap 的 percpu 引用计数。引用数量 = 当前 range 内总页数。每一页都持有 pgmap 的一个引用。

10.5.4 memmap_init_zone_device()

vm_memremap_pages() > memremap_pages() > pagemap_range() > memmap_init_zone_device()

```
6110 #ifdef CONFIG_ZONE_DEVICE
6111 void __ref memmap_init_zone_device(struct zone *zone,
6112                   unsigned long start_pfn,
6113                   unsigned long nr_pages,
6114                   struct dev_pagemap *pgmap)
6115 {
6116     unsigned long pfn, end_pfn = start_pfn + nr_pages;
6117     struct pglist_data *pgdat = zone->zone_pgdat;
6118     struct vmem_altmap *altmap = pgmap_altmap(pgmap);
6119     unsigned long zone_idx = zone_idx(zone);
6120     unsigned long start = jiffies;
6121     int nid = pgdat->node_id;
6122 
6123     if (WARN_ON_ONCE(!pgmap || zone_idx(zone) != ZONE_DEVICE))
6124         return;
6125 
6126     /*
6127      * The call to memmap_init_zone should have already taken care
6128      * of the pages reserved for the memmap, so we can just jump to
6129      * the end of that region and start processing the device pages.
6130      */
6131     if (altmap) {
6132         start_pfn = altmap->base_pfn + vmem_altmap_offset(altmap);
6133         nr_pages = end_pfn - start_pfn;
6134     }
6135 
6136     for (pfn = start_pfn; pfn < end_pfn; pfn++) {
6137         struct page *page = pfn_to_page(pfn);
6138 
6139         __init_single_page(page, pfn, zone_idx, nid);
6140 
6141         /*
6142          * Mark page reserved as it will need to wait for onlining
6143          * phase for it to be fully associated with a zone.
6144          *
6145          * We can use the non-atomic __set_bit operation for setting
6146          * the flag as we are still initializing the pages.
6147          */
6148         __SetPageReserved(page);
6149 
6150         /*
6151          * ZONE_DEVICE pages union ->lru with a ->pgmap back pointer
6152          * and zone_device_data.  It is a bug if a ZONE_DEVICE page is
6153          * ever freed or placed on a driver-private list.
6154          */
6155         page->pgmap = pgmap;
6156         page->zone_device_data = NULL;
6157 
6158         /*
6159          * Mark the block movable so that blocks are reserved for
6160          * movable at startup. This will force kernel allocations
6161          * to reserve their blocks rather than leaking throughout
6162          * the address space during boot when many long-lived
6163          * kernel allocations are made.
6164          *
6165          * Please note that MEMINIT_HOTPLUG path doesn't clear memmap
6166          * because this is done early in section_activate()
6167          */
6168         if (IS_ALIGNED(pfn, pageblock_nr_pages)) {
6169             set_pageblock_migratetype(page, MIGRATE_MOVABLE);
6170             cond_resched();
```

```
6171            }
6172        }
6173
6174        pr_info("%s initialised %lu pages in %ums\n", __func__,
6175            nr_pages, jiffies_to_msecs(jiffies - start));
6176    }
6177
6178    #endif
```

- 6111 ___*ref*(定义如下) 表示将目标函数放到 section(段):*.ref.text*，*.ref.text*属于常驻内存段，boot 完成之后不会被释放。

  ```
  #define __ref            __section(".ref.text") noinline
  ```

  如果该函数直接放普通*.text* 段 (section)，而不是放*.ref.text* 段，功能运行没问题。但是只要有 ___*init* 修饰的函数调用它，modpost 会在构建阶段强制输出"section mismatch" 警告。

  内核构建阶段 modpost 扫描目标文件的符号引用关系，如果一个**可被释放**段里的代码，引用了另一个常驻段的代码/数据 (或反过来)，就报"section mismatch"，用来捕获**代码在段释放之后，还被调用访问**的严重 bug。

  因此, 如果*.init.text* 里的代码如果调用了*.text* 段的函数，编译时就会触发"section mismatch" 告警; 若反过来调用，运行时则会触发 bug。

  ___*ref* 的首要作用对象是内核构建阶段的 **modpost** 工具。modpost 内置规则放行*.init.text* → *.ref.text* 引用, 其余所有 *.init* → 普通*.text* 的调用，继续保留告警检测。

  noinline 强制 GCC 不要 inline 该函数。因为一旦该函数被 inline，函数会被调用者就地展开，*.ref.text* 标记会丢失，sparse 会重新报 *section mismatch*。

- 6116 计算这段内存的终止 PFN(开区间，$pfn < end_pfn$)。
- 6117 获取当前 zone 所属的 NUMA 节点描述符。
- 6118 获取取 pgmap 里的 altmap 指针。

  ZONE_DEVICE 可以不在系统主 memmap 里分配 *struct page*，而是通过驱动预先分配一块独立内存 (不是设备内存或显存) 放 page 数组。

- 6119 获取 zone 类型的索引编号，这里一定是 *ZONE_DEVICE*。
- 6123-6124 *pgmap* 不能为空，zone 类型必须是 *ZONE_DEVICE*。不满足则不初始化，直接 return。
- 6131-6134 跳过部分保留区域，从设备页 *struct page* 对应的存放区起点开始初始化。

  altmap 这块 host 内存不是全部都给 *struct page* 用, 头部要留一段存 *altmap* 内部元数据，不能存放 *struct page* 要跳过。

  - 6133 重新计算 nr_pages。

    新 *start_pfn* 现在变成 *altmap* 里可用区的起始 PFN。重新计算从新起点到终点，一共还有多少 PFN 可以初始化 *struct page*。

    memmap_init_zone() 已经处理了 memmap 本身占用的页面，可以直接跳过这部分预留区域，从设备页对应的 *struct page* 存放区起点开始初始化。

- 6136-6172 遍历每一个 PFN，逐个初始化 *struct page*。
  - 6137 根据 PFN 拿到对应的 *struct page* 指针。
  - 6139 填充 page 基础字段。
  - 6148 将页面标记为 reserved。页面需要等待内存 onlining 阶段完成，才能完全和 zone 关联。
  - 6155-6156 *ZONE_DEVICE* 页面复用 page 里的 union：原来的 lru 链表成员，被重载为 *pgmap* 回指指针和 *zone_device_data*。

    struct page 里 *lru* 和 *pgmap* 的复用见图 10–3。

    page 反向指向 *dev_pagemap*。后续 HMM、migrate_vma 框架、get_dev_pagemap() 靠这个指针**识别这是设备页**。`page->zone_device_data` 驱动私有数据域，Nouveau/AMDGPU 可以在这里挂载私有描述符，初始置空。
  - 6168 当前 pfn 是这个 pageblock 的起始页号才进入这个代码段设置。

    只需要在每个 pageblock 的第一页 (pfn 对齐到 *pageblock_nr_pages*) 设置迁移类型。同 pageblock 内剩下所有页面复用这个 block 属性，不需要每页重复调用。
  - 6169 将 pageblock 迁移类型设置为 *MIGRATE_MOVABLE*。*ZONE_DEVICE* 页面 block 统一标记为可移动。

11

DMA-BUF

dma-buf(*struct dma_buf*) 是跨设备共享缓冲区框架，在多个硬件设备 (GPU、编码器、显示、NIC、AI 加速器) 之间共享同一块物理内存，避免多次拷贝。

硬件驱动 (譬如 camera 设备驱动) 将物理内存 export 成 dma-buf 对象，生成文件描述符 (fd), 用户空间程序通过这个文件描述符 (fd) 传递 buffer 给 GPU 驱动,GPU 驱动就可以找到这块物理内存。多个设备把同一块物理内存各自映射到自己的地址空间 (GPU VA、camera 等)，避免了 CPU 中间的介入拷贝。满足了不同设备之间共享 buffer **零拷贝**的需求。*dma-buf* 就是设备之间共享物理内存的缓冲区抽象，是面向设备端的 DMA。

dma-buf 本身不分 RAM 还是 VRAM，它只是一个**缓冲区句柄框架**。内存在哪，由导出这个 dma-buf 的驱动决定。如果是 VRAM 的 dma-buf(AMDGPU/NVIDIA 场景), 那么这块物理内存固定在 VRAM，**不会自动迁移**。这点和 HMM SVM 不一样。

11.1 dma-buf 在用户空间的共享

用户空间跨进程共享 dma-buf 一般通过 unix 域的本地套接字 (***AF_UNIX***) 来传递 dma 内存对象的 fd。这时候会利用 ***SCM_RIGHTS*** 控制消息, 并使用 ***sendmsg()*** 和 ***recvmsg()*** 接口来收发。

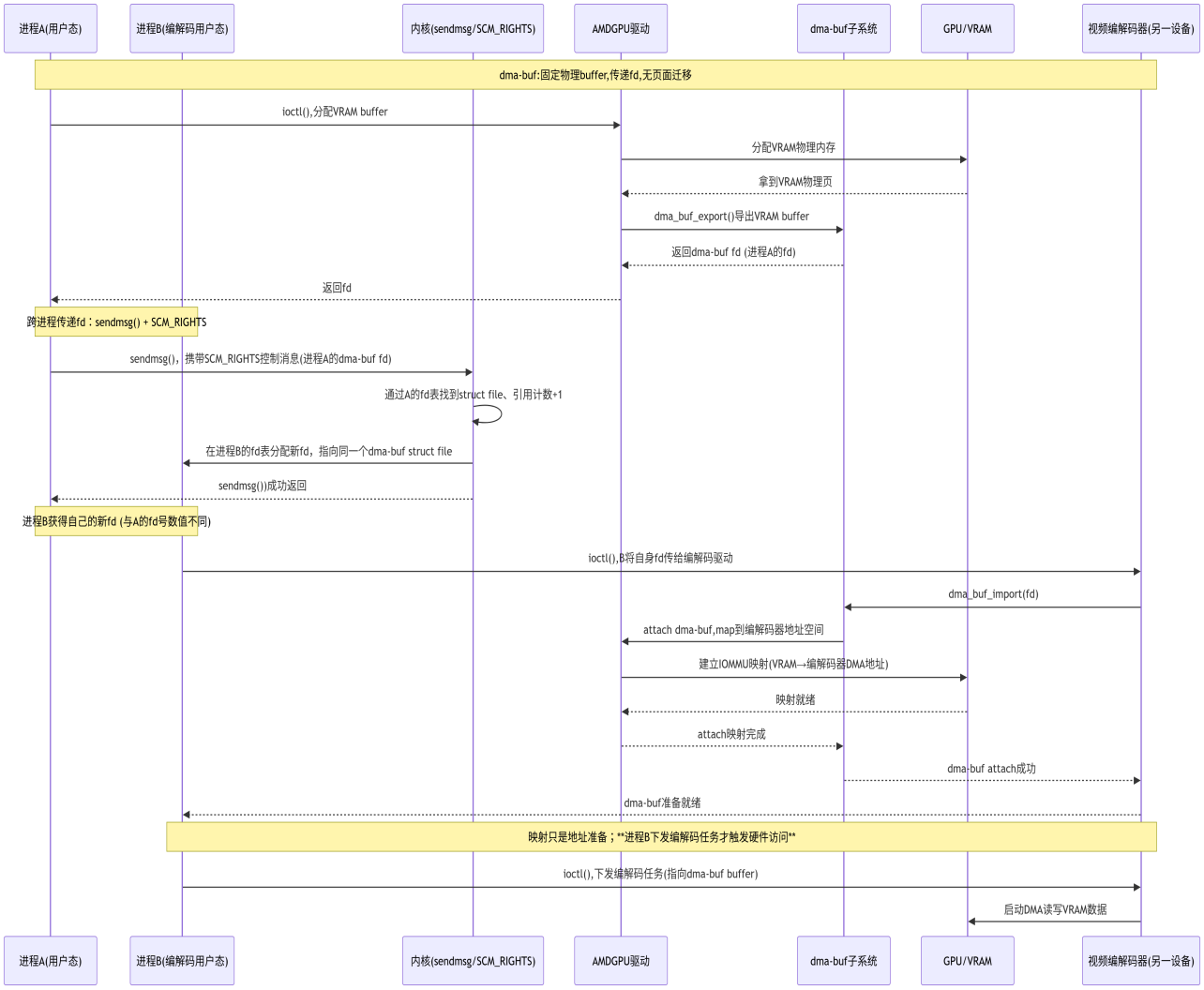
SCM_RIGHTS 是 unix 域 socket 控制消息的特殊方式: 让内核跨进程复制已打开文件的访问权限，不是单纯传 fd 号。dma-buf 跨进程共享就是靠这种机制，接收方拿到新 fd，但底层是同一个 dma-buf 内存对象。换言之, 传递的不是 fd 数字本身, 数字只是进程 (局部) 的文件索引，真正跨进程共享的是内核 *struct file*。

内核拿到进程 A 的 *struct file**[①]，把 file 引用计数 +1，在进程 B 的 fd 表里分配全新 fd 编号，指向内核中同一个 *struct file*，而这个 file 则指向同一个 *struct dma_buf*(内存) 对象。换言之，也就是内核在做跨进程的 *dup()/dup2()*[②]，而不是拷贝内存。

[①] unix/linux 中万物皆是文件。file 可以是内存、设备等对象。

[②] 系统调用 dup()/dup2() 是用户空间同一进程中复制 (副本)fd 的方式，执行完后得到一个新的 fd。这样同时新旧两个编号的 fd 都指向同一个 struct file。

下面是 AMD GPU 共享 dma buf 渲染输出帧，送给编码器的内存动作序列图的例子。



11.2 dma-buf 的映射接口

dma-buf 提供了两套可选映射接口，驱动可以选择实现:

- *dma_buf_vmap()* 在内核态临时把 dma 映射到内核虚拟地址;
- *dma_buf_ops -> mmap()* 对用户态 mmap 的支持。

11.2.1 dma_buf_ops->mmap()

`drivers/gpu/drm/amd/amdgpu/amdgpu_dma_buf.c` 文件中的 *struct dma_buf_ops amdgpu_dmabuf_ops* 是 **AMD GPU** 驱动回调操作集。其中*.mmap()* 是对用户空间 mmap 的支持，换言之，也就是用户态系统调用 mmap() 的内核处理入口。

11.2.1.1 drm_gem_dmabuf_mmap()

AMD GPU 驱动中 *struct dma_buf_ops amdgpu_dmabuf_ops->mmap()* 的实现是函数 drm_gem_dmabuf_mmap()(如下)。因为相比于纯内存管理子系统而言，代码变化较快。所以该章节中仍以 6.10 中的代码来解析。

drm_gem_dmabuf_mmap()

```
780 int drm_gem_dmabuf_mmap(struct dma_buf *dma_buf, struct vm_area_struct *vma)
781 {
782     struct drm_gem_object *obj = dma_buf->priv;
783 
784     return drm_gem_prime_mmap(obj, vma);
785 }
786 EXPORT_SYMBOL(drm_gem_dmabuf_mmap);
```

11.2.1.2 drm_gem_prime_mmap()

实际调用的 *drm_gem_prime_mmap()* 如下，该函数是 dma-buf 导出 GEM 对象的 mmap 回调函数，挂载在 dma-buf 的 file_operations，当用户调用 mmap() 时触发。

drm_gem_dmabuf_mmap() *drm_gem_prime_mmap()*

```
722 int drm_gem_prime_mmap(struct drm_gem_object *obj, struct vm_area_struct *vma)
723 {
724     struct drm_file *priv;
725     struct file *fil;
726     int ret;
727 
728     /* Add the fake offset */
729     vma->vm_pgoff += drm_vma_node_start(&obj->vma_node);
730 
731     if (obj->funcs && obj->funcs->mmap) {
732         vma->vm_ops = obj->funcs->vm_ops;
733 
734         drm_gem_object_get(obj);
735         ret = obj->funcs->mmap(obj, vma);
736         if (ret) {
737             drm_gem_object_put(obj);
738             return ret;
739         }
740         vma->vm_private_data = obj;
741         return 0;
742     }
743 
744     priv = kzalloc(sizeof(*priv), GFP_KERNEL);
745     fil = kzalloc(sizeof(*fil), GFP_KERNEL);
746     if (!priv || !fil) {
747         ret = -ENOMEM;
748         goto out;
749     }
750 
751     /* Used by drm_gem_mmap() to lookup the GEM object */
752     priv->minor = obj->dev->primary;
753     fil->private_data = priv;
754 
755     ret = drm_vma_node_allow(&obj->vma_node, priv);
756     if (ret)
757         goto out;
758 
759     ret = obj->dev->driver->fops->mmap(fil, vma);
760 
761     drm_vma_node_revoke(&obj->vma_node, priv);
762 out:
763     kfree(priv);
```

```
764     kfree(fil);
765
766     return ret;
767 }
768 EXPORT_SYMBOL(drm_gem_prime_mmap);
```

- 729 修正 vma 的页偏移。

  VMA 表示用户进程调用 mmap() 之后内核创建的一段虚拟地址区间。该 VMA 对应的物理/设备内存，相对于映射文件起始位置的页偏移。把当前 VMA 的页偏移，加上 GEM 对象在 DRM 文件地址空间的起始页号。

  - $vma->vm_pgoff$ 是页粒度偏移 (单位 PAGE_SIZE)。
  - $obj->vma_node$ 即 *struct drm_vma_node*，DRM(Direct Rendering Manager, 直接渲染管理器) VMA 节点。DRM 用结构体 *drm_vma_node* 来管理 GEM 对象在 DRM 设备文件的全局地址空间里的页偏移。
  - $drm_vma_node_start(node)$ 返回目标 VMA node 在 DRM 设备文件地址空间中的起始页偏移 (页为单位)。

  DRM 设备文件本身不是普通磁盘文件，是虚拟文件，它的地址空间被切分成很多段，每一段对应一个 GEM buffer 对象。

  drm_vma_node 负责为每个 GEM 对象在这个虚拟文件里分配一段页区间 (相当于给每个对象分配文件内的“页地址”)。用户 mmap() 传入的 offset，是对象内相对偏移，不是 DRM 全局文件偏移，所以在 mmap 处理函数里，必须把对象内部页偏移+ 对象在 DRM 文件中的起始页，填入 *vm_pgoff*。

  后续缺页异常 fault 处理时，会读取 *vma->vm_pgoff*，根据这个全局页偏移找到对应的 GEM 对象，再做物理页面映射。

- 731-742 如果 GEM 对象提供了自定义 mmap 回调，那么优先走这个自定义实现。
  - 735-739 驱动自定义 mmap 逻辑，填充 vma，建立页表。
  - 740 把 GEM 对象挂到 vma 私有数据域，后续缺页 close 时能拿到 obj。
- 744-745 分配并清零内存。

  内核原生 *drm_fops->mmap* 函数要求传入合法的 DRM 设备的 *struct file ** 和 *drm_file* 上下文。但现在是通过 *dma-buf fd* 做 mmap，不是直接打开*/dev/dri/cardX*，没有真实的 *file/drm_file*。所以临时伪造这两个结构体，用来调用驱动原生 mmap 函数。

  `priv->minor = obj->dev->primary` 把临时 *drm_file* 绑定到 DRM minor 设备 (/dev/dri/-cardX)。

  `fil->private_data = priv` linux file 结构惯例，*private_data* 指向驱动私有上下文 *drm_file*。

- 755-757 DRM VMA 节点有访问控制，防止未授权访问上下文 mmap GEM。这里给临时 priv 加上访问权限。
- 759 调用 DRM 设备驱动顶层的文件操作 mmap 回调，也就是 cardX 设备 fops 里的 mmap 函数。等价于 mmap 系统调用在打开*/dev/dri/cardX* 场景下的原生入口。这里伪造了 fil + priv，驱动的 *fops.mmap* 可以正常执行，完成 vma 设置、建立 CPU 页表映射。

- 761 撤销刚才临时授予的访问权限。

11.2.2 dma_buf_vmap()

该函数在**内核态**把一块 dma-buf 对应的离散物理页，映射成一段连续的内核虚拟地址，让内核驱动可以直接用指针读写整个 buffer。

API	映射到	调用方	用途
dma_buf_vmap()	内核虚拟地址	内核驱动	内核代码直接指针读写整个 *dma-buf*
drm_gem_prime_mmap()	用户态进程虚拟地址	用户态 *mmap()* 系统调用	用户态应用读写 *buffer*

Table 11–1: dma_buf_vmap() vs. drm_gem_prime_mmap()

dma_buf_vmap()

```
1199 void *dma_buf_vmap(struct dma_buf *dmabuf)
1200 {
1201     void *ptr;
1202
1203     if (WARN_ON(!dmabuf))
1204         return NULL;
1205
1206     if (!dmabuf->ops->vmap)
1207         return NULL;
1208
1209     mutex_lock(&dmabuf->lock);
1210     if (dmabuf->vmapping_counter) {
1211         dmabuf->vmapping_counter++;
1212         BUG_ON(!dmabuf->vmap_ptr);
1213         ptr = dmabuf->vmap_ptr;
1214         goto out_unlock;
1215     }
1216
1217     BUG_ON(dmabuf->vmap_ptr);
1218
1219     ptr = dmabuf->ops->vmap(dmabuf);
1220     if (WARN_ON_ONCE(IS_ERR(ptr)))
1221         ptr = NULL;
1222     if (!ptr)
1223         goto out_unlock;
1224
1225     dmabuf->vmap_ptr = ptr;
1226     dmabuf->vmapping_counter = 1;
1227
1228 out_unlock:
1229     mutex_unlock(&dmabuf->lock);
1230     return ptr;
1231 }
1232 EXPORT_SYMBOL_GPL(dma_buf_vmap);
```

- 1203-1204 防御性检查，捕获传递空指针。
- 1206-1207 如果驱动没有实现 vmap 回调，直接返回。

  `dmabuf->ops->vmap` 是 buffer 导出方 (AMDGPU 驱动) 实现的底层映射。这个 vmap 回调是可选的，驱动可以不支持。

- 1209 防止内核多线程并发 *dma_buf_vmap()/dma_buf_vunmap()* 产生竞态。
- 1210-1214 已经存在有效的 vmap 映射, 则复用已有映射不重新创建。
  - 1211 允许多处内核代码同时持有 vmap, 引用计数 +1。
  - 1212 如果计数 > 0 但 vmap_ptr 是空，说明内核内部状态损坏，直接触发 BUG 崩溃。
  - 1213 直接拿缓存好的内核虚拟地址。
- 1217 防御断言. 此时 vmap_ptr 必须是 NULL, 如果不为 NULL，说明状态不一致，触发 BUG。
- 1219-1223 真正创建内核虚拟映射, 调用 dma-buf 导出驱动的 vmap 回调。
- 1225 映射创建成功, 保存返回的内核虚拟地址到 *dmabuf->vmap_ptr*。
- 1226 初始化引用计数。

12

NUMA blance

numa balancing 的本质，是把谁在访问哪块内存这个信息，通过人为制造的缺页异常，从 MM(内存管理) 子系统采集出来，送到调度器做两件事：

- 1. 迁移内存: 把被某个 CPU 频繁访问、却位于远端 NUMA 节点的物理页，迁移到该 CPU 所在节点；
- 2. 迁移任务: 把任务调度到它大部分内存所在的节点上。

它不是一个独立子系统，而是调度器 (scheduler) 与内存管理 (mm) 之间的一条双向反馈回路。

numa balancing 是**内存局部性 (locality)**[①]的均衡，而不是**内存负载均衡**，简言之，不是内存总容量在 numa 各个节点平摊。

12.1 实现方法

内核无法硬件级地统计哪个 CPU 访问了哪一页。可行的近似手段是制造缺页：

定期把已存在的、可迁移用户页的 PTE 的 ***_PAGE_PRESENT*** 位清掉置上 ***_PAGE_PROTNONE*** 位,PFN(物理页帧) 号保留不变。由于 PTE 仍然指向真实物理页 (PFN 未变)，所以没有真的换出页。

任务下次访问时，由于 present 位为 0，于是触发 page fault。内核在缺页处理执行路径中调用 *do_numa_page()* 中：

- 先把 PTE 恢复成正常可访问
- 顺便读取 `page->_last_cpupid`(记录上次访问该页的 **CPU + PID**) 得到访问历史。
- 判断页是否” 错位”(misplaced)。是, 则异步迁移。
- 把” 本地/远端、私有/共享” 的统计喂给调度器。

[①] **内存局部性**指程序访问内存不是完全随机的, 访问的地址存在明显规律，倾向于重复访问一小片区域的现象。

需要注意的是 page->_last_cpupid 中的 *_last_cpupid* 不一定是 *struct page* 结构体的独立成员，有可能会打包在 *page->flags* 字段。这是由宏值 ***LAST_CPUPID_NOT_IN_PAGE_FLAGS*** 决定的。如下:

```
struct page {
  ... ...

#ifdef LAST_CPUPID_NOT_IN_PAGE_FLAGS
    int _last_cpupid;
#endif
} _struct_page_alignment;
```

numa balance 机制下的迁移分两种:

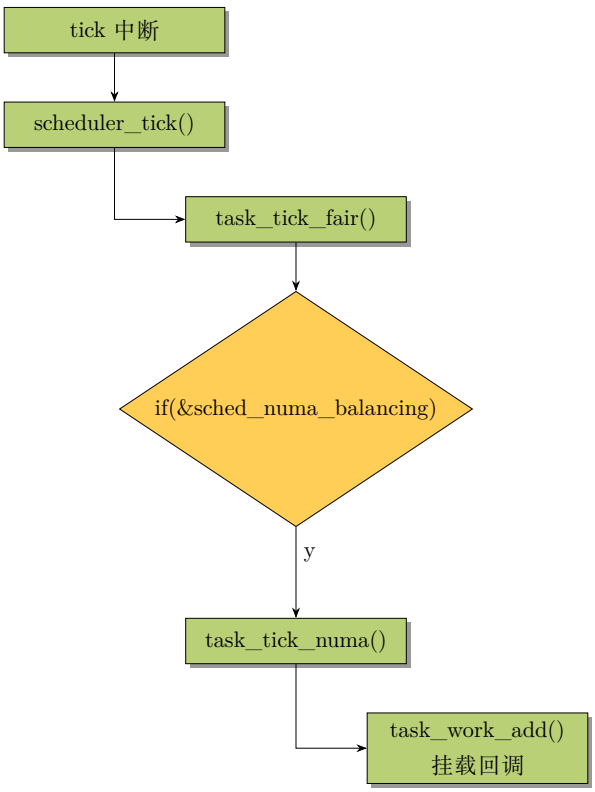
- **页迁移** 顾名思义，就是页迁移到目标节点的 cpu。这会由任务在 page fault 上下文中自己操作。
- **任务 (task) 迁移** 任务迁移到目标页所在节点的 cpu。这会在 page fault 处理的回程中由 task 的 work 回调操作。

页迁移是滞后的, 已经被访问了才发现错位; 任务迁移是前瞻的, 根据统计把任务 task 送到内存所在节点。两者互为补充。

12.2 NUMA balance 执行的三个阶段

12.2.1 扫描和标记 (NUMA hinting PTE)

在 tick 中断中会判断当前进程 (current) 是否需要进行 numa balance，若是，则给当前进程挂上回调 work。



PTE 扫描的入口函数是 *task_tick_fair()*, 在时钟中断上下文中由 *scheduler_tick()* 调用该函数。

task_tick_fair()

```
10693 static void task_tick_fair(struct rq *rq, struct task_struct *curr, int queued)
10694 {
10695     struct cfs_rq *cfs_rq;
10696     struct sched_entity *se = &curr->se;
10697
10698     for_each_sched_entity(se) {
10699         cfs_rq = cfs_rq_of(se);
10700         entity_tick(cfs_rq, se, queued);
10701     }
10702
10703     if (static_branch_unlikely(&sched_numa_balancing))
10704         task_tick_numa(rq, curr);
10705
10706     update_misfit_status(curr, rq);
10707     update_overutilized_status(task_rq(curr));
10708 }
```

- 10696 取当前任务的调度实体。
- 10698–10701 沿调度实体链逐级调用 entity_tick()(更新时间片、检查是否需要抢占)。这是调度器本身的逻辑。
- 10703 静态键判定。

  判断 *sched_numa_balancing* 这个静态 key 是否打开，也就是判断全局是否开启 NUMA 自动均衡。

  编译器编译这一行的时候, 识别出这是 static key，那么不生成 $load + cmp + jump$(普通 if 编译出的汇编步骤)。而是只生成指令 $jump \quad XXXX$。

  因为目前是 *static_branch_unlikely*, 所以默认这里的 if 判断返回 false, 也就是跳过 if 里面的代码块:*task_tick_numa()*。所以此时标签 XXXX 对应是 10706 行语句的地址。也就是默认预埋跳转到 10706 行的指令。

  当 `echo 1 > /proc/sys/kernel/numa_balancing`, 内核调用 `static_key_enable(&sched_numa_balancing)` 原地改写这条 jump 机器码, 把它换成 *nop*(空指令), 直接执行 *task_tick_numa()*。

  示意性代码如下，默认:

```
    jmp XXX
task_tick_numa:
    ...
XXXX:
    ...     # update_misfit_status()对应的汇编代码
```

  开启 numa balance 后，就类似代码被修改为 (实际修改的是内存中的机器码):

```
    nop
task_tick_numa:
    ...
XXXX:
    ...     # update_misfit_status()对应的汇编代码
```

- 10704 调用 *task_tick_numa()* **判断**要不要扫 NUMA，不做实际扫描 (实际扫描被推后到 task_work 里)。

tick 中之所以只做判断不实际扫描，是因为 tick 运行在中断上下文，不能做页表遍历 (要拿 mmap_lock、要刷 TLB、可能睡眠)。所以这里只做极轻量的计时判断，把重活推到安全上下文，也就是下面的函数来处理。

task_tick_fair() > *task_tick_numa()*

```
2905 static void task_tick_numa(struct rq *rq, struct task_struct *curr)
2906 {
2907     struct callback_head *work = &curr->numa_work;
2908     u64 period, now;
2909 
2910     /*
2911      * We don't care about NUMA placement if we don't have memory.
2912      */
2913     if ((curr->flags & (PF_EXITING | PF_KTHREAD)) || work->next != work)
2914         return;
2915 
2916     /*
2917      * Using runtime rather than walltime has the dual advantage that
2918      * we (mostly) drive the selection from busy threads and that the
2919      * task needs to have done some actual work before we bother with
2920      * NUMA placement.
2921      */
2922     now = curr->se.sum_exec_runtime;
2923     period = (u64)curr->numa_scan_period * NSEC_PER_MSEC;
2924 
2925     if (now > curr->node_stamp + period) {
2926         if (!curr->node_stamp)
2927             curr->numa_scan_period = task_scan_start(curr);
2928         curr->node_stamp += period;
2929 
2930         if (!time_before(jiffies, curr->mm->numa_next_scan))
2931             task_work_add(curr, work, TWA_RESUME);
2932     }
2933 }
```

- 2907 获取取任务内嵌的 task_work 节点。
- 2913-2914 三种早退的场景。
  - ***PF_EXITING*** 表示任务正在退出，*exit_mm()* 已可能摘掉 mm，此时访问 *curr->mm* 是危险的，必须立刻退出。
  - ***PF_KTHREAD*** 表示内核线程没有用户地址空间，NUMA balancing 无意义。
  - ***work->next != work*** 说明该 work 已经挂在队列上尚未执行 (*work->next = work* 是" 空闲" 的自表示)。
- 2922 获取纳秒级累计运行时间。
- 2923 把扫描周期从毫秒换算成纳秒。

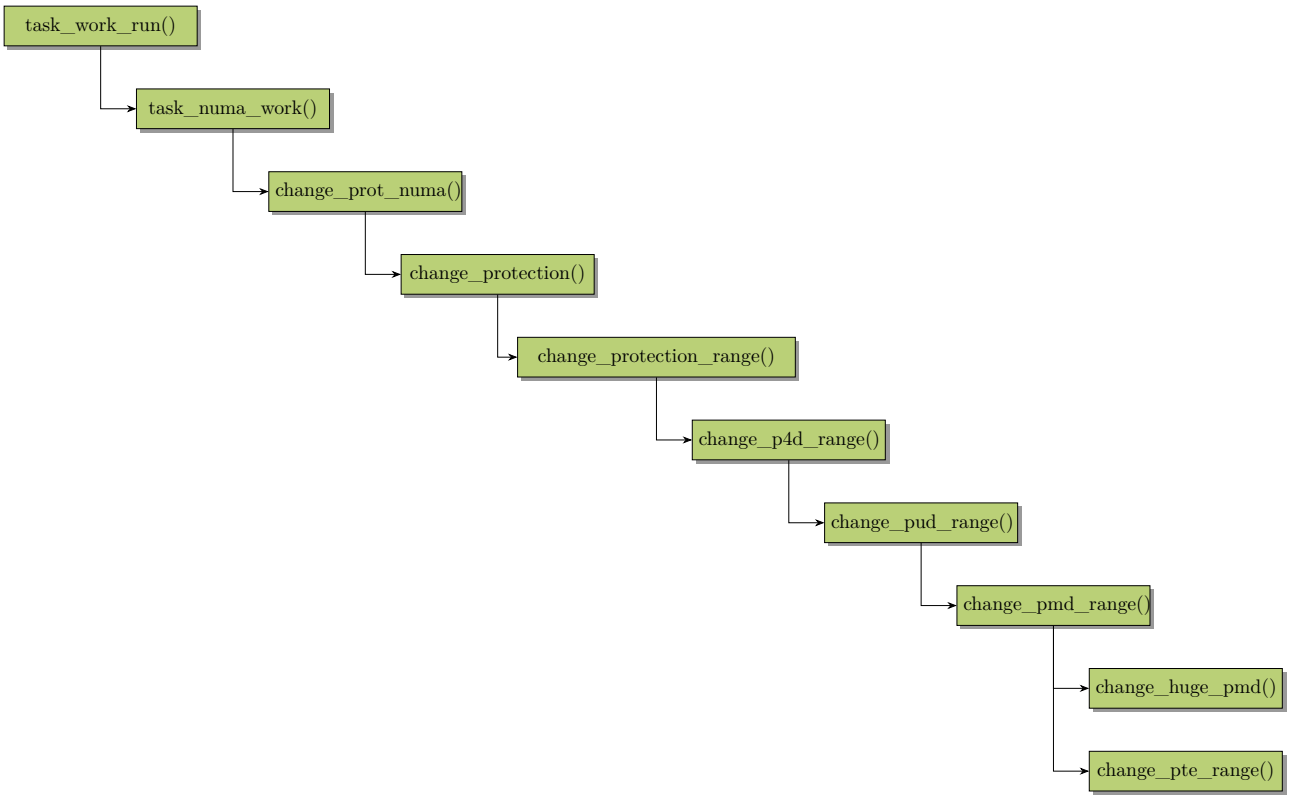
  扫描周期控制一个进程什么时候启动一轮 NUMA 页面扫描 (安装 hint PTE，采集访问局部性)。不是每次时钟中断都扫，按任务的执行时间做周期调度。
- 2925 判断距离上次扫描是否累计运行时间 > 一个扫描周期。

- 2926-2927 通过判断 task 的上次扫描时间点是否是 0，来判断本 task 是否是首次触发扫描。如果是首次出发扫描，那么会初始化扫描周期。
- 2928 更新 task 的最近一次扫描时间点。

  $node_stamp \mathrel{+}= period$ 而不是 $= now$。如果任务长时间没被调度，$now - node_stamp$ 可能远大于 period，用 $\mathrel{+}= period$ 能保证时间线不漂移，且不会因为一次长睡眠就补偿出连续多次扫描。
- 2930 `mm->numa_next_scan` 是全 mm 共享的节拍。若还没到点 ($jiffies < numa_next_scan$)，即使本 task 时间戳到了也不扫—-这是多线程的进程内” 只让一个线程扫描” 的一层过滤。
- 2931 挂入 task work，***TWA_RESUME*** 表示” 在返回用户态时执行”。这里只是入队。

在 task 从内核返回用户空间时，会执行 PTE 扫描和标记的工作。把一部分用户页 PTE 改成 PROT_NONE(NUMA hinting PTE)，造成不可访问。这由 *task_work_run()* 执行。

task_work_run() 会在返回用户空间前的 *resume_user_mode_work()* 中被调用。所以扫描代码运行在 task 自己的进程上下文，此时可以睡眠。如果扫描耗时长，影响的是这个 task 自己的时间片，天然公平。



task_work_run() *task_numa_work()*

```
2716 static void task_numa_work(struct callback_head *work)
2717 {
2718     unsigned long migrate, next_scan, now = jiffies;
2719     struct task_struct *p = current;
2720     struct mm_struct *mm = p->mm;
2721     u64 runtime = p->se.sum_exec_runtime;
```

```
2722     struct vm_area_struct *vma;
2723     unsigned long start, end;
2724     unsigned long nr_pte_updates = 0;
2725     long pages, virtpages;
2726 
2727     SCHED_WARN_ON(p != container_of(work, struct task_struct, numa_work));
2728 
2729     work->next = work;
2730     /*
2731      * Who cares about NUMA placement when they're dying.
2732      *
2733      * NOTE: make sure not to dereference p->mm before this check,
2734      * exit_task_work() happens _after_ exit_mm() so we could be called
2735      * without p->mm even though we still had it when we enqueued this
2736      * work.
2737      */
2738     if (p->flags & PF_EXITING)
2739         return;
2740 
2741     if (!mm->numa_next_scan) {
2742         mm->numa_next_scan = now +
2743             msecs_to_jiffies(sysctl_numa_balancing_scan_delay);
2744     }
2745 
2746     /*
2747      * Enforce maximal scan/migration frequency..
2748      */
2749     migrate = mm->numa_next_scan;
2750     if (time_before(now, migrate))
2751         return;
2752 
2753     if (p->numa_scan_period == 0) {
2754         p->numa_scan_period_max = task_scan_max(p);
2755         p->numa_scan_period = task_scan_start(p);
2756     }
2757 
2758     next_scan = now + msecs_to_jiffies(p->numa_scan_period);
2759     if (cmpxchg(&mm->numa_next_scan, migrate, next_scan) != migrate)
2760         return;
2761 
2762     /*
2763      * Delay this task enough that another task of this mm will likely win
2764      * the next time around.
2765      */
2766     p->node_stamp += 2 * TICK_NSEC;
2767 
2768     start = mm->numa_scan_offset;
2769     pages = sysctl_numa_balancing_scan_size;
2770     pages <<= 20 - PAGE_SHIFT; /* MB in pages */
2771     virtpages = pages * 8;     /* Scan up to this much virtual space */
2772     if (!pages)
2773         return;
2774 
2775 
2776     if (!mmap_read_trylock(mm))
2777         return;
2778     vma = find_vma(mm, start);
2779     if (!vma) {
2780         reset_ptenuma_scan(p);
2781         start = 0;
2782         vma = mm->mmap;
2783     }
2784     for (; vma; vma = vma->vm_next) {
2785         if (!vma_migratable(vma) || !vma_policy_mof(vma) ||
```

```
2786                is_vm_hugetlb_page(vma) || (vma->vm_flags & VM_MIXEDMAP)) {
2787                continue;
2788            }
2789
2790            /*
2791             * Shared library pages mapped by multiple processes are not
2792             * migrated as it is expected they are cache replicated. Avoid
2793             * hinting faults in read-only file-backed mappings or the vdso
2794             * as migrating the pages will be of marginal benefit.
2795             */
2796            if (!vma->vm_mm ||
2797                (vma->vm_file && (vma->vm_flags & (VM_READ|VM_WRITE)) == (VM_READ)))
2798                continue;
2799
2800            /*
2801             * Skip inaccessible VMAs to avoid any confusion between
2802             * PROT_NONE and NUMA hinting ptes
2803             */
2804            if (!vma_is_accessible(vma))
2805                continue;
2806
2807            do {
2808                start = max(start, vma->vm_start);
2809                end = ALIGN(start + (pages << PAGE_SHIFT), HPAGE_SIZE);
2810                end = min(end, vma->vm_end);
2811                nr_pte_updates = change_prot_numa(vma, start, end);
2812
2813                /*
2814                 * Try to scan sysctl_numa_balancing_size worth of
2815                 * hpages that have at least one present PTE that
2816                 * is not already pte-numa. If the VMA contains
2817                 * areas that are unused or already full of prot_numa
2818                 * PTEs, scan up to virtpages, to skip through those
2819                 * areas faster.
2820                 */
2821                if (nr_pte_updates)
2822                    pages -= (end - start) >> PAGE_SHIFT;
2823                virtpages -= (end - start) >> PAGE_SHIFT;
2824
2825                start = end;
2826                if (pages <= 0 || virtpages <= 0)
2827                    goto out;
2828
2829                cond_resched();
2830            } while (end != vma->vm_end);
2831        }
2832
2833    out:
2834        /*
2835         * It is possible to reach the end of the VMA list but the last few
2836         * VMAs are not guaranteed to the vma_migratable. If they are not, we
2837         * would find the !migratable VMA on the next scan but not reset the
2838         * scanner to the start so check it now.
2839         */
2840        if (vma)
2841            mm->numa_scan_offset = start;
2842        else
2843            reset_ptenuma_scan(p);
2844        mmap_read_unlock(mm);
2845
2846        /*
2847         * Make sure tasks use at least 32x as much time to run other code
2848         * than they used here, to limit NUMA PTE scanning overhead to 3% max.
2849         * Usually update_task_scan_period slows down scanning enough; on an
```

```
2850         * overloaded system we need to limit overhead on a per task basis.
2851         */
2852        if (unlikely(p->se.sum_exec_runtime != runtime)) {
2853            u64 diff = p->se.sum_exec_runtime - runtime;
2854            p->node_stamp += 32 * diff;
2855        }
2856    }
```

- 2727 work 必须是 *task_struct->numa_work* 成员。这里纯粹是防御性检查。
- 2729 把 work->next 重置为 work 自身。

  这是 work 标记为空闲的方式。之后判定 work 是否可用就是通过 *work->next != work* 进行判断。
- 2738-2739 判断当前进程是否正在退出，若是，则立即返回。

  必须放在任何 current->mm 解引用 (使用) 之前。原因是 exit_mm() 在 *exit_task_work()* 之前执行, $current- > mm$ 到这里可能已经为 NULL。
- 2741-2744 设置 mm 共享扫描节拍: 下次扫描时刻 = 当前时刻 + *scan_delay*。
- 2749 把 mm 共享节拍读入局部变量。
- 2750-2751 没到扫描时间点就直接返回。
- 2753-2756 若 current 的 numa 扫描周期 (*numa_scan_period*) 还未未初始化，则此刻初始化。
- 2758 计算 current 的下一次扫描时刻。
- 2759-2760 *->numa_next_scan* 等于 migrate 时才成功写入。否则，说明同 mm 的另一个线程已经抢先扫描并推后了时间戳，本线程立刻返回。
- 2766 进行扫描退避，延迟自己 ($node_stamp+ = 2 \times TICK_N SEC$), 让同 mm 的其它线程在下一轮更容易” 抢到” 扫描权。
- 2768 start 从上次扫描到的位置继续。
- 2769-2770 scan_size 换算成页数。
- 2771-2773 允许扫描的虚拟地址空间上限是物理页数的 8 倍。

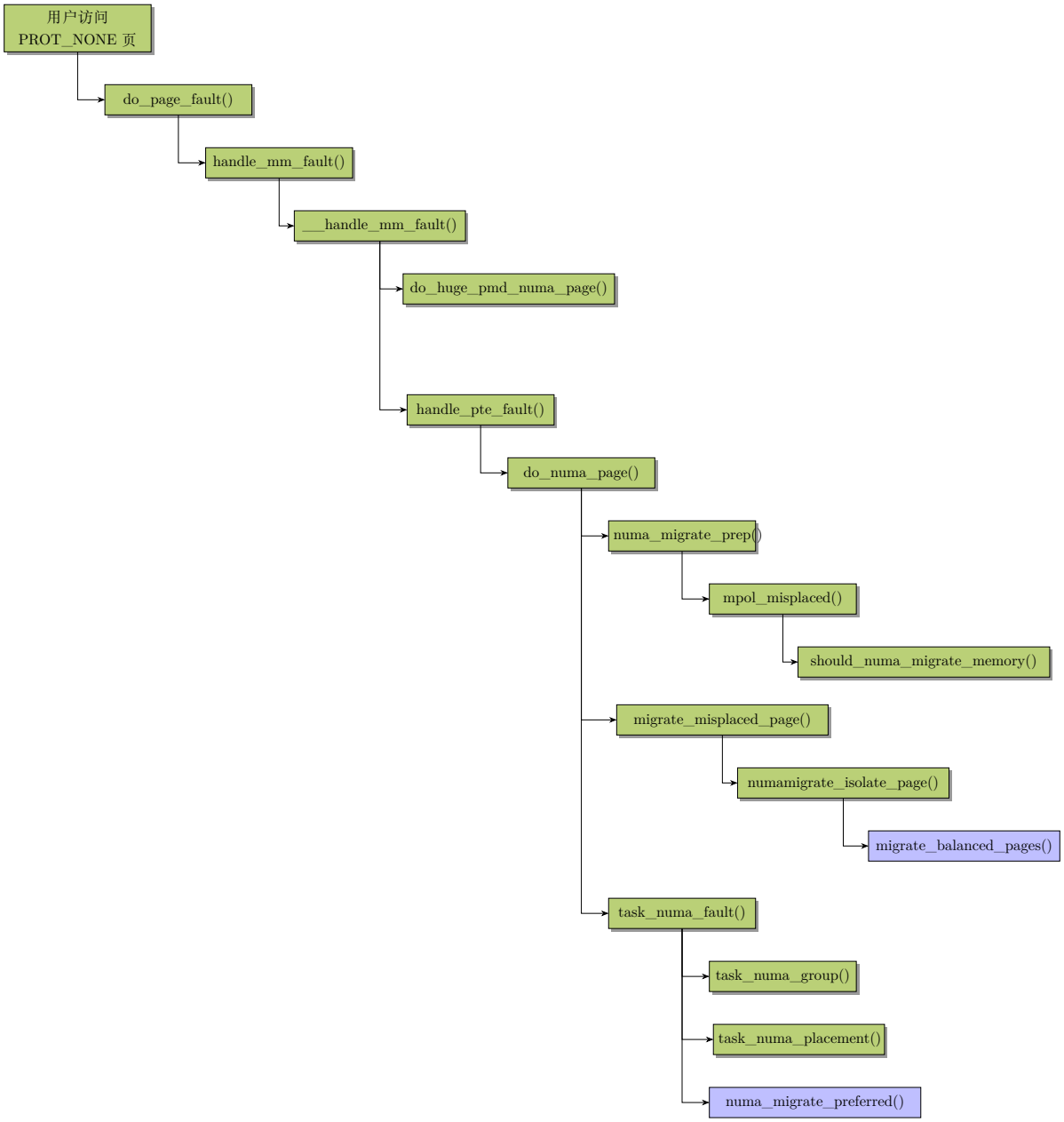
  因为 VMA 里常有” 未分配的空洞” 或” 已经被标记过 prot_numa 的页”，这些不产生 PTE 更新。如果只看物理页数，遇到大段空洞时就会卡住扫不完。用 8 倍虚拟空间做上限，让扫描器能快速穿过这些” 低产出” 区域。

  用户把 scan_size 设成 0，则直接返回。
- 2778 从游标位置找 VMA。
- 2779-2783 若找不到 (游标已过最后一个 VMA)，说明一轮扫描完成。若如此则使游标归零，然后从 $mm- > mmap$ 重新开始。
- 2784 遍历 VMA 链表。
- 2785-2788 跳过下面四种场景:

  - *!vma_migratable(vma)* 不可迁移 (如 mlock/特殊映射)，迁不了就不采样。
  - *!vma_policy_mof(vma)* 没有设置” 跟随 CPU 迁移” 策略。这是尊重用户 *mbind/set_mempolicy* 的显式意图。
  - *is_vm_hugetlb_page(vma)* hugetlb 页有自己的缺页路径。
  - *VM_MIXEDMAP* 混合映射 (如 DAX、device memory)，页不归 LRU 管，迁不了。
- 2796-2798 跳过两种场景。
  - *!vma->vm_mm* **无 mm 的 VMA 是内核内部映射**，不该动。
  - 只读文件映射 (vm_file 存在且权限恰为 VM_READ) 典型是共享库和 vdso。这些页在多个进程间属” 缓存复制”，迁移收益极小。
- 2804-2805 跳过不可访问的 VMA。这个检查至关重要：真正的 PROT_NONE 页和 NUMA hinting 的 PROT_NONE 页在硬件层面完全一样，唯一的区分依据就是”VMA 本身是否可访问”。如果给一个本来就是 PROT_NONE 的 VMA 打 hinting 标记，缺页时无法分辨是用户真的想保护还是内核在采样。
- 2807 在单个 VMA 内按 pages 大小切片推进。
- 2809-2810 计算本次区间末尾, 按 2MB 对齐，再钳位到 VMA 末尾。
- 2811 调用接口返回实际被修改的页数。
- 2821-2822 仅当真的改了页时才扣减 pages 预算: 如果这一段全是空洞或已标记过的页 ($nr_pte_updates == 0$)，就不算” 产出”，预算不消耗, 扫描器可以继续往后推进。
- 2823 virtpages 无论有无产出都扣减——这是防死循环的硬性上限。
- 2825 游标前进。
- 2827-2828 预算都耗尽, 跳出。
- 2840-2843 如果 vma 非空，说明循环是从 2826/2827 的预算耗尽退出的，游标停在中间，保存游标。如果 vma 为 NULL，说明 VMA 链表走到了尽头, 但” 走到尽头” 可能是被 2785-2805 的 continue 跳过的不可迁移 VMA 造成的。所以必须在这里检查并 reset，否则下一轮找不到可扫的 VMA 会一直停在末尾，扫描器” 卡死”。
- 2852-2855 开销自我限制。如果本次扫描消耗了 diff 纳秒的 CPU 时间，要求任务至少花 32 倍的时间干别的事，才允许再扫一次。这把扫描开销硬性压到约 3%(1/33) 以内。这是在自适应机制 (*update_task_scan_period()*) 失效时 (如系统过载导致统计失真) 的最后一道保险。

12.2.2 缺页采样

用户下一次访问被标记的页，触发 NUMA hinting fault，恢复 PTE 可访问，并记录” 哪个 CPU 访问了哪个节点的页”。

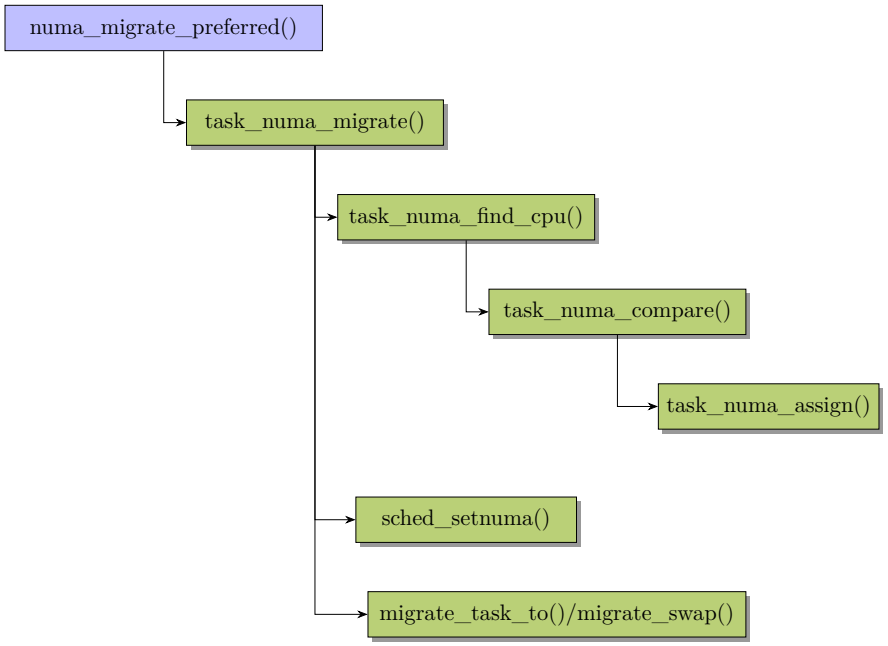


12.2.3 决策迁移

该操作在 page fault 上下文进行, 决定迁移进程还是迁移页。

12.2.3.1 迁移进程

判断要迁移任务，则调用 numa_migrate_perfered() 函数，如下:



12.2.3.2 迁移页

should_numa_migrate_memory() 缺页时判断要不要搬内存页。若需要，则调用 ***migrate_misplaced_page()*** 进行页迁移。

12.3 防止 NUMA balance 破坏全局负载均衡

numa balancing 不替代 cpu 负载均衡，而是在 (cpu) 负载均衡的约束下工作：

- 迁移前会检查目标节点是否有空闲算力、迁移后是否破坏负载均衡 (*load_too_imbalanced()*)。
- 负载均衡选 cpu core 时会参考 NUMA 统计，避免把已本地化的任务拉走 (*migrate_degrades_locality()*、*fbq_classify_rq()*、*find_idlest_group()*)；
- 用 *rq->numa_migrate_on* 互斥，避免同一目标 CPU 被并发 numa 迁移抢占。

13

与用户空间的交互

13.1 查看空闲内存

13.1.1 free

用命令行工具 free 可以获得当前可用内存数。
free 打印的信息如下:

```
comlee:~/project/github/linux/mm >>>free
              total        used        free      shared  buff/cache   available
Mem:       16053896     2835104    10012924      526844     3205868    12349412
Swap:      46874620           0    46874620
comlee:~/project/github/linux/mm >>>
```

图 13–1: free 的输出信息

- ***total*** 系统中可使用的物理内存总量。
- ***used*** 当前正在使用、且在内存紧张时无法轻易释放的内存估算值；其值等于可用总物理内存减去通过启发式计算得到的可用内存。
- ***free*** 空闲内存。
- ***shared*** 共享内存总量。包括进程间共享内存，基于内存的文件系统的内存 (譬如 tmpfs 等)。
- ***buff/cache*** 不需要 swap out 即可回收的内存总数。

  buff 是总的块缓存页 (block device pages) 的数量，是被内核使用的缓存。它属于文件页 (file-backed pages). 但它是一种特殊的文件页。块缓存页是裸磁盘视角的文件页，是块设备层的缓存。普通文件页是文件系统视角的页，是文件系统层的缓存。换言之，块缓存页上的数据对应磁盘或外存上物理上连续的单元，是对“裸块 (raw) 设备”直接读写时的缓存 (不经过文件系统)。而普通文件页上的数据经过了文件系统层，文件逻辑上连续的，但映射到磁盘上不是物理上连续的单元。

NOTE

文件页、匿名页都是用户空间在使用。而块缓冲页是内核使用，所以被单独统计在 *Buffers* 里。这也是块缓冲 (文件) 页和普通文件页的区别之一。

swapcache 页是已经有对应 swap 分区的内存页缓存。它本质是匿名页。但是内核在统计总的文件页时会将 swapcache 页也包含进去。在内核实现里文件页都挂在 address_space. 为了方便管理，匿名页换出到 swap 分区时，也给它造了一个虚拟的 address_space, 于是内核统计时，错误地把 swapcache 页也包含进去了。

address_space 可以理解成" 一页页磁盘数据在内存里的管理目录"。内核会给每一类有磁盘后备的数据页配一个 struct address_space，用来：管理这个缓存页: 记录偏移、页面的映射关系、记录回写、脏页、LRU 等信息。

三类对象拥有 address_space：

- 普通文件 inode, 管理这个文件的 page cache
- 块设备 bdev, 管理 block device pages(也就是 Buffers)
- swap 分区, 管理 swap cache 页面 (也就是已经有 swap 分区映射的内存页)

综上，

$$buffers = block_device_pages$$
$$cached = total_filed_backed - (block_device_pages + swap_cache_pages)$$

于是，它们之和为:

$$\begin{aligned} buff/cache &= cached + buffers \\ &= total_filed_backed - (block_device_pages + swap_cache_pages) \\ &\quad + block_device_pages \\ &= total_filed_backed - swap_cache_pages \end{aligned}$$

slab_reclaimable 页是内核可回收对象，也要算进可回收内存，所以:

$$buff/cache = slab_reclaimable + total_filed_backed - swap_cache_pages$$

- ***available*** 估算安全可用的内存量。与 cache 或 free 字段提供的数据不同的是，它还会考虑到，由于 slab 正在使用中，并非所有可回收 slab 内存都能被实际回收。该值是经验估算值— 为了避免内存颠簸，必须保留一定比例的页缓存；为了避免内存回收被触发并进而引发 OOM killer 风险，我们必须同时保证两者：将空闲页面维持在高水位线之上，高于低内存预留值 (min 水位线)。

  按上面的理解，应该 $available \leq (buff/cache)$ 才对, 但是违反直觉的是，我们看到的是相反的情况。这是因为:

  这里显示出来的 available 是从 buff/cache 里算出来的一部分, 真正的安全内存量应该还要在这个值基础上加上 free 这部分:

$$available = free + (buff/cache)_partial_usable$$

13.1.2 /proc/meminfo 节点

通过 *cat /proc/meminfo* 节点同样可以获得和 free 相似的数据。

```
comlee:~/project/mem2/memory_mgt/picture >>>cat /proc/meminfo
MemTotal:       16053896 kB
MemFree:        10020832 kB
MemAvailable:   12392828 kB
Buffers:          351180 kB
Cached:          2638228 kB
SwapCached:            0 kB
Active:          1339656 kB
Inactive:        3705760 kB
Active(anon):       2292 kB
Inactive(anon):  2570176 kB
Active(file):    1337364 kB
Inactive(file):  1135584 kB
Unevictable:      324160 kB
Mlocked:              16 kB
SwapTotal:      46874620 kB
SwapFree:       46874620 kB
Dirty:              240 kB
Writeback:            0 kB
AnonPages:       2380192 kB
Mapped:           776696 kB
Shmem:            516452 kB
KReclaimable:     241584 kB
Slab:             405144 kB
SReclaimable:     241584 kB
SUnreclaim:       163560 kB
KernelStack:       12768 kB
PageTables:        29036 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
WritebackTmp:          0 kB
CommitLimit:    54901568 kB
Committed_AS:    9912916 kB
VmallocTotal:   34359738367 kB
VmallocUsed:       50444 kB
VmallocChunk:          0 kB
Percpu:            14720 kB
HardwareCorrupted:     0 kB
AnonHugePages:         0 kB
ShmemHugePages:        0 kB
ShmemPmdMapped:        0 kB
FileHugePages:         0 kB
FilePmdMapped:         0 kB
HugePages_Total:       0
HugePages_Free:        0
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:               0 kB
DirectMap4k:      363636 kB
DirectMap2M:     7706624 kB
DirectMap1G:     9437184 kB
comlee:~/project/mem2/memory_mgt/picture >>>
```

图 13–2: /proc/meminfo 节点的输出信息

下面列出了怎样通过/proc/meminfo 各项计算得到 free 打印的值:

free	*/proc/meminfo*	**说明**
total	*MemTotal*	系统中可使用的物理内存总量
used	$MemTotal - MemAvailable$	当前正在使用、且在内存紧张时无法轻易释放的内存估算值
free	*MemFree*	空闲内存
shared	*Shmem*	共享内存总量
buff/cache	$Buffers + Cached + SReclaimable$	不需要 *swap out* 即可回收的内存总数
available	*MemAvailable*	安全可用内存量的估算值

13.1.2.1 查看根 LRU 向量

通过 proc 节点/proc/meminfo 也能够查看全局 (global) 的 lruvec(即根 LRU 向量)。

```
comlee:~ >>>cat /proc/meminfo |grep -i active --color=no
Active:          1218368 kB
Inactive:        3106660 kB
Active(anon):       2588 kB
Inactive(anon):  2111172 kB
Active(file):    1215780 kB
Inactive(file):   995488 kB
```

13.2 检测内存碎片

可以通过 *cat /proc/buddyinfo* 节点来查看伙伴系统中各阶 (order-n) 连续内存的剩余数量，以此判断内存碎片情况。

13.2.1 /proc/buddyinfo

cat /proc/buddyinfo 得到的信息如下:

```
comlee:~/project/mem2/memory_mgt/picture >>>cat /proc/buddyinfo
Node 0, zone      DMA      0      0      0      0      0      0      0      0      1      2      2
Node 0, zone    DMA32      3      6      9      9      8      6      2      3      5      2    347
Node 0, zone   Normal   1715   1374   2241   2488   3821   1894    724    155    197     43   1769
comlee:~/project/mem2/memory_mgt/picture >>>
```

图 13–3: /proc/buddyinfo 节点的输出信息

图中每一列对应不同的页块阶 (order), 从左往右依次增大 1 阶。每行代表不同的内存 zone(譬如 DMA、DMA32 和 Normal 等).

13.3 查看进程的内存占用

13.3.1 /proc/$pid/status

cat /proc/$pid/status 可以得到进程相关的信息，下面截图主要截取了内存相关的输出。其中 $pid 是进程 ID 号。

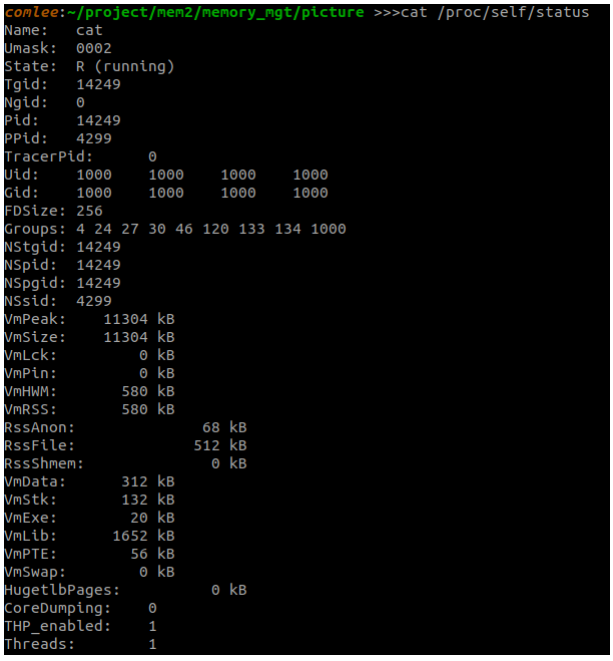


图 13–4: /proc/$pid/status 节点的输出信息

其中各项的说明如下:

- ***VmHWM*** 进程物理内存的峰值使用量，即进程在整个运行期间实际占用物理内存的最大值。
- ***VmRSS*** 当前实际占用的物理内存大小。等于以下三项之和:
  - ***RssAnon*** 目标进程匿名页当前所占物理内存的大小。
  - ***RssFile*** 目标进程文件页当前所占物理内存的大小。
  - ***RssShmem*** 用于共享内存的内存大小，包括 tmpfs、匿名共享内存、System V 共享内存等。

这些数值来源于内核中追踪内存事件的统计计数器。为避免对性能造成影响，这些计数器的更新会被限流。因此，这些数值并非完全精确。

13.4 查看内核工作集页的状态

13.4.1 /proc/vmstat

内核通过一组 *WORKINGSET_** 工作集计数器跟踪不同类型页面的活跃性, 这些计数器反映了系统对匿名/文件内存的动态管理行为。

宏名称	用途
WORKINGSET_FAULT_ANON	记录匿名页面发生缺页异常的次数。当进程访问的匿名页面不在物理内存中时，触发缺页异常，该计数器加 *1*。用于帮助分析系统的内存使用情况和性能瓶颈。频繁的匿名页面缺页可能意味着内存不足或页面置换算法不合理。
WORKINGSET_FAULT_FILE	记录与文件映射相关的页面发生缺页异常的次数。文件映射是将磁盘文件的内容映射到进程的虚拟地址空间，当这些映射页面不在物理内存中时，使用该宏计数。有助于了解文件映射操作对系统性能的影响，分析文件访问的局部性和磁盘 *I/O* 压力。如果文件映射页面频繁缺页，可能需要优化文件读取策略或增加内存。
WORKINGSET_REFAULT_ANON	记录匿名页面再次发生缺页异常 *(refault)* 的次数。当一个匿名页面之前已经发生过缺页异常被加载到内存，之后由于某些原因（如被换出内存）再次发生缺页异常时进行计数。可用于分析系统中匿名页面的访问模式和内存使用效率。频繁的匿名页面再次缺页可能意味着内存紧张，或者页面置换算法不够优化。
WORKINGSET_REFAULT_FILE	记录与文件映射相关的页面再次发生缺页异常的次数。文件映射是将磁盘文件的内容映射到进程的虚拟地址空间，当这些映射页面之前已处理过缺页异常，之后又因被换出等原因再次发生缺页时，使用该宏计数。用于帮助了解文件映射页面的访问特性，评估文件映射机制的性能，以及发现可能存在的磁盘 *I/O* 瓶颈。
WORKINGSET_ACTIVATE_ANON	匿名页面从欠活跃状态 *(*链表*)* 转变成为活跃状态 *(*链表*)* 的操作次数。用于衡量系统中匿名页面加入活跃链表操作的频繁程度，反映进程内存访问模式和内存管理策略的有效性。
WORKINGSET_ACTIVATE_FILE	文件页从欠活跃状态 *(*链表*)* 转换成为活跃状态 *(*链表*)* 的操作次数。用于分析文件映射页面的使用情况，有助于优化文件映射相关的内存管理。
WORKINGSET_RESTORE_ANON	从交换分区 *(swapin)* 换回内存的页面数。通常在系统进行内存回收后，当需要再次使用之前被换出的匿名页面时，会从交换分区 *(Swap)* 换回内存。可以了解系统中匿名页面的换入换出情况，评估内存回收策略对匿名页面的影响。
WORKINGSET_RESTORE_FILE	记录与文件映射相关页面 *restore* 操作的次数。当文件映射页面被换出后又需要重新使用时，系统会从磁盘重新加载文件页。用于帮助分析文件映射页面的换入换出行为，优化文件映射的内存管理策略。
WORKINGSET_NODERECLAIM	记录节点 *(node)* 进行内存回收的次数。在 *NUMA* 系统中，当某个节点的内存资源紧张时，会进行内存回收操作，此宏用于统计这些操作的次数。用于监控 *NUMA* 系统中各个节点的内存使用情况和回收频率，有助于优化多节点系统的内存分配和管理策略。

可以通过 /proc/vmstat 获取这些计数器的值。此处因为系统内存充足所以显示值都为 0：

```
comlee:~ >>>sudo cat /proc/vmstat |grep "workingset_" --color=no
workingset_nodes 0
workingset_refault_anon 0
workingset_refault_file 0
workingset_activate_anon 0
workingset_activate_file 0
workingset_restore_anon 0
workingset_restore_file 0
workingset_nodereclaim 0
```

其中 WORKINGSET_ACTIVATE_ANON 和 NR_ACTIVE_ANON 计数器的区别在于：NR_ACTIVE_ANON 是系统中当前处于活跃状态的匿名页面数量计数器，是一个静态的统计。而 WORKINGSET_ACTIVATE_ANON 计数器记录的是匿名页面从欠活跃链表转移到活跃状态链表的操作次数，是对一个动态行为的统计，也就是这里的 refaults 变量值。

14

初始化

14.1 bootmem_init()

bootmem_init()

```
398 void __init bootmem_init(void)
399 {
400     unsigned long min, max;
401
402     min = PFN_UP(memblock_start_of_DRAM());
403     max = PFN_DOWN(memblock_end_of_DRAM());
404
405     early_memtest(min << PAGE_SHIFT, max << PAGE_SHIFT);
406
407     max_pfn = max_low_pfn = max;
408     min_low_pfn = min;
409
410     arm64_numa_init();
411
412     /*
413      * must be done after arm64_numa_init() which calls numa_init() to
414      * initialize node_online_map that gets used in hugetlb_cma_reserve()
415      * while allocating required CMA size across online nodes.
416      */
417 #if defined(CONFIG_HUGETLB_PAGE) && defined(CONFIG_CMA)
418     arm64_hugetlb_cma_reserve();
419 #endif
420
421     dma_pernuma_cma_reserve();
422
423     /*
424      * sparse_init() tries to allocate memory from memblock, so must be
425      * done after the fixed reservations
426      */
427     sparse_init();
428     zone_sizes_init(min, max);
429
430     memblock_dump_all();
431 }
```

- 402 将系统 DRAM 最小起始物理地址按页 (通常是 4 k 字节) 对齐，并向上取整。将对齐取整后得

到的物理页帧号返回。PFN_UP 的实现如下：

假设物理起始地址为 0x80001010，那么取整后得到的物理页帧号是:

$$\begin{aligned}(0x80001010 + 2^{12} - 1) >> 12 &= (0x80001010 + 4096 - 1) >> 12 \\ &= 0x8000200F >> 12 \\ &= 0x80002\end{aligned}$$

实际对应的物理地址是 0x80002000(即,$0x80002 << 12$)，这里可以看到 $[0x80001010, 0x80002000)$ 这段内存在起始页帧号之外。因为伙伴系统只管理完整的页帧，而这部分不足一页，所以不再纳入内存管理，相当于舍弃不足一页的部分。当然，硬件内存起始地址出现非页对齐的场景比较罕见。

- 403 同理, 得到系统 DRAM 最大物理地址按页对齐取整后，返回的物理页帧号。这次也舍弃了不能按页对齐而多出的部分。需要注意的是，这次是向下取整。
- 405 启动早期对内存硬件进行自检。
- 407-408 给低端内存 ($< 896M$) 的最低和最高物理页帧号的全局变量赋值。这里没有判断内存地址是否 $< 896M$, 是因为这段代码是 arm64 的。
- 410 arm64 平台 numa 节点初始化。
- 418 arm64_hugetlb_cma_reserve() 是 arm64 平台巨页 CMA 内存预留专用函数，读取启动参数确定预留容量，遍历所有 NUMA 在线节点、在各节点本地内存分别预留连续物理内存，专门供给 hugetlb 巨页使用；

  该函数必须在打开配置项 *CONFIG_HUGETLB_PAGE* 和 *CONFIG_CMA* 后才会编译。并且必须在 arm64_numa_init() 完成之后再执行；因为 arm64_numa_init() 内部会调用 numa_init() 初始化 node 节点的全局位图, 该位图会在 arm64_hugetlb_cma_reserve() 中被使用。
- 421 为每个 NUMA 节点单独预留 DMA 专用 CMA 内存池，给设备 DMA 传输使用，解决 DMA 需要物理连续内存的需求。
- 427 该函数是 Linux SPARSE 稀疏内存的初始化主函数，负责建立稀疏内存的页表管理框架，适配物理内存不连续、存在内存空洞、内存分散在多个 NUMA 节点的服务器。
- 428 遍历所有在线 NUMA 节点与稀疏内存段，按 zone 内存域统计各分区总物理页数量，初始化每个 zone 基础管理元数据，并将 zone 挂载到对应节点的 pgdat 结构中，为后续 CMA 预留、伙伴系统、内存水位线、kswapd 回收提供准确的分区内存拓扑数据。
- 430 把内核启动阶段 memblock 子系统记录的全部物理内存拓扑完整打印到内核日志，方便查看物理内存分布、预留区域、空洞、CMA/设备保留内存等信息。

14.1.1 arm64_hugetlb_cma_reserve()

arm64_hugetlb_cma_reserve() 是系统为 Hugetlb 巨页预留 CMA 内存的入口函数。

bootmem_init() > *arm64_hugetlb_cma_reserve()*

```
39 void __init arm64_hugetlb_cma_reserve(void)
40 {
41     int order;
42
43 #ifdef CONFIG_ARM64_4K_PAGES
44     order = PUD_SHIFT - PAGE_SHIFT;
45 #else
46     order = CONT_PMD_SHIFT + PMD_SHIFT - PAGE_SHIFT;
47 #endif
48     /*
49      * HugeTLB CMA reservation is required for gigantic
50      * huge pages which could not be allocated via the
51      * page allocator. Just warn if there is any change
52      * breaking this assumption.
53      */
54     WARN_ON(order <= MAX_ORDER);
55     hugetlb_cma_reserve(order);
56 }
```

- 43-47 根据内核编译时选择的基页尺寸, 算出当前平台硬件能支持的最大的巨页对应的页阶数值。

  43-44 在基页的 size 是 4 k 字节的情况下，*PUD_SHIFT - PAGE_SHIFT* 算出 PUD 级别映射的超大巨页的 order。

  46 计算非 4 k 基页下，系统支持的最大巨页的 order。

  超大巨页无法通过普通页分配器分配，必须依靠 Hugetlb CMA 预留内存。

- 54 *order <= MAX_ORDER* 代表最大巨页阶数 ≤ 伙伴系统最大阶，但此时巨页完全可以直接从普通页分配器获取，不需要 CMA 预留，违背该函数设计因此触发警告。

- 55 调用通用接口，为超大巨页在各 NUMA 节点预留 CMA 连续内存。

14.1.2 hugetlb_cma_reserve()

bootmem_init() > *arm64_hugetlb_cma_reserve()* > *hugetlb_cma_reserve()*

```
5605 void __init hugetlb_cma_reserve(int order)
5606 {
5607     unsigned long size, reserved, per_node;
5608     int nid;
5609
5610     cma_reserve_called = true;
5611
5612     if (!hugetlb_cma_size)
5613         return;
5614
5615     if (hugetlb_cma_size < (PAGE_SIZE << order)) {
5616         pr_warn("hugetlb_cma: cma area should be at least %lu MiB\n",
5617             (PAGE_SIZE << order) / SZ_1M);
5618         return;
5619     }
5620
5621     /*
5622      * If 3 GB area is requested on a machine with 4 numa nodes,
```

```
5623         * let's allocate 1 GB on first three nodes and ignore the last one.
5624         */
5625        per_node = DIV_ROUND_UP(hugetlb_cma_size, nr_online_nodes);
5626        pr_info("hugetlb_cma: reserve %lu MiB, up to %lu MiB per node\n",
5627            hugetlb_cma_size / SZ_1M, per_node / SZ_1M);
5628
5629        reserved = 0;
5630        for_each_node_state(nid, N_ONLINE) {
5631            int res;
5632            char name[CMA_MAX_NAME];
5633
5634            size = min(per_node, hugetlb_cma_size - reserved);
5635            size = round_up(size, PAGE_SIZE << order);
5636
5637            snprintf(name, sizeof(name), "hugetlb%d", nid);
5638            res = cma_declare_contiguous_nid(0, size, 0, PAGE_SIZE << order,
5639                             0, false, name,
5640                             &hugetlb_cma[nid], nid);
5641            if (res) {
5642                pr_warn("hugetlb_cma: reservation failed: err %d, node %d",
5643                    res, nid);
5644                continue;
5645            }
5646
5647            reserved += size;
5648            pr_info("hugetlb_cma: reserved %lu MiB on node %d\n",
5649                size / SZ_1M, nid);
5650
5651            if (reserved >= hugetlb_cma_size)
5652                break;
5653        }
5654    }
```

- 5612-5613 若启动参数未配置 ***hugetlb_cma=*** (全局预留大小为 0)，直接退出函数，不做任何 CMA 预留。换言之，如果要内核启动时预留巨页专用的 CMA 内存区域，就必须在启动参数里通过 *hugetlb_cma=* 传入参数，否则不会创建巨页专属 cma 区域。

  hugetlb_cma= 启动命令行参数的解析逻辑，会把用户传入的数值 (如 hugetlb_cma=16G) 统一转换成字节存入全局变量 *hugetlb_cma_size*。

- 5615-5619 $PAGE_SIZE << order$ 表示最大巨页 1 页所占的字节数。若用户配置的预留巨页 CMA 大小，连一张最大巨页都放不下，打印警告日志并直接退出。

- 5625-5627 计算单节点平均预留大小。

- 5630 遍历系统中所有在线可用 NUMA 节点，按前面算出的平均预留大小，逐个节点分配 CMA 内存。

- 5634 计算当前节点本次实际要预留的大小。

  per_node 是单节点平均预留内存的上限；$hugetlb_cma_size - reserved$ 是还未分配完的剩余总容量。本次实际要预留的大小取两者较小值。

- 5635 CMA 预留内存必须是最大巨页大小的整数倍 (向上对齐)，以此保证这片 CMA 区域可以完整存放多张超大巨页。

- 5637-5645 给当前节点的 CMA 池命名，并在指定 NUMA 节点声明预留一段连续 CMA 内存。

- 5647 预留成功则把当前节点预留的 size，累加进全局已预留总容量。

- 5651-5652 累加的预留总量字节数 ≥ 预留总量。

14.1.3 arm64_numa_init()

bootmem_init() > *arm64_numa_init()*

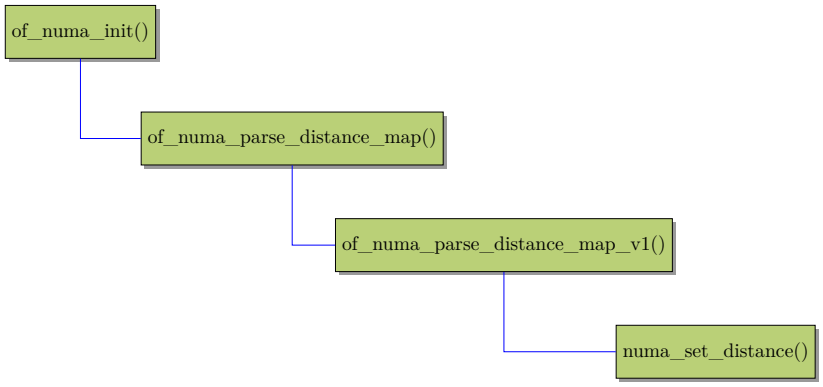
```
454 void __init arm64_numa_init(void)
455 {
456     if (!numa_off) {
457         if (!acpi_disabled && !numa_init(arm64_acpi_numa_init))
458             return;
459         if (acpi_disabled && !numa_init(of_numa_init))
460             return;
461     }
462 
463     numa_init(dummy_numa_init);
464 }
```

- 456-461 如果 numa 没有被禁用，则进入此代码块。

  如果 ACPI 没有被禁用，则使用 ACPI 解析 numa 节点信息。如果被禁用或解析失败，则使用设备树解析 numa 信息。

  需要注意的是在 arm64_acpi_numa_init() 中会填充真实的 numa node 的距离信息到二维距离矩阵中。如果是以 dts 设备树的形式进行 numa init, 那么会解析出设备树中的 node 距离信息来填充距离矩阵。

  以 dts 树的解析为例，从 of_numa_init() 到 numa_set_distance() 将距离信息填充矩阵的整个流程如下:



- 463 这是兜底方案，把全局内存划为唯一内存节点。

14.1.4 numa_init()

bootmem_init() > *arm64_numa_init()* > *numa_init()*

```
383 static int __init numa_init(int (*init_func)(void))
384 {
385     int ret;
386 
```

```
387     nodes_clear(numa_nodes_parsed);
388     nodes_clear(node_possible_map);
389     nodes_clear(node_online_map);
390
391     ret = numa_alloc_distance();
392     if (ret < 0)
393         return ret;
394
395     ret = init_func();
396     if (ret < 0)
397         goto out_free_distance;
398
399     if (nodes_empty(numa_nodes_parsed)) {
400         pr_info("No NUMA configuration found\n");
401         ret = -EINVAL;
402         goto out_free_distance;
403     }
404
405     ret = numa_register_nodes();
406     if (ret < 0)
407         goto out_free_distance;
408
409     setup_node_to_cpumask_map();
410
411     return 0;
412 out_free_distance:
413     numa_free_distance();
414     return ret;
415 }
```

- 387-389 清空 NUMA 解析的节点的临时位图、硬件拓扑存在的节点的位图、当前可用的节点的位图。
- 391-393 分配用于存储 NUMA 节点距离信息矩阵的内存，并填入默认值。在紧接着 395 行，会利用传入的执行句柄通过 ACPI 或 dts 设备树解析出真正的距离值来覆盖默认值。

  以 dts 树为例，这些预设的距离信息通常是多核 SOC 厂商在硬件手册/NUMA 拓扑手册中官方给出的访问延迟决定的数值。

  在命令行通过*numactl –harware*可以看到系统里的 numa node 的距离矩阵，如下。不过因为我的 PC 是单节点的内存，所以只看到一个节点。

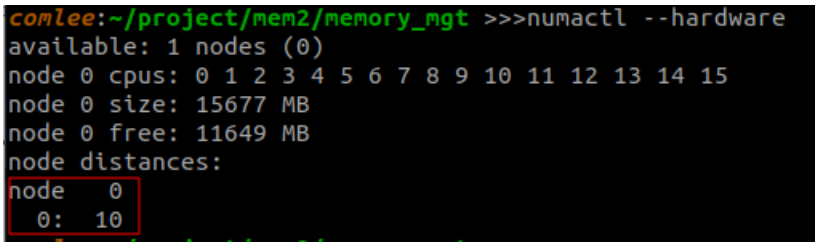


图 14–1: numa node 信息示例

- 395-397 执行解析，如果有 NUMA 节点就在节点的临时位图里对应位置 1。没有解析出节点则跳转，释放距离信息矩阵申请的内存，退出。

  以传入的执行函数 *of_numa_init()* 为例，在执行中通过解析 dts 数得到 numa node 信息，并以此填充二维距离矩阵。

- 399-403 本次 NUMA 拓扑解析没有识别到任何有效 NUMA 节点，出错返回。

- 405-407 numa_register_nodes() 根据 init_func() 解析出来、存放在临时位图 numa_nodes_parsed 中的 NUMA 节点信息，把节点注册到内核全局标准节点管理位图。完成拓扑信息从“临时解析缓存”转入“内核正式管理结构”。

14.1.5 numa_register_nodes()

bootmem_init() > *arm64_numa_init()* > *numa_init()* > *numa_register_nodes()*

```
351 static int __init numa_register_nodes(void)
352 {
353     int nid;
354     struct memblock_region *mblk;
355 
356     /* Check that valid nid is set to memblks */
357     for_each_mem_region(mblk) {
358         int mblk_nid = memblock_get_region_node(mblk);
359 
360         if (mblk_nid == NUMA_NO_NODE || mblk_nid >= MAX_NUMNODES) {
361             pr_warn("Warning: invalid memblk node %d [mem %#010Lx-%#010Lx]\n",
362                 mblk_nid, mblk->base,
363                 mblk->base + mblk->size - 1);
364             return -EINVAL;
365         }
366     }
367 
368     /* Finally register nodes. */
369     for_each_node_mask(nid, numa_nodes_parsed) {
370         unsigned long start_pfn, end_pfn;
371 
372         get_pfn_range_for_nid(nid, &start_pfn, &end_pfn);
373         setup_node_data(nid, start_pfn, end_pfn);
374         node_set_online(nid);
375     }
376 
377     /* Setup online nodes to actual nodes*/
378     node_possible_map = numa_nodes_parsed;
379 
380     return 0;
381 }
```

- 357-366 遍历每个 memblk 内存块，检查并确认每个 memblk 都有归属某个 numa node。也就是分配了 numa 节点 id。

  360-364 如果 memblk 所在 numa id 无效，则打印错误并返回。

- 369-372 遍历迭代从 dts 里解析出的 node id.

  获取当前迭代的目标 node id 对应的节点范围的起始和结束物理页帧。注意最后得到的值是页对齐后的。

- 373 为每个目标 node 初始化一个节点描述符 pglist_data。

- 374 将目标节点设为” 在线”。

- 378 将在线节点赋值给当前真实节点位图。

14.1.6 numa_alloc_distance()

bootmem_init() → arm64_numa_init() → numa_init() → numa_alloc_distance()

```
276 static int __init numa_alloc_distance(void)
277 {
278     size_t size;
279     u64 phys;
280     int i, j;
281
282     size = nr_node_ids * nr_node_ids * sizeof(numa_distance[0]);
283     phys = memblock_find_in_range(0, PFN_PHYS(max_pfn),
284                                   size, PAGE_SIZE);
285     if (WARN_ON(!phys))
286         return -ENOMEM;
287
288     memblock_reserve(phys, size);
289
290     numa_distance = __va(phys);
291     numa_distance_cnt = nr_node_ids;
292
293     /* fill with the default distances */
294     for (i = 0; i < numa_distance_cnt; i++)
295         for (j = 0; j < numa_distance_cnt; j++)
296             numa_distance[i * numa_distance_cnt + j] = i == j ?
297                 LOCAL_DISTANCE : REMOTE_DISTANCE;
298
299     pr_debug("Initialized distance table, cnt=%d\n", numa_distance_cnt);
300
301     return 0;
302 }
```

- 282 计算 numa 节点距离矩阵的存储需要多少字节。

  假设有 N 个内存节点，每亮点之间都要记录距离，包括自身到自身。所以:

  $$\text{整个矩阵占用的内存} = N \times N \times \text{单条距离数据的 size}$$

- 283-286 寻找可以容纳整个矩阵数据的一块内存，返回物理首地址。找不到则出错返回。
- 290 将找到的存储距离矩阵信息的物理首地址转换成内核线性地址。
- 294-297 给距离矩阵填充默认值，如下图（示例中有 4 个 numa 节点），自身到自身的距离是宏值:10(*LOCAL_DISTANCE*)，否则填充:20(*REMOTE_DISTANECE*)。

二维 *numa* 节点距离矩阵 $distance[nid_{src}][nid_{dst}]$

	node0	node1	node2	node3
node0	L	R	R	R
node1	R	L	R	R
node2	R	R	L	R
node3	R	R	R	L

说明:
1. L 表示宏 LOCAL_DISTANCE, 值为 10;R 表示 REMOTE_DISTANECE, 值为 20
2. 第 1 个黄色框表示 node 0 和 node 0 的距离；第 2 个黄色框表示 node 0 和 node 1 的距离，以此类推...

14.2 CMA 内存

有几种不同方式预留 CMA, 它们的用途也有所不同:

- **全局通用 *CMA***, 通过内核启动参数 *cma=* 来指定需要预留多大字节的连续物理内存作为 CMA 区域。通过这种方式内核会在启动早期从总内存中画出部分内存用于通用 CMA。
- **巨页专用 *CMA***, *hugetlb_cma=* 用于预留内存给巨页。用于运行时动态分配巨页。但是闲置时这块内存可给页缓存、匿名页等正常使用。
- ***per-node* 的 *DMA* 专用 *CMA***, *cma_pernuma=xxM*给每个在线 NUMA 节点单独创建 *xxM* 大小的 CMA 内存。这块内存供 DMA 专用, 被 *dma_alloc_coherent()*优先取用。普通 alloc_page(),kmalloc() 不会用到这个区域。和全局 CMA 的区别在于仅服务本节点 DMA 设备, 不供给 hugetlb 等其他子系统。
- ***DTS* 内存池**, 在 dts 树文件中通过 *compatible = "shared-dma-pool"* 来标识是 DMA/CMA 内存池。如果 cma-reserved 节点不用 *shared-dma-pool* 标识, 系统会锁定预留, 但是得用户手动 map(ioremap() 等) 再使用, 不能直接用标准 DMA KPI 接口来申请这块区域的内存。

  shared-dma-pool 标识的内存既可以做成全局通用 CMA, 也可以做成私有的 CMA 或独占的 DMA 池。

  私有表示这块内存专属某个设备, 但是空闲时系统可以分配给 movable 页。而**独占**表示这块内存只能该设备使用, 系统不能碰。

  这两个属性靠 ***reusable*** 属性划分:

  - 有 reusable 属性, 那么表示可复用给 movable 页, 属于 CMA, 如下图:

```
/ {
        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

                linux_default_cma: cma@0x800000000 {
                        compatible = "shared-dma-pool";
                        reusable;           // 标记为CMA可回收内存
                        linux,cma-default; // 关键: 设为系统全局默认CMA
                        reg = <0x00 0x800000000 0x00 0x40000000>; // 1GB
                };
        };
};
```

  - 不带 reusable 属性, 搭配 *no-map*。那么这块内存不属于 CMA, 并且内核伙伴系统不管理, 是固定**独占**的 reserve 的 DMA 池。这种方式和直接在启动命令行传递 ***reserve=*** 的效果一致。

```
/ {
        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

                r5_firmware_dma: dma-static@0xa00000000 {
                        compatible = "shared-dma-pool";
                        no-map;  // 内核不建立线性映射
                        // 无 reusable 属性, 不是CMA
                        reg = <0x00 0xa00000000 0x00 0x8000000>; // 128MB
                };
        };
```

```
        //独占的DMA内存
        mcu_r5@20000000 {
                reg = <0x00 0x20000000 0x00 0x1000>;
                memory-region = <&r5_firmware_dma>;
        };
};
```

下面的 dts 配置表示目标 CMA 仅被 memory-region 引用的设备私有，是独立的私有 CMA 池。但是空闲时系统可以给 movable 页使用。

```
/ {
        reserved-memory {
                #address-cells = <2>;
                #size-cells = <2>;
                ranges;

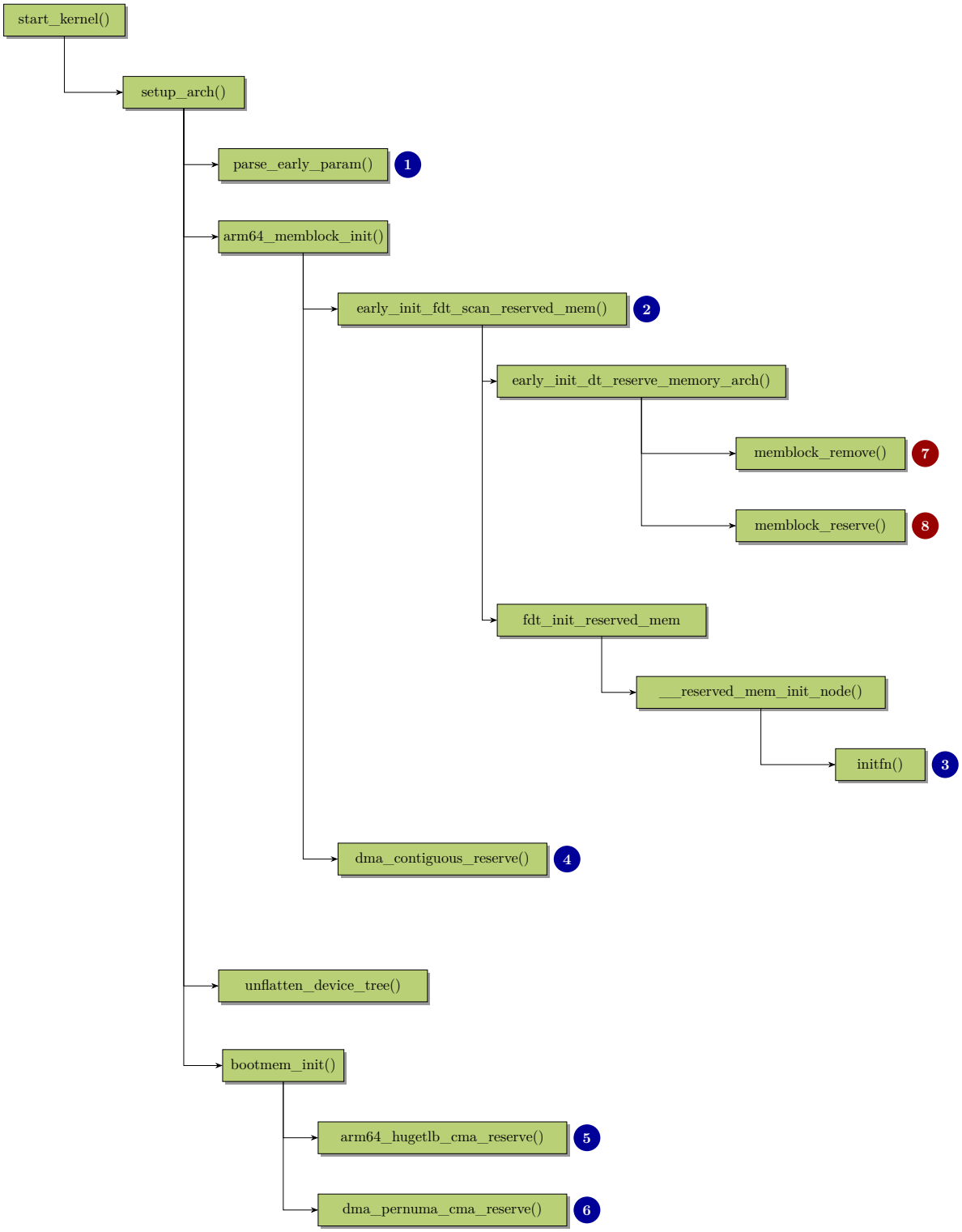
                video_private_cma: video-cma@0x900000000 {
                        compatible = "shared-dma-pool";
                        reusable; // 属于CMA内存池，可回收复用
                        reg = <0x00 0x900000000 0x00 0x20000000>; // 512MB
                };
        };

        // 多媒体视频设备，绑定私有CMA池
        video_decoder@10000000 {
                reg = <0x00 0x10000000 0x00 0x1000>;
                memory-region = <&video_private_cma>; // 绑定私有CMA
        };
};
```

下表是 dts 树中关于预留内存的配置和预留内存的性质的总结:

dts 配置	身份	闲置时内核可用	绑定对象
shared-dma-pool+ reusable+ linux,cma-default	全局默认 *CMA*	可	通用
shared-dma-pool+ reusable+ memory-region 绑设备	设备私有 *CMA*	可	指定 *dev*
*shared-dma-pool+ no-map (*无 *reusable)*	*DMA coherent* 缓存池	否	指定 *dev*
无 *compatible+ no-map*	纯 *reserved*	否	无

下面这幅图展示了启动时从命令行或 dts 树解析预留 CMA 内存的参数，并预留 CMA 区域的函数调用拓扑图:



说明:

- 1. ***parse_early_param()***遍历内核启动命令行的参数，并执行所有由 *early_param()* 宏注册的早期启动参数的处理函数，来解析出启动参数赋值给各自对应全局变量。

  如下图，早期启动参数 *cma*、*cma_pernuma* 分别注册的回调处理函数如下:

```
static int __init early_cma(char *p)
{
    if (!p) {
        pr_err("Config string not provided\n");
```

```
        return -EINVAL;
    }

    size_cmdline = memparse(p, &p);
    if (*p != '@')
        return 0;
    base_cmdline = memparse(p + 1, &p);
    if (*p != '-') {
        limit_cmdline = base_cmdline + size_cmdline;
        return 0;
    }
    limit_cmdline = memparse(p + 1, &p);

    return 0;
}
early_param("cma", early_cma);

/*********************************************/
static int __init early_cma_pernuma(char *p)
{
    pernuma_size_bytes = memparse(p, &p);
    return 0;
}
early_param("cma_pernuma", early_cma_pernuma);
```

- 2. ***early_init_fdt_scan_reserved_mem()*** 在启动早期，直接解析原始 Flat DTB(未展开的设备树二进制 blob)，扫描*/reserved-memory* 节点，调用预留内存对应的回调函数，进行内存预留。

  此时还没有执行 unflatten_device_tree()，内核还没有构建 struct device_node 树。它依靠底层 fdt_xxx 系列函数直接解析二进制 DTB。

- 3. ***initfn()*** 表示 dtb 中通过*compatible = "shared-dma-pool"*属性标识的，需要预留的内存对应的初始化函数。

  主要是 rmem_cma_setup() 和 rmem_dma_setup()(详见后面解析)。

  rmem_cma_setup() 中会判断是否有通过启动参数 *cma=* 传递全局默认 CMA 大小，如有，则忽略 dtb 中的全局默认 CMA 的预留设置。若否，但是 dtb 中有节点"linux,cma-default"，则按 dtb 中的配置来预留全局默认 CMA。

  rmem_dma_setup() 用于注册设备独占的 DMA 一致性内存池，不是 CMA。

- 4. ***dma_contiguous_reserve()*** 创建全局默认 CMA 区，供没有私有内存池的 DMA 设备分配连续内存。
  - 如果启动命令行没有通过 *cma=* 传递全局默认 CMA 大小，也没有通过内核配置项配置 CMA 大小，则跳过预留全局默认 CMA 区。
  - 如果启动命令行通过 *cma=* 传递了全局默认 CMA 大小，则按此预留全局默认 CMA 区。
  - 如果通过内核配置项传递了全局默认 CMA 大小，并且 dtb 中没有"linux,cma-default" 节点的配置，则按内核配置项预留全局默认 CMA 区。
  - 如果通过内核配置项传递了全局默认 CMA 大小，但 dtb 中有"linux,cma-default" 节点的配置，说明第 3 步已经预留了全局默认 CMA 区。则跳过预留全局默认 CMA 区。

- 5. ***arm64_hugetlb_cma_reserve()*** 专门处理 ***hugetlb_cma=XXM*** 启动参数，预留一块独立 CMA 区域，专供巨页 (HugeTLB) 分配使用。这块 CMA 池和普通 DMA/CMA、DTS shared-dma-pool、全局 CMA 互相隔离。

- 6. ***dma_pernuma_cma_reserve()*** 为每个在线 NUMA 节点分别创建独立的 DMA 专用 CMA 区域 (per-node CMA), 由启动参数 *cma_pernuma=XXM* 控制，未配置则直接返回，不执行任何预留的动作。

- 7. ***memblock_remove()*** 从 memblock 区域链表中，彻底删除一段物理内存，不再认为这段物理地址存在可用内存。之后内核完全无视该段物理地址，不会建立任何线性映射，驱动必须 ioremap() 才能使用。

  这段内存对应的就是 dts 树里没有*compatible = "shared-dma-pool"*属性，但是有*no-map*属性的纯 reserve 的内存区域。

  NOTE

  在启动命令行通过 *reserve=* 传递参数进行所谓的预留，和此处的预留不是一个概念。

  reserve= 传递的参数的回调处理函数是 *reserve_setup()*, 它是通过宏 *__setup("reserve=", reserve_setup)* 来进行参数和回调挂钩的。它的回调函数的调用比 *early_param("xxx",foo)* 注册的回调调用要晚得多，此时基本的内存管理子系统已经初始化完成。

  reserve= 传递的参数的回调处理函数操作的是内核资源树，具体是 *ioport_resource* (*I/O* 端口空间) 或 *iomem_resource* (*I/O* 内存空间)。这种方式是防止冲突而进行的逻辑占用而非物理预留或隔离，它告诉内核的资源管理器：这段地址范围已经被占用了，其他驱动程序不要通过 *request_region* 或 *request_mem_region* 来申请这段范围。而 *dts* 树中的 *no-map* 是为了物理内存底层预留。

- 8. ***memblock_reserve()*** 调用 memblock_add_range() 函数把内存区域加入 memblock.reserved，内存还留在 memory 链表，只是标记预留。

函数 ___reserved_mem_init_node() 遍历 dts 中各个预留的内存节点，执行各个内存区域专属的初始化回调函数。

early_init_fdt_scan_reserved_mem() > *fdt_init_reserved_mem()* > *__reserved_mem_init_node()*

```
170 static int __init __reserved_mem_init_node(struct reserved_mem *rmem)
171 {
172     extern const struct of_device_id __reservedmem_of_table[];
173     const struct of_device_id *i;
174     int ret = -ENOENT;
175
176     for (i = __reservedmem_of_table; i < &__rmem_of_table_sentinel; i++) {
177         reservedmem_of_init_fn initfn = i->data;
178         const char *compat = i->compatible;
179
180         if (!of_flat_dt_is_compatible(rmem->fdt_node, compat))
181             continue;
182
183         ret = initfn(rmem);
184         if (ret == 0) {
185             pr_info("initialized node %s, compatible id %s\n",
186                 rmem->name, compat);
187             break;
188         }
189     }
190     return ret;
191 }
```

- 176 循环遍历存放在段 (section)___reservedmem_of_table 内的所有 struct of_device_id 结构体数组成员。___reservedmem_of_table 是 section 的起始地址，也是第一个元素的地址。内核链接时所

有被 ___section("___reservedmem_of_table") 修饰的结构体被放在了同一个 section(段)。

在 5.10 内核，通用的放在该 section 的 struct of_device_id 结构体只有两个，名称分别是 *___of_table_cma* 和 *___of_table_dma*。分别通过如下宏来定义:

```
RESERVEDMEM_OF_DECLARE(cma, "shared-dma-pool", rmem_cma_setup);
RESERCEDMEM_OF_DECLARE(dma, "shared-dma-pool", rmem_dma_setup);
```

宏原型如下:

```
#define RESERVEDMEM_OF_DECLARE(name, compat, init)          \
    _OF_DECLARE(reservedmem, name, compat, init, reservedmem_of_init_fn)

#define _OF_DECLARE(table, name, compat, fn, fn_type)           \
    static const struct of_device_id __of_table_##name      \
        __used __section("__" #table "_of_table")       \
         = { .compatible = compat,              \
             .data = (fn == (fn_type)NULL) ? fn : fn  }
```

展开后的结构体如下:

```
    static const struct of_device_id __of_table_cma
    __used __section("__reservedmem_of_table") __aligned(__alignof__(struct
        of_device_id))
    = { .compatible = "shared-dma-pool",
        .data = rmem_cma_setup
     }
```

这里的 ___used 等价于 ___attribute___((used)), 用于告诉编译器不要把这段代码优化删除掉。因为这里的结构体在 c 语言层面没有直接调用，是靠链接脚本放到指定的 section, 运行时靠指针来引用的。不加 ___used，编译器会判断无人使用，直接优化丢弃。

___alignof___(type) 是获取 type 类型的对齐要求。___aligned(___alignof___(type)) 表示按 type 类型的对齐要求来对齐目标结构体。

更常见的写法是:

```
    static const struct of_device_id __of_table_cma = {
        .compatible = "shared-dma-pool",
        .data = rmem_cma_setup
     }
__used __section("__reservedmem_of_table") __aligned(__alignof__(struct of_device_id
    ))
```

- 180-181 将 dts 中预留的内存的节点的 compatible 和 ___reservedmem_of_table 段里结构体的 compatible 比较，如果不匹配，则 continue 继续。
- 183 如果 dts 里预留内存节点找到了 table 中匹配的结构体，则调用初始化回调函数。

  根据前面的描述可知，对于通用 CMA 和私有 CMA/DMA 分别有初始化函数 rmem_cma_setup() 和 rmem_dma_setup()。但是这两种的 compatible 都是"*shared-dma-pool*"，因此此处对于这些 dts 节点，会依次调用两次 initfn()，分别对应 rmem_cma_setup()、rmem_dma_setup()。

  而在这两个函数内部会根据属性来区分是否继续执行还是及早退出。

14.2.1 预留通用 CMA

14.2.1.1 通过 dts 节点预留通用 CMA

如果在内核启动时没有通过 ***cma=*** 传递参数预留通用 CMA。那么会辗转调用到 rmem_cma_setup()、rmem_dma_setup()。

arm64_memblock_init() ··· *__reserved_mem_init_node()* *initfn()* *rmem_cma_setup()*

```
400 static int __init rmem_cma_setup(struct reserved_mem *rmem)
401 {
402     phys_addr_t align = PAGE_SIZE << max(MAX_ORDER - 1, pageblock_order);
403     phys_addr_t mask = align - 1;
404     unsigned long node = rmem->fdt_node;
405     bool default_cma = of_get_flat_dt_prop(node, "linux,cma-default", NULL);
406     struct cma *cma;
407     int err;
408
409     if (size_cmdline != -1 && default_cma) {
410         pr_info("Reserved memory: bypass %s node, using cmdline CMA params instead\n",
411             rmem->name);
412         return -EBUSY;
413     }
414
415     if (!of_get_flat_dt_prop(node, "reusable", NULL) ||
416         of_get_flat_dt_prop(node, "no-map", NULL))
417         return -EINVAL;
418
419     if ((rmem->base & mask) || (rmem->size & mask)) {
420         pr_err("Reserved memory: incorrect alignment of CMA region\n");
421         return -EINVAL;
422     }
423
424     err = cma_init_reserved_mem(rmem->base, rmem->size, 0, rmem->name, &cma);
425     if (err) {
426         pr_err("Reserved memory: unable to setup CMA region\n");
427         return err;
428     }
429     /* Architecture specific contiguous memory fixup. */
430     dma_contiguous_early_fixup(rmem->base, rmem->size);
431
432     if (default_cma)
433         dma_contiguous_default_area = cma;
434
435     rmem->ops = &rmem_cma_ops;
436     rmem->priv = cma;
437
438     pr_info("Reserved memory: created CMA memory pool at %pa, size %ld MiB\n",
439         &rmem->base, (unsigned long)rmem->size / SZ_1M);
440
441     return 0;
442 }
```

- 405 读取 dts 属性"***linux,cma-default***"。如果存在该属性，那么这块 CMA 池将可能作为系统默认全局 CMA。
- 409-412 全局默认 CMA 的方式只能二选一，启动命令行传递参数的方式将覆盖 dts 节点的方式。
- 415-417 条件满足任意一条直接返回。
  - 不存在 reusable 属性。也就是有 *shared-dma-pool* 属性—具有该属性才会调用到该函数—但不带 *reusable* 属性。那么是静态 DMA 预留内存，不是 CMA。

- 存在 no-map 属性。CMA 区域要求内核线性映射，不允许 no-map。

  满足上面条件的交给 rmem_dma_setup() 来预留静态 DMA 内存。

- 419-421 检查对齐，内存起始物理地址 base、内存大小 size 都必须是前面 align 的整数倍。
- 424-427 真正创建 CMA 区域的核心函数。(见后面的解析)
- 432-433 如果 DTS 节点带 ***linux,cma-default*** 属性，将此 cma 指针赋值给全局默认 CMA 指针。
- 435-436 给 reserved_mem 结构体挂载操作集与私有数据。

14.2.1.2 通过启动参数预留通用 CMA

通用 CMA 区域如果在初始化时传递了 ***cma=***，那么会调用 dma_contiguous_reserve() 来创建通用 CMA 内存。该函数名称里带有"dma" 字样并不表示这块 CMA 是 DMA 专用。而是由于早期 linux 只有这一块 CMA，主要用于 DMA coherent 连续内存分配。由于这一历史原因，所以名称中带有"dma" 字样。

arm64_memblock_init() > *dma_contiguous_reserve()*

```
167 void __init dma_contiguous_reserve(phys_addr_t limit)
168 {
169     phys_addr_t selected_size = 0;
170     phys_addr_t selected_base = 0;
171     phys_addr_t selected_limit = limit;
172     bool fixed = false;
173
174     pr_debug("%s(limit %08lx)\n", __func__, (unsigned long)limit);
175
176     if (size_cmdline != -1) {
177         selected_size = size_cmdline;
178         selected_base = base_cmdline;
179         selected_limit = min_not_zero(limit_cmdline, limit);
180         if (base_cmdline + size_cmdline == limit_cmdline)
181             fixed = true;
182     } else {
183 #ifdef CONFIG_CMA_SIZE_SEL_MBYTES
184         selected_size = size_bytes;
185 #elif defined(CONFIG_CMA_SIZE_SEL_PERCENTAGE)
186         selected_size = cma_early_percent_memory();
187 #elif defined(CONFIG_CMA_SIZE_SEL_MIN)
188         selected_size = min(size_bytes, cma_early_percent_memory());
189 #elif defined(CONFIG_CMA_SIZE_SEL_MAX)
190         selected_size = max(size_bytes, cma_early_percent_memory());
191 #endif
192     }
193
194     if (selected_size && !dma_contiguous_default_area) {
195         pr_debug("%s: reserving %ld MiB for global area\n", __func__,
196              (unsigned long)selected_size / SZ_1M);
197
198         dma_contiguous_reserve_area(selected_size, selected_base,
199                         selected_limit,
200                         &dma_contiguous_default_area,
201                         fixed);
202     }
203 }
```

- 176-179 如果命令行有传递准备预留的 cma 区域的大小等参数，就按解析出的参数设置值。参数一般形式是: $cma = size@base, limit$。分别设置 cma 区域的大小、起始地址、上限地址。其中上限取系统的默认值和用户传入值的最小，但非 0 的值。
- 180-181 如果用户同时指定了大小，起始地址和上限地址，并且这三个参数满足:

  $$\text{起始地址} + \text{cma 的大小} = \text{地址上限}$$

  表示这个 cma 区域的地址已经锁定，不需要系统遍历内存找空闲块来给这个 cma 区域。
- 182-192 如果用户没有从启动参数传递 cmd= 的参数，那么有可能在内核配置项中进行了配置。下面这个代码就直接从配置参数中取值，或使用默认值来设置 cma 区域的大小。
  - 183-184 直接取编译时配置选项 CONFIG_SIZE_SEL_MBYTES 的配置来配置大小。
  - 185-186 如果开启和配置了 CONFIG_SIZE_SEL_PERCENTAGE，那么按整个系统物理内存的百分比 ($CONFIG_SIZE_SEL_PERCENTAGE\%$，即 $\frac{CONFIG_SIZE_SEL_PERCENTAGE}{100}$) 来设置 cma 大小。
  - 187-191 在开启了 CONFIG_SIZE_SEL_MIN 的场景下，取上面两者的最小值作为大小。同理，取最大值作为 cma 区域的大小。
- 194-202 这里主要用于判断是否需要创建，并且要防止重复创建全局默认 CMA 区域。如果 dma_contiguous_default_area 不为 NULL，表示全局 CMA 区域已经存在，此时就跳过创建的动作。直接返回。

  全局默认 CMA 区域只能全局唯一，但是系统中独立的 CMA 区可以创建多个。

arm64_memblock_init() › dma_contiguous_reserve() › dma_contiguous_reserve_area()

```
227 int __init dma_contiguous_reserve_area(phys_addr_t size, phys_addr_t base,
228                                         phys_addr_t limit, struct cma **res_cma,
229                                         bool fixed)
230 {
231     int ret;
232
233     ret = cma_declare_contiguous(base, size, limit, 0, 0, fixed,
234                                  "reserved", res_cma);
235     if (ret)
236         return ret;
237
238     /* Architecture specific contiguous memory fixup. */
239     dma_contiguous_early_fixup(cma_get_base(*res_cma),
240                                cma_get_size(*res_cma));
241
242     return 0;
243 }
```

该函数通过启动早期的内存组件完成内存预留。

cma_declare_contiguous() 主要是 cma_declare_contiguous_nid() 的包裹函数。

arm64_memblock_init() › ... › ... › cma_declare_contiguous() › cma_declare_contiguous_nid()

```
236 int __init cma_declare_contiguous_nid(phys_addr_t base,
237                                        phys_addr_t size, phys_addr_t limit,
238                                        phys_addr_t alignment, unsigned int order_per_bit,
```

```
239             bool fixed, const char *name, struct cma **res_cma,
240             int nid)
241 {
242     phys_addr_t memblock_end = memblock_end_of_DRAM();
243     phys_addr_t highmem_start;
244     int ret = 0;
245 
246     /*
247      * We can't use __pa(high_memory) directly, since high_memory
248      * isn't a valid direct map VA, and DEBUG_VIRTUAL will (validly)
249      * complain. Find the boundary by adding one to the last valid
250      * address.
251      */
252     highmem_start = __pa(high_memory - 1) + 1;
253     pr_debug("%s(size %pa, base %pa, limit %pa alignment %pa)\n",
254         __func__, &size, &base, &limit, &alignment);
255 
256     if (cma_area_count == ARRAY_SIZE(cma_areas)) {
257         pr_err("Not enough slots for CMA reserved regions!\n");
258         return -ENOSPC;
259     }
260 
261     if (!size)
262         return -EINVAL;
263 
264     if (alignment && !is_power_of_2(alignment))
265         return -EINVAL;
266 
267     /*
268      * Sanitise input arguments.
269      * Pages both ends in CMA area could be merged into adjacent unmovable
270      * migratetype page by page allocator's buddy algorithm. In the case,
271      * you couldn't get a contiguous memory, which is not what we want.
272      */
273     alignment = max(alignment,  (phys_addr_t)PAGE_SIZE <<
274               max_t(unsigned long, MAX_ORDER - 1, pageblock_order));
275     if (fixed && base & (alignment - 1)) {
276         ret = -EINVAL;
277         pr_err("Region at %pa must be aligned to %pa bytes\n",
278             &base, &alignment);
279         goto err;
280     }
281     base = ALIGN(base, alignment);
282     size = ALIGN(size, alignment);
283     limit &= ~(alignment - 1);
284 
285     if (!base)
286         fixed = false;
287 
288     /* size should be aligned with order_per_bit */
289     if (!IS_ALIGNED(size >> PAGE_SHIFT, 1 << order_per_bit))
290         return -EINVAL;
291 
292     /*
293      * If allocating at a fixed base the request region must not cross the
294      * low/high memory boundary.
295      */
296     if (fixed && base < highmem_start && base + size > highmem_start) {
297         ret = -EINVAL;
298         pr_err("Region at %pa defined on low/high memory boundary (%pa)\n",
299             &base, &highmem_start);
300         goto err;
301     }
302 
```

```
303     /*
304      * If the limit is unspecified or above the memblock end, its effective
305      * value will be the memblock end. Set it explicitly to simplify further
306      * checks.
307      */
308     if (limit == 0 || limit > memblock_end)
309         limit = memblock_end;
310 
311     if (base + size > limit) {
312         ret = -EINVAL;
313         pr_err("Size (%pa) of region at %pa exceeds limit (%pa)\n",
314             &size, &base, &limit);
315         goto err;
316     }
317 
318     /* Reserve memory */
319     if (fixed) {
320         if (memblock_is_region_reserved(base, size) ||
321             memblock_reserve(base, size) < 0) {
322             ret = -EBUSY;
323             goto err;
324         }
325     } else {
326         phys_addr_t addr = 0;
327 
328         /*
329          * All pages in the reserved area must come from the same zone.
330          * If the requested region crosses the low/high memory boundary,
331          * try allocating from high memory first and fall back to low
332          * memory in case of failure.
333          */
334         if (base < highmem_start && limit > highmem_start) {
335             addr = memblock_alloc_range_nid(size, alignment,
336                     highmem_start, limit, nid, true);
337             limit = highmem_start;
338         }
339 
340        if (!addr) {
341             addr = memblock_alloc_range_nid(size, alignment, base,
342                     limit, nid, true);
343             if (!addr) {
344                 ret = -ENOMEM;
345                 goto err;
346             }
347         }
348 
349         /*
350          * kmemleak scans/reads tracked objects for pointers to other
351          * objects but this address isn't mapped and accessible
352          */
353         kmemleak_ignore_phys(addr);
354         base = addr;
355     }
356 
357     ret = cma_init_reserved_mem(base, size, order_per_bit, name, res_cma);
358     if (ret)
359         goto free_mem;
360 
361     pr_info("Reserved %ld MiB at %pa\n", (unsigned long)size / SZ_1M,
362         &base);
363     return 0;
364 
365 free_mem:
366     memblock_free(base, size);
```

```
367 err:
368     pr_err("Failed to reserve %ld MiB\n", (unsigned long)size / SZ_1M);
369     return ret;
370 }
```

- 242 返回系统可用内存的最大物理地址。
- 252 获取内核内存直接映射区 (线性映射区) 上限对应的虚拟地址。内核线性映射区的虚拟地址的范围是 $\in [PAGE_OFFSET, highmem)$,highmem 是边界，不是有效的线性映射地址，无对应的物理内存。因此 -1 得到有效线性地址再进行转换。
- 256-259 检查 CMA 内存池数量是否已达到限制数量。

  cma_area_count 是系统中已创建的 CMA 内存池数目。cma_areas 是全局 cma 池的管理结构体数组，创建的 CMA 池数量不能超过该结构体数组的成员数。
- 261-262 参数合法性检测。传入要预留大小若为 0，则出错返回。
- 264-265 物理内存对齐要求按 2 的幂对齐。
- 273-280 提升对齐粒度，更新用户传入的过小 alignment。

  $PAGE_SIZE << max_t(unsigned\ long,\ MAX_ORDER - 1, pageblock_order)$ 得到的是系统中最大的连续页块对应的页阶。MAX_ORDER-1 是伙伴系统支持的最大分配页阶，pageblock_order 是一个页块 (pageblock, 此处是专有名词) 的页阶数。取两者中的最大值，代表系统中最大连续页块对应的页阶 (2^{max} 个页)。最后

$$\begin{aligned} PAGE_SIZE \times 2^{max} &= \text{字节数/页} \times \text{页数} \\ &= \text{一页的字节数} \times 2^{max} \\ &= \text{系统中最大连续页块对应的字节数} \end{aligned}$$

- 281-283 CMA 区域起始地址的对齐必须按照上面算出的 alignment。

  如果 CMA 区域对齐粒度太小，那么伙伴系统有可能会将 CMA 区域的首尾页和 *unmovable* 页合并，造成无法取出满足要求的 CMA 区域的连续内存。

  首先 CMA 区域的首尾页和 *movable* 页合并的话，需要大块 CMA 内存时, 系统可以将 *movable* 页迁移到别处，归还完整连续的 CMA 页。但 *unmovable* 页不可迁移，一旦和 CMA 页合并，就会牢牢卡死在 CMA 边界无法挪走，形成永久碎片。

  伙伴系统进行跨 pageblock 页合并时，要求两个 pageblock 的迁移类型一致。所以如果相邻两个 pageblock 的迁移类型不同是无法合并页的。因此首尾页不按 pageblock 对齐，那么有可能在另一个 pageblock 里, 从而被合并。
- 285-286 如果没有指定起始地址，那么取消固定地址模式，改为自动搜寻内存。
- 296-301 不允许 CMA 跨高低内存的边界，会破坏伙伴系统 zone 管理。
- 308-309 修正上限，若上限超过 DRAM 的末尾，则修正为末尾值。
- 311-316 CMA 区域的尾部不能超出上限，若超出出错返回。
- 319-324 在固定地址模式下，检查指定内存区是否被其他模块预留，若没有则锁定这块区域。

- 326 进入自动搜寻合适内存区域模式。
- 334-338 CMA 区域必须落在同一个 zone 区。优先从高端内存分配。
- 340-347 分配失败再从低端分配。
- 357-359 完成 cma 元数据的初始化，初始化空闲页位图。

arm64_memblock_init() ··· *cma_declare_contiguous_nid()* *cma_init_reserved_mem()*

```
167 int __init cma_init_reserved_mem(phys_addr_t base, phys_addr_t size,
168                                   unsigned int order_per_bit,
169                                   const char *name,
170                                   struct cma **res_cma)
171 {
172     struct cma *cma;
173     phys_addr_t alignment;
174 
175     /* Sanity checks */
176     if (cma_area_count == ARRAY_SIZE(cma_areas)) {
177         pr_err("Not enough slots for CMA reserved regions!\n");
178         return -ENOSPC;
179     }
180 
181     if (!size || !memblock_is_region_reserved(base, size))
182         return -EINVAL;
183 
184     /* ensure minimal alignment required by mm core */
185     alignment = PAGE_SIZE <<
186             max_t(unsigned long, MAX_ORDER - 1, pageblock_order);
187 
188     /* alignment should be aligned with order_per_bit */
189     if (!IS_ALIGNED(alignment >> PAGE_SHIFT, 1 << order_per_bit))
190         return -EINVAL;
191 
192     if (ALIGN(base, alignment) != base || ALIGN(size, alignment) != size)
193         return -EINVAL;
194 
195     /*
196      * Each reserved area must be initialised later, when more kernel
197      * subsystems (like slab allocator) are available.
198      */
199     cma = &cma_areas[cma_area_count];
200 
201     if (name)
202         snprintf(cma->name, CMA_MAX_NAME, name);
203     else
204         snprintf(cma->name, CMA_MAX_NAME,  "cma%d\n", cma_area_count);
205 
206     cma->base_pfn = PFN_DOWN(base);
207     cma->count = size >> PAGE_SHIFT;
208     cma->order_per_bit = order_per_bit;
209     *res_cma = cma;
210     cma_area_count++;
211     totalcma_pages += (size / PAGE_SIZE);
212 
213     return 0;
214 }
```

在已经预留的大块内存的基础上，创建 CMA 管理元数据和空闲页位图。使这块内存具有 CMA 分配、回收能力。

- 176-179 cma 内存区域数量将超出 cma 管理数组能管理的上限，出错返回。
- 181-182 cma 预留大小为 0，或 cma 区域已经提前被其他模块预留，则出错返回。
- 192-193 如果区域大小和起始地址没有按 pageblock 对齐，则返回。
- 199 从 cma 全局管理数组分配一个空闲成员用于此 cma 区域的管理。
- 201-204 将自定义的 CMA 池名称赋值给 CMA 结构体的 name 成员。如果没有传入名称，则生成默认 CMA 池的名称，格式是 cmaX(X 代表序号)。
- 206 CMA 区物理地址前向对齐到页边界得到的页帧号。
- 207 赋值 CMA 区的总页数。
- 208 CMA 空闲位图中 1 bit 管理 $2^{order_per_bit}$ 个连续页。
- 210 CMA 区的全局计数 +1。
- 211 累加当前 CMA 区的页数到全局 CMA 页总数中。

  这个值可以在***/proc/meminfo*** 节点打印的信息的 *CmaTotal* 一栏显示。

14.2.2 创建设备私有的或默认全局的 DMA 池

如果解析设备树 reserved-memory 时，发现有 *compatible = "shared-dma-pool"* 和 *no-map* 属性，那么会创建一块静态预留 DMA 池，供设备 DMA 直接使用。但它不是 CMA，在空闲时不能分配给 movable 页，是独立预留的内存池。

arm64_memblock_init() … *__reserved_mem_init_node()* *initfn()* *rmem_dma_setup()*

```
352 static int __init rmem_dma_setup(struct reserved_mem *rmem)
353 {
354     unsigned long node = rmem->fdt_node;
355
356     if (of_get_flat_dt_prop(node, "reusable", NULL))
357         return -EINVAL;
358
359 #ifdef CONFIG_ARM
360     if (!of_get_flat_dt_prop(node, "no-map", NULL)) {
361         pr_err("Reserved memory: regions without no-map are not yet supported\n");
362         return -EINVAL;
363     }
364
365     if (of_get_flat_dt_prop(node, "linux,dma-default", NULL)) {
366         WARN(dma_reserved_default_memory,
367              "Reserved memory: region for default DMA coherent area is redefined\n");
368         dma_reserved_default_memory = rmem;
369     }
370 #endif
371
372     rmem->ops = &rmem_dma_ops;
373     pr_info("Reserved memory: created DMA memory pool at %pa, size %ld MiB\n",
374         &rmem->base, (unsigned long)rmem->size / SZ_1M);
375     return 0;
376 }
```

- 356-357 这里会创建或注册静态的设备独占的 DMA 池，不允许回收，存在 *reusable* 属性直接报错，终止该 reserved-memory 节点初始化。
- 360-363 这里是 ARM 平台强制校验: 必须携带 no-map 属性。

  no-map 意味着内核不创建线性虚拟地址映射。
- 365 判断是否设置 ***linux,dma-default*** 属性, 若有，标记这块内存作为系统默认预留的 DMA coherent 内存池。

  NOTE

  CMA 闲时系统可以用，需要 *DMA* 时腾出来给 *DMA*。如果 *CMA* 区域正在被文件缓存占用，分配大块 *DMA* 内存时需要回收页面，存在延时、甚至分配失败；

  专用 *DMA* 池内存系统启动时就直接划走，系统永远碰不到，分配延迟稳定，不会出现回收给 *DMA* 用的动作。
- 366 如果已经存在默认 DMA 池，打印警告。
- 367 赋值全局指针，更新默认 DMA 池。
- 372 挂载操作函数集。

14.2.3 预留 per-node DMA 专用的 CMA

per-node DMA 专用的 CMA 通过 dma_pernuma_cma_reserve() 来预留。

在内核配置项 *CONFIG_DMA_PERNUMA_CMA=y* 的情况下，dma_pernuma_cma_reserve() 才会真正编译进 img, 否则是个空函数。

start_kernel() > *setup_arch()* > *bootmem_init()* > *dma_pernuma_cma_reserve()*

```
130 #ifdef CONFIG_DMA_PERNUMA_CMA
131 void __init dma_pernuma_cma_reserve(void)
132 {
133     int nid;
134
135     if (!pernuma_size_bytes)
136         return;
137
138     for_each_online_node(nid) {
139         int ret;
140         char name[CMA_MAX_NAME];
141         struct cma **cma = &dma_contiguous_pernuma_area[nid];
142
143         snprintf(name, sizeof(name), "pernuma%d", nid);
144         ret = cma_declare_contiguous_nid(0, pernuma_size_bytes, 0, 0,
145                         0, false, name, cma, nid);
146         if (ret) {
147             pr_warn("%s: reservation failed: err %d, node %d", __func__,
148                 ret, nid);
149             continue;
150         }
151
152         pr_debug("%s: reserved %llu MiB on node %d\n", __func__,
153             (unsigned long long)pernuma_size_bytes / SZ_1M, nid);
```

```
154     }
155 }
156 #endif
```

- 135-136 如果没有配置 per-node DMA CMA(指定预留给 per-node DMA 专用的 CMA 区的字节大小为 0)，则返回。这里对变量 pernuma_size_bytes 的值进行了检查，但是这个值的原始来源是: 内核启动时通过命令行参数 ***cma_pernuma=***来指定的。在 5.10 内核中这是唯一来源。
- 138-141 遍历迭代在线 NUMA 节点的编号，在每次迭代中取出目标节点的 (DMA 专属)CMA 管理结构体的指针。dma_contiguous_pernuma_area[] 是一个静态全局数组，每个元素对应一个在线 NUMA 节点的 (DMA 专属) CMA 管理结构体指针。后续 DMA 分配时，可以直接通过 node 节点索引该数组找到目标节点专属 CMA。
- 144-145 这里进行实际的内存预留，调用的 cma_declare_contiguous_nid() 函数 (见前面解析) 和预留通用 CMA 时一致。

  需要注意的是，这里传递的第 1 个入参 (起始地址) 是 0，意味着需要系统自动找寻合适的内存区域作为 per-node 的 DMA 专用 CMA。

15

其它

15.1 xarrays(可扩展数组)

可扩展数组 xarray 可以存储两类数据：

- 指针 (必须按 4 字节对齐)；
- 能够容纳在指针内的非负数值 (即取值范围从 0 到 LONG_MAX)。

每个可扩展数组都由一个 struct xarray 对象 (obj) 表示 (内存管理中该 obj 的核心使用者是页缓存的 struct address_space->i_pages)。

```
struct xarray {
    spinlock_t  xa_lock;
/* private: The rest of the data structure is not to be used directly. */
    gfp_t       xa_flags;
    void __rcu *    xa_head;
};
```

该结构体本身占用空间极小，因此一般不建议单独分配内存，再通过指针在其他结构中引用它。通常是将其内嵌到其他数据结构中。

```
struct address_space {
    ... ...
    struct xarray i_pages;
    ... ...
}
```

如上所示，也就是 address_space 里直接包含一个 xarray，而不是指针。

struct xarray 结构包含一个自旋锁 (xa_lock)，用于处理无法仅通过 RCU 完成的操作；还包含标志位 (xa_flags)，用于设置 xarray 的相关标记；以及 xarray 树的根节点 (xa_head)。后两个字段是私有的，不允许在 xarray 代码外部直接使用。

一个新的 xarray 对象 (obj) 可以通过宏 ***XARRAY_INIT*** 来初始化。注意:一个 xarray 树只有一个 xarray 对象。

```
#define XARRAY_INIT(name, flags) {              \
    .xa_lock = __SPIN_LOCK_UNLOCKED(name.xa_lock),      \
```

```
    .xa_flags = flags,                  \
    .xa_head = NULL,                    \
}
```

如果数组中的所有项都为 NULL，那么 xa_head 是一个空指针；如果数组仅在索引 0 处包含元素，那么 xa_head 直接存储该元素项；如果数组中存在索引 0 之外的其他非空表项，那么 xa_head 指向一个 xa_node 节点。

BITS_PER_XA_VALUE 定义了 xarray 的项 (entry) 里整数值可以占多少位。

```
#define BITS_PER_XA_VALUE   (BITS_PER_LONG - 1)
```

xa_mk_value() 会把一个整数值编码成一个假指针，存放在指针 size(大小) 的槽位 (slot)。

- 如果值存在根节点 (index=0 元素), 那么会存在 xarray->xa_head 成员:

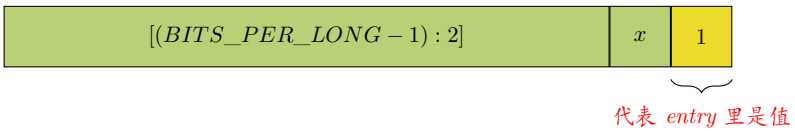
```
struct xarray {
        ... ...
        void __rcu *    xa_head;
};
```

- 如果值存入树节点，那么会放在 slot 槽里:

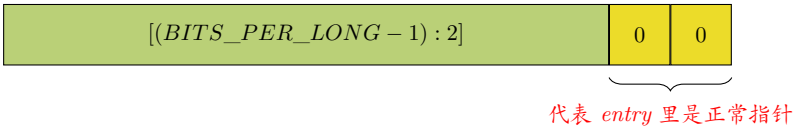
```
struct xa_node {
        ... ...
        void __rcu      *slots[XA_CHUNK_SIZE];
        ... ...
};
```

XA_CHUNK_SIZE 一般是 64.

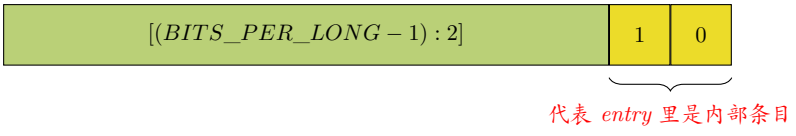
从宏 BITS_PER_XA_VALUE 的定义可以看出:xarray 中的整数值必须是不大于 LONG_MAX 的非负值—需要牺牲一位来标记这些数据为数值类型。这个位是最低位([0] 位)。



内存地址 4 字节对齐，意味着地址一定是 4 的整数倍，二进制最低两位固定为 00，XArray 用这两位做类型标记用户指针。低 2 位为 00,XArray 就识别为合法用户指针。



如果低两位是 10，则代表存放的内部项。包含：xa_node 指针、Sibling、Retry、Zero 等内部标记。



上面 entry 最低位和条目类型的关系总结在如下表格:

条目类型	最低两位	核心特征	使用者	判断函数
用户指针	*00*	*4* 字节对齐地址	业务代码	*NA*
内部条目	*10*	节点/状态标记	*XArray* 核心	*xa_is_internal()*
值条目 *(整数)*	*x1*	编码后的整数	业务代码	*xa_is_value()*

更进一步，内部项 (条目) 的不同值范围代表不同的内部项的意义:

最低两位 *[1:0]*	类型	原始值	编码后的值	含义
10	兄弟项	$0 \approx 62$	$(0 \approx 62 \ll 2) \mid 2$	表示多个索引共用同一个条目，数值用于标明当前条目相对目标条目所处的偏移位置。
10	重试项	*256*	$(256 \ll 2) \mid 2$	表示其他线程已持有 *xa_lock* 锁，且正在更新当前条目。为避免获取到临时失效的引用，需要重新执行本次操作
10	*Zero*	*257*	$(257 \ll 2) \mid 2$	表示该条目已被占用，但存储内容为空 *(等效空指针)*
10	*Error*	$-4094 \approx -2$	$(neg_val \ll 2) \mid 2$	*NA*
10	*xa_node*	指针地址	$(addr \ll 2) \mid 2$	*NA*

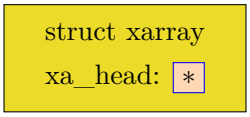
每个独立的节点包含 XA_CHUNK_SIZE 个槽位 (slots)，用于存储指针。对于除极小规模嵌入式系统之外的所有环境，该值为 2^6(即 64) 个指针。每个槽位中存放着条目 (entry)，其中部分条目可能指向更深层的节点。与动态数组类似，当 XArray 空间耗尽时，其容量会成倍增长。但与动态数组不同的是，XArray 通过新增 struct xa_node 节点来实现扩容。譬如一开始 xa_head 直接指向最底层的叶子节点，首次扩容时，会创建一个新节点，然后把 xa_head 的值改成指向这个新节点。xa_head 从" 指向旧节点" 变成了" 指向新父节点"。并将指向旧节点的指针放入该新节点的第一个槽位。也就是把原来的叶子节点放进新节点的 slot[0]。

这个父节点被插入到树的头部，其位移值 (shift) 会增加 XA_CHUNK_SIZE 对应的位数。位移值用于标识每个节点条目所覆盖的索引范围。

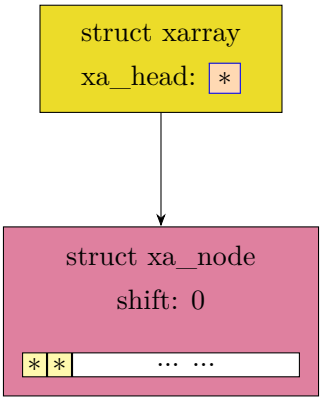
譬如，旧节点是 head 直接指向的叶子节点：shift=6，有 64 个 slot。第一次扩容，会新建一个节点，并把原旧节点放入新节点的第一个槽位，这样新节点就成了父节点。新节点也是 64 个 slot。新节点 (父节点) 一个槽位 (slot) 覆盖 $2^6 = 64$ 个索引 (范围)。所以新节点的 shift 是 6。

由上可知,shift 就是各层的一个槽位可以覆盖的索引数的 (以 2 为底的) 指数值。各层一个槽位可以覆盖的索引数是 2^{shift}。需要注意的是，每个叶子节点的槽位没有指向更深一层的节点，所以每个槽位 (slot) 的索引覆盖范围是 1(即,2^0), 因此叶子节点的 $shift = 0$。

下面是从 head 开始对 xarray 进行扩容的示意图:



挂接第一个节点, 粉红色矩形框表示新建 node(节点)(下同), 如下:



如果第一个子节点的 XA_CHUNK_SIZE 个 slot(槽位) 被填满，现在需要新增一个数据项。那么保留原有的 xa_node 结构体，新建一个 xa_node 作为父节点，在新节点的槽位中存入指向原来父节点的指针 (图中红色箭头)。如下,

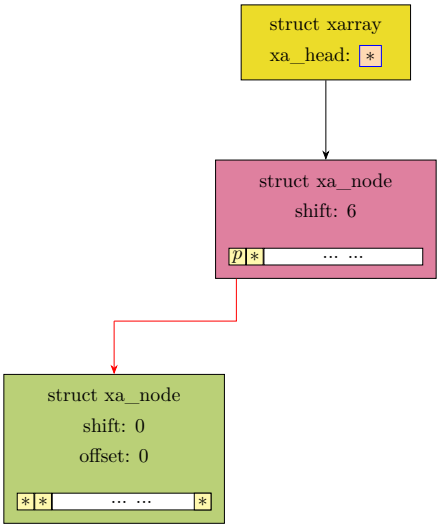


图 15–1: 新建节点 (shift=6) 的 slot[0] 槽填入指向原 node 的指针

之后新分配的子节点会存入顶层父节点数组的第二个槽位，这个节点的下标偏移为 1。通过父节点 shift 值 + 叶子节点内部 offset 偏移，就能定位数组中任意数据的存储位置。

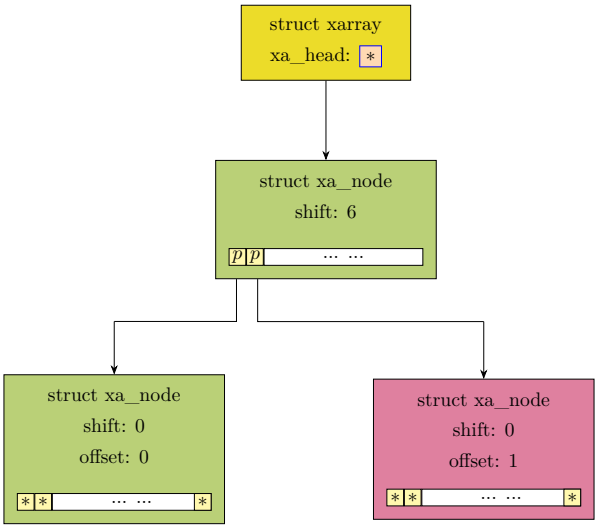


图 15–2: 新建的第二个叶节点填充到父节点 (shift=6) 的 slot[1] 槽

以此类推，后续新分配的节点依次存入该槽，直到第 64 个数据项。

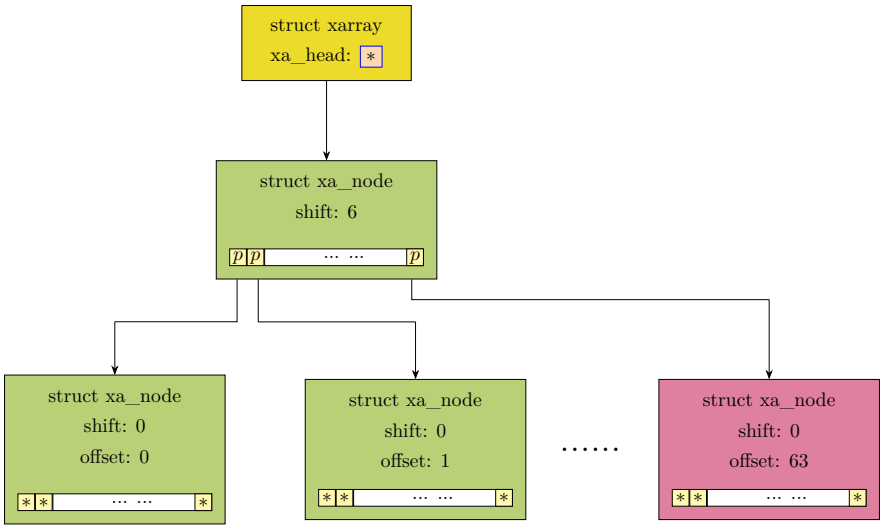
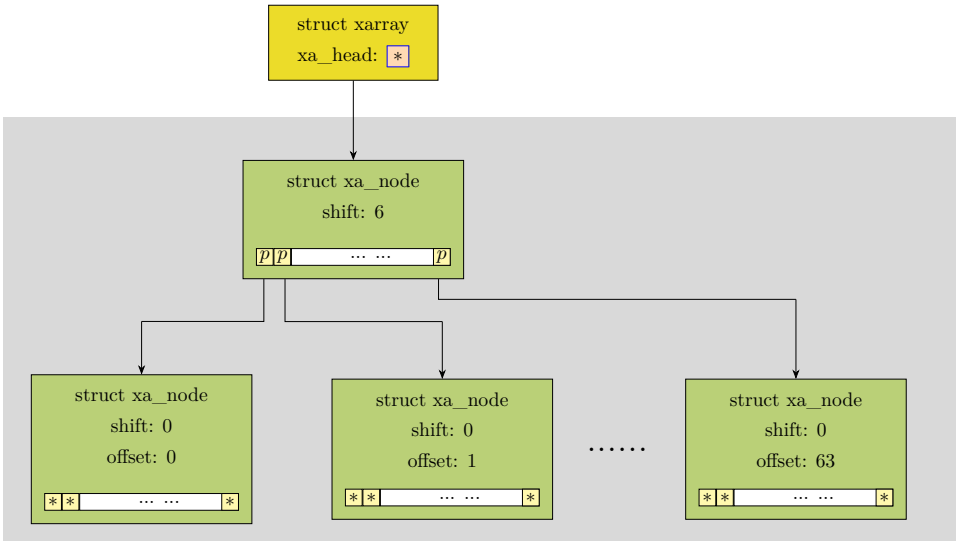


图 15–3: 子节点陆续填满父节点 (shift=6) 的 slot 槽

后续扩容触发条件是顶层 xa_node **节点**全部填满。现在是两层 node(xa_head 不是 node), 单个节点拥有 XA_CHUNK_SIZE(64) 个槽位，顶层节点的每个槽位都指向一个叶子节点，每个叶子节点又能存放 XA_CHUNK_SIZE 个数据。XA_CHUNK_SIZE=64，因此两层结构总共可存储 $64^2 = 4096$ 条数据；填满 4096 项后，XArray 需要再次向上扩容。

此时会将下图中灰色框内的部分挂接在新创建的顶层父节点上。



挂接后的示意图如下，新顶层父 node 的第一个 slot 内存有指向原父节点的指针 (图中红色箭头):

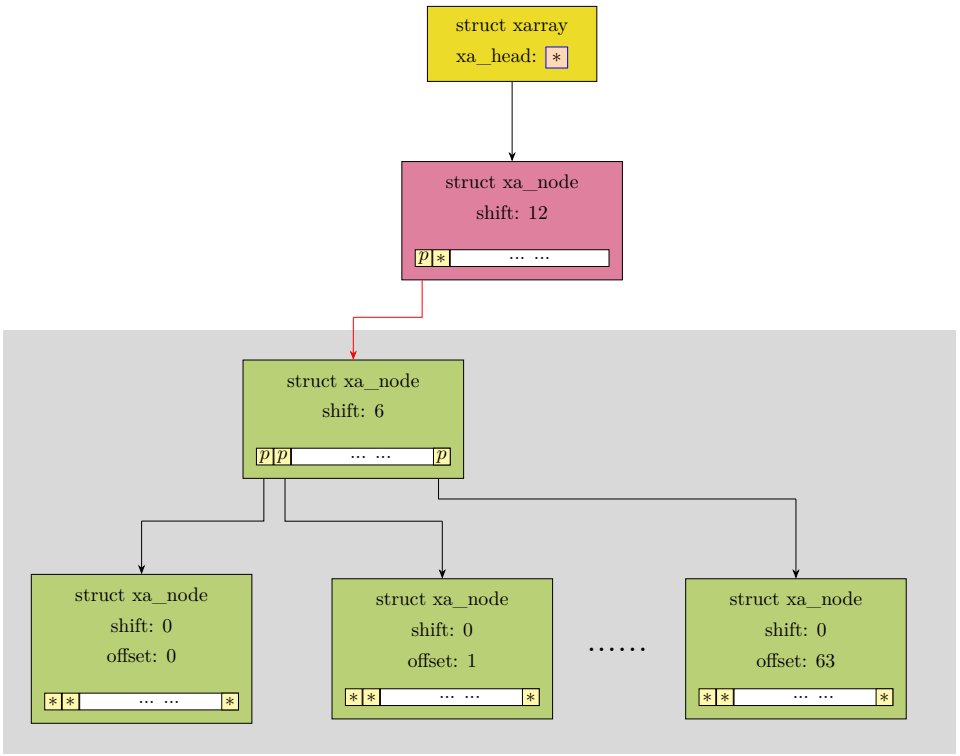


图 15–4: xarray 扩容，挂接到 shift=12 的新顶层父节点

当 shift=12 的层级的节点及其子节点的 slot 槽都填满后，若要新增节点就必须继续扩容，如下:

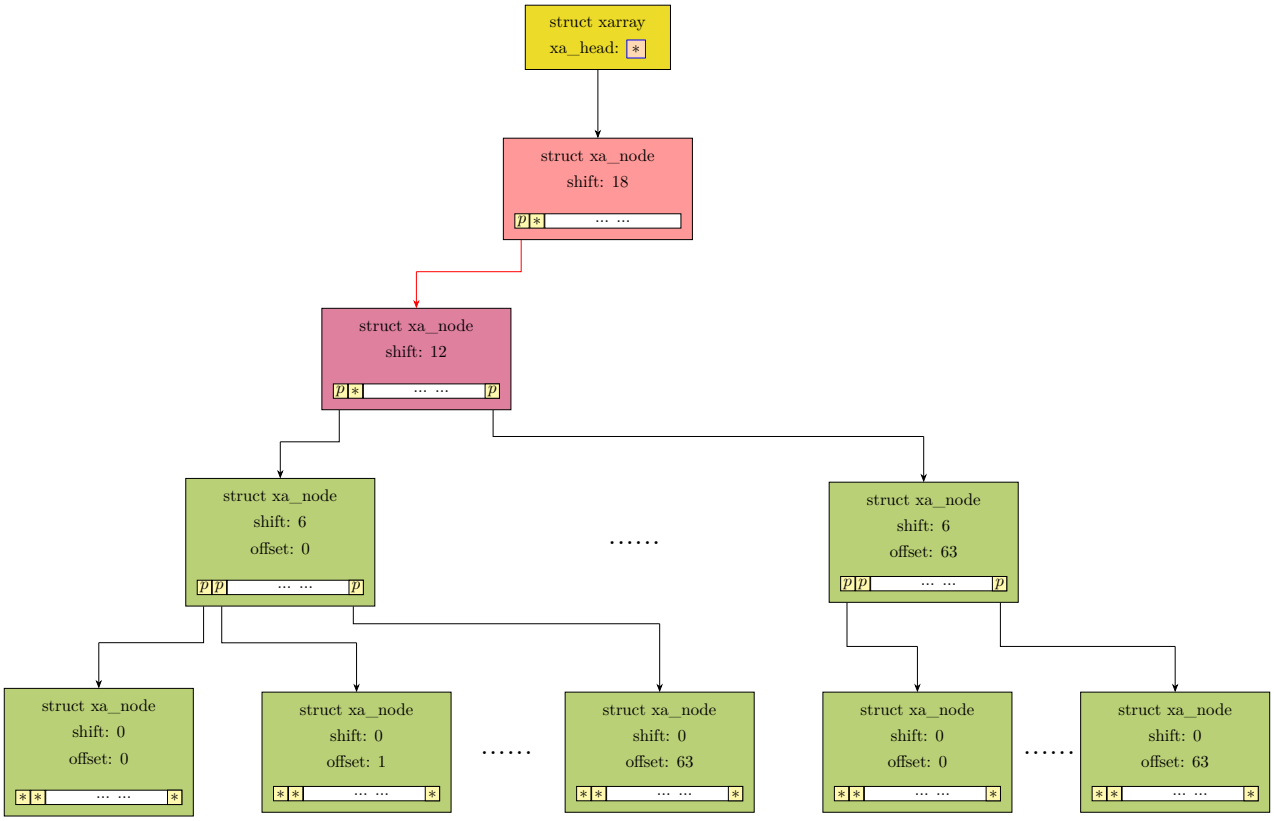


图 15–5: xarray 继续扩容，挂接到 shift=18 的新顶层父节点

和前面类似，新 (父) 节点 (shift=18) 的槽位中存入指向原来顶层父节点的指针，见上图红色线条。之后再扩容的话，图形原理和上面类似。

15.1.1 xas_store()

xas_store()

```
772 void *xas_store(struct xa_state *xas, void *entry)
773 {
774     struct xa_node *node;
775     void __rcu **slot = &xas->xa->xa_head;
776     unsigned int offset, max;
777     int count = 0;
778     int values = 0;
779     void *first, *next;
780     bool value = xa_is_value(entry);
781
782     if (entry) {
783         bool allow_root = !xa_is_node(entry) && !xa_is_zero(entry);
784         first = xas_create(xas, allow_root);
785     } else {
786         first = xas_load(xas);
787     }
788
789     if (xas_invalid(xas))
790         return first;
791     node = xas->xa_node;
792     if (node && (xas->xa_shift < node->shift))
793         xas->xa_sibs = 0;
```

```
794     if ((first == entry) && !xas->xa_sibs)
795         return first;
796 
797     next = first;
798     offset = xas->xa_offset;
799     max = xas->xa_offset + xas->xa_sibs;
800     if (node) {
801         slot = &node->slots[offset];
802         if (xas->xa_sibs)
803             xas_squash_marks(xas);
804     }
805     if (!entry)
806         xas_init_marks(xas);
807 
808     for (;;) {
809         /*
810          * Must clear the marks before setting the entry to NULL,
811          * otherwise xas_for_each_marked may find a NULL entry and
812          * stop early.  rcu_assign_pointer contains a release barrier
813          * so the mark clearing will appear to happen before the
814          * entry is set to NULL.
815          */
816         rcu_assign_pointer(*slot, entry);
817         if (xa_is_node(next) && (!node || node->shift))
818             xas_free_nodes(xas, xa_to_node(next));
819         if (!node)
820             break;
821         count += !next - !entry;
822         values += !xa_is_value(first) - !value;
823         if (entry) {
824             if (offset == max)
825                 break;
826             if (!xa_is_sibling(entry))
827                 entry = xa_mk_sibling(xas->xa_offset);
828         } else {
829             if (offset == XA_CHUNK_MASK)
830                 break;
831         }
832         next = xa_entry_locked(xas->xa, node, ++offset);
833         if (!xa_is_sibling(next)) {
834             if (!entry && (offset > max))
835                 break;
836             first = next;
837         }
838         slot++;
839     }
840 
841     update_node(xas, node, count, values);
842     return first;
843 }
844 EXPORT_SYMBOL_GPL(xas_store);
```

- 775 获取目标 xarray 的头结点成员变量 xa_head 的地址。
- 780 判断 entry 的内容是不是 “数值”。
- 783 判断是否允许 entry 直接存在根节点。条件是：entry 不是内部节点类型，也不是 zero 类型。

  entry 如果是内部节点，则最低两位 =0b10(XA_INTERNAL)，且地址 (entry 值) > 4096。要往 xarray 存一个子树节点，不能把节点地址直接丢进 xa_head，必须分配上层节点来挂接。

  判断是不是 ZERO 特殊占位项，就是看最低位是不是 1(XA_ZERO_ENTRY=(void)1).ZERO 项不能直接存在根指针。它和空指针是两回事。

在前面判断后, 如果 allow_root = true, 说明 entry 具备存在根节点的资格，但能不能存在根节点还要看其他条件。也就是下一行的 xas_crete() 内部会判断。

NOTE

entry 是普通指针或者 NULL (0)，才允许直接赋值根节点。

- 784 创建或返回一个 slot 用于存 entry。
- 786 entry 为 NULL，也就是 NULL= 删除原有数据。不用新建节点，先读旧值。

  xas_store() 同时兼容了存储和删除 entry 的操作。
- 789-790 判断 xas->xa_node 里面存的是不是有效指针. 如果不是，直接返回原来的值，不往下执行。

  xas->xa_node 字段是复用的，既能存 xa_node 指针 (记录游标变量 xas 当前走到哪层节点, 也就是游标变量的遍历指针), 也能存操作异常时的特殊标记值/错误码，此时 xas->xa_node 不存指针，而是存数值: 1、2、3、负数错误码。这些错误码只存在 xas 上下文 (xas->xa_node), 而不是 node->slots[] 槽位。它和 slot 里存 XA_[ZERO/RETRY/ERR]_ENTRY 编码值 (见前面的表) 是两套空间、两套编码。xas_invalid 只判断游标变量当前的状态 (xas->xa_node), 不判断 slot 内 entry 存储的编码。

NOTE

让一个指针返回值同时承载”正常指针”和”状态码”是 linux 内核常用的经典技巧。

类似的内核场景有: 创建结构体成功返回结构体指针。若失败，则用指针存放状态码 (错误码)。详见辅助函数 (宏)ERR_PTR(打包错误码到指针)、IS_ERR(判断是不是错误) 和 PTR_ERR(取出真正的错误码) 对指针的处理。(内核中 sizeof(unsigned long) = 指针的 size)

这样做的好处在于统一了返回接口，不需要额外的传出参数。

正常遍历成功时 xas->xa_node 是合法指针 struct xa_node * 。最低 2bit=00，所以 xas_invalid()=false。

xas->xa_node 最低两位的编码规则如下:

最低两位 [1:0]	存入 xa_node 的值	xas_invalid 结果	编码宏	场景
00	有效结构体地址	&3=0 → valid	合法 xa_node 结构体指针	正常遍历树，游标指向真实节点
01	(void *)1	&3 0 → invalid	XAS_BOUNDS	索引越界/遍历无数据
10	(void *)2	&3 0 → invalid	XAS_ERROR(errno)	内存分配失败/参数错误
11	(void *)3	&3 0 → invalid	XAS_RESTART	并发修改路径临时解锁

- 791 获取游标当前停留的 node(节点).
- 792 游标当前停留节点所在的层级和游标缓存的当前 (旧) 层级比较。游标旧层级 < 节点真实层级, 说明游标停留的节点跑到了上层节点。这种场景通常是 xarray 被扩容了，在原来的节点上面新加了一层节点 (父节点), 游标停留在这父节点上。xas_create() 中会有此细节 (待解析)

  游标的停留 node 从下层 (小 shift) 节点跳到了上层 (大 shift) 节点，原来的 xa_sibs 是下层节点的连续 slot 计数，和新上层节点 slots 完全无关，因此作废清零。因为 xas 是游标量所以要更新到当前层。等下次遍历到下层时，会重新计算出新游标的 xas->sibs. 也就是说 xas->sibs 只是丢掉内存里临时数字，真实的 sibling 信息全写死在 node->slots 数组里，下次进节点遍历 slot 统计，自然会还原 sibs 数量。

- 794-795 如果写入值和原值一样, 并且没有后续连续兄弟 slot 项要批量修改，那么直接返回退出。
- 797 获取要写入的**旧**值。

  此处虽然变量名称是 next, 但获取的是旧值。之所以称为"next" 是因为在下面的大循环遍历时变量迭代时存储 next 值。这也可以通过前面 xas_create() 的返回值—也就是这里的变量 first—的说明看出 next(或 first) 存的旧值：如果槽位已经存在，返回该槽位的内容 (旧值)。如果槽位是新创建的，返回 NULL。
- 798 获取当要写的槽位 (slot)
- 799 计算 (从当前要写的槽位开始) 要连续写到的最大槽位。
- 800 判断是否在根节点 (根节点无 node 结构，直接存在 xa_head)
- 802-803 针对由一个起始基准 slot + 一串 sibling 兄弟 slot 组成的大块条目，把这一整段区间内分散的 slot 的 mark 位全部收拢合并. 只在起始基准 offset 的槽位保留 mark，后面 sibling 项对应 slot 的 mark 位图全部清掉。

  mark 是附着在 slot/entry 上的 3 种状态标记位，用位图方式存储在 xa_node->marks[3]，用来给索引打标签。内核在内存管理中的经典用法：XA_MARK_0 = XA_FREE_MARK 空闲标记（page 已经释放、索引闲置）。MARK1：待回写 (Pending)。MARK2：保留 (Reserved)，用户也可自定义用途。
- 805-806 如果我们要写入的新值是 NULL(空), 也就是要删除数据。就把当前这个 slot 对应的所有 mark 全部清零。

  下面的 for 循环是对同一个 node 内 slot 的批量处理。
- 816 把 *slot 这个受 RCU 保护的指针安全地更新为新值 entry，并强制保证：所有之前的操作 (清 mark、squash) 必须先完成，让其他 CPU 永远看不到半修改状态。强制 CPU 和编译器保证：赋值前的所有操作（如清 mark）一定先完成，绝对不乱序。rcu_assign_pointer 包含一个释放内存屏障，这是防止 CPU 乱序执行：不让 “设为 NULL” 先执行, 而 “清标记” 后执行。因此清除标记的操作，会在条目被设为 NULL 之前对其他 CPU 可见。

  必须在把条目设为 NULL 之前先清除标记位, 否则 xas_for_each_marked 可能会遍历到一个 NULL 条目，并提前终止遍历。

  *slot 的类型是 void *, 表示槽位里面存储的指针 (entry/子节点/NULL)。
- 817 next 是即将写入的新值。如果这个新值是一个 xa_node 节点指针, 说明即将要覆盖/替换掉一个完整子树. 并且当前在根节点或者当前不是叶子节点。那么释放整个子树的所有节点内存。因为原来的子树再也用不到了，必须回收内存，避免泄漏。
- 819-820 如果当前已经处于根节点无子 node(node == NULL)，说明已经到最顶层了，不能再往上回退遍历，直接跳出循环，结束整个写入操作.
- 821 计算本次操作让数组里多了或少了几个有效 slot。这里是 bool 值的运算，计算结果的范围是 $count\,\{-1, 0, 1\}$。

首先，如前述，我们已经知道:next 是旧值，entry 是新值。那么譬如: 新值是 NULL, 旧值非 NULL. 也就是 next=NULL,entry=!NULL。此时，

$$\begin{aligned} tmp =& !next - !entry \\ =& !NULL - !\,(!NULL) \\ =& 1 - !1 \\ =& 1 - 0 \\ =& 1 \end{aligned}$$

此时 count 应该 +1。

- 822 更新 value 类型条目的计数。计算原理同上。
- 823-830 这是 xarray 批量写入兄弟条目 (sibling) 的核心循环逻辑.
  - 823 写入有效值, 不是 NULL(不是删除)
  - 824-825 写到本次批量的最后一个槽，已经写完了，退出循环
  - 827 基准槽写真实数据, 后面所有槽都标记为 sibling，指向基准槽。
  - 829-830 如果是写入 NULL（删除操作），已经写到节点最后一个槽，不能再写了，退出。
- 833-837 这几行是 XArray 批量处理连续索引时，跳到下一个槽位、读取下一个值、判断是不是 sibling、决定要不要继续循环的核心步进逻辑。
- 833 先把 $offset + 1$(移动到下一个槽)，然后读取这个新槽里的值，存到 next。
- 834-835 如果正在执行删除操作 (写入 NULL)，已经处理完所有要删的索引 (已经越过了本次批量处理的最后位置) , 退出循环。
- 836 只要碰到非兄弟条目的槽，就更新基准头 (槽)
- 838 slot 移向下一个，继续循环。
- 841 把前面循环里统计出来的 count 和 values，更新到当前节点的 node->count 和 node->nr_values。

15.1.2 xas_create()

xa_store() › *xas_create()*

```
638 static void *xas_create(struct xa_state *xas, bool allow_root)
639 {
640     struct xarray *xa = xas->xa;
641     void *entry;
642     void __rcu **slot;
643     struct xa_node *node = xas->xa_node;
644     int shift;
645     unsigned int order = xas->xa_shift;
646 
647     if (xas_top(node)) {
648         entry = xa_head_locked(xa);
649         xas->xa_node = NULL;
650         if (!entry && xa_zero_busy(xa))
```

```
651             entry = XA_ZERO_ENTRY;
652         shift = xas_expand(xas, entry);
653         if (shift < 0)
654             return NULL;
655         if (!shift && !allow_root)
656             shift = XA_CHUNK_SHIFT;
657         entry = xa_head_locked(xa);
658         slot = &xa->xa_head;
659     } else if (xas_error(xas)) {
660         return NULL;
661     } else if (node) {
662         unsigned int offset = xas->xa_offset;
663 
664         shift = node->shift;
665         entry = xa_entry_locked(xa, node, offset);
666         slot = &node->slots[offset];
667     } else {
668         shift = 0;
669         entry = xa_head_locked(xa);
670         slot = &xa->xa_head;
671     }
672 
673     while (shift > order) {
674         shift -= XA_CHUNK_SHIFT;
675         if (!entry) {
676             node = xas_alloc(xas, shift);
677             if (!node)
678                 break;
679             if (xa_track_free(xa))
680                 node_mark_all(node, XA_FREE_MARK);
681             rcu_assign_pointer(*slot, xa_mk_node(node));
682         } else if (xa_is_node(entry)) {
683             node = xa_to_node(entry);
684         } else {
685             break;
686         }
687         entry = xas_descend(xas, node);
688         slot = &node->slots[xas->xa_offset];
689     }
690 
691     return entry;
692 }
```

- 647 判断当前 node 存放的是不是一个正常的 struct xa_node 节点指针。如果不是正常节点，而是特殊标记 (越界/重启/错误)。则返回 true。
- 648 获取根节点的 xa_head。
- 649 清空游标节点。

  当 xas->xa_node 是特殊标记 (1/2/3) 时，原有遍历路径作废，必须从头 (树根) 重新开始遍历整棵 xarray 树。
- $xa_head == NULL$ 说明整棵树没有分配任何 xa_node 树形节点。但是 index0 逻辑上已占用。换言之，只预留 index0、没存真实数据，整棵树没创建任何节点，xa_head 维持 NULL。

  xa_zero_busy() 会通过 check xa->xa_flags 是否置了 XA_FLAGS_ZERO_BUSY 标志来判断 index0 是否已 reserve(预留)。
- 650-651 根 (xa_head) 是 NULL（没节点、没数据），但逻辑上 index 0 已经被占用（reserved），所以把 entry(即 xa_head) 强行赋值成 XA_ZERO_ENTRY，让遍历/创建逻辑知道：index 0 是被占

的，不是空的。对外的 KPI 接口读时会返回 NULL，但是 xarray 内部实现机制会用这个编码 (entry 最低两位 $[1:0] = 0b10$) 来区分**空槽 NULL** 和**已预留空槽**。

XA_ZERO_ENTRY 和 XA_FLAGS_ZERO_BUSY 的关系:

- XA_FLAGS_ZERO_BUSY：flag 标记，只存在 xa->xa_flags，只用 flag 标记 0 号预留，不写入 slot。
- XA_ZERO_ENTRY：slot 填充值，树形节点分配后写入 slot，物理存储占位标记。

- 652-654 根据当前树根 entry(xa_head)，计算整棵 XArray 树的顶层的 shift。如果树高度不够，就自动创建更高层的节点把树撑大；如果扩容失败 (内存不足)，返回负数。xas_expand() 会根据目标索引 xas->xa_index 计算需要多大的树高度。

- 655-656 如果当前树是只有 index 0 的极简模式 (shift=0)，并且当前不允许使用这种极简模式，就强制 shift=6，让后续代码创建一个 64 槽的子节点。

- 657-658 因为刚才的 xas_expand() 可能新建了节点，所以：重新读取最新的根数据 (xa_head)，并把指针定位到根槽位 (slot)，准备从最顶端开始向下找目标 index。

  xas_expand() 扩容时可能创建一层新 (父) 节点，老节点挂在新节点 slot [0]，xa_head 会重新指向新分配的上层 (父节点) 的 xa_node。也就是 xa->xa_head = 新 xa_node 指针。所以此处会重新读取。

- 659-660 判断游标状态是否发生错误（内存不足、非法操作），若是，直接失败返回。

- 661-666 如果 xas 游标是正常有效的节点（不是顶层、不是错误），就直接从游标里取出：当前层偏移量、shift、当前槽里的值、当前槽指针，为接下来向下遍历树做准备。

- 667-671 执行到这个代码段，意味着 xas 游标已经在根节点之上，没有上层节点了，就在树的最顶端。换言之，进入这里的唯一条件是：node == NULL。
  整个代码逻辑判断的流程如下:

```
if (xas_top(node)) {
    //游标位置失效，需要重启/从头遍历。
} else if (xas_error(xas)) {
    //操作本身出错，不能继续
} else if (node) {
    //正常有效节点
} else {
    //node == NULL
}
```

  其中前两个 if 的判断的意义如下:

  - 650-651 xas_top(node)=true 表示游标位置失效了，需要重置。
  - 659-660 xas_error(xas)=true 表示内存不足、参数非法、内核错误不能继续执行，必须终止!

15.2 外设寄存器 (MMIO) 的映射

外设 (或片上外设) 寄存器的地址是物理地址，但内核不能直接访问物理地址，必须通过页表建立内核虚拟地址到设备物理地址的映射。*ioremap()* 就是用来新建一段内核页表映射专门给 *MMIO* 设备。让内核可以通过指针直接访问设备寄存器。映射页表自动配置为非缓存属性，保证寄存器读写直接作用于硬件。使用完毕需调用 *iounmap()* 销毁映射。该函数仅用于内核态访问 *MMIO* 区域，不可用于普通系统 RAM。返回的 *__iomem* 指针不能直接用普通指针解引用，最好用内核 IO 访问函数: *readl()/writel()/readb()/writeb()*。保证访问顺序（内存屏障），避免编译器优化打乱寄存器读写时序。

适用场景:PCI 设备 BAR 空间映射。SOC 片上外设 UART、GPIO、定时器等寄存器。

ioremap() 是内核提供的架构无关 API，但其底层实现由各架构自行定义。x86_64 在 *arch/x86/mm/ioremap.c* 基于 PAT 寄存器编码 MMIO 页属性；arm64 在 *arch/arm64/mm/io.c* 使用 MAIR 寄存器，PTE 通过 AttrIndx 索引内存属性。二者均在 vmalloc 虚拟地址域创建内核页表映射，调用 *remap_pfn_range()* 填充页表项，但硬件内存属性管理机制不同。

15.2.1 ioremap()

========= 待加入

A

附录

Index

/proc/sys/vm/min_unmapped_ratio, 123, 124
/proc/sys/vm/watermark_scale_factor, 120

buffers,cached,swapcache 的区别, 284

cpu 和 task 各自的脏页阈值的作用, 392

hwpoison swap entry, 463

IO 拥塞场景 A, 249

kmem_cache->min 的作用意义, 80
kswapd 回收限流, 223
kswapd 进入回收, 307

lazyfree: 文件页 lru 链表上的匿名页, 287

memcg elow 事件的上报, 226
min、low、high 水位线之间的 Δ 及由来, 121

swapcache 的准确含义, 375

ZONE_DEVICE 区域页如何构建的, 459
ZONELIST_FALLBACK 备选顺序如何调整, 113

不受全局脏页数约束的线程标记, 402
为什么文件页的 write 重载页不是 workingset 页, 214
为什么计算全局最小预留内存时将 ZONE_HIGHMEM 排除, 104
什么时候从 CMA 分配, 340
什么时候填充和 invalid TLB, 468
什么时候无条件执行内存整理, 338
什么时候用严格限制模式来计算阈值, 400
什么是脏页泄漏, 392
什么是高代价分配, 333
什么条件触发降低文件页回收权重主动回收匿名页, 222
修剪模式 (trim mode) 是什么意思, 212
允许使用紧急内存的场景 A, 277
内存整理积极程度如何设置, 366
内核高精度时间间隔测量的接口, 183
动态决定是否允许活跃 LRU 页降级的点, 206
匿名页和文件页的回收损耗 (cost), 205
匿名页降级的时机, 295
向用户空间通知内存压力和 lmkd, 190
回写压力的判断, 223
在文件 lru 链表上的匿名页, 287
如何找出 cpu 所在 numa 节点号, 110
如何改变伙伴系统能分配的字节上限, 126
如何遍历系统中所有 slub 缓存, 46
工作页集是什么, 239
应用层如何查看碎片指数, 333

文件页 pageout 和匿名页 swapout 的差别, 286
文件页的二次机会和匿名页的一次机会, 260
欠活跃链表移除事件的计数, 217
活跃页降级到欠活跃链表的时机, 295

直接回收什么时候运行直接文件回写, 268
碎片或内存不足的判断, 333
碎片指数的计算, 335

设置 workingset 页的时机 B, 298

隔离的是文件可执行页，第一次访问就放到活跃链表, 292
需要 invalid TLB 的场景 A, 462
预留内存和紧急预留内存的区别, 122